

.NET MAUI - praktyczna dokumentacja od podstaw

Ten dokument jest praktycznym przewodnikiem po **.NET MAUI**. Prowadzi od podstaw: czym jest MAUI, jak wygląda projekt, jak pisać XAML, jak układać interfejs, jak dobierać kontrolki i jak pisać prosty, czytelny code-behind w C#.

Materiał skupia się na aplikacjach użytkowych: formularzach, listach, obrazach, suwakach, przyciskach, komunikatach, pracy z plikami i prostych danych. Zaawansowane wzorce architektoniczne są pominięte albo ograniczone do minimum, bo na początku ważniejsze jest sprawne zbudowanie działającego ekranu i poprawna obsługa zdarzeń.

Przykłady kodu pokazują zwykle dwa pliki: widok `.xaml` oraz logikę `.xaml.cs`. To najprostszy i najbardziej bezpośredni sposób nauki MAUI: najpierw budujesz ekran, potem podpinasz zdarzenia i aktualizujesz widok z code-behind.

Spis treści

- 1. Wprowadzenie do .NET MAUI
 - 1.1. Czym jest .NET MAUI
 - 1.2. Do czego służy .NET MAUI i jakie aplikacje można w nim tworzyć
 - 1.3. Jakie platformy obsługuje .NET MAUI
 - 1.4. Czym różni się aplikacja mobilna od desktopowej
 - 1.5. Czym różni się .NET MAUI od WPF
 - 1.6. Czym różni się .NET MAUI od WinForms
 - 1.7. Czym różni się .NET MAUI od Xamarin.Forms
 - 1.8. Aplikacja wieloplatformowa i model „jeden projekt, wiele platform”
 - 1.9. Zalety i ograniczenia .NET MAUI
 - 1.10. Kiedy warto używać .NET MAUI, a kiedy wybrać inną technologię
- 2. Środowisko pracy
 - 2.1. Visual Studio
 - 2.2. Visual Studio Code
 - 2.3. .NET SDK
 - 2.4. Workload .NET MAUI
 - 2.5. Android SDK
 - 2.6. Emulator Androida a fizyczny telefon
 - 2.7. Uruchamianie aplikacji na Windows
 - 2.8. Uruchamianie aplikacji na Androidzie i na emulatorze
 - 2.9. Uruchamianie aplikacji na fizycznym telefonie
 - 2.10. Tryb Debug i tryb Release
 - 2.11. Hot Reload
 - 2.12. Pierwszy projekt - utworzenie i struktura
 - 2.13. Typowe problemy z konfiguracją i najczęstsze błędy pierwszego uruchomienia
- 3. Pierwszy projekt i struktura projektu
 - 3.1. Przegląd struktury projektu
 - 3.2. Plik `.csproj`
 - 3.3. `MauiProgram.cs`
 - 3.4. `App.xaml` i `App.xaml.cs`
 - 3.5. `AppShell.xaml` i `AppShell.xaml.cs`
 - 3.6. `MainPage.xaml` i `MainPage.xaml.cs`
 - 3.7. Folder `Resources`
 - 3.8. Folder `Platforms`
 - 3.9. Obrazy, czcionki, pliki lokalne i zasoby aplikacji
 - 3.10. Jak organizować projekt od początku
- 4. Podstawy C#: typy, operatory i tekst

- 4.1. Jak działa program i z czego składa się kod
- 4.2. Zmienne - deklaracja, inicjalizacja i nazewnictwo
- 4.3. Typy liczbowe całkowite - int, long, short, byte
- 4.4. Typy zmiennoprzecinkowe - double, float, decimal
- 4.5. Typ logiczny - bool
- 4.6. Typ znakowy - char
- 4.7. Typ tekstowy - string
- 4.8. Stałe - const i readonly
- 4.9. Operatory arytmetyczne
- 4.10. Operatory przypisania
- 4.11. Operatory porównania
- 4.12. Operatory logiczne
- 4.13. Operatory null: ??, ?, ??=
- 4.14. Konwersje typów
- 4.15. Interpolacja i formatowanie tekstu
- 5. Podstawy C#: warunki, pętle i metody
 - 5.1. Instrukcja warunkowa if / else if / else
 - 5.2. Operator warunkowy (trójargumentowy) ?:
 - 5.3. Instrukcja switch i switch expression
 - 5.4. Pętla for
 - 5.5. Pętla foreach
 - 5.6. Pętla while
 - 5.7. Pętla do-while
 - 5.8. break i continue
 - 5.9. Metody
- 6. C#: kolekcje, klasy i modele danych
 - 6.1. Tablice
 - 6.2. List
 - 6.3. ObservableCollection
 - 6.4. Słowniki - Dictionary
 - 6.5. Klasy i obiekty
 - 6.6. Właściwości - get i set
 - 6.7. Konstruktory
 - 6.8. Modyfikatory dostępu
 - 6.9. Pola, this i składowe statyczne
 - 6.10. Dziedziczenie i polimorfizm (podstawy)
 - 6.11. Typy wyliczeniowe - enum
- 7. C#: daty, losowanie, async, wyjątki i LINQ
 - 7.1. Losowanie - Random
 - 7.2. Operacje matematyczne - Math
 - 7.3. Data i czas - DateTime i TimeSpan
 - 7.4. Programowanie asynchroniczne - async, await, Task
 - 7.5. Obsługa wyjątków - try, catch, finally
 - 7.6. LINQ - wygodne operacje na kolekcjach
 - 7.7. Łączenie elementów - prosta logika aplikacji
- 8. Podstawy XAML
 - 8.1. Czym jest XAML
 - 8.2. Znaczniki (tagi)
 - 8.3. Atrybuty i właściwości
 - 8.4. Property element syntax (właściwości jako elementy)

- 8.5. Znaczniki samozamykające
- 8.6. Elementy zagnieżdżone - element nadrzędny i potomny
- 8.7. Komentarze w XAML
- 8.8. Przestrzenie nazw - `xmlns` i `xmlns:x`
- 8.9. Typowe błędy składni XAML
- 8.10. Markup extensions - rozszerzenia znaczników
- 8.11. `x:Static`, `x:Reference`, `x:Null` i `x:Array`
- 8.12. Zasoby w XAML - `ResourceDictionary`
- 8.13. Style w XAML - `Style`, `Setter`, `TargetType`
- 8.14. `OnPlatform` i `OnIdiom` w XAML
- 8.15. Pełny szablon strony XAML - podsumowanie składni
- 9. XAML w projekcie MAUI i code-behind
 - 9.1. Relacja XAML i C#
 - 9.2. `x:Class` - powiązanie z klasą C#
 - 9.3. `x:Name` i `Name` - nazwy kontrolerek
 - 9.4. Pliki `.xaml`, `.xaml.cs`, klasa partial i `InitializeComponent()`
- 10. Strony i cykl życia aplikacji
 - 10.1. `ContentPage`
 - 10.2. `TabbedPage`
 - 10.3. `FlyoutPage`
 - 10.4. Kiedy używać poszczególnych typów stron
 - 10.5. Cykl życia strony
 - 10.6. Konstruktor a `OnAppearing` - gdzie ładować dane
 - 10.7. Ładowanie i odświeżanie danych po wejściu i powrocie
- 11. Nawigacja i Shell
 - 11.1. `NavigationPage`
 - 11.2. Shell jako główny sposób budowy aplikacji
 - 11.3. Podstawy nawigacji
 - 11.4. Nawigacja modalna
 - 11.5. Shell, `AppShell` i `ShellContent`
 - 11.6. `FlyoutItem`, `TabBar` i `Tab`
 - 11.7. Routing i `GoToAsync`
 - 11.8. Przekazywanie danych między stronami
 - 11.9. `[QueryProperty]` i `IQueryAttributable`
 - 11.10. Typowe błędy przy nawigacji
- 12. Layout i rozmieszczanie elementów
 - 12.1. Czym jest layout
 - 12.2. Element nadrzędny i potomny, zagnieżdżanie
 - 12.3. `VerticalStackLayout`
 - 12.4. `HorizontalStackLayout`
 - 12.5. Grid - siatka wierszy i kolumn
 - 12.6. Wymiary w Grid: Auto, gwiazdka i wartości stałe
 - 12.7. `FlexLayout`
 - 12.8. `AbsoluteLayout`
 - 12.9. `ScrollView`
 - 12.10. `Border` i `ContentView`
 - 12.11. Rozmiary: `WidthRequest`, `HeightRequest`
 - 12.12. Odstępy: `Margin`, `Padding`, `Spacing`
 - 12.13. Wyrównanie: `HorizontalOptions` i `VerticalOptions`
 - 12.14. Responsywność i różnice mobile/desktop

- 12.15. Kiedy używać którego layoutu
- 12.16. Typowe błędy przy rozmieszczaniu elementów
- 12.17. Pełne tabele atrybutów layoutów
- 13. Design, kolory, style i animacje
 - 13.1. Kolory w XAML - sposoby zapisu
 - 13.2. Kolory nazwane
 - 13.3. Zapis HEX, RGB i ARGB - przezroczystość
 - 13.4. Właściwości kolorów: BackgroundColor, TextColor, BorderColor, Color
 - 13.5. Dynamiczna zmiana koloru
 - 13.6. Suwaki RGB - wzornik kolorów
 - 13.7. Dynamiczna zmiana rozmiaru czcionki
 - 13.8. Dynamiczna zmiana widoczności elementów
 - 13.9. Tryb jasny i ciemny - AppThemeBinding
 - 13.10. Przykład: dynamiczny wygląd zależny od wartości
 - 13.11. Typowe błędy
 - 13.12. Pełna lista nazwanych kolorów
 - 13.13. Wszystkie sposoby ustawiania koloru w XAML i C#
 - 13.14. Triggery - reakcja na zmianę właściwości
 - 13.15. VisualStateManager - stany wizualne
 - 13.16. Animacje
 - 13.17. ControlTemplate i DataTemplateSelector
- 14. Kontrolki tekstowe
 - 14.1. Label
 - 14.2. Entry
 - 14.3. Editor
 - 14.4. SearchBar
 - 14.5. Label - pełna tabela atrybutów
 - 14.6. Entry - pełna tabela atrybutów
 - 14.7. Editor - pełna tabela atrybutów i porównanie z Entry
 - 14.8. Receptury kontrolek tekstowych
 - 14.9. Label - receptury praktyczne
 - 14.10. Entry - receptury praktyczne
 - 14.11. Editor - receptury praktyczne
 - 14.12. SearchBar - receptury praktyczne
 - 14.13. Dodatkowe receptury kontrolek
- 15. Przyciski i akcje użytkownika
 - 15.1. Button
 - 15.2. ImageButton
 - 15.3. Button - pełna tabela atrybutów
 - 15.4. ImageButton - pełna tabela atrybutów
 - 15.5. Button - receptury praktyczne
 - 15.6. ImageButton - receptury praktyczne
- 16. Kontrolki wyboru i wartości liczbowe
 - 16.1. Slider - odczyt wartości i ValueChanged
 - 16.2. CheckBox
 - 16.3. RadioButton
 - 16.4. Switch
 - 16.5. Switch a CheckBox - porównanie
 - 16.6. CheckBox - pełna tabela atrybutów
 - 16.7. RadioButton - pełna tabela atrybutów

- 16.8. Switch - pełna tabela atrybutów
- 16.9. Picker
- 16.10. DatePicker
- 16.11. TimePicker
- 16.12. Slider
- 16.13. Stepper
- 16.14. Slider a Stepper - porównanie
- 16.15. Pełne tabele atrybutów kontrolki wyboru danych
- 16.16. CheckBox - receptury praktyczne
- 16.17. RadioButton - receptury praktyczne
- 16.18. Switch - receptury praktyczne
- 17. Kontrolki graficzne i prezentacyjne
 - 17.1. Image
 - 17.2. ActivityIndicator
 - 17.3. ProgressBar
 - 17.4. BoxView
 - 17.5. Border
 - 17.6. Frame (starszy kontener)
 - 17.7. Porównanie kontrolki wyświetlania
 - 17.8. Pełne tabele atrybutów kontrolki wyświetlania
 - 17.9. TableView - formularze i ustawienia
 - 17.10. WebView - strona internetowa w aplikacji
 - 17.11. Kształty (Shapes) - Rectangle, Ellipse, Line, Polygon, Path
 - 17.12. Pozostałe elementy: ToolbarItem, MenuBar, BlazorWebView
- 18. Kontrolki listowe i widoki kolekcji
 - 18.1. CollectionView - pełna tabela atrybutów
 - 18.2. ListView - pełna tabela atrybutów
 - 18.3. CarouselView i IndicatorView
 - 18.4. RefreshView - pociągnij, by odświeżyć
 - 18.5. SwipeView - gesty przesunięcia na elemencie
- 19. Wspólne właściwości kontrolki
 - 19.1. Właściwości treści: Text, Content, Placeholder, Source
 - 19.2. Właściwości wyboru: SelectedItem, SelectedIndex, ItemsSource
 - 19.3. Właściwości rozmiaru: WidthRequest, HeightRequest, Minimum...
 - 19.4. Właściwości odstępów: Margin, Padding
 - 19.5. Właściwości wyrównania: HorizontalOptions, VerticalOptions
 - 19.6. Właściwości wyglądu: BackgroundColor, TextColor, FontSize, FontAttributes, Opacity, Minimum, Maximum, Value
 - 19.7. Porównanie: Margin a Padding
 - 19.8. Porównanie: IsVisible a IsEnabled
 - 19.9. Porównanie: Text a Placeholder
 - 19.10. Porównanie: SelectedItem a SelectedIndex
 - 19.11. Porównanie: List a ObservableCollection
 - 19.12. Porównanie: Content a Text
 - 19.13. Porównanie: Source jako zasób, URL i plik lokalny
 - 19.14. Wspólne właściwości wszystkich kontrolki
 - 19.15. Typ Thickness - zapis marginesów i paddingów
 - 19.16. Podsumowanie kontrolki
- 20. Zdarzenia, gesty i code-behind
 - 20.1. Czym jest zdarzenie
 - 20.2. Czym jest event handler

- 20.3. Podpinanie zdarzeń w XAML i w C#
- 20.4. Parametry sender i e
- 20.5. Najważniejsze zdarzenia - przegląd
- 20.6. Przykłady najważniejszych zdarzeń
- 20.7. Gesty i TapGestureRecognizer
- 20.8. Kliknięcie w obraz - wzorzec praktyczny
- 20.9. Schemat: akcja → zmiana stanu → aktualizacja UI
- 20.10. Typowe błędy w obsłudze zdarzeń
- 20.11. Receptury - gotowe przykłady łączone (XAML + C#)
- 21. Stan aplikacji i aktualizacja interfejsu
 - 21.1. Czym jest stan aplikacji
 - 21.2. Gdzie przechowywać stan - pola klasy
 - 21.3. Stan liczbowy - licznik
 - 21.4. Stan tekstowy - aktualny komunikat
 - 21.5. Stan logiczny - włączone/wyłączone
 - 21.6. Stan jako indeks - aktualny element
 - 21.7. Stan jako wybrany element lub obiekt
 - 21.8. Stan a wygląd - ważne rozróżnienie
 - 21.9. Aktualizacja interfejsu po zmianie stanu
 - 21.10. Przykład: gra w kości (stan złożony)
 - 21.11. Przykład: wzornik kolorów RGB (stan liczbowy ↔ wygląd)
 - 21.12. Typowe rodzaje stanu - podsumowanie
- 22. Logika poza widokiem i organizacja kodu
 - 22.1. Klasy pomocnicze
 - 22.2. Modele jako nośnik danych i prostej logiki
 - 22.3. Klasa do pracy na danych
 - 22.4. Logika walidacji
 - 22.5. Logika losowania
 - 22.6. Logika generowania hasła
 - 22.7. Logika obliczeń
 - 22.8. Logika zmiany stanu i pracy na liście obiektów
 - 22.9. Logika szyfrowania (prosty przykład)
 - 22.10. Testowanie logiki bez interfejsu
 - 22.11. Typowe błędy
- 23. Formularze i pobieranie danych
 - 23.1. Budowa prostego formularza
 - 23.2. Układ formularza w VerticalStackLayout i w Grid
 - 23.3. Pobieranie danych z różnych kontroltek
 - 23.4. Składanie podsumowania z wielu pól
 - 23.5. Aktualizacja etykiety po kliknięciu przycisku
 - 23.6. Czyszczenie formularza
 - 23.7. Blokowanie przycisku do czasu poprawnego uzupełnienia
 - 23.8. Formularz logowania - kompletny przykład
 - 23.9. Formularz rejestracji - kompletny przykład
 - 23.10. Formularz z wieloma typami kontroltek - kompletny przykład
 - 23.11. Typowe błędy przy formularzach
 - 23.12. Formularz logowania - kompletny przykład
 - 23.13. Formularz z każdym typem pola - kompletny przykład
 - 23.14. Formularz wielosekcyjny z Border
 - 23.15. Receptury formularzy i walidacji

- 23.16. Formularz logowania
- 23.17. Formularz rejestracji (zgodność haseł, walidacja e-mail z @)
- 23.18. Formularz kontaktowy
- 23.19. Formularz dodawania produktu
- 23.20. Ankieta (RadioButton i CheckBox)
- 23.21. Formularz ustawień (Switch)
- 23.22. Formularz wielosekcyjny (Border)
- 23.23. Kalkulator BMI
- 23.24. Formularz z dynamicznym podsumowaniem
- 23.25. Formularz rejestracji na wydarzenie (walidacja długości tekstu i zakresu liczbowego)
- 23.26. Formularz zmiany hasła (porównanie starego i nowego)
- 23.27. Formularz opinii z oceną gwiazdkową (Slider jako rating)
- 24. Walidacja i komunikaty dla użytkownika
 - 24.1. Po co walidować dane
 - 24.2. Sprawdzanie pustych pól - IsNullOrEmpty
 - 24.3. Sprawdzanie długości tekstu
 - 24.4. Sprawdzanie, czy tekst zawiera znak (np. e-mail z „@“)
 - 24.5. Porównywanie dwóch pól (zgodność haseł)
 - 24.6. Sprawdzanie, czy tekst składa się tylko z cyfr
 - 24.7. Konwersja tekstu na liczbę: Parse, TryParse, zakres
 - 24.8. Pełniejsza walidacja e-maila (wyrażenia regularne)
 - 24.9. Wartość domyślna przy błędnych danych
 - 24.10. Prezentacja błędów: Label a DisplayAlert
 - 24.11. Walidacja na żywo (TextChanged) i blokowanie przycisku
 - 24.12. Kompletny przykład walidacji formularza
 - 24.13. Typowe błędy walidacji
 - 24.14. Przegląd okien dialogowych
 - 24.15. DisplayAlert - komunikat z jednym przyciskiem
 - 24.16. DisplayAlert - pytanie z dwoma przyciskami
 - 24.17. DisplayActionSheet - menu akcji
 - 24.18. DisplayPromptAsync - pobieranie tekstu
 - 24.19. Komunikat po walidacji, zapisie i akcji
 - 24.20. Kiedy etykieta, a kiedy alert
 - 24.21. Typowe błędy
 - 24.22. Podsumowanie technik walidacji
- 25. Listy, kolekcje i praktyczne receptury
 - 25.1. List a ObservableCollection - najważniejsza różnica
 - 25.2. Lista tekstów i lista obiektów
 - 25.3. CollectionView
 - 25.4. DataTemplate i ItemTemplate - wygląd elementu
 - 25.5. ListView - alternatywa klasyczna
 - 25.6. Dodawanie elementów i automatyczne odświeżanie
 - 25.7. Usuwanie elementów
 - 25.8. Edycja i wybór elementu
 - 25.9. Przechodzenie po elementach: Poprzedni / Następny
 - 25.10. Lista obiektów wczytana z pliku
 - 25.11. Pusta lista - EmptyView
 - 25.12. Typowe błędy przy listach
 - 25.13. Receptury list i kolekcji
 - 25.14. ObservableCollection - dodawanie elementów

- 25.15. ObservableCollection - usuwanie elementów
- 25.16. ObservableCollection - edycja elementu
- 25.17. ObservableCollection - czyszczenie kolekcji
- 25.18. CollectionView - prosty DataTemplate
- 25.19. CollectionView - złożony DataTemplate z obiektem
- 25.20. CollectionView - GridItemsLayout (siatka)
- 25.21. CollectionView - EmptyView
- 25.22. CollectionView - SelectionChanged
- 25.23. CollectionView - Header i Footer
- 25.24. CollectionView - lista pozioma
- 25.25. ListView - ViewCell z wieloma elementami
- 25.26. ListView - ItemTapped
- 25.27. ListView - grupowanie
- 25.28. ListView - Pull-to-Refresh
- 25.29. CarouselView + IndicatorView
- 25.30. SwipeView w CollectionView
- 25.31. Wyszukiwanie - SearchBar + filtrowanie LINQ
- 25.32. Sortowanie kolekcji
- 25.33. Lista obiektów z sortowaniem po właściwości
- 25.34. Lista z pliku tekstowego
- 25.35. Lista z pliku CSV (obiekty)
- 25.36. Nawigacja Poprzedni/Następny z zawijaniem
- 25.37. CollectionView - wielokrotny wybór (Multi-Select)
- 25.38. Filtrowanie obiektów z wieloma kryteriami
- 25.39. CollectionView - RefreshView (pull-to-refresh)
- 25.40. ObservableCollection - przenoszenie elementów (Move)
- 25.41. CollectionView - SelectionMode None z TapGestureRecognizer
- 25.42. Lista z licznikiem - dodawanie i usuwanie z podsumowaniem
- 25.43. Nawigacja Poprzedni/Następny z obiektami (szczegóły)
- 25.44. Podsumowanie
- 26. Binding danych w praktyce
 - 26.1. Czym jest binding
 - 26.2. Po co stosuje się binding
 - 26.3. BindingContext
 - 26.4. Binding w XAML i w C#
 - 26.5. Ścieżka i StringFormat
 - 26.6. Tryby bindingu: OneWay, TwoWay, OneTime
 - 26.7. Binding do tekstu, liczby, wartości logicznej i listy
 - 26.8. Binding w CollectionView
 - 26.9. INotifyPropertyChanged - temat opcjonalny
 - 26.10. Odświeżanie listy - ObservableCollection
 - 26.11. Typowe błędy z bindingiem
- 27. Obrazy i zasoby graficzne
 - 27.1. Dodawanie obrazów do projektu
 - 27.2. Nazewnictwo plików graficznych
 - 27.3. Image.Source - wyświetlanie obrazu
 - 27.4. Źródła obrazu: zasób, internet, plik lokalny
 - 27.5. Aspect - dopasowanie obrazu
 - 27.6. Opacity - przezroczystość
 - 27.7. Zmiana obrazu w czasie działania aplikacji

- 27.8. Budowanie nazwy obrazka na podstawie danych
- 27.9. Obsługa braku obrazka
- 27.10. Klikalny obraz - TapGestureRecognizer
- 27.11. Wiele obrazów na ekranie i rozpoznawanie kliknięcia
- 27.12. Obrazy jako element stanu - przykład gry w kości
- 27.13. Inne praktyczne zastosowania obrazów
- 27.14. Typowe błędy z obrazami
- 27.15. Obraz z zasobu (Resources/Images)
- 27.16. Obraz z URL
- 27.17. Obraz z pliku lokalnego
- 27.18. Podmiana obrazu w runtime + budowanie nazwy
- 27.19. Klikalny obraz - TapGestureRecognizer
- 28. Pliki lokalne i zasoby Raw
 - 28.1. Czym jest plik lokalny
 - 28.2. Plik projektu a plik użytkownika
 - 28.3. Zasób a plik zapisywany przez aplikację
 - 28.4. Foldery Resources/Raw i Resources/Images
 - 28.5. FileSystem.AppDataDirectory i budowanie ścieżek
 - 28.6. Ścieżki względne i bezwzględne
 - 28.7. Odczyt pliku tekstowego
 - 28.8. Zapis pliku tekstowego
 - 28.9. Dopisywanie do pliku
 - 28.10. Sprawdzanie istnienia i usuwanie pliku
 - 28.11. Odczyt danych z pliku „dane.txt” i parsowanie
 - 28.12. Tworzenie listy obiektów na podstawie pliku
 - 28.13. Zapis wyniku działania aplikacji i dużego tekstu
 - 28.14. Obsługa błędów przy pracy z plikami
 - 28.15. Wybór wielu plików i filtrowanie typów
 - 28.16. Uprawnienia związane z plikami i multimediami
 - 28.17. Typowe problemy ze ścieżkami i dostępem do plików
 - 28.18. Receptury plików, obrazów, danych i API
 - 28.19. Zapis pliku tekstowego
 - 28.20. Odczyt pliku tekstowego
 - 28.21. Dopisywanie do pliku
 - 28.22. Usuwanie pliku
 - 28.23. Odczyt zasobu z Resources/Raw
 - 28.24. Parsowanie linii na listę obiektów
- 29. Preferences i ustawienia aplikacji
 - 29.1. Czym są Preferences
 - 29.2. Kiedy używać Preferences
 - 29.3. Zapis i odczyt różnych typów
 - 29.4. Sprawdzanie, usuwanie i czyszczenie
 - 29.5. Przykład: zapamiętanie loginu
 - 29.6. Przykład: zapamiętanie motywu i rozmiaru czcionki
 - 29.7. Przykład: zapamiętanie ostatniej wartości i ustawień użytkownika
 - 29.8. Typowe błędy
 - 29.9. Preferences - zapis i odczyt ustawień
 - 29.10. Preferences - zapamiętanie motywu
- 30. SQLite i dane lokalne
 - 30.1. Kiedy wystarczy plik, a kiedy potrzeba bazy

- 30.2. Czym jest SQLite
- 30.3. Instalacja pakietu i model danych
- 30.4. Klasa bazy danych i tworzenie tabeli
- 30.5. Dodawanie rekordu (Create)
- 30.6. Odczyt rekordów (Read)
- 30.7. Aktualizacja rekordu (Update)
- 30.8. Usuwanie rekordu (Delete)
- 30.9. Wzorzec „Zapisz” (dodaj lub aktualizuj)
- 30.10. Połączenie SQLite z CollectionView
- 30.11. Kompletny przykład: aplikacja zadań z SQLite
- 30.12. Typowe błędy
- 30.13. SQLite - model + klasa bazy CRUD
- 30.14. SQLite - pełna aplikacja zadań z CollectionView
- 31. API, JSON i HttpClient - temat opcjonalny
 - 31.1. Czym jest API
 - 31.2. REST API i endpointy
 - 31.3. Kody statusu HTTP
 - 31.4. Czym jest JSON
 - 31.5. HttpClient - pobieranie danych (GET)
 - 31.6. Wysyłanie danych (POST)
 - 31.7. Serializacja i deserializacja - System.Text.Json
 - 31.8. Model danych pod JSON
 - 31.9. Wyświetlanie danych z API w CollectionView
 - 31.10. Obsługa błędów API
 - 31.11. Typowe błędy
 - 31.12. HttpClient GET + JSON (GetFromJsonAsync)
 - 31.13. HttpClient POST
 - 31.14. ActivityIndicator + try/catch + Connectivity
 - 31.15. Podsumowanie
- 32. Funkcje urządzenia i uprawnienia - temat opcjonalny
 - 32.1. Czym są uprawnienia
 - 32.2. Uprawnienia na Androidzie i iOS
 - 32.3. Sprawdzanie i proszenie o uprawnienie
 - 32.4. Najważniejsze funkcje urządzenia
 - 32.5. Przykłady: schowek, przeglądarka, telefon, e-mail
 - 32.6. Geolokalizacja, latarka, udostępnianie
 - 32.7. Connectivity i DeviceInfo
 - 32.8. Typowe błędy
- 33. Kod platformowy i różnice systemowe - temat opcjonalny
 - 33.1. Folder Platforms
 - 33.2. Kompilacja warunkowa: #if ANDROID, #if IOS, #if WINDOWS
 - 33.3. OnPlatform - różne wartości per platforma
 - 33.4. OnIdiom - telefon, tablet, komputer
 - 33.5. Partial classes dla kodu natywnego
 - 33.6. Typowe różnice między platformami
 - 33.7. Testowanie na różnych platformach
 - 33.8. Typowe błędy
- 34. Debugowanie
 - 34.1. Czym jest debugowanie
 - 34.2. Breakpointy i kroki: Step Over, Step Into, Step Out

- 34.3. Podgląd zmiennych: Watch, Locals, Output
- 34.4. Debugowanie XAML i bindingu
- 34.5. Obsługa wyjątków: try, catch, finally
- 34.6. Wyjątki przy konwersji, plikach, API i SQLite
- 34.7. Komunikaty dla użytkownika i logowanie błędów
- 34.8. Typowe błędy początkujących
- 35. Gotowe aplikacje - kompletne przykłady
 - 35.1. Jak korzystać z gotowych aplikacji
 - 35.2. Kalkulatory i przeliczniki
 - 35.3. Kalkulator BMI
 - 35.4. Kalkulator napiwku
 - 35.5. Przelicznik jednostek
 - 35.6. Kalkulator zamówienia pizzy
 - 35.7. Generator hasła
 - 35.8. Formularze i rezerwacje
 - 35.9. Formularz rejestracji
 - 35.10. Rezerwacja wizyty
 - 35.11. Ankieta satysfakcji
 - 35.12. Formularz kontaktowy
 - 35.13. Zamówienie biletu
 - 35.14. Listy i kolekcje
 - 35.15. Lista zakupów
 - 35.16. Lista notatek
 - 35.17. Planer zadań
 - 35.18. Dziennik wydatków
 - 35.19. Licznik punktów graczy
 - 35.20. Obrazy, gry i interakcje
 - 35.21. Gra w kości
 - 35.22. Kości z blokowaniem
 - 35.23. Galeria obrazów
 - 35.24. Quiz z pytaniami
 - 35.25. Wzornik kolorów RGB
 - 35.26. Pliki i dane lokalne
 - 35.27. Notatnik z zapisem do pliku
 - 35.28. Szyfr Cezara z zapisem wyniku
 - 35.29. Przeglądarka albumów z pliku
 - 35.30. Ustawienia aplikacji w Preferences
 - 35.31. Katalog produktów SQLite
 - 35.32. Czas, API i funkcje urządzenia
 - 35.33. Minutnik
 - 35.34. Panel urządzenia domowego
 - 35.35. Pogoda z API
 - 35.36. Lista użytkowników z API
 - 35.37. Informacje o urządzeniu i sieci
- 36. Najczęstsze błędy (Wyjątki i błędy kompilacji)
 - 36.1. NullReferenceException
 - 36.2. Błąd XFC0000 (Błąd parsowania XAML)
 - 36.3. Błąd XFC0009 (Brak właściwości)
 - 36.4. MissingMethodException (Brak konstruktora bezparametrowego)
 - 36.5. InvalidOperationException

- 36.6. HttpRequestException
- 36.7. TaskCanceledException
- 36.8. FileNotFoundException
- 36.9. Błąd CS0103 (Kompilacja)
- 36.10. Błąd CS1061 (Kompilacja)
- 36.11. FormatException
- 36.12. ArgumentOutOfRangeException
- 36.13. UnauthorizedAccessException
- 36.14. InvalidCastException
- 36.15. JsonException (System.Text.Json)
- 36.16. Błąd NETSDK1005 (Błąd Budowania)
- 36.17. Błąd CS0246 (Brak using)
- 36.18. TargetInvocationException (Błędy wiązań i XAML)
- 36.19. Java.Lang.Exception (Wyjątki specyficzne dla Androida)
- 36.20. TypeLoadException

1. Wprowadzenie do .NET MAUI

1.1. Czym jest .NET MAUI

.NET MAUI (pełna nazwa: **.NET Multi-platform App UI**) to nowoczesny **framework** firmy Microsoft, który pozwala tworzyć aplikacje z **jednego, wspólnego kodu** działające na wielu systemach naraz: **Android, iOS, macOS** oraz **Windows**. Słowo „framework” oznacza tu gotowy zestaw bibliotek, narzędzi i zasad, na których budujemy własny program - nie musimy pisać od zera obsługi okien, przycisków czy dotyku, bo dostarcza je MAUI. Najważniejsza idea brzmi: **piszesz interfejs i logikę raz, a framework uruchamia je na różnych platformach**. Pod spodem MAUI tłumaczy nasze kontrolki na **natywne** komponenty danego systemu - przycisk zdefiniowany w MAUI staje się prawdziwym przyciskiem Androida na telefonie i prawdziwym przyciskiem Windows na komputerze. Dzięki temu aplikacja wygląda i działa naturalnie na każdym urządzeniu. Do opisu wyglądu używamy języka **XAML**, a do logiki - języka **C#**. Całość działa na platformie **.NET**, czyli środowisku uruchomieniowym, które zarządza pamięcią i wykonywaniem kodu.

.NET MAUI służy do **budowania aplikacji z interfejsem graficznym** - takich, które użytkownik obsługuje dotykiem lub myszką, widząc na ekranie przyciski, pola, listy i obrazy. Najczęściej tworzy się w nim aplikacje **mobilne** (na telefony i tablety) oraz **desktopowe** (na komputery). Zamiast utrzymywać osobny projekt dla Androida (np. w Kotlinie) i osobny dla iOS (np. w Swift), używamy **jednej bazy kodu** w C# i XAML. To znacząco skraca czas tworzenia i obniża koszt utrzymania, bo poprawkę wprowadzamy raz, a trafia ona na wszystkie platformy. MAUI jest następcą technologii **Xamarin.Forms** i stanowi „oficjalną drogę” Microsoftu do tworzenia wieloplatformowych aplikacji w ekosystemie .NET. W praktyce programista spędza większość czasu w plikach `.xaml` (wygląd) i `.xaml.cs` (logika), rzadko schodząc do kodu specyficznego dla konkretnego systemu.

Kiedy używać?

Po .NET MAUI sięgamy wtedy, gdy chcemy **jedną aplikację na wiele platform** i pracujemy (lub chcemy pracować) w języku **C#**. Jest to świetny wybór dla zespołów znających .NET, które nie chcą uczyć się osobno Kotliny, Swifta i technologii desktopowych. Sprawdza się w aplikacjach biznesowych, narzędziach wewnętrznych, aplikacjach z formularzami, listami i prostą logiką, a także w wielu aplikacjach konsumenckich. MAUI wybierzemy też wtedy, gdy zależy nam na **natywnym wyglądzie** i wydajności, a nie chcemy budować aplikacji jako „opakowanej strony internetowej”. Jeżeli aplikacja ma działać i na telefonie, i na komputerze, jedna baza kodu MAUI jest bardzo atrakcyjna.

NAJWAŻNIEJSZE INFORMACJE

- **MAUI = jeden kod, wiele platform** (Android, iOS, macOS, Windows).
- Interfejs opisujemy w **XAML**, logikę piszemy w **C#**.
- Kontrolki są tłumaczone na **natywne** komponenty systemu.
- MAUI to **następca Xamarin.Forms**, oparty na nowoczesnym **.NET**.
- Architektura **jednego projektu** (Single Project) trzyma cały kod i zasoby w jednym miejscu.

*W tym podręczniku najczęściej mówimy o zastosowaniu **mobilnym** (telefon), bo to ono dominuje w nauce. Pamiętaj jednak, że niemal wszystkie pokazane mechanizmy działają identycznie również na komputerze.*

Na co uważać:

Początkujący często mylą .NET MAUI z technologiami webowymi (jak React czy Blazor). MAUI to aplikacje **natywne**, a nie strony w przeglądarce - choć istnieje hybrydowy `BlazorWebView`, to nie jest to domyślny sposób pracy. Druga pułapka to mylenie XAML z HTML: składnia bywa podobna, ale to różne języki o innych regułach. Wreszcie, choć kod jest wspólny, **niektóre rzeczy różnią się między platformami** (uprawnienia, ścieżki plików, wygląd kontroltek) - o tym piszemy w osobnym rozdziale.

1.2. Do czego służy .NET MAUI i jakie aplikacje można w nim tworzyć

Zakres zastosowań .NET MAUI jest bardzo szeroki, ponieważ framework daje dostęp do natywnych możliwości urządzeń: ekranu dotykowego, aparatu, lokalizacji, plików, sieci czy powiadomień. Dzięki temu w MAUI można zbudować praktycznie każdy typ aplikacji z interfejsem graficznym, która nie wymaga ekstremalnej, niskopoziomowej wydajności (jak zaawansowane gry 3D). Aplikacja MAUI to skompilowany program instalowany na urządzeniu, a nie strona otwierana w przeglądarce. Może działać offline, korzystać z lokalnej bazy danych i zapisywać pliki na urządzeniu.

NAJWAŻNIEJSZE INFORMACJE

Poniższa tabela pokazuje przykładowe kategorie aplikacji, które dobrze pasują do .NET MAUI:

KATEGORIA	PRZYKŁADY	DLACZEGO MAUI PASUJE
Aplikacje biznesowe	CRM, magazyn, raporty	formularze, listy, baza danych
Narzędzia produktywności	notatnik, lista zadań, kalkulator	prosta logika, zapis lokalny
Aplikacje z danymi z sieci	pogoda, wiadomości, sklep	komunikacja z API i JSON
Aplikacje katalogowe	katalog produktów, galeria	listy, obrazy, szczegóły
Panele i sterowanie	smart home, ustawienia	stan on/off, przełączniki
Proste gry i aplikacje edukacyjne	quizy, gry logiczne	obrazy, stan, interakcje

Kiedy używać?

Wyberzemy MAUI, gdy aplikacja ma standardowy interfejs (przyciski, pola, listy, obrazy) i ma działać na kilku platformach. Idealnie nadaje się do aplikacji, które pobierają dane, pokazują je w listach, pozwalają je edytować i zapisują lokalnie lub wysyłają do serwera.

Na co uważać:

Do bardzo wymagających graficznie gier (silniki 3D, zaawansowana fizyka) lepiej nadają się dedykowane silniki, np. Unity. MAUI obsługuje proste gry i animacje, ale nie zastąpi specjalistycznego silnika gier.

1.3. Jakie platformy obsługuje .NET MAUI

Platforma docelowa to system operacyjny, na którym uruchomi się aplikacja. .NET MAUI z jednego projektu potrafi zbudować wersje dla czterech platform. Każda z nich ma swoje wymagania narzędziowe, ale wspólny kod pozostaje ten sam. Wybór platform docelowych zapisany jest w pliku projektu (`.csproj`) w polu `TargetFrameworks` .

NAJWAŻNIEJSZE INFORMACJE

PLATFORMA	URZĄDZENIA	WYMAGANIA DO BUDOWANIA
Android	telefony i tablety z Androidem	Android SDK (działa na Windows i macOS)
iOS	iPhone, iPad	komputer Mac z Xcode
macOS	komputery Apple (Mac Catalyst)	komputer Mac z Xcode
Windows	komputery z Windows (WinUI 3)	Windows + Windows App SDK

Aby tworzyć i testować aplikacje na **iOS** oraz **macOS**, potrzebny jest komputer **Mac** - to wymóg Apple. Na Windows zbudujesz aplikację na **Android** i **Windows** bez przeszkód. Android można budować zarówno na Windows, jak i na macOS.

Na co uważać:

Nie trzeba od razu wspierać wszystkich platform. Na początku nauki zwykle wystarczy jedna (np. Windows lub Android), aby szybko uruchamiać aplikację. Dodatkowe platformy można dołączyć później, rozszerzając `TargetFrameworks` .

1.4. Czym różni się aplikacja mobilna od desktopowej

Aplikacja mobilna działa na telefonie lub tablecie, a **aplikacja desktopowa** - na komputerze. Choć MAUI pozwala pisać je z jednego kodu, warto rozumieć różnice w sposobie używania, bo wpływają one na projekt interfejsu. Telefon ma mały, dotykowy ekran trzymany zwykle w pionie, ograniczoną przestrzeń i pracuje na baterii. Komputer ma duży ekran (często w poziomie), mysz i klawiaturę oraz zwykle stały dostęp do zasilania. Te różnice sprawiają, że ten sam układ interfejsu może świetnie wyglądać na komputerze, a być ciasny na telefonie - i odwrotnie.

NAJWAŻNIEJSZE INFORMACJE

CECHA	APLIKACJA MOBILNA	APLIKACJA DESKTOPOWA
Ekran	mały, zwykle pionowy	duży, zwykle poziomy
Obsługa	dotyk (palec)	mysz i klawiatura
Przestrzeń	ograniczona	duża
Zasilanie	bateria (oszczędność energii)	zwykle stałe
Nawigacja	ekran po ekranie, gesty	okna, wiele paneli naraz

Kiedy używać?

Projektując ekran, myśl o tym, gdzie aplikacja będzie używana. Dla telefonu stosuj układy pionowe (`VerticalStackLayout`), duże, łatwe do dotknięcia przyciski i `ScrollView` dla dłuższych treści. Dla komputera możesz wykorzystać szerokość ekranu, np. dwie kolumny obok siebie w `Grid` .

Na co uważać:

Najczęstszy błąd to projektowanie wyłącznie pod jeden rodzaj urządzenia. Element o szerokości ustawionej „na sztywno” na 1000 pikseli będzie dobrze wyglądał na komputerze, ale wyjdzie poza ekran telefonu. Dlatego preferuj **elastyczne** rozmiary (gwiazdka `*` w `Grid` , `HorizontalOptions="Fill"`) zamiast sztywnych wartości.

1.5. Czym różni się .NET MAUI od WPF

WPF (Windows Presentation Foundation) to starsza technologia Microsoftu do budowy aplikacji **wyłącznie na Windows**. Podobnie jak MAUI, używa XAML do opisu interfejsu i C# do logiki, dlatego wiedza przenosi się między nimi. Kluczowa różnica jest taka, że **WPF działa tylko na Windows**, a **MAUI jest wieloplatformowe**. WPF ma za to bardzo dojrzały, rozbudowany system stylów, szablonów i bindowania, dopracowany przez lata na jednej platformie.

NAJWAŻNIEJSZE INFORMACJE

CECHA	WPF	.NET MAUI
Platformy	tylko Windows	Android, iOS, macOS, Windows
Język wyglądu	XAML	XAML
Język logiki	C#	C#
Kontener główny	<code>Window</code>	<code>Page</code> (np. <code>ContentPage</code>)
Dotyk	dodatkowo	natywnie (priorytet)
Zastosowanie	desktop Windows	mobilne + desktop

Kiedy używać?

WPF wybierzemy, gdy aplikacja ma działać **tylko na Windows** i chcemy wykorzystać jego dojrzałe możliwości desktopowe. MAUI wybierzemy, gdy zależy nam na **wielu platformach**, w tym mobilnych.

Na co uważać:

Mimo podobieństwa XAML, nazwy niektórych kontrolerek i layoutów różnią się (np. WPF ma `StackPanel` , MAUI ma `VerticalStackLayout` / `HorizontalStackLayout` ; WPF ma `Window` , MAUI ma `ContentPage`). Kod WPF nie uruchomi się bez zmian w MAUI i odwrotnie.

1.6. Czym różni się .NET MAUI od WinForms

WinForms (Windows Forms) to bardzo stara, ale wciąż używana technologia desktopowa Microsoftu na Windows. W odróżnieniu od MAUI i WPF, WinForms **nie używa XAML** - interfejs buduje się tam głównie przez przeciąganie kontrolki w projektancie, a kod jest generowany automatycznie. WinForms jest prosty i szybki do prototypowania prostych narzędzi, ale ma ograniczone możliwości stylowania i działa tylko na Windows.

NAJWAŻNIEJSZE INFORMACJE

CECHA	WINFORMS	.NET MAUI
Platformy	tylko Windows	wiele platform
Opis interfejsu	projektant + kod C#	XAML + C#
Stylowanie	ograniczone	bogate (style, zasoby)
Dotyk	słabe wsparcie	natywne
Binding danych	podstawowy	wygodny przy listach i formularzach

Kiedy używać?

WinForms bywa wybierany do bardzo prostych, wewnętrznych narzędzi na Windows, gdzie liczy się tempo, a nie wygląd. MAUI to nowocześniejszy wybór, szczególnie gdy potrzebujemy wielu platform lub dopracowanego interfejsu.

Na co uważać:

W WinForms układ często opiera się na bezwzględnych pozycjach kontrolki, co źle skaluje się na różnych ekranach. MAUI promuje układy elastyczne (layouty), dlatego sposób myślenia o rozmieszczaniu elementów jest tu inny.

1.7. Czym różni się .NET MAUI od Xamarin.Forms

Xamarin.Forms to bezpośredni **poprzednik** .NET MAUI. Był pierwszą szeroko stosowaną technologią Microsoftu do wieloplatformowego interfejsu w XAML i C#. MAUI to jego **ewolucja**: przepisano go tak, by korzystał z jednego, wspólnego projektu i nowego, szybszego mechanizmu renderowania kontrolki zwanego **Handlers** (zamiast starszych **Renderers**). Wiele nazw kontrolki pozostało identycznych (**Label**, **Button**, **Entry**), więc wiedza z Xamarin.Forms w dużej części przenosi się na MAUI.

NAJWAŻNIEJSZE INFORMACJE

CECHA	XAMARIN.FORMS	.NET MAUI
Struktura	osobne projekty per platforma	jeden wspólny projekt
Renderowanie	Renderers	Handlers (szybsze)
Podstawa	starszy stos Xamarin	nowoczesny .NET
Wsparcie	zakończone	aktywnie rozwijane
Nazwy kontrolki	Label , Button ...	w większości identyczne

Kiedy używać?

Nowe projekty zawsze zaczynamy w **.NET MAUI** - Xamarin.Forms nie jest już rozwijany. Znajomość Xamarin.Forms przydaje się głównie przy utrzymaniu starszych aplikacji i przy czytaniu starszych przykładów w internecie.

Na co uważać:

Część przykładów znalezionych w sieci dotyczy Xamarin.Forms. Zwykle da się je przenieść do MAUI z drobnymi zmianami (inne przestrzenie nazw, **VerticalStackLayout** zamiast **StackLayout**), ale nie zawsze działają „bez ruszania”.

1.8. Aplikacja wieloplatformowa i model „jeden projekt, wiele platform”

Aplikacja wieloplatformowa (ang. *cross-platform*) to aplikacja, która z jednego kodu działa na wielu systemach. W MAUI realizuje to **architektura jednego projektu** (Single Project): cały wspólny kod, zasoby (obrazy, czcionki) i ustawienia trzymane są w jednym miejscu, a framework sam buduje z nich wersję dla wybranej platformy. Elementy specyficzne dla danego systemu (np. uprawnienia Androida) trafiają do folderu **Platforms**, podzielonego na podkatalogi **Android**, **iOS**, **MacCatalyst** i **Windows**. W codziennej pracy programista w ogromnej większości przypadków pracuje wyłącznie ze wspólnym kodem.

NAJWAŻNIEJSZE INFORMACJE

- Wspólny kod i wygląd piszemy **raz**.
- Kod specyficzny dla platformy mieszka w folderze `Platforms`.
- Framework wybiera odpowiednie elementy w zależności od budowanej platformy.
- Zasoby (obrazy, czcionki) są **współdzielone** i automatycznie dostosowywane.

Na początku nauki możesz całkowicie zignorować folder `Platforms`. Cała Twoja praca odbywa się w plikach `.xaml` i `.xaml.cs` w głównym katalogu projektu.

Na co uważać:

„Jeden kod” nie znaczy „wszystko zawsze identyczne”. Czasem trzeba dodać drobny fragment dla konkretnej platformy (np. inne zachowanie klawiatury czy ścieżki plików). MAUI daje na to mechanizmy (`OnPlatform`), dyrektywy (`#if ANDROID`), które omawiamy w dalszej części.

1.9. Zalety i ograniczenia .NET MAUI

NAJWAŻNIEJSZE INFORMACJE

Każda technologia ma mocne i słabe strony. Świadomość obu pomaga podejmować dobre decyzje projektowe.

Zalety:

ZALETA	WYJAŚNIENIE
Jeden kod, wiele platform	mniej pracy i niższy koszt utrzymania
Język C# i .NET	spójność z ekosystemem Microsoftu
Natywny wygląd i wydajność	kontrolki tłumaczone na natywne
Bogaty XAML i binding	czytelny opis interfejsu i danych
Dostęp do funkcji urządzenia	aparat, GPS, pliki, sieć

Ograniczenia:

OGRANICZENIE	WYJAŚNIENIE
Konfiguracja środowiska	wymaga SDK, workloadów, czasem Maca
Niedoskonałości wieloplatformowe	drobne różnice między systemami
Nie do ciężkich gier 3D	lepsze są dedykowane silniki
Młodszy ekosystem niż natywny	mniej bibliotek niż czysty Android/iOS

Na co uważać:

Najczęstszą barierą na starcie jest **konfiguracja środowiska** (instalacja SDK, workloadu MAUI, emulatora). To jednorazowy koszt - po poprawnym ustawieniu praca jest już wygodna. Cierpliwość przy pierwszym uruchomieniu się opłaca.

1.10. Kiedy warto używać .NET MAUI, a kiedy wybrać inną technologię

Kiedy używać?

- Chcesz **jedną aplikację na telefon i komputer** z jednego kodu.
- Twój zespół zna lub chce poznać **C# i .NET**.
- Aplikacja ma typowy interfejs: formularze, listy, obrazy, prosta logika.
- Zależy Ci na **natywnym** wyglądzie i działaniu offline.

Kiedy wybrać inną technologię?

- Aplikacja ma działać **tylko na Windows** i jest czysto desktopowa → rozważ **WPF**.
- Tworzysz **zaawansowaną grę 3D** → rozważ silnik gier (np. Unity).
- Budujesz **stronę internetową / aplikację webową** → rozważ technologie webowe (np. Blazor, React).
- Potrzebujesz **maksymalnej, niskopoziomowej wydajności** na jednej platformie → rozważ kod natywny (Kotlin/Swift).

Na co uważać:

Wybór technologii to kompromis. MAUI błyszczy przy aplikacjach wieloplatformowych z typowym interfejsem. Gdy wymagania są skrajne (tylko jedna platforma, ekstremalna wydajność, czysty web), inne narzędzia mogą pasować lepiej. Dla większości aplikacji biznesowych i narzędziowych MAUI jest jednak bardzo dobrym, uniwersalnym wyborem.

W kolejnym rozdziale przygotujemy środowisko pracy: zainstalujemy potrzebne narzędzia, utworzymy pierwszy projekt i uruchomimy go. To fundament, bez którego nie ruszymy z praktyką.

2. Środowisko pracy

Zanim napiszemy pierwszą aplikację, musimy przygotować **środowisko pracy** - czyli zestaw narzędzi, które pozwolą pisać kod, kompilować go i uruchamiać na różnych urządzeniach. W tym rozdziale poznasz wszystkie potrzebne elementy, dowiesz się, jak je zainstalować, jak uruchomić aplikację na komputerze, w emulatorze i na prawdziwym telefonie, a także jak radzić sobie z najczęstszymi problemami. To jednorazowa inwestycja czasu - gdy środowisko jest gotowe, dalsza praca jest już wygodna i szybka.

2.1. Visual Studio

Visual Studio to **zintegrowane środowisko programistyczne** (ang. *IDE - Integrated Development Environment*) firmy Microsoft, dostępne na systemie Windows. IDE to zaawansowany program, który łączy w jednym miejscu edytor kodu, kompilator, debugger, narzędzia do projektowania interfejsu i zarządzania projektem. Dla .NET MAUI Visual Studio jest najwygodniejszym wyborem na Windows, bo ma wbudowane wsparcie dla MAUI: szablony projektów, podgląd XAML, **Hot Reload** i zarządzanie urządzeniami. Istnieje też wersja **Visual Studio dla Maca**, choć Microsoft kieruje użytkowników Maca raczej w stronę VS Code. Visual Studio występuje w edycji **Community**, która jest darmowa do nauki i wielu zastosowań.

Visual Studio służy do **pisania, uruchamiania i debugowania** aplikacji MAUI. Pozwala utworzyć projekt z gotowego szablonu, edytować pliki XAML i C# z podpowiedziami składni (IntelliSense), uruchamiać aplikację jednym kliknięciem na wybranym urządzeniu, zatrzymywać działanie programu w wybranym miejscu (breakpoint) i podglądać wartości zmiennych. To centrum dowodzenia całą pracą programisty.

NAJWAŻNIEJSZE INFORMACJE

- Visual Studio **Community** jest darmowe do nauki.
- Działa na **Windows** (na Macu używamy VS Code).
- Ma wbudowane wsparcie dla MAUI po zainstalowaniu odpowiedniego **workloadu**.
- Zawiera edytor kodu, debugger, podgląd XAML i menedżer urządzeń.

*Jeśli dopiero zaczynasz na Windows, **Visual Studio Community** to najprostsza droga. Większość rzeczy zrobisz tam „klikając”, bez ręcznego używania terminala.*

Na co uważać:

Nie myl **Visual Studio** z **Visual Studio Code** - to dwa różne programy o podobnej nazwie. Visual Studio to duże, pełne IDE (głównie Windows), a VS Code to lekki, uniwersalny edytor działający na wszystkich systemach. Oba można wykorzystać do MAUI, ale konfiguruje się je inaczej.

2.2. Visual Studio Code

Visual Studio Code (w skrócie **VS Code**) to lekki, darmowy i **wieloplatformowy** edytor kodu (Windows, macOS, Linux). Sam w sobie jest prostym edytorem, ale dzięki **rozszerzeniom** (extensions) można go zamienić w pełnoprawne środowisko do MAUI. Kluczowe jest tu rozszerzenie **.NET MAUI** oraz **C# Dev Kit**, które dodają obsługę projektów, budowania i uruchamiania aplikacji. VS Code jest popularny na komputerach Mac i tam często stanowi główne narzędzie pracy z MAUI.

VS Code służy do edycji kodu, budowania i uruchamiania aplikacji MAUI - szczególnie na systemach, gdzie nie ma pełnego Visual Studio (macOS, Linux). Po doinstalowaniu rozszerzeń pozwala wybrać platformę docelową, uruchomić aplikację i debugować ją, choć część czynności wykonuje się przez **paletę poleceń** lub terminal.

NAJWAŻNIEJSZE INFORMACJE

- VS Code jest **wieloplatformowy** i darmowy.
- Do MAUI potrzebuje rozszerzeń: **.NET MAUI** i **C# Dev Kit**.
- Wiele operacji wykonuje się przez paletę poleceń (**Ctrl/Cmd + Shift + P**) lub terminal.

- Jest lżejszy niż Visual Studio, ale wymaga więcej ręcznej konfiguracji.

Na co uważać:

W VS Code więcej rzeczy robi się „ręcznie” niż w Visual Studio (np. przez polecenia `dotnet`). To świetne narzędzie, ale początkującym na Windows zwykle łatwiej zacząć od Visual Studio. Pamiętaj też, by zainstalować właściwe rozszerzenia - bez nich VS Code nie „zna” projektów MAUI.

2.3. .NET SDK

.NET SDK (*Software Development Kit*) to podstawowy zestaw narzędzi platformy .NET, bez którego nic nie zbudujemy. Zawiera **kompilator** (zamienia kod C# na program), **środowisko uruchomieniowe** (.NET runtime) oraz narzędzie wiersza poleceń `dotnet`, którym tworzymy, budujemy i uruchamiamy projekty. To fundament całej pracy - zarówno Visual Studio, jak i VS Code korzystają pod spodem z .NET SDK. SDK występuje w wersjach numerowanych (np. .NET 8), a MAUI wymaga odpowiednio nowej wersji.

.NET SDK dostarcza wszystko, co potrzebne, by **z kodu źródłowego powstała działająca aplikacja**. Dzięki poleceniu `dotnet` możemy pracować nawet bez graficznego IDE - utworzyć projekt, skompilować go i uruchomić z terminala.

NAJWAŻNIEJSZE INFORMACJE

- SDK zawiera **kompilator**, **runtime** i narzędzie `dotnet`.
- MAUI wymaga odpowiednio nowej wersji .NET (np. .NET 8 lub nowszy).
- SDK instaluje się raz; może współistnieć kilka wersji obok siebie.

PRZYKŁAD C#

Sprawdzenie zainstalowanej wersji SDK z terminala:

```
# Wyświetla wersję zainstalowanego .NET SDK
dotnet --version

# Wyświetla szczegółowe informacje o zainstalowanych wersjach i środowisku
dotnet --info
```

Na co uważać:

Jeśli polecenie `dotnet` zwraca błąd „nie rozpoznano polecenia”, oznacza to, że SDK nie jest zainstalowany lub nie został dodany do zmiennej środowiskowej `PATH`. Po instalacji uruchom terminal ponownie, by odświeżyć ścieżki.

2.4. Workload .NET MAUI

Workload to dodatkowy pakiet rozszerzający .NET SDK o wsparcie dla konkretnego typu projektów. Workload o nazwie `maui` dodaje szablony projektów MAUI oraz biblioteki potrzebne do budowania aplikacji na Android, iOS, macOS i Windows. Bez tego workloadu .NET SDK „nie wie”, jak zbudować aplikację MAUI - zabraknie szablonów i bibliotek. Workload instaluje się raz i aktualizuje wraz z nowymi wersjami.

Workload MAUI **uzupełnia SDK o wszystko, co specyficzne dla MAUI**: szablony (`dotnet new maui`), kontrolki, mechanizm Handlers oraz integrację z platformami. To on sprawia, że w ogóle możemy tworzyć projekty MAUI.

NAJWAŻNIEJSZE INFORMACJE

POLECENIE	DZIAŁANIE
<code>dotnet workload list</code>	pokazuje zainstalowane workloady
<code>dotnet workload install maui</code>	instaluje workload MAUI
<code>dotnet workload update</code>	aktualizuje workloady do najnowszych wersji
<code>dotnet workload repair</code>	naprawia uszkodzoną instalację workloadów

PRZYKŁAD C#

```
# Sprawdź, czy workload MAUI jest zainstalowany
dotnet workload list

# Zainstaluj workload MAUI (jeśli go brakuje)
dotnet workload install maui

# Zaktualizuj workloady
dotnet workload update
```

Na co uważać:

W Visual Studio workload MAUI instaluje się automatycznie, gdy w instalatorze zaznaczysz opcję „**.NET Multi-platform App UI development**”. Jeśli brak szablonu MAUI przy tworzeniu projektu, to najczęstszy znak, że workload nie został zainstalowany.

2.5. Android SDK

Android SDK to zestaw narzędzi Google'a niezbędny do budowania i uruchamiania aplikacji na Androidzie. Zawiera m.in. kompilatory, biblioteki systemu Android, narzędzia do podpisywania aplikacji oraz menedżer urządzeń wirtualnych (emulatorów). MAUI korzysta z Android SDK „pod spodem”, gdy budujemy wersję aplikacji na Android. W Visual Studio Android SDK instaluje się razem z workloadem MAUI, więc zwykle nie trzeba robić tego ręcznie.

Android SDK umożliwia **zbudowanie pliku aplikacji Androida** (APK/AAB) oraz **uruchomienie jej** na emulatorze lub fizycznym telefonie. Bez niego nie zbudujemy wersji na Android.

NAJWAŻNIEJSZE INFORMACJE

- Android SDK jest potrzebny tylko do platformy **Android**.
- W Visual Studio instaluje się zwykle automatycznie z workloadem MAUI.
- Wymaga zaakceptowania **licencji** Google'a (czasem trzeba to zrobić ręcznie).
- Zawiera **Android Emulator** do testów bez fizycznego urządzenia.

Na co uważać:

Częstym problemem jest brak akceptacji licencji Android SDK, co blokuje budowanie. Wtedy trzeba uruchomić odpowiednie narzędzie (np. menedżer SDK w Visual Studio) i zaakceptować warunki. Android SDK potrafi też zajmować dużo miejsca na dysku.

2.6. Emulator Androida a fizyczny telefon

Emulator Androida to wirtualny telefon uruchamiany na komputerze - pozwala testować aplikację bez posiadania fizycznego urządzenia. **Fizyczny telefon** to prawdziwe urządzenie podłączone do komputera (kablem USB lub po sieci), na którym uruchamiamy aplikację. Oba sposoby pozwalają sprawdzić działanie aplikacji na Androidzie, ale różnią się wygodą i wiernością.

NAJWAŻNIEJSZE INFORMACJE

CECHA	EMULATOR	FIZYCZNY TELEFON
Koszt	brak (część SDK)	trzeba mieć urządzenie
Wydajność	wolniejszy, obciąża komputer	szybki, realny
Wierność	bardzo dobra, ale nie 100%	w pełni realne zachowanie
Aparat, GPS, czujniki	symulowane	prawdziwe
Konfiguracja	utworzenie urządzenia wirtualnego (AVD)	włączenie trybu dewelopera + USB

Kiedy używać?

Do codziennej pracy nad wyglądem i logiką wystarcza **emulator**. Gdy testujesz funkcje sprzętowe (aparat, GPS, czujniki) lub realną wydajność i dotyk, użyj **fizycznego telefonu**.

Na co uważać:

Emulator potrafi być wolny, zwłaszcza na słabszych komputerach lub bez włączonej akceleracji sprzętowej (Hyper-V / wirtualizacja w BIOS). Fizyczny telefon wymaga włączenia **trybu dewelopera** i **debugowania USB** w ustawieniach systemu Android.

2.7. Uruchamianie aplikacji na Windows

Uruchomienie aplikacji „na Windows” oznacza zbudowanie i odpalenie jej jako natywnej aplikacji **Windows (WinUI 3)** bezpośrednio na komputerze, na którym pracujesz. To zwykle **najszybszy** sposób uruchomienia aplikacji podczas nauki, bo nie wymaga emulatora ani telefonu – aplikacja startuje jak zwykły program na pulpicie.

NAJWAŻNIEJSZE INFORMACJE

- To najszybsza ścieżka uruchomienia podczas nauki na Windows.
- Wymaga komponentu **Windows App SDK** (instalowany z workloadem).
- W Visual Studio wybierasz cel **Windows Machine** i klikasz uruchom.

PRZYKŁAD C#

```
# Uruchomienie aplikacji na Windows z terminala
dotnet build -t:Run -f net8.0-windows10.0.19041.0
```

Na co uważać:

Aby uruchomić aplikację na Windows w trybie deweloperskim, system może wymagać włączenia **trybu programisty** w ustawieniach Windows. Wygląd na Windows może drobnie różnić się od telefonu – to normalne.

2.8. Uruchamianie aplikacji na Androidzie i na emulatorze

Uruchomienie na Androidzie to zbudowanie wersji aplikacji dla tego systemu i odpalenie jej na **emulatorze** lub **fizycznym telefonie**. W Visual Studio wybieramy urządzenie z listy (emulator lub podłączony telefon) i klikamy uruchom; środowisko zbuduje aplikację, zainstaluje ją na urządzeniu i uruchomi.

NAJWAŻNIEJSZE INFORMACJE

- Najpierw musi istnieć **urządzenie wirtualne (AVD)** lub podłączony telefon.
- Pierwsze uruchomienie trwa dłużej (budowanie + instalacja).
- Emulator trzeba czasem wcześniej uruchomić, by pojawił się na liście urządzeń.

PRZYKŁAD C#

```
# Uruchomienie na podłączonym urządzeniu/emulatorze Androida
dotnet build -t:Run -f net8.0-android
```

Na co uważać:

Jeśli emulator nie pojawia się na liście, sprawdź, czy został utworzony w menedżerze urządzeń i czy jest uruchomiony. Brak akceleracji sprzętowej sprawia, że emulator działa bardzo wolno.

2.9. Uruchamianie aplikacji na fizycznym telefonie

To uruchomienie aplikacji na **prawdziwym urządzeniu** podłączonym do komputera. Daje najwierniejszy obraz działania aplikacji – realny dotyk, wydajność i dostęp do sprzętu. Aby to zrobić, telefon musi mieć włączony **tryb dewelopera** i **debugowanie USB**.

NAJWAŻNIEJSZE INFORMACJE

1. W telefonie wejdź w **Ustawienia** → **Informacje o telefonie** i kilka razy dotknij „Numer kompilacji”, aby włączyć tryb dewelopera.
2. W opcjach deweloperskich włącz **Debugowanie USB**.
3. Podłącz telefon kablem i zaakceptuj prośbę o zaufanie komputerowi.
4. Wybierz urządzenie w środowisku i uruchom aplikację.

Na co uważać:

Niektóre telefony wymagają dodatkowych sterowników USB (zwłaszcza na Windows). Jeśli urządzenie nie jest wykrywane, sprawdź kabel (musi przesyłać dane, nie tylko ładować) oraz potwierdzenie zaufania na ekranie telefonu.

2.10. Tryb Debug i tryb Release

To dwie podstawowe **konfiguracje budowania** aplikacji. **Debug** jest przeznaczony dla programisty - kod nie jest zoptymalizowany, ale zawiera pełne informacje diagnostyczne, dzięki czemu można ustawiać breakpointy i krok po kroku analizować działanie. **Release** służy do dystrybucji - kod jest zoptymalizowany pod kątem wydajności i rozmiaru, a dane debugowania są usuwane. Aplikacje publikowane w sklepach budujemy zawsze w trybie **Release**.

NAJWAŻNIEJSZE INFORMACJE

CECHA	DEBUG	RELEASE
Optimalizacja kodu	nie	tak
Informacje diagnostyczne	pełne	usunięte
Breakpointy	tak	ograniczone
Wydajność	niższa	wyższa
Dyrektywy <code>#if DEBUG</code>	aktywne	nieaktywne
Przeznaczenie	praca programisty	publikacja

PRZYKŁAD C#

```
#if DEBUG
// Ten kod wykona się tylko w trybie Debug
Console.WriteLine("Tryb debugowania - dodatkowe logi");
#endif
```

Na co uważać:

Aplikację, którą oddajesz użytkownikom, zawsze testuj również w trybie **Release** - działa szybciej i bez danych debugowania, więc niektóre błędy ujawniają się dopiero tam. W trybie Debug aplikacja bywa wyraźnie wolniejsza, co jest normalne.

2.11. Hot Reload

Hot Reload to funkcja pozwalająca **wprowadzać zmiany w kodzie bez ponownego uruchamiania aplikacji**. Gdy zmienisz coś w XAML (np. kolor, tekst, odstęp) lub w ciele metody C#, efekt pojawia się natychmiast na działającym urządzeniu czy emulatorze. To ogromnie przyspiesza pracę nad wyglądem, bo nie trzeba za każdym razem czekać kilkadziesiąt sekund na pełne przebudowanie.

Hot Reload służy do **szybkiego iterowania** nad interfejsem i logiką. Zmieniasz `Padding`, `FontSize` czy treść etykiety i od razu widzisz rezultat - idealne do „dopieszczania” wyglądu.

NAJWAŻNIEJSZE INFORMACJE

- Działa najlepiej przy zmianach w **XAML** oraz w **ciągach metod**.
- Niektóre zmiany strukturalne (nowe pole, zmiana sygnatury metody, nowa kontrolka z `x:Name`) wymagają **pełnego restartu**.
- Dostępny w Visual Studio i VS Code podczas debugowania.

Na co uważać:

Gdy zmiana „nie chce się pojawić”, zwykle wystarczy ponownie uruchomić aplikację. Nie wszystkie modyfikacje da się zastosować na żywo - to normalne ograniczenie, a nie błąd.

2.12. Pierwszy projekt - utworzenie i struktura

Pierwszy projekt to świeża aplikacja utworzona z gotowego **szablону MAUI**. Szablon tworzy kompletną, działającą aplikację „Hello World”, którą można od razu uruchomić, a następnie modyfikować. To najlepszy punkt startu, bo dostajemy poprawnie skonfigurowaną strukturę plików.

PRZYKŁAD C#

```
# Utworzenie nowego projektu MAUI o nazwie MojaPierwszaApp
dotnet new maui -n MojaPierwszaApp
```

```
# Wejście do katalogu projektu
cd MojaPierwszaApp

# Zbudowanie projektu (sprawdzenie, czy wszystko działa)
dotnet build
```

NAJWAŻNIEJSZE INFORMACJE

Po utworzeniu projektu zobaczysz m.in. takie elementy:

```
MojaPierwszaApp/
├─ App.xaml / App.xaml.cs      # punkt startowy aplikacji
├─ AppShell.xaml / .xaml.cs     # nawigacja (powłoka)
├─ MainPage.xaml / .xaml.cs    # pierwszy ekran (widok + logika)
├─ MauiProgram.cs              # konfiguracja aplikacji
├─ Resources/                  # obrazy, czcionki, style
└─ Platforms/                  # kod specyficzny dla platform
```

Na co uważać:

W Visual Studio projekt tworzysz przez **Plik → Nowy → Projekt** i wybór szablonu **.NET MAUI App**. Upewnij się, że wybierasz właśnie ten szablon, a nie „MAUI Class Library” czy „MAUI Blazor”.

2.13. Typowe problemy z konfiguracją i najczęstsze błędy pierwszego uruchomienia

NAJWAŻNIEJSZE INFORMACJE

Pierwsze uruchomienie bywa źródłem frustracji, bo środowisko składa się z wielu współpracujących elementów. Poniższa tabela zbiera najczęstsze problemy i ich rozwiązania:

OBJAW	PRAWDOPODOBNA PRZYCZYNA	ROZWIĄZANIE
Brak szablonu MAUI	brak workloadu	<code>dotnet workload install maui</code>
Emulator się nie uruchamia	brak urządzenia wirtualnego	utwórz AVD w menedżerze urządzeń
Emulator bardzo wolny	brak akceleracji sprzętowej	włącz wirtualizację w BIOS/Hyper-V
Błąd licencji Android SDK	nieakceptowana licencja	zaakceptuj licencję w menedżerze SDK
Telefon niewykrywany	brak debugowania USB / zły kabel	włącz tryb dewelopera, zmień kabel
<code>dotnet</code> nierozpoznane	brak SDK lub złe <code>PATH</code>	zainstaluj SDK, zrestartuj terminal
Długie pierwsze budowanie	pobieranie zależności	poczekaj - to jednorazowe
Zmiana się nie pojawia	nieodświeżony build	przebuduj projekt (Rebuild)

Na co uważać:

Większość problemów ze środowiskiem rozwiązuje jedna z trzech czynności: **przebudowanie projektu** (Clean + Rebuild), **aktualizacja workloadów** (`dotnet workload update`) lub **ponowne uruchomienie środowiska/komputera**. Jeśli coś nie działa po instalacji, najpierw spróbuj tych kroków, zanim zaczniesz szukać bardziej złożonych przyczyn.

Pierwsze budowanie projektu zawsze trwa najdłużej, bo pobierane są zależności i przygotowywane są zasoby. Kolejne uruchomienia są znacznie szybsze. Nie zniechęcaj się długim startem za pierwszym razem.

Gdy utworzysz projekt MAUI, środowisko wygeneruje zestaw plików i folderów o ściśle określonych rolach. Zrozumienie, za co odpowiada każdy z nich, jest fundamentem - pozwala szybko odnaleźć właściwe miejsce na nowy kod i uniknąć chaosu. W tym rozdziale omówimy plik po pliku i folder po folderze całą strukturę projektu, a na końcu pokażemy, jak rozsądnie zorganizować projekt od samego początku.

3. Pierwszy projekt i struktura projektu

3.1. Przegląd struktury projektu

Struktura projektu to **uporządkowany zestaw plików i katalogów**, z których składa się aplikacja. MAUI używa **jednego projektu** dla wszystkich platform, więc cały wspólny kod znajduje się w jednym miejscu, a elementy zależne od systemu - w folderze **Platforms**. Poniższy diagram pokazuje typowy układ świeżo utworzonej aplikacji.

NAJWAŻNIEJSZE INFORMACJE



ELEMENT	ROLA
.csproj	ustawienia projektu i budowania
MauiProgram.cs	startowa konfiguracja aplikacji
App.xaml(.cs)	start aplikacji, zasoby globalne
AppShell.xaml(.cs)	struktura nawigacji
MainPage.xaml(.cs)	pierwszy ekran (widok + logika)
Resources/	obrazy, czcionki, style, pliki surowe
Platforms/	kod zależny od systemu

Na co uważać:

Nie usuwaj ani nie zmieniaj po prostu plików **App**, **AppShell** i **MauiProgram** - to one uruchamiają i konfiguruje aplikację. Folder **Platforms** na początku można zostawić w spokoju; zajrzyć tam dopiero przy zaawansowanych funkcjach.

3.2. Plik **.csproj**

Plik **.csproj** (np. **MojaApp.csproj**) to **plik projektu** w formacie XML, który opisuje, jak aplikacja ma zostać zbudowana. Znajdziesz w nim m.in. listę platform docelowych, nazwę i wersję aplikacji, dołączone obrazy oraz czcionki, a także listę pakietów **NuGet** (zewnętrznych

bibliotek). To „instrukcja budowy” całego projektu.

`.csproj` służy do **konfiguracji budowania**: określa, na jakie systemy budujemy (`TargetFrameworks`), jak nazywa się aplikacja, jaki ma identyfikator i wersję, oraz jakie zasoby i biblioteki dołączamy. Edytujemy go rzadko, ale przy dodawaniu pakietów czy zmianie platform docelowych zaglądamy właśnie tutaj.

NAJWAŻNIEJSZE INFORMACJE

WPIS	ZNACZENIE
<code>TargetFrameworks</code>	platformy docelowe (android, ios, windows...)
<code>UseMaui</code>	włącza funkcje MAUI
<code>SingleProject</code>	architektura jednego projektu
<code>ApplicationTitle</code>	nazwa aplikacji widoczna dla użytkownika
<code>ApplicationId</code>	unikalny identyfikator (np. <code>com.firma.app</code>)
<code>ApplicationDisplayVersion</code>	wersja widoczna (np. <code>1.0</code>)
<code>MauiImage</code> / <code>MauiFont</code>	dołączane obrazy i czcionki
<code>PackageReference</code>	pakiet NuGet (zewnętrzna biblioteka)

PRZYKŁAD

```
<Project Sdk="Microsoft.NET.Sdk">
  <PropertyGroup>
    <!-- Platformy docelowe -->
    <TargetFrameworks>net8.0-android;net8.0-ios;net8.0-windows10.0.19041.0</TargetFrameworks>
    <OutputType>Exe</OutputType>
    <UseMaui>true</UseMaui>
    <SingleProject>true</SingleProject>

    <!-- Tożsamość aplikacji -->
    <ApplicationTitle>Moja App</ApplicationTitle>
    <ApplicationId>com.firma.mojaapp</ApplicationId>
    <ApplicationDisplayVersion>1.0</ApplicationDisplayVersion>
    <ApplicationVersion>1</ApplicationVersion>
  </PropertyGroup>

  <ItemGroup>
    <!-- Ikona, splash, obrazy, czcionki -->
    <MauiIcon Include="Resources\AppIcon\appicon.svg" />
    <MauiSplashScreen Include="Resources\Splash\splash.svg" Color="#512BD4" />
    <MauiImage Include="Resources\Images\*" />
    <MauiFont Include="Resources\Fonts\*" />
  </ItemGroup>

  <ItemGroup>
    <!-- Pakiet NuGet, np. do lokalnej bazy SQLite -->
    <PackageReference Include="sqlite-net-pcl" Version="1.9.172" />
  </ItemGroup>
</Project>
```

TYPOWE BŁĘDY

- Ręczna edycja z błędem składni XML (np. niezamknięty znacznik) - projekt przestaje się budować.

- Dodanie pakietu w złej wersji, niezgodnej z używanym .NET.
- Zapomnienie o **przebudowaniu** projektu po zmianie `.csproj`.

Na co uważać:

Po każdej zmianie w `.csproj` wykonaj **przebudowę** (Rebuild). Środowisko musi ponownie wczytać konfigurację, inaczej zmiany nie zadziałają.

3.3. `MauiProgram.cs`

`MauiProgram.cs` to plik startowej konfiguracji aplikacji. Zawiera metodę `CreateMauiApp()`, która buduje aplikację MAUI, wskazuje klasę `App`, włącza fonty i ustawia podstawowe opcje projektu. W prostych aplikacjach najczęściej zaglądasz tu tylko wtedy, gdy dodajesz własne czcionki albo dodatkową bibliotekę.

Nie umieszczaj tutaj logiki aplikacji. Kod obsługi przycisków, formularzy, list i obrazów pisz w `MainPage.xaml.cs` albo w prostych klasach pomocniczych.

PRZYKŁAD C#

```
using Microsoft.Extensions.Logging;

namespace MojaApp;

public static class MauiProgram
{
    public static MauiApp CreateMauiApp()
    {
        var builder = MauiApp.CreateBuilder();
        builder
            .UseMauiApp<App>()
            .ConfigureFonts(fonts =>
            {
                fonts.AddFont("OpenSans-Regular.ttf", "OpenSansRegular");
            });

        #if DEBUG
            builder.Logging.AddDebug();
        #endif

        return builder.Build();
    }
}
```

TYPOWE BŁĘDY

- Brak wywołania `UseMauiApp<App>()` - aplikacja nie wie, co uruchomić.
- Literówka w nazwie pliku czcionki - font nie zostanie załadowany.
- Wpisywanie logiki widoku w `MauiProgram.cs` - ten plik powinien tylko konfigurować aplikację.

Na co uważać:

Po zmianach w `MauiProgram.cs` wykonaj ponowne uruchomienie aplikacji. Hot Reload nie zawsze wystarcza przy zmianach konfiguracji.

3.4. `App.xaml` i `App.xaml.cs`

Para plików `App.xaml` i `App.xaml.cs` definiuje klasę `App`, czyli obiekt reprezentujący całą aplikację. `App.xaml` przechowuje **zasoby globalne** (style, kolory, słowniki) dostępne na każdej stronie. `App.xaml.cs` zawiera kod tworzony przy starcie i ustawia **stronę startową** (`MainPage` lub `AppShell`).

App decyduje, **co użytkownik zobaczy najpierw**, oraz udostępnia zasoby wspólne dla całej aplikacji. To także miejsce na reakcję na zdarzenia cyklu życia aplikacji (start, uśpienie, wznowienie).

PRZYKŁAD

```

<!-- App.xaml -->
<Application xmlns="http://schemas.microsoft.com/dotnet/2021/maui"
              xmlns:x="http://schemas.microsoft.com/winfx/2009/xaml"
              x:Class="MojaApp.App">
  <Application.Resources>
    <ResourceDictionary>
      <Color x:Key="KolorPrimary">#512BD4</Color>
    </ResourceDictionary>
  </Application.Resources>
</Application>

```

```

// App.xaml.cs
namespace MojaApp;

public partial class App : Application
{
    public App()
    {
        InitializeComponent();
        MainPage = new AppShell(); // strona startowa (lub new MainPage())
    }
}

```

TYPOWE BŁĘDY

- Usunięcie `InitializeComponent()` w konstruktorze - zasoby z `App.xaml` nie zostaną wczytane.
- Próba odwołania się do zasobu, którego nie zdefiniowano w `App.xaml`.

Na co uważać:

Zasoby globalne (kolory, style) najwygodniej trzymać właśnie w `App.xaml`, by były dostępne wszędzie. Dla prostych aplikacji możesz ustawić `MainPage = new MainPage();` zamiast `AppShell`.

3.5. `AppShell.xaml` i `AppShell.xaml.cs`

`AppShell` to klasa opisująca **strukturę nawigacji** aplikacji za pomocą mechanizmu **Shell**. W `AppShell.xaml` deklarujemy zakładki, menu boczne i ekrany, a w `AppShell.xaml.cs` rejestrujemy dodatkowe trasy (routing). Shell upraszcza przemieszczanie się między ekranami.

`AppShell` definiuje „szkielet” aplikacji: jakie ma sekcje, jak się między nimi przechodzi i jak wygląda menu. To wygodny sposób organizacji większych aplikacji z wieloma ekranami.

PRZYKŁAD

```

<!-- AppShell.xaml -->
<Shell xmlns="http://schemas.microsoft.com/dotnet/2021/maui"
       xmlns:x="http://schemas.microsoft.com/winfx/2009/xaml"
       xmlns:local="clr-namespace:MojaApp"
       x:Class="MojaApp.AppShell">
  <ShellContent Title="Start"

```

```
        ContentTemplate="{DataTemplate local:MainPage}"
        Route="start" />
</Shell>
```

Na co uważać:

W bardzo prostych aplikacjach jednoekranowych Shell nie jest konieczny - można ustawić `MainPage = new MainPage();` w `App.xaml.cs`. Shell zaczyna się oplotać przy kilku ekranach i nawigacji między nimi.

3.6. MainPage.xaml i MainPage.xaml.cs

`MainPage` to pierwszy ekran aplikacji. Składa się z dwóch plików: `MainPage.xaml` (widok - co widać) oraz `MainPage.xaml.cs` (logika - co się dzieje, czyli *code-behind*). Oba tworzą jedną klasę dzięki słowu kluczowemu `partial`.

To na `MainPage` (i kolejnych stronach) budujemy interfejs i obsługujemy interakcje użytkownika. Większość pracy programisty MAUI odbywa się właśnie w takich parach plików `.xaml` + `.xaml.cs`.

PRZYKŁAD

```
<!-- MainPage.xaml -->
<ContentPage xmlns="http://schemas.microsoft.com/dotnet/2021/maui"
              xmlns:x="http://schemas.microsoft.com/winfx/2009/xaml"
              x:Class="MojaApp.MainPage">
    <VerticalStackLayout Padding="20" Spacing="10">
        <Label x:Name="Etykieta" Text="Witaj!" FontSize="24" />
        <Button Text="Kliknij" Clicked="OnKliknij" />
    </VerticalStackLayout>
</ContentPage>
```

```
// MainPage.xaml.cs
namespace MojaApp;

public partial class MainPage : ContentPage
{
    public MainPage()
    {
        InitializeComponent();
    }

    private void OnKliknij(object sender, EventArgs e)
    {
        Etykieta.Text = "Kliknięto przycisk!";
    }
}
```

TYPOWE BŁĘDY

- Niezgodność `x:Class` w XAML z nazwą/przestrzenią klasy w `.cs`.
- Odwołanie do kontrolki bez nadania jej `x:Name`.

Na co uważać:

`x:Class` w pliku XAML musi dokładnie odpowiadać nazwie klasy i przestrzeni nazw w pliku `.cs`. To ten atrybut „spina” widok z logiką.

3.7. Folder `Resources`

Folder `Resources` przechowuje **zasoby współdzielone** aplikacji: obrazy, czcionki, style, ikonę i ekran powitalny oraz dowolne pliki surowe. MAUI automatycznie dba o ich przygotowanie dla różnych platform i rozdzielczości.

NAJWAŻNIEJSZE INFORMACJE

PODFOLDER	ZAWARTOŚĆ
<code>Resources/Images</code>	obrazy (PNG, SVG) używane w aplikacji
<code>Resources/Fonts</code>	czcionki (TTF/OTF) dołączone do aplikacji
<code>Resources/Raw</code>	dowolne pliki surowe, np. <code>dane.txt</code> , JSON
<code>Resources/Styles</code>	globalne style i kolory (<code>Styles.xml</code> , <code>Colors.xml</code>)
<code>Resources/AppIcon</code>	ikona aplikacji
<code>Resources/Splash</code>	ekran powitalny (splash screen)

Nazwy plików obrazów muszą być pisane **małymi literami**, bez spacji i myślników (np. `logo.png`, `kostka1.png`). Plik `Moja Grafika.png` spowoduje błąd budowania.

Na co uważać:

Po dodaniu nowego obrazu lub czcionki często trzeba **przebudować** projekt, aby MAUI je zarejestrował. Pliki z `Resources/Raw` to dobre miejsce na dane startowe aplikacji wczytywane przy uruchomieniu.

3.8. Folder `Platforms`

Folder `Platforms` zawiera **kod i ustawienia specyficzne dla konkretnego systemu**. Ma podkatalogi `Android`, `iOS`, `MacCatalyst` i `Windows`. Znajdują się tam m.in. pliki startowe każdej platformy, deklaracje uprawnień (np. `AndroidManifest.xml`) czy konfiguracja `Info.plist` dla iOS.

`Platforms` pozwala dostosować aplikację do wymagań danego systemu - zadeklarować uprawnienia, ustawić specyficzne opcje startu czy napisać kod działający tylko na jednej platformie.

Na co uważać:

Na początku nauki zwykle nie zaglądasz do `Platforms`. Zajrzysz tam dopiero, gdy będziesz potrzebować uprawnień (np. do aparatu na Androidzie) lub kodu specyficznego dla platformy.

3.9. Obrazy, czcionki, pliki lokalne i zasoby aplikacji

To wszystkie **dodatkowe materiały**, z których korzysta aplikacja. **Obrazy i czcionki** dołączamy w `Resources` i odwołujemy się do nich po nazwie. **Pliki lokalne** to dane zapisywane lub odczytywane na urządzeniu w trakcie działania. **Zasoby aplikacji** (kolory, style) trzymamy w `ResourceDictionary`.

NAJWAŻNIEJSZE INFORMACJE

- Obraz** w XAML: `Source="logo.png"` (plik z `Resources/Images`).
- Czcionka**: rejestrujemy w `MauiProgram.cs` z aliasem, potem `FontFamily="OpenSansRegular"`.
- Plik startowy** (np. `dane.txt`) umieszczamy w `Resources/Raw`.
- Pliki użytkownika** zapisujemy w katalogu `FileSystem.AppDataDirectory`.

Na co uważać:

Rozróżniam **zasoby projektu** (dołączone do aplikacji, tylko do odczytu, np. obraz w `Resources/Images`) od **plików zapisywanych przez aplikację** (tworzone w trakcie działania w `AppDataDirectory`). To dwa różne światy - pierwszego nie nadpiszesz, drugi służy do trwałego zapisu danych użytkownika.

3.10. Jak organizować projekt od początku

NAJWAŻNIEJSZE INFORMACJE

Dobra organizacja od pierwszego dnia oszczędza wielu problemów później. Zalecany, prosty układ to:

```
MojaApp/
├─ Models/      # klasy danych
├─ Views/       # ekrany (.xaml)
├─ Helpers/     # walidacja, obliczenia, klasy pomocnicze
├─ Data/        # baza, pliki, dostęp do danych
├─ Helpers/     # narzędzia pomocnicze
├─ Converters/  # konwertery bindingu
└─ Resources/   # obrazy, czcionki, style
```

Zasady:

- **Jedna odpowiedzialność na plik** - model trzyma dane, klasa pomocnicza logikę, widok wygląd.
- **Czytelne nazwy** - `ProduktyPage`, `ProduktyBaza`, `Produkt`.
- **Nie wrzucaj wszystkiego do** `MainPage` - dziel kod na warstwy.

Na co uważać:

Nie musisz od razu używać wszystkich folderów. Ale nawet w prostym projekcie warto wydzielić przynajmniej `Models`, `Helpers` i `Data` - to ułatwia późniejszą rozbudowę i sprawia, że kod jest czytelniejszy.

Porządek w projekcie to nie „ozdoba”, lecz realna oszczędność czasu. W uporządkowanym projekcie szybciej znajdujesz pliki, łatwiej dodajesz funkcje i rzadziej powielasz kod.

4. Podstawy C#: typy, operatory i tekst

Logikę aplikacji MAUI piszemy w języku **C#** (czytamy „si szarp”). Ten rozdział to **kurs C# od zera** - zakładamy, że nigdy wcześniej nie programowałeś. Tłumaczymy każde pojęcie dokładnie, z wieloma przykładami, abyś po jego przerobieniu rozumiał kod we wszystkich pozostałych rozdziałach i potrafił napisać własną logikę. Czytaj po kolei i przepisuj przykłady samodzielnie - programowania uczymy się przez praktykę, a nie przez samo czytanie.

4.1. Jak działa program i z czego składa się kod

Program komputerowy to **ciąg instrukcji**, które komputer wykonuje **po kolei, od góry do dołu**. Każda instrukcja to jedno polecenie - „zapamiętaj liczbę”, „dodaj dwie wartości”, „pokaż tekst”. W C# większość instrukcji kończymy **średnikiem** `;`, który mówi „to koniec tego polecenia”. Brak średnika to jeden z najczęstszych błędów początkujących i powoduje, że kod się nie kompiluje.

Kod grupujemy w **bloki** otoczone nawiasami klamrowymi `{ }`. Blok to zestaw instrukcji traktowanych jako całość - np. ciało metody, wewnątrz pętli czy warunku. Instrukcje wewnątrz bloku zwykle **wcinamy** (przesuwamy w prawo), aby kod był czytelny; wcięcia nie są wymagane przez język, ale są obowiązkowym dobrym nawykiem.

W C# **wielkość liter ma znaczenie** - `wiek`, `Wiek` i `WIEK` to trzy różne nazwy. Język rozróżnia też **słowa kluczowe** (zarezerwowane, np. `int`, `if`, `for`) od nazw, które sami nadajemy (zmiennym, metodom, klasom). Komentarze - czyli notatki dla człowieka, ignorowane przez komputer - zapisujemy po `//` (do końca linii) lub między `/*` a `*/` (blok).

```
// To jest komentarz jednoliniowy - komputer go pomija.

/* To jest
   komentarz blokowy
   na kilka linii. */

int wiek = 25;           // instrukcja zakończona średnikiem
if (wiek >= 18)          // początek warunku
{                         // początek bloku
```

```
bool dorosly = true; // instrukcja wewnątrz bloku (wcięta)
} // koniec bloku
```

Zasada na start: każda „czynność” to jedna instrukcja zakończona `;`, a powiązane instrukcje grupujemy w bloku `{ }`. Trzymaj porządek w wcięciach od pierwszego dnia - zaoszczędzi Ci to mnóstwo błędów.

4.2. Zmienne - deklaracja, inicjalizacja i nazewnictwo

Zmienna to nazwane „pudełko” w pamięci, w którym przechowujemy wartość - liczbę, tekst, wartość logiczną. Nazywamy ją zmienną, bo jej zawartość można **zmieniać** w trakcie działania programu. Każda zmienna ma trzy cechy: **typ** (jaką wartość może przechowywać), **nazwę** (jak się do niej odwołujemy) oraz **wartość** (co w niej jest). Zanim użyjemy zmiennej, musimy ją **zadeklarować** - czyli powiedzieć językowi, jaki ma typ i nazwę.

Deklaracja to podanie typu i nazwy: `int wiek;`. **Inicjalizacja** to nadanie pierwszej wartości: `wiek = 25;`. Najczęściej robimy obie rzeczy naraz: `int wiek = 25;`. Po deklaracji możemy dowolnie zmieniać wartość, ale **nie możemy zmienić typu** - zmienna `int` zawsze przechowuje liczbę całkowitą.

Nazwy zmiennych powinny być **opisowe** i pisane w stylu **camelCase** (pierwsze słowo małą literą, kolejne wielką: `wiekUzytkownika`, `LiczbaPolubien`). Nazwa nie może zaczynać się od cyfry, zawierać spacji ani być słowem kluczowym. Dobra nazwa mówi, co przechowuje zmienna - `cena` jest lepsze niż `x`.

```
// Deklaracja + inicjalizacja (najczęstsza forma)
int wiek = 25;
string imie = "Anna";
double cena = 19.99;
bool zalogowany = false;

// Deklaracja, a potem inicjalizacja
int liczbaPunktow;
liczbaPunktow = 100;

// Zmiana wartości (zmienna „zmienia się”)
wiek = 26; // teraz wiek to 26
wiek = wiek + 1; // teraz 27 - po prawej liczymy, wynik wraca do zmiennej

// Kilka zmiennych tego samego typu w jednej linii
int a = 1, b = 2, c = 3;
```

POJĘCIE	ZNACZENIE	PRZYKŁAD
Deklaracja	podanie typu i nazwy	<code>int wiek;</code>
Inicjalizacja	nadanie pierwszej wartości	<code>wiek = 25;</code>
Przypisanie	zmiana wartości	<code>wiek = 30;</code>
camelCase	styl nazw zmiennych	<code>LiczbaPolubien</code>

Na co uważać: użycie zmiennej przed nadaniem jej wartości to błąd („użyto nieprzypisanej zmiennej lokalnej”). Zawsze inicjalizuj zmienną, zanim jej użyjesz. Unikaj nic niemówiących nazw (`x`, `temp`, `dane1`) - po tygodniu sam nie będziesz wiedział, co znaczą.

4.3. Typy liczbowe całkowite - int, long, short, byte

Liczby całkowite (bez części po przecinku) przechowujemy w typach całkowitych. Najczęściej używamy `int` - wystarcza w zdecydowanej większości przypadków (liczniki, wiek, indeksy, wyniki). Pozostałe typy całkowite różnią się **zakresem** (jak duże liczby

pomieszczą) i zużyciem pamięci. Większy zakres to większe zużycie pamięci, ale na nowoczesnych urządzeniach różnice są nieistotne - dlatego domyślnie wybieramy `int`.

`int` mieści liczby od ok. -2,1 miliarda do +2,1 miliarda. Gdy potrzebujemy większych liczb (np. liczba milisekund od 1970 roku, bardzo duże liczniki), używamy `long`. Typy `short` i `byte` mają mały zakres i stosujemy je rzadko, zwykle przy oszczędzaniu pamięci lub pracy z danymi binarnymi (`byte` mieści 0-255, idealny dla składowej koloru).

```
int wiek = 30;           // typowa liczba całkowita
int temperatura = -15;   // może być ujemna
int liczbaUzytkownikow = 1500000;

long bardzoDuzaLiczba = 9000000000; // poza zakresem int - potrzebny long
byte skladowaKoloru = 255;           // 0..255 - np. wartość RGB
short maleLiczby = 1000;

// Operacje na liczbach całkowitych
int suma = wiek + 5;           // 35
int podwojony = wiek * 2;      // 60
```

TYP	ZAKRES (W PRZYBLIŻENIU)	KIEDY UŻYWAĆ
<code>byte</code>	0 ... 255	składowe koloru, dane binarne
<code>short</code>	-32 768 ... 32 767	rzadko, oszczędność pamięci
<code>int</code>	ok. ±2,1 mld	domyślny wybór dla liczb całkowitych
<code>long</code>	ok. ±9,2 tryliona	bardzo duże liczby, czas w ms

Na co uważać: dzielenie dwóch liczb całkowitych daje **liczbę całkowitą** - `7 / 2` to `3` (a nie `3.5`), bo reszta jest odrzucana. Jeśli chcesz wynik z częścią ułamkową, użyj typu `double` (patrz 5.4). Przekroczenie zakresu typu (np. dodanie 1 do maksymalnego `int`) powoduje „przewinięcie” do wartości ujemnej - przy dużych liczbach użyj `long`.

4.4. Typy zmiennoprzecinkowe - `double`, `float`, `decimal`

Gdy potrzebujemy liczb z **częścią ułamkową** (po przecinku), używamy typów zmiennoprzecinkowych. `double` to domyślny wybór dla zwykłych obliczeń (wzrost, średnia, BMI, wartość suwaka). `float` to „mniejszy double” - rzadziej używany. `decimal` jest **najdokładniejszy** i przeznaczony do **pieniędzy** oraz obliczeń finansowych, gdzie błędy zaokrągleń są niedopuszczalne.

W C# część ułamkową zapisujemy **kropką**, nie przecinkiem: `3.14`, nie `3,14`. Aby zaznaczyć, że literał jest typu `decimal`, dodajemy przyrostek `m` (`19.99m`), a dla `float` - `f` (`1.5f`). Bez przyrostka liczba z kropką jest traktowana jako `double`.

```
double wzrost = 1.75;           // metry
double srednia = 4.85;
double wartoscSuwaka = 23.7;

decimal cena = 19.99m;          // pieniądze - przyrostek 'm'
decimal saldoKonta = 12345.67m;

float wspolczynnik = 1.5f;      // przyrostek 'f'

// Obliczenia
double bmi = 70 / (1.75 * 1.75); // ok. 22.86
double polowa = 7.0 / 2.0;       // 3.5 (bo to double, nie int)
```

TYP	DOKŁADNOŚĆ	PRZYROSTEK	KIEDY UŻYWAĆ
<code>float</code>	mniejsza	<code>f</code>	rzadko, oszczędność pamięci
<code>double</code>	duża	(brak)	domyślny dla obliczeń
<code>decimal</code>	bardzo duża	<code>m</code>	pieniądze, finanse

Do kwot pieniężnych używaj `decimal`, nie `double`. `double` przechowuje liczby w przybliżeniu, przez co `0.1 + 0.2` może dać `0.30000000000000004`. Dla cen i sald takie błędy są niedopuszczalne - `decimal` ich nie ma.

Na co uważać: liczby z kropką bez przyrostka są typu `double` - przypisanie `decimal cena = 19.99;` (bez `m`) to błąd. Pamiętaj o kropce zamiast przecinka. Przy dzieleniu, jeśli choć jedna liczba jest `double` (np. `7.0`), wynik będzie `double` z częścią ułamkową.

4.5. Typ logiczny - bool

`bool` to typ przechowujący jedną z **dwóch wartości**: `true` (prawda) lub `false` (fałsz). To fundament podejmowania decyzji w programie - każdy warunek (`if`) sprowadza się do wartości `bool`. Zmienne logiczne nazywamy tak, by ich nazwa brzmiała jak pytanie tak/nie: `czyZalogowany`, `jestPusty`, `wlaczony`.

Wartość `bool` powstaje też jako **wynik porównania** (np. `wiek ≥ 18` daje `true` lub `false`) oraz operacji logicznych (omówionych w 5.12). Wartość logiczną możemy zanegować operatorem `!` (nie): `!true` to `false`.

```
bool zalogowany = false;
bool pelnoletni = true;

// bool jako wynik porównania
int wiek = 20;
bool czyDorosly = wiek ≥ 18; // true

// Przetaczanie wartości na przeciwną
bool wlaczony = false;
wlaczony = !wlaczony; // teraz true
wlaczony = !wlaczony; // znów false

// Użycie w warunku
if (czyDorosly)
{
    // wykona się, bo czyDorosly = true
}
```

Na co uważać: w warunku `if` piszemy `if (zalogowany)`, a nie `if (zalogowany == true)` - to drugie jest poprawne, ale zbędne. Nie myl przypisania `=` z porównaniem `==`: `if (x = 5)` to błąd, powinno być `if (x == 5)`. Wzorzec `wlaczony = !wlaczony;` to najprostszy sposób na przełącznik (np. w panelu urządzenia).

4.6. Typ znakowy - char

`char` przechowuje **pojedynczy znak**: literę, cyfrę, spację lub symbol. Wartość `char` zapisujemy w **apostrofach** (pojedynczych cudzysłowach): `'A'`, `'7'`, `'@'`. To różni `char` od `string` (tekst), który zapisujemy w cudzysłowach podwójnych: `"A"`. Pojedynczy tekst może być pusty lub mieć wiele znaków, a `char` to zawsze dokładnie jeden znak.

`char` przydaje się, gdy analizujemy tekst znak po znaku - np. sprawdzamy, czy znak jest literą (`char.IsLetter`), cyfrą (`char.IsDigit`) czy wielką literą (`char.IsUpper`). Każdy znak ma też swój numer w tablicy kodów (Unicode), dzięki czemu można na nim wykonywać operacje arytmetyczne (wykorzystywane np. w szyfrze Cezara).

```

char litera = 'A';
char cyfra = '7';
char symbol = '@';

// Przydatne sprawdzenia
bool czyLitera = char.IsLetter('A'); // true
bool czyCyfra = char.IsDigit('7'); // true
bool czyWielka = char.IsUpper('a'); // false

// Pobranie znaku z tekstu (po indeksie, licząc od 0)
string słowo = "Kot";
char pierwszy = słowo[0]; // 'K'
char drugi = słowo[1]; // 'o'

// Iteracja po znakach tekstu
foreach (char c in "abc")
{
    // c kolejno: 'a', 'b', 'c'
}

```

Na co uważać: `'A'` (apostrofy) to `char`, a `"A"` (cudzysłowy) to `string` - to różne typy i nie można ich mieszać. Indeksy znaków w tekście liczymy **od zera** (`słowo[0]` to pierwszy znak). Odwołanie do indeksu poza długością tekstu powoduje błąd.

4.7. Typ tekstowy - string

`string` przechowuje **tekst** - od pojedynczego słowa po całe zdania. Wartość zapisujemy w **podwójnych cudzysłowach**: `"Witaj"`. Tekst to jeden z najczęściej używanych typów, bo aplikacje stale operują na napisach: imionach, komunikatach, danych z pól. `string` ma bardzo wiele wbudowanych **metod** ułatwiających pracę z tekstem - poznaj te najważniejsze, bo będziesz ich używać bez przerwy.

Teksty można **łączyć** (konkatenacja) operatorem `+`, ale czytelniej jest używać **interpolacji** (`$"..."`, omówionej w 5.15). Długość tekstu zwraca właściwość `.Length`. Tekst może być pusty (`""`) lub mieć wartość `null` (brak wartości) - dlatego przy danych od użytkownika sprawdzamy go metodą `string.IsNullOrEmpty`.

```

string imie = "Anna";
string nazwisko = "Kowalska";

// Łączenie tekstów
string pelne = imie + " " + nazwisko; // "Anna Kowalska"

// Długość
int dlugosc = imie.Length; // 4

// Najważniejsze metody tekstu
string tekst = " Witaj Świecie ";
string bezSpacji = tekst.Trim(); // "Witaj Świecie" (usuwa spacje z brzegów)
string wielkie = tekst.ToUpper(); // zamienia na WIELKIE litery
string male = tekst.ToLower(); // zamienia na małe litery
bool zawiera = tekst.Contains("Witaj"); // true - czy zawiera fragment
bool zaczyna = tekst.Trim().StartsWith("W"); // true
string zamiana = "kot".Replace("k", "l"); // "lot"
string[] czesci = "a,b,c".Split(','); // tablica: ["a","b","c"]

```

```
string fragment = "abcdef".Substring(0, 3); // "abc" (od indeksu 0, 3 znaki)
int pozycja = "abcabc".IndexOf("b"); // 1 (pierwsze wystąpienie)
```

METODA / WŁAŚCIWOŚĆ	DZIAŁANIE	PRZYKŁAD WYNIKU
<code>.Length</code>	liczba znaków	<code>"Kot".Length</code> → 3
<code>.Trim()</code>	usuwa spacje z początku i końca	<code>" a ".Trim()</code> → "a"
<code>.ToUpper()</code> / <code>.ToLower()</code>	zmiana wielkości liter	<code>"a".ToUpper()</code> → "A"
<code>.Contains(x)</code>	czy zawiera fragment	true / false
<code>.StartsWith(x)</code> / <code>.EndsWith(x)</code>	czy zaczyna/kończy się	true / false
<code>.Replace(a, b)</code>	zamienia fragment	<code>"kot".Replace("k", "l")</code> → "lot"
<code>.Split(z)</code>	dzieli na tablicę	<code>"a,b".Split(',')</code> → ["a", "b"]
<code>.Substring(i, n)</code>	wycina fragment	<code>"abcd".Substring(1,2)</code> → "bc"
<code>.IndexOf(x)</code>	pozycja fragmentu (lub -1)	<code>"ab".IndexOf("b")</code> → 1

Na co uważać: tekst dla pustego pola może mieć wartość `null` - wywołanie metody na `null` (np. `tekst.Trim()`) gdy `tekst` jest `null`) powoduje błąd `NullReferenceException` . Dlatego zanim użyjesz tekstu od użytkownika, sprawdź `string.IsNullOrEmpty(tekst)` . Pamiętaj, że metody jak `ToUpper` **nie zmieniają** oryginału, lecz **zwracają nowy** tekst - wynik trzeba przypisać do zmiennej.

4.8. Stałe - const i readonly

Czasem wartość **nie powinna się zmieniać** w trakcie działania programu - np. liczba PI, stawka VAT, maksymalna liczba prób. Taką wartość zapisujemy jako **stałą**. Słowo kluczowe `const` tworzy stałą ustaloną w momencie pisania kodu - jej wartość musi być znana od razu i nie da się jej zmienić nigdzie w programie. Próba przypisania nowej wartości do stałej to błąd kompilacji.

Pokrewne `readonly` oznacza pole, które ustawiamy raz (przy deklaracji lub w konstruktorze) i potem już nie zmieniamy. Różnica: `const` to wartość „wmurowana” w kod, a `readonly` może być ustalona w trakcie tworzenia obiektu. Stałe nazywamy zwykle opisowo, np. `MaksymalnaLiczbaProb` .

```
// Stała - wartość znana od razu, niezmienna
const double Pi = 3.14159;
const int MaksymalnaLiczbaProb = 3;
const string NazwaAplikacji = "Moja App";

// readonly - ustalana raz, np. w konstruktorze
public class Konfiguracja
{
    public readonly DateTime DataUtworzenia;
    public Konfiguracja()
    {
        DataUtworzenia = DateTime.Now; // ustawiamy raz, potem niezmiennie
    }
}

// Użycie stałej
double obwod = 2 * Pi * promien;
```

Na co uważać: używaj stałych dla wartości, które są „magiczne” w kodzie (np. liczba 3 oznaczająca limit prób) - nazwana stała `MaksymalnaLiczbaProb` jest czytelniejsza niż gołe `3` rozsiane po kodzie. `const` musi mieć wartość znaną od razu (nie może to być np. wynik metody) - wtedy użyj `readonly`.

4.9. Operatory arytmetyczne

Operatory arytmetyczne służą do obliczeń na liczbach. Podstawowe to dodawanie `+`, odejmowanie `-`, mnożenie `*` i dzielenie `/`. Dodatkowo mamy **modulo** `%`, które zwraca **resztę z dzielenia** - bardzo przydatne np. do sprawdzania parzystości (`x % 2 == 0`) czy zawiązywania indeksów. Obowiązuje normalna **kolejność działań**: najpierw mnożenie i dzielenie, potem dodawanie i odejmowanie; nawiasy `( )` zmieniają kolejność.

Bardzo często zwiększamy lub zmniejszamy liczbę o 1 - służą do tego operatory **inkrementacji** `++` i **dekrementacji** `--`. Zapis `licznik++` znaczy `licznik = licznik + 1`. To podstawa liczników i pętli.

```
int a = 10, b = 3;

int suma = a + b;      // 13
int roznica = a - b;    // 7
int iloczyn = a * b;    // 30
int iloraz = a / b;     // 3 (dzielenie całkowite - reszta odrzucona!)
int reszta = a % b;     // 1 (10 dzielone przez 3 daje resztę 1)

// Kolejność działań i nawiasy
int wynik1 = 2 + 3 * 4; // 14 (najpierw 3*4)
int wynik2 = (2 + 3) * 4; // 20 (nawias najpierw)

// Inkrementacja / dekrementacja
int licznik = 5;
licznik++; // 6
licznik--; // 5

// Modulo - sprawdzanie parzystości
bool parzysta = (a % 2 == 0); // false (10 jest parzyste? tak → true)
```

OPERATOR	DZIAŁANIE	PRZYKŁAD	WYNIK
<code>+</code>	dodawanie	<code>5 + 2</code>	<code>7</code>
<code>-</code>	odejmowanie	<code>5 - 2</code>	<code>3</code>
<code>*</code>	mnożenie	<code>5 * 2</code>	<code>10</code>
<code>/</code>	dzielenie	<code>7 / 2</code>	<code>3</code> (całkowite!)
<code>%</code>	reszta (modulo)	<code>7 % 2</code>	<code>1</code>
<code>++</code>	zwiększ o 1	<code>x++</code>	<code>x = x + 1</code>
<code>--</code>	zmniejsz o 1	<code>x--</code>	<code>x = x - 1</code>

Na co uważać: dzielenie dwóch `int` daje `int` (reszta odrzucona): `7 / 2` to `3`, nie `3.5`. Aby dostać ułamek, użyj `double`: `7.0 / 2` daje `3.5`. Operator `%` często wykorzystujemy do zawiązywania indeksów: `(indeks + 1) % dlugosc` po ostatnim elemencie wraca do zera.

4.10. Operatory przypisania

Operator **przypisania** `=` umieszcza wartość z prawej strony w zmiennej po lewej. To **nie jest** porównanie - `x = 5` znaczy „włóż 5 do x”, a nie „czy x równa się 5”. Oprócz zwykłego `=` mamy **operatory złożone**, które łączą działanie z przypisaniem: `+=`, `-=`, `*=`, `/=`,

`%=`. Zapis `x += 3` to skrót `x = x + 3`. Są wygodne i czytelne, szczególnie przy aktualizacji liczników i sum.

```
int x = 10;

x += 5;    // x = x + 5 → 15
x -= 3;    // x = x - 3 → 12
x *= 2;    // x = x * 2 → 24
x /= 4;    // x = x / 4 → 6
x %= 4;    // x = x % 4 → 2

// Działa też na tekstach (+=)
string log = "Start";
log += " → krok 1";    // "Start → krok 1"
log += " → krok 2";    // "Start → krok 1 → krok 2"

// Typowe użycie: sumowanie w pętli
int suma = 0;
suma += 10;    // 10
suma += 25;    // 35
```

OPERATOR	ZNACZENIE	RÓWNOWAŻNE
=	przypisz	-
+=	dodaj i przypisz	<code>x = x + y</code>
-=	odejmij i przypisz	<code>x = x - y</code>
*=	pomnóż i przypisz	<code>x = x * y</code>
/=	podziel i przypisz	<code>x = x / y</code>
%=	reszta i przypisz	<code>x = x % y</code>

Na co uważać: nie myl `=` (przypisanie) z `==` (porównanie). `+=` na tekstach **dokleja** kolejny fragment - wygodne do budowania komunikatów, ale do wielu skleiń wydajniejsza jest interpolacja lub `StringBuilder`.

4.11. Operatory porównania

Operatory porównania sprawdzają relację między dwiema wartościami i zawsze zwracają `bool` (`true` lub `false`). To na nich opierają się warunki `if`. Najważniejsze to równość `==`, nierówność `!=` oraz porównania `<`, `>`, `≤`, `≥`. Działają na liczbach (porównanie wartości) i na tekstach (`==` porównuje zawartość napisów).

```
int a = 5, b = 8;

bool rowne = (a == b);    // false
bool rozne = (a != b);    // true
bool mniejsze = (a < b);  // true
bool wieksze = (a > b);   // false
bool mniejszeRowne = (a ≤ 5); // true
bool wiekszeRowne = (b ≥ 8); // true

// Porównanie tekstów (porównuje zawartość)
string s1 = "kot", s2 = "kot";
bool takiSam = (s1 == s2);    // true
```

```
// Wynik porównania używany w warunku
if (a < b)
{
    // wykona się, bo 5 < 8
}
```

OPERATOR	ZNACZENIE	PRZYKŁAD	WYNIK
=	równe	5 = 5	true
≠	różne	5 ≠ 3	true
<	mniejsze	3 < 5	true
>	większe	3 > 5	false
≤	mniejsze lub równe	5 ≤ 5	true
≥	większe lub równe	5 ≥ 8	false

Na co uważać: do porównania używamy `=` (podwójny znak), nie `=`. Porównanie tekstów `=` rozróżnia wielkość liter - `"Kot" = "kot"` to `false`. Aby porównać ignorując wielkość, użyj `string.Equals(a, b, StringComparison.OrdinalIgnoreCase)` lub porównaj po `ToLower()`.

4.12. Operatory logiczne

Operatory logiczne łączą warunki. `&&` (i) daje `true`, gdy **oba** warunki są prawdziwe. `||` (lub) daje `true`, gdy **przynajmniej jeden** jest prawdziwy. `!` (nie) **odwraca** wartość logiczną. Dzięki nim budujemy złożone warunki, np. „wiek ≥ 18 i ma zgodę”.

Operatory `&&` i `||` działają **leniwie** (short-circuit): jeśli wynik jest już znany po pierwszym warunku, drugi nie jest sprawdzany. To ważne, bo pozwala bezpiecznie pisać np. `tekst ≠ null && tekst.Length > 0` - gdy `tekst` jest `null`, drugi warunek nie zostanie wykonany i nie będzie błędu.

```
int wiek = 20;
bool maZgode = true;

bool moze = (wiek ≥ 18) && maZgode;    // true - oba spełnione
bool ktokolwiek = (wiek < 13) || maZgode; // true - jeden wystarczy
bool niepełnoletni = !(wiek ≥ 18);    // false - negacja

// Leniwe wartościowanie chroni przed błędem
string tekst = null;
if (tekst ≠ null && tekst.Length > 0)
{
    // bezpieczne: gdy tekst = null, Length NIE jest sprawdzane
}

// Złożony warunek
bool poprawny = !string.IsNullOrEmpty(email)
    && email.Contains('@')
    && haslo.Length ≥ 6;
```

OPERATOR	NAZWA	DAJE <code>TRUE</code> , GDY
<code>&&</code>	i (AND)	oba warunki prawdziwe
<code> </code>	lub (OR)	przynajmniej jeden prawdziwy
<code>!</code>	nie (NOT)	warunek jest fałszywy

Na co uważać: kolejność `&&` przed `||` (jak mnożenie przed dodawaniem) - przy złożonych warunkach używaj **nawiasów** dla jasności: `(a && b) || c`. Wykorzystuj leniwość: w warunku `obiekt != null && obiekt.Cos` sprawdzenie `null` musi być **pierwsze**.

4.13. Operatory null: `??`, `?.`, `??=`

Wartość `null` oznacza „brak wartości” - np. niewypełnione pole tekstowe czy niewybrany element listy. Odwołanie do czegoś, co jest `null`, powoduje błąd `NullReferenceException` - jeden z najczęstszych błędów. C# ma operatory, które bezpiecznie radzą sobie z `null`. `??` (null-coalescing) zwraca wartość zastępczą, gdy lewa strona jest `null`. `?.` (null-conditional) bezpiecznie wywołuje składową - gdy obiekt jest `null`, zwraca `null` zamiast rzucać błąd. `??=` przypisuje wartość tylko, gdy zmienna jest `null`.

```
string imie = null;

// ?? - wartość zastępcza dla null
string doWyswietlenia = imie ?? "Gość"; // "Gość" (bo imie = null)

// ?. - bezpieczne wywołanie (gdy null, zwraca null, nie błąd)
int? dlugosc = imie?.Length;           // null (zamiast wyjątku)

// łączenie ?. i ??
string wybor = listaWyboru.SelectedItem?.ToString() ?? "nie wybrano";

// ??= - przypisz tylko, gdy null
string nazwa = null;
nazwa ??= "Domyślna"; // nazwa staje się "Domyślna"
nazwa ??= "Inna";     // bez zmian - nazwa już nie jest null
```

OPERATOR	DZIAŁANIE
<code>??</code>	zwróć prawą stronę, gdy lewa jest <code>null</code>
<code>?.</code>	wywołaj składową bezpiecznie (gdy <code>null</code> → <code>null</code>)
<code>??=</code>	przypisz tylko, gdy zmienna jest <code>null</code>

4.14. Konwersje typów

Konwersja to zamiana wartości jednego typu na inny. Dzielimy ją na **niejawną** (automatyczną, gdy nie ma ryzyka utraty danych - np. `int` → `double`) oraz **jawną** (gdy musimy ją wymusić, bo grozi utrata danych - np. `double` → `int`, co odrzuca część ułamkową). Jawną konwersję liczb robimy **rzutowaniem** - typ w nawiasie przed wartością: `(int)3.9` daje `3`.

Najważniejszy przypadek w praktyce to zamiana **tekstu na liczbę**, bo dane z pól (`Entry`) przychodzą jako `string`. Robimy to bezpiecznie metodą `int.TryParse` / `double.TryParse`, która zwraca `true` / `false` i nie rzuca wyjątku przy błędnych danych. `int.Parse` też zamienia tekst na liczbę, ale **rzuca wyjątek** przy złym wpisie - dlatego na danych użytkownika używamy `TryParse`. Odwrotnie - liczbę na tekst zamieniamy metodą `.ToString()`.

```
// Konwersja niejawną (bezpieczną) - int → double
int calkowita = 5;
double ulamkowa = calkowita; // 5.0 automatycznie
```

```
// Konwersja jawna (rzutowanie) - double → int (ucina część ułamkową!)
double d = 3.9;
int i = (int)d;           // 3 (NIE 4 - obcięcie, nie zaokrąglenie)

// Tekst → liczba: bezpiecznie przez TryParse
string wpis = "42";
if (int.TryParse(wpis, out int liczba))
{
    // konwersja się udała, 'liczba' = 42
}
else
{
    // wpis nie był liczbą
}

double.TryParse("3.14", out double pi); // pi = 3.14

// Liczba → tekst
int wiek = 30;
string tekst = wiek.ToString();          // "30"
string cena = (19.99).ToString("0.00"); // "19.99"

// Zaokrąglenie (gdy chcemy 4, nie 3)
int zaokraglone = (int)Math.Round(3.9); // 4
```

KONWERSJA	SPOSÓB	UWAGA
<code>int</code> → <code>double</code>	niejawna (automat)	bezpieczna
<code>double</code> → <code>int</code>	<code>(int)x</code>	ucina część ułamkową
<code>string</code> → <code>int</code>	<code>int.TryParse</code>	bezpieczne, zwraca <code>bool</code>
<code>string</code> → <code>int</code>	<code>int.Parse</code>	rzuca wyjątek przy błędzie
liczba → <code>string</code>	<code>.ToString()</code>	opcjonalny format

Na danych od użytkownika **zawsze** używaj `TryParse`, nie `Parse`. `int.Parse("abc")` rzuci wyjątek i zatrzyma aplikację; `int.TryParse("abc", out var x)` bezpiecznie zwróci `false`.

Na co uważać: rzutowanie `(int)` na `double` **obcina**, a nie zaokrągla - `(int)3.9` to `3`. Aby zaokrąglić, użyj `Math.Round`. Pamiętaj, że suwak (`Slider.Value`) zwraca `double` - do liczby całkowitej rzutuj: `(int)Suwak.Value`.

4.15. Interpolacja i formatowanie tekstu

Najwygodniejszym sposobem budowania tekstu ze zmiennych jest **interpolacja stringów**. Stawiamy znak `$` przed cudzysłowem i wstawiamy zmienne w klamrach `{ }`. To czytelniejsze niż sklejanie operatorem `+`. Wewnątrz klamr można umieszczać wyrażenia (np. `{a + b}`) oraz **format** po dwukropku (np. `{cena:0.00}` dla dwóch miejsc po przecinku).

Formatowanie pozwala ładnie wyświetlać liczby: stała liczba miejsc po przecinku, waluta, procenty, separatory tysięcy. To samo formatowanie działa w `.ToString("format")` i w wiązaniach (`StringFormat`).

```
string imie = "Anna";
int wiek = 28;
double cena = 19.9;
```

```
// Interpolacja - wstawianie zmiennych
string opis = $"{imie} ma {wiek} lat";           // "Anna ma 28 lat"

// Wyrażenia w klamrach
int a = 3, b = 4;
string suma = $"{a} + {b} = {a + b}";           // "3 + 4 = 7"

// Formatowanie liczb
string cenaTxt = $"{cena:0.00} zł";             // "19,90 zł"
string procent = $"{0.25:P0}";                 // "25%"
string zSeparatorem = $"{1234567:N0}";         // "1 234 567"

// Wieloliniowy tekst (znak nowej linii \n)
string podsumowanie = $"Imię: {imie}\nWiek: {wiek}";

// To samo formatowanie w ToString
string r = (3.14159).ToString("0.00");         // "3,14"
```

FORMAT	ZNACZENIE	PRZYKŁAD -> WYNIK
0.00	2 miejsca po przecinku	19.9 → 19,90
0	liczba całkowita	3.7 → 4 (zaokrągła)
N0	z separatorem tysięcy	1234567 → 1 234 567
P0	procent	0.25 → 25%
C	waluta	19.9 → 19,90 zł
\n	nowa linia	przejdźcie do nowej linii

Na co uważać: pamiętaj o znaku `$` przed cudzysłowem - bez niego `"{wiek}"` to dosłowny tekst, a nie wartość. Separator dziesiętny i waluta zależą od ustawień regionalnych urządzenia. Do składania komunikatu z wielu pól interpolacja jest najczytelniejsza: `$"Klient: {imie}, kwota: {kwota:0.00} zł"`.

5. Podstawy C#: warunki, pętle i metody

5.1. Instrukcja warunkowa if / else if / else

Instrukcja warunkowa pozwala wykonać kod **tylko wtedy, gdy spełniony jest warunek**. To podstawa podejmowania decyzji. Składa się z słowa `if`, warunku w nawiasach `( )` (wyrażenie typu `bool`) i bloku `{ }`, który wykona się, gdy warunek jest prawdziwy. Opcjonalne `else` obsługuje przypadek, gdy warunek jest fałszywy, a `else if` pozwala sprawdzać kolejne warunki po kolei.

Działa to tak: komputer sprawdza warunek `if`; jeśli `true`, wykonuje jego blok i pomija resztę. Jeśli `false`, przechodzi do kolejnego `else if`, a gdy żaden nie pasuje - do `else`. Warunki sprawdzane są **po kolei**, więc kolejność ma znaczenie. Bloku `{ }` można teoretycznie pominąć przy jednej instrukcji, ale **zawsze go pisz** - to zapobiega błędom.

```
int punkty = 72;

// Pojedynczy warunek
if (punkty ≥ 50)
{
    // wykona się, bo 72 ≥ 50
}
```

```

// if / else
if (punkty ≥ 50)
{
    Wynik.Text = "Zaliczone";
}
else
{
    Wynik.Text = "Niezaliczone";
}

// if / else if / else - wiele możliwości po kolei
string ocena;
if (punkty ≥ 90)      ocena = "bardzo dobry";
else if (punkty ≥ 75) ocena = "dobry";
else if (punkty ≥ 50) ocena = "dostateczny";
else                  ocena = "niedostateczny";

// Zagnieżdżone warunki
if (zaalogowany)
{
    if (maUprawnienia)
    {
        // wykona się tylko, gdy oba warunki prawdziwe
    }
}

// Warunek złożony zamiast zagnieżdżenia (czytelniej)
if (zaalogowany && maUprawnienia)
{
    // to samo, krócej
}

```

Wzorec **wczesnego wyjścia** (early return) bardzo poprawia czytelność walidacji: sprawdzaj warunki po kolei i przerywaj `return`, gdy coś jest nie tak, zamiast budować głębokie zagnieżdżenia `if`.

```

private void OnZapisz()
{
    if (string.IsNullOrWhiteSpace(Imie.Text)) { Pokaz("Podaj imię"); return; }
    if (!Email.Text.Contains('@'))             { Pokaz("Zły e-mail"); return; }

    // tu dochodzimy tylko, gdy wszystko poprawne
    Pokaz("Zapisano");
}

```

Na co uważać: warunek musi być typu `bool`. Używaj `=` (porównanie), nie `=` (przypisanie). Zawsze pisz nawiasy klamrowe, nawet dla jednej instrukcji - brak klamr to częste źródło błędów przy późniejszej rozbudowie. Kolejność `else if` ma znaczenie: bardziej szczegółowe warunki sprawdzaj wcześniej.

5.2. Operator warunkowy (trójargumentowy) ?:

Gdy chcemy **wybrać jedną z dwóch wartości** zależnie od warunku, zamiast pełnego `if/else` możemy użyć krótkiego **operatora warunkowego** `?:` (zwanego trójargumentowym lub ternarnym). Składnia: `warunek ? wartośćGdyPrawda : wartośćGdyFałsz`.

Jest zwięzły i świetnie nadaje się do prostych przypisań oraz ustawiania właściwości.

```
int wiek = 20;

// Zamiast if/else przypisujemy jedną z dwóch wartości
string status = (wiek ≥ 18) ? "dorosły" : "niepełnoletni";

// Ustawianie koloru zależnie od warunku
Wynik.TextColor = poprawne ? Colors.Green : Colors.Red;

// Ustawianie tekstu przycisku przełącznika
Przycisk.Text = włączone ? "Wyłącz" : "Włącz";

// Można zagnieżdżać (ale uważaj na czytelność)
string ocena = punkty ≥ 75 ? "dobry" : punkty ≥ 50 ? "dostateczny" : "słaby";
```

Na co uważać: operator `?:` świetnie zastępuje proste `if/else` przypisujące wartość, ale **nie nadużywaj zagnieżdżania** - kilka zagnieżdżonych `?:` staje się nieczytelne, wtedy lepszy jest `if/else` lub `switch`. Obie gałęzie (`:`) muszą zwracać ten sam typ.

5.3. Instrukcja switch i switch expression

Gdy porównujemy **jedną wartość z wieloma możliwościami**, czytelniejszy od długiego `else if` bywa `switch`. Dla każdej możliwej wartości definiujemy `case`, a `default` obsługuje wszystkie pozostałe przypadki. W klasycznym `switch` każdy `case` musi kończyć się `break` (lub `return`). Można połączyć kilka `case` (gdy mają wspólne działanie), pisząc je jeden pod drugim.

Nowoczesny C# oferuje też zwięzłą formę **switch expression**, która **zwraca wartość** - idealna do przypisań. Używa składni `wartość switch { wzorzec ⇒ wynik, ... }`, gdzie `_` oznacza „każdy inny przypadek” (odpowiednik `default`).

```
string dzien = "sobota";

// Klasyczny switch
switch (dzien)
{
    case "sobota":
    case "niedziela":           // kilka case razem
        Info.Text = "Weekend";
        break;
    case "piątek":
        Info.Text = "Prawie weekend";
        break;
    default:                   // wszystkie pozostałe
        Info.Text = "Dzień roboczy";
        break;
}

// switch expression (zwraca wartość - krócej)
string typ = dzien switch
{
    "sobota" or "niedziela" ⇒ "Weekend",
    "piątek"                ⇒ "Prawie weekend",
    -                        ⇒ "Dzień roboczy"
};

// switch na liczbach z zakresami (wzorce relacyjne)
```

```
int punkty = 85;
string ocena = punkty switch
{
    ≥ 90 ⇒ "bardzo dobry",
    ≥ 75 ⇒ "dobry",
    ≥ 50 ⇒ "dostateczny",
    -    ⇒ "niedostateczny"
};
```

ELEMENT	KLASYCZNY SWITCH	SWITCH EXPRESSION
Zwraca wartość	nie	tak
Zakończenie gałęzi	<code>break;</code>	<code>,</code> (przecinek)
„Każdy inny”	<code>default:</code>	<code>_</code>
Łączenie wartości	kolejne <code>case</code>	<code>or</code>

Na co uważać: w klasycznym `switch` brak `break` to błąd kompilacji w C#. Switch expression jest krótszy i zwraca wartość, ale to inna składnia (`⇒`, przecinki, `_`). Dla 2–3 przypadków wystarczy `if/else`; `switch` opłaca się przy większej liczbie wartości.

5.4. Pętla for

Pętla powtarza blok kodu wiele razy - zamiast pisać tę samą instrukcję 100 razy, piszemy ją raz w pętli. **Pętla** `for` to pętla z **licznikiem**; używamy jej, gdy **znamy liczbę powtórzeń** (np. „zrób coś 5 razy”, „przejdź po elementach tablicy po indeksie”). Jej nagłówek ma trzy części oddzielone średnikami: **inicjalizacja** licznika, **warunek** kontynuacji i **krok** (zmiana licznika po każdym obrocie).

Anatomia `for (int i = 0; i < 5; i++)`: najpierw raz wykonuje się `int i = 0` (start). Potem sprawdzany jest warunek `i < 5` - jeśli `true`, wykonuje się blok pętli. Po bloku wykonuje się krok `i++` i znów sprawdzany jest warunek. Tak w kółko, aż warunek stanie się `false`. Powyższa pętla wykona się dla `i` równego 0, 1, 2, 3, 4 - czyli **5 razy**.

```
// Najprostsza pętla - wykona się 5 razy (i = 0,1,2,3,4)
for (int i = 0; i < 5; i++)
{
    Console.WriteLine($"Obrót numer {i}");
}

// Liczenie od 1 do 10
for (int i = 1; i ≤ 10; i++)
{
    // i przyjmuje 1,2,...,10
}

// Co drugi (krok 2)
for (int i = 0; i < 10; i += 2)
{
    // i = 0,2,4,6,8
}

// W dół (od 5 do 1)
for (int i = 5; i ≥ 1; i--)
{
    // i = 5,4,3,2,1
}
```



```
// Sumowanie liczb od 1 do 100
int suma = 0;
for (int i = 1; i ≤ 100; i++)
{
    suma += i;    // dodaje kolejne liczby
}

// suma = 5050

// Iteracja po tablicy po indeksie
string[] owoce = { "jabłko", "banan", "gruszka" };
for (int i = 0; i < owoce.Length; i++)
{
    Console.WriteLine($"{i}: {owoce[i]}");
}

// Pętla zagnieżdżona (pętla w pętli) - np. tabliczka mnożenia
for (int w = 1; w ≤ 3; w++)
{
    for (int k = 1; k ≤ 3; k++)
    {
        Console.WriteLine($"{w} x {k} = {w * k}");
    }
}
```

CZĘŚĆ NAGŁÓWKA	ROLA	PRZYKŁAD
inicjalizacja	start licznika (raz)	<code>int i = 0</code>
warunek	dopóki prawda - powtarzaj	<code>i < 5</code>
krok	zmiana po każdym obrocie	<code>i++</code>

Na co uważać: najczęstszy błąd to **wyjście poza zakres** tablicy - używaj warunku `i < tablica.Length` (z `<`, nie `≤`), bo indeksy idą od `0` do `Length - 1`. Druga pułapka to **pętla nieskończona**, gdy warunek nigdy nie staje się fałszywy (np. zapomniany `i++`). Pętle zagnieżdżone wykonują się „warunek razy warunek” razy - uważaj na wydajność przy dużych zakresach.

5.5. Pętla foreach

Pętla `foreach` przechodzi **po wszystkich elementach kolekcji** (tablicy, listy) - od pierwszego do ostatniego - bez konieczności pilnowania indeksu. Jest czytelniejsza i bezpieczniejsza niż `for`, gdy chcemy po prostu „przejsć po każdym elemencie”. Składnia: `foreach (typ element in kolekcja)`. W każdym obrocie zmienna `element` przyjmuje kolejną wartość z kolekcji.

```
string[] owoce = { "jabłko", "banan", "gruszka" };

// Przejszcie po każdym elemencie
foreach (string owoc in owoce)
{
    Console.WriteLine(owoc);    // kolejno: jabłko, banan, gruszka
}

// Sumowanie wartości z listy
List<int> liczby = new() { 10, 20, 30 };
int suma = 0;
foreach (int liczba in liczby)
{
```

```

        suma += liczba;           // 10, potem 30, potem 60
    }

    // var - kompilator sam wykryje typ elementu
    foreach (var produkt in produkty)
    {
        Console.WriteLine(produkt.Nazwa);
    }

    // foreach po znakach tekstu
    int literyA = 0;
    foreach (char c in "abrakadabra")
    {
        if (c == 'a') literyA++;    // policz litery 'a'
    }

```

Kiedy używać? `foreach`, gdy chcesz przejść po **wszystkich** elementach i nie potrzebujesz numeru pozycji. `for`, gdy potrzebujesz **indeksu** (np. by porównać sąsiednie elementy lub modyfikować po indeksie).

Na co uważać: w `foreach` **nie wolno modyfikować kolekcji** (dodawać/usuwać elementów) w trakcie iteracji - powoduje to błąd. Jeśli musisz usuwać elementy podczas przechodzenia, użyj pętli `for` (od końca) lub utwórz kopię. Zmienna pętli jest „tylko do odczytu” - nie przypiszesz jej nowej wartości.

5.6. Pętla while

Pętla `while` powtarza blok **dopóki warunek jest prawdziwy**. Używamy jej, gdy **nie znamy z góry liczby powtórzeń**, a jedynie warunek zakończenia (np. „czytaj, dopóki są dane”, „losuj, aż trafisz”). Warunek sprawdzany jest **przed** każdym obrotem - jeśli od początku jest `false`, pętla nie wykona się ani razu.

```

// Odliczanie
int n = 5;
while (n > 0)
{
    Console.WriteLine(n);
    n--;           // WAŻNE: zmieniamy warunek, by pętla się skończyła
}

// Powtarzaj, aż warunek przestanie być spełniony
int suma = 0, i = 1;
while (suma < 100)
{
    suma += i;
    i++;
}

// Losuj, aż wypadnie 6
Random los = new Random();
int rzut;
int proby = 0;
while ((rzut = los.Next(1, 7)) != 6)
{
    proby++;
}

```

Na co uważać: w pętli `while` **musisz** wewnątrz bloku zmieniać coś, co wpływa na warunek (np. `n--`) - inaczej powstanie **pętla nieskończona**, która zawiesi aplikację. To najczęstszy błąd przy `while`. Jeśli warunek od początku jest fałszywy, blok nie wykona się wcale.

5.7. Pętla do-while

Pętla `do-while` działa jak `while`, ale warunek sprawdzany jest **na końcu**, po wykonaniu bloku. Dzięki temu blok wykona się **co najmniej raz**, nawet gdy warunek od początku jest fałszywy. Używamy jej, gdy chcemy najpierw coś zrobić, a potem zdecydować, czy powtórzyć (np. pokaż pytanie, a potem sprawdź odpowiedź).

```
// Blok wykona się przynajmniej raz
int n = 0;
do
{
    Console.WriteLine($"Wartość: {n}");
    n++;
}
while (n < 3); // warunek na końcu; uwaga na średnik!

// Przykład: powtarzaj, aż dane będą poprawne (konceptyjnie)
string wpis;
do
{
    wpis = PobierzWpis(); // najpierw pobierz
}
while (string.IsNullOrEmpty(wpis)); // powtarzaj, gdy pusto
```

PĘTLA	WARUNEK SPRAWDZANY	WYKONA SIĘ MINIMUM
<code>while</code>	przed blokiem	0 razy
<code>do-while</code>	po bloku	1 raz

Na co uważać: `do-while` kończy się **średnikiem** po `while(...)` - to częsta literówka. Wybieraj `do-while`, gdy blok musi wykonać się przynajmniej raz; w pozostałych przypadkach zwykle wystarczy `while`.

5.8. break i continue

W pętlach przydają się dwa słowa sterujące. `break` **natychmiast przerywa** całą pętlę i wychodzi z niej. `continue` **pomija resztę bieżącego obrotu** i przechodzi do następnego (nie kończy całej pętli). Oba pozwalają precyzyjnie kontrolować przebieg pętli.

```
// break - przerwij, gdy znajdziesz szukany element
int[] liczby = { 3, 7, 2, 9, 5 };
int szukana = 9;
int pozycja = -1;
for (int i = 0; i < liczby.Length; i++)
{
    if (liczby[i] == szukana)
    {
        pozycja = i;
        break; // znaleziono - nie ma sensu szukać dalej
    }
}

// continue - pomij elementy niespełniające warunku
```

```

int sumaParzystych = 0;
for (int i = 1; i ≤ 10; i++)
{
    if (i % 2 ≠ 0) continue; // pomini nieparzyste
    sumaParzystych += i;      // dodaj tylko parzyste: 2+4+6+8+10
}

// continue w grze w kości - pomini zablokowane
for (int i = 0; i < 5; i++)
{
    if (zablokowana[i]) continue; // nie losuj zablokowanej kości
    wartosci[i] = los.Next(1, 7);
}

```

SŁOWO	DZIAŁANIE
<code>break</code>	przerywa całą pętlę
<code>continue</code>	omija bieżący obrót, przechodzi do następnego

Na co uważać: `break` przerywa tylko **najbliższą** pętlę (w pętlach zagnieżdżonych - wewnętrzną). `continue` jest świetny do pomijania elementów niespełniających warunku, dzięki czemu unikasz głębokiego zagnieżdżenia `if` wewnątrz pętli.

5.9. Metody

Metoda to nazwany fragment kodu, który wykonuje określone zadanie i można go **wielokrotnie wywoływać**. Dzięki metodom unikamy powtarzania kodu i dzielimy program na czytelne kawałki. Metoda może przyjmować **parametry** (dane wejściowe) i **zwracać wynik** (typ przed nazwą). Gdy nic nie zwraca, jej typem jest `void`. Metodę **wywołujemy**, podając jej nazwę i argumenty w nawiasach.

Metody mogą mieć **parametry opcjonalne** (z wartością domyślną), być wywoływane z **argumentami nazwanymi**, a także występować w kilku wersjach o tej samej nazwie, lecz różnych parametrach - to **przeciążanie** (overloading). Krótkie metody zwracające jedną wartość można zapisać skróconą składnią `⇒` (expression body).

```

// Metoda bez parametrów, nic nie zwraca (void)
void Przywitaj()
{
    Console.WriteLine("Cześć!");
}

// Metoda z parametrami, zwraca wynik
int Dodaj(int a, int b)
{
    return a + b; // return zwraca wynik i kończy metodę
}

// Wywołanie
Przywitaj();
int suma = Dodaj(3, 5); // suma = 8

// Skrócony zapis (expression body) - dla krótkich metod
int Kwadrat(int x) ⇒ x * x;
double Srednia(double a, double b) ⇒ (a + b) / 2;

// Parametry opcjonalne (wartość domyślna)
string Powitanie(string imie, string przed = "Witaj")

```

```

{
    return $"{przed}, {imie}!";
}

Powitanie("Anna");           // "Witaj, Anna!"
Powitanie("Anna", "Dzień dobry"); // "Dzień dobry, Anna!"

// Argumenty nazwane (kolejność dowolna, czytelność)
Powitanie(imie: "Piotr", przed: "Hej");

// Przeciążanie - ta sama nazwa, różne parametry
int Pomnoz(int a, int b) => a * b;
double Pomnoz(double a, double b) => a * b;

// Parametr out - metoda zwraca dodatkowy wynik przez parametr
bool SprobujPodzielic(int a, int b, out int wynik)
{
    if (b == 0) { wynik = 0; return false; }
    wynik = a / b;
    return true;
}

```

ELEMENT	ZNACZENIE
typ zwracany	typ wyniku (<code>int</code> , <code>string</code> ...) lub <code>void</code>
<code>void</code>	metoda nic nie zwraca
<code>return</code>	zwraca wynik i kończy metodę
parametry	dane wejściowe w nawiasach
<code>=></code>	skrótowy zapis krótkiej metody
przeciążanie	ta sama nazwa, różne parametry

Na co uważać: metoda `void` nie używa `return` z wartością (może użyć samego `return;` do wcześniejszego wyjścia). Typ i liczba argumentów przy wywołaniu muszą pasować do definicji. Dziel długie metody na mniejsze, nazwane - kod staje się czytelniejszy i łatwiejszy do testowania. To podstawa wydzielenia logiki.

6. C#: kolekcje, klasy i modele danych

6.1. Tablice

Tablica to zbiór wielu wartości **tego samego typu** pod jedną nazwą, o **stałym rozmiarze** ustalonym przy tworzeniu. Do elementów odwołujemy się przez **indeks** (numer pozycji), liczony **od zera** - pierwszy element ma indeks 0, drugi 1 itd. Tablicę tworzymy, podając typ z nawiasami `[]` i rozmiar, albo od razu z wartościami w nawiasach klamrowych.

Liczbę elementów zwraca właściwość `.Length`. Po tablicy przechodzimy pętlą `for` (gdy potrzebujemy indeksu) lub `foreach` (gdy chcemy każdy element). Tablice mogą być też **wielowymiarowe** (np. siatka, tabela) oraz **postrzępione** (tablica tablic o różnej długości).

```

// Utworzenie tablicy o rozmiarze 3 (puste - wartości domyślne 0)
int[] liczby = new int[3];

liczby[0] = 10; // pierwszy element
liczby[1] = 20;
liczby[2] = 30;

```

```
// Utworzenie od razu z wartościami
string[] owoce = { "jabłko", "banan", "gruszka" };
int[] oceny = new int[] { 5, 4, 3, 5, 2 };

// Dostęp po indeksie (od 0!)
string pierwszy = owoce[0];           // "jabłko"
string ostatni = owoce[owoce.Length - 1]; // "gruszka"

// Liczba elementów
int ile = owoce.Length;    // 3

// Iteracja
for (int i = 0; i < oceny.Length; i++)
    Console.WriteLine($"Ocena {i}: {oceny[i]}");

foreach (string owoc in owoce)
    Console.WriteLine(owoc);

// Tablica dwuwymiarowa (siatka 2x3)
int[,] siatka = new int[2, 3];
siatka[0, 0] = 1;
siatka[1, 2] = 9;

// Tablice równoległe - ten sam indeks dotyczy tej samej rzeczy (np. gra w kości)
int[] wartosci = new int[5];
bool[] zablokowana = new bool[5];
```

OPERACJA	ZAPIS	UWAGA
Utworzenie pustej	<code>new int[3]</code>	rozmiar stały
Z wartościami	<code>{ 1, 2, 3 }</code>	rozmiar z liczby elementów
Dostęp	<code>tab[i]</code>	indeks od 0
Liczba elementów	<code>tab.Length</code>	
Ostatni element	<code>tab[tab.Length - 1]</code>	

Na co uważać: indeksy idą od `0` do `Length - 1` - odwołanie do `tab[tab.Length]` lub indeksu ujemnego powoduje błąd `IndexOutOfRangeException`. Tablica ma **stały rozmiar** - nie da się jej powiększyć po utworzeniu; gdy potrzebujesz zmiennej liczby elementów, użyj `List<T>` (następny podrozdział). Tablice równoległe (ten sam indeks w kilku tablicach) to prosty sposób przechowywania powiązanych danych.

6.2. List

`List<T>` to **dynamiczna lista** - zbiór elementów typu `T`, do którego można **dodawać i usuwać** elementy w trakcie działania (w przeciwieństwie do tablicy o stałym rozmiarze). To najczęściej używana kolekcja do przechowywania danych w pamięci. Litera `T` w `List<T>` oznacza typ elementów - `List<string>` to lista tekstów, `List<int>` to lista liczb, `List<Produkt>` to lista obiektów.

Najważniejsze operacje: `Add` (dodaj na koniec), `Remove` (usuń element), `RemoveAt` (usuń po indeksie), `Insert` (wstaw na pozycję), `Clear` (wyczyść), `Contains` (czy zawiera), `Count` (liczba elementów - odpowiednik `Length` tablicy), dostęp po indeksie `lista[i]`.

```
using System.Collections.Generic;

// Pusta lista
List<string> imiona = new List<string>();
imiona.Add("Anna");
imiona.Add("Piotr");
imiona.Add("Ewa");

// Lista z wartościami początkowymi
List<int> oceny = new List<int> { 5, 4, 3 };

// Liczba elementów
int ile = imiona.Count;           // 3

// Dostęp po indeksie (od 0)
string pierwsza = imiona[0];     // "Anna"

// Usuwanie
imiona.Remove("Piotr");           // usuwa po wartości
imiona.RemoveAt(0);               // usuwa po indeksie (tu: "Anna")

// Wstawianie na konkretną pozycję
oceny.Insert(0, 6);               // wstaw 6 na początek

// Sprawdzanie i czyszczenie
bool czyJest = oceny.Contains(5); // true
oceny.Clear();                    // usuwa wszystkie

// Iteracja
foreach (string imie in imiona)
    Console.WriteLine(imie);

// Lista obiektów
List<Produkt> produkty = new()
{
    new Produkt { Nazwa = "Kawa", Cena = 19.99 },
    new Produkt { Nazwa = "Herbata", Cena = 12.50 }
};
produkty.Add(new Produkt { Nazwa = "Sok", Cena = 8.00 });
```

METODA / WŁAŚCIWOŚĆ	DZIAŁANIE
<code>.Add(x)</code>	dodaje element na koniec
<code>.Remove(x)</code>	usuwa pierwszy pasujący element
<code>.RemoveAt(i)</code>	usuwa element o indeksie <code>i</code>
<code>.Insert(i, x)</code>	wstawia <code>x</code> na pozycję <code>i</code>
<code>.Clear()</code>	usuwa wszystkie elementy
<code>.Contains(x)</code>	czy lista zawiera <code>x</code>
<code>.Count</code>	liczba elementów
<code>lista[i]</code>	dostęp do elementu o indeksie <code>i</code>

Na co uważać: `List<T>` ma `.Count` (nie `.Length` jak tablica). Dostęp po indeksie poza zakresem (`lista[lista.Count]`) to błąd. `List<T>` świetnie nadaje się do danych „roboczych”, ale **nie powiadamia interfejsu** o zmianach – do list **wyświetlanych** na ekranie używaj `ObservableCollection<T>` (następny podrozdział).

6.3. ObservableCollection

`ObservableCollection<T>` to specjalna lista, która przy każdym **dodaniu lub usunięciu** elementu **automatycznie powiadamia** powiązany widok (np. `CollectionView`), dzięki czemu lista na ekranie odświeża się sama. Pod względem użycia jest niemal identyczna jak `List<T>` (`Add`, `Remove`, `Count`, indeks), ale ma tę dodatkową, kluczową dla interfejsu cechę.

```
using System.Collections.ObjectModel;

// Tworzenie i wypełnianie
ObservableCollection<string> notatki = new ObservableCollection<string>();
notatki.Add("Kup mleko"); // jeśli podpięta pod CollectionView, pojawi się od razu

// Z wartościami początkowymi
ObservableCollection<string> zadania = new() { "Zadanie 1", "Zadanie 2" };

// Operacje jak w List
zadania.Add("Zadanie 3"); // widok odświeża się automatycznie
zadania.RemoveAt(0);
zadania.Clear();
int ile = zadania.Count;
```

Na co uważać: to **najczęstszy błąd** przy listach - gdy lista na ekranie „nie odświeża się” po dodaniu elementu, prawie zawsze przyczyną jest użycie `List<T>` zamiast `ObservableCollection<T>`. Reguła: lista **wyświetlana i zmieniana** w trakcie działania → `ObservableCollection`; dane robocze w pamięci → `List`. `ObservableCollection` reaguje na dodanie/usunięcie **elementu**, ale nie na zmianę **właściwości** elementu - do tego obiekt musi implementować `INotifyPropertyChanged`.

6.4. Słowniki - Dictionary

`Dictionary<K, V>` to kolekcja **par klucz-wartość**: każdemu **kluczowi** (typu `K`) przypisana jest **wartość** (typu `V`). Działa jak prawdziwy słownik - podajesz hasło (klucz), dostajesz definicję (wartość). Klucze są **unikalne**. Słownik świetnie nadaje się do szybkiego wyszukiwania wartości po kluczu - np. cena produktu po nazwie, liczba wystąpień po słowie.

```
using System.Collections.Generic;

// Słownik: nazwa produktu → cena
Dictionary<string, double> cennik = new Dictionary<string, double>();
```



```

cennik["Kawa"] = 19.99;
cennik["Herbata"] = 12.50;

// Z wartościami początkowymi
Dictionary<string, int> stanMagazynu = new()
{
    { "Kawa", 10 },
    { "Herbata", 5 }
};

// Odczyt po kluczu
double cena = cennik["Kawa"];    // 19.99

// Bezpieczny odczyt (gdy klucz może nie istnieć)
if (cennik.TryGetValue("Sok", out double cenaSoku))
{
    // klucz istnieje - użyj cenaSoku
}

// Sprawdzenie i usunięcie
bool jest = cennik.ContainsKey("Kawa"); // true
cennik.Remove("Herbata");

// Iteracja po parach
foreach (var para in cennik)
{
    Console.WriteLine($"{para.Key}: {para.Value} zł");
}

```

OPERACJA	ZAPIS
Dodanie/zmiana	<code>słownik[klucz] = wartosc</code>
Odczyt	<code>słownik[klucz]</code>
Bezpieczny odczyt	<code>TryGetValue(klucz, out x)</code>
Czy zawiera klucz	<code>ContainsKey(klucz)</code>
Usunięcie	<code>Remove(klucz)</code>
Liczba par	<code>.Count</code>

Na co uważać: odwołanie do **nieistniejącego klucza** (`cennik["NieMa"]`) rzuca wyjątek - używaj `TryGetValue` lub `ContainsKey`, gdy nie masz pewności, że klucz istnieje. Klucze muszą być unikalne - przypisanie do istniejącego klucza **nadpisuje** poprzednią wartość. Dictionary to świetny wybór, gdy często szukasz danych „po nazwie”.

6.5. Klasy i obiekty

Klasa to „przepis” (szablon) opisujący, jak zbudowany jest pewien typ - jakie ma **pola/właściwości** (dane) i **metody** (zachowania). **Obiekt** to konkretny egzemplarz utworzony na podstawie klasy słowem `new`. Analogia: klasa to projekt domu, obiekt to konkretny zbudowany dom. W aplikacjach klasy reprezentują modele danych (np. `Produkt`, `Uzytkownik`), strony i klasy pomocnicze.

```

// Definicja klasy
public class Produkt
{

```

```

// Właściwości (dane obiektu)
public string Nazwa { get; set; }
public double Cena { get; set; }
public int Ilosc { get; set; }

// Metoda (zachowanie obiektu)
public double WartoscCalkowita()
{
    return Cena * Ilosc;
}

public string Opis() => $"{Nazwa} x{Ilosc} = {WartoscCalkowita():0.00} zł";
}

// Tworzenie obiektu (instancji)
Produkt p = new Produkt();
p.Nazwa = "Kawa";
p.Cena = 19.99;
p.Ilosc = 3;

// Tworzenie z inicjalizacją właściwości (krócej)
Produkt q = new Produkt { Nazwa = "Herbata", Cena = 12.50, Ilosc = 2 };

// Użycie metod i właściwości obiektu
double wartosc = p.WartoscCalkowita(); // 59.97
string opis = q.Opis(); // "Herbata x2 = 25,00 zł"

// Lista obiektów
List<Produkt> koszyk = new() { p, q };

```

Na co uważać: nie myl klasy (przepisu) z obiektem (konkretnym egzemplarzem). Zanim użyjesz obiektu, musisz go utworzyć (`new`) - odwołanie do obiektu, który jest `null`, powoduje `NullReferenceException`. Każdy obiekt ma własny zestaw danych - zmiana `p.Cena` nie wpływa na `q`.

6.6. Właściwości - get i set

Właściwość to „inteligentne pole” obiektu, które można odczytywać (`get`) i ustawiać (`set`). Najprostsza forma to **właściwość automatyczna**: `public string Nazwa { get; set; }`. Właściwość może być **tylko do odczytu** (`{ get; }` lub `{ get; private set; }`), mieć **wartość domyślną**, a także **wyliczać** wynik na podstawie innych pól (właściwość obliczana).

```

public class Osoba
{
    // Właściwość automatyczna (odczyt i zapis)
    public string Imie { get; set; }
    public string Nazwisko { get; set; }

    // Z wartością domyślną
    public bool Aktywny { get; set; } = true;

    // Tylko do odczytu z zewnątrz (zmiana tylko wewnątrz klasy)
    public int Id { get; private set; }

    // Właściwość obliczana (tylko get, liczy w locie)
    public string PelneImie => $"{Imie} {Nazwisko}";
}

```

```
// Właściwość z logiką w set (walidacja)
private int wiek;
public int Wiek
{
    get => wiek;
    set => wiek = value < 0 ? 0 : value; // nie pozwól na ujemny
}
}

// Użycie
Osoba o = new Osoba { Imie = "Anna", Nazwisko = "Kowalska" };
string pelne = o.PelneImie; // "Anna Kowalska" (obliczane)
o.Wiek = -5; // zapisane jako 0 (walidacja w set)
```

FORMA	ZAPIS	ZNACZENIE
Automatyczna	{ get; set; }	odczyt i zapis
Z domyślną	{ get; set; } = true;	wartość początkowa
Tylko odczyt	{ get; private set; }	zapis tylko wewnątrz klasy
Obliczana	=> wyrażenie	liczona w locie

Na co uważać: w setterze słowo kluczowe `value` oznacza przypisywaną wartość. Właściwości obliczane (`=>`) nie przechowują wartości - liczą ją przy każdym odczycie. Słowo `value` istnieje tylko w `set`.

6.7. Konstruktory

Konstruktor to specjalna metoda wywoływana **automatycznie przy tworzeniu obiektu** (`new`). Służy do nadania wartości początkowych. **Ma nazwę taką samą jak klasa i nie ma typu zwracanego.** Jeśli nie napiszemy żadnego konstruktora, klasa ma domyślny (bezparametrowy). Możemy mieć kilka konstruktorów o różnych parametrach (przeciążanie).

```
public class Konto
{
    public string Wlasciciel { get; set; }
    public decimal Saldo { get; private set; }

    // Konstruktor z parametrami
    public Konto(string wlasciciel, decimal saldoPoczkowe)
    {
        Wlasciciel = wlasciciel;
        Saldo = saldoPoczkowe;
    }

    // Drugi konstruktor (przeciążony) - saldo domyślnie 0
    public Konto(string wlasciciel) : this(wlasciciel, 0)
    {
        // ': this(...)' wywołuje pierwszy konstruktor
    }
}

// Użycie
```

```
Konto k1 = new Konto("Anna", 1000m);
Konto k2 = new Konto("Piotr"); // saldo = 0
```

Na co uważać: gdy dodasz **własny** konstruktor z parametrami, domyślny bezparametrowy **przestaje istnieć** - jeśli go potrzebujesz, dopisz go ręcznie. Zapis `: this(...)` pozwala jednemu konstruktorowi wywołać drugi, by nie powtarzać kodu. Konstruktor strony MAUI zawsze zaczyna się od `InitializeComponent();`.

6.8. Modyfikatory dostępu

Modyfikator dostępu określa, **kto może korzystać** z klasy, pola, właściwości czy metody. To mechanizm **hermetyzacji** - ukrywania szczegółów i udostępniania tylko tego, co potrzebne. Domyślnie ukrywamy dane (`private`) i udostępniamy operacje (`public`), co chroni poprawność obiektu.

```
public class Licznik
{
    private int wartosc = 0; // private - tylko wewnątrz klasy

    public void Zwiększ() // public - dostępne z zewnątrz
    {
        wartosc++;
    }

    public int Pobierz() => wartosc;
}

Licznik l = new Licznik();
l.Zwiększ();
int w = l.Pobierz(); // 1
// l.wartosc - BŁĄD: pole jest private, niedostępne z zewnątrz
```

MODYFIKATOR	DOSTĘP
<code>public</code>	z dowolnego miejsca
<code>private</code>	tylko w tej samej klasie
<code>protected</code>	w klasie i klasach dziedziczących
<code>internal</code>	w obrębie tego samego projektu

Na co uważać: domyślnie pola powinny być `private`, a dostęp do nich kontrolowany przez `public` właściwości lub metody. Bezpośredni publiczny dostęp do wszystkich pól utrudnia kontrolę poprawności danych. To zasada hermetyzacji - fundament programowania obiektowego.

6.9. Pola, this i składowe statyczne

Pole to zmienna należąca do obiektu (lub klasy). Różni się od właściwości tym, że nie ma `get` / `set` - to „surowa” zmienna. Słowo `this` odnosi się do **bieżącego obiektu** i przydaje się, gdy nazwa parametru pokrywa się z nazwą pola. Składowa `static` należy do **klasy, a nie do obiektu** - istnieje jedna, wspólna dla wszystkich, i wywołujemy ją przez nazwę klasy, bez tworzenia obiektu.

```
public class Uzytkownik
{
    private string imie; // pole prywatne
    public static int LiczbaUzytkownikow = 0; // pole statyczne (wspólne)
```

```

public Uzytkownik(string imie)
{
    this.imie = imie;           // this.imie = pole; imie = parametr
    LiczbaUzytkownikow++;      // zwiększamy wspólny licznik
}

// Metoda statyczna - wywołujemy bez obiektu
public static string Powitanie() => "Witamy w aplikacji";
}

// Użycie statycznych składowych (przez nazwę klasy)
string tekst = Uzytkownik.Powitanie();
new Uzytkownik("Anna");
new Uzytkownik("Piotr");
int ilu = Uzytkownik.LiczbaUzytkownikow; // 2 (wspólne dla wszystkich)

```

Na co uważać: `this` rozróżnia pole od parametru o tej samej nazwie. Składowe `static` są **wspólne** - zmiana wpływa na „wszystkich”. Metody pomocnicze bez stanu (np. `Math.Round`, własne walidatory) zwykle są `static`. Nie nadużywaj statycznego, mutowalnego stanu - może powodować trudne błędy.

6.10. Dziedziczenie i polimorfizm (podstawy)

Dziedziczenie pozwala stworzyć nową klasę na bazie istniejącej - klasa **pochodna** przejmuje pola, właściwości i metody klasy **bazowej** i może dodać własne. Zapisujemy je dwukropkiem: `class Pies : Zwierze`. **Polimorfizm** pozwala klasie pochodnej **nadpisać** zachowanie metody bazowej - metodę w bazie oznaczamy `virtual`, a w pochodnej `override`. To zaawansowane podstawy programowania obiektowego, przydatne m.in. przy stronach MAUI (każda strona dziedziczy po `ContentPage` i nadpisuje `OnAppearing`).

```

// Klasa bazowa
public class Zwierze
{
    public string Imie { get; set; }
    public virtual string Dzwiek() => "..."; // virtual = można nadpisać
}

// Klasa pochodna - dziedziczy i nadpisuje
public class Pies : Zwierze
{
    public override string Dzwiek() => "Hau hau"; // override = nadpisanie
}

public class Kot : Zwierze
{
    public override string Dzwiek() => "Miau";
}

// Polimorfizm w działaniu
List<Zwierze> zwierzeta = new() { new Pies(), new Kot() };
foreach (Zwierze z in zwierzeta)
{
    Console.WriteLine(z.Dzwiek()); // "Hau hau", potem "Miau"
}

// Strona MAUI dziedziczy po ContentPage i nadpisuje metodę cyklu życia

```

```
public partial class MojaPage : ContentPage
{
    protected override void OnAppearing()
    {
        base.OnAppearing();    // base = wywołanie wersji z klasy bazowej
        // własny kod
    }
}
```

POJĘCIE	ZNACZENIE
: Bazowa	dziedziczenie po klasie bazowej
virtual	metoda, którą można nadpisać
override	nadpisanie metody w klasie pochodnej
base	odwołanie do wersji z klasy bazowej

6.11. Typy wyliczeniowe - enum

enum to typ przechowujący jedną z **z góry określonych, nazwanych wartości**. Zamiast „magicznych” liczb czy tekstów (np. status 0, 1, 2) używamy czytelnych nazw (`Status.Nowy`, `Status.Wyslany`). To zwiększa czytelność i zapobiega błędom - kompilator pilnuje, by wartość była jedną z dozwolonych.

```
// Definicja typu wyliczeniowego
public enum StatusZamowienia
{
    Nowe,
    Opłaczone,
    Wysłane,
    Dostarczone,
    Anulowane
}

// Użycie
StatusZamowienia status = StatusZamowienia.Nowe;

// W warunkach i switch
if (status == StatusZamowienia.Anulowane)
{
    // ...
}

string opis = status switch
{
    StatusZamowienia.Nowe      ⇒ "Oczekuje na płatność",
    StatusZamowienia.Opłaczone ⇒ "W realizacji",
    StatusZamowienia.Wysłane   ⇒ "W drodze",
    StatusZamowienia.Dostarczone ⇒ "Zakończone",
    -                          ⇒ "Anulowane"
};

// Jako właściwość modelu
public class Zamowienie
```

```
{
    public StatusZamowienia Status { get; set; } = StatusZamowienia.Nowe;
}
```

Na co uważać: `enum` jest znacznie czytelniejszy i bezpieczniejszy niż przechowywanie statusu jako liczby czy tekstu - kompilator nie pozwoli przypisać wartości spoza zestawu. Świetnie współgra ze `switch`. Używaj go wszędzie tam, gdzie wartość pochodzi z zamkniętego zbioru możliwości (status, kategoria, kierunek).

7. C#: daty, losowanie, async, wyjątki i LINQ

7.1. Losowanie - Random

Klasa `Random` generuje **liczby losowe** - przydatne w grach (rzut kością), losowym doborze czy generowaniu danych. Metoda `Next(min, max)` zwraca losową liczbę całkowitą od `min` (włącznie) do `max` (**wyłącznie**), `NextDouble()` zwraca ułamek z zakresu 0-1, a `Next(max)` - liczbę od 0 do `max-1`.

```
Random los = new Random();

int kostka = los.Next(1, 7);    // 1..6 (7 jest WYŁĄCZONE)
int procent = los.Next(0, 101); // 0..100
int indeks = los.Next(Lista.Count); // losowy indeks 0..Count-1
double ułamek = los.NextDouble(); // 0.0 .. ~1.0

// Losowy element z listy
string[] kolory = { "czerwony", "zielony", "niebieski" };
string losowy = kolory[los.Next(kolory.Length)];

// Rzut pięcioma kośćmi
int[] wyniki = new int[5];
for (int i = 0; i < 5; i++)
    wyniki[i] = los.Next(1, 7);
```

Na co uważać: górna granica `Next(min, max)` jest **wyłączona** - dla kości używamy `Next(1, 7)`, a nie `Next(1, 6)`. Twórz jeden obiekt `Random` (np. jako pole klasy: `readonly Random los = new();`) i używaj go wielokrotnie - tworzenie nowego w pętli w krótkim czasie może dawać powtarzalne wyniki.

7.2. Operacje matematyczne - Math

Klasa `Math` udostępnia gotowe funkcje matematyczne: zaokrąglanie, potęgowanie, pierwiastek, wartość bezwzględna, minimum/maksimum, zaokrąglanie w górę/dół. Wszystkie są statyczne - wywołujemy je przez `Math.Nazwa(...)`.

```
double a = Math.Round(3.567, 2); // 3.57 (zaokrąglenie do 2 miejsc)
double b = Math.Floor(3.9);      // 3 (w dół)
double c = Math.Ceiling(3.1);    // 4 (w górę)
double d = Math.Abs(-7);         // 7 (wartość bezwzględna)
double e = Math.Pow(2, 10);      // 1024 (2 do potęgi 10)
double f = Math.Sqrt(144);       // 12 (pierwiastek)
int g = Math.Max(5, 9);          // 9
int h = Math.Min(5, 9);          // 5
double pi = Math.PI;             // 3.14159...
```

METODA	DZIAŁANIE	PRZYKŁAD -> WYNIK
<code>Math.Round(x, n)</code>	zaokrąglij do n miejsc	<code>Round(3.567,2)</code> → <code>3.57</code>
<code>Math.Floor(x)</code>	zaokrąglij w dół	<code>Floor(3.9)</code> → <code>3</code>
<code>Math.Ceiling(x)</code>	zaokrąglij w górę	<code>Ceiling(3.1)</code> → <code>4</code>
<code>Math.Abs(x)</code>	wartość bezwzględna	<code>Abs(-7)</code> → <code>7</code>
<code>Math.Pow(a, b)</code>	potęga	<code>Pow(2,3)</code> → <code>8</code>
<code>Math.Sqrt(x)</code>	pierwiastek	<code>Sqrt(9)</code> → <code>3</code>
<code>Math.Max/Min(a,b)</code>	większa/mniejsza	<code>Max(5,9)</code> → <code>9</code>

Na co uważać: `Math.Round` domyślnie zaokrągla „do parzystej” przy połówce - dla typowych potrzeb wystarcza. `Math.Pow` i `Sqrt` zwracają `double`, więc do `int` trzeba rzutować. Nie myl `Math.Floor` (zawsze w dół) z rzutowaniem `(int)` (obcięcie) - dla liczb dodatnich dają to samo, ale dla ujemnych różnią się.

7.3. Data i czas - DateTime i TimeSpan

```

DateTime teraz = DateTime.Now;           // data + godzina
DateTime dzis = DateTime.Today;          // data, godzina 00:00
DateTime data = new DateTime(2025, 6, 15); // konkretna data

// Składowe
int rok = teraz.Year;
int miesiac = teraz.Month;
int dzien = teraz.Day;
int godzina = teraz.Hour;

// Dodawanie/odejmowanie czasu
DateTime jutro = dzis.AddDays(1);
DateTime zaGodzine = teraz.AddHours(1);
DateTime tydzienTemu = dzis.AddDays(-7);

// Różnica dat → TimeSpan
DateTime start = new DateTime(2025, 1, 1);
TimeSpan roznica = dzis - start;
int dni = roznica.Days;

// Formatowanie do tekstu
string tekst = teraz.ToString("dd.MM.yyyy"); // np. "15.06.2025"
string zCzasem = teraz.ToString("dd.MM.yyyy HH:mm");

// TimeSpan z TimePicker
TimeSpan godz = WybórGodziny.Time;
string h = godz.ToString(@"hh\:mm");

```


ELEMENT	ZWRACA	PRZYKŁAD
<code>DateTime.Now</code>	data + czas teraz	-
<code>DateTime.Today</code>	dzisiejsza data (00:00)	-
<code>.AddDays(n)</code>	data przesunięta	<code>Today.AddDays(7)</code>
<code>data1 - data2</code>	<code>TimeSpan</code> (różnica)	<code>.Days</code> , <code>.Hours</code>
<code>.ToString("format")</code>	tekst daty	<code>"dd.MM.yyyy"</code>

7.4. Programowanie asynchroniczne - `async`, `await`, `Task`

Niektóre operacje **trwają** (pobieranie z sieci, odczyt pliku, zapytanie do bazy). Gdyby wykonać je „normalnie”, interfejs by **zamarł** na ten czas. Dlatego wykonujemy je **asynchronicznie**: słowo `async` oznacza metodę asynchroniczną, `await` czeka na wynik **bez blokowania** ekranu, a `Task` reprezentuje „pracę w toku”. Dzięki temu podczas pobierania danych użytkownik wciąż może korzystać z aplikacji.

```
// Metoda asynchroniczna zwracająca wynik
async Task<string> PobierzDaneAsync()
{
    await Task.Delay(2000);    // symulacja długiej operacji (2 s)
    return "Pobrane dane";
}

// Handler zdarzenia - async void (wyjątek dla obsługi zdarzeń)
private async void OnPobierz(object sender, EventArgs e)
{
    Loader.IsRunning = true;

    try
    {
        string dane = await PobierzDaneAsync(); // czeka, nie blokując UI
        Wynik.Text = dane;
    }
    finally
    {
        Loader.IsRunning = false;
    }
}

// Typowe operacje asynchroniczne w MAUI
string tresc = await File.ReadAllTextAsync(sciezka); // plik
await DisplayAlert("Tytuł", "Treść", "OK");         // okno dialogowe
await Navigation.PushAsync(new SzczegolyPage());     // nawigacja
```

ELEMENT	ZNACZENIE
<code>async</code>	metoda asynchroniczna (umożliwia <code>await</code>)
<code>await</code>	czeka na wynik bez blokowania UI
<code>Task</code>	operacja bez wyniku
<code>Task<T></code>	operacja zwracająca wartość typu <code>T</code>

7.5. Obsługa wyjątków - try, catch, finally

Wyjątek to błąd pojawiający się w trakcie działania programu (brak pliku, błędna konwersja, brak sieci, dostęp do `null`). Aby aplikacja nie „wysypała się”, otaczamy ryzykowny kod blokiem `try`, a w `catch` reagujemy na błąd. Blok `finally` wykonuje się **zawsze** (niezależnie od tego, czy był błąd) - idealny do sprzątnięcia (np. ukrycia wskaźnika ładowania).

```
try
{
    string tresc = await File.ReadAllTextAsync(sciezka);
    Podglad.Text = tresc;
}
catch (FileNotFoundException)
{
    // konkretny rodzaj błędu - brak pliku
    await DisplayAlert("Błąd", "Plik nie istnieje.", "OK");
}
catch (Exception ex)
{
    // dowolny inny błąd - ex.Message zawiera opis
    await DisplayAlert("Błąd", $"Coś poszło nie tak: {ex.Message}", "OK");
}
finally
{
    Loader.IsRunning = false; // wykona się zawsze
}
```

BLOK	KIEDY SIĘ WYKONUJE
<code>try</code>	kod, który może rzucić błąd
<code>catch</code>	gdy wystąpi błąd (można mieć kilka)
<code>finally</code>	zawsze, na końcu (sprzątnięcie)

Na co uważać: lepiej **zapobiegać** błędom niż je łapać - używaj `TryParse` zamiast `Parse`, `File.Exists` przed odczytem, sprawdzaj `null`. `try/catch` rezerwuj na rzeczy nieprzewidywalne (sieć, pliki). Łap **konkretne** wyjątki przed ogólnym `Exception`. Nie zostawiaj pustego `catch` - to ukrywa błędy; przynajmniej zaloguj lub poinformuj użytkownika.

7.6. LINQ - wygodne operacje na kolekcjach

LINQ to zestaw metod do **wygodnego przetwarzania kolekcji** - filtrowania, sortowania, liczenia, sumowania - bez ręcznych pętli. Wymaga `using System.Linq;`. Najważniejsze metody: `Where` (filtruj), `Select` (przekształć), `OrderBy` / `OrderByDescending` (sortuj), `Count`, `Sum`, `Average`, `Max`, `Min`, `Any` (czy istnieje), `First` / `FirstOrDefault` (pierwszy). To bardzo praktyczne narzędzie przy listach.

```
using System.Linq;

List<int> liczby = new() { 5, 2, 8, 1, 9, 3 };

// Filtrowanie - tylko większe od 4
List<int> duze = liczby.Where(x => x > 4).ToList(); // [5, 8, 9]

// Sortowanie
List<int> rosnaco = liczby.OrderBy(x => x).ToList(); // [1,2,3,5,8,9]
List<int> malejaco = liczby.OrderByDescending(x => x).ToList();
```

```
// Liczenie, suma, średnia, max/min
int ile = liczby.Count(x => x > 4); // 3
int suma = liczby.Sum();           // 28
double srednia = liczby.Average(); // ~4.67
int najw = liczby.Max();           // 9

// Czy istnieje / pierwszy pasujący
bool jestParzysta = liczby.Any(x => x % 2 == 0); // true
int pierwszyDuzy = liczby.First(x => x > 4);     // 5

// Na liście obiektów (np. filtrowanie i sortowanie produktów)
var tanie = produkty
    .Where(p => p.Cena < 20)
    .OrderBy(p => p.Nazwa)
    .ToList();

// Filtrowanie tekstem (wyszukiwarka)
string fraza = "ka";
var pasujace = produkty
    .Where(p => p.Nazwa.ToLower().Contains(fraza.ToLower()))
    .ToList();
```

METODA	DZIAŁANIE
<code>Where(warunek)</code>	wybiera pasujące elementy
<code>OrderBy(klucz)</code>	sortuje rosnąco
<code>OrderByDescending</code>	sortuje malejąco
<code>Select(przekształcenie)</code>	przekształca elementy
<code>Count()</code> / <code>Sum()</code> / <code>Average()</code>	liczy / sumuje / średnia
<code>Max()</code> / <code>Min()</code>	największy / najmniejszy
<code>Any(warunek)</code>	czy istnieje pasujący
<code>First()</code> / <code>FirstOrDefault()</code>	pierwszy (lub domyślny)
<code>ToList()</code>	zamienia wynik na listę

Na co uważać: zapis `x => x > 4` to **wyrażenie lambda** - „dla elementu `x` zwróć warunek”. Większość metod LINQ zwraca „leniwy” wynik - `ToList()` materializuje go do listy. `First` rzuca wyjątek, gdy nic nie pasuje; `FirstOrDefault` zwraca wartość domyślną (`null` dla obiektów). LINQ świetnie nadaje się do wyszukiwarek i filtrowania list.

7.7. Łączenie elementów - prosta logika aplikacji

Na koniec połączmy poznane elementy w typowy scenariusz handlera. Niemal każda reakcja na akcję użytkownika to sekwencja: **odczytaj dane** → **sprawdź (waliduj)** → **przetwórz** → **pokaż wynik**. Poniższy przykład łączy odczyt z pól, walidację (`TryParse`, warunki), obliczenia i aktualizację interfejsu.

```
private void OnPrzelicz(object sender, EventArgs e)
{
    // 1. ODCZYT danych z kontrolek
    string nazwa = PoleNazwa.Text;
    string cenaTekst = PoleCena.Text;
```

```

string iloscTekst = PoleIlosc.Text;

// 2. WALIDACJA
if (string.IsNullOrEmpty(nazwa))
{
    Pokaz("Podaj nazwę produktu.", false);
    return;
}

if (!double.TryParse(cenaTekst, out double cena) || cena ≤ 0)
{
    Pokaz("Cena musi być liczbą większą od zera.", false);
    return;
}

if (!int.TryParse(iloscTekst, out int ilosc) || ilosc < 1)
{
    Pokaz("Ilość musi być liczbą całkowitą ≥ 1.", false);
    return;
}

// 3. PRZETWARZANIE (logika)
double wartosc = cena * ilosc;
string poziom = wartosc > 100 ? "duże zamówienie" : "małe zamówienie";

// 4. WYŚWIETLENIE wyniku
Podsumowanie.Text =
    $"Produkt: {nazwa}\n" +
    $"Cena: {cena:0.00} zł\n" +
    $"Ilość: {ilosc}\n" +
    $"Wartość: {wartosc:0.00} zł ({poziom})";
Podsumowanie.TextColor = Colors.Black;
}

private void Pokaz(string tekst, bool ok)
{
    Podsumowanie.Text = tekst;
    Podsumowanie.TextColor = ok ? Colors.Green : Colors.Red;
}

```

Na co uważać: trzymaj się schematu **odczytaj → sprawdź → policz → pokaż**. Najpierw waliduj (z wczesnym wyjściem `return`), potem licz, na końcu aktualizuj widok. Pomocnicza metoda (`Pokaz`) eliminuje powtarzanie. Ten wzorzec to fundament niemal każdej logiki w aplikacji - opanuj go, a poradzisz sobie z większością zadań.

To koniec kursu podstaw C#. Jeśli rozumiesz zmienne i typy, operatory, warunki, wszystkie pętle, tablice i listy, metody oraz klasy - masz solidny fundament. Reszta podręcznika pokazuje, jak użyć tej wiedzy do budowania interfejsu i logiki aplikacji MAUI. Wróć do tego rozdziału, gdy coś w kodzie będzie niejasne.

8. Podstawy XAML

XAML to język, w którym opisujemy **wygląd** aplikacji MAUI. Bez jego dobrego zrozumienia trudno świadomie budować interfejs. W tym rozdziale wyjaśniamy od zera, czym jest XAML, jak się go zapisuje, jak łączy się z C#, oraz omawiamy wszystkie pojęcia potrzebne do swobodnej pracy: znaczniki, atrybuty, właściwości, przestrzenie nazw, `x:Name`, `x:Class`, code-behind i typowe błędy składni.

8.1. Czym jest XAML

XAML (czytane „zamel”, pełna nazwa *eXtensible Application Markup Language*) to **język znaczników** służący do **opisu interfejsu użytkownika**. Jest oparty na XML, czyli zapisuje strukturę za pomocą **znaczników** (tagów) ujętych w nawiasy ostre `< >`. W MAUI w XAML deklarujemy, jakie elementy mają pojawić się na ekranie (etykiety, przyciski, pola, obrazy) i jak mają wyglądać. XAML jest **deklaratywny** - opisujemy „co ma być”, a nie „jak to krok po kroku narysować”. To podejście jest czytelne i pozwala oddzielić wygląd od logiki. Każdy plik XAML ma powiązany plik C# (*code-behind*), w którym piszemy zachowanie.

XAML służy do **budowania warstwy widoku**: rozmieszczania kontroltek, ustawiania ich właściwości (tekst, kolor, rozmiar) i tworzenia struktury ekranu. Dzięki niemu wygląd aplikacji jest opisany w jednym, przejrzystym miejscu, oddzielony od kodu logiki. To ułatwia czytanie, utrzymanie i współpracę w zespole.

Kiedy używać?

XAML stosujemy zawsze, gdy budujemy interfejs aplikacji MAUI - to domyślny i zalecany sposób. Teoretycznie cały interfejs można zbudować również w C#, ale XAML jest znacznie czytelniejszy dla struktury widoku. W praktyce: **wygląd → XAML, logika → C#**.

NAJWAŻNIEJSZE INFORMACJE

- XAML opiera się na **XML** (znaczniki w nawiasach ostrych).
- Jest **deklaratywny** - opisuje strukturę i wygląd.
- Każdy plik `.xaml` ma powiązany plik `.xaml.cs` (code-behind).
- Te same elementy można tworzyć w XAML lub w C#, ale XAML jest czytelniejszy dla widoku.

PRZYKŁAD XAML

```
<ContentPage xmlns="http://schemas.microsoft.com/dotnet/2021/maui"
              xmlns:x="http://schemas.microsoft.com/winfx/2009/xaml"
              x:Class="MojaApp.MainPage">
    <VerticalStackLayout Padding="20">
        <Label Text="To jest XAML" FontSize="24" />
    </VerticalStackLayout>
</ContentPage>
```

Na co uważać:

XAML przypomina HTML, ale to **inny język** o innych regułach (m.in. rozróżnia wielkość liter w nazwach kontroltek i właściwości). Nie zakładaj, że znaczniki HTML zadziałają w XAML.

8.2. Znaczniki (tagi)

Znacznik (tag) to podstawowy element składni XAML - nazwa kontrolki lub elementu ujęta w nawiasy ostre. Znaczniki występują parami: **otwierający** `<Label>` i **zamykający** `</Label>`. Pomiędzy nimi może znaleźć się treść lub inne znaczniki. Nazwa znacznika odpowiada nazwie klasy kontrolki w MAUI (np. `Label`, `Button`, `Grid`).

NAJWAŻNIEJSZE INFORMACJE

- Znacznik otwierający: `<Label>`; zamykający: `</Label>`.
- Nazwa znacznika = nazwa kontrolki/klasa.
- Wielkość liter ma znaczenie (`Label`, nie `label`).

PRZYKŁAD XAML

```
←!— Znacznik otwierający i zamykający z treścią między nimi —→
<Label>Tekst etykiety</Label>

←!— Znacznik z elementem potomnym —→
<VerticalStackLayout>
    <Label>Element wewnątrz layoutu</Label>
</VerticalStackLayout>
```

TYPOWE BŁĘDY

- Brak znacznika zamykającego (`</Label>`).
- Literówka w nazwie znacznika lub zła wielkość liter.

Na co uważać:

Każdy znacznik otwierający musi mieć odpowiadający mu zamykający - albo być znacznikiem **samozamykającym** (patrz 4.6). Nieparzysty znacznik to częsty błąd składni.

8.3. Atrybuty i właściwości

Atrybut to zapis wewnątrz znacznika ustawiający **właściwość** kontrolki, w formacie `Nazwa="wartość"`. **Właściwość** to cecha kontrolki, np. tekst, kolor, rozmiar. W XAML większość właściwości ustawiamy właśnie jako atrybuty. Wartości zapisujemy w cudzysłowie. Jeden znacznik może mieć wiele atrybutów oddzielonych spacjami.

NAJWAŻNIEJSZE INFORMACJE

- Atrybut: `Nazwa="wartość"` wewnątrz znacznika.
- Atrybut ustawia **właściwość** kontrolki.
- Wartości zawsze w cudzysłowie.
- Kilka atrybutów oddzielamy spacjami.

PRZYKŁAD XAML

```
←!— Trzy atrybuty ustawiające trzy właściwości →  
<Label Text="Witaj"  
      TextColor="DarkBlue"  
      FontSize="22" />
```

TYPOWE BŁĘDY

- Brak cudzysłowu wokół wartości (`FontSize=22` zamiast `FontSize="22"`).
- Zła nazwa właściwości lub jej wielkość liter.

Na co uważać:

Nazwy właściwości muszą dokładnie odpowiadać nazwom w MAUI (np. `TextColor`, nie `textcolor`). Wartości liczbowe i logiczne również zapisujemy w cudzysłowie - to wymóg składni XAML.

8.4. Property element syntax (właściwości jako elementy)

Niektóre właściwości mają **wartość zbyt złożoną**, by zapisać ją w cudzysłowie (np. cały layout, gradient, kształt). Wtedy używamy **składni właściwości jako elementu** (*property element syntax*): zapisujemy właściwość jako zagnieżdżony znacznik w formacie `<Kontrolka.Właściwość>`. To pozwala przypisać właściwości bogatą zawartość, a nie tylko prosty tekst.

NAJWAŻNIEJSZE INFORMACJE

- Składnia: `<Kontrolka.Właściwość> ... </Kontrolka.Właściwość>`.
- Używana, gdy wartość jest złożona (obiekt, kolekcja).
- Często przy `GestureRecognizers`, `Resources`, `StrokeShape`.

PRZYKŁAD XAML

```
←!— Złożona właściwość zapisana jako element →  
<Label Text="Dotknij mnie"  
      <Label.GestureRecognizers>  
        <TapGestureRecognizer Tapped="OnDotkniecie" />  
      </Label.GestureRecognizers>  
</Label>  
  
←!— Border z kształtem jako elementem →  
<Border>
```

```

<Border.StrokeShape>
    <Rectangle CornerRadius="12" />
</Border.StrokeShape>
<Label Text="Karta" />
</Border>

```

Na co uważać:

Składnia „z kropką” (`Label.GestureRecognizers`) zawsze odnosi się do właściwości tej samej kontrolki, w której jest zagnieżdżona. To nie jest osobna kontrolka, tylko sposób ustawienia właściwości.

8.5. Znaczniki samozamykające

Gdy kontrolka **nie ma zawartości między znacznikami** (wszystko ustawiamy atrybutami), możemy użyć **znacznika samozamykającego** - kończącego się ukośnikiem `</>`. To skrót zamiast pisania osobnego znacznika zamykającego. Jest powszechnie stosowany dla prostych kontroltek.

NAJWAŻNIEJSZE INFORMACJE

- Samozamykający: `<Label Text="Cześć" />`.
- Równoważny zapisowi `<Label Text="Cześć"></Label>`.
- Używany, gdy kontrolka nie ma elementów potomnych.

PRZYKŁAD XAML

```

<!-- Te dwa zapisy są równoważne -->
<Button Text="OK" />
<Button Text="OK"></Button>

```

TYPOWE BŁĘDY

- Pominięcie ukośnika `/` w znaczniku bez zawartości (brak zamknięcia).

Na co uważać:

Jeśli kontrolka ma elementy potomne (np. layout zawierający inne kontrolki), **nie może** być samozamykająca - musi mieć pełny znacznik zamykający.

8.6. Elementy zagnieżdżone - element nadrzędny i potomny

XAML buduje **hierarchię**: kontrolki umieszczone wewnątrz innych są ich **elementami potomnymi** (dziećmi), a kontrolka je zawierająca jest **elementem nadrzędnym** (rodzicem). To zagnieżdżanie tworzy **drzewo wizualne** całego ekranu. Layouty (np. `VerticalStackLayout`, `Grid`) są typowymi rodzicami zawierającymi wiele dzieci.

NAJWAŻNIEJSZE INFORMACJE

- **Rodzic** zawiera **dzieci**; dzieci mogą mieć własne dzieci.
- Cała strona to drzewo zagnieżdżonych elementów.
- `ContentPage` ma tylko **jedno** dziecko bezpośrednie (zwykle layout).

PRZYKŁAD XAML

```

<ContentPage ...>                                <!-- korzeń -->
  <VerticalStackLayout>                             <!-- rodzic -->
    <Label Text="Nagłówek" />                       <!-- dziecko 1 -->
    <HorizontalStackLayout>                         <!-- dziecko 2 (i rodzic) -->
      <Button Text="Tak" />                          <!-- wnuk 1 -->
      <Button Text="Nie" />                         <!-- wnuk 2 -->
    </HorizontalStackLayout>
  </VerticalStackLayout>
</ContentPage>

```

```
</VerticalStackLayout>
</ContentPage>
```

Na co uważać:

`ContentPage` przyjmuje **dokładnie jeden** element bezpośredni. Aby pokazać wiele kontroltek, opakuj je w jeden layout. Próba umieszczenia dwóch elementów wprost w `ContentPage` zakończy się błędem.

8.7. Komentarze w XAML

Komentarz to fragment tekstu ignorowany przez kompilator, służący do objaśnień dla człowieka. W XAML komentarze zapisujemy tak jak w XML/HTML: między `<!--` a `-->`. Dobrze stosowane komentarze ułatwiają zrozumienie kodu.

PRZYKŁAD XAML

```
<!-- To jest komentarz - nie wpływa na działanie -->
<VerticalStackLayout>
    <!-- Nagłówek ekranu -->
    <Label Text="Ustawienia" FontSize="24" />
</VerticalStackLayout>
```

TYPOWE BŁĘDY

- Próba zagnieżdżenia komentarza w komentarzu (nie dozwolone).
- Umieszczenie komentarza wewnątrz znacznika (między atrybutami) - to błąd.

Na co uważać:

Komentarza nie wolno umieszczać wewnątrz znacznika otwierającego (między atrybutami). Komentarze stawiaj **pośród** znacznikami, nie w środku nich.

8.8. Przestrzeń nazw - `xmlns` i `xmlns:x`

Przestrzeń nazw (namespace) mówi XAML-owi, skąd pochodzą używane znaczniki. Na początku pliku deklarujemy je atrybutami `xmlns`. Domyślna przestrzeń (`xmlns="...maui"`) udostępnia standardowe kontrolki MAUI (`Label`, `Button`). Przestrzeń `xmlns:x="...xaml"` dodaje elementy języka XAML, takie jak `x:Name` i `x:Class`. Możemy też deklarować własne przestrzenie, by używać naszych klas.

NAJWAŻNIEJSZE INFORMACJE

DEKLARACJA	CO UDOSTĘPNI
<code>xmlns="...maui"</code>	standardowe kontrolki MAUI
<code>xmlns:x="...xaml"</code>	elementy języka XAML (<code>x:Name</code> , <code>x:Class</code> , <code>x:Key</code>)
<code>xmlns:local="clr-namespace:MojaApp"</code>	nasze własne klasy z projektu

PRZYKŁAD XAML

```
<ContentPage xmlns="http://schemas.microsoft.com/dotnet/2021/maui"
              xmlns:x="http://schemas.microsoft.com/winfx/2009/xaml"
              xmlns:local="clr-namespace:MojaApp"
              x:Class="MojaApp.MainPage">
    <!-- local:MojWidok pochodzi z naszego projektu -->
    <local:MojWidok />
</ContentPage>
```

TYPOWE BŁĘDY

- Brak deklaracji przestrzeni nazw dla własnej klasy (`xmlns:local`).

- Zła ścieżka w `clr-namespace` (niezgodna z przestrzenią nazw klasy).

Na co uważać:

Aby użyć własnej klasy w XAML, musisz zadeklarować dla niej przestrzeń nazw (`xmlns:local="clr-namespace: ..."`) i odwołać się przez prefiks (`local:`). Prefiks (`x`, `local`) możesz nazwać dowolnie, ale przyjęło się `x` dla XAML i `local` dla własnych klas.

8.9. Typowe błędy składni XAML

NAJWAŻNIEJSZE INFORMACJE

XAML jest wymagający co do składni. Poniższa tabela zbiera najczęstsze błędy i ich przyczyny:

BŁĄD	PRZYCZYNA	ROZWIĄZANIE
Niezamknięty znacznik	brak <code></...></code> lub <code>>/</code>	zamknij każdy znacznik
Brak cudzysłowu	<code>FontSize=20</code>	zapisz <code>FontSize="20"</code>
Zła wielkość liter	<code>textcolor</code> zamiast <code>TextColor</code>	zachowaj dokładną pisownię
Dwa dzieci w <code>ContentPage</code>	brak layoutu opakowującego	opakuj w jeden layout
Nieznana właściwość	literówka lub zła kontrolka	sprawdź nazwę właściwości
Brak przestrzeni nazw	użycie własnej klasy bez <code>xmlns</code>	zadeklaruj <code>xmlns:local</code>
Niezgodność <code>x:Class</code>	inna nazwa klasy w <code>.cs</code>	ujednolić nazwy

Na co uważać:

Większość błędów XAML to drobne pomyłki składni: niezamknięty znacznik, brak cudzysłowu, zła wielkość liter. Czytaj komunikaty błędów - zwykle wskazują linię i element, w którym jest problem. Edytor z podpowiedziami (IntelliSense) znacząco redukuje takie pomyłki.

Gdy XAML „nie chce się skompilować”, zacznij od sprawdzenia trzech rzeczy: czy wszystkie znaczniki są zamknięte, czy wszystkie wartości mają cudzysłowy oraz czy `ContentPage` ma tylko jedno dziecko. To pokrywa większość typowych błędów.

8.10. Markup extensions - rozszerzenia znaczników

Markup extension (rozszerzenie znaczników) to specjalny zapis w klamrach `{ }` używany jako wartość atrybutu, gdy potrzebujemy czegoś więcej niż zwykłego tekstu - np. odwołania do zasobu, wiązania danych czy wartości stałej. To „mini-funkcje” XAML wykonywane przy ładowaniu widoku. Najważniejsze z nich poznasz w praktyce na każdym kroku.

ROZSZERZENIE	DO CZEGO	PRZYKŁAD
<code>{Binding ...}</code>	wiązanie danych	<code>Text="{Binding Imie}"</code>
<code>{StaticResource klucz}</code>	zasób (pobrany raz)	<code>Color="{StaticResource Primary}"</code>
<code>{DynamicResource klucz}</code>	zasób śledzący zmiany	<code>BackgroundColor="{DynamicResource Tlo}"</code>
<code>{x:Static ...}</code>	wartość statyczna z C#	<code>Text="{x:Static local:Stałe.Wersja}"</code>
<code>{x:Reference nazwa}</code>	odwołanie do innej kontrolki	<code>BindingContext="{x:Reference Suwak}"</code>
<code>{OnPlatform ...}</code>	inna wartość per platforma	<code>FontSize="{OnPlatform iOS=18, Android=16}"</code>
<code>{OnIdiom ...}</code>	inna wartość per urządzenie	<code>FontSize="{OnIdiom Phone=16, Desktop=24}"</code>
<code>{AppThemeBinding ...}</code>	wartość zależna od motywu	<code>TextColor="{AppThemeBinding Light=Black, Dark=White}"</code>
<code>{x:Null}</code>	wartość <code>null</code>	<code>BackgroundColor="{x:Null}"</code>

←— Kilka rozszerzeń naraz →
`<Label Text="{Binding Tytuł}"`

```
TextColor="{AppThemeBinding Light=Black, Dark=White}"
```

```
FontSize="{OnIdiom Phone=16, Desktop=22}" />
```

←!— Odwołanie do innej kontrolki przez x:Reference →

```
<Slider x:Name="Suwak" Maximum="100" />
```

```
<Label Text="{Binding Source={x:Reference Suwak}, Path=Value, StringFormat='{0:0}'}" />
```

Na co uważać: rozszerzenia zapisujemy w klamrach `{ }` bez cudzysłowu wewnątrz (cudzysłów obejmuje całość). Jeśli wartość atrybutu ma dosłownie zaczynać się od `{`, poprzedź ją `{ }` (escape). Najczęściej używasz `Binding`, `StaticResource` i `AppThemeBinding`.

8.11. x:Static, x:Reference, x:Null i x:Array

Rozszerzenia z przestrzeni `x:` dają dostęp do elementów języka. `x:Static` wstawia wartość **statycznego** pola/właściwości z C# (np. stałą). `x:Reference` odwołuje się do innej **kontrolki** po nazwie (przydatne do wiązań między kontrolkami). `x:Null` to wartość `null`. `x:Array` tworzy tablicę wartości wprost w XAML.

←!— x:Static - stała z klasy C# →

```
<Label Text="{x:Static local:Konfiguracja.NazwaAplikacji}" />
```

←!— x:Reference - etykieta pokazuje wartość suwaka bez code-behind →

```
<Slider x:Name="Glosnosc" Maximum="100" Value="50" />
```

```
<Label Text="{Binding Source={x:Reference Glosnosc}, Path=Value, StringFormat='Głośność: {0:0}%'}" />
```

←!— x:Array - tablica tekstów jako ItemsSource Pickera →

```
<Picker Title="Kolor">
```

```
    <Picker.ItemsSource>
```

```
        <x:Array Type="{x:Type x:String}">
```

```
            <x:String>Czerwony</x:String>
```

```
            <x:String>Zielony</x:String>
```

```
            <x:String>Niebieski</x:String>
```

```
        </x:Array>
```

```
    </Picker.ItemsSource>
```

```
</Picker>
```

Na co uważać: `x:Reference` pozwala związać dwie kontrolki bez pisania kodu - np. etykieta „na żywo” pokazująca wartość suwaka. `x:Static` wymaga, by składowa była `static`. Te rozszerzenia poznasz głębiej przy data bindingu.

8.12. Zasoby w XAML - ResourceDictionary

Zasób to obiekt wielokrotnego użytku (kolor, styl, szablon) przechowywany w `ResourceDictionary` i przywoływany po kluczu (`x:Key`). Zasoby możemy umieścić na poziomie kontrolki, strony (`ContentPage.Resources`) lub całej aplikacji (`App.xaml`). Zasoby aplikacji są dostępne wszędzie, zasoby strony - tylko na niej. Odwołujemy się przez `{StaticResource klucz}` (pobranie raz) lub `{DynamicResource klucz}` (śledzi zmiany, np. motyw).

←!— Zasoby strony →

```
<ContentPage.Resources>
```

```
    <ResourceDictionary>
```

```
        <Color x:Key="KolorAkcent">#FF5722</Color>
```

```
        <x:Double x:Key="DuzaCzcionka">24</x:Double>
```

```
        <Thickness x:Key="StandardowyPadding">20</Thickness>
```

```
    </ResourceDictionary>
```

```
</ContentPage.Resources>
```

```
<VerticalStackLayout Padding="{StaticResource StandardowyPadding}">
    <Label Text="Nagłówek"
        TextColor="{StaticResource KolorAkcent}"
        FontSize="{StaticResource DuzaCzcionka}" />
</VerticalStackLayout>
```

Na co uważać: każdy zasób potrzebuje `x:Key` (klucza), po którym go przywołujesz. Odwołanie do nieistniejącego klucza powoduje błąd przy ładowaniu strony. Kolory i style najlepiej trzymać globalnie w `App.xaml`, by były spójne w całej aplikacji.

8.13. Style w XAML - Style, Setter, TargetType

Styl pozwala zdefiniować zestaw właściwości raz i zastosować do wielu kontroltek. Składa się z elementów `Setter` (każdy ustawia jedną właściwość) i ma `TargetType` (typ kontrolki). Styl **nazwany** (z `x:Key`) stosujemy ręcznie przez `Style="{StaticResource ...}"`; styl **niejawnny** (bez `x:Key`) obejmuje automatycznie wszystkie kontrolki danego typu.

```
<ContentPage.Resources>
    <!-- Styl nazwany dla przycisku głównego -->
    <Style x:Key="PrzyciskGłówny" TargetType="Button">
        <Setter Property="BackgroundColor" Value="#2196F3" />
        <Setter Property="TextColor" Value="White" />
        <Setter Property="CornerRadius" Value="8" />
        <Setter Property="HeightRequest" Value="48" />
        <Setter Property="FontAttributes" Value="Bold" />
    </Style>

    <!-- Styl niejawnny - wszystkie Label bez x:Key -->
    <Style TargetType="Label">
        <Setter Property="FontSize" Value="16" />
        <Setter Property="TextColor" Value="#333333" />
    </Style>
</ContentPage.Resources>

<VerticalStackLayout Padding="20" Spacing="12">
    <Label Text="Mam styl niejawnny" />
    <Button Text="Zapisz" Style="{StaticResource PrzyciskGłówny}" />
    <Button Text="Wyślij" Style="{StaticResource PrzyciskGłówny}" />
</VerticalStackLayout>
```

Na co uważać: styl niejawnny w `App.xaml` obejmuje całą aplikację - to najszybszy sposób na spójną typografię. Styl nazwany daje warianty (np. przycisk główny i drugorzędny). Style omawiamy szerzej w kontekście wyglądu; tutaj ważne, że to też element XAML.

8.14. OnPlatform i OnIdiom w XAML

W XAML możemy podać **różne wartości właściwości** zależnie od platformy (`OnPlatform`) lub typu urządzenia (`OnIdiom`) - bez pisania kodu. To podstawa responsywności i dostosowania do systemów.

```
<!-- Inny rozmiar czcionki per platforma (zapis inline) -->
<Label Text="Tytuł" FontSize="{OnPlatform Android=18, iOS=20, WinUI=24}" />

<!-- Inny padding per urządzenie (zapis rozbudowany z typem) -->
<ContentPage.Padding>
    <OnIdiom x:TypeArguments="Thickness" Phone="16" Tablet="28" Desktop="40" />
</ContentPage.Padding>
```

```
</ContentPage.Padding>
```

←!— Połączenie z innymi właściwościami →

```
<Button Text="DaLej"
        WidthRequest="{OnIdiom Phone=200, Desktop=320}"
        HorizontalOptions="Center" />
```

Na co uważać: w zapisie rozbudowanym `OnPlatform` / `OnIdiom` podaj `x:TypeArguments` (typ wartości, np. `Thickness`, `x:Double`). Zapis inline (`{OnIdiom Phone=..., Desktop=...}`) jest krótszy dla prostych typów. To czytelniejsze niż dyrektywy `#if` dla różnic w wartościach.

8.15. Pełny szablon strony XAML - podsumowanie składni

Poniżej kompletny, opisany szablon strony łączący poznane elementy składni XAML: deklaracje przestrzeni nazw, `x:Class`, zasoby, layout główny, kontrolki z `x:Name`, wiązania i rozszerzenia. To wzorzec, od którego zaczyna się każda strona.

```
<?xml version="1.0" encoding="utf-8" ?>
<ContentPage xmlns="http://schemas.microsoft.com/dotnet/2021/maui"
             xmlns:x="http://schemas.microsoft.com/winfx/2009/xaml"
             xmlns:local="clr-namespace:MojaApp"
             x:Class="MojaApp.PrzykladPage"
             Title="Przykład"
             BackgroundColor="{AppThemeBinding Light=White, Dark=#1E1E1E}">

    ←!— Zasoby strony →
    <ContentPage.Resources>
        <Style x:Key="Naglowek" TargetType="Label">
            <Setter Property="FontSize" Value="24" />
            <Setter Property="FontAttributes" Value="Bold" />
        </Style>
    </ContentPage.Resources>

    ←!— Jedyny element główny: layout →
    <ScrollView>
        <VerticalStackLayout Padding="20" Spacing="12">
            <Label Text="Tytuł ekranu" Style="{StaticResource Naglowek}" />
            <Entry x:Name="PoleImie" Placeholder="Wpisz imię" />
            <Button Text="Zatwierdź" Clicked="OnZatwierdz" />
            <Label x:Name="Wynik" />
        </VerticalStackLayout>
    </ScrollView>
</ContentPage>
```

Na co uważać: zawsze pilnuj: poprawne `xmlns`, zgodny `x:Class` z code-behind, jeden element główny w `ContentPage`, `x:Name` dla kontrolki używanych w C#. Ten szablon to fundament - kopij go jako punkt startu nowej strony.

Strona to pełnoekranowa jednostka interfejsu - pojedynczy ekran, który widzi użytkownik. Aplikacja składa się zwykle z wielu stron, między którymi przemieszczamy się za pomocą nawigacji. W tym rozdziale poznasz typy stron dostępne w MAUI, ich zastosowania oraz cykl życia strony, czyli zdarzenia od jej utworzenia po zniknięcie z ekranu. Zrozumienie cyklu życia jest kluczowe, by poprawnie ładować i odświeżać dane.

9. XAML w projekcie MAUI i code-behind

9.1. Relacja XAML i C#

XAML i C# to dwie połówki tej samej aplikacji. XAML opisuje, **co widać** (widok), a C# definiuje, **co się dzieje** (logika). Łączy je mechanizm **code-behind**: każdy plik `.xaml` ma plik `.xaml.cs` o tej samej nazwie, a oba tworzą jedną klasę dzięki słowu kluczowemu `partial`. Podczas kompilacji XAML jest tłumaczony na obiekty C#, a metoda `InitializeComponent()` „spina” zadeklarowany widok z kodem.

Ten podział pozwala **oddzielić wygląd od zachowania**. Dzięki temu można zmieniać interfejs bez ruszania logiki i odwrotnie. To jedna z najważniejszych zalet MAUI.

NAJWAŻNIEJSZE INFORMACJE

- Plik `.xaml` → **widok**; plik `.xaml.cs` → **logika** (code-behind).
- Oba pliki to jedna klasa `partial`.
- `InitializeComponent()` w konstruktorze łączy XAML z C#.
- Kontrolce nadajemy `x:Name`, by sięgnąć po nią z C#.

PRZYKŁAD XAML

```
<!-- Widok: definiuje przycisk i etykietę -->
<VerticalStackLayout Padding="20" Spacing="10">
    <Label x:Name="Wynik" Text="0" FontSize="28" />
    <Button Text="Dodaj 1" Clicked="OnDodaj" />
</VerticalStackLayout>
```

PRZYKŁAD C#

```
// Logika: obsługuje kliknięcie i aktualizuje etykietę
public partial class MainPage : ContentPage
{
    int licznik = 0;

    public MainPage()
    {
        InitializeComponent(); // łączy widok z tą klasą
    }

    private void OnDodaj(object sender, EventArgs e)
    {
        licznik++;
        Wynik.Text = licznik.ToString();
    }
}
```

TYPOWE BŁĘDY

- Brak `InitializeComponent()` - kontrolki z XAML są niedostępne (`null`).
- Niezgodność `x:Class` z nazwą klasy w `.cs`.

Na co uważać:

Nie traktuj XAML i C# jako osobnych światów - to jedna klasa. Kontrolka zadeklarowana w XAML jest dostępna w C# tylko wtedy, gdy ma `x:Name` i gdy `InitializeComponent()` zostało wywołane.

9.2. `x:Class` - powiązanie z klasą C#

Atrybut `x:Class` w głównym znaczniku pliku XAML wskazuje **klasę C#**, z którą ten widok jest powiązany. To on „spina” plik `.xaml` z plikiem `.xaml.cs`. Wartość `x:Class` musi dokładnie odpowiadać przestrzeni nazw i nazwie klasy w code-behind.

PRZYKŁAD XAML

```
←!— Widok powiązany z klasą MojaApp.MainPage →  
<ContentPage ... x:Class="MojaApp.MainPage">  
    ...  
</ContentPage>
```

```
// Code-behind musi mieć dokładnie tę samą przestrzeń i nazwę  
namespace MojaApp;  
public partial class MainPage : ContentPage { ... }
```

TYPOWE BŁĘDY

- Niezgodność `x:Class` z nazwą lub przestrzenią klasy w `.cs` - błąd kompilacji.

Na co uważać:

`x:Class` to klej między widokiem a logiką. Jeśli zmienisz nazwę klasy w `.cs`, zmień ją także w `x:Class`, i odwrotnie.

9.3. `x>Name` i `Name` - nazwy kontroltek

`x>Name` nadaje kontrolce **nazwę**, dzięki której można odwołać się do niej z C#. Po skompilowaniu MAUI tworzy w klasie pole o tej nazwie wskazujące na daną kontrolkę. Istnieje też zwykła właściwość `Name`, ale w praktyce do odwołań z code-behind używamy `x>Name`.

NAJWAŻNIEJSZE INFORMACJE

- `x>Name` umożliwia dostęp do kontrolki z C# (`Etykieta.Text = ...`).
- Nazwy powinny być czytelne i opisowe (`PoleEmail`, `PrzyciskZapisz`).
- `Name` i `x>Name` w MAUI w praktyce działają podobnie; standardem jest `x>Name`.

PRZYKŁAD XAML I C#

```
<Label x>Name="Komunikat" Text="Gotowy" />  
<Entry x>Name="PoleImie" Placeholder="imię" />
```

```
private void OnKliknij(object sender, EventArgs e)  
{  
    Komunikat.Text = $"Witaj, {PoleImie.Text}!";  
}
```

TYPOWE BŁĘDY

- Literówka w `x>Name` lub w odwołaniu z C# - kontrolka niezaleziona.
- Dwie kontrolki o tej samej nazwie `x>Name` - błąd.

Na co uważać:

`x>Name` musi być unikalne w obrębie strony. Po dodaniu `x>Name` czasem trzeba przebudować projekt, by wygenerowało się powiązane pole, zanim C# „zobaczy” kontrolkę.

9.4. Pliki `.xaml`, `.xaml.cs`, klasa `partial` i `InitializeComponent()`

Para `.xaml` + `.xaml.cs` tworzy jedną klasę częściową (`partial`). Słowo `partial` oznacza, że definicja klasy jest podzielona na kilka plików, a kompilator łączy je w całość. Metoda `InitializeComponent()`, wywoływana w konstruktorze, wczytuje strukturę z XAML i inicjalizuje wszystkie kontrolki oraz powiązania.

NAJWAŻNIEJSZE INFORMACJE

- `partial` pozwala podzielić klasę na pliki (XAML + code-behind).
- `InitializeComponent()` musi być wywołane w konstruktorze.
- Bez `InitializeComponent()` kontrolki z XAML będą `null`.

PRZYKŁAD C#

```
public partial class MainPage : ContentPage
{
    public MainPage()
    {
        InitializeComponent(); // wczytuje widok z XAML
    }
}
```

TYPOWE BŁĘDY

- Najczęstszy błąd: brak `InitializeComponent()` - aplikacja rzuca wyjątek lub kontrolki są niedostępne.
- Usunięcie słowa `partial` z deklaracji klasy.

Na co uważać:

Nigdy nie usuwaj `InitializeComponent()` z konstruktora strony. To ono buduje widok z XAML. Jego brak to jeden z najczęstszych błędów początkujących.

10. Strony i cykl życia aplikacji

10.1. `ContentPage`

`ContentPage` to najczęściej używany typ strony - reprezentuje pojedynczy ekran z dowolną zawartością. Ma jedną główną właściwość `Content`, do której przypisujemy jeden element (zwykle layout zawierający kontrolki). Każda nowa strona w projekcie domyślnie dziedziczy po `ContentPage`. To na niej budujemy formularze, listy, ekrany szczegółów i praktycznie wszystko, co użytkownik widzi.

`ContentPage` służy jako płótno dla interfejsu pojedynczego ekranu. Wewnątrz umieszczamy layout, a w nim kontrolki. Ustawiamy też tytuł (`Title`) widoczny w pasku nawigacji oraz tło (`BackgroundColor`).

NAJWAŻNIEJSZE WŁAŚCIWOŚCI

WŁAŚCIWOŚĆ	OPIS	PRZYKŁAD
<code>Content</code>	jedyny element główny strony	layout z kontrolkami
<code>Title</code>	tytuł strony (pasek nawigacji)	<code>Title="Ustawienia"</code>
<code>BackgroundColor</code>	kolor tła strony	<code>BackgroundColor="White"</code>
<code>Padding</code>	wewnętrzny margines treści	<code>Padding="20"</code>

PRZYKŁAD PODSTAWOWY

```
<?xml version="1.0" encoding="utf-8" ?>
<ContentPage xmlns="http://schemas.microsoft.com/dotnet/2021/maui"
              xmlns:x="http://schemas.microsoft.com/winfx/2009/xaml"
```

```

        x:Class="MojaApp.MainPage"
        Title="Strona główna"
        BackgroundColor="White">
<VerticalStackLayout Padding="20" Spacing="10">
    <Label Text="Treść strony" FontSize="22" />
</VerticalStackLayout>
</ContentPage>

```

PRZYKŁAD W C#

```

// Stronę można też zbudować w całości w C#
public class PrzykladPage : ContentPage
{
    public PrzykladPage()
    {
        Title = "Przykład";
        Content = new VerticalStackLayout
        {
            Padding = 20,
            Children = { new Label { Text = "Treść", FontSize = 22 } }
        };
    }
}

```

TYPOWE ZASTOSOWANIA

- Ekran formularza (logowanie, rejestracja).
- Ekran listy (np. produktów).
- Ekran szczegółów elementu.

TYPOWE BŁĘDY

- Próba umieszczenia kilku elementów wprost w `ContentPage` (przyjmuje tylko jeden).
- Brak layoutu opakowującego kontrolki.

10.2. TabbedPage

`TabbedPage` prezentuje kilka stron jako **zakładki** przełączane jednym dotknięciem. Każda zakładka to osobna strona. To model dla aplikacji o kilku równorzędnych sekcjach, np. „Dziś”, „Kalendarz”, „Profil”.

PRZYKŁAD PODSTAWOWY

```

<TabbedPage xmlns="http://schemas.microsoft.com/dotnet/2021/maui"
    xmlns:x="http://schemas.microsoft.com/winfx/2009/xaml"
    xmlns:local="clr-namespace:MojaApp"
    x:Class="MojaApp.GłównaTabbedPage">
    <local:StronaDzis Title="Dziś" />
    <local:StronaProfil Title="Profil" />
</TabbedPage>

```

TYPOWE ZASTOSOWANIA

- Aplikacje z 2–5 głównymi sekcjami dostępnymi od razu.

TYPOWE BŁĘDY

- Zbyt wiele zakładek (nie mieszczą się czytelnie na ekranie).

10.3. FlyoutPage

`FlyoutPage` dzieli ekran na **wysuwane menu boczne** (`Flyout`) i **treść główną** (`Detail`). Użytkownik otwiera menu gestem lub ikoną „hamburgera” i wybiera sekcję, a treść zmienia się w obszarze głównym. To wzorzec dla aplikacji z większą liczbą sekcji.

TYPOWE ZASTOSOWANIA

- Rozbudowane aplikacje z wieloma sekcjami w menu bocznym.

Na co uważać:

Współcześnie tę samą funkcję (menu boczne) realizuje wygodniej **Shell** przez `FlyoutItem` . `FlyoutPage` warto znać, ale w nowych projektach częściej używamy Shell.

10.4. Kiedy używać poszczególnych typów stron

NAJWAŻNIEJSZE INFORMACJE

TYP STRONY	KIEDY UŻYWAĆ
<code>ContentPage</code>	każdy pojedynczy ekran
<code>NavigationPage</code>	nawigacja „w głąb” z przyciskiem wstecz
<code>TabbedPage</code>	kilka równorzędnych sekcji jako zakładki
<code>FlyoutPage</code>	rozbudowane menu boczne
<code>Shell</code>	nowoczesna, zunifikowana nawigacja (zalecane)

Na co uważać:

`ContentPage` to fundament - używasz go zawsze do budowy pojedynczego ekranu. Pozostałe typy (`NavigationPage` , `TabbedPage` , `FlyoutPage` , Shell) decydują o **strukturze nawigacji** między ekranami. W nowych projektach najczęściej wybieramy Shell.

10.5. Cykl życia strony

Cykl życia strony to sekwencja zdarzeń, przez które przechodzi strona: od utworzenia (konstruktor), przez pojawienie się na ekranie (`OnAppearing`), aż po zniknięcie (`OnDisappearing`). Zrozumienie tej kolejności pozwala poprawnie ładować i odświeżać dane oraz sprzątać zasoby.

NAJWAŻNIEJSZE INFORMACJE

ETAP	METODA	KIEDY SIĘ WYKONUJE
Utworzenie	konstruktor	raz, przy tworzeniu strony
Pojawienie	<code>OnAppearing()</code>	przy każdym wejściu na stronę
Zniknięcie	<code>OnDisappearing()</code>	przy każdym opuszczeniu strony

PRZYKŁAD C#

```
public partial class ListaPage : ContentPage
{
    public ListaPage()
    {
        InitializeComponent(); // raz - inicjalizacja widoku
    }

    protected override void OnAppearing()
    {
        base.OnAppearing();
        ZaładujDane(); // za każdym razem, gdy wchodzimy na stronę
    }
}
```

```
protected override void OnDisappearing()
{
    base.OnDisappearing();
    // np. zapis stanu, zatrzymanie zadań
}

private void ZaładujDane() { /* pobranie i pokazanie danych */ }
}
```

Na co uważać:

Pamiętaj o wywołaniu `base.OnAppearing()` i `base.OnDisappearing()` przy przestanianiu tych metod. Pomińnięcie wywołania metody bazowej może spowodować nieoczekiwane zachowanie.

10.6. Konstruktor a `OnAppearing` - gdzie ładować dane

To jedno z najważniejszych rozróżnień przy pracy ze stronami. **Konstruktor** wykonuje się **tylko raz** - przy tworzeniu strony.

`OnAppearing` wykonuje się **za każdym razem**, gdy strona pojawia się na ekranie (także po powrocie z innej strony). Dlatego dane, które mają być **aktualne po każdym wejściu**, ładujemy w `OnAppearing`, a jednorazową inicjalizację - w konstruktorze.

PRZYKŁAD C#

```
public ListaPage()
{
    InitializeComponent();
    // Tu: jednorazowa konfiguracja, podpięcie kolekcji itp.
}

protected override void OnAppearing()
{
    base.OnAppearing();
    // Tu: odświeżenie listy - wykona się też po powrocie z ekranu dodawania
    OdswiezListe();
}
```

TYPOWE BŁĘDY

- Ładowanie danych w konstruktorze → lista nie odświeża się po powrocie.
- Brak `base.OnAppearing()` przy przestanianiu metody.

Na co uważać:

Jeśli po dodaniu elementu na innym ekranie wracasz na listę i **nie widzisz zmian**, prawie na pewno ładujesz dane w konstruktorze zamiast w `OnAppearing`. Przenieś ładowanie do `OnAppearing`, a problem zniknie.

10.7. Ładowanie i odświeżanie danych po wejściu i powrocie

Typowy scenariusz: ekran listy pokazuje dane, a osobny ekran pozwala dodać nowy element. Po powrocie z ekranu dodawania lista powinna **pokazać nowy element**. Realizujemy to, odświeżając dane w `OnAppearing`, które wykonuje się przy każdym powrocie na stronę.

PRZYKŁAD C#

```
protected override async void OnAppearing()
{
    base.OnAppearing();
    Loader.IsRunning = true;
}
```

```

try
{
    var dane = await baza.PobierzWszystkie(); // świeże dane z klasy bazy
    Lista.ItemsSource = dane;
}
finally
{
    Loader.IsRunning = false;
}
}

```

Na co uważać:

Odświeżanie w `OnAppearing` jest proste, ale uważaj, by nie wykonywać **ciężkich** operacji za każdym wejściem, jeśli dane się nie zmieniły. W większych aplikacjach warto sprawdzać, czy odświeżenie jest naprawdę potrzebne (np. flagą „dane zmienione”).

Zapamiętaj prostą zasadę: **konstruktor = raz**, `OnAppearing` = **za każdym razem**. To rozróżnienie rozwiązuje większość problemów z „nieodświeżającymi się” danymi po powrocie z innego ekranu.

Nawigacja to przemieszczanie się użytkownika między ekranami aplikacji. Prawie każda aplikacja ma więcej niż jeden ekran: listę i szczegóły, formularz i podsumowanie, ustawienia. W tym rozdziale poznasz dwa podejścia do nawigacji - klasyczny **stos stron** (`NavigationPage`) oraz nowoczesny **Shell** - a także sposoby przekazywania danych między ekranami.

11. Nawigacja i Shell

11.1. NavigationPage

`NavigationPage` to kontener zarządzający **stosem stron** i dodający na górze ekranu pasek nawigacji z tytułem oraz przyciskiem powrotu. Działa jak stos: nowe strony „wkładamy” na wierzch (`PushAsync`), a powrót „zdejmuje” bieżącą (`PopAsync`). To klasyczny model nawigacji „w głąb i z powrotem”.

`NavigationPage` umożliwia przechodzenie między ekranami z zachowaniem historii i automatycznym przyciskiem wstecz. Zwykle owija pierwszą stronę aplikacji.

PRZYKŁAD W C#

```

// Ustawienie strony startowej w NavigationPage (App.xaml.cs)
MainPage = new NavigationPage(new MainPage());

```

```

// Przejście do nowej strony i powrót
await Navigation.PushAsync(new SzczegolyPage());
await Navigation.PopAsync();

```

TYPOWE ZASTOSOWANIA

- Aplikacje z hierarchią ekranów (lista → szczegóły → edycja).

TYPOWE BŁĘDY

- Wywołanie `PushAsync`, gdy strona nie jest w `NavigationPage` ani w Shell.

11.2. Shell jako główny sposób budowy aplikacji

Shell to nowoczesny, zunifikowany system budowy struktury aplikacji. Zamiast ręcznie składać `NavigationPage`, `TabbedPage` i `FlyoutPage`, opisujemy całą nawigację deklaratywnie w jednym pliku `AppShell.xaml`. Shell oferuje menu boczne, zakładki i nawigację URI (routing) - wszystko spójnie i z minimalną ilością kodu.

Shell upraszcza nawigację: przejście do dowolnego ekranu sprowadza się do podania jego „adresu” (`GoToAsync`). Automatycznie zarządza paskiem nawigacji, przyciskiem wstecz i historią.

Kiedy używać?

Dla większości aplikacji z kilkoma ekranami Shell jest zalecanym wyborem. Dla bardzo prostych, jednoekranowych aplikacji można go pominąć.

11.3. Podstawy nawigacji

Nawigacja to mechanizm przechodzenia z jednego ekranu na drugi i wracania. W modelu klasycznym opiera się na **stosie** (ang. *stack*): wchodząc głębiej, kładziemy nową stronę na wierzch, a wracając, zdejmujemy ją, odstawiając poprzednią. Stos pamięta historię, dzięki czemu przycisk „wstecz” wie, dokąd wrócić.

Nawigacja pozwala **podzielić aplikację na logiczne ekrany** i prowadzić użytkownika przez kolejne kroki: od listy do szczegółów, od formularza do potwierdzenia. Bez niej cała aplikacja musiałaby zmieścić się na jednym ekranie.

NAJWAŻNIEJSZE INFORMACJE

- Model stosu: `PushAsync` (wejście), `PopAsync` (powrót).
- `PopToRootAsync` wraca do pierwszej strony stosu.
- Metody nawigacji są **asynchroniczne** (`await`).

PRZYKŁAD C#

```
// Wejście w głąb - nowa strona na wierzch stosu
await Navigation.PushAsync(new SzczegolyPage());

// Powrót o jeden ekran
await Navigation.PopAsync();

// Powrót do pierwszej strony
await Navigation.PopToRootAsync();
```

TYPOWE BŁĘDY

- Wywołanie `PushAsync` bez `NavigationPage`/Shell - nawigacja nie działa.
- Brak `await` przed metodą nawigacji.

Na co uważać:

Aby nawigacja stosowa działała, pierwsza strona musi być w `NavigationPage` (albo używamy Shell). Metody nawigacji są asynchroniczne - wywołuj je z `await` w metodzie `async`.

11.4. Nawigacja modalna

Nawigacja modalna otwiera stronę „na wierzchu” - jako okno wymagające zamknięcia, bez automatycznego przycisku wstecz. Używamy jej do ekranów, które przerywają główny przepływ: logowanie, formularz, który trzeba zatwierdzić lub anulować, ważny komunikat. Strona modalna zastania poprzednią do czasu jej zamknięcia.

PRZYKŁAD C#

```
// Otwarcie strony modalnej
await Navigation.PushModalAsync(new LogowaniePage());
```

```
// Zamknięcie strony modalnej (powrót)
await Navigation.PopModalAsync();
```

Na co uważać:

W oknie modalnym sam zadbaj o przycisk zamykający (np. „Anuluj”/„Zapisz”), bo nie ma domyślnego przycisku wstecz jak w nawigacji stosowej.

11.5. Shell, AppShell i ShellContent

Shell to system nawigacji, w którym całą strukturę aplikacji opisujemy deklaratywnie w pliku `AppShell.xaml`. Podstawowym elementem jest `ShellContent` - reprezentuje pojedynczy ekran osadzony w Shell. Shell potrafi też tworzyć menu boczne i zakładki. Strona startowa aplikacji ustawiana jest wtedy na `new AppShell()`.

PRZYKŁAD PODSTAWOWY

```
<?xml version="1.0" encoding="utf-8" ?>
<Shell xmlns="http://schemas.microsoft.com/dotnet/2021/maui"
      xmlns:x="http://schemas.microsoft.com/winfx/2009/xaml"
      xmlns:local="clr-namespace:MojaApp"
      x:Class="MojaApp.AppShell">

  <ShellContent Title="Start"
                ContentTemplate="{DataTemplate local:MainPage}"
                Route="start" />

</Shell>
```

```
// App.xaml.cs - Shell jako strona startowa
MainPage = new AppShell();
```

Na co uważać:

`ContentTemplate="{DataTemplate local:MainPage}"` sprawia, że strona tworzona jest dopiero przy pierwszym wejściu (leniwie), co jest wydajne. Pamiętaj o zadeklarowaniu przestrzeni `xmlns:local`.

11.6. FlyoutItem, TabBar i Tab

Shell pozwala budować dwa popularne układy nawigacji. `FlyoutItem` tworzy pozycję w menu bocznym (szufladzie). `TabBar` wraz z `Tab` tworzy dolny pasek zakładek. Oba zawierają w środku `ShellContent` wskazujący konkretne ekrany.

PRZYKŁAD PODSTAWOWY

```
<Shell ...>
  <!-- Menu boczne -->
  <FlyoutItem Title="Start" Icon="dom.png">
    <ShellContent ContentTemplate="{DataTemplate local:MainPage}" Route="start" />
  </FlyoutItem>
  <FlyoutItem Title="Ustawienia" Icon="ustawienia.png">
    <ShellContent ContentTemplate="{DataTemplate local:UstawieniaPage}" Route="ustawienia" />
  </FlyoutItem>

  <!-- Dolne zakładki -->
  <TabBar>
    <Tab Title="Lista" Icon="lista.png">
      <ShellContent ContentTemplate="{DataTemplate local:ListaPage}" />
    </Tab>
  </TabBar>
</Shell>
```

```

</Tab>
<Tab Title="Profil" Icon="profil.png">
    <ShellContent ContentTemplate="{DataTemplate local:ProfilPage}" />
</Tab>
</TabBar>
</Shell>

```

Na co uważać:

`FlyoutItem` daje menu boczne, a `TabBar` / `Tab` - zakładki. Można je łączyć, ale rozważ, czy nie komplikujesz nawigacji. Dla prostych aplikacji wystarczy kilka `ShellContent`.

11.7. Routing i GoToAsync

W Shellu do nawigacji służy `Shell.Current.GoToAsync("trasa")`, która przenosi użytkownika pod wskazany adres (URI). Ekran z menu/zakładek mają trasy zdefiniowane w `AppShell.xaml`, a ekrany „szczegółowe” trzeba dodatkowo zarejestrować przez `Routing.RegisterRoute`. Trasy mogą być względne (`"szczegoly"`) lub absolutne (`"//start"`), a `".."` oznacza powrót.

NAJWAŻNIEJSZE INFORMACJE

ZAPIS	ZNACZENIE
<code>"szczegoly"</code>	nawigacja względna (dotóż na stos)
<code>"//start"</code>	nawigacja absolutna (sekcja od korzenia)
<code>".."</code>	powrót o jeden ekran
<code>"..?id=5"</code>	powrót z parametrem

PRZYKŁAD C#

```

// Rejestracja trasy ekranu szczegółów (AppShell.xaml.cs)
public AppShell()
{
    InitializeComponent();
    Routing.RegisterRoute("szczegoly", typeof(SzczegolyPage));
}

```

```

// Nawigacja do trasy i powrót
await Shell.Current.GoToAsync("szczegoly");
await Shell.Current.GoToAsync("..");

```

TYPOWE BŁĘDY

- Nawigacja do trasy, która nie jest zarejestrowana ani zadeklarowana - błąd.
- Mylenie trasy względnej z absolutną (`//`).

Na co uważać:

Ekran dostępny tylko przez `GoToAsync` (np. szczegóły) **muszą** być zarejestrowane przez `Routing.RegisterRoute`. To częsta przyczyna błędów „nie można przejść do trasy”.

11.8. Przekazywanie danych między stronami

Często nowy ekran potrzebuje danych z poprzedniego - np. ekran szczegółów musi wiedzieć, który element pokazać. Dane przekazujemy na dwa sposoby: przez **konstruktor** strony (model klasyczny) lub przez **parametry w adresie** (Shell). Najlepszą praktyką jest przekazywanie **identyfikatora** elementu, a nie całego obiektu.

PRZYKŁAD C# (KONSTRUKTOR)

```
// Przekazanie obiektu przez konstruktor
await Navigation.PushAsync(new SzczegolyPage(wybranyProdukt));
```

```
public partial class SzczegolyPage : ContentPage
{
    public SzczegolyPage(Produkt produkt)
    {
        InitializeComponent();
        BindingContext = produkt; // dane gotowe do wyświetlenia
    }
}
```

Na co uważać:

Przekazywanie całego obiektu przez konstruktor jest proste, ale lepiej przekazywać **ID** i pobierać świeże dane na ekranie docelowym - wtedy zawsze pracujesz na aktualnej wersji danych.

11.9. [QueryProperty] i IQueryable

Przy nawigacji Shell dane przekazujemy w adresie jako parametry (`szczegoly?id=5`). Po stronie odbiorcy mamy dwa mechanizmy: atrybut `[QueryProperty]`, który automatycznie wpisuje parametr do właściwości, oraz interfejs `IQueryable` z metodą `ApplyQueryAttributes`, która odbiera wszystkie parametry jako słownik (wygodne, gdy parametrów jest więcej).

PRZYKŁAD C#

```
// Nawigacja z parametrem
await Shell.Current.GoToAsync($"szczegoly?id={produkt.Id}");
```

```
// Sposób 1: QueryProperty
[QueryProperty(nameof(Id), "id")]
public partial class SzczegolyPage : ContentPage
{
    public string Id { get; set; } // wypełnione automatycznie z adresu

    protected override void OnAppearing()
    {
        base.OnAppearing();
        // użyj Id, by pobrać dane
    }
}
```

```
// Sposób 2: IQueryable (więcej parametrów)
public partial class SzczegolyPage : ContentPage, IQueryable
{
    public void ApplyQueryAttributes(IDictionary<string, object> query)
    {
        string id = query["id"].ToString();
        // załaduj obiekt po id
    }
}
```

```
}  
  
}
```

Na co uważać:

`[QueryProperty(nameof(Id), "id")]` łączy parametr `id` z adresu z właściwością `Id`. Przy wielu parametrach wygodniejszy bywa `IQueryAttributable`. Pamiętaj o pobraniu danych po ustawieniu parametru (np. w `OnAppearing`).

11.10. Typowe błędy przy nawigacji

NAJWAŻNIEJSZE INFORMACJE

BŁĄD	PRZYCZYNA	ROZWIĄZANIE
<code>PushAsync</code> nie działa	brak <code>NavigationPage</code> /Shell	owiń stroną startową w <code>NavigationPage</code>
Błąd trasy w Shell	trasa niezarejestrowana	<code>Routing.RegisterRoute(...)</code>
Brak <code>await</code>	metoda nie czeka na nawigację	dodaj <code>await</code> i <code>async</code>
Dane nie docierają	zły parametr/QueryProperty	sprawdź nazwy parametrów
Podwójna nawigacja	szybkie dwukrotne kliknięcie	blokuj przycisk na czas nawigacji

Na co uważać:

Najczęstsze problemy to brak kontenera nawigacji (`NavigationPage` /Shell) oraz brak rejestracji trasy w Shell. Zawsze wywołuj metody nawigacji z `await`. Aby uniknąć podwójnej nawigacji przy szybkim dwukrotnym kliknięciu, możesz na chwilę wyłączyć przycisk (`IsEnabled = false`).

*Dla większości aplikacji z kilkoma ekranami **Shell** jest najwygodniejszym wyborem - łączy menu, zakładki i nawigację URI w jednym, spójnym mechanizmie. Klasyczny `NavigationPage` świetnie sprawdza się w prostych aplikacjach „lista → szczegóły”.*

12. Layout i rozmieszczanie elementów

Layout to fundament każdego ekranu - to on decyduje, gdzie i jak duże będą kontrolki. Dobór właściwego layoutu to pierwsza decyzja przy budowie ekranu, a umiejętność ich zagnieżdżania pozwala stworzyć dowolnie złożony interfejs z kilku prostych „klocków”. W tym rozdziale dokładnie omówimy wszystkie ważne layouty MAUI, właściwości rozmiaru i odstępów, wyrównanie oraz zasady responsywności.

12.1. Czym jest layout

Layout to specjalny kontener, którego jedynym zadaniem jest **rozmieszczanie elementów potomnych** na ekranie. Sam nic nie wyświetla - decyduje tylko o położeniu i rozmiarze kontrolki, które w nim umieścimy. Pod spodem layout działa w dwóch fazach: **pomiar** (pyta każde dziecko, ile miejsca potrzebuje) i **rozmieszczenie** (przydziela każdemu konkretny obszar). Dlatego ten sam zestaw kontrolki wygląda inaczej w `VerticalStackLayout` (jeden pod drugim) niż w `Grid` (w siatce).

Layout służy do **organizacji interfejsu**: ustawienia kontrolki w pionie, poziomie, siatce lub w dowolnym układzie. Bez layoutu nie da się czytelnie rozmieścić więcej niż jednej kontrolki, bo `ContentPage` przyjmuje tylko jeden element główny.

NAJWAŻNIEJSZE INFORMACJE

- Layout **nie wyświetla** własnej treści - rozmieszcza dzieci.
- `ContentPage` przyjmuje **jeden** element główny - zwykle layout.
- Layouty można **zagnieżdżać** (layout w layoutcie).
- Wybór layoutu zależy od pożądanego układu (pion, poziom, siatka).

Na co uważać:

Najpierw zastanów się, jaki układ chcesz uzyskać, a potem dobierz layout. Pion → `VerticalStackLayout`, poziom → `HorizontalStackLayout`, siatka/dwa wymiary → `Grid`. Złożone ekrany budujemy, zagnieżdżając te proste layouty.

12.2. Element nadrzędny i potomny, zagnieżdżanie

W layoucie obowiązuje relacja **rodzic-dziecko**: layout jest **rodzicem**, a kontrolki w nim - jego **dziećmi**. Dziecko może samo być layoutem z własnymi dziećmi - to **zagnieżdżanie**. Cała strona to drzewo zagnieżdżonych elementów. Zagnieżdżanie to najważniejsza technika budowy interfejsu: cały ekran owijamy w jeden layout główny, a poszczególne fragmenty realizujemy mniejszymi layoutami.

PRZYKŁAD XAML

```
<VerticalStackLayout Padding="20" Spacing="12">      <!-- rodzic główny -->
  <Label Text="Nagłówek" FontSize="24" />      <!-- dziecko -->

  <HorizontalStackLayout Spacing="10">          <!-- dziecko (i rodzic) -->
    <Button Text="Tak" />                        <!-- wnuk -->
    <Button Text="Nie" />                       <!-- wnuk -->
  </HorizontalStackLayout>

  <Grid ColumnDefinitions="*,*" ColumnSpacing="10"> <!-- kolejne dziecko -->
    <Label Text="Lewa" Grid.Column="0" />
    <Label Text="Prawa" Grid.Column="1" />
  </Grid>
</VerticalStackLayout>
```

Na co uważać:

Nie zagnieżdżaj layoutów „na zapas” - każdy dodatkowy poziom to nieco więcej pracy przy rozmieszczaniu. Używaj tylu, ile naprawdę potrzeba, by uzyskać układ. Zbyt głębokie zagnieżdżanie pogarsza czytelność i wydajność.

12.3. VerticalStackLayout

VerticalStackLayout układa dzieci **jeden pod drugim**, w pionowej kolumnie, w kolejności zapisu w XAML. To najczęściej używany layout w prostych aplikacjach, bo większość ekranów mobilnych to pionowa lista pól i przycisków.

Służy do budowy **pionowych układów**: formularzy, ekranów logowania, list ustawień. Każde dziecko domyślnie zajmuje pełną szerokość, a wysokość dobiera do zawartości.

NAJWAŻNIEJSZE WŁAŚCIWOŚCI

WŁAŚCIWOŚĆ	OPIS	PRZYKŁAD
Spacing	odstęp między dziećmi	Spacing="12"
Padding	wewnętrzny margines kontenera	Padding="20"
HorizontalOptions	wyrównanie poziome całego layoutu	Center

PRZYKŁAD PODSTAWOWY

```
<VerticalStackLayout Padding="20" Spacing="12">
  <Label Text="Imię" />
  <Entry Placeholder="Wpisz imię" />
  <Label Text="Nazwisko" />
  <Entry Placeholder="Wpisz nazwisko" />
  <Button Text="Zapisz" />
</VerticalStackLayout>
```

TYPOWE ZASTOSOWANIA

- Formularze (pola jeden pod drugim).
- Pionowe listy ustawień.
- Ekran z nagłówkiem i przyciskami.

TYPOWE BŁĘDY

- Zbyt wiele elementów bez `ScrollView` - część nie mieści się na ekranie.
- Mylenie z `HorizontalStackLayout` (kierunek układania).

12.4. HorizontalStackLayout

`HorizontalStackLayout` układa dzieci **obok siebie, w poziomie**, od lewej do prawej. Sprawdza się, gdy chcemy umieścić kilka elementów w jednym rzędzie: parę przycisków, ikonę z podpisem, serię obrazków.

NAJWAŻNIEJSZE WŁAŚCIWOŚCI

WŁAŚCIWOŚĆ	OPIS	PRZYKŁAD
<code>Spacing</code>	odstęp między dziećmi	<code>Spacing="10"</code>
<code>Padding</code>	margines wewnętrzny	<code>Padding="8"</code>
<code>HorizontalOptions</code>	położenie całego rzędu	<code>Center</code>

PRZYKŁAD PODSTAWOWY

```
<HorizontalStackLayout Spacing="10" HorizontalOptions="Center">
  <Button Text="-" WidthRequest="60" />
  <Label x:Name="Licznik" Text="0" FontSize="28" VerticalOptions="Center" />
  <Button Text="+" WidthRequest="60" />
</HorizontalStackLayout>
```

TYPOWE ZASTOSOWANIA

- Przyciski `+` / `-` obok licznika.
- Ikona i tekst w jednym rzędzie.
- Pasek akcji.

TYPOWE BŁĘDY

- Zbyt wiele elementów - nie zawijają się, lecz wychodzą poza ekran.

Na co uważać:

`HorizontalStackLayout` **nie zawija** elementów do nowej linii. Jeśli ich liczba jest zmienna i mają się zawijać, użyj `FlexLayout` z `Wrap="Wrap"`.

12.5. Grid - siatka wierszy i kolumn

`Grid` rozmieszcza elementy w **siatce wierszy i kolumn**, podobnie jak tabela. Definiujemy wiersze (`RowDefinitions`) i kolumny (`ColumnDefinitions`), a każdemu dziecku przypisujemy pozycję właściwościami `Grid.Row` i `Grid.Column` (numeracja od zera). Element może rozciągać się na kilka komórek przez `Grid.RowSpan` / `Grid.ColumnSpan`.

`Grid` jest idealny do układów **dwuwymiarowych**: formularzy z etykietami po lewej i polami po prawej, paneli przycisków, siatek obrazków, dashboardów. Daje pełną kontrolę nad proporcjami przestrzeni.

NAJWAŻNIEJSZE WŁAŚCIWOŚCI

WŁAŚCIWOŚĆ	OPIS	PRZYKŁAD
RowDefinitions	definicje wierszy	Auto,*,Auto
ColumnDefinitions	definicje kolumn	Auto,*
Grid.Row	wiersz dziecka (od 0)	Grid.Row="1"
Grid.Column	kolumna dziecka (od 0)	Grid.Column="0"
Grid.RowSpan	rozciągnięcie na wiersze	Grid.RowSpan="2"
Grid.ColumnSpan	rozciągnięcie na kolumny	Grid.ColumnSpan="2"
RowSpacing / ColumnSpacing	odstęp w siatce	RowSpacing="10"

PRZYKŁAD PODSTAWOWY

```
<Grid RowSpacing="10" ColumnSpacing="10" Padding="20"
      RowDefinitions="Auto,Auto" ColumnDefinitions="Auto,*">

  <Label Text="Login:" Grid.Row="0" Grid.Column="0" VerticalOptions="Center" />
  <Entry Placeholder="login" Grid.Row="0" Grid.Column="1" />

  <Label Text="Hasło:" Grid.Row="1" Grid.Column="0" VerticalOptions="Center" />
  <Entry IsPassword="True" Grid.Row="1" Grid.Column="1" />

</Grid>
```

PRZYKŁAD W C#

```
var grid = new Grid
{
    RowDefinitions = { new RowDefinition { Height = GridLength.Auto } },
    ColumnDefinitions =
    {
        new ColumnDefinition { Width = GridLength.Auto },
        new ColumnDefinition { Width = GridLength.Star }
    }
};

var etykieta = new Label { Text = "Login:" };
var pole = new Entry();
grid.Add(etykieta, 0, 0); // kolumna 0, wiersz 0
grid.Add(pole, 1, 0);     // kolumna 1, wiersz 0
```

TYPOWE ZASTOSOWANIA

- Formularze (etykieta + pole w jednym wierszu).
- Pasek: ikona + tytuł + przycisk.
- Siatki kafelków i dashboardy.

TYPOWE BŁĘDY

- Pomylenie kolejności `Grid.Row` i `Grid.Column`.
- Brak definicji wiersza/kolumny dla użytego indeksu.
- Liczenie od 1 zamiast od 0.

12.6. Wymiary w Grid: Auto, gwiazdka i wartości stałe

Rozmiar wiersza lub kolumny w **Grid** określamy na trzy sposoby. **Auto** = dokładnie tyle, ile potrzebuje zawartość. ***** (gwiazdka) = cała pozostała, wolna przestrzeń (mechanizm proporcjonalny - odpowiada za responsywność). **Liczba** (np. **100**) = stały rozmiar. Gwiazdki można ważyć: **2*** i **1*** dzielą miejsce w proporcji 2:1.

NAJWAŻNIEJSZE INFORMACJE

ZAPIS	ZNACZENIE	KIEDY UŻYWAĆ
Auto	rozmiar do zawartości	etykieta, ikona
*	cała wolna przestrzeń	główny obszar treści
2*, 1*	proporcjonalny podział	kolumny w proporcji
100	stała wartość	przycisk o stałej szerokości

PRZYKŁAD XAML

```

<!-- ikona (Auto) + tytuł (cała reszta) + przycisk (stała szerokość) -->
<Grid ColumnDefinitions="Auto,*,80" ColumnSpacing="10">
    <Image Source="logo.png" Grid.Column="0" WidthRequest="40" />
    <Label Text="Tytuł" Grid.Column="1" VerticalOptions="Center" />
    <Button Text="OK" Grid.Column="2" />
</Grid>

```

Na co uważać:

To gwiazdka ***** zapewnia **responsywność** - kolumna z ***** rośnie i kurczy się z ekranem. Łącz **Auto** (dla elementów o stałym rozmiarze) z ***** (dla obszaru, który ma wypełnić resztę), aby układ dobrze wyglądał na ekranach różnej wielkości.

12.7. FlexLayout

FlexLayout to elastyczny layout wzorowany na CSS Flexbox. Jego siłą jest **automatyczne zawijanie** elementów do nowej linii, gdy zabraknie miejsca (**Wrap**), oraz precyzyjna kontrola rozkładu (**JustifyContent**) i wyrównania (**AlignItems**). Idealny do układów kafelkowych i list o zmiennej liczbie elementów.

NAJWAŻNIEJSZE WŁAŚCIWOŚCI

WŁAŚCIWOŚĆ	OPIS	PRZYKŁAD
Direction	kierunek układania	Row, Column
Wrap	zawijanie elementów	Wrap, NoWrap
JustifyContent	rozkład wzdłuż osi głównej	SpaceAround
AlignItems	wyrównanie w poprzek	Center

PRZYKŁAD PODSTAWOWY

```

<FlexLayout Direction="Row" Wrap="Wrap"
    JustifyContent="SpaceAround" AlignItems="Center">
    <Button Text="Tag 1" Margin="5" />
    <Button Text="Tag 2" Margin="5" />
    <Button Text="Tag 3" Margin="5" />
    <Button Text="Tag 4" Margin="5" />
</FlexLayout>

```

TYPOWE ZASTOSOWANIA

- Zawijane „tagi” lub przyciski kategorii.
- Responsywne kafelki.

Na co uważać:

`FlexLayout` jest potężny, ale bywa trudniejszy do opanowania niż stack layouty. Do prostych układów wystarczą `VerticalStackLayout` / `HorizontalStackLayout`. `FlexLayout` wybieraj, gdy potrzebujesz zawijania.

12.8. AbsoluteLayout

`AbsoluteLayout` umieszcza elementy w **dokładnie określonych pozycjach** i pozwala je nakładać. Pozycję i rozmiar dziecka ustawiamy przez `AbsoluteLayout.LayoutBounds` (X, Y, szerokość, wysokość), a `AbsoluteLayout.LayoutFlags` decyduje, czy wartości są bezwzględne, czy **proporcjonalne** (0–1 względem kontenera).

PRZYKŁAD PODSTAWOWY

```
<AbsoluteLayout>
  <Image Source="tlo.png"
    AbsoluteLayout.LayoutBounds="0,0,1,1"
    AbsoluteLayout.LayoutFlags="All" />
  <Label Text="Nakładka"
    AbsoluteLayout.LayoutBounds="0.5,0.5,AutoSize,AutoSize"
    AbsoluteLayout.LayoutFlags="PositionProportional" />
</AbsoluteLayout>
```

TYPOWE ZASTOSOWANIA

- Nakładki (przycisk pływający nad obrazem, znaczek na ikonie).

Na co uważać:

Unikaj wartości bezwzględnych dla pozycji - psują responsywność. Jeśli musisz użyć `AbsoluteLayout`, preferuj tryb proporcjonalny (`PositionProportional` / `All`), by układ skalował się z ekranem. Do typowych formularzy `AbsoluteLayout` jest zbyt sztywny.

12.9. ScrollView

`ScrollView` dodaje **przewijanie**, gdy zawartość nie mieści się na ekranie. Przyjmuje **jeden** element (zwykle layout), który można przewijać palcem. Niezbędny przy dłuższych formularzach i treściach.

NAJWAŻNIEJSZE WŁAŚCIWOŚCI

WŁAŚCIWOŚĆ	OPIS	PRZYKŁAD
<code>Orientation</code>	kierunek przewijania	<code>Vertical</code> , <code>Horizontal</code>
<code>Content</code>	jedyny element wewnątrz	layout

PRZYKŁAD PODSTAWOWY

```
<ScrollView>
  <VerticalStackLayout Padding="20" Spacing="15">
    <Label Text="Długi formularz..." FontSize="24" />
    <Entry Placeholder="Pole 1" />
    <Entry Placeholder="Pole 2" />
    <!-- ...wiele pól... -->
    <Button Text="Zapisz" />
  </VerticalStackLayout>
</ScrollView>
```

TYPOWE ZASTOSOWANIA

- Dłgie formularze.
- Ekrany z dużą ilością treści.

TYPOWE BŁĘDY

- Umieszczenie kilku elementów wprost w `ScrollView` (przyjmuje tylko jeden).
- Wkładanie `CollectionView` / `ListView` do pionowego `ScrollView` (konflikt przewijania).

Na co uważać:

Nie zagnieżdżaj listy przewijanej (`CollectionView`, `ListView`) w pionowym `ScrollView` - listy przewijają się same, a połączenie powoduje konflikt. `ScrollView` stosuj do statycznej treści, która bywa dłuższa niż ekran.

12.10. Border i ContentView

`Border` to kontener otaczający **jeden element** obramowaniem i pozwalający na zaokrąglone rogi oraz tło - podstawa do budowy „kart”.

`ContentView` to bazowy kontener do tworzenia **własnych, wielokrotnie używalnych komponentów** (np. własna karta produktu jako osobna kontrolka).

PRZYKŁAD PODSTAWOWY (BORDER)

```
<Border Stroke="#DDDDDD" StrokeThickness="1"
        BackgroundColor="White" Padding="16" Margin="10">
  <Border.StrokeShape>
    <RoundRectangle CornerRadius="12" />
  </Border.StrokeShape>
  <VerticalStackLayout Spacing="6">
    <Label Text="Nazwa produktu" FontAttributes="Bold" FontSize="18" />
    <Label Text="49,99 zł" TextColor="Green" />
  </VerticalStackLayout>
</Border>
```

PRZYKŁAD (CONTENTVIEW JAKO WŁASNY KOMPONENT)

```
<!-- KartaProduktu.xaml -->
<ContentView xmlns="http://schemas.microsoft.com/dotnet/2021/maui"
              xmlns:x="http://schemas.microsoft.com/winfx/2009/xaml"
              x:Class="MojaApp.KartaProduktu">
  <Border Padding="12">
    <Label x:Name="Nazwa" FontSize="18" />
  </Border>
</ContentView>
```

Na co uważać:

`Border` (jak `ContentPage` i `ScrollView`) przyjmuje **jeden** element - by pokazać wiele kontrolki, wóń w niego layout. `Border` zastąpił starszą kontrolkę `Frame`; w nowym kodzie preferuj `Border`.

12.11. Rozmiary: WidthRequest, HeightRequest

`WidthRequest` i `HeightRequest` to **sugerowane** wymiary kontrolki w jednostkach niezależnych od urządzenia. Słowo „request” (prośba) jest istotne - to sugestia, którą layout może uwzględnić lub zmodyfikować, zależnie od dostępnego miejsca. Istnieją też

`MinimumWidthRequest` i `MinimumHeightRequest` (minimalne wymiary).

PRZYKŁAD XAML

```
<Button Text="Wąski przycisk" WidthRequest="160" HeightRequest="48" />
<Image Source="logo.png" WidthRequest="100" HeightRequest="100" />
```

TYPOWE BŁĘDY

- Ustawianie sztywnych rozmiarów wszędzie → interfejs źle skaluje się na różnych ekranach.

Na co uważać:

Nie ustawiaj rozmiarów „na sztywno” wszędzie - to psuje responsywność. Używaj ich tam, gdzie naprawdę trzeba (np. stały przycisk, ikona), a w pozostałych miejscach pozwól layoutom dobrać rozmiar automatycznie (gwiazdka `*`, `Fill`).

12.12. Odstępy: Margin, Padding, Spacing

To trzy mechanizmy kontroli przestrzeni. `Margin` to odstęp **na zewnątrz** kontrolki (od sąsiadów). `Padding` to odstęp **wewnątrz** kontenera (od krawędzi do dzieci). `Spacing` to jednolity odstęp między dziećmi w layoutach stosowych i `Grid`.

NAJWAŻNIEJSZE INFORMACJE

MECHANIZM	GDZIE	CO ROBI
<code>Margin</code>	dowolna kontrolka	odstęp na zewnątrz
<code>Padding</code>	kontener	odstęp wewnątrz
<code>Spacing</code>	stack layout, Grid	odstęp między dziećmi

Zapis odstępów: jedna liczba (wszystkie strony), dwie (`lewo-prawo, góra-dół`), cztery (`lewo, góra, prawo, dół`).

PRZYKŁAD XAML

```
<VerticalStackLayout Padding="20" Spacing="12">
  <Label Text="Tytuł" Margin="0,0,0,10" />
  <Button Text="Dalej" Margin="40,5" />
</VerticalStackLayout>
```

←!— wewnątrz + między dziećmi →
←!— 10 odstępu na dole →
←!— 40 lewo/prawo, 5 góra/dół →

TYPOWE BŁĘDY

- Mylenie `Margin` (na zewnątrz) z `Padding` (wewnątrz).
- Ręczne dodawanie marginesów zamiast użycia `Spacing`.

Na co uważać:

Do równych odstępów między elementami używaj `Spacing` (czytelniejsze niż marginesy na każdym elemencie). `Margin` rezerwuj dla wyjątków, gdy jeden element potrzebuje innego odstępu niż reszta.

12.13. Wyrównanie: HorizontalOptions i VerticalOptions

Każda kontrolka ma właściwości `HorizontalOptions` (wyrównanie poziome) i `VerticalOptions` (pionowe), decydujące o tym, jak zachowa się w przydzielonym miejscu. Cztery wartości: `Start`, `Center`, `End`, `Fill`. Domyślnie wiele kontrolki używa `Fill` (rozciągnięty), dlatego np. przycisk zajmuje pełną szerokość, dopóki nie ustawimy `Center`.

NAJWAŻNIEJSZE INFORMACJE

WARTOŚĆ	DZIAŁANIE
<code>Start</code>	do lewej/góry
<code>Center</code>	wyśrodkowanie
<code>End</code>	do prawej/dółu
<code>Fill</code>	rozciągnięcie na całą przestrzeń

PRZYKŁAD XAML

```
<VerticalStackLayout>
  <Label Text="Nagłówek" HorizontalOptions="Center" />
  <Button Text="Wąski, wyśrodkowany" HorizontalOptions="Center" />
  <Button Text="Pełna szerokość" HorizontalOptions="Fill" />
</VerticalStackLayout>
```

Na co uważać:

Różnica między `Fill` a `Center` jest kluczowa: `Fill` rozciąga element na całą dostępną przestrzeń, `Center` zostawia go w naturalnym rozmiarze i centruje. To najczęstsze narzędzie do „dopieszczania” położenia kontrolki.

12.14. Responsywność i różnice mobile/desktop

Responsywność to zdolność interfejsu do dobrego wyglądu na ekranach różnej wielkości - od małego telefonu po duży monitor. W MAUI uzyskujemy ją głównie przez **elastyczne rozmiary** (gwiazdka `*` w `Grid`, `HorizontalOptions="Fill"`) zamiast wartości sztywnych, oraz przez `ScrollView` dla treści, która może nie zmieścić się na małym ekranie.

NAJWAŻNIEJSZE INFORMACJE

- Telefon: układy pionowe, duże przyciski, `ScrollView`.
- Komputer: można wykorzystać szerokość (np. dwie kolumny w `Grid`).
- Elastyczne rozmiary (`*`, `Fill`) dostosowują się do ekranu.
- `OnIdiom` pozwala podać inną wartość dla telefonu i dla komputera.

PRZYKŁAD XAML

```
←!— Inny padding na telefonie i na komputerze →
<VerticalStackLayout>
  <VerticalStackLayout.Padding>
    <OnIdiom x:TypeArguments="Thickness" Phone="16" Desktop="40" />
  </VerticalStackLayout.Padding>
  <Label Text="Treść dopasowana do urządzenia" />
</VerticalStackLayout>
```

Na co uważać:

Projektuj z myślą o **najmniejszym** ekranie (telefon), a potem wykorzystaj dodatkową przestrzeń na większych. Unikaj sztywnych szerokości - to najczęstsza przyczyna „rozjeżdżania się” interfejsu między urządzeniami.

12.15. Kiedy używać którego layoutu

NAJWAŻNIEJSZE INFORMACJE

LAYOUT	KIEDY UŻYWAĆ
<code>VerticalStackLayout</code>	elementy jeden pod drugim (formularze)
<code>HorizontalStackLayout</code>	kilka elementów w rzędzie
<code>Grid</code>	układy dwuwymiarowe, proporcje, formularze
<code>FlexLayout</code>	zawijane, zmienne listy elementów
<code>AbsoluteLayout</code>	nakładki, pozycjonowanie absolutne
<code>ScrollView</code>	treść dłuższa niż ekran
<code>Border</code>	karty, obramowania

Na co uważać:

Nie ma jednego „najlepszego” layoutu - liczy się dobór do zadania. W praktyce 90% ekranów zbudujesz z `VerticalStackLayout`, `HorizontalStackLayout` i `Grid`, zagnieżdżając je w razie potrzeby i owijając w `ScrollView`.

12.16. Typowe błędy przy rozmieszczaniu elementów

NAJWAŻNIEJSZE INFORMACJE

BŁĄD	SKUTEK	ROZWIĄZANIE
Kilka dzieci w <code>ContentPage</code> / <code>ScrollView</code> / <code>Border</code>	błąd lub brak widoku	opakuj w jeden layout
Sztuczne rozmiary wszędzie	złe skalowanie	użyj <code>*</code> , <code>Fill</code>
Brak <code>ScrollView</code> przy długiej treści	część treści niewidoczna	owin w <code>ScrollView</code>
Lista w pionowym <code>ScrollView</code>	konflikt przewijania	usuń <code>ScrollView</code> wokół listy
Złe indeksy w <code>Grid</code>	element w złej komórce	sprawdź <code>Grid.Row</code> / <code>Grid.Column</code> (od 0)

Na co uważać:

Większość problemów z układem wynika z trzech przyczyn: zbyt wielu dzieci tam, gdzie dozwolone jest jedno; sztywnych rozmiarów psujących responsywność; oraz braku `ScrollView` przy dłuższej treści. Pilnując tych trzech rzeczy, unikniesz większości kłopotów.

Buduj ekran „od zewnątrz do środka”: najpierw wybierz layout główny (zwykle `VerticalStackLayout` lub `Grid` w `ScrollView`), potem dodawaj wewnątrz mniejsze layouty dla poszczególnych fragmentów. Taka warstwowa budowa jest czytelna i łatwa w modyfikacji.

12.17. Pełne tabele atrybutów layoutów

Poniżej komplet atrybutów każdego layoutu (oprócz wspólnych właściwości takich jak `Padding`, `Margin`, `BackgroundColor`, `HorizontalOptions`, `VerticalOptions`, `WidthRequest` / `HeightRequest`).

VerticalStackLayout / HorizontalStackLayout

ATRYBUT	TYP	OPIS
<code>Spacing</code>	<code>double</code>	odstęp między kolejnymi dziećmi

```
<VerticalStackLayout Spacing="12" Padding="20">
  <Label Text="Jeden pod drugim" />
  <Button Text="OK" />
</VerticalStackLayout>
```

Grid

ATRYBUT	TYP	OPIS
RowDefinitions	kolekcja	definicje wierszy (Auto , *, liczba)
ColumnDefinitions	kolekcja	definicje kolumn
RowSpacing	double	odstęp między wierszami
ColumnSpacing	double	odstęp między kolumnami
Grid.Row (dołączana)	int	wiersz dziecka (od 0)
Grid.Column (dołączana)	int	kolumna dziecka (od 0)
Grid.RowSpan	int	rozciągnięcie na wiersze
Grid.ColumnSpan	int	rozciągnięcie na kolumny

```
<Grid RowDefinitions="Auto,*,Auto" ColumnDefinitions="*,2*"
      RowSpacing="8" ColumnSpacing="8">
  <Label Text="Nagłówek" Grid.Row="0" Grid.ColumnSpan="2" />
  <BoxView Grid.Row="1" Grid.Column="0" Color="LightBlue" />
  <BoxView Grid.Row="1" Grid.Column="1" Color="LightGreen" />
  <Button Text="Dół" Grid.Row="2" Grid.ColumnSpan="2" />
</Grid>
```

FlexLayout

ATRYBUT	TYP	OPIS
Direction	enum	Row , Column , RowReverse , ColumnReverse
Wrap	enum	NoWrap , Wrap , WrapReverse
JustifyContent	enum	rozkład wzdłuż osi (Start , Center , SpaceBetween , SpaceAround , SpaceEvenly)
AlignItems	enum	wyrównanie w poprzek (Start , Center , End , Stretch)
AlignContent	enum	wyrównanie linii (przy zawijaniu)
FlexLayout.Grow (dołączana)	float	jak bardzo dziecko rośnie
FlexLayout.Basis (dołączana)	FlexBasis	bazowy rozmiar dziecka

```
<FlexLayout Direction="Row" Wrap="Wrap" JustifyContent="SpaceAround" AlignItems="Center">
  <Button Text="Tag 1" Margin="4" />
  <Button Text="Tag 2" Margin="4" />
  <Button Text="Tag 3" Margin="4" />
</FlexLayout>
```

AbsoluteLayout

ATRYBUT (DOŁĄCZANA)	TYP	OPIS
AbsoLuteLayout.LayoutBounds	Rect	X, Y, szerokość, wysokość
AbsoLuteLayout.LayoutFlags	enum	None , All , PositionProportional , SizeProportional

```
<AbsoluteLayout>
    <Image Source="tlo.png" AbsoluteLayout.LayoutBounds="0,0,1,1" AbsoluteLayout.LayoutFlags="All" />
    <Label Text="Nakładka" AbsoluteLayout.LayoutBounds="0.5,0.5,AutoSize,AutoSize"
        AbsoluteLayout.LayoutFlags="PositionProportional" />
</AbsoluteLayout>
```

ScrollView

ATRYBUT	TYP	OPIS
Orientation	enum	Vertical, Horizontal, Both, Neither
Content	View	jedyny element wewnątrz
HorizontalScrollBarVisibility	enum	widoczność poziomego paska
VerticalScrollBarVisibility	enum	widoczność pionowego paska

```
<ScrollView Orientation="Vertical">
    <VerticalStackLayout Padding="20" Spacing="12">
        <!-- długa treść -->
    </VerticalStackLayout>
</ScrollView>
```

```
// Programowe przewinięcie do elementu
await Widok.ScrollToAsync(0, 500, animated: true);
```

Border

ATRYBUT	TYP	OPIS
Stroke	Brush / Color	kolor obramowania
StrokeThickness	double	grubość
StrokeShape	Shape	kształt (RoundedRectangle, Ellipse)
StrokeDashArray	kolekcja	wzór linii przerywanej
StrokeLineCap / StrokeLineJoin	enum	zakończenia/tączenia linii
Content	View	jeden element wewnątrz

ContentView

ATRYBUT	TYP	OPIS
Content	View	zawartość komponentu
ControlTemplate	ControlTemplate	szablon kontrolki

ContentView to baza do tworzenia **własnych, wielokrotnie używalnych komponentów** (np. własna karta produktu jako osobna kontrolka).

Na co uważać: layouty przyjmujące **jeden** element to **ScrollView**, **Border**, **ContentView**, **ContentPage** - by pokazać wiele kontroltek, włoż layout. Właściwości dołączane (np. **Grid.Row**, **AbsoluteLayout.LayoutBounds**) ustawiamy na **dzieciach**, nie na

layoucie.

Ten rozdział łączy trzy powiązane tematy: **kolory** (jak je zapisywać i ustawiać), **suwaki** (jak odczytywać wartość i reagować na zmianę) oraz **dynamiczny wygląd** (jak zmieniać wygląd interfejsu w czasie działania). Na koniec pokazujemy **tryb jasny i ciemny** z `AppThemeBinding`. To praktyczna wiedza potrzebna do tworzenia interaktywnych, „żywych” interfejsów.

13. Design, kolory, style i animacje

13.1. Kolory w XAML - sposoby zapisu

Kolory w MAUI można zapisać na kilka sposobów: przez **nazwę** (`Red`), zapis **HEX** (`#RRGGBB`), **ARGB** (`#AARRGGBB` z kanałem przezroczystości) lub w C# przez `Color.FromRgb`. Kolory ustawiamy m.in. we właściwościach `BackgroundColor`, `TextColor`, `Stroke`.

NAJWAŻNIEJSZE INFORMACJE

FORMAT	PRZYKŁAD	OPIS
Nazwa	<code>Red</code> , <code>DarkBlue</code>	gotowe kolory z <code>Colors</code>
HEX	<code>#2196F3</code>	czerwony, zielony, niebieski (00–FF)
ARGB	<code>#802196F3</code>	alfa (przezroczystość) + kolor
RGB w C#	<code>Color.FromRgb(33,150,243)</code>	składowe 0–255
Z HEX w C#	<code>Color.FromArgb("#2196F3")</code>	parsowanie napisu

PRZYKŁAD XAML

```
<Label Text="Nazwa" TextColor="DarkBlue" />
<Label Text="HEX" TextColor="#E91E63" />
<BoxView Color="#802196F3" HeightRequest="30" /> 
```

Na co uważać:

Zapis HEX `#RRGGBB` to najpopularniejszy sposób precyzyjnego określenia koloru. Każda para znaków (00–FF) to składowa: czerwona, zielona, niebieska.

13.2. Kolory nazwane

MAUI udostępnia dziesiątki **gotowych kolorów** w klasie `Colors` (np. `Colors.Red`, `Colors.Green`). W XAML używamy ich po nazwie (`TextColor="Green"`), a w C# przez `Colors.Green`.

NAJWAŻNIEJSZE INFORMACJE

NAZWA	NAZWA	NAZWA
<code>Black</code>	<code>White</code>	<code>Gray</code>
<code>Red</code>	<code>Green</code>	<code>Blue</code>
<code>Yellow</code>	<code>Orange</code>	<code>Purple</code>
<code>LightGray</code>	<code>DarkGray</code>	<code>Transparent</code>

PRZYKŁAD C#

```
Tytuł.TextColor = Colors.DarkGreen;
Tło.BackgroundColor = Colors.LightYellow;
```

```
Ramka.BackgroundColor = Colors.Transparent; // przezroczyste
```

Na co uważać:

Kolory nazwane są wygodne i czytelne, ale ograniczone do predefiniowanego zestawu. Dla dokładnych odcieni marki użyj zapisu HEX.

13.3. Zapis HEX, RGB i ARGB - przezroczystość

HEX (`#RRGGBB`) opisuje kolor trzema składowymi w systemie szesnastkowym. ARGB (`#AARRGGBB`) dodaje na początku **kanał alfa** (przezroczystość): `00` = całkowicie przezroczysty, `FF` = pełne krycie. W C# kolor składamy z liczb 0–255 przez `Color.FromRgb` lub `Color.FromRgba`.

PRZYKŁAD C#

```
// Z trzech składowych 0-255
Color niebieski = Color.FromRgb(33, 150, 243);

// Z przezroczystością (ostatni parametr alfa 0-1)
Color półprzezroczysty = Color.FromRgba(0, 0, 0, 0.5); // czarny 50%

// Z napisu HEX/ARGB
Color zHex = Color.FromArgb("#FF5722");
Color zArgb = Color.FromArgb("#80000000"); // czarny 50%
```

Na co uważać:

Kanał alfa jest bardzo przydatny do **nakładek** i półprzezroczystych tła (np. przyciemnienie ekranu pod oknem dialogowym). W zapisie ARGB pierwsza para to alfa - `#80...` oznacza ok. 50% krycia.

13.4. Właściwości kolorów: BackgroundColor, TextColor, BorderColor, Color

Różne kontrolki mają różne właściwości kolorów. `BackgroundColor` - tło (większość kontrol). `TextColor` - kolor tekstu (kontrolki tekstowe). `Stroke` / `BorderColor` - kolor obramowania (`Border` / `Frame`). `Color` - wypełnienie (`BoxView`).

NAJWAŻNIEJSZE INFORMACJE

WŁAŚCIWOŚĆ	KONTROLKA	USTAWIA
<code>BackgroundColor</code>	większość	kolor tła
<code>TextColor</code>	<code>Label</code> , <code>Entry</code> , <code>Button</code>	kolor tekstu
<code>Stroke</code>	<code>Border</code>	kolor obramowania
<code>BorderColor</code>	<code>Frame</code>	kolor obramowania
<code>Color</code>	<code>BoxView</code>	wypełnienie

PRZYKŁAD XAML

```
<Label Text="Tekst" TextColor="White" BackgroundColor="#2196F3" />
<BoxView Color="Green" HeightRequest="20" />
<Border Stroke="LightGray" StrokeThickness="1"><Label Text="Karta" /></Border>
```

Na co uważać:

Pamiętaj, że `BoxView` używa `Color` (nie `BackgroundColor`), a `Border` - `Stroke` (nie `BorderColor`), które należy do starszego `Frame`). To częste pomyłki.

13.5. Dynamiczna zmiana koloru

Kolor kontrolki można zmieniać **w czasie działania** - np. zależnie od wartości lub stanu. Wystarczy przypisać nową wartość do właściwości koloru.

PRZYKŁAD C#

```
private void OcenWynik(int punkty)
{
    // Kolor zależny od wartości
    if (punkty ≥ 75) Wynik.TextColor = Colors.Green;
    else if (punkty ≥ 50) Wynik.TextColor = Colors.Orange;
    else Wynik.TextColor = Colors.Red;

    Wynik.Text = $"Wynik: {punkty}";
}
```

Na co uważać:

Dynamiczna zmiana koloru to świetny sposób na czytelną informację zwrotną (zielony = OK, czerwony = błąd). Zmiana właściwości koloru natychmiast przerysowuje kontrolkę.

13.6. Suwaki RGB - wzornik kolorów

Klasyczne ćwiczenie łączące suwaki i kolory: trzy suwaki (R, G, B) o zakresie 0–255, których wartości składają kolor pokazywany na podglądzie. To wzorcowy przykład „stan → wygląd”.

PRZYKŁAD XAML

```
<VerticalStackLayout Padding="20" Spacing="12">
    <BoxView x:Name="Podglad" HeightRequest="120" Color="Black" />
    <Label x:Name="EtykietaRgb" Text="RGB(0, 0, 0)" HorizontalOptions="Center" />

    <Label Text="Czerwony (R)" />
    <Slider x:Name="SuwakR" Minimum="0" Maximum="255" ValueChanged="OnKolor" />
    <Label Text="Zielony (G)" />
    <Slider x:Name="SuwakG" Minimum="0" Maximum="255" ValueChanged="OnKolor" />
    <Label Text="Niebieski (B)" />
    <Slider x:Name="SuwakB" Minimum="0" Maximum="255" ValueChanged="OnKolor" />
</VerticalStackLayout>
```

PRZYKŁAD C#

```
private void OnKolor(object sender, ValueChangedEventArgs e)
{
    int r = (int)SuwakR.Value;
    int g = (int)SuwakG.Value;
    int b = (int)SuwakB.Value;

    Podglad.Color = Color.FromRgb(r, g, b); // kolor z trzech liczb
    EtykietaRgb.Text = $"RGB({r}, {g}, {b})";
}
```

Na co uważać:

Każdy suwak zwraca `double` - rzutuj na `int`, bo składowe to liczby całkowite 0–255. `Color.FromRgb(r, g, b)` składa kolor. To kompletny, działający wzornik - jeden handler obsługuje wszystkie trzy suwaki.

13.7. Dynamiczna zmiana rozmiaru czcionki

Suwak może też sterować **rozmiarem czcionki** etykiety - przesunięcie natychmiast zmienia `FontSize`. To popularne ustawienie dostępności (większy/mniejszy tekst).

PRZYKŁAD C#

```
private void OnRozmiar(object sender, ValueChangedEventArgs e)
{
    int rozmiar = (int)e.NewValue;
    Podglad.FontSize = rozmiar;           // zmiana rozmiaru na żywo
    EtykietaWartosci.Text = $"Rozmiar: {rozmiar}";
}
```

Na co uważać:

`FontSize` przyjmuje `double`, ale dla czytelności zwykle rzutujemy na `int`. Ustaw sensowny zakres suwaka (np. 12–40), by tekst nie był zbyt mały ani zbyt duży.

13.8. Dynamiczna zmiana widoczności elementów

Wygląd zmieniamy też przez **pokazywanie i ukrywanie** elementów (`IsVisible`). Np. komunikat błędu pokazujemy tylko, gdy jest błąd; dodatkowe pola - po zaznaczeniu opcji.

PRZYKŁAD C#

```
private void OnOpcja(object sender, CheckedChangedEventArgs e)
{
    // Pokaż dodatkowe pole tylko, gdy opcja zaznaczona
    DodatkowePole.IsVisible = e.Value;
}
```

Na co uważać:

`IsVisible="False"` całkowicie ukrywa element (znika z układu). To różni się od `IsEnabled="False"`, które pokazuje element jako nieaktywny. Wybierz zależnie od intencji.

13.9. Tryb jasny i ciemny - AppThemeBinding

Systemy oferują **tryb jasny i ciemny**, a dobre aplikacje powinny je respektować. `AppThemeBinding` pozwala podać dwie wartości właściwości - dla trybu jasnego (`Light`) i ciemnego (`Dark`) - a MAUI sam wybierze odpowiednią i **automatycznie** ją zmieni, gdy użytkownik przełączy motyw systemu.

PRZYKŁAD XAML

```
<ContentPage BackgroundColor="{AppThemeBinding Light=White, Dark=#1E1E1E}">
    <Label Text="Tekst dopasowany do motywu"
           TextColor="{AppThemeBinding Light=Black, Dark=White}" />
</ContentPage>
```

PRZYKŁAD C#

```
// Odczyt i wymuszenie motywu w kodzie
AppTheme aktualny = Application.Current.RequestedTheme;
```

```
Application.Current.UserAppTheme = AppTheme.Dark; // wymuś ciemny
Application.Current.UserAppTheme = AppTheme.Unspecified; // jak w systemie
```

Na co uważać:

AppThemeBinding to najwygodniejszy sposób obsługi motywów - deklarujesz oba warianty w XAML, a zmiana motywu dzieje się automatycznie. Projektując kolory, zadбай, by tekst był czytelny w obu trybach (ciemny tekst na jasnym tle i odwrotnie).

13.10. Przykład: dynamiczny wygląd zależny od wartości

PRZYKŁAD C#

```
private void OnTemperatura(object sender, ValueChangedEventArgs e)
{
    int temp = (int)e.NewValue;
    Wartosc.Text = $"{temp}°C";

    // Kolor i komunikat zależne od wartości (stan → wygląd)
    if (temp < 0) { Tlo.BackgroundColor = Colors.LightBlue; Opis.Text = "Mróz"; }
    else if (temp < 20){ Tlo.BackgroundColor = Colors.LightGreen; Opis.Text = "Chłodno"; }
    else if (temp < 30){ Tlo.BackgroundColor = Colors.LightYellow;Opis.Text = "Ciepło"; }
    else { Tlo.BackgroundColor = Colors.OrangeRed; Opis.Text = "Gorąco"; }
}
```

Na co uważać:

To przykład „żywego” interfejsu: jedna wartość (z suwaka) steruje kilkoma elementami wyglądu naraz (tekst, kolor tła, opis). Trzymaj się schematu „stan → wygląd” - najpierw odczytaj wartość, potem ustaw wszystkie zależne właściwości.

13.11. Typowe błędy

NAJWAŻNIEJSZE INFORMACJE

BŁĄD	SKUTEK	ROZWIĄZANIE
<code>BoxView.BackgroundColor</code> zamiast <code>Color</code>	brak koloru	użyj <code>Color</code>
Brak rzutowania <code>Value</code> na <code>int</code>	ułamkowe wartości	<code>(int)e.NewValue</code>
Zły zakres suwaka	neczytelny efekt	dobierz <code>Minimum</code> / <code>Maximum</code>
Neczytelny tekst w trybie ciemnym	tekst niewidoczny	użyj <code>AppThemeBinding</code>
Składowe RGB spoza 0–255	błędny kolor	trzymaj zakres 0–255

Na co uważać:

Najczęstsze potknięcia to mylenie właściwości kolorów (`Color` vs `BackgroundColor`) oraz brak rzutowania wartości suwaka na `int`. Pamiętaj też o czytelności kolorów w obu trybach motywu.

Kolory i suwaki to podstawa „dynamicznego” interfejsu. Połączenie `Slider` + `ValueChanged` + zmiana właściwości (kolor, rozmiar, widoczność) daje natychmiastową, efektywną reakcję na działania użytkownika. Wzornik RGB to świetne ćwiczenie łączące te elementy.

13.12. Pełna lista nazwanych kolorów

MAUI udostępnia w klasie `Colors` dziesiątki gotowych, **nazwanych kolorów** (zgodnych ze standardem nazw kolorów z technologii webowych). Użyjesz ich w XAML po nazwie (`TextColor="Crimson"`) albo w C# przez `Colors.Crimson`. Poniżej kompletny, pogrupowany wykaz wraz z zapisem HEX - gdy chcesz dokładnie ten sam odcień bez polegania na nazwie, użyj wartości HEX.

Czerwienie i róże

NAZWA	HEX	NAZWA	HEX
Red	#FF0000	DarkRed	#8B0000
Crimson	#DC143C	FireBrick	#B22222
IndianRed	#CD5C5C	LightCoral	#F08080
Salmon	#FA8072	DarkSalmon	#E9967A
LightSalmon	#FFA07A	Tomato	#FF6347
Pink	#FFC0CB	LightPink	#FFB6C1
HotPink	#FF69B4	DeepPink	#FF1493
PaleVioletRed	#DB7093	MediumVioletRed	#C71585

Pomarańcze i żółcie

NAZWA	HEX	NAZWA	HEX
Orange	#FFA500	DarkOrange	#FF8C00
OrangeRed	#FF4500	Coral	#FF7F50
Gold	#FFD700	Yellow	#FFFF00
LightYellow	#FFFFE0	LemonChiffon	#FFFACD
Khaki	#F0E68C	DarkKhaki	#BDB76B
Moccasin	#FFE4B5	PeachPuff	#FFDAB9
PapayaWhip	#FFEFD5	Goldenrod	#DAA520

Zielenie

NAZWA	HEX	NAZWA	HEX
Green	#008000	DarkGreen	#006400
Lime	#00FF00	LimeGreen	#32CD32
LightGreen	#90EE90	PaleGreen	#98FB98
ForestGreen	#228B22	SeaGreen	#2E8B57
MediumSeaGreen	#3CB371	SpringGreen	#00FF7F
Olive	#808000	OliveDrab	#6B8E23
DarkOliveGreen	#556B2F	YellowGreen	#9ACD32
Teal	#008080	Chartreuse	#7FFF00

Błękity i niebieskie

NAZWA	HEX	NAZWA	HEX
Blue	#0000FF	DarkBlue	#00008B
Navy	#000080	MidnightBlue	#191970
RoyalBlue	#4169E1	MediumBlue	#0000CD
DodgerBlue	#1E90FF	CornflowerBlue	#6495ED
SteelBlue	#4682B4	LightSteelBlue	#B0C4DE
SkyBlue	#87CEEB	LightSkyBlue	#87CEFA
LightBlue	#ADD8E6	PowderBlue	#B0E0E6
Cyan / Aqua	#00FFFF	DarkCyan	#008B8B
Turquoise	#40E0D0	Aquamarine	#7FFFD4

Fiolety i purpury

NAZWA	HEX	NAZWA	HEX
Purple	#800080	Indigo	#4B0082
Violet	#EE82EE	Magenta / Fuchsia	#FF00FF
Orchid	#DA70D6	Plum	#DDA0DD
MediumPurple	#9370DB	BlueViolet	#8A2BE2
DarkViolet	#9400D3	DarkOrchid	#9932CC
Lavender	#E6E6FA	Thistle	#D8BFD8
SlateBlue	#6A5ACD	MediumSlateBlue	#7B68EE

Brązy

NAZWA	HEX	NAZWA	HEX
Brown	#A52A2A	Maroon	#800000
SaddleBrown	#8B4513	Sienna	#A0522D
Chocolate	#D2691E	Peru	#CD853F
SandyBrown	#F4A460	Tan	#D2B48C
RosyBrown	#BC8F8F	BurlyWood	#DEB887
Wheat	#F5DEB3	NavajoWhite	#FFDEAD

Biele, szarości i czerni

NAZWA	HEX	NAZWA	HEX
White	#FFFFFF	Black	#000000
Snow	#FFFAFA	Ivory	#FFFFF0
WhiteSmoke	#F5F5F5	Gainsboro	#DCDCDC
LightGray	#D3D3D3	Silver	#C0C0C0
DarkGray	#A9A9A9	Gray	#808080
DimGray	#696969	SlateGray	#708090
LightSlateGray	#778899	DarkSlateGray	#2F4F4F
Transparent	#00000000	Beige	#F5F5DC

Wszystkie powyższe nazwy działają zarówno w XAML (np. `BackgroundColor="SteelBlue"`), jak i w C# (`Colors.SteelBlue`).
`Transparent` to kolor w pełni przezroczysty - przydatny do „niewidocznego” tła. Gdy potrzebujesz odcienia spoza listy, użyj zapisu HEX.

13.13. Wszystkie sposoby ustawiania koloru w XAML i C#

Kolor można podać i ustawić na wiele sposobów. Poniżej komplet - od nazwy, przez HEX i ARGB, po tworzenie w C# ze składowych. Każdy sposób działa z dowolną właściwością koloru (`BackgroundColor`, `TextColor`, `Stroke`, `BoxView.Color`, `ProgressColor` itd.).

```

<!-- 1. Nazwa koloru -->
<Label Text="Nazwa" TextColor="Crimson" BackgroundColor="LightYellow" />

<!-- 2. HEX skrócony (#RGB) -->
<Label Text="HEX skrócony" TextColor="#F00" />      <!-- czerwony -->

<!-- 3. HEX pełny (#RRGGBB) -->
<Label Text="HEX pełny" TextColor="#2196F3" />

<!-- 4. HEX z przezroczystością (#AARRGGBB) -->
<BoxView Color="#802196F3" HeightRequest="30" />      <!-- ok. 50% krycia -->

<!-- 5. Kolor jako zasób (zdefiniowany w App.xaml / Resources) -->
<Button Text="Z zasobu" BackgroundColor="{StaticResource KolorPrimary}" />

<!-- 6. Kolor zależny od motywu (jasny/ciemny) -->
<ContentPage BackgroundColor="{AppThemeBinding Light=White, Dark=#1E1E1E}" />
```

```

// 1. Nazwany kolor z klasy Colors
Tytuł.TextColor = Colors.Crimson;
Tło.BackgroundColor = Colors.LightYellow;

// 2. Z zapisu HEX / ARGB (parsowanie napisu)
Ramka.Stroke = Color.FromArgb("#2196F3");
Naklejka.BackgroundColor = Color.FromArgb("#80000000"); // czarny 50%

// 3. Ze składowych RGB 0-255
Podgląd.Color = Color.FromRgb(33, 150, 243);
```

```
// 4. Ze składowych RGBA (z przezroczystością 0-1)
Naklejka.BackgroundColor = Color.FromRgba(0, 0, 0, 0.5);

// 5. Z modelu HSL/HSV (barwa, nasycenie, jasność)
Akcent.BackgroundColor = Color.FromHsla(0.6, 0.8, 0.5);

// 6. Modyfikacje istniejącego koloru
Color bazowy = Colors.Blue;
Color jasnieszy = bazowy.AddLuminosity(0.2f); // jaśniejszy odcień
Color polprzezroczysty = bazowy.WithAlpha(0.5f); // 50% krycia
```

SPOSÓB	XAML	C#
Nazwa	"Crimson"	Colors.Crimson
HEX	"#2196F3"	Color.FromArgb("#2196F3")
ARGB (alfa)	"#80RRGGBB"	Color.FromRgba(r, g, b, a)
RGB 0-255	-	Color.FromRgb(r, g, b)
HSL/HSV	-	Color.FromHsla(h, s, l)
Zasób	{StaticResource ...}	z Resources
Motyw	{AppThemeBinding ...}	RequestedTheme

Na co uważać: w HEX `#AARRGGBB` pierwsza para to **alfa** (przezroczystość): `FF` = pełne krycie, `00` = całkowicie przezroczysty, `80` ≈ 50%. `Color.FromRgb` przyjmuje składowe 0-255, a `Color.FromRgba` dodatkowo alfę 0-1. Metody jak `WithAlpha` i `AddLuminosity` zwracają nowy kolor - wynik trzeba przypisać.

13.14. Triggery - reakcja na zmianę właściwości

Trigger pozwala zmienić wygląd kontrolki **automatycznie**, gdy spełniony jest warunek - bez pisania kodu w code-behind. Najczęstszy to **Trigger** (reaguje na właściwość tej samej kontrolki) oraz **DataTrigger** (reaguje na wartość z bindingu). Definiujemy je w **Triggers** kontrolki lub w stylu.

```
<!-- Trigger: gdy Entry ma fokus, zmień kolor tła -->
<Entry Placeholder="Wpisz coś">
    <Entry.Triggers>
        <Trigger TargetType="Entry" Property="IsFocused" Value="True">
            <Setter Property="BackgroundColor" Value="LightYellow" />
        </Trigger>
    </Entry.Triggers>
</Entry>

<!-- DataTrigger: gdy przełącznik włączony, zmień tekst etykiety na zielony -->
<Label Text="Status">
    <Label.Triggers>
        <DataTrigger TargetType="Label"
            Binding="{Binding Source={x:Reference Przełącznik}, Path=IsToggled}"
            Value="True">
            <Setter Property="TextColor" Value="Green" />
        </DataTrigger>
    </Label.Triggers>
</Label>
```

```
</Label>

<Switch x:Name="Przełacznik" />
```

Na co uważać: triggery są deklaratywne - wygląd zmienia się sam, bez handlerów. **DataTrigger** z **x:Reference** to wygodny sposób powiązania wyglądu jednej kontrolki ze stanem innej. Istnieje też **EventTrigger** (reakcja na zdarzenie) i **MultiTrigger** (kilka warunków naraz).

13.15. VisualStateManager - stany wizualne

VisualStateManager (VSM) pozwala zdefiniować **wygląd dla różnych stanów** kontrolki: normalny, naciśnięty, wyłączony, zaznaczony. To standardowy sposób na reakcje wizualne na interakcję (np. przycisk ciemnieje przy naciśnięciu).

```
<Button Text="Naciśnij mnie">
  <VisualStateManager.VisualStateGroups>
    <VisualStateGroup x:Name="CommonStates">
      <VisualState x:Name="Normal">
        <VisualState.Setters>
          <Setter Property="BackgroundColor" Value="#2196F3" />
          <Setter Property="Scale" Value="1" />
        </VisualState.Setters>
      </VisualState>
      <VisualState x:Name="Pressed">
        <VisualState.Setters>
          <Setter Property="BackgroundColor" Value="#1565C0" />
          <Setter Property="Scale" Value="0.96" />
        </VisualState.Setters>
      </VisualState>
      <VisualState x:Name="Disabled">
        <VisualState.Setters>
          <Setter Property="BackgroundColor" Value="LightGray" />
        </VisualState.Setters>
      </VisualState>
    </VisualStateGroup>
  </VisualStateManager.VisualStateGroups>
</Button>
```

Na co uważać: VSM jest świetny do spójnych reakcji wizualnych. Najczęściej definiujemy go w **stylu** (raz dla wszystkich przycisków), a nie na każdej kontrolce osobno. Nazwy stanów (**Normal**, **Pressed**, **Disabled**, **Focused**, **Selected**) są ustalone.

13.16. Animacje

MAUI udostępnia gotowe **animacje** wywoływane z kodu jako metody asynchroniczne na kontrolce. Najważniejsze: **FadeTo** (przezroczystość), **TranslateTo** (przesunięcie), **ScaleTo** (skala), **RotateTo** (obróć). Można je łączyć (**await** po kolei lub równolegle przez **Task.WhenAll**).

METODA	ANIMUJE	PRZYKŁAD
<code>FadeTo(0..1, ms)</code>	przezroczystość	<code>await x.FadeTo(0, 500)</code>
<code>TranslateTo(x, y, ms)</code>	przesunięcie	<code>await x.TranslateTo(100, 0, 300)</code>
<code>ScaleTo(skała, ms)</code>	rozmiar	<code>await x.ScaleTo(1.2, 200)</code>
<code>RotateTo(stopnie, ms)</code>	obróć	<code>await x.RotateTo(360, 500)</code>

```
<Image x:Name="Logo" Source="logo.png" HeightRequest="120" />
<Button Text="Animuj" Clicked="OnAnimuj" />
```

```
private async void OnAnimuj(object sender, EventArgs e)
{
    await Logo.FadeTo(0, 250);           // znika
    await Logo.FadeTo(1, 250);           // pojawia się
    await Logo.ScaleTo(1.3, 200);         // powiększ
    await Logo.ScaleTo(1.0, 200);         // wróć
    await Logo.RotateTo(360, 500);         // pełny obrót
    Logo.Rotation = 0;                   // reset kąta

    // Animacje równoległe
    await Task.WhenAll(
        Logo.TranslateTo(50, 0, 300),
        Logo.FadeTo(0.5, 300));
    await Logo.TranslateTo(0, 0, 300);
}

// Animacja „wejścia” elementu w OnAppearing
protected override async void OnAppearing()
{
    base.OnAppearing();
    Logo.Opacity = 0;
    await Logo.FadeTo(1, 400);           // płynne pojawienie się
}
```

Na co uważać: animacje są asynchroniczne - `await` czeka na ich zakończenie (sekwencja), a `Task.WhenAll` uruchamia kilka naraz. Po `RotateTo(360)` ustaw `Rotation = 0`, by uniknąć narastania kąta. Używaj animacji oszczędnie - mają wzbogacać, nie rozpraszać.

13.17. ControlTemplate i DataTemplateSelector

`ControlTemplate` pozwala zdefiniować **wygląd (szablon)** całej kontrolki lub strony - np. wspólny układ z nagłówkiem dla wielu ekranów. `DataTemplateSelector` pozwala wybrać **różny szablon elementu** listy zależnie od danych (np. inny wygląd wiadomości wysłanej i odebranej).

```
// Selektor szablonów - inny wygląd zależnie od danych
public class WiadomoscSelector : DataTemplateSelector
{
    public DataTemplate Wyslana { get; set; }
    public DataTemplate Odebrana { get; set; }

    protected override DataTemplate OnSelectTemplate(object item, BindableObject container)
        => ((Wiadomosc)item).CzyMoja ? Wyslana : Odebrana;
}
```

```
<CollectionView ItemsSource="{Binding Wiadomosci}"
```

```
ItemTemplate="{StaticResource SelektorWiadomosci}" />
```

Na co uważać: **ControlTemplate** to temat bardziej zaawansowany - przydaje się przy wspólnym „szkieletcie” ekranów i własnych kontrolkach. **DataTemplateSelector** jest praktyczny w listach o niejednorodnych elementach (czaty, mieszane typy wpisów).

14. Kontrolki tekstowe

Kontrolki tekstowe to najczęściej używane elementy interfejsu - służą do **wyświetlania** tekstu (**Label**) oraz do jego **wprowadzania** (**Entry** , **Editor** , **SearchBar**). W tym rozdziale omawiamy każdą z nich według jednolitego schematu: czym jest, do czego służy, najważniejsze właściwości i zdarzenia (w tabelach), przykłady oraz typowe błędy.

14.1. Label

Label to kontrolka służąca **wyłącznie do wyświetlania tekstu** - to mobilny odpowiednik etykiety. Nie pozwala wpisywać tekstu; pokazuje go użytkownikowi. To przez **Label** aplikacja „mówi”: prezentuje wyniki, komunikaty, liczniki i podpisy pól.

Label służy do **prezentacji informacji**: tytułów, opisów, wyników obliczeń, komunikatów walidacji, wartości liczników. W kodzie najczęściej zmieniamy jego właściwość **Text** , aby pokazać aktualny stan aplikacji.

NAJWAŻNIEJSZE WŁAŚCIWOŚCI

WŁAŚCIWOŚĆ	OPIS	PRZYKŁAD
Text	wyświetlany tekst	Text ="Witaj"
TextColor	kolor tekstu	TextColor ="DarkBlue"
FontSize	rozmiar czcionki	FontSize ="22"
FontAttributes	styl: None / Bold / Italic	FontAttributes ="Bold"
HorizontalTextAlignment	wyrównanie tekstu w poziomie	Center
VerticalTextAlignment	wyrównanie w pionie	Center
LineBreakMode	sposób łamania długiego tekstu	WordWrap
FontFamily	rodzina czcionki	FontFamily ="OpenSansRegular"

NAJWAŻNIEJSZE ZDARZENIA

ZDARZENIE	KIEDY WYSTĘPUJE	PRZYKŁAD UŻYCIA
(brak typowych)	Label rzadko obsługuje zdarzenia	do kliknięć dodaj TapGestureRecognizer

PRZYKŁAD PODSTAWOWY

```
<Label Text="Wynik: 0"
      TextColor="DarkGreen"
      FontSize="22"
      FontAttributes="Bold"
      HorizontalTextAlignment="Center" />
```

PRZYKŁAD W C#

```
// Dynamiczna zmiana tekstu i koloru
Wynik.Text = $"Liczba punktów: {punkty}";
Wynik.TextColor = punkty >= 50 ? Colors.Green : Colors.Red;
```

TYPOWE ZASTOSOWANIA

- Tytuły i nagłówki.
- Komunikaty walidacji.
- Wyświetlanie liczników i wyników.

TYPOWE BŁĘDY

- Próba „pobrania danych” z `Label` (to kontrolka tylko do wyświetlania).
- Zapomnienie `$` przy interpolacji (`Text = "Wynik: " + x` zamiast `$"Wynik: {x}"`).

TEKST WIELOLINIOWY I FORMATTEDTEXT

`Label` potrafi wyświetlić tekst w wielu liniach (gdy zawiera znaki nowej linii lub gdy `LineBreakMode="WordWrap"`). Może też łączyć fragmenty o różnym stylu dzięki `FormattedText` i elementom `Span` :

```
<Label>
  <Label.FormattedText>
    <FormattedString>
      <Span Text="Cena: " FontAttributes="Bold" />
      <Span Text="49,99 zł" TextColor="Green" />
    </FormattedString>
  </Label.FormattedText>
</Label>
```

Do dynamicznego budowania tekstu używaj interpolacji: `Wynik.Text = $"Suma: {suma} zł";` . To czytelniejsze i mniej podatne na błędy niż sklejanie operatorem `+` .

14.2. Entry

`Entry` to **jednoliniowe pole tekstowe**, w które użytkownik **wpisuje dane**: login, e-mail, hasło, liczbę. To podstawowe narzędzie zbierania danych wejściowych w formularzach.

`Entry` służy do **pobierania krótkiego tekstu** od użytkownika. Wpisaną wartość odczytujemy z właściwości `Text` , a `Placeholder` pokazuje podpowiedź, gdy pole jest puste.

NAJWAŻNIEJSZE WŁAŚCIWOŚCI

WŁAŚCIWOŚĆ	OPIS	PRZYKŁAD
<code>Text</code>	realna wartość wpisana przez użytkownika	odczyt w C#
<code>Placeholder</code>	szara podpowiedź w pustym polu	<code>Placeholder="e-mail"</code>
<code>PlaceholderColor</code>	kolor podpowiedzi	<code>PlaceholderColor="Gray"</code>
<code>IsPassword</code>	ukrywa znaki (pole hasła)	<code>IsPassword="True"</code>
<code>Keyboard</code>	typ klawiatury	<code>Keyboard="Email"</code>
<code>MaxLength</code>	maksymalna liczba znaków	<code>MaxLength="20"</code>
<code>ClearButtonVisibility</code>	przycisk czyszczenia	<code>WhileEditing</code>
<code>IsReadOnly</code>	pole tylko do odczytu	<code>IsReadOnly="True"</code>

NAJWAŻNIEJSZE ZDARZENIA

ZDARZENIE	KIEDY WYSTĘPUJE	PRZYKŁAD UŻYCIA
TextChanged	przy każdej zmianie tekstu	walidacja na żywo
Completed	po naciśnięciu Enter/Gotowe	zatwierdzenie pola
Focused	gdy pole otrzyma fokus	podświetlenie
Unfocused	gdy pole straci fokus	walidacja po wyjściu

PRZYKŁAD PODSTAWOWY

```
<VerticalStackLayout Padding="20" Spacing="10">
    <Entry x:Name="PoleEmail" Placeholder="adres e-mail" Keyboard="Email" />
    <Entry x:Name="PoleHaslo" Placeholder="hasło" IsPassword="True" />
    <Button Text="Zaloguj" Clicked="OnZaloguj" />
</VerticalStackLayout>
```

PRZYKŁAD W C#

```
private void OnZaloguj(object sender, EventArgs e)
{
    string email = PoleEmail.Text;    // odczyt wpisanej wartości
    string haslo = PoleHaslo.Text;

    if (string.IsNullOrEmpty(email))
    {
        DisplayAlert("Uwaga", "Podaj e-mail", "OK");
        return;
    }

    // dalsza logika logowania...
}
```

TYPY Klawiatury (KEYBOARD)

WARTOŚĆ	ZASTOSOWANIE
Default	zwykły tekst
Numeric	liczby
Email	adresy e-mail
Telephone	numery telefonu
Url	adresy stron
Text	tekst z poprawkami

TYPOWE ZASTOSOWANIA

- Pola formularza (imię, e-mail, login).
- Pole hasła (IsPassword="True").
- Pole liczbowe (Keyboard="Numeric").

TYPOWE BŁĘDY

- Założenie, że Entry.Text nigdy nie jest null (dla pustego pola bywa null).
- Pobieranie liczby bez TryParse (wyjątek przy złym wpisie).

- Mylenie `Text` (realna wartość) z `Placeholder` (tylko podpowiedź).

Zanim porównasz lub przetworzysz `Entry.Text`, sprawdź `string.IsNullOrEmpty(PoleEmail.Text)`. Dla pustego pola `Text` może być `null`, a operacje na `null` powodują błąd.

14.3. Editor

`Editor` to pole do wprowadzania **dłuższego, wielowierszowego tekstu** - opisu, komentarza, notatki. Działa podobnie do `Entry`, ale automatycznie zawija tekst i pozwala wpisywać znaki nowej linii.

`Editor` służy do **pobierania dłuższych wypowiedzi**: treści notatki, opisu produktu, komentarza. Wybór między `Entry` a `Editor` jest prosty: krótki, jednoliniowy tekst → `Entry`; dłuższy → `Editor`.

NAJWAŻNIEJSZE WŁAŚCIWOŚCI

WŁAŚCIWOŚĆ	OPIS	PRZYKŁAD
<code>Text</code>	wpisana treść	odczyt w C#
<code>Placeholder</code>	podpowiedź w pustym polu	<code>Placeholder="Opis..."</code>
<code>AutoSize</code>	automatyczne rośnięcie	<code>TextChanges</code>
<code>MaxLength</code>	maksymalna długość	<code>MaxLength="500"</code>

NAJWAŻNIEJSZE ZDARZENIA

ZDARZENIE	KIEDY WYSTĘPUJE	PRZYKŁAD UŻYCIA
<code>TextChanged</code>	przy każdej zmianie tekstu	licznik znaków
<code>Completed</code>	po zakończeniu edycji	zapis treści

PRZYKŁAD PODSTAWOWY

```
<Editor x:Name="PoleOpis"
        Placeholder="Wpisz dłuższy opis..."
        AutoSize="TextChanges"
        HeightRequest="120" />
```

PRZYKŁAD W C#

```
private void OnZapisz(object sender, EventArgs e)
{
    string opis = PoleOpis.Text ?? "";
    LicznikZnakow.Text = $"Znaków: {opis.Length}";
}
```

TYPOWE ZASTOSOWANIA

- Notatki i komentarze.
- Opisy w formularzach.

TYPOWE BŁĘDY

- Użycie `Entry` tam, gdzie potrzebny jest tekst wieloliniowy.
- Brak ustawienia wysokości - pole może być za małe.

14.4. SearchBar

`SearchBar` to wyspecjalizowane pole do **wyszukiwania**. Wygląda jak `Entry`, ale ma wbudowaną ikonę lupy i dedykowane zdarzenia. Najczęściej służy do **filtrowania listy**.

SearchBar pozwala użytkownikowi szybko **znaleźć** element w długiej liście - filtrujemy dane na podstawie wpisanej frazy, na żywo lub po zatwierdzeniu.

NAJWAŻNIEJSZE WŁAŚCIWOŚCI

WŁAŚCIWOŚĆ	OPIS	PRZYKŁAD
Text	wpisana fraza	odczyt w C#
Placeholder	podpowiedź	Placeholder ="Szukaj..."
CancelButtonColor	kolor przycisku czyszczenia	CancelButtonColor ="Gray"

NAJWAŻNIEJSZE ZDARZENIA

ZDARZENIE	KIEDY WYSTĘPUJE	PRZYKŁAD UŻYCIA
TextChanged	przy każdej zmianie frazy	filtrowanie na żywo
SearchButtonPressed	po naciśnięciu przycisku szukaj	wyszukiwanie na żądanie

PRZYKŁAD PODSTAWOWY

```
<SearchBar x:Name="Wyszukiwarka" Placeholder="Szukaj owocu..."
           TextChanged="OnSzukaj" />
<CollectionView x:Name="Wyniki">
  <CollectionView.ItemTemplate>
    <DataTemplate>
      <Label Text="{Binding .}" Padding="8" />
    </DataTemplate>
  </CollectionView.ItemTemplate>
</CollectionView>
```

PRZYKŁAD W C#

```
readonly List<string> wszystkie = new()
{ "Jabłko", "Banan", "Cytryna", "Gruszka", "Malina" };

private void OnSzukaj(object sender, TextChangedEventArgs e)
{
    string fraza = (e.NewTextValue ?? "").ToLower();
    // Filtrowanie listy na podstawie frazy
    Wyniki.ItemsSource = wszystkie
        .Where(p => p.ToLower().Contains(fraza))
        .ToList();
}
```

TYPOWE ZASTOSOWANIA

- Wyszukiwanie w liście lokalnej.
- Wyszukiwanie danych z API.

TYPOWE BŁĘDY

- Porównywanie bez ujednolicenia wielkości liter (**ToLower**) - wyszukiwanie „rozróżnia” wielkość.
- Brak obsługi pustej frazy (powinna pokazać całą listę).

Do filtrowania „na żywo” używaj **TextChanged** oraz metod LINQ **Where** i **Contains**. Porównuj po **ToLower()**, aby wyszukiwanie ignorowało wielkość liter.

14.5. Label - pełna tabela atrybutów

Oprócz wspólnych właściwości `Label` ma własne atrybuty sterujące wyświetlaniem tekstu. Poniżej komplet.

ATRYBUT	TYP	OPIS
<code>Text</code>	<code>string</code>	wyświetlany tekst
<code>TextColor</code>	<code>Color</code>	kolor tekstu
<code>FontSize</code>	<code>double</code>	rozmiar czcionki
<code>FontFamily</code>	<code>string</code>	rodzina czcionki (alias z rejestracji)
<code>FontAttributes</code>	enum	<code>None</code> , <code>Bold</code> , <code>Italic</code>
<code>FontAutoScalingEnabled</code>	<code>bool</code>	skalowanie wg ustawień systemu
<code>HorizontalTextAlignment</code>	enum	<code>Start</code> , <code>Center</code> , <code>End</code>
<code>VerticalTextAlignment</code>	enum	wyrównanie pionowe tekstu
<code>LineBreakMode</code>	enum	<code>WordWrap</code> , <code>CharacterWrap</code> , <code>TruncationTail</code> , <code>NoWrap</code> ...
<code>MaxLines</code>	<code>int</code>	maksymalna liczba linii
<code>LineHeight</code>	<code>double</code>	wysokość linii (interlinia)
<code>CharacterSpacing</code>	<code>double</code>	odstęp między znakami
<code>TextDecorations</code>	enum	<code>Underline</code> , <code>Strikethrough</code> , <code>None</code>
<code>TextType</code>	enum	<code>Text</code> lub <code>Html</code>
<code>Padding</code>	<code>Thickness</code>	wewnętrzny odstęp tekstu
<code>FormattedText</code>	<code>FormattedString</code>	tekst złożony z fragmentów (<code>Span</code>)

```
<Label Text="Pełny przykład etykiety"
      TextColor="#333333"
      FontSize="20"
      FontAttributes="Bold,Italic"
      HorizontalTextAlignment="Center"
      LineBreakMode="WordWrap"
      MaxLines="2"
      CharacterSpacing="1"
      TextDecorations="Underline"
      Padding="8" />
```

```
Etykieta.Text = "Tekst z kodu";
Etykieta.TextColor = Colors.DarkSlateGray;
Etykieta.FontSize = 22;
Etykieta.FontAttributes = FontAttributes.Bold;
Etykieta.HorizontalTextAlignment = TextAlignment.Center;
Etykieta.LineBreakMode = LineBreakMode.TailTruncation;
Etykieta.MaxLines = 1;
Etykieta.TextDecorations = TextDecorations.Strikethrough;
```

Tekst złożony z fragmentów o różnym stylu budujemy przez `FormattedText` i `Span` :

```
<Label>
  <Label.FormattedText>
    <FormattedString>
      <Span Text="Cena: " FontAttributes="Bold" />
      <Span Text="49,99 zł " TextColor="Green" FontSize="20" />
      <Span Text="(promocja)" TextColor="Red" FontAttributes="Italic" />
    </FormattedString>
  </Label.FormattedText>
</Label>
```

14.6. Entry - pełna tabela atrybutów

Entry (jednoliniowe pole) ma bogaty zestaw atrybutów sterujących wpisywaniem.

ATRYBUT	TYP	OPIS
Text	string	wpisana wartość
Placeholder	string	podpowiedź w pustym polu
PlaceholderColor	Color	kolor podpowiedzi
TextColor	Color	kolor wpisywanego tekstu
IsPassword	bool	ukrywa znaki (pole hasła)
Keyboard	Keyboard	typ klawiatury (Default , Numeric , Email , Telephone , Url , Chat)
MaxLength	int	maksymalna liczba znaków
IsReadOnly	bool	tylko do odczytu
IsTextPredictionEnabled	bool	podpowiedzi tekstu
ReturnType	enum	etykieta klawisza Enter (Done , Next , Search , Send , Go)
ClearButtonVisibility	enum	przycisk czyszczenia (Never , WhileEditing)
HorizontalTextAlignment	enum	wyrównanie tekstu
VerticalTextAlignment	enum	wyrównanie pionowe
CursorPosition	int	pozycja kursora
SelectionLength	int	długość zaznaczenia
FontSize / FontAttributes / FontFamily	-	jak w Label
CharacterSpacing	double	odstęp między znakami

```
<Entry Placeholder="adres e-mail"
  PlaceholderColor="Gray"
  Keyboard="Email"
  MaxLength="100"
  ClearButtonVisibility="WhileEditing"
  ReturnType="Next"
  IsSpellCheckEnabled="False" />

<Entry Placeholder="hasło" IsPassword="True" MaxLength="32" />
```

```
<Entry Placeholder="kwota" Keyboard="Numeric" HorizontalTextAlignment="End" />
```

```
// Odczyt i ustawianie
string email = PoleEmail.Text;
PoleEmail.Text = ""; // wyczyść
PoleEmail.Placeholder = "wpisz e-mail";
PoleEmail.IsReadOnly = true; // zablokuj edycję
PoleEmail.CursorPosition = 0;
```

Zdarzenia **Entry**: **TextChanged** (każda zmiana), **Completed** (Enter), **Focused** / **Unfocused**.

```
private void OnTekstZmieniony(object sender, TextChangedEventArgs e)
{
    string nowy = e.NewTextValue; // aktualny tekst
    string stary = e.OldTextValue; // poprzedni
    LicznikZnakow.Text = $"{nowy?.Length ?? 0}/100";
}
```

14.7. Editor - pełna tabela atrybutów i porównanie z Entry

Editor to pole wielowierszowe; ma większość atrybutów **Entry** oraz kilka własnych.

ATRYBUT	TYP	OPIS
Text	string	wpisana treść
Placeholder	string	podpowiedź
AutoSize	enum	Disabled lub TextChanges (rośnie z tekstem)
MaxLength	int	maksymalna długość
IsReadOnly	bool	tylko do odczytu
Keyboard	Keyboard	typ klawiatury
FontSize / FontAttributes	-	jak w Label

```
<Editor Placeholder="Wpisz dłuższy opis..."
        AutoSize="TextChanges"
        MaxLength="500"
        HeightRequest="120" />
```

CECHA	ENTRY	EDITOR
Liczba linii	jedna	wiele
Zawijanie tekstu	nie	tak
IsPassword	tak	nie
AutoSize	nie	tak
Typowe użycie	login, e-mail, liczba	opis, komentarz, notatka

Na co uważać: **Editor** nie ma **IsPassword** (do haseł zawsze **Entry**). Ustaw **AutoSize="TextChanges"**, by pole rośło wraz z wpisywanym tekstem. Dla obu kontrolki pamiętaj o sprawdzaniu **null** /pustości przy odczycie **Text**.

Ten rozdział omawia kontrolki, którymi użytkownik **wykonuje akcje** (**Button**, **ImageButton**) oraz **dokonuje wyboru** (**CheckBox**, **RadioButton**, **Switch**). To one zamieniają statyczny ekran w interaktywną aplikację. Każdą kontrolkę opisujemy według schematu z właściwościami, zdarzeniami i przykładami.

14.8. Receptury kontrolki tekstowych

Poniżej znajdziesz praktyczne przykłady najważniejszych kontrolki tekstowych i przycisków w .NET MAUI. Każdy przykład zawiera kompletny kod XAML oraz odpowiadający mu kod C# - gotowy do skopiowania i uruchomienia.

14.9. Label - receptury praktyczne

PRZYKŁAD 1: LABEL Z RÓŻNYMI STYLAMI TEKSTU

Label to podstawowa kontrolka do wyświetlania tekstu. Można ustawiać czcionkę, kolor, rozmiar i styl.

```

<!-- XAML: Różne style Label -->
<VerticalStackLayout Padding="20" Spacing="10">
    <Label Text="Tekst pogrubiony"
           FontAttributes="Bold"
           FontSize="18"
           TextColor="DarkBlue" />

    <Label Text="Tekst kursywą"
           FontAttributes="Italic"
           FontSize="16"
           TextColor="Green" />

    <Label Text="Tekst pogrubiony i kursywa"
           FontAttributes="Bold,Italic"
           FontSize="20"
           TextColor="Red" />

    <Label Text="Tekst z podkreśleniem"
           FontSize="14"
           TextDecorations="Underline"
           TextColor="Purple" />

    <Label Text="Tekst przekreślony"
           FontSize="14"
           TextDecorations="Strikethrough"

```

```
        TextColor="Gray" />
</VerticalStackLayout>
```

```
// C#: Różne style Label
using Microsoft.Maui.Controls;

public partial class LabelStylePage : ContentPage
{
    public LabelStylePage()
    {
        var layout = new VerticalStackLayout { Padding = 20, Spacing = 10 };

        layout.Children.Add(new Label
        {
            Text = "Tekst pogrubiony",
            FontAttributes = FontAttributes.Bold,
            FontSize = 18,
            TextColor = Colors.DarkBlue
        });

        layout.Children.Add(new Label
        {
            Text = "Tekst kursywą",
            FontAttributes = FontAttributes.Italic,
            FontSize = 16,
            TextColor = Colors.Green
        });

        layout.Children.Add(new Label
        {
            Text = "Tekst pogrubiony i kursywa",
            FontAttributes = FontAttributes.Bold | FontAttributes.Italic,
            FontSize = 20,
            TextColor = Colors.Red
        });

        layout.Children.Add(new Label
        {
            Text = "Tekst z podkreśleniem",
            FontSize = 14,
            TextDecorations = TextDecorations.Underline,
            TextColor = Colors.Purple
        });

        layout.Children.Add(new Label
        {
            Text = "Tekst przekreślony",
            FontSize = 14,
            TextDecorations = TextDecorations.Strikethrough,
            TextColor = Colors.Gray
        });
    }
}
```



```
        Content = layout;
    }
}
```

PRZYKŁAD 2: LABEL Z FORMATTEDTEXT I SPAN

FormattedText pozwala na mieszanie różnych stylów w jednym Label - każdy Span może mieć inny kolor, rozmiar i styl.

←!— XAML: FormattedText z wieloma Span →

```
<Label>
    <Label.FormattedText>
        <FormattedString>
            <Span Text="Witaj " FontSize="16" TextColor="Black" />
            <Span Text="w świecie " FontSize="16" FontAttributes="Bold" TextColor="Blue" />
            <Span Text=".NET MAUI!" FontSize="20" FontAttributes="Italic" TextColor="OrangeRed" />
        </FormattedString>
    </Label.FormattedText>
</Label>
```

```
// C#: FormattedText z wieloma Span
var label = new Label();

var formattedString = new FormattedString();

formattedString.Spans.Add(new Span
{
    Text = "Witaj ",
    FontSize = 16,
    TextColor = Colors.Black
});

formattedString.Spans.Add(new Span
{
    Text = "w świecie ",
    FontSize = 16,
    FontAttributes = FontAttributes.Bold,
    TextColor = Colors.Blue
});

formattedString.Spans.Add(new Span
{
    Text = ".NET MAUI!",
    FontSize = 20,
    FontAttributes = FontAttributes.Italic,
    TextColor = Colors.OrangeRed
});

label.FormattedText = formattedString;
```

PRZYKŁAD 3: LABEL - WYRÓWNIANIA TEKSTU

Właściwości `HorizontalTextAlignment` i `VerticalTextAlignment` kontrolują położenie tekstu wewnątrz `Label`.

```
←!— XAML: Wyrównania tekstu w Label →  
<VerticalStackLayout Padding="20" Spacing="15">  
    <Label Text="Wyrównanie do lewej"  
        HorizontalTextAlignment="Start"  
        BackgroundColor="LightGray"  
        HeightRequest="50"  
        VerticalTextAlignment="Center" />  
  
    <Label Text="Wyrównanie do środka"  
        HorizontalTextAlignment="Center"  
        BackgroundColor="LightYellow"  
        HeightRequest="50"  
        VerticalTextAlignment="Center" />  
  
    <Label Text="Wyrównanie do prawej"  
        HorizontalTextAlignment="End"  
        BackgroundColor="LightCyan"  
        HeightRequest="50"  
        VerticalTextAlignment="Center" />  
  
    <Label Text="Tekst u góry"  
        VerticalTextAlignment="Start"  
        HorizontalTextAlignment="Center"  
        BackgroundColor="LightPink"  
        HeightRequest="80" />  
  
    <Label Text="Tekst na dole"  
        VerticalTextAlignment="End"  
        HorizontalTextAlignment="Center"  
        BackgroundColor="LightGreen"  
        HeightRequest="80" />  
</VerticalStackLayout>
```

```
// C#: Wyrównania tekstu w Label  
var layout = new VerticalStackLayout { Padding = 20, Spacing = 15 };  
  
layout.Children.Add(new Label  
{  
    Text = "Wyrównanie do lewej",  
    HorizontalTextAlignment = TextAlignment.Start,  
    VerticalTextAlignment = TextAlignment.Center,  
    BackgroundColor = Colors.LightGray,  
    HeightRequest = 50  
});  
  
layout.Children.Add(new Label  
{  
    Text = "Wyrównanie do środka",  
    HorizontalTextAlignment = TextAlignment.Center,
```

```

        VerticalTextAlignment = TextAlignment.Center,
        BackgroundColor = Colors.LightYellow,
        HeightRequest = 50
    });

    layout.Children.Add(new Label
    {
        Text = "Wyrównanie do prawej",
        HorizontalTextAlignment = TextAlignment.End,
        VerticalTextAlignment = TextAlignment.Center,
        BackgroundColor = Colors.LightCyan,
        HeightRequest = 50
    });

    Content = layout;

```

PRZYKŁAD 4: LABEL WIELOLINIOWY Z LINEBREAKMODE

Właściwość `MaxLines` ogranicza liczbę widocznych linii, a `LineBreakMode` decyduje o sposobie łamania i obcinania tekstu.

```

<!-- XAML: Label wieloliniowy -->
<VerticalStackLayout Padding="20" Spacing="10">
    <Label Text="To jest bardzo długi tekst, który powinien się zawinąć do wielu linii automatycznie, gdy nie zmieści się w jednej
linii na ekranie."
        MaxLines="3"
        LineBreakMode="WordWrap"
        FontSize="14" />

    <Label Text="Ten tekst zostanie obcięty na końcu z wielokropkiem jeśli jest za długi."
        MaxLines="1"
        LineBreakMode="TailTruncation"
        FontSize="14" />

    <Label Text="Linia pierwsza&#10;Linia druga&#10;Linia trzecia"
        FontSize="14"
        LineBreakMode="WordWrap" />
</VerticalStackLayout>

```

```

// C#: Label wieloliniowy
var layout = new VerticalStackLayout { Padding = 20, Spacing = 10 };

layout.Children.Add(new Label
{
    Text = "To jest bardzo długi tekst, który powinien się zawinąć do wielu linii automatycznie, gdy nie zmieści się w jednej linii
na ekranie.",
    MaxLines = 3,
    LineBreakMode = LineBreakMode.WordWrap,
    FontSize = 14
});

```

```

layout.Children.Add(new Label
{
    Text = "Ten tekst zostanie obcięty na końcu z wielokropkiem jeśli jest za długi.",
    MaxLines = 1,
    LineBreakMode = LineBreakMode.TailTruncation,
    FontSize = 14
});

// Tekst wieloliniowy z jawnym znakiem nowej linii
layout.Children.Add(new Label
{
    Text = "Linia pierwsza\nLinia druga\nLinia trzecia",
    FontSize = 14,
    LineBreakMode = LineBreakMode.WordWrap
});

Content = layout;

```

14.10. Entry - receptury praktyczne

PRZYKŁAD 5: ENTRY - RÓŻNE TYPY KŁAWIATUR (KEYBOARD)

Entry to pole tekstowe jednoliniowe. Właściwość Keyboard określa typ klawiatury wyświetlanej na urządzeniu mobilnym.

```

<!-- XAML: Entry z różnymi klawiaturami -->
<VerticalStackLayout Padding="20" Spacing="10">
    <Entry Placeholder="Domyślna klawiatura"
        Keyboard="Default" />

    <Entry Placeholder="Adres e-mail"
        Keyboard="Email" />

    <Entry Placeholder="Numer telefonu"
        Keyboard="Telephone" />

    <Entry Placeholder="Adres URL"
        Keyboard="Url" />

    <Entry Placeholder="Tylko cyfry"
        Keyboard="Numeric" />

    <Entry Placeholder="Czat (emotikony)"
        Keyboard="Chat" />

    <Entry Placeholder="Tekst zwykły"
        Keyboard="Plain" />
</VerticalStackLayout>

```

```

// C#: Entry z różnymi klawiaturami
var layout = new VerticalStackLayout { Padding = 20, Spacing = 10 };

```

```

layout.Children.Add(new Entry { Placeholder = "Domyślna klawiatura", Keyboard = Keyboard.Default });
layout.Children.Add(new Entry { Placeholder = "Adres e-mail", Keyboard = Keyboard.Email });
layout.Children.Add(new Entry { Placeholder = "Numer telefonu", Keyboard = Keyboard.Telephone });
layout.Children.Add(new Entry { Placeholder = "Adres URL", Keyboard = Keyboard.Url });
layout.Children.Add(new Entry { Placeholder = "Tylko cyfry", Keyboard = Keyboard.Numeric });
layout.Children.Add(new Entry { Placeholder = "Czat (emotikony)", Keyboard = Keyboard.Chat });
layout.Children.Add(new Entry { Placeholder = "Tekst zwykły", Keyboard = Keyboard.Plain });

```

```
Content = layout;
```

PRZYKŁAD 6: ENTRY - HASŁO (ISPASSWORD)

Ustawienie `IsPassword=true` ukrywa wpisywane znaki (wyświetla kropki).

```

<!-- XAML: Entry jako pole hasła -->
<VerticalStackLayout Padding="20" Spacing="10">
    <Entry Placeholder="Wpisz hasło"
           IsPassword="True"
           MaxLength="20" />

    <Entry Placeholder="Potwierdź hasło"
           IsPassword="True"
           MaxLength="20" />
</VerticalStackLayout>

```

```

// C#: Entry jako pole hasła
var layout = new VerticalStackLayout { Padding = 20, Spacing = 10 };

var passwordEntry = new Entry
{
    Placeholder = "Wpisz hasło",
    IsPassword = true,
    MaxLength = 20
};

var confirmEntry = new Entry
{
    Placeholder = "Potwierdź hasło",
    IsPassword = true,
    MaxLength = 20
};

layout.Children.Add(passwordEntry);
layout.Children.Add(confirmEntry);
Content = layout;

```

PRZYKŁAD 7: ENTRY - WALIDACJA NA ŻYWO (TEXTCHANGED)

Zdarzenie `TextChanged` pozwala reagować na każdą zmianę tekstu - idealnie nadaje się do walidacji w czasie rzeczywistym.

```

<!-- XAML: Entry z walidacją na żywo -->
<VerticalStackLayout Padding="20" Spacing="10">
    <Entry x:Name="EmailEntry"
        Placeholder="Wpisz adres e-mail"
        Keyboard="Email"
        TextChanged="OnEmailTextChanged" />

    <Label x:Name="ValidationLabel"
        Text=""
        TextColor="Red"
        FontSize="12" />
</VerticalStackLayout>

```

```

// C#: Entry z walidacją na żywo (code-behind)
public partial class ValidationPage : ContentPage
{
    public ValidationPage()
    {
        InitializeComponent();

        private void OnEmailTextChanged(object sender, TextChangedEventArgs e)
        {
            string email = e.NewTextValue;

            if (string.IsNullOrEmpty(email))
            {
                ValidationLabel.Text = "";
                EmailEntry.BackgroundColor = Colors.White;
            }
            else if (email.Contains("@") && email.Contains("."))
            {
                ValidationLabel.Text = "✓ Poprawny format";
                ValidationLabel.TextColor = Colors.Green;
                EmailEntry.BackgroundColor = Colors.LightGreen;
            }
            else
            {
                ValidationLabel.Text = "✗ Niepoprawny format e-mail";
                ValidationLabel.TextColor = Colors.Red;
                EmailEntry.BackgroundColor = Colors.LightPink;
            }
        }
    }
}

```

PRZYKŁAD 8: ENTRY - MAXLENGTH I CLEARBUTTONVISIBILITY

MaxLength ogranicza liczbę znaków, a ClearButtonVisibility dodaje przycisk „X” do czyszczenia pola.

←!— XAML: Entry z MaxLength i przyciskiem czyszczenia →

```
<VerticalStackLayout Padding="20" Spacing="10">
    <Entry Placeholder="Kod pocztowy (max 6 znaków)"
        MaxLength="6"
        Keyboard="Numeric"
        ClearButtonVisibility="WhileEditing" />

    <Entry Placeholder="Imię (max 30 znaków)"
        MaxLength="30"
        ClearButtonVisibility="WhileEditing" />

    <Entry Placeholder="Identyfikator klienta (11 cyfr)"
        MaxLength="11"
        Keyboard="Numeric"
        ClearButtonVisibility="WhileEditing" />
</VerticalStackLayout>
```

```
// C#: Entry z MaxLength i przyciskiem czyszczenia
var layout = new VerticalStackLayout { Padding = 20, Spacing = 10 };

layout.Children.Add(new Entry
{
    Placeholder = "Kod pocztowy (max 6 znaków)",
    MaxLength = 6,
    Keyboard = Keyboard.Numeric,
    ClearButtonVisibility = ClearButtonVisibility.WhileEditing
});

layout.Children.Add(new Entry
{
    Placeholder = "Imię (max 30 znaków)",
    MaxLength = 30,
    ClearButtonVisibility = ClearButtonVisibility.WhileEditing
});

layout.Children.Add(new Entry
{
    Placeholder = "Identyfikator klienta (11 cyfr)",
    MaxLength = 11,
    Keyboard = Keyboard.Numeric,
    ClearButtonVisibility = ClearButtonVisibility.WhileEditing
});

Content = layout;
```

PRZYKŁAD 9: ENTRY - ZDARZENIE COMPLETED

Zdarzenie Completed jest wywoływane po naciśnięciu klawisza Enter/Return. Przydatne do zatwierdzania formularza.

```

<!-- XAML: Entry z Completed -->
<VerticalStackLayout Padding="20" Spacing="10">
    <Entry x:Name="NameEntry"
        Placeholder="Wpisz imię i naciśnij Enter"
        Completed="OnEntryCompleted"
        ReturnType="Done" />

    <Label x:Name="GreetingLabel"
        Text=""
        FontSize="18"
        TextColor="DarkGreen" />
</VerticalStackLayout>

```

```

// C#: Entry z Completed (code-behind)
public partial class CompletedPage : ContentPage
{
    public CompletedPage()
    {
        InitializeComponent();

        private void OnEntryCompleted(object sender, EventArgs e)
        {
            var entry = (Entry)sender;
            GreetingLabel.Text = $"Cześć, {entry.Text}! 🍕";
        }
    }
}

```

PRZYKŁAD 10: ENTRY - RETURNTYPE (RÓŻNE IKONY KŁAWISZA RETURN)

Właściwość `ReturnType` zmienia wygląd klawisza Return na klawiaturze.

```

<!-- XAML: Entry z różnymi ReturnType -->
<VerticalStackLayout Padding="20" Spacing="10">
    <Entry Placeholder="Done" ReturnType="Done" />
    <Entry Placeholder="Go" ReturnType="Go" />
    <Entry Placeholder="Next" ReturnType="Next" />
    <Entry Placeholder="Search" ReturnType="Search" />
    <Entry Placeholder="Send" ReturnType="Send" />
</VerticalStackLayout>

```

```

// C#: Entry z różnymi ReturnType
var layout = new VerticalStackLayout { Padding = 20, Spacing = 10 };

layout.Children.Add(new Entry { Placeholder = "Done", ReturnType = ReturnType.Done });
layout.Children.Add(new Entry { Placeholder = "Go", ReturnType = ReturnType.Go });
layout.Children.Add(new Entry { Placeholder = "Next", ReturnType = ReturnType.Next });

```



```
layout.Children.Add(new Entry { Placeholder = "Search", ReturnType = ReturnType.Search });
layout.Children.Add(new Entry { Placeholder = "Send", ReturnType = ReturnType.Send });

Content = layout;
```

14.11. Editor - receptury praktyczne

PRZYKŁAD 11: EDITOR - WIELOLINIOWE POLE TEKSTOWE

Editor to odpowiednik TextArea - wieloliniowe pole tekstowe do dłuższych wpisów. Obsługuje Placeholder, MaxLength i automatyczne dopasowanie wysokości.

```
<!-- XAML: Editor wieloliniowy -->
<VerticalStackLayout Padding="20" Spacing="10">
    <Editor Placeholder="Wpisz tutaj swoją wiadomość..."
            HeightRequest="150"
            MaxLength="500"
            FontSize="14"
            AutoSize="TextChanges" />

    <Label x:Name="CharCountLabel"
           Text="0/500 znaków"
           FontSize="12"
           TextColor="Gray"
           HorizontalTextAlignment="End" />
</VerticalStackLayout>
```

```
// C#: Editor wieloliniowy z licznikiem znaków
public partial class EditorPage : ContentPage
{
    private Editor _editor;
    private Label _charCountLabel;

    public EditorPage()
    {
        var layout = new VerticalStackLayout { Padding = 20, Spacing = 10 };

        _editor = new Editor
        {
            Placeholder = "Wpisz tutaj swoją wiadomość...",
            HeightRequest = 150,
            MaxLength = 500,
            FontSize = 14,
            AutoSize = EditorAutoSizeOption.TextChanges
        };
        _editor.TextChanged += OnEditorTextChanged;

        _charCountLabel = new Label
        {
            Text = "0/500 znaków",
            FontSize = 12,
```

```

        TextColor = Colors.Gray,
        HorizontalTextAlignment = TextAlignment.End
    };

    layout.Children.Add(_editor);
    layout.Children.Add(_charCountLabel);
    Content = layout;
}

private void OnEditorTextChanged(object sender, TextChangedEventArgs e)
{
    int count = e.NewTextValue?.Length ?? 0;
    _charCountLabel.Text = $"{count}/500 znaków";
    _charCountLabel.TextColor = count > 450 ? Colors.Red : Colors.Gray;
}
}

```

PRZYKŁAD 12: EDITOR - ZDARZENIA FOCUSED I UNFOCUSED

Zdarzenia Focused/Unfocused pozwalają reagować na moment kliknięcia w pole i opuszczenia go.

```

<!-- XAML: Editor z Focused/Unfocused -->
<VerticalStackLayout Padding="20" Spacing="10">
    <Editor x:Name="NotesEditor"
        Placeholder="Kliknij, żeby pisać..."
        HeightRequest="120"
        Focused="OnEditorFocused"
        Unfocused="OnEditorUnfocused" />

    <Label x:Name="StatusLabel"
        Text="Nieaktywny"
        FontSize="12"
        TextColor="Gray" />
</VerticalStackLayout>

```

```

// C#: Editor – obsługa Focused i Unfocused
public partial class EditorEventsPage : ContentPage
{
    public EditorEventsPage()
    {
        InitializeComponent();
    }

    private void OnEditorFocused(object sender, FocusEventArgs e)
    {
        StatusLabel.Text = "✎ Edytujesz...";
        StatusLabel.TextColor = Colors.Blue;
        NotesEditor.BackgroundColor = Colors.LightYellow;
    }
}

```

```

private void OnEditorUnfocused(object sender, FocusEventArgs e)
{
    StatusLabel.Text = "Nieaktywny";
    StatusLabel.TextColor = Colors.Gray;
    NotesEditor.BackgroundColor = Colors.White;
}
}

```

14.12. SearchBar - receptury praktyczne

PRZYKŁAD 13: SEARCHBAR - FILTROWANIE LISTY

SearchBar wyświetla pole wyszukiwania z przyciskiem. Poniżej filtrujemy listę owoców w czasie rzeczywistym.

```

<!-- XAML: SearchBar z ListView -->
<VerticalStackLayout Padding="20" Spacing="10">
    <SearchBar x:Name="FruitSearchBar"
        Placeholder="Szukaj owocu..."
        TextChanged="OnSearchTextChanged"
        SearchButtonPressed="OnSearchButtonPressed" />

    <ListView x:Name="FruitListView">
        <ListView.ItemTemplate>
            <DataTemplate>
                <TextCell Text="{Binding}" />
            </DataTemplate>
        </ListView.ItemTemplate>
    </ListView>
</VerticalStackLayout>

```

```

// C#: SearchBar - filtrowanie listy (code-behind)
public partial class SearchPage : ContentPage
{
    private List<string> _allFruits = new()
    {
        "Jabłko", "Gruszka", "Banan", "Pomarańcza", "Winogrono",
        "Truskawka", "Malina", "Borówka", "Ananas", "Mango",
        "Kiwi", "Arbuz", "Śliwka", "Wiśnia", "Brzoskwinia"
    };

    public SearchPage()
    {
        InitializeComponent();
        FruitListView.ItemsSource = _allFruits;
    }

    private void OnSearchTextChanged(object sender, TextChangedEventArgs e)
    {
        string filter = e.NewTextValue?.ToLower() ?? "";
        FruitListView.ItemsSource = _allFruits
            .Where(f => f.ToLower().Contains(filter))

```

```

        .ToList();
    }

    private void OnSearchButtonPressed(object sender, EventArgs e)
    {
        // Opcjonalnie: dodatkowa akcja po naciśnięciu lupy
        string query = FruitSearchBar.Text;
        DisplayAlert("Szukam", $"Szukasz: {query}", "OK");
    }
}

```

PRZYKŁAD 14: SEARCHBAR - FILTROWANIE Z COLLECTIONVIEW

Nowoczesne podejście z CollectionView i ObservableCollection.

```

<!-- XAML: SearchBar + CollectionView -->
<VerticalStackLayout Padding="20">
    <SearchBar x:Name="CitySearch"
        Placeholder="Wpisz nazwę miasta..."
        TextChanged="OnCitySearchChanged" />

    <CollectionView x:Name="CityCollectionView">
        <CollectionView.ItemTemplate>
            <DataTemplate>
                <Label Text="{Binding}" Padding="10" FontSize="16" />
            </DataTemplate>
        </CollectionView.ItemTemplate>
    </CollectionView>
</VerticalStackLayout>

```

```

// C#: SearchBar + CollectionView
public partial class CitySearchPage : ContentPage
{
    private List<string> _cities = new()
    {
        "Warszawa", "Kraków", "Gdańsk", "Wrocław", "Poznań",
        "Łódź", "Katowice", "Szczecin", "Lublin", "Bydgoszcz"
    };

    public CitySearchPage()
    {
        InitializeComponent();
        CityCollectionView.ItemsSource = _cities;
    }

    private void OnCitySearchChanged(object sender, TextChangedEventArgs e)
    {
        string query = e.NewTextValue?.ToLower() ?? "";

        if (string.IsNullOrEmpty(query))

```

```

    {
        CityCollectionView.ItemsSource = _cities;
    }
    else
    {
        CityCollectionView.ItemsSource = _cities
            .Where(c => c.ToLower().Contains(query))
            .ToList();
    }
}
}

```

14.13. Dodatkowe receptury kontrolek

PRZYKŁAD 31: ENTRY Z IKONĄ (WEWNĄTRZ FRAME)

Symulacja Entry z ikoną - opakowujemy Entry w ramkę z obrazkiem.

```

<!-- XAML: Entry z ikoną w ramce -->
<Frame Padding="5" CornerRadius="10" BorderColor="LightGray" HasShadow="False">
    <HorizontalStackLayout Spacing="10">
        <Image Source="search_icon.png" WidthRequest="20" HeightRequest="20"
            VerticalOptions="Center" />
        <Entry Placeholder="Szukaj..."
            HorizontalOptions="FillAndExpand"
            FontSize="14" />
    </HorizontalStackLayout>
</Frame>

```

```

// C#: Entry z ikoną w ramce
var frame = new Frame
{
    Padding = 5,
    CornerRadius = 10,
    BorderColor = Colors.LightGray,
    HasShadow = false
};

var row = new HorizontalStackLayout { Spacing = 10 };
row.Children.Add(new Image
{
    Source = "search_icon.png",
    WidthRequest = 20,
    HeightRequest = 20,
    VerticalOptions = LayoutOptions.Center
});
row.Children.Add(new Entry
{
    Placeholder = "Szukaj...",
    HorizontalOptions = LayoutOptions.FillAndExpand,
    FontSize = 14
}

```

```
});

frame.Content = row;
```

PRZYKŁAD 33: LABEL Z GESTAMI (TAPGESTURERECOGNIZER NA SPAN)

Span wewnątrz Label może reagować na dotknięcie - przydatne do tworzenia klikalnych linków.

```
←!— XAML: Klikalny Span w Label →

<Label FontSize="14">
    <Label.FormattedText>
        <FormattedString>
            <Span Text="Rejestrując się, akceptujesz " />
            <Span Text="regulamin"
                  TextColor="Blue"
                  TextDecorations="Underline">
                <Span.GestureRecognizers>
                    <TapGestureRecognizer Tapped="OnTermsTapped" />
                </Span.GestureRecognizers>
            </Span>
            <Span Text=" oraz " />
            <Span Text="politykę prywatności"
                  TextColor="Blue"
                  TextDecorations="Underline">
                <Span.GestureRecognizers>
                    <TapGestureRecognizer Tapped="OnPrivacyTapped" />
                </Span.GestureRecognizers>
            </Span>
            <Span Text="." />
        </FormattedString>
    </Label.FormattedText>
</Label>
```

```
// C#: Klikalne Span w Label (code-behind)
public partial class LinkLabelPage : ContentPage
{
    public LinkLabelPage()
    {
        InitializeComponent();
    }

    private async void OnTermsTapped(object sender, EventArgs e)
    {
        await DisplayAlert("Regulamin", "Tu byłby tekst regulaminu...", "OK");
    }

    private async void OnPrivacyTapped(object sender, EventArgs e)
    {
        await DisplayAlert("Prywatność", "Tu byłaby polityka prywatności...", "OK");
    }
}
```

```
}  
  
}
```

PRZYKŁAD 34: ENTRY - WALIDACJA NUMERYCZNA NA ŻYWO

Kontrola, czy użytkownik wpisuje poprawną liczbę w zakresie.

```
<!-- XAML: Entry z walidacją numeryczną -->  
<VerticalStackLayout Padding="20" Spacing="10">  
    <Label Text="Podaj wiek (1-120):" FontSize="14" />  
    <Entry x:Name="AgeEntry"  
        Placeholder="Wiek"  
        Keyboard="Numeric"  
        MaxLength="3"  
        TextChanged="OnAgeTextChanged" />  
    <Label x:Name="AgeValidationLabel" Text="" FontSize="12" />  
</VerticalStackLayout>
```

```
// C#: Walidacja numeryczna  
public partial class NumericValidationPage : ContentPage  
{  
    public NumericValidationPage()  
    {  
        InitializeComponent();  
    }  
  
    private void OnAgeTextChanged(object sender, TextChangedEventArgs e)  
    {  
        if (string.IsNullOrEmpty(e.NewTextValue))  
        {  
            AgeValidationLabel.Text = "";  
            return;  
        }  
  
        if (int.TryParse(e.NewTextValue, out int age))  
        {  
            if (age ≥ 1 && age ≤ 120)  
            {  
                AgeValidationLabel.Text = "✓ Poprawny wiek";  
                AgeValidationLabel.TextColor = Colors.Green;  
            }  
            else  
            {  
                AgeValidationLabel.Text = "x Wiek musi być w zakresie 1-120";  
                AgeValidationLabel.TextColor = Colors.Red;  
            }  
        }  
        else  
        {  
            AgeValidationLabel.Text = "x Wpisz liczbę";  
        }  
    }  
}
```

```

        AgeValidationLabel.TextColor = Colors.Red;
    }
}
}

```

PRZYKŁAD 35: KOMPLETNY FORMULARZ REJESTRACJI

Kompleksowy przykład łączący wiele kontroltek: Entry, CheckBox, RadioButton, Button.

```

<!-- XAML: Formularz rejestracji -->
<ScrollView>
<VerticalStackLayout Padding="20" Spacing="12">
    <Label Text="Rejestracja" FontSize="24" FontAttributes="Bold"
        HorizontalTextAlignment="Center" />

    <Entry x:Name="RegNameEntry" Placeholder="Imię i nazwisko"
        ClearButtonVisibility="WhileEditing" />

    <Entry x:Name="RegEmailEntry" Placeholder="Adres e-mail"
        Keyboard="Email" ClearButtonVisibility="WhileEditing" />

    <Entry x:Name="RegPasswordEntry" Placeholder="Hasło (min. 8 znaków)"
        IsPassword="True" MaxLength="30" />

    <Entry x:Name="RegPasswordConfirm" Placeholder="Powtórz hasło"
        IsPassword="True" MaxLength="30" />

    <Label Text="Typ konta:" FontAttributes="Bold" Margin="0,10,0,0" />
    <RadioButton Content="Osobiste" GroupName="AccountType" IsChecked="True" />
    <RadioButton Content="Firmowe" GroupName="AccountType" />

    <HorizontalStackLayout Spacing="8" Margin="0,10,0,0">
        <CheckBox x:Name="RegTermsCheck" Color="DodgerBlue" />
        <Label Text="Akceptuję regulamin klasy pomocniczej"
            VerticalTextAlignment="Center" FontSize="13" />
    </HorizontalStackLayout>

    <HorizontalStackLayout Spacing="8">
        <CheckBox x:Name="RegNewsletterCheck" Color="Green" />
        <Label Text="Chcę otrzymywać newsletter"
            VerticalTextAlignment="Center" FontSize="13" />
    </HorizontalStackLayout>

    <Button x:Name="RegSubmitButton"
        Text="Zarejestruj się"
        BackgroundColor="DodgerBlue"
        TextColor="White"
        CornerRadius="10"
        FontSize="16"
        Margin="0,15,0,0"
        Clicked="OnRegisterClicked" />

```



```
</VerticalStackLayout>

</ScrollView>
```

```
// C#: Formularz rejestracji (code-behind)
public partial class RegistrationPage : ContentPage
{
    public RegistrationPage()
    {
        InitializeComponent();
    }

    private async void OnRegisterClicked(object sender, EventArgs e)
    {
        // Walidacja
        if (string.IsNullOrEmpty(RegNameEntry.Text))
        {
            await DisplayAlert("Błąd", "Wpisz imię i nazwisko", "OK");
            return;
        }

        if (string.IsNullOrEmpty(RegEmailEntry.Text) ||
            !RegEmailEntry.Text.Contains("@"))
        {
            await DisplayAlert("Błąd", "Wpisz poprawny adres e-mail", "OK");
            return;
        }

        if (string.IsNullOrEmpty(RegPasswordEntry.Text) ||
            RegPasswordEntry.Text.Length < 8)
        {
            await DisplayAlert("Błąd", "Hasło musi mieć co najmniej 8 znaków", "OK");
            return;
        }

        if (RegPasswordEntry.Text != RegPasswordConfirm.Text)
        {
            await DisplayAlert("Błąd", "Hasła nie są takie same", "OK");
            return;
        }

        if (!RegTermsCheck.IsChecked)
        {
            await DisplayAlert("Błąd", "Musisz zaakceptować regulamin", "OK");
            return;
        }

        await DisplayAlert("Sukces", $"Konto utworzone dla: {RegNameEntry.Text}", "OK");
    }
}
```

PRZYKŁAD 36: EDITOR Z PLACEHOLDER I STYLIZACJĄ

```
←!— XAML: Editor stylizowany →

<VerticalStackLayout Padding="20" Spacing="10">

    <Label Text="Twoja opinia:" FontSize="16" FontAttributes="Bold" />

    <Frame BorderColor="LightGray" CornerRadius="8" Padding="5" HasShadow="False">

        <Editor Placeholder="Napisz co myślisz o naszej aplikacji..."

            PlaceholderColor="Silver"

            HeightRequest="120"

            FontSize="14"

            AutoSize="TextChanges"

            MaxLength="1000" />

    </Frame>

    <Button Text="Wyślij opinię"

        BackgroundColor="MediumSeaGreen"

        TextColor="White"

        CornerRadius="8" />

</VerticalStackLayout>
```

```
// C#: Editor stylizowany w ramce

var layout = new VerticalStackLayout { Padding = 20, Spacing = 10 };

layout.Children.Add(new Label
{
    Text = "Twoja opinia:",
    FontSize = 16,
    FontAttributes = FontAttributes.Bold
});

var editor = new Editor
{
    Placeholder = "Napisz co myślisz o naszej aplikacji...",
    PlaceholderColor = Colors.Silver,
    HeightRequest = 120,
    FontSize = 14,
    AutoSize = EditorAutoSizeOption.TextChanges,
    MaxLength = 1000
};

var frame = new Frame
{
    BorderColor = Colors.LightGray,
    CornerRadius = 8,
    Padding = 5,
    HasShadow = false,
    Content = editor
};

layout.Children.Add(frame);
layout.Children.Add(new Button
{
    Text = "Wyślij opinię",
```

```
        BackgroundColor = Colors.MediumSeaGreen,
        TextColor = Colors.White,
        CornerRadius = 8
    });

    Content = layout;
```

PRZYKŁAD 37: LABEL - BINDING DO WIELU WŁAŚCIWOŚCI (STRINGFORMAT)

```
←!— XAML: Label z StringFormat —→

<VerticalStackLayout Padding="20" Spacing="10">
    <Entry x:Name="FirstNameEntry" Placeholder="Imię" />
    <Entry x:Name="LastNameEntry" Placeholder="Nazwisko" />

    <Label x:Name="FullNameLabel"
        FontSize="18"
        TextColor="DarkSlateGray" />
</VerticalStackLayout>
```

```
// C#: Dynamiczne łączenie tekstu z wielu Entry
public partial class StringFormatPage : ContentPage
{
    public StringFormatPage()
    {
        InitializeComponent();
        FirstNameEntry.TextChanged += OnNameChanged;
        LastNameEntry.TextChanged += OnNameChanged;
    }

    private void OnNameChanged(object sender, TextChangedEventArgs e)
    {
        string first = FirstNameEntry.Text ?? "";
        string last = LastNameEntry.Text ?? "";
        FullNameLabel.Text = $"Witaj, {first} {last}!".Trim();
    }
}
```

PRZYKŁAD 38: BUTTON - ANIMACJA PO KLIKNIĘCIU

Przycisk z prostą animacją skalowania po kliknięciu - daje użytkownikowi wizualny feedback.

```
←!— XAML: Button z animacją —→

<VerticalStackLayout Padding="20" Spacing="20" VerticalOptions="Center">
    <Button x:Name="AnimatedButton"
        Text="Naciśnij mnie! 🍷"
        BackgroundColor="OrangeRed"
        TextColor="White"
        FontSize="18"
```

```

        CornerRadius="15"
        Clicked="OnAnimatedButtonClicked" />
</VerticalStackLayout>

```

```

// C#: Button z animacją skalowania
public partial class AnimatedButtonPage : ContentPage
{
    public AnimatedButtonPage()
    {
        InitializeComponent();

        private async void OnAnimatedButtonClicked(object sender, EventArgs e)
        {
            // Animacja: zmniejsz, potem przywróć rozmiar
            await AnimatedButton.ScaleTo(0.9, 100);
            await AnimatedButton.ScaleTo(1.0, 100);

            AnimatedButton.Text = "Kliknięto! ✓";
            await Task.Delay(1000);
            AnimatedButton.Text = "Naciśnij mnie! 🍷 ";
        }
    }
}

```

PRZYKŁAD 40: SEARCHBAR - FILTROWANIE Z WYRÓŻNIENIEM WYNIKU

```

<!-- XAML: SearchBar z liczbą wyników -->
<VerticalStackLayout Padding="20" Spacing="10">
    <SearchBar x:Name="ProductSearch"
        Placeholder="Szukaj produktu..."
        TextChanged="OnProductSearchChanged" />

    <Label x:Name="ResultCountLabel"
        Text="Produktów: 0"
        FontSize="12"
        TextColor="Gray" />

    <CollectionView x:Name="ProductsView">
        <CollectionView.ItemTemplate>
            <DataTemplate>
                <Frame Margin="0,3" Padding="10" CornerRadius="5" BorderColor="LightGray">
                    <Label Text="{Binding}" FontSize="14" />
                </Frame>
            </DataTemplate>
        </CollectionView.ItemTemplate>
    </CollectionView>
</VerticalStackLayout>

```

```
// C#: SearchBar z wyświetlaniem liczby wyników
public partial class ProductSearchPage : ContentPage
{
    private List<string> _products = new()
    {
        "Laptop ASUS", "Laptop Dell", "Laptop HP", "Monitor LG 27\"",
        "Monitor Samsung 24\"", "Klawiatura mechaniczna", "Klawiatura membranowa",
        "Myszka bezprzewodowa", "Myszka gamingowa", "Słuchawki Bluetooth",
        "Słuchawki nauszne", "Pendrive 64GB", "Dysk SSD 1TB"
    };

    public ProductSearchPage()
    {
        InitializeComponent();
        UpdateList(_products);
    }

    private void OnProductSearchChanged(object sender, TextChangedEventArgs e)
    {
        string query = e.NewTextValue?.ToLower() ?? "";
        var filtered = string.IsNullOrEmpty(query)
            ? _products
            : _products.Where(p => p.ToLower().Contains(query)).ToList();

        UpdateList(filtered);
    }

    private void UpdateList(List<string> items)
    {
        ProductsView.ItemsSource = items;
        ResultCountLabel.Text = $"Produktów: {items.Count}";
    }
}
```

15. Przyciski i akcje użytkownika

15.1. Button

Button służy do **uruchamiania akcji**: zatwierdzenia formularza, dodania elementu, obliczenia wyniku, nawigacji. Reakcję na naciśnięcie piszemy w metodzie obsługi zdarzenia.

NAJWAŻNIEJSZE WŁAŚCIWOŚCI

WŁAŚCIWOŚĆ	OPIS	PRZYKŁAD
Text	napis na przycisku	Text="Zapisz"
IsEnabled	czy przycisk jest aktywny	IsEnabled="False"
BackgroundColor	kolor tła	BackgroundColor="#2196F3"
TextColor	kolor napisu	TextColor="White"
CornerRadius	zaokrąglenie rogów	CornerRadius="8"
FontSize	rozmiar czcionki	FontSize="18"
ImageSource	ikona na przycisku	ImageSource="ikona.png"

NAJWAŻNIEJSZE ZDARZENIA

ZDARZENIE	KIEDY WYSTĘPUJE	PRZYKŁAD UŻYCIA
Pressed	w momencie wciśnięcia	efekt wizualny
Released	po puszczeniu	efekt wizualny

PRZYKŁAD PODSTAWOWY

```
<Button x:Name="PrzyciskZapisz"
        Text="ZAPISZ"
        BackgroundColor="#2196F3"
        TextColor="White"
        CornerRadius="8"
        Clicked="OnZapisz" />
```

PRZYKŁAD W C#

```
private void OnZapisz(object sender, EventArgs e)
{
    Komunikat.Text = "Dane zapisane!";
}

// Jeden handler dla wielu przycisków - rozpoznanie po sender
private void OnDowolny(object sender, EventArgs e)
{
    var przycisk = (Button)sender;
    Komunikat.Text = $"Kliknięto: {przycisk.Text}";
}
```

TYPOWE ZASTOSOWANIA

- Zatwierdzanie formularza.
- Dodawanie/usuwanie elementów.
- Przyciski  /  w liczniku.

TYPOWE BŁĘDY

- Umieszczenie wielu odpowiedzialności w jednym przycisku.

Jeden handler może obsłużyć kilka przycisków - rzutuj `sender` na `Button` i sprawdź np. jego `Text` lub `ClassId`, aby rozpoznać, który

przycisk wysłał zdarzenie.

15.2. ImageButton

`ImageButton` stosujemy, gdy naturalnym elementem akcji jest **ikona**: przycisk zamykania (krzyżyk), usuwania (kosz), edycji (ołówki), ulubionych (serce).

NAJWAŻNIEJSZE WŁAŚCIWOŚCI

WŁAŚCIWOŚĆ	OPIS	PRZYKŁAD
<code>Source</code>	obraz przycisku	<code>Source="kosz.png"</code>
<code>Aspect</code>	dopasowanie obrazu	<code>AspectFit</code>
<code>IsEnabled</code>	aktywność	<code>IsEnabled="True"</code>

NAJWAŻNIEJSZE ZDARZENIA

ZDARZENIE	KIEDY WYSTĘPUJE	PRZYKŁAD UŻYCIA
-----------	-----------------	-----------------

PRZYKŁAD PODSTAWOWY

```
<ImageButton Source="kosz.png" HeightRequest="40" WidthRequest="40"
    Clicked="OnUsun" />
```

PRZYKŁAD W C#

```
private async void OnUsun(object sender, EventArgs e)
{
    bool ok = await DisplayAlert("Usuwanie", "Usunąć element?", "Tak", "Nie");
    if (ok) { /* usuń element */ }
}
```

TYPOWE ZASTOSOWANIA

- Ikony akcji na paskach i w listach.

TYPOWE BŁĘDY

- Użycie zwykłego `Image` + `TapGestureRecognizer` tam, gdzie `ImageButton` byłby czytelniejszy.

15.3. Button - pełna tabela atrybutów

ATRYBUT	TYP	OPIS
Text	string	napis na przycisku
TextColor	Color	kolor napisu
FontSize / FontAttributes / FontFamily	-	czcionka napisu
BackgroundColor	Color	kolor tła
BorderColor	Color	kolor obramowania
BorderWidth	double	grubość obramowania
CornerRadius	int	zaokrąglenie rogów
Padding	Thickness	wewnętrzny odstęp
ImageSource	ImageSource	ikona na przycisku
ContentLayout	ButtonContentLayout	układ ikony względem tekstu
CharacterSpacing	double	odstęp między znakami
LineBreakMode	enum	łamanie długiego napisu
IsEnabled	bool	czy aktywny

```
<Button Text="ZAPISZ"
        TextColor="White"
        BackgroundColor="#2196F3"
        BorderColor="#1565C0"
        BorderWidth="1"
        CornerRadius="10"
        FontAttributes="Bold"
        FontSize="16"
        Padding="20,12"
        ImageSource="zapisz.png"
        ContentLayout="Left, 8"
        Clicked="OnZapisz" />
```

```
PrzyciskZapisz.Text = "Zapisano";
PrzyciskZapisz.BackgroundColor = Colors.Green;
PrzyciskZapisz.IsEnabled = false;      // zablokuj
PrzyciskZapisz.CornerRadius = 8;
```


15.4. ImageButton - pełna tabela atrybutów

ATRYBUT	TYP	OPIS
Source	ImageSource	obraz przycisku
Aspect	enum	dopasowanie obrazu (AspectFit , AspectFill , Fill)
BackgroundColor	Color	tło
BorderColor / BorderWidth	-	obramowanie
CornerRadius	int	zaokrąglenie
Padding	Thickness	wewnętrzny odstęp
IsEnabled	bool	czy aktywny

```
<ImageButton Source="kosz.png" Aspect="AspectFit"
              HeightRequest="44" WidthRequest="44"
              BackgroundColor="Transparent" Clicked="OnUsun" />
```

15.5. Button - receptury praktyczne

```
←!— XAML: Prosty Button →
<VerticalStackLayout Padding="20" Spacing="10">
    <Button Text="Kliknij mnie!"
            Clicked="OnButtonClicked"
            BackgroundColor="DodgerBlue"
            TextColor="White"
            FontSize="16"
            CornerRadius="10" />

    <Label x:Name="ClickCountLabel"
           Text="Liczba kliknięć: 0"
           FontSize="14" />
</VerticalStackLayout>
```

```
// C#: Prosty Button (code-behind)
public partial class ButtonPage : ContentPage
{
    private int _clickCount = 0;

    public ButtonPage()
    {
        InitializeComponent();
    }

    private void OnButtonClicked(object sender, EventArgs e)
    {
        _clickCount++;
        ClickCountLabel.Text = $"Liczba kliknięć: {_clickCount}";
    }
}
```

```
}  
  
}
```

PRZYKŁAD 16: BUTTON - RÓŻNE STYLE (CORNERRADIUS, BORDERWIDTH, KOLORY)

Przycisk można dowolnie stylizować zmieniając tło, obramowanie i zaokrąglenie narożników.

←!— XAML: Przyciski z różnymi stylami →

```
<VerticalStackLayout Padding="20" Spacing="15">  
    <Button Text="Zaokrąglony"  
        BackgroundColor="MediumPurple"  
        TextColor="White"  
        CornerRadius="25"  
        HeightRequest="50" />  
  
    <Button Text="Z obramowaniem"  
        BackgroundColor="Transparent"  
        TextColor="DarkOrange"  
        BorderColor="DarkOrange"  
        BorderWidth="2"  
        CornerRadius="8" />  
  
    <Button Text="Cień i duży"  
        BackgroundColor="ForestGreen"  
        TextColor="White"  
        FontSize="20"  
        FontAttributes="Bold"  
        CornerRadius="12"  
        Padding="20,10"  
        Shadow="{Shadow Brush=Black, Offset='3,3', Radius=5, Opacity=0.3}" />  
</VerticalStackLayout>
```

```
// C#: Przyciski z różnymi stylami  
var layout = new VerticalStackLayout { Padding = 20, Spacing = 15 };  
  
layout.Children.Add(new Button  
{  
    Text = "Zaokrąglony",  
    BackgroundColor = Colors.MediumPurple,  
    TextColor = Colors.White,  
    CornerRadius = 25,  
    HeightRequest = 50  
});  
  
layout.Children.Add(new Button  
{  
    Text = "Z obramowaniem",  
    BackgroundColor = Colors.Transparent,  
    TextColor = Colors.DarkOrange,  
    BorderColor = Colors.DarkOrange,
```

```

        BorderWidth = 2,
        CornerRadius = 8
    });

    layout.Children.Add(new Button
    {
        Text = "Duży i zielony",
        BackgroundColor = Colors.ForestGreen,
        TextColor = Colors.White,
        FontSize = 20,
        FontAttributes = FontAttributes.Bold,
        CornerRadius = 12,
        Padding = new Thickness(20, 10)
    });

    Content = layout;

```

PRZYKŁAD 17: BUTTON Z IKONĄ (IMAGESOURCE)

Przycisk może zawierać ikonę obok tekstu dzięki właściwości `ImageSource`.

```

<!-- XAML: Button z ikoną -->
<VerticalStackLayout Padding="20" Spacing="10">
    <Button Text="Zapisz"
        ImageSource="save_icon.png"
        ContentLayout="Left,10"
        BackgroundColor="SteelBlue"
        TextColor="White"
        CornerRadius="8" />

    <Button Text="Usuń"
        ImageSource="delete_icon.png"
        ContentLayout="Left,10"
        BackgroundColor="Crimson"
        TextColor="White"
        CornerRadius="8" />

    <Button Text="Pobierz"
        ImageSource="download_icon.png"
        ContentLayout="Top,5"
        BackgroundColor="Teal"
        TextColor="White"
        CornerRadius="8" />
</VerticalStackLayout>

```

```

// C#: Button z ikoną
var layout = new VerticalStackLayout { Padding = 20, Spacing = 10 };

layout.Children.Add(new Button
{

```

```

        Text = "Zapisz",
        ImageSource = "save_icon.png",
        ContentLayout = new Button.ButtonContentLayout(Button.ButtonContentLayout.ImagePosition.Left, 10),
        BackgroundColor = Colors.SteelBlue,
        TextColor = Colors.White,
        CornerRadius = 8
    });

    layout.Children.Add(new Button
    {
        Text = "Usuń",
        ImageSource = "delete_icon.png",
        ContentLayout = new Button.ButtonContentLayout(Button.ButtonContentLayout.ImagePosition.Left, 10),
        BackgroundColor = Colors.Crimson,
        TextColor = Colors.White,
        CornerRadius = 8
    });

    Content = layout;

```

PRZYKŁAD 19: BUTTON - ISENABLED (AKTYWNY/NIEAKTYWNY)

IsEnabled pozwala dynamicznie włączać i wyłączać przycisk, np. dopóki formularz nie jest wypełniony.

```

<!-- XAML: Button z dynamicznym IsEnabled -->
<VerticalStackLayout Padding="20" Spacing="10">
    <Entry x:Name="UsernameEntry"
        Placeholder="Wpisz login"
        TextChanged="OnFormChanged" />

    <Entry x:Name="PasswordEntry"
        Placeholder="Wpisz hasło"
        IsPassword="True"
        TextChanged="OnFormChanged" />

    <Button x:Name="LoginButton"
        Text="Zaloguj się"
        IsEnabled="False"
        BackgroundColor="DodgerBlue"
        TextColor="White"
        Clicked="OnLoginClicked" />
</VerticalStackLayout>

```

```

// C#: Button z dynamicznym IsEnabled (code-behind)
public partial class LoginPage : ContentPage
{
    public LoginPage()
    {
        InitializeComponent();
    }
}

```

```

private void OnFormChanged(object sender, TextChangedEventArgs e)
{
    // Przycisk aktywny tylko gdy oba pola są wypełnione
    bool isValid = !string.IsNullOrEmpty(UsernameEntry.Text)
        && !string.IsNullOrEmpty>PasswordEntry.Text);
    LoginButton.IsEnabled = isValid;
}

private async void OnLoginClicked(object sender, EventArgs e)
{
    await DisplayAlert("Sukces", $"Zalogowano jako: {UsernameEntry.Text}", "OK");
}
}

```

←!— XAML: Wiele przycisków, jeden handler —→

```

<VerticalStackLayout Padding="20" Spacing="10">
    <Button Text="Czerwony" StyleId="Red" Clicked="OnColorButtonClicked" />
    <Button Text="Zielony" StyleId="Green" Clicked="OnColorButtonClicked" />
    <Button Text="Niebieski" StyleId="Blue" Clicked="OnColorButtonClicked" />
    <Button Text="Żółty" StyleId="Yellow" Clicked="OnColorButtonClicked" />

    <BoxView x:Name="ColorBox" HeightRequest="100" Color="LightGray" CornerRadius="10" />
</VerticalStackLayout>

```

```
// C#: Jeden handler dla wielu przycisków
public partial class MultiButtonPage : ContentPage
{
    public MultiButtonPage()
    {
        InitializeComponent();
    }

    private void OnColorButtonClicked(object sender, EventArgs e)
    {
        var button = (Button)sender;
        string colorName = button.StyleId;

        ColorBox.Color = colorName switch
        {
            "Red" => Colors.Red,
            "Green" => Colors.Green,
            "Blue" => Colors.Blue,
            "Yellow" => Colors.Yellow,
            _ => Colors.LightGray
        };
    }
}
```

15.6. ImageButton - receptury praktyczne

PRZYKŁAD 21: IMAGEBUTTON - PRZYCISK OBRAZKOWY

ImageButton wyświetla sam obraz (bez tekstu) jako przycisk klikalny. Przydatny do ikon akcji.

```
<!-- XAML: ImageButton -->
<VerticalStackLayout Padding="20" Spacing="15" HorizontalOptions="Center">
    <ImageButton Source="heart_icon.png"
        WidthRequest="60"
        HeightRequest="60"
        CornerRadius="30"
        BackgroundColor="LightPink"
        Clicked="OnHeartClicked"
        Aspect="AspectFit"
        Padding="10" />

    <ImageButton Source="settings_icon.png"
        WidthRequest="50"
        HeightRequest="50"
        BackgroundColor="LightGray"
        CornerRadius="10"
        Clicked="OnSettingsClicked"
        Aspect="AspectFit"
        Padding="8" />
</VerticalStackLayout>
```

```
<Label x:Name="ImageButtonLabel" Text="" HorizontalTextAlignment="Center" />
</VerticalStackLayout>
```

```
// C#: ImageButton
public partial class ImageButtonPage : ContentPage
{
    public ImageButtonPage()
    {
        InitializeComponent();

        private void OnHeartClicked(object sender, EventArgs e)
        {
            ImageButtonLabel.Text = "❤️ Polubiono!";
        }

        private void OnSettingsClicked(object sender, EventArgs e)
        {
            ImageButtonLabel.Text = "⚙️ Otwarto ustawienia";
        }
    }
}
```

PRZYKŁAD 22: IMAGEBUTTON - TWORZONY PROGRAMOWO

```
// C#: ImageButton tworzony w kodzie
var layout = new VerticalStackLayout { Padding = 20, Spacing = 15 };
var statusLabel = new Label { HorizontalTextAlignment = TextAlignment.Center };

var playButton = new ImageButton
{
    Source = "play_icon.png",
    WidthRequest = 70,
    HeightRequest = 70,
    CornerRadius = 35,
    BackgroundColor = Colors.LightGreen,
    Aspect = Aspect.AspectFit,
    Padding = 15
};

playButton.Clicked += (s, e) => statusLabel.Text = "▶️ Odtwarzanie...";

var stopButton = new ImageButton
{
    Source = "stop_icon.png",
    WidthRequest = 70,
    HeightRequest = 70,
    CornerRadius = 35,
    BackgroundColor = Colors.LightCoral,
    Aspect = Aspect.AspectFit,
    Padding = 15
};
```

```
stopButton.Clicked += (s, e) => statusLabel.Text = "🛑 Zatrzymano";

layout.Children.Add(playButton);
layout.Children.Add(stopButton);
layout.Children.Add(statusLabel);
Content = layout;
```

16. Kontrolki wyboru i wartości liczbowe

16.1. Slider - odczyt wartości i ValueChanged

Slider to suwak zwracający wartość typu **double** w zakresie **Minimum** – **Maximum**. Zdarzenie **ValueChanged** reaguje w trakcie przesuwania, dostarczając nową wartość w **e.NewValue**.

PRZYKŁAD C#

```
private void OnSuwak(object sender, ValueChangedEventArgs e)
{
    double wartosc = e.NewValue; // nowa wartość (double)
    int calkowita = (int)e.NewValue; // rzutowanie na int
    Etykieta.Text = $"Wartość: {calkowita}";
}
```

Na co uważać:

Korzystaj z **e.NewValue** w handlerze zamiast odczytywać **Suwak.Value**. Rzutuj na **int**, gdy potrzebujesz liczby całkowitej. **ValueChanged** wywołuje się wielokrotnie podczas przeciągania - nadaje się do aktualizacji na żywo.

16.2. CheckBox

NAJWAŻNIEJSZE WŁAŚCIWOŚCI

WŁAŚCIWOŚĆ	OPIS	PRZYKŁAD
Color	kolor zaznaczenia	Color="Green"

NAJWAŻNIEJSZE ZDARZENIA

ZDARZENIE	KIEDY WYSTĘPUJE	PRZYKŁAD UŻYCIA
-----------	-----------------	-----------------

PRZYKŁAD PODSTAWOWY

```
<HorizontalStackLayout Spacing="8">
    <CheckBox x:Name="Zgoda" CheckedChanged="OnZgoda" />
    <Label Text="Akceptuję regulamin" VerticalOptions="Center" />
</HorizontalStackLayout>
```

PRZYKŁAD W C#

```
private void OnZgoda(object sender, CheckedChangedEventArgs e)
{
    PrzyciskDalej.IsEnabled = e.Value; // aktywny tylko po zaznaczeniu zgody
}
```


TYPOWE ZASTOSOWANIA

- Akceptacja regulaminu.
- Zaznaczanie wielu opcji niezależnie.

TYPOWE BŁĘDY

- Mylenie `CheckBox` (niezależne) z `RadioButton` (wzajemnie wykluczające się).

16.3. RadioButton

`RadioButton` służy do wyboru **dokładnie jednej** opcji z grupy. Przyciski o tej samej `GroupName` wykluczają się wzajemnie - zaznaczenie jednego odznacza pozostałe.

NAJWAŻNIEJSZE WŁAŚCIWOŚCI

WŁAŚCIWOŚĆ	OPIS	PRZYKŁAD
<code>GroupName</code>	nazwa grupy (wzajemne wykluczanie)	<code>GroupName="rozmiar"</code>
<code>Content</code>	etykieta obok przycisku	<code>Content="Średni"</code>
<code>Value</code>	wartość przypisana opcji	<code>Value="M"</code>

NAJWAŻNIEJSZE ZDARZENIA

ZDARZENIE	KIEDY WYSTĘPUJE	PRZYKŁAD UŻYCIA
-----------	-----------------	-----------------

PRZYKŁAD PODSTAWOWY

```
<VerticalStackLayout>
    <Label Text="Wybierz rozmiar:" FontAttributes="Bold" />
    <RadioButton Content="Mały" GroupName="rozmiar" Value="S" CheckedChanged="OnRozmiar" />
    <RadioButton Content="Średni" GroupName="rozmiar" Value="M" IsChecked="True" CheckedChanged="OnRozmiar" />
    <RadioButton Content="Duży" GroupName="rozmiar" Value="L" CheckedChanged="OnRozmiar" />
    <Label x:Name="Wybrany" Text="Wybrano: M" />
</VerticalStackLayout>
```

PRZYKŁAD W C#

```
private void OnRozmiar(object sender, CheckedChangedEventArgs e)
{
    if (!e.Value) return; // reaguj tylko na zaznaczenie, nie na odznaczenie
    var rb = (RadioButton)sender;
    Wybrany.Text = $"Wybrano: {rb.Value}";
}
```

TYPOWE ZASTOSOWANIA

- Wybór płci, rozmiaru, kategorii, sposobu płatności.

TYPOWE BŁĘDY

- Brak `GroupName` - przyciski nie wykluczają się i można zaznaczyć kilka naraz.

Bez wspólnego `GroupName` przyciski radiowe nie tworzą grupy. Zawsze nadawaj tę samą nazwę grupy opcjom, które mają się wzajemnie wykluczać.

16.4. Switch

`Switch` to **przełącznik** w stylu mobilnym reprezentujący stan włączone/wyłączone. Stan przechowuje właściwość logiczna `IsToggled`. Świetnie pasuje do ustawień typu „włącz powiadomienia”.

NAJWAŻNIEJSZE WŁAŚCIWOŚCI

WŁAŚCIWOŚĆ	OPIS	PRZYKŁAD
IsToggled	stan włączony/wyłączony (bool)	IsToggled="True"
OnColor	kolor w stanie włączonym	OnColor="Green"
ThumbColor	kolor suwaka	ThumbColor="White"

NAJWAŻNIEJSZE ZDARZENIA

ZDARZENIE	KIEDY WYSTĘPUJE	PRZYKŁAD UŻYCIA
Toggled	po przełączeniu	reakcja na zmianę stanu

PRZYKŁAD PODSTAWOWY

```
<HorizontalStackLayout Spacing="10">
  <Switch x:Name="Powiadomienia" Toggled="OnPrzelaczono" />
  <Label x:Name="StatusPowiadomien" Text="Powiadomienia: wyłączone"
    VerticalOptions="Center" />
</HorizontalStackLayout>
```

PRZYKŁAD W C#

```
private void OnPrzelaczono(object sender, ToggledEventArgs e)
{
    StatusPowiadomien.Text = e.Value
        ? "Powiadomienia: włączone"
        : "Powiadomienia: wyłączone";
}
```

TYPOWE ZASTOSOWANIA

- Ustawienia on/off (powiadomienia, tryb ciemny).
- Przełączanie stanu urządzenia.

TYPOWE BŁĘDY

16.5. Switch a CheckBox - porównanie

NAJWAŻNIEJSZE INFORMACJE

CECHA	SWITCH	CHECKBOX
Wygląd	przełącznik (suwak)	kwadrat z „ptaszkiem"
Typowe użycie	ustawienia on/off	akceptacja, wybór wielu opcji

Kiedy używać?

- Switch - gdy chodzi o włączenie/wyłączenie funkcji (ustawienia).
- CheckBox - gdy zaznaczamy pozycje lub akceptujemy warunek.
- RadioButton - gdy wybieramy jedną opcję z kilku.

Na co uważać:

Wszystkie te kontrolki przechowują **stan logiczny** (bool). W logice aplikacji odczytujesz ten stan i podejmujesz decyzję instrukcją if . To prosty, ale fundamentalny wzorzec interakcji.

16.6. CheckBox - pełna tabela atrybutów

ATRYBUT	TYP	OPIS
Color	Color	kolor zaznaczenia
IsEnabled	bool	czy aktywne

```
<HorizontalStackLayout Spacing="8">
    <CheckBox x:Name="Zgoda" Color="Green" CheckedChanged="OnZgoda" />
    <Label Text="Akceptuję regulamin" VerticalOptions="Center" />
</HorizontalStackLayout>
```

```
private void OnZgoda(object sender, CheckedChangedEventArgs e)
{
    PrzyciskDalej.IsEnabled = e.Value; // aktywny tylko po zaznaczeniu
}
```

16.7. RadioButton - pełna tabela atrybutów

ATRYBUT	TYP	OPIS
GroupName	string	grupa wzajemnie wykluczających się opcji
Content	object	etykieta/treść obok przycisku
Value	object	wartość przypisana opcji
IsEnabled	bool	czy aktywny

```
<VerticalStackLayout>
    <Label Text="Sposób płatności:" FontAttributes="Bold" />
    <RadioButton Content="Karta" GroupName="platnosc" Value="karta" CheckedChanged="OnPlatnosc" IsChecked="True" />
    <RadioButton Content="Gotówka" GroupName="platnosc" Value="gotowka" CheckedChanged="OnPlatnosc" />
    <RadioButton Content="Przelew" GroupName="platnosc" Value="przelew" CheckedChanged="OnPlatnosc" />
    <Label x:Name="Wybrana" Text="Wybrano: karta" />
</VerticalStackLayout>
```

```
private void OnPlatnosc(object sender, CheckedChangedEventArgs e)
{
    if (!e.Value) return; // reaguj tylko na zaznaczenie
    var rb = (RadioButton)sender;
    Wybrana.Text = $"Wybrano: {rb.Value}";
}
```

16.8. Switch - pełna tabela atrybutów

ATRYBUT	TYP	OPIS
IsToggled	bool	stan włączony/wyłączony
OnColor	Color	kolor w stanie włączonym
ThumbColor	Color	kolor suwaka (kółka)
IsEnabled	bool	czy aktywny

```
<HorizontalStackLayout Spacing="10">
    <Switch x:Name="Powiadomienia" OnColor="Green" ThumbColor="White" Toggled="OnPrzełączono" />
    <Label x:Name="StatusP" Text="Powiadomienia: wyłączone" VerticalOptions="Center" />
</HorizontalStackLayout>
```

```
private void OnPrzełączono(object sender, ToggledEventArgs e)
{
    StatusP.Text = e.Value ? "Powiadomienia: włączone" : "Powiadomienia: wyłączone";
}
```

16.9. Picker

Picker to rozwijana lista wyboru. Używaj go wtedy, gdy użytkownik ma wybrać jedną pozycję z krótkiej lub średniej listy opcji.

NAJWAŻNIEJSZE WŁAŚCIWOŚCI

WŁAŚCIWOŚĆ	OPIS	PRZYKŁAD
ItemsSource	kolekcja pozycji	List<string>
SelectedIndex	indeks wybranej pozycji (-1 = brak)	odczyt w C#
SelectedItem	wybrany obiekt	odczyt w C#
Title	tytuł/podpowiedź listy	Title="Wybierz"

NAJWAŻNIEJSZE ZDARZENIA

ZDARZENIE	KIEDY WYSTĘPUJE	PRZYKŁAD UŻYCIA
SelectedIndexChanged	po zmianie wyboru	reakcja na wybór

PRZYKŁAD PODSTAWOWY

```
<Picker x:Name="ListaGatunkow" Title="Wybierz gatunek"
    SelectedIndexChanged="OnGatunek" />
```

PRZYKŁAD W C#

```
public MainPage()
{
    InitializeComponent();
    // Wypełnienie listy opcji
    ListaGatunkow.ItemsSource = new List<string> { "Pies", "Kot", "Chomik", "Papuga" };
}
```

```

    }

    private void OnGatunek(object sender, EventArgs e)
    {
        if (ListaGatunkow.SelectedIndex == -1) return; // nic nie wybrano
        string gatunek = ListaGatunkow.SelectedItem.ToString();
        Komunikat.Text = $"Wybrano: {gatunek}";
    }

```

```

<Picker x:Name="ListaProduktow" Title="Produkt"
        ItemDisplayBinding="{Binding Nazwa}" />

```

```

ListaProduktow.ItemsSource = new List<Produkt>
{
    new Produkt { Nazwa = "Kawa", Cena = 19.99 },
    new Produkt { Nazwa = "Herbata", Cena = 12.50 }
};
// po wyborze:
var wybrany = (Produkt)ListaProduktow.SelectedItem;

```

TYPOWE ZASTOSOWANIA

- Wybór kategorii, gatunku, koloru.
- Wybór z listy obiektów (produkt, kontakt).

TYPOWE BŁĘDY

- Odwołanie do `SelectedItem` bez sprawdzenia `SelectedIndex  $\neq$  -1` (na starcie nic nie jest wybrane $\rightarrow$ `null`).
- Brak `ItemDisplayBinding` przy liście obiektów (pokazuje nazwę typu zamiast właściwości).

Zawsze sprawdzaj `SelectedIndex  $\neq$  -1` przed użyciem `SelectedItem`. Na starcie nic nie jest wybrane, a `SelectedItem` jest `null` - odwołanie do niego spowoduje błąd.

16.10. DatePicker

`DatePicker` służy do wyboru daty. Zwraca wartość typu `DateTime`, dlatego dobrze pasuje do dat urodzenia, rezerwacji, wydarzeń i terminów.

NAJWAŻNIEJSZE WŁAŚCIWOŚCI

WŁAŚCIWOŚĆ	OPIS	PRZYKŁAD
<code>Date</code>	wybrana data (<code>DateTime</code>)	odczyt w C#
<code>MinimumDate</code>	najwcześniejsza możliwa data	<code>MinimumDate="2020-01-01"</code>
<code>MaximumDate</code>	najpóźniejsza możliwa data	ograniczenie zakresu
<code>Format</code>	format wyświetlania	<code>Format="dd.MM.yyyy"</code>

NAJWAŻNIEJSZE ZDARZENIA

ZDARZENIE	KIEDY WYSTĘPUJE	PRZYKŁAD UŻYCIA
<code>DateSelected</code>	po wybraniu daty	reakcja na zmianę

PRZYKŁAD PODSTAWOWY

```
<DatePicker x:Name="WyborDaty" Format="dd.MM.yyyy"
            DateSelected="OnData" />
```

PRZYKŁAD W C#

```
private void OnData(object sender, DateChangedEventArgs e)
{
    DateTime data = e.NewDate;
    Komunikat.Text = $"Wybrano: {data:dd.MM.yyyy}";
}
```

TYPOWE ZASTOSOWANIA

- Data urodzenia, data rezerwacji, data wydarzenia.

TYPOWE BŁĘDY

- Brak ograniczeń `MinimumDate` / `MaximumDate` tam, gdzie data powinna być z określonego zakresu.

16.11. TimePicker

`TimePicker` służy do wyboru godziny. Zwraca wartość typu `TimeSpan`, więc odczytujesz z niego czas bez konkretnej daty.

NAJWAŻNIEJSZE WŁAŚCIWOŚCI

WŁAŚCIWOŚĆ	OPIS	PRZYKŁAD
<code>Time</code>	wybrana godzina (<code>TimeSpan</code>)	odczyt w C#
<code>Format</code>	format wyświetlania	<code>Format="HH:mm"</code>

PRZYKŁAD PODSTAWOWY

```
<TimePicker x:Name="WyborGodziny" Format="HH:mm" />
<Button Text="Pokaż" Clicked="OnPokaż" />
<Label x:Name="Wynik" />
```

PRZYKŁAD W C#

```
private void OnPokaż(object sender, EventArgs e)
{
    TimeSpan czas = WyborGodziny.Time;
    Wynik.Text = $"Godzina: {czas:hh\\:mm}";
}
```

TYPOWE ZASTOSOWANIA

- Godzina wizyty, alarmu, rezerwacji.

TYPOWE BŁĘDY

- Mylenie typu `Time` (`TimeSpan`) z `Date` (`DateTime`).

16.12. Slider

`Slider` to suwak pozwalający wybrać wartość liczbową przez przesuwanie uchwytu. Definiują go `Minimum`, `Maximum` i `Value` (typu `double`). Zdarzenie `ValueChanged` reaguje w trakcie przesuwania, co pozwala natychmiast pokazywać efekt.

Slider stosujemy do **płynnego wyboru wartości z zakresu**: głośność, jasność, rozmiar czcionki, ocena. Każda zmiana może natychmiast wpływać na inny element interfejsu.

NAJWAŻNIEJSZE WŁAŚCIWOŚCI

WŁAŚCIWOŚĆ	OPIS	PRZYKŁAD
Minimum	wartość minimalna	Minimum="0"
Maximum	wartość maksymalna (można zmieniać w runtime)	Maximum="100"
Value	aktualna wartość (double)	Value="20"
ThumbColor	kolor uchwyty	ThumbColor="Blue"

NAJWAŻNIEJSZE ZDARZENIA

ZDARZENIE	KIEDY WYSTĘPUJE	PRZYKŁAD UŻYCIA
ValueChanged	przy każdej zmianie wartości	aktualizacja na żywo

PRZYKŁAD PODSTAWOWY

```
<Slider x:Name="Suwak" Minimum="10" Maximum="60" Value="20"
        ValueChanged="OnRozmiar" />
<Label x:Name="Podglad" Text="Tekst" FontSize="20" />
```

PRZYKŁAD W C#

```
private void OnRozmiar(object sender, ValueChangedEventArgs e)
{
    int rozmiar = (int)e.NewValue; // double -> int
    Podglad.FontSize = rozmiar;
    EtykietaWartosci.Text = $"Rozmiar: {rozmiar}";
}
```

TYPOWE ZASTOSOWANIA

- Rozmiar czcionki, głośność, jasność.
- Składowe koloru w wzorniku RGB.
- Dynamiczna zmiana wyglądu w czasie rzeczywistym.

TYPOWE BŁĘDY

- Pominięcie rzutowania na **int** przy wartościach całkowitych (np. **23.7**).
- Odczyt **Suwak.Value** zamiast wygodnego **e.NewValue** w handlerze.

16.13. Stepper

Stepper to para przycisków +/- do zmiany wartości o ustalony krok. W odróżnieniu od **Slider** (płynny wybór z szerokiego zakresu), **Stepper** służy do **precyzyjnych, małych zmian** - liczby sztuk, osób, porcji.

NAJWAŻNIEJSZE WŁAŚCIWOŚCI

WŁAŚCIWOŚĆ	OPIS	PRZYKŁAD
Minimum	minimalna wartość	Minimum="1"
Maximum	maksymalna wartość	Maximum="10"
Increment	krok zmiany	Increment="1"
Value	aktualna wartość	Value="1"

NAJWAŻNIEJSZE ZDARZENIA

ZDARZENIE	KIEDY WYSTĘPUJE	PRZYKŁAD UŻYCIA
ValueChanged	po zmianie wartości	aktualizacja licznika

PRZYKŁAD PODSTAWOWY

```
<HorizontalStackLayout Spacing="12">
    <Label Text="Liczba sztuk:" VerticalOptions="Center" />
    <Stepper x:Name="Ilosc" Minimum="1" Maximum="10" Increment="1" Value="1"
        ValueChanged="OnIlosc" />
    <Label x:Name="EtykietaIlosci" Text="1" VerticalOptions="Center" />
</HorizontalStackLayout>
```

PRZYKŁAD W C#

```
private void OnIlosc(object sender, ValueChangedEventArgs e)
{
    EtykietaIlosci.Text = ((int)e.NewValue).ToString();
}
```

TYPOWE ZASTOSOWANIA

- Liczba sztuk produktu w koszyku.
- Liczba osób, porcji.

TYPOWE BŁĘDY

- Użycie **Stepper** tam, gdzie wygodniejszy byłby **Slider** (i odwrotnie).

16.14. Slider a Stepper - porównanie

NAJWAŻNIEJSZE INFORMACJE

CECHA	SLIDER	STEPPER
Sposób wyboru	przesuwanie uchwyty	przyciski +/-
Zakres	szeroki, płynny	wąski, precyzyjny
Typowe użycie	głośność, rozmiar czcionki	liczba sztuk
Wartość	double	double (często rzutowana na int)

Na co uważać:

Wybierz **Slider**, gdy ważna jest płynność i szeroki zakres (np. głośność). Wybierz **Stepper**, gdy chcesz precyzyjnych, małych zmian liczby i chcesz uniknąć ręcznego wpisywania. Obie kontrolki zwracają **double** - do liczb całkowitych rzutuj na **int**.

Wszystkie kontrolki z tego rozdziału świetnie współpracują z **Label** pokazującym aktualnie wybraną wartość. Po każdej zmianie aktualizuj etykietę - to natychmiastowa, czytelna informacja zwrotna dla użytkownika.

16.15. Pełne tabele atrybutów kontrolek wyboru danych

ATRYBUT	TYP	OPIS
ItemsSource	IList	kolekcja pozycji
SelectedItem	object	wybrany obiekt
SelectedIndex	int	indeks wyboru (-1 = brak)
Title	string	tytuł/podpowiedź
TitleColor	Color	kolor tytułu
ItemDisplayBinding	Binding	która właściwość obiektu ma być wyświetlana
TextColor	Color	kolor tekstu
FontSize / FontAttributes	-	czcionka
HorizontalTextAlignment	enum	wyrównanie

```
<Picker x:Name="ListaProduktow" Title="Wybierz produkt"
        TitleColor="Gray" ItemDisplayBinding="{Binding Nazwa}"
        SelectedIndexChanged="OnWybor" />
```

```
ListaProduktow.ItemsSource = produkty;           // lista obiektów
ListaProduktow.SelectedIndex = 0;                 // wybierz pierwszy
var wybrany = (Produkt)ListaProduktow.SelectedItem;
```

ATRYBUT	TYP	OPIS
Date	DateTime	wybrana data
MinimumDate	DateTime	najwcześniejsza data
MaximumDate	DateTime	najpóźniejsza data
Format	string	format wyświetlania (dd.MM.yyyy)
TextColor	Color	kolor tekstu
FontSize / FontAttributes	-	czcionka

```
<DatePicker x:Name="Data" Format="dd.MM.yyyy"
            MinimumDate="2020-01-01" MaximumDate="2030-12-31"
            DateSelected="OnData" />
```

ATRYBUT	TYP	OPIS
Time	TimeSpan	wybrana godzina
Format	string	format (HH:mm)
TextColor	Color	kolor tekstu
FontSize	double	rozmiar czcionki

```
<TimePicker x:Name="Godzina" Format="HH:mm" />
```

Slider (suwak)

ATRYBUT	TYP	OPIS
Minimum	double	wartość minimalna
Maximum	double	wartość maksymalna
Value	double	aktualna wartość
MinimumTrackColor	Color	kolor przebytej części
MaximumTrackColor	Color	kolor pozostałej części
ThumbColor	Color	kolor uchwyty

```
<Slider x:Name="Glosnosc" Minimum="0" Maximum="100" Value="50"
        MinimumTrackColor="#2196F3" ThumbColor="#1565C0"
        ValueChanged="OnGlosnosc" />
```

```
private void OnGlosnosc(object sender, ValueChangedEventArgs e)
{
    int v = (int)e.NewValue;           // double -> int
    Etykieta.Text = $"Głośność: {v}%";
}
```

Stepper (licznik +/-)

ATRYBUT	TYP	OPIS
Minimum	double	wartość minimalna
Maximum	double	wartość maksymalna
Increment	double	krok zmiany
Value	double	aktualna wartość

```
<Stepper x:Name="Ilosc" Minimum="1" Maximum="99" Increment="1" Value="1"
        ValueChanged="OnIlosc" />
```

Ten rozdział omawia kontrolki, które **pokazują** treści wizualne i informacje o stanie aplikacji: obrazy (`Image`), wskaźniki pracy (`ActivityIndicator` , `ProgressBar`) oraz proste elementy graficzne (`BoxView` , `Border` , `Frame`). Obrazy są szczególnie ważne, bo bardzo często zmieniają się dynamicznie w zależności od stanu.

16.16. CheckBox - receptury praktyczne

PRZYKŁAD 23: CHECKBOX - ZAZNACZANIE OPCJI

```
←!— XAML: CheckBox →  
  
<VerticalStackLayout Padding="20" Spacing="10">  
    <HorizontalStackLayout Spacing="10">  
        <CheckBox x:Name="TermsCheckBox"  
            CheckedChanged="OnCheckBoxChanged"  
            Color="DodgerBlue" />  
        <Label Text="Akceptuję regulamin"  
            VerticalTextAlignment="Center" />  
    </HorizontalStackLayout>  
  
    <HorizontalStackLayout Spacing="10">  
        <CheckBox x:Name="NewsletterCheckBox"  
            CheckedChanged="OnCheckBoxChanged"  
            Color="Green" />  
        <Label Text="Chcę otrzymywać newsletter"  
            VerticalTextAlignment="Center" />  
    </HorizontalStackLayout>  
  
    <HorizontalStackLayout Spacing="10">  
        <CheckBox x:Name="DarkModeCheckBox"  
            CheckedChanged="OnCheckBoxChanged"  
            Color="Purple" />  
        <Label Text="Tryb ciemny"  
            VerticalTextAlignment="Center" />  
    </HorizontalStackLayout>  
  
    <Label x:Name="CheckBoxStatusLabel"  
        Text=""  
        FontSize="12"  
        TextColor="Gray" />  
</VerticalStackLayout>
```

```
// C#: CheckBox (code-behind)  
public partial class CheckBoxPage : ContentPage  
{  
    public CheckBoxPage()  
    {  
        InitializeComponent();  
    }  
  
    private void OnCheckBoxChanged(object sender, CheckedChangedEventArgs e)  
    {  
        var statuses = new List<string>();  
  
        if (TermsCheckBox.IsChecked) statuses.Add("Regulamin ✓");  
        if (NewsletterCheckBox.IsChecked) statuses.Add("Newsletter ✓");  
        if (DarkModeCheckBox.IsChecked) statuses.Add("Tryb ciemny ✓");  
  
        CheckBoxStatusLabel.Text = statuses.Count > 0
```

```

        ? string.Join(", ", statuses)
        : "Nic nie zaznaczono";
    }
}

```

PRZYKŁAD 24: CHECKBOX - LISTA ZADAŃ (TODO)

Praktyczne zastosowanie CheckBox jako listy zadań do odhaczania.

```

<!-- XAML: CheckBox jako lista TODO -->
<VerticalStackLayout Padding="20" Spacing="5">
    <Label Text="Lista zadań:" FontSize="18" FontAttributes="Bold" />

    <HorizontalStackLayout Spacing="8">
        <CheckBox IsChecked="True" Color="Green" />
        <Label Text="Zrobić zakupy" VerticalTextAlignment="Center"
            TextDecorations="Strikethrough" TextColor="Gray" />
    </HorizontalStackLayout>

    <HorizontalStackLayout Spacing="8">
        <CheckBox IsChecked="False" Color="Green" />
        <Label Text="Napisać raport" VerticalTextAlignment="Center" />
    </HorizontalStackLayout>

    <HorizontalStackLayout Spacing="8">
        <CheckBox IsChecked="False" Color="Green" />
        <Label Text="Umyć samochód" VerticalTextAlignment="Center" />
    </HorizontalStackLayout>

    <HorizontalStackLayout Spacing="8">
        <CheckBox IsChecked="True" Color="Green" />
        <Label Text="Wysłać maila" VerticalTextAlignment="Center"
            TextDecorations="Strikethrough" TextColor="Gray" />
    </HorizontalStackLayout>
</VerticalStackLayout>

```

```

// C#: CheckBox jako lista TODO (dynamicznie generowana)
var layout = new VerticalStackLayout { Padding = 20, Spacing = 5 };
layout.Children.Add(new Label { Text = "Lista zadań:", FontSize = 18, FontAttributes = FontAttributes.Bold });

var tasks = new[] { "Zrobić zakupy", "Napisać raport", "Umyć samochód", "Wysłać maila" };

foreach (var task in tasks)
{
    var row = new HorizontalStackLayout { Spacing = 8 };
    var label = new Label { Text = task, VerticalTextAlignment = TextAlignment.Center };
    var checkBox = new CheckBox { Color = Colors.Green };

    checkBox.CheckedChanged += (s, e) =>
    {

```

```

        label.TextDecorations = e.Value ? TextDecorations.Strikethrough : TextDecorations.None;
        label.TextColor = e.Value ? Colors.Gray : Colors.Black;
    };

    row.Children.Add(checkBox);
    row.Children.Add(label);
    layout.Children.Add(row);
}

Content = layout;

```

16.17. RadioButton - receptury praktyczne

PRZYKŁAD 25: RADIOBUTTON Z GROUPNAME

RadioButton pozwala wybrać jedną opcję z grupy. Przyciski o tym samym GroupName tworzą jedną grupę wzajemnie wykluczającą.

```

<!-- XAML: RadioButton z GroupName -->
<VerticalStackLayout Padding="20" Spacing="10">
    <Label Text="Wybierz rozmiar:" FontSize="16" FontAttributes="Bold" />

    <RadioButton Content="Mały" GroupName="SizeGroup"
        CheckedChanged="OnSizeChanged" />
    <RadioButton Content="Średni" GroupName="SizeGroup"
        IsChecked="True" CheckedChanged="OnSizeChanged" />
    <RadioButton Content="Duży" GroupName="SizeGroup"
        CheckedChanged="OnSizeChanged" />
    <RadioButton Content="Bardzo duży" GroupName="SizeGroup"
        CheckedChanged="OnSizeChanged" />

    <Label x:Name="SizeLabel" Text="Wybrany rozmiar: Średni"
        FontSize="14" TextColor="DarkBlue" />
</VerticalStackLayout>

```

```

// C#: RadioButton z GroupName (code-behind)
public partial class RadioButtonPage : ContentPage
{
    public RadioButtonPage()
    {
        InitializeComponent();
    }

    private void OnSizeChanged(object sender, CheckedChangedEventArgs e)
    {
        if (e.Value) // Reaguj tylko na zaznaczenie (nie odznaczenie)
        {
            var radio = (RadioButton)sender;
            SizeLabel.Text = $"Wybrany rozmiar: {radio.Content}";
        }
    }
}

```

```
}  
  
}
```

PRZYKŁAD 26: RADIOBUTTON - WIELE GRUP NA JEDNEJ STRONIE

Można mieć kilka niezależnych grup RadioButton na jednej stronie.

←!— XAML: Wiele grup RadioButton →

```
<VerticalStackLayout Padding="20" Spacing="10">  
    <Label Text="Kolor tła:" FontAttributes="Bold" />  
    <RadioButton Content="Biały" GroupName="ColorGroup" IsChecked="True" />  
    <RadioButton Content="Jasny niebieski" GroupName="ColorGroup" />  
    <RadioButton Content="Jasny zielony" GroupName="ColorGroup" />  
  
    <BoxView HeightRequest="1" Color="LightGray" />  
  
    <Label Text="Czcionka:" FontAttributes="Bold" />  
    <RadioButton Content="Mała (12)" GroupName="FontGroup" />  
    <RadioButton Content="Średnia (16)" GroupName="FontGroup" IsChecked="True" />  
    <RadioButton Content="Duża (22)" GroupName="FontGroup" />  
  
    <BoxView HeightRequest="1" Color="LightGray" />  
  
    <Label Text="Motyw:" FontAttributes="Bold" />  
    <RadioButton Content="Jasny" GroupName="ThemeGroup" IsChecked="True" />  
    <RadioButton Content="Ciemny" GroupName="ThemeGroup" />  
    <RadioButton Content="Systemowy" GroupName="ThemeGroup" />  
</VerticalStackLayout>
```

```
// C#: Wiele grup RadioButton  
var layout = new VerticalStackLayout { Padding = 20, Spacing = 10 };  
  
// Grupa kolorów  
layout.Children.Add(new Label { Text = "Kolor tła:", FontAttributes = FontAttributes.Bold });  
layout.Children.Add(new RadioButton { Content = "Biały", GroupName = "ColorGroup", IsChecked = true });  
layout.Children.Add(new RadioButton { Content = "Jasny niebieski", GroupName = "ColorGroup" });  
layout.Children.Add(new RadioButton { Content = "Jasny zielony", GroupName = "ColorGroup" });  
  
// Grupa czcionek  
layout.Children.Add(new Label { Text = "Czcionka:", FontAttributes = FontAttributes.Bold });  
layout.Children.Add(new RadioButton { Content = "Mała (12)", GroupName = "FontGroup" });  
layout.Children.Add(new RadioButton { Content = "Średnia (16)", GroupName = "FontGroup", IsChecked = true });  
layout.Children.Add(new RadioButton { Content = "Duża (22)", GroupName = "FontGroup" });  
  
// Grupa motywów  
layout.Children.Add(new Label { Text = "Motyw:", FontAttributes = FontAttributes.Bold });  
layout.Children.Add(new RadioButton { Content = "Jasny", GroupName = "ThemeGroup", IsChecked = true });  
layout.Children.Add(new RadioButton { Content = "Ciemny", GroupName = "ThemeGroup" });  
layout.Children.Add(new RadioButton { Content = "Systemowy", GroupName = "ThemeGroup" });
```

```
Content = layout;
```

PRZYKŁAD 27: RADIOBUTTON - FORMULARZ ZAMÓWIENIA Z REAKCJĄ

Kompleksowy przykład: formularz z wyborem opcji i aktualizacją podsumowania.

←!— XAML: RadioButton w formularzu zamówienia →

```
<VerticalStackLayout Padding="20" Spacing="10">
    <Label Text="Sposób dostawy:" FontAttributes="Bold" FontSize="16" />

    <RadioButton Content="Kurier (15 zł)" GroupName="DeliveryGroup"
        Value="15" CheckedChanged="OnDeliveryChanged" />
    <RadioButton Content="Paczkomat (10 zł)" GroupName="DeliveryGroup"
        Value="10" IsChecked="True" CheckedChanged="OnDeliveryChanged" />
    <RadioButton Content="Odbiór osobisty (0 zł)" GroupName="DeliveryGroup"
        Value="0" CheckedChanged="OnDeliveryChanged" />

    <Label Text="Płatność:" FontAttributes="Bold" FontSize="16" Margin="0,10,0,0" />

    <RadioButton Content="BLIK" GroupName="PaymentGroup" IsChecked="True" />
    <RadioButton Content="Karta płatnicza" GroupName="PaymentGroup" />
    <RadioButton Content="Przelew tradycyjny" GroupName="PaymentGroup" />

    <Frame BorderColor="LightGray" Padding="15" CornerRadius="8" Margin="0,15,0,0">
        <Label x:Name="SummaryLabel" Text="Koszt dostawy: 10 zł" FontSize="16" />
    </Frame>
</VerticalStackLayout>
```

```
// C#: RadioButton w formularzu zamówienia (code-behind)
public partial class OrderFormPage : ContentPage
{
    public OrderFormPage()
    {
        InitializeComponent();
    }

    private void OnDeliveryChanged(object sender, CheckedChangedEventArgs e)
    {
        if (e.Value)
        {
            var radio = (RadioButton)sender;
            SummaryLabel.Text = $"Koszt dostawy: {radio.Value} zł";
        }
    }
}
```

16.18. Switch - receptury praktyczne

PRZYKŁAD 28: SWITCH - PRZEŁĄCZNIK ON/OFF

Switch to prosty przełącznik dwustanowy (włączony/wyłączony). Zdarzenie Toggled informuje o zmianie.

```
<!-- XAML: Switch -->
<VerticalStackLayout Padding="20" Spacing="15">
    <HorizontalStackLayout Spacing="15">
        <Switch x:Name="WifiSwitch"
            IsToggled="True"
            Toggled="OnWifiToggled"
            OnColor="DodgerBlue"
            ThumbColor="White" />

        <Label Text="Wi-Fi" VerticalTextAlignment="Center" FontSize="16" />
        <Label x:Name="WifiStatusLabel" Text="Włączone"
            VerticalTextAlignment="Center" TextColor="Green" />
    </HorizontalStackLayout>

    <HorizontalStackLayout Spacing="15">
        <Switch x:Name="BluetoothSwitch"
            IsToggled="False"
            Toggled="OnBluetoothToggled"
            OnColor="Blue"
            ThumbColor="LightGray" />

        <Label Text="Bluetooth" VerticalTextAlignment="Center" FontSize="16" />
        <Label x:Name="BluetoothStatusLabel" Text="Wyłączone"
            VerticalTextAlignment="Center" TextColor="Red" />
    </HorizontalStackLayout>

    <HorizontalStackLayout Spacing="15">
        <Switch x:Name="NotifSwitch"
            IsToggled="True"
            Toggled="OnNotifToggled"
            OnColor="Green"
            ThumbColor="White" />

        <Label Text="Powiadomienia" VerticalTextAlignment="Center" FontSize="16" />
        <Label x:Name="NotifStatusLabel" Text="Włączone"
            VerticalTextAlignment="Center" TextColor="Green" />
    </HorizontalStackLayout>
</VerticalStackLayout>
```

```
// C#: Switch (code-behind)
public partial class SwitchPage : ContentPage
{
    public SwitchPage()
    {
        InitializeComponent();
    }

    private void OnWifiToggled(object sender, ToggledEventArgs e)
    {
        WifiStatusLabel.Text = e.Value ? "Włączone" : "Wyłączone";
    }
}
```



```

        WifiStatusLabel.TextColor = e.Value ? Colors.Green : Colors.Red;
    }

    private void OnBluetoothToggled(object sender, ToggledEventArgs e)
    {
        BluetoothStatusLabel.Text = e.Value ? "Włączone" : "Wyłączone";
        BluetoothStatusLabel.TextColor = e.Value ? Colors.Green : Colors.Red;
    }

    private void OnNotifToggled(object sender, ToggledEventArgs e)
    {
        NotifStatusLabel.Text = e.Value ? "Włączone" : "Wyłączone";
        NotifStatusLabel.TextColor = e.Value ? Colors.Green : Colors.Red;
    }
}

```

PRZYKŁAD 29: SWITCH - ZMIANA MOTYWU (JASNY/CIEMNY)

Praktyczne zastosowanie Switch do przełączania motywu aplikacji.

```

<!-- XAML: Switch do zmiany motywu -->
<VerticalStackLayout x:Name="MainLayout" Padding="20" Spacing="15"
    BackgroundColor="White">
    <Label Text="Ustawienia wyświetlania" FontSize="20" FontAttributes="Bold"
        x:Name="TitleLabel" TextColor="Black" />

    <HorizontalStackLayout Spacing="15">
        <Label Text="🌙 Tryb ciemny" VerticalTextAlignment="Center" FontSize="16"
            x:Name="DarkModeLabel" TextColor="Black" />
        <Switch x:Name="DarkModeSwitch"
            IsToggled="False"
            Toggled="OnDarkModeToggled"
            OnColor="MediumPurple" />
    </HorizontalStackLayout>

    <Label x:Name="PreviewLabel"
        Text="To jest przykładowy tekst do podglądu motywu."
        FontSize="14" TextColor="Black" />
</VerticalStackLayout>

```

```

// C#: Switch do zmiany motywu (code-behind)
public partial class ThemeSwitchPage : ContentPage
{
    public ThemeSwitchPage()
    {
        InitializeComponent();
    }

    private void OnDarkModeToggled(object sender, ToggledEventArgs e)
    {

```

```

        if (e.Value) // Tryb ciemny
        {
            MainLayout.BackgroundColor = Color.FromArgb("#1E1E1E");
            TitleLabel.TextColor = Colors.White;
            DarkModeLabel.TextColor = Colors.White;
            PreviewLabel.TextColor = Colors.LightGray;
        }
        else // Tryb jasny
        {
            MainLayout.BackgroundColor = Colors.White;
            TitleLabel.TextColor = Colors.Black;
            DarkModeLabel.TextColor = Colors.Black;
            PreviewLabel.TextColor = Colors.Black;
        }
    }
}

```

PRZYKŁAD 30: SWITCH TWORZONY PROGRAMOWO Z BINDINGIEM

```

// C#: Switch z bindingiem do obiektu ustawień
public class UstawieniaPowiadomien : INotifyPropertyChanged
{
    private bool _isNotificationsEnabled = true;
    private bool _isSoundEnabled = true;
    private bool _isVibrationEnabled = false;

    public bool IsNotificationsEnabled
    {
        get => _isNotificationsEnabled;
        set { _isNotificationsEnabled = value; OnPropertyChanged(); OnPropertyChanged(nameof(StatusText)); }
    }

    public bool IsSoundEnabled
    {
        get => _isSoundEnabled;
        set { _isSoundEnabled = value; OnPropertyChanged(); OnPropertyChanged(nameof(StatusText)); }
    }

    public bool IsVibrationEnabled
    {
        get => _isVibrationEnabled;
        set { _isVibrationEnabled = value; OnPropertyChanged(); OnPropertyChanged(nameof(StatusText)); }
    }

    public string StatusText => $"Powiadomienia: {(IsNotificationsEnabled ? "TAK" : "NIE")}, " +
        $"Dźwięk: {(IsSoundEnabled ? "TAK" : "NIE")}, " +
        $"Wibracja: {(IsVibrationEnabled ? "TAK" : "NIE")}";

    public event PropertyChangedEventHandler PropertyChanged;
    protected void OnPropertyChanged([CallerMemberName] string name = null)
        => PropertyChanged?.Invoke(this, new PropertyChangedEventArgs(name));
}

```

```

}

// Użycie w kodzie strony:
public class SettingsPage : ContentPage
{
    public SettingsPage()
    {
        BindingContext = new UstawieniaPowiadomien();

        var layout = new VerticalStackLayout { Padding = 20, Spacing = 15 };

        // Powiadomienia
        var notifRow = new HorizontalStackLayout { Spacing = 10 };
        var notifSwitch = new Switch { OnColor = Colors.Green };
        notifSwitch.SetBinding(Switch.IsToggledProperty, "IsNotificationsEnabled");
        notifRow.Children.Add(notifSwitch);
        notifRow.Children.Add(new Label { Text = "Powiadomienia", VerticalTextAlignment = TextAlignment.Center });
        layout.Children.Add(notifRow);

        // Dźwięk
        var soundRow = new HorizontalStackLayout { Spacing = 10 };
        var soundSwitch = new Switch { OnColor = Colors.Blue };
        soundSwitch.SetBinding(Switch.IsToggledProperty, "IsSoundEnabled");
        soundRow.Children.Add(soundSwitch);
        soundRow.Children.Add(new Label { Text = "Dźwięk", VerticalTextAlignment = TextAlignment.Center });
        layout.Children.Add(soundRow);

        // Wibracja
        var vibRow = new HorizontalStackLayout { Spacing = 10 };
        var vibSwitch = new Switch { OnColor = Colors.Orange };
        vibSwitch.SetBinding(Switch.IsToggledProperty, "IsVibrationEnabled");
        vibRow.Children.Add(vibSwitch);
        vibRow.Children.Add(new Label { Text = "Wibracja", VerticalTextAlignment = TextAlignment.Center });
        layout.Children.Add(vibRow);

        // Status
        var statusLabel = new Label { FontSize = 12, TextColor = Colors.Gray };
        statusLabel.SetBinding(Label.TextProperty, "StatusText");
        layout.Children.Add(statusLabel);

        Content = layout;
    }
}

```

17. Kontrolki graficzne i prezentacyjne

17.1. Image

Image wyświetla **grafikę** - zdjęcie, ikonę, ilustrację. Źródło obrazu ustawiamy właściwością **Source**, najczęściej podając nazwę pliku z folderu **Resources/Images**. Sposób dopasowania określa **Aspect**, a przezroczystość - **Opacity**.

Image służy do **prezentacji grafiki**: logo, zdjęć produktów, ikon, obrazów stanu (np. różne grafiki kości w grze). Obraz bardzo często zmienia się w trakcie działania - podmieniamy **Source** zależnie od stanu.

NAJWAŻNIEJSZE WŁAŚCIWOŚCI

WŁAŚCIWOŚĆ	OPIS	PRZYKŁAD
Source	źródło obrazu	Source="Logo.png"
Aspect	dopasowanie do obszaru	AspectFit
Opacity	przezroczystość (0–1)	Opacity="0.4"
WidthRequest / HeightRequest	sugerowany rozmiar	HeightRequest="100"
IsVisible	czy widoczny	IsVisible="False"

NAJWAŻNIEJSZE ZDARZENIA

ZDARZENIE	KIEDY WYSTĘPUJE	PRZYKŁAD UŻYCIA
(brak własnych)	do kliknięć dodaj TapGestureRecognizer	klikalny obraz

WARTOŚCI **ASPECT**

WARTOŚĆ	DZIAŁANIE
AspectFit	mieści cały obraz, zachowuje proporcje (mogą być puste pasy)
AspectFill	wypełnia obszar, zachowuje proporcje (przycina nadmiar)
Fill	rozciąga na cały obszar (może zniekształcić)
Center	wyśrodkowuje w oryginalnym rozmiarze

PRZYKŁAD PODSTAWOWY

```
<Image x:Name="ObrazKostki"
    Source="kostka1.png"
    Aspect="AspectFit"
    HeightRequest="100"
    Opacity="1" />
```

PRZYKŁAD W C#

```
// Dynamiczna podmiana obrazu i przezroczystości
ObrazKostki.Source = "kostka6.png";
ObrazKostki.Opacity = 0.4; // przygaszenie (np. element zablokowany)

// Budowanie nazwy obrazu na podstawie danych
int wartosc = 3;
ObrazKostki.Source = $"kostka{wartosc}.png"; // → "kostka3.png"
```

ŹRÓDŁA OBRAZU

ŹRÓDŁO	ZAPIS
Zasób aplikacji	Source="Logo.png" (plik z Resources/Images)
Internet (URL)	Source="https://example.com/foto.png"
Plik lokalny	Source="{Binding SciezkaDoPliku}"

TYPOWE ZASTOSOWANIA

- Logo, zdjęcia, ikony.
- Obrazy zmieniane w runtime (kości, statusy).
- Klikalny obraz (z `TapGestureRecognizer`).

TYPOWE BŁĘDY

- Nazwa pliku z wielką literą lub spacją (błąd budowania).
- Brak pliku w `Resources/Images`.
- Zapomnienie o przebudowaniu projektu po dodaniu obrazu.

Nazwy plików obrazów muszą być pisane **małymi literami**, bez spacji i myślników (`kostka1.png` , nie `Kostka 1.png`). Po dodaniu nowego obrazu **przebuduj** projekt.

17.2. `ActivityIndicator`

`ActivityIndicator` to animowana „kręciotka” (spinner) sygnalizująca, że **aplikacja pracuje** - pobiera dane, zapisuje plik, czeka na serwer. Sterujemy nim właściwością `IsRunning` (uruchamia animację) i `IsVisible`.

Pokazuje użytkownikowi, że trwa **operacja o nieznanym czasie**. Bez niego aplikacja podczas ładowania sprawia wrażenie zawieszonej.

NAJWAŻNIEJSZE WŁAŚCIWOŚCI

WŁAŚCIWOŚĆ	OPIS	PRZYKŁAD
<code>IsRunning</code>	czy animacja działa	<code>IsRunning="True"</code>
<code>IsVisible</code>	czy widoczny	<code>IsVisible="True"</code>
<code>Color</code>	kolor wskaźnika	<code>Color="#2196F3"</code>

PRZYKŁAD PODSTAWOWY

```
<ActivityIndicator x:Name="Loader" IsRunning="False" IsVisible="False"
                  Color="#2196F3" />
```

PRZYKŁAD W C#

```
private async void OnPobierz(object sender, EventArgs e)
{
    Loader.IsVisible = Loader.IsRunning = true; // pokaż spinner
    try
    {
        await Task.Delay(2000); // symulacja długiej operacji
    }
    finally
    {
        Loader.IsVisible = Loader.IsRunning = false; // ukryj spinner
    }
}
```

TYPOWE ZASTOSOWANIA

- Pobieranie danych z API.
- Zapis do bazy lub pliku.

TYPOWE BŁĘDY

- Brak ukrycia wskaźnika po zakończeniu (kręci się w nieskończoność).
- Ukrycie poza blokiem `finally` (pozostaje przy błędzie).

Ukrywanie wskaźnika umieszczaj w bloku `finally` - zniknie nawet, gdy operacja zakończy się błędem.

17.3. ProgressBar

`ProgressBar` pokazuje **postęp znanej operacji** - w odróżnieniu od `ActivityIndicator`, informuje, jaka część pracy została wykonana. Sterujemy nim właściwością `Progress` w zakresie od `0.0` (0%) do `1.0` (100%).

NAJWAŻNIEJSZE WŁAŚCIWOŚCI

WŁAŚCIWOŚĆ	OPIS	PRZYKŁAD
<code>Progress</code>	postęp 0.0–1.0	<code>Progress="0.5"</code>
<code>ProgressColor</code>	kolor paska	<code>ProgressColor="Green"</code>

PRZYKŁAD PODSTAWOWY

```
<ProgressBar x:Name="Pasek" Progress="0" ProgressColor="#4CAF50" />
<Button Text="Start" Clicked="OnStart" />
```

PRZYKŁAD W C#

```
private async void OnStart(object sender, EventArgs e)
{
    for (int i = 0; i <= 10; i++)
    {
        Pasek.Progress = i / 10.0; // 0.0 .. 1.0
        await Task.Delay(200);
    }
}
```

TYPOWE ZASTOSOWANIA

- Postęp pobierania pliku.
- Postęp wieloetapowego formularza.

TYPOWE BŁĘDY

- Przekazanie wartości spoza zakresu 0–1 (np. procentów 0–100).

`Progress` przyjmuje wartość od `0.0` do `1.0`. Jeśli masz procenty (0–100), podziel je przez 100: `Pasek.Progress = procent / 100.0;`.

17.4. BoxView

`BoxView` to najprostsza kontrolka graficzna - **kolorowy prostokąt**. Mimo prostoty bywa bardzo przydatny jako separator (cienka linia), element dekoracyjny czy wskaźnik koloru/statusu.

NAJWAŻNIEJSZE WŁAŚCIWOŚCI

WŁAŚCIWOŚĆ	OPIS	PRZYKŁAD
<code>Color</code>	kolor wypełnienia	<code>Color="LightGray"</code>
<code>WidthRequest</code>	szerokość	<code>WidthRequest="14"</code>
<code>HeightRequest</code>	wysokość	<code>HeightRequest="1"</code>
<code>CornerRadius</code>	zaokrąglenie	<code>CornerRadius="7"</code>

PRZYKŁAD PODSTAWOWY

```
<VerticalStackLayout Spacing="10">
  <Label Text="Sekcja A" />
  <BoxView Color="LightGray" HeightRequest="1" /> <!-- separator -->
  <Label Text="Sekcja B" />
  <HorizontalStackLayout Spacing="8">
    <BoxView Color="Green" WidthRequest="14" HeightRequest="14" />
    <Label Text="Aktywny" VerticalOptions="Center" />
  </HorizontalStackLayout>
</VerticalStackLayout>
```

PRZYKŁAD W C#

```
// Wskaźnik koloru sterowany stanem
StatusKropka.Color = aktywne ? Colors.Green : Colors.Red;
```

TYPOWE ZASTOSOWANIA

- Separator (cienka linia).
- Kropka statusu obok tekstu.
- Podgląd koloru we wzorniku.

TYPOWE BŁĘDY

- Brak ustawienia wysokości/szerokości (`BoxView` nie ma rozmiaru domyślnego).

17.5. Border

`Border` to kontener otaczający **jeden element** obramowaniem, z możliwością zaokrąglenia rogów i tła. To podstawowe narzędzie do budowy **kart** (cards). Zastąpił starszą kontrolkę `Frame`.

NAJWAŻNIEJSZE WŁAŚCIWOŚCI

WŁAŚCIWOŚĆ	OPIS	PRZYKŁAD
<code>Stroke</code>	kolor obramowania	<code>Stroke="#DDDDDD"</code>
<code>StrokeThickness</code>	grubość obramowania	<code>StrokeThickness="1"</code>
<code>StrokeShape</code>	kształt (np. zaokrąglony)	<code>RoundRectangle</code>
<code>BackgroundColor</code>	tło karty	<code>BackgroundColor="White"</code>
<code>Padding</code>	margines wewnętrzny	<code>Padding="16"</code>

PRZYKŁAD PODSTAWOWY

```
<Border Stroke="#DDDDDD" StrokeThickness="1"
  BackgroundColor="White" Padding="16" Margin="10">
  <Border.StrokeShape>
    <RoundRectangle CornerRadius="12" />
  </Border.StrokeShape>
  <VerticalStackLayout Spacing="6">
    <Label Text="Nazwa produktu" FontAttributes="Bold" FontSize="18" />
    <Label Text="Krótki opis" TextColor="Gray" />
    <Label Text="49,99 zł" TextColor="Green" FontAttributes="Bold" />
  </VerticalStackLayout>
</Border>
```

```
</VerticalStackLayout>
</Border>
```

TYPOWE ZASTOSOWANIA

- Karty produktów, kontaktów, wiadomości.
- Wizualne grupowanie treści.

TYPOWE BŁĘDY

- Próba umieszczenia kilku elementów wprost w `Border` (przyjmuje jeden).

17.6. Frame (starszy kontener)

`Frame` to **starsza** kontrolka kontenera z obramowaniem, cieniem i zaokrągleniem, znana z Xamarin.Forms. W MAUI nadal działa i często spotyka się ją w przykładach, ale w nowym kodzie zaleca się `Border`, który daje większą kontrolę nad wyglądem.

NAJWAŻNIEJSZE WŁAŚCIWOŚCI

WŁAŚCIWOŚĆ	OPIS	PRZYKŁAD
<code>BorderColor</code>	kolor obramowania	<code>BorderColor="LightGray"</code>
<code>CornerRadius</code>	zaokrąglenie rogów	<code>CornerRadius="10"</code>
<code>HasShadow</code>	czy ma cień	<code>HasShadow="True"</code>
<code>Padding</code>	margines wewnętrzny	<code>Padding="16"</code>

PRZYKŁAD PODSTAWOWY

```
<Frame BorderColor="LightGray" CornerRadius="10" Padding="16" HasShadow="True">
  <Label Text="Treść w ramce" />
</Frame>
```

Na co uważać:

`Frame` jest prosty, ale w nowych projektach preferuj `Border` - jest bardziej elastyczny (dowolne kształty przez `StrokeShape`) i zgodny z kierunkiem rozwoju MAUI. `Frame` warto znać głównie po to, by rozumieć starsze przykłady.

17.7. Porównanie kontrolek wyświetlania

NAJWAŻNIEJSZE INFORMACJE

KONTROLKA	POKAZUJE	TYPOWE UŻYCIE
<code>Image</code>	grafikę	logo, zdjęcia, ikony stanu
<code>ActivityIndicator</code>	„trwa praca” (nieokreślony czas)	pobieranie, zapis
<code>ProgressBar</code>	postęp (0–100%)	pobieranie z paskiem
<code>BoxView</code>	kolorowy prostokąt	separator, status
<code>Border</code>	obramowanie/karta	grupowanie treści
<code>Frame</code>	starszy kontener z cieniem	starsze projekty

Na co uważać:

Dobierz wskaźnik do sytuacji: gdy znasz postęp - `ProgressBar`; gdy nie - `ActivityIndicator`. Do kart używaj `Border` (nowoczesny) zamiast `Frame` (starszy). `Image` to najczęściej zmieniana dynamicznie kontrolka, więc wróć do niej przy pracy z obrazami.

Wskaźniki postępu i aktywności bardzo poprawiają odbiór aplikacji. Użytkownik, który widzi „kręciolkę” podczas ładowania, wie, że aplikacja działa - bez niej mógłby pomyśleć, że się zawiesiła.

17.8. Pełne tabele atrybutów kontrolki wyświetlania

Image (obraz)

ATRYBUT	TYP	OPIS
Source	ImageSource	źródło obrazu (plik/URL)
Aspect	enum	AspectFit, AspectFill, Fill, Center
Opacity	double	przezroczystość 0–1
IsAnimationPlaying	bool	odtwarzanie animacji (GIF)
WidthRequest / HeightRequest	double	rozmiar

```
<Image Source="logo.png" Aspect="AspectFit" Opacity="1" HeightRequest="120" />
```

```
Obraz.Source = "kostka3.png";           // zasób
Obraz.Source = "https://...";           // z internetu
Obraz.Source = ImageSource.FromFile(sciezka); // plik lokalny
Obraz.Opacity = 0.4;                     // przygaszenie
```

ActivityIndicator (wskaźnik pracy)

ATRYBUT	TYP	OPIS
IsRunning	bool	czy animacja działa
Color	Color	kolor wskaźnika
IsVisible	bool	czy widoczny

```
<ActivityIndicator x:Name="Loader" IsRunning="False" IsVisible="False" Color="#2196F3" />
```

ProgressBar (pasek postępu)

ATRYBUT	TYP	OPIS
Progress	double	postęp 0.0–1.0
ProgressColor	Color	kolor paska

```
<ProgressBar x:Name="Pasek" Progress="0.5" ProgressColor="#4CAF50" />
```

```
Pasek.Progress = procent / 100.0;           // z procentów
await Pasek.ProgressTo(1.0, 500, Easing.Linear); // płynna animacja do 100%
```

BoxView (kolorowy prostokąt)

ATRYBUT	TYP	OPIS
Color	CoLoR	wypełnienie
CornerRadius	CornerRadius	zaokrąglenie rogów
WidthRequest / HeightRequest	double	rozmiar

```
<BoxView Color="LightGray" HeightRequest="1" />           <!-- separator -->
<BoxView Color="Green" WidthRequest="16" HeightRequest="16" CornerRadius="8" /> <!-- kropka -->
```

Border (obramowanie / karta)

ATRYBUT	TYP	OPIS
Stroke	Brush / CoLoR	kolor obramowania
StrokeThickness	double	grubość obramowania
StrokeShape	Shape	kształt (RoundRectangle , Ellipse ...)
StrokeDashArray	kolekcja	wzór linii przerywanej
BackgroundColor	CoLoR	tło
Padding	Thickness	wewnętrzny odstęp

```
<Border Stroke="#DDDDDD" StrokeThickness="1" BackgroundColor="White" Padding="16">
  <Border.StrokeShape>
    <RoundRectangle CornerRadius="12" />
  </Border.StrokeShape>
  <Label Text="Zawartość karty" />
</Border>
```

Frame (starszy kontener)

ATRYBUT	TYP	OPIS
BorderColor	CoLoR	kolor obramowania
CornerRadius	float	zaokrąglenie
HasShadow	bool	cień
Padding	Thickness	wewnętrzny odstęp

```
<Frame BorderColor="LightGray" CornerRadius="10" HasShadow="True" Padding="16">
  <Label Text="Treść w ramce" />
</Frame>
```

Na co uważać: Image.Source w C# przyjmuje napis (nazwę pliku) lub ImageSource.FromFile / FromUri . BoxView używa Color (nie BackgroundColor), a Border - Stroke (nie BorderColor , które należy do starszego Frame). W nowych projektach preferuj Border zamiast Frame .

17.9. TableView - formularze i ustawienia

TableView wyświetla **statyczne** wiersze pogrupowane w sekcje - idealne do ekranów ustawień. Każdy wiersz to gotowa komórka (**TextCell**, **SwitchCell**, **EntryCell**).

KOMÓRKA	ZASTOSOWANIE
TextCell	tekst + opis
SwitchCell	wiersz z przełącznikiem
EntryCell	wiersz z polem tekstowym
ImageCell	tekst + obraz

```
<TableView Intent="Settings">
  <TableRoot>
    <TableSection Title="Ogólne">
      <SwitchCell Text="Powiadomienia" On="True" />
      <SwitchCell Text="Tryb ciemny" />
      <EntryCell Label="Nazwa" Placeholder="Wpisz nazwę" />
    </TableSection>
    <TableSection Title="O aplikacji">
      <TextCell Text="Wersja" Detail="1.0.0" />
    </TableSection>
  </TableRoot>
</TableView>
```

17.10. WebView - strona internetowa w aplikacji

WebView osadza **stronę internetową** lub treść HTML w aplikacji.

ATRYBUT	OPIS
Source	adres URL lub HTML
CanGoBack / CanGoForward	nawigacja

```
<WebView Source="https://example.com" HeightRequest="400" />
```

```
// Wyświetlenie własnego HTML
Przegladarka.Source = new HtmlWebViewSource { Html = "<h1>Witaj</h1><p>Treść.</p>" };
```

17.11. Kształty (Shapes) - Rectangle, Ellipse, Line, Polygon, Path

Przestrzeń **Microsoft.Maui.Controls.Shapes** udostępnia **kształty wektorowe**: **Rectangle**, **Ellipse**, **Line**, **Polyline**, **Polygon**, **Path**. Sterujemy ich wyglądem przez **Fill** (wypełnienie), **Stroke** (kontur), **StrokeThickness**.

```
<!-- Wymaga: xmlns:shapes="clr-namespace:Microsoft.Maui.Controls.Shapes;assembly=Microsoft.Maui.Controls" -->
<Ellipse Fill="LightBlue" Stroke="Blue" StrokeThickness="2"
  WidthRequest="80" HeightRequest="80" />

<Rectangle Fill="LightGreen" RadiusX="10" RadiusY="10"
  WidthRequest="120" HeightRequest="60" />
```

```
<Line X1="0" Y1="0" X2="100" Y2="0" Stroke="Gray" StrokeThickness="2" />

<Polygon Points="0,0 50,0 25,50" Fill="Orange" />
```

17.12. Pozostałe elementy: ToolbarItem, MenuBar, BlazorWebView

ToolbarItem dodaje przycisk akcji na **pasku tytułu** strony (np. „Zapisz”, „Dodaj”). **MenuBar** tworzy menu (głównie desktop). **BlazorWebView** osadza komponenty Blazor (zaawansowane, hybrydowe).

```
<ContentPage.ToolbarItems>
    <ToolbarItem Text="Dodaj" IconImageSource="dodaj.png" Clicked="OnDodaj" />
    <ToolbarItem Text="Zapisz" Order="Primary" Clicked="OnZapisz" />
</ContentPage.ToolbarItems>
```

```
private void OnDodaj(object sender, EventArgs e) { /* akcja z paska tytułu */ }
```

Na co uważać: do długich, dynamicznych list używaj **CollectionView** (wydajny); **ListView** wybieraj dla grupowania i „pull to refresh” bez **RefreshView**. **CarouselView** + **IndicatorView** to zestaw na banery/onboarding. **TableView** jest świetny do statycznych ustawień. **ToolbarItem** to standardowe miejsce na główne akcje ekranu.

Wiele właściwości powtarza się w różnych kontrolkach. Warto poznać je raz, dobrze, bo używamy ich nieustannie. Ten rozdział zbiera **najważniejsze właściwości** w przejrzystych tabelach oraz wyjaśnia często mylone pary przez **porównania**. To rozdział referencyjny - wracaj do niego, gdy nie pamiętasz, za co odpowiada dana właściwość.

18. Kontrolki listowe i widoki kolekcji

18.1. CollectionView - pełna tabela atrybutów

CollectionView to nowoczesna kontrolka listy. Wyświetla kolekcję podpiętą do **ItemsSource**, używając szablonu **ItemTemplate**. Obsługuje wybór, różne układy (lista pionowa/pozioma, siatka), nagłówki/stopkę i pusty stan.

ATRYBUT	TYP	OPIS
ItemsSource	IEnumerable	kolekcja danych
ItemTemplate	DataTemplate	wygląd pojedynczego elementu
ItemsLayout	IItemsLayout	układ (lista/siatka, pion/poziom)
SelectionMode	enum	None , Single , Multiple
SelectedItem	object	zaznaczony element
SelectedItems	IList	zaznaczone (tryb Multiple)
EmptyView	object	widok przy pustej liście
Header / Footer	object	nagłówek / stopka listy
ItemsUpdatingScrollMode	enum	zachowanie przewijania przy dodaniu
VerticalScrollBarVisibility	enum	widoczność paska przewijania

```

<CollectionView x:Name="Lista" SelectionMode="Single" SelectionChanged="OnWybor">
    <CollectionView.ItemsLayout>
        <GridItemsLayout Orientation="Vertical" Span="2" /> <!-- siatka 2 kolumny -->
    </CollectionView.ItemsLayout>
    <CollectionView.EmptyView>
        <Label Text="Brak danych" HorizontalOptions="Center" TextColor="Gray" />
    </CollectionView.EmptyView>
    <CollectionView.ItemTemplate>
        <DataTemplate>
            <Border Padding="10" Margin="5">
                <VerticalStackLayout>
                    <Label Text="{Binding Nazwa}" FontAttributes="Bold" />
                    <Label Text="{Binding Cena, StringFormat='{0:0.00} zł'}" TextColor="Green" />
                </VerticalStackLayout>
            </Border>
        </DataTemplate>
    </CollectionView.ItemTemplate>
</CollectionView>

```

```

private void OnWybor(object sender, SelectionChangedEventArgs e)
{
    if (e.CurrentSelection.FirstOrDefault() is Produkt p)
        Szczegoly.Text = p.Nazwa;
    ((CollectionView)sender).SelectedItem = null; // odznacz po wejściu
}

```

Układ listy ustawiamy przez **ItemsLayout**: **LinearItemsLayout** (lista) lub **GridItemsLayout** (siatka). Można też użyć skróconych wartości **Vertical** / **Horizontal**.

```

<!-- Lista pozioma -->
<CollectionView ItemsLayout="HorizontalList" ... />

<!-- Siatka 3 kolumny -->
<CollectionView>
    <CollectionView.ItemsLayout>
        <GridItemsLayout Orientation="Vertical" Span="3" HorizontalItemSpacing="8" VerticalItemSpacing="8" />
    </CollectionView.ItemsLayout>
</CollectionView>

```

18.2. ListView - pełna tabela atrybutów

ListView to starsza kontrolka listy (każdy element w **ViewCell**). W nowych projektach preferuj **CollectionView**, ale **ListView** ma wbudowane nagłówki sekcji, grupowanie i „pociągnij, by odświeżyć”.

ATRYBUT	TYP	OPIS
ItemsSource	IEnumerable	dane
ItemTemplate	DataTemplate	szablon (z ViewCell)
SelectedItem	object	zaznaczony element
HasUnevenRows	bool	wiersze o różnej wysokości
RowHeight	int	stała wysokość wiersza
IsGroupingEnabled	bool	grupowanie
IsPullToRefreshEnabled	bool	pociągnij, by odświeżyć
IsRefreshing	bool	stan odświeżania

```
<ListView x:Name="ListaLV" HasUnevenRows="True" ItemTapped="OnTap">
  <ListView.ItemTemplate>
    <DataTemplate>
      <ViewCell>
        <Grid Padding="10" ColumnDefinitions="*,Auto">
          <Label Text="{Binding Nazwa}" Grid.Column="0" />
          <Label Text="{Binding Cena}" Grid.Column="1" />
        </Grid>
      </ViewCell>
    </DataTemplate>
  </ListView.ItemTemplate>
</ListView>
```

```
private void OnTap(object sender, ItemTappedEventArgs e)
{
    if (e.Item is Produkt p) Szczegoly.Text = p.Nazwa;
}
```

18.3. CarouselView i IndicatorView

CarouselView wyświetla elementy jako przewijane karuzelą (jeden duży element naraz, przesuwany w bok) - idealne do banerów czy wprowadzenia. IndicatorView pokazuje kropki wskazujące bieżącą pozycję karuzeli.

ATRYBUT (CAROUSELVIEW)	OPIS
ItemsSource	dane
ItemTemplate	szablon elementu
Position	indeks bieżącego elementu
CurrentItem	bieżący element
IndicatorView	powiązany wskaźnik kropek
Loop	zapętlanie przewijania

```

<CarouselView x:Name="Karuzela" IndicatorView="Wskaznik">
    <CarouselView.ItemTemplate>
        <DataTemplate>
            <Image Source="{Binding .}" Aspect="AspectFill" />
        </DataTemplate>
    </CarouselView.ItemTemplate>
</CarouselView>

<IndicatorView x:Name="Wskaznik" IndicatorColor="LightGray"
    SelectedIndicatorColor="#2196F3" HorizontalOptions="Center" />

```

```
Karuzela.ItemsSource = new List<string> { "baner1.png", "baner2.png", "baner3.png" };
```

18.4. RefreshView - pociągnij, by odświeżyć

RefreshView opakowuje przewijaną treść (np. **CollectionView**) i dodaje gest „pociągnij w dół, by odświeżyć”.

ATRYBUT	TYP	OPIS
IsRefreshing	bool	czy trwa odświeżanie
RefreshColor	Color	kolor wskaźnika

```

<RefreshView x:Name="Odswiezanie" Refreshing="OnOdswiez">
    <CollectionView ItemsSource="{Binding Elementy}" />
</RefreshView>

```

```

private async void OnOdswiez(object sender, EventArgs e)
{
    Odswiezanie.IsRefreshing = true;
    await ZaladujDane();           // pobierz świeże dane
    Odswiezanie.IsRefreshing = false;
}

```

18.5. SwipeView – gesty przesunięcia na elemencie

SwipeView dodaje **akcje wysuwane** po przesunięciu elementu w bok (np. usuń/edytuj na elemencie listy).

```

<SwipeView>
    <SwipeView.RightItems>
        <SwipeItems>
            <SwipeItem Text="Usuń" BackgroundColor="Red" Invoked="OnUsun" />
            <SwipeItem Text="Edytuj" BackgroundColor="Orange" Invoked="OnEdytuj" />
        </SwipeItems>
    </SwipeView.RightItems>
    <Grid Padding="16" BackgroundColor="White">
        <Label Text="{Binding Nazwa}" />
    </Grid>
</SwipeView>

```

19. Wspólne właściwości kontrolek

19.1. Właściwości treści: Text, Content, Placeholder, Source

NAJWAŻNIEJSZE INFORMACJE

WŁAŚCIWOŚĆ	WYSTĘPUJE W	CO USTAWIA
Text	Label, Entry, Editor, Button	wyświetlany lub wpisany tekst
Content	ContentPage, Border, ContentView, ScrollView	jedyny element potomny (kontener)
Placeholder	Entry, Editor, SearchBar	szara podpowiedź w pustym polu
Source	Image, ImageButton	źródło obrazu (plik/URL)

Na co uważać:

Text to tekst, a Content to element (np. layout). Label ma Text, ale Border ma Content. To częsta pomyłka: do Border nie wpiszesz Text, tylko umieścisz w nim kontrolkę przez Content.

NAJWAŻNIEJSZE INFORMACJE

WŁAŚCIWOŚĆ	WYSTĘPUJE W	ZNACZENIE
IsVisible	każda kontrolka	czy element jest widoczny
IsEnabled	każda kontrolka	czy element jest aktywny (reaguje)
IsToggled	Switch	czy włączony

PRZYKŁAD C#

```
Komunikat.IsVisible = true; // pokaż etykietę
PrzyciskZapisz.IsEnabled = false; // zablokuj przycisk
bool zgoda = Zgoda.IsChecked; // odczyt zaznaczenia
bool tryb = Przełącznik.IsToggled; // odczyt przełącznika
```

Na co uważać:

IsVisible="False" ukrywa element całkowicie (znika z układu), a IsEnabled="False" pokazuje go, ale jako nieaktywny (wyszarzony). To kluczowa różnica - opisana szerzej w porównaniu 13.9.

19.2. Właściwości wyboru: SelectedItem, SelectedIndex, ItemsSource

NAJWAŻNIEJSZE INFORMACJE

WŁAŚCIWOŚĆ	WYSTĘPUJE W	ZNACZENIE
SelectedItem	jw.	wybrany obiekt

PRZYKŁAD C#

```
ListaGatunkow.ItemsSource = new List<string> { "Pies", "Kot" };
if (ListaGatunkow.SelectedIndex != -1)
{
    string wybrany = ListaGatunkow.SelectedItem.ToString();
}
```

Na co uważać:

`SelectedIndex` to numer pozycji (liczba), a `SelectedItem` to sam obiekt. Na starcie `SelectedIndex` wynosi `-1`, a `SelectedItem` jest `null` - zawsze to sprawdzaj przed użyciem.

19.3. Właściwości rozmiaru: `WidthRequest`, `HeightRequest`, `Minimum...`

NAJWAŻNIEJSZE INFORMACJE

WŁAŚCIWOŚĆ	ZNACZENIE
<code>WidthRequest</code>	sugerowana szerokość
<code>HeightRequest</code>	sugerowana wysokość
<code>MinimumWidthRequest</code>	minimalna szerokość
<code>MinimumHeightRequest</code>	minimalna wysokość

Na co uważać:

Słowo „request” (prośba) oznacza, że to **sugestia** - layout może ją zmodyfikować zależnie od dostępnego miejsca. Nie ustawiaj sztywnych rozmiarów wszędzie, bo psuje to responsywność; używaj ich tam, gdzie naprawdę potrzeba.

19.4. Właściwości odstępów: `Margin`, `Padding`

NAJWAŻNIEJSZE INFORMACJE

WŁAŚCIWOŚĆ	GDZIE	CO ROBI
<code>Margin</code>	dowolna kontrolka	odstęp na zewnątrz
<code>Padding</code>	kontenery	odstęp wewnątrz

Zapis: jedna liczba (wszystkie strony), dwie (`poziom`, `pion`), cztery (`lewo`, `góra`, `prawo`, `dół`).

Na co uważać:

To jedna z najczęściej mylonych par - szczegółowe porównanie w 13.8.

19.5. Właściwości wyrównania: `HorizontalOptions`, `VerticalOptions`

NAJWAŻNIEJSZE INFORMACJE

WARTOŚĆ	DZIAŁANIE
<code>Start</code>	do lewej/góry
<code>Center</code>	wyśrodkowanie
<code>End</code>	do prawej/dółu
<code>Fill</code>	rozciągnięcie na całą przestrzeń

Na co uważać:

Domyślnie wiele kontrolki używa `Fill`, dlatego np. przycisk zajmuje pełną szerokość. Ustaw `Center`, by zostawić go w naturalnym rozmiarze.

19.6. Właściwości wyglądu: BackgroundColor, TextColor, FontSize, FontAttributes, Opacity, Minimum, Maximum, Value

NAJWAŻNIEJSZE INFORMACJE

WŁAŚCIWOŚĆ	WYSTĘPUJE W	ZNACZENIE
BackgroundColor	większość kontroltek	kolor tła
TextColor	kontrolki tekstowe	kolor tekstu
FontSize	kontrolki tekstowe	rozmiar czcionki
FontAttributes	kontrolki tekstowe	Bold, Italic, None
Opacity	każda kontrolka	przezroczystość 0–1
Minimum / Maximum	Slider, Stepper, ProgressBar	zakres wartości
Value	Slider, Stepper	aktualna wartość

PRZYKŁAD C#

```
Tytuł.BackgroundColor = Colors.LightYellow;
Tytuł.TextColor = Colors.DarkBlue;
Tytuł.FontSize = 24;
Tytuł.FontAttributes = FontAttributes.Bold;
Obraz.Opacity = 0.5; // półprzezroczysty
```

Na co uważać:

Opacity przyjmuje wartości od 0.0 (niewidoczny) do 1.0 (pełne krycie). Wartość pośrednia (np. 0.4) daje efekt „przygaszenia” - przydatny do oznaczania elementów nieaktywnych bez ich ukrywania.

19.7. Porównanie: Margin a Padding

NAJWAŻNIEJSZE INFORMACJE

To najczęściej mylona para odstępów.

CECHA	MARGIN	PADDING
Działa	na zewnątrz kontrolki	wewnątrz kontenera
Odsuwa	element od sąsiadów	zawartość od krawędzi
Dotyczy	każdej kontrolki	tylko kontenerów

```
←!— Padding: odstęp od krawędzi layoutu do dzieci →
←!— Margin: odstęp przycisku od sąsiadów →
<VerticalStackLayout Padding="20">
  <Button Text="OK" Margin="0,10,0,0" />
</VerticalStackLayout>
```

Na co uważać:

Margines odpycha element od otoczenia, padding odpycha zawartość od krawędzi kontenera. Gdy element „przykleja się” do sąsiada - dodaj Margin. Gdy treść „przykleja się” do krawędzi kontenera - dodaj Padding.

19.8. Porównanie: IsVisible a IsEnabled

NAJWAŻNIEJSZE INFORMACJE

CECHA	ISVISIBLE	ISENABLED
<code>False</code> powoduje	element znika z układu	element widoczny, ale nieaktywny
Zajmuje miejsce	nie (gdy ukryty)	tak
Typowe użycie	pokazywanie/ukrywanie sekcji	blokowanie przycisku

Na co uważać:

Wybór zależy od intencji: jeśli element ma **zniknąć** - `IsVisible="False"`; jeśli ma być **widoczny, lecz niedostępny** (wyszarzony) - `IsEnabled="False"`. Np. przycisk „Zapłać” zwykle blokujemy (`IsEnabled`), a komunikat błędu ukrywamy (`IsVisible`).

19.9. Porównanie: Text a Placeholder

NAJWAŻNIEJSZE INFORMACJE

CECHA	TEXT	PLACEHOLDER
Co to	realna wartość	podpowiedź
Znika po wpisaniu	nie	tak
Odczytujemy w C#	tak	nie (to tylko hint)

Na co uważać:

`Placeholder` to tylko wizualna podpowiedź - **nie jest** wartością pola. Realną zawartość wpisaną przez użytkownika czytamy zawsze z `Text`. Mylenie tych dwóch to częsty błąd przy pobieraniu danych.

19.10. Porównanie: SelectedItem a SelectedIndex

NAJWAŻNIEJSZE INFORMACJE

CECHA	SELECTEDITEM	SELECTEDINDEX
Zwraca	wybrany obiekt	numer pozycji (od 0)
Gdy nic nie wybrano	<code>null</code>	<code>-1</code>
Typowe użycie	praca na obiekcie	sprawdzenie, czy coś wybrano

Na co uważać:

Najpierw sprawdź `SelectedIndex  $\neq$  -1`, a dopiero potem użyj `SelectedItem`. To dwie strony tej samej monety: indeks mówi „która pozycja”, a item daje „sam obiekt”.

19.11. Porównanie: List a ObservableCollection

NAJWAŻNIEJSZE INFORMACJE

CECHA	LIST<T>	OBSERVABLECOLLECTION<T>
Powiadamia widok o zmianach	nie	tak
Odświeżanie listy po dodaniu	brak/ręczne	automatyczne
Typowe użycie	dane robocze	listy na ekranie

Na co uważać:

To kluczowa różnica przy listach. Gdy lista **wyświetlana** ma się odświeżać po dodaniu elementu - użyj `ObservableCollection<T>`. Zwykła `List<T>` nie powiadomi widoku i lista „nie zareaguje” na zmiany. To najczęstszy błąd przy pracy z listami.

19.12. Porównanie: Content a Text

NAJWAŻNIEJSZE INFORMACJE

CECHA	CONTENT	TEXT
Przyjmuje	element (kontrolkę/layout)	napis
Występuje w	kontenerach (<code>Border</code> , <code>ContentPage</code>)	kontrolkach tekstowych (<code>Label</code> , <code>Entry</code>)

Na co uważać:

Do kontenera (`Border` , `ScrollView` , `ContentPage`) wstawiasz **element** przez `Content` . Do kontrolki tekstowej wpisujesz **napis** przez `Text` . Nie pomył tych dwóch - `Border` nie ma `Text` , a `Label` nie ma `Content` .

19.13. Porównanie: Source jako zasób, URL i plik lokalny

NAJWAŻNIEJSZE INFORMACJE

ŹRÓDŁO	ZAPIS	UWAGI
Zasób aplikacji	<code>Source="logo.png"</code>	plik z <code>Resources/Images</code> , małe litery
URL (internet)	<code>Source="https:// ..."</code>	wymaga połączenia z siecią
Plik lokalny	<code>ImageSource.FromFile(ściezka)</code>	zapisany na urządzeniu

PRZYKŁAD C#

```
// Zasób aplikacji
Obraz.Source = "logo.png";

// Z internetu
Obraz.Source = "https://example.com/foto.png";

// Z pliku na urządzeniu
Obraz.Source = ImageSource.FromFile(ściezkaDoPliku);
```

Na co uważać:

Obraz z zasobów wymaga poprawnej nazwy (małe litery, bez spacji). Obraz z URL wymaga sieci i może się nie załadować (warto przewidzieć obraz zastępczy). Obraz z pliku lokalnego podajemy przez pełną ścieżkę (zwykle w `AppDataDirectory`).

Ten rozdział to Twoja „ściągą” z właściwości. Gdy nie pamiętasz, czy użyć `Text` czy `Content` , `IsVisible` czy `IsEnabled` , `List` czy `ObservableCollection` - wróć do odpowiedniego porównania. Znajomość tych par różnic eliminuje większość typowych pomyłek.

19.14. Wspólne właściwości wszystkich kontroltek

Każda kontrolka w MAUI dziedziczy po wspólnych klasach bazowych (`VisualElement` , `View` , `Element`), dzięki czemu **ma ten sam zestaw podstawowych właściwości** - niezależnie od tego, czy to `Label` , `Button` , `Image` czy layout. Poniżej kompletny wykaz tych wspólnych atrybutów, pogrupowany tematycznie. Te właściwości możesz ustawić na **dowolnej** kontrolce, w XAML i w C#.

Rozmiar i położenie

WŁAŚCIWOŚĆ	TYP	OPIS
WidthRequest	double	sugerowana szerokość
HeightRequest	double	sugerowana wysokość
MinimumWidthRequest	double	minimalna szerokość
MinimumHeightRequest	double	minimalna wysokość
MaximumWidthRequest	double	maksymalna szerokość
MaximumHeightRequest	double	maksymalna wysokość
Width	double	rzeczywista szerokość (tylko odczyt)
Height	double	rzeczywista wysokość (tylko odczyt)
HorizontalOptions	LayoutOptions	wyrównanie poziome (Start / Center / End / Fill)
VerticalOptions	LayoutOptions	wyrównanie pionowe
Margin	Thickness	odstęp na zewnątrz

Wygląd i tło

WŁAŚCIWOŚĆ	TYP	OPIS
BackgroundColor	Color	kolor tła
Background	Brush	tło (np. gradient)
Opacity	double	przezroczystość 0.0–1.0
Shadow	Shadow	cień rzucany przez element
Clip	Geometry	przycięcie kształtem

Widoczność i stan

WŁAŚCIWOŚĆ	TYP	OPIS
IsVisible	bool	czy element jest widoczny (i zajmuje miejsce)
IsEnabled	bool	czy element jest aktywny (reaguje)
InputTransparent	bool	czy element przepuszcza dotyk „przez siebie”

Transformacje (przekształcenia wizualne)

WŁAŚCIWOŚĆ	TYP	OPIS
Scale	double	skala (1.0 = normalna)
ScaleX / ScaleY	double	skala w jednej osi
Rotation	double	obrót w stopniach
RotationX / RotationY	double	obrót w przestrzeni 3D
TranslationX / TranslationY	double	przesunięcie względem pozycji
AnchorX / AnchorY	double	punkt zaczepienia transformacji (0–1)
ZIndex	int	kolejność nakładania (wyższy = na wierzchu)

Identyfikacja i pozostałe

WŁAŚCIWOŚĆ	TYP	OPIS
<code>x:Name</code>	-	nazwa do odwołań z C#
<code>ClassId</code>	<code>string</code>	dowolny identyfikator (np. do rozróżniania)
<code>StyleId</code>	<code>string</code>	identyfikator stylu
<code>Style</code>	<code>Style</code>	przypisany styl
<code>BindingContext</code>	<code>object</code>	źródło danych dla bindingu
<code>FlowDirection</code>	enum	kierunek układu (LTR/RTL)
<code>GestureRecognizers</code>	kolekcja	rozpoznawanie gestów (np. dotknięcie)

←! Wspólne właściwości na przykładzie Label (działają na każdej kontrolce) →

```
<Label Text="Przykład"
      x:Name="Etykieta"
      WidthRequest="200"
      HeightRequest="60"
      Margin="10"
      BackgroundColor="LightYellow"
      Opacity="0.9"
      HorizontalOptions="Center"
      VerticalOptions="Start"
      Rotation="0"
      Scale="1"
      IsVisible="True"
      IsEnabled="True" />
```

```
// Te same właściwości ustawione z kodu (na dowolnej kontrolce)
Etykieta.WidthRequest = 200;
Etykieta.Margin = new Thickness(10); // jednolity margines
Etykieta.Margin = new Thickness(10, 5, 10, 5); // lewo, góra, prawo, dół
Etykieta.BackgroundColor = Colors.LightYellow;
Etykieta.Opacity = 0.9;
Etykieta.HorizontalOptions = LayoutOptions.Center;
Etykieta.IsVisible = false; // ukryj
Etykieta.IsEnabled = false; // zablokuj
Etykieta.Scale = 1.2; // powiększ o 20%
Etykieta.Rotation = 15; // obróć o 15 stopni
Etykieta.TranslationX = 30; // przesun w prawo o 30
```

Skoro te właściwości ma **każda** kontrolka, opanowanie ich raz pozwala sterować wyglądem i zachowaniem wszystkich elementów. Najczęściej używane to `WidthRequest`, `HeightRequest`, `Margin`, `BackgroundColor`, `Opacity`, `IsVisible`, `IsEnabled` oraz `HorizontalOptions`, `VerticalOptions`.

19.15. Typ Thickness - zapis marginesów i paddingów

Marginesy (`Margin`) i wypełnienia (`Padding`) używają typu `Thickness`, który określa odstępy z czterech stron. W XAML zapisujemy go jako jedną, dwie lub cztery liczby; w C# tworzymy obiekt `new Thickness(...)`.

ZAPIS	ZNACZENIE
<code>Margin="20"</code>	20 ze wszystkich stron
<code>Margin="20,10"</code>	20 lewo+prawo, 10 góra+dół
<code>Margin="5,10,15,20"</code>	lewo=5, góra=10, prawo=15, dół=20

```
<Button Text="A" Margin="20" />      <!-- wszystkie strony -->
<Button Text="B" Margin="20,10" />    <!-- poziomo, pionowo -->
<Button Text="C" Margin="5,10,15,20" /> <!-- L, G, P, D -->
<VerticalStackLayout Padding="16,24" /> <!-- padding też to Thickness -->
```

```
przycisk.Margin = new Thickness(20);      // wszystkie strony
przycisk.Margin = new Thickness(20, 10);   // poziom, pion
przycisk.Margin = new Thickness(5, 10, 15, 20); // L, G, P, D
layout.Padding = new Thickness(16, 24);
```

Na co uważać: kolejność czterech wartości to **lewo, góra, prawo, dół** (zgodnie z ruchem wskazówek zegara od lewej). Dwie wartości to „poziom, pion”. `Padding` (wewnątrz kontenera) i `Margin` (na zewnątrz kontrolki) używają tego samego typu `Thickness`.

19.16. Podsumowanie kontrolek

KONTROLKA	TYP	GŁÓWNE ZDARZENIE	KLUCZOWE WŁAŚCIWOŚCI
Label	Wyświetlanie tekstu	-	Text, FormattedText, FontSize, TextColor
Entry	Pole jednoliniowe	TextChanged, Completed	Keyboard, IsPassword, MaxLength, ClearButtonVisibility
Editor	Pole wieloliniowe	TextChanged, Completed	AutoSize, MaxLength, Placeholder
SearchBar	Pole wyszukiwania	TextChanged, SearchButtonPressed	Placeholder, Text
Switch	Przełącznik	Toggled	IsToggled, OnColor, ThumbColor

20. Zdarzenia, gesty i code-behind

Aplikacje mobilne są **sterowane zdarzeniami** (event-driven): użytkownik coś robi (klika, wpisuje, przesuwa), a aplikacja reaguje. Zrozumienie zdarzeń jest absolutnie kluczowe - to one ożywiają interfejs. W tym rozdziale wyjaśniamy od zera, czym jest zdarzenie i handler, jak je podpinąć, jak czytać parametry `sender` i `e`, omawiamy najważniejsze zdarzenia oraz gesty, a na końcu pokazujemy uniwersalny schemat: **akcja użytkownika → zmiana stanu → aktualizacja interfejsu**.

20.1. Czym jest zdarzenie

Zdarzenie (event) to **sygnał**, że coś się stało - użytkownik nacisnął przycisk, zmienił tekst, przesunął suwak. Kontrolki „wysyłają” zdarzenia, a my możemy na nie zareagować, podpinając do nich **metodę obsługi** (handler). Gdy zdarzenie wystąpi, MAUI automatycznie wywoła naszą metodę.

Zdarzenia pozwalają aplikacji **reagować na działania użytkownika**. Bez nich interfejs byłby statyczny. Dzięki zdarzeniom kliknięcie przycisku uruchamia obliczenie, a przesunięcie suwaka zmienia rozmiar tekstu.

NAJWAŻNIEJSZE INFORMACJE

- Kontrolka **wysyła** zdarzenie, my **reagujemy** handlerem.
- Handler to metoda o sygnaturze `(object sender, EventArgs e)`.
- Zdarzenie podpinamy w XAML (atrybut) lub w C# (operator `+=`).

Na co uważać:

20.2. Czym jest event handler

Event handler (metoda obsługi zdarzenia) to **metoda wywoływana, gdy wystąpi zdarzenie**. Ma ustaloną sygnaturę: pierwszy parametr `object sender` (kontrolka, która wysłała zdarzenie), drugi - argumenty zdarzenia (`EventArgs` lub typ pochodny z dodatkowymi danymi).

PRZYKŁAD C#

```
// Typowy handler zdarzenia Clicked
private void OnKliknij(object sender, EventArgs e)
{
    Komunikat.Text = "Kliknięto!";
}
```

NAJWAŻNIEJSZE INFORMACJE

- Sygnatura: `(object sender, EventArgs e)`.
- Nazwa metody jest dowolna, ale przyjęło się przedrostek `On...`.
- Handler musi pasować typem argumentów do zdarzenia.

Na co uważać:

20.3. Podpinanie zdarzeń w XAML i w C#

PRZYKŁAD XAML

```
<!-- Podpięcie w XAML -->
<Button Text="Kliknij" Clicked="OnKliknij" />
```

PRZYKŁAD W C#

```
// Podpięcie w code-behind operatorem +=
public MainPage()
{
    InitializeComponent();
    PrzyciskDynamiczny.Clicked += OnKliknij;
}

// Tworzenie kontrolki i zdarzenia w całości w C#
var przycisk = new Button { Text = "Dynamiczny" };
przycisk.Clicked += (s, e) => Komunikat.Text = "Kliknięto dynamiczny";
Layout.Children.Add(przycisk);
```

TYPOWE BŁĘDY

- W XAML: nazwa metody niezgodna z code-behind.
- W C#: użycie `=` zamiast `+=` przy podpinaniu.

Na co uważać:

Podpinanie w XAML jest preferowane dla kontroltek zdefiniowanych w widoku. Operator `+=` w C# stosuj dla kontroltek tworzonych dynamicznie. Możesz też użyć wyrażenia lambda `(s, e) => ...` jako krótkiego handlera.

20.4. Parametry sender i e

Każdy handler dostaje dwa parametry. `sender` to obiekt, który wysłał zdarzenie - czyli kontrolka (np. przycisk). Rzutując go na właściwy typ, możemy odczytać jego właściwości. `e` to argumenty zdarzenia - zawiera dodatkowe dane specyficzne dla zdarzenia (np. `e.NewValue` dla suwaka, `e.Value` dla przełącznika).

PRZYKŁAD C#

```
// Rozpoznanie kontrolki przez sender
private void OnDowolnyPrzycisk(object sender, EventArgs e)
{
    var przycisk = (Button)sender;          // rzutowanie
    Komunikat.Text = $"Kliknięto: {przycisk.Text}";
}

// Odczyt danych zdarzenia z e
private void OnSuwak(object sender, ValueChangedEventArgs e)
{
    double nowa = e.NewValue; // nowa wartość suwaka
    double stara = e.OldValue; // poprzednia wartość
}
```

NAJWAŻNIEJSZE INFORMACJE

PARAMETR	ZAWIERA
<code>sender</code>	kontrolkę, która wysłała zdarzenie
<code>e</code>	dodatkowe dane zdarzenia

Na co uważać:

`sender` ma typ `object`, więc aby sięgnąć po właściwości kontrolki, trzeba go rzutować (`(Button)sender`). To pozwala obsłużyć wiele kontrolek jednym handlerem. Dane w `e` zależą od zdarzenia - korzystaj z nich, zamiast odczytywać kontrolkę po `x.Name`.

20.5. Najważniejsze zdarzenia - przegląd

NAJWAŻNIEJSZE INFORMACJE

Poniższa tabela zbiera najczęściej używane zdarzenia w MAUI:

ZDARZENIE	KONTROLKA	KIEDY WYSTĘPUJE	TYP ARGUMENTU
<code>TextChanged</code>	<code>Entry</code> , <code>Editor</code> , <code>SearchBar</code>	przy każdej zmianie tekstu	<code>TextChangedEventArgs</code>
<code>Completed</code>	<code>Entry</code>	po naciśnięciu Enter	<code>EventArgs</code>
<code>Toggled</code>	<code>Switch</code>	po przełączeniu	<code>ToggledEventArgs</code>
<code>SelectionChanged</code>	<code>CollectionView</code>	po zmianie zaznaczenia	<code>SelectionChangedEventArgs</code>
<code>ValueChanged</code>	<code>Slider</code> , <code>Stepper</code>	przy zmianie wartości	<code>ValueChangedEventArgs</code>
<code>Unfocused</code>	pola tekstowe	gdy pole traci fokus	<code>FocusEventArgs</code>
<code>Loaded</code>	każda kontrolka	gdy element został załadowany	<code>EventArgs</code>

Na co uważać:

20.6. Przykłady najważniejszych zdarzeń

PRZYKŁAD: TEXTCHANGED

```
private void OnTekstZmieniony(object sender, TextChangedEventArgs e)
{
    string nowy = e.NewTextValue; // aktualny tekst
    LicznikZnakow.Text = $"Znaków: {nowy?.Length ?? 0}";
}
```

PRZYKŁAD: COMPLETED

```
private void OnZatwierdzono(object sender, EventArgs e)
{
    // wykonuje się po naciśnięciu Enter w polu Entry
    Szukaj(((Entry)sender).Text);
}
```

PRZYKŁAD: SELECTIONCHANGED (COLLECTIONVIEW)

```
private void OnWybor(object sender, SelectionChangedEventArgs e)
{
    if (e.CurrentSelection.FirstOrDefault() is Produkt p)
        Szczegoly.Text = $"Wybrano: {p.Nazwa}";
}
```

Na co uważać:

Każde zdarzenie dostarcza w `e` przydatne dane: `TextChanged` → `e.NewTextValue` / `e.OldTextValue`, `ValueChanged` → `e.NewValue` / `e.OldValue`, `SelectionChanged` → `e.CurrentSelection`. Korzystaj z nich zamiast ponownie odczytywać kontrolkę.

20.7. Gesty i TapGestureRecognizer

Gesty pozwalają reagować na dotyk elementów, które same nie są przyciskami: klikalny obraz, kafelek, cała sekcja. To kluczowe w grach i interaktywnych ekranach opartych na obrazach.

PRZYKŁAD XAML

```
<Image Source="zdjecie.png" HeightRequest="80">
    <Image.GestureRecognizers>
        <TapGestureRecognizer Tapped="OnObrazDotkniete" />
    </Image.GestureRecognizers>
</Image>
```

PRZYKŁAD W C#

```
private void OnObrazDotknieto(object sender, EventArgs e)
{
    var obraz = (Image)sender;
    // np. przełączenie przezroczystości po dotknięciu
    obraz.Opacity = obraz.Opacity == 1.0 ? 0.4 : 1.0;
}
```

INNE GESTY

GEST	KLASA	WYKRYWA
Dotknięcie	<code>TapGestureRecognizer</code>	pojedyncze/podwójne dotknięcie
Przesunięcie	<code>SwipeGestureRecognizer</code>	szybki ruch w kierunku
Przeciąganie	<code>PanGestureRecognizer</code>	ciągłe przesuwanie
Szczypanie	<code>PinchGestureRecognizer</code>	zoom dwoma palcami

TYPOWE BŁĘDY

- Brak `GestureRecognizers` - element nie reaguje na dotyk.

Gdy kilka obrazów współdzieli jeden handler, ustaw im unikalne `ClassId` (np. numer) w XAML i odczytaj `((Image)sender).ClassId` w kodzie, by rozpoznać, który obraz dotknięto.

20.8. Kliknięcie w obraz - wzorec praktyczny

Najczęstsze zastosowanie gestu to **klikalny obraz**. Łączymy `Image` z `TapGestureRecognizer`, a w handlerze rozpoznajemy obraz przez `sender` i reagujemy - zmieniamy `Source`, `Opacity` lub stan aplikacji.

PRZYKŁAD XAML I C#

```
<HorizontalStackLayout Spacing="8">
    <Image Source="ikona1.png" ClassId="0" HeightRequest="60">
        <Image.GestureRecognizers>
            <TapGestureRecognizer Tapped="OnIkona" />
        </Image.GestureRecognizers>
    </Image>
    <Image Source="ikona2.png" ClassId="1" HeightRequest="60">
        <Image.GestureRecognizers>
            <TapGestureRecognizer Tapped="OnIkona" />
        </Image.GestureRecognizers>
    </Image>
</HorizontalStackLayout>
```

```
private void OnIkona(object sender, EventArgs e)
{
    var obraz = (Image)sender;
    int indeks = int.Parse(obraz.ClassId); // który obraz
    Komunikat.Text = $"Dotknięto obraz nr {indeks}";
}
```

Na co uważać:

20.9. Schemat: akcja -> zmiana stanu -> aktualizacja UI

To **najważniejszy schemat** w aplikacjach event-driven. Niemal każdy handler wygląda tak samo: (1) **akcja użytkownika** wywołuje zdarzenie, (2) w handlerze **zmieniamy stan** aplikacji (zmienne, kolekcje), (3) **aktualizujemy interfejs**, by pokazać nowy stan. Opanowanie tego schematu to klucz do tworzenia działających aplikacji.

PRZYKŁAD C#

```
public partial class MainPage : ContentPage
{
    int licznik = 0; // STAN

    public MainPage() => InitializeComponent();

    private void OnDodaj(object sender, EventArgs e)
    {
        // 1. akcja: kliknięcie przycisku (to zdarzenie)
        // 2. zmiana stanu
        licznik++;
        // 3. aktualizacja UI
        Wynik.Text = $"Liczba: {licznik}";
    }
}
```

Na co uważać:

Jedna akcja użytkownika często zmienia **kilka rzeczy** na ekranie naraz (tekst, kolor, obraz, dostępność przycisku). Trzymaj się schematu: zmien stan, a potem konsekwentnie zaktualizuj wszystkie zależne elementy interfejsu.

20.10. Typowe błędy w obsłudze zdarzeń

NAJWAŻNIEJSZE INFORMACJE

BŁĄD	SKUTEK	ROZWIĄZANIE
Nazwa metody niezgodna z XAML	błąd kompilacji	ujednolić nazwy
Zły typ parametru <code>e</code>	błąd kompilacji	dopasuj typ argumentu
Brak rzutowania <code>sender</code>	brak dostępu do właściwości	<code>(Button)sender</code>
<code>async void</code> poza handlerem	trudne błędy	użyj <code>async Task</code>
Podwójne podpięcie (<code>+=</code> wielokrotnie)	handler wywołany kilka razy	podpinaj raz

Na co uważać:

Najczęstsze problemy to niezgodność nazwy metody z XAML oraz zły typ parametru `e`. Pamiętaj też, by nie podpinąć tego samego handlera wielokrotnie operatorem `+=` (np. w `OnAppearing`), bo wtedy wykona się kilka razy.

20.11. Receptury - gotowe przykłady łączone (XAML + C#)

Poniżej zestaw krótkich, kompletnych „receptur” na najczęstsze interakcje. Każda zawiera widok i logikę - gotowe do skopiowania i dostosowania.

Pokaż/ukryj element przyciskiem

```
<Button Text="Pokaż szczegóły" Clicked="OnPrzełącz" />
<Label x:Name="Szczegoly" Text="Treść szczegółów" IsVisible="False" />
```

```
private void OnPrzełącz(object sender, EventArgs e)
    => Szczegoly.IsVisible = !Szczegoly.IsVisible; // przełącz widoczność
```

Licznik znaków w polu

```
<Entry x:Name="Pole" MaxLength="100" TextChanged="OnTekst" />
<Label x:Name="Licznik" Text="0/100" HorizontalTextAlignment="End" />
```

```
private void OnTekst(object sender, TextChangedEventArgs e)
    => Licznik.Text = $"{e.NewValue?.Length ?? 0}/100";
```

Suwak zmieniający etykietę na żywo

```
<Slider x:Name="Suwak" Minimum="0" Maximum="100" ValueChanged="OnSuwak" />
<Label x:Name="Wartosc" Text="0" />
```

```
private void OnSuwak(object sender, ValueChangedEventArgs e)
    => Wartosc.Text = ((int)e.NewValue).ToString();
```

```
<Picker x:Name="Lista" Title="Wybierz" SelectedIndexChanged="OnWybor" />
<Label x:Name="Echo" />
```

```
public MainPage()
{
    InitializeComponent();
    Lista.ItemsSource = new List<string> { "A", "B", "C" };
}

private void OnWybor(object sender, EventArgs e)
{
    if (Lista.SelectedIndex == -1) return;
    Echo.Text = $"Wybrano: {Lista.SelectedItem}";
}
```

Switch sterujący stanem i kolorem

```
<Switch x:Name="Tryb" Toggled="OnTryb" />
<Label x:Name="Status" Text="Tryb: standard" />
```

```
private void OnTryb(object sender, ToggledEventArgs e)
{
```

```
Status.Text = e.Value ? "Tryb: nocny" : "Tryb: standard";
Status.TextColor = e.Value ? Colors.Orange : Colors.Black;
}
```

Klikalny obraz przełączający stan (polubienie)

```
<Image x:Name="Serce" Source="serce_puste.png" HeightRequest="48">
    <Image.GestureRecognizers>
        <TapGestureRecognizer Tapped="OnPoLub" />
    </Image.GestureRecognizers>
</Image>
```

```
bool polubione = false;
private void OnPoLub(object sender, EventArgs e)
{
    polubione = !polubione;
    Serce.Source = polubione ? "serce_pełne.png" : "serce_puste.png";
}
```

Dodawanie do listy na żywo

```
<Entry x:Name="Nowy" Placeholder="Nowy element" />
<Button Text="Dodaj" Clicked="OnDodaj" />
<CollectionView x:Name="Lista2">
    <CollectionView.ItemTemplate>
        <DataTemplate><Label Text="{Binding .}" Padding="6" /></DataTemplate>
    </CollectionView.ItemTemplate>
</CollectionView>
```

```
ObservableCollection<string> elementy = new();
public MainPage() { InitializeComponent(); Lista2.ItemsSource = elementy; }
private void OnDodaj(object sender, EventArgs e)
{
    if (string.IsNullOrWhiteSpace(Nowy.Text)) return;
    elementy.Add(Nowy.Text);
    Nowy.Text = string.Empty;
}
```

Dynamiczny kolor tła zależny od wartości

```
<Entry x:Name="PoLeTemp" Placeholder="Temperatura" Keyboard="Numeric" TextChanged="OnTemp" />
<BoxView x:Name="Wskaznik" HeightRequest="40" Color="Gray" />
```

```
private void OnTemp(object sender, TextChangedEventArgs e)
{
}
```

```

if (!int.TryParse(e.NewValue, out int t)) { Wskaznik.Color = Colors.Gray; return; }
Wskaznik.Color = t < 0 ? Colors.LightBlue : t < 25 ? Colors.LightGreen : Colors.OrangeRed;
}

```

Blokowanie przycisku do czasu zaznaczenia zgody

```

<CheckBox x:Name="Zgoda" CheckedChanged="OnZgoda" />
<Button x:Name="Dalej" Text="Dalej" IsEnabled="False" />

```

```

private void OnZgoda(object sender, CheckedChangedEventArgs e)
⇒ Dalej.IsEnabled = e.Value;

```

Potwierdzenie przed usunięciem (DisplayAlert)

```

private async void OnUsun(object sender, EventArgs e)
{
    bool ok = await DisplayAlert("Usuwanie", "Na pewno usunąć?", "Tak", "Nie");
    if (ok) { /* usuń */ }
}

```

Na co uważać: to gotowe „klocki” - łącz je, by budować pełne ekrany. Wszystkie opierają się na schemacie „akcja → zmiana stanu → aktualizacja UI”. Kopiuj i dostosowuj nazwy kontrolerek do swoich.

Stan aplikacji to wszystko, co aplikacja musi „pamiętać” w trakcie działania: aktualny licznik, wybrany element, wynik, kolor, to, czy coś jest włączone. Zrozumienie stanu to klucz do tworzenia działających aplikacji - bez niego interfejs „zapomina” o wszystkim po każdej akcji. Ten rozdział tłumaczy stan bardzo prosto i pokazuje go na praktycznych przykładach: liczniku, przełączniku, przeglądarce listy, grze w kości i wzorniku kolorów.

21. Stan aplikacji i aktualizacja interfejsu

21.1. Czym jest stan aplikacji

Stan to **zbiór danych, które aplikacja przechowuje w danym momencie**. Gdy klikasz „+” i licznik rośnie, jego aktualna wartość to stan. Gdy przełącznik jest włączony, ten fakt to stan. Stan zmienia się w reakcji na akcje użytkownika, a interfejs **pokazuje** aktualny stan.

Stan pozwala aplikacji **pamiętać** dane między kolejnymi akcjami i **reagować** na ich zmiany. Bez stanu każda akcja zaczynałaby od zera - licznik nigdy by nie rósł, lista nie zapamiętałaby pozycji.

NAJWAŻNIEJSZE INFORMACJE

- Stan przechowujemy w **polach klasy** strony (nie w zmiennych lokalnych metod).
- Po zmianie stanu **aktualizujemy interfejs**, by pokazać nową wartość.
- Stan może być liczbą, tekstem, wartością logiczną, indeksem, obiektem, kolekcją.

Na co uważać:

Najważniejsze rozróżnienie: zmienna **lokalna** (w metodzie) żyje tylko podczas jej wykonania, a **pole klasy** żyje przez cały czas istnienia strony. Stan, który ma przetrwać między akcjami, **musi** być polem klasy.

21.2. Gdzie przechowywać stan - pola klasy

Stan przechowujemy w **polach klasy** strony - zmiennych zadeklarowanych wewnątrz klasy, ale poza metodami. Obiekt strony żyje, dopóki użytkownik widzi ekran, więc jego pola zachowują wartości między kliknięciami.

PRZYKŁAD C#

```
public partial class MainPage : ContentPage
{
    // STAN aplikacji - pola klasy (pamięć ekranu)
    int licznik = 0;
    string aktualnyTekst = "";
    bool wlaczone = false;
    int aktualnyIndeks = 0;

    public MainPage() => InitializeComponent();
}
```

TYPOWE BŁĘDY

- Deklaracja zmiennej stanu **wewnątrz metody** handlera → resetuje się przy każdym kliknięciu.

Na co uważać:

Gdyby `int licznik = 0;` umieścić w metodzie `OnDodaj`, byłby tworzony od nowa przy każdym kliknięciu i zawsze wynosiłby 1. Stan trzymaj jako **pole klasy**, a nie zmienną lokalną.

21.3. Stan liczbowy - licznik

Stan liczbowy to wartość liczbowa pamiętana między akcjami - najprostszy przykład to **licznik**. Zwiększamy go lub zmniejszamy w reakcji na kliknięcia i pokazujemy w etykiecie.

PRZYKŁAD XAML

```
<VerticalStackLayout Padding="20" Spacing="16" VerticalOptions="Center">
    <Label x:Name="EtykietaLicznika" Text="Licznik: 0"
        FontSize="28" HorizontalOptions="Center" />
    <HorizontalStackLayout Spacing="20" HorizontalOptions="Center">
        <Button x:Name="PrzyciskMinus" Text="-" WidthRequest="70" Clicked="OnOdejmij" IsEnabled="False" />
        <Button Text="+" WidthRequest="70" Clicked="OnDodaj" />
    </HorizontalStackLayout>
</VerticalStackLayout>
```

PRZYKŁAD C#

```
int licznik = 0; // stan liczbowy

private void OnDodaj(object sender, EventArgs e)
{
    licznik++;
    Odswiez();
}

private void OnOdejmij(object sender, EventArgs e)
{
    if (licznik > 0) licznik--; // blokada zejścia poniżej zera
    Odswiez();
}
```



```

    }

    private void Odswiez()
    {
        EtykietaLicznika.Text = $"Licznik: {licznik}";
        PrzyciskMinus.IsEnabled = licznik > 0; // wyłącz '-' przy zerze
    }

```

Na co uważać:

Wydzielenie metody `Odswiez()` to dobry nawyk: każda akcja zmienia stan, a potem woła jedną metodę aktualizującą interfejs. Dzięki temu logika wyświetlania jest w jednym miejscu i nie trzeba jej powielać.

21.4. Stan tekstowy - aktualny komunikat

Stan tekstowy to przechowywany napis - np. aktualny komunikat, ostatnio wpisana wartość, bieżąca nazwa. Trzymamy go w polu typu `string` i pokazujemy w `Label`.

PRZYKŁAD C#

```

string aktualnyKomunikat = "Gotowy";

private void OnZmien(object sender, EventArgs e)
{
    aktualnyKomunikat = $"Ostatnia akcja: {DateTime.Now:HH:mm:ss}";
    Status.Text = aktualnyKomunikat;
}

```

Na co uważać:

Stan tekstowy bywa wygodny, gdy ten sam komunikat jest używany w kilku miejscach lub musi przetrwać między akcjami. Dla prostych przypadków często wystarczy bezpośrednio ustawić `Label.Text`.

21.5. Stan logiczny - włączone/wyłączone

Stan logiczny (`bool`) reprezentuje dwie możliwości: włączone/wyłączone, tak/nie, aktywne/nieaktywne. To podstawa paneli urządzeń i przełączników. Przełączamy go operatorem negacji `!`.

PRZYKŁAD C#

```

bool wlaczone = false; // stan on/off

private void OnPrzełącz(object sender, EventArgs e)
{
    wlaczone = !wlaczone; // przełącz na przeciwny
    if (wlaczone)
    {
        Status.Text = "Status: Włączone";
        Status.TextColor = Colors.Green;
        PrzyciskPrzełącz.Text = "Wyłącz";
    }
    else
    {
        Status.Text = "Status: Wyłączone";
        Status.TextColor = Colors.Gray;
        PrzyciskPrzełącz.Text = "Włącz";
    }
}

```

```
}  
}
```

Na co uważać:

Wzorec `wlaczona = !wlaczona;` to najprostszy przełącznik. Jedna zmienna `bool` jest **jedynym źródłem prawdy** o stanie, a wszystkie elementy interfejsu (tekst, kolor, podpis przycisku) wyprowadzamy z niej w instrukcji `if`.

21.6. Stan jako indeks - aktualny element

Gdy przeglądamy listę element po elemencie (np. galeria, cytaty), stanem jest **indeks aktualnego elementu**. Zwiększamy go lub zmniejszamy, a interfejs pokazuje element spod tego indeksu.

PRZYKŁAD C#

```
string[] cytaty =  
{  
    "Kto pyta, nie błądzi.",  
    "Ćwiczenie czyni mistrza.",  
    "Lepszy wróbel w garści niż gołąb na dachu."  
};  
int aktualnyIndeks = 0; // stan: który cytat  
  
private void OnNastepny(object sender, EventArgs e)  
{  
    aktualnyIndeks++;  
    if (aktualnyIndeks ≥ cytaty.Length) aktualnyIndeks = 0; // zawinięcie  
    Cytat.Text = cytaty[aktualnyIndeks];  
}  
  
private void OnPoprzedni(object sender, EventArgs e)  
{  
    aktualnyIndeks--;  
    if (aktualnyIndeks < 0) aktualnyIndeks = cytaty.Length - 1; // zawinięcie  
    Cytat.Text = cytaty[aktualnyIndeks];  
}
```

Na co uważać:

Pilnuj **zakresu indeksu**, by nie wyjść poza tablicę. Popularny wzorec to **zawijanie**: po ostatnim elemencie wracamy do pierwszego (`= 0`), a przed pierwszym przechodzimy do ostatniego (`= Length - 1`). Wyjście poza zakres tablicy to częsty błąd.

21.7. Stan jako wybrany element lub obiekt

Stanem może być cały **obiekt** - np. aktualnie wybrany produkt, edytowany rekord, bieżący użytkownik. Trzymamy go w polu odpowiedniego typu i odwołujemy się do jego właściwości.

PRZYKŁAD C#

```
Produkt wybranyProdukt; // stan: aktualnie wybrany obiekt  
  
private void OnWybor(object sender, SelectionChangedEventArgs e)  
{  
    if (e.CurrentSelection.FirstOrDefault() is Produkt p)  
    {  
        wybranyProdukt = p;  
        Szczegoly.Text = $"{p.Nazwa} - {p.Cena} zł";  
    }
```

```
}  
  
}
```

Na co uważać:

Gdy stanem jest obiekt, może on być `null` (np. nic nie wybrano). Zawsze sprawdzaj `null` przed odwołaniem do jego właściwości, by uniknąć błędu wykonania.

21.8. Stan a wygląd - ważne rozróżnienie

To kluczowe rozróżnienie. **Stan** to dane (np. `wlaczzone = true`, `licznik = 5`). **Wygląd** to to, co widać (zielony napis „Włączone”, tekst „Licznik: 5”). Wygląd jest **po pochodną** stanu - najpierw zmieniamy stan, potem aktualizujemy wygląd, by go odzwierciedlał.

NAJWAŻNIEJSZE INFORMACJE

- **Stan** = dane w pamięci (pola klasy).
- **Wygląd** = właściwości kontrolek (Text, kolor, obraz).
- Kierunek: **stan** → **wygląd** (zmień dane, potem odśwież widok).

Na co uważać:

Nie „trzymaj stanu w wyglądzie”. Np. nie odczytuj tekstu z `Label`, by dowiedzieć się, ile wynosi licznik - trzymaj liczbę w polu klasy, a `Label` tylko ją wyświetla. Stan to źródło prawdy, wygląd to jego odbicie.

21.9. Aktualizacja interfejsu po zmianie stanu

Po każdej zmianie stanu trzeba **zaktualizować interfejs**, by pokazać nową wartość. W code-behind robimy to ręcznie, ustawiając właściwości kontrolek. Przy bardziej rozbudowanym bindingu można też użyć `INotifyPropertyChanged`.

PRZYKŁAD C#

```
private void Odswiez()  
{  
    // Jedna metoda aktualizująca cały interfejs na podstawie stanu  
    EtykietaLicznika.Text = $"Licznik: {licznik}";  
    PrzyciskMinus.IsEnabled = licznik > 0;  
    Tlo.BackgroundColor = licznik > 10 ? Colors.LightGreen : Colors.White;  
}
```

Na co uważać:

Jedna akcja często zmienia **kilka** elementów interfejsu. Wydzielenie wspólnej metody odświeżającej (`Odswiez`) zapewnia spójność - wszystkie zależne elementy aktualizują się razem, na podstawie aktualnego stanu.

21.10. Przykład: gra w kości (stan złożony)

Gra w kości łączy kilka rodzajów stanu: **wartości** wylosowane na kościach, **stan blokady** każdej kości oraz **sumę** oczek. To dobry przykład stanu przechowywanego w tablicach.

PRZYKŁAD C#

```
int[] wartosci = new int[5]; // stan: wartości kości  
bool[] zablokowana = new bool[5]; // stan: które kości zablokowane  
readonly Random los = new Random();  
  
private void OnRzut(object sender, EventArgs e)  
{  
    for (int i = 0; i < 5; i++)  
    {  
        if (zablokowana[i]) continue; // pomiń zablokowane
```

```

        wartosci[i] = los.Next(1, 7); // losuj 1..6
        kostki[i].Source = $"kostka{wartosci[i]}.png";
    }
    PokazSume();
}

private void PokazSume()
{
    int suma = 0;
    foreach (int w in wartosci) suma += w;
    EtykietaSumy.Text = $"Suma: {suma}";
}

```

Na co uważać:

Złożony stan dobrze przechowywać w **tablicach równoległych** (`wartosci`, `zablokowana`), gdzie ten sam indeks dotyczy tej samej kości. To pozwala obsłużyć wiele elementów w jednej pętli. Alternatywą jest lista obiektów klasy `Kosc`.

21.11. Przykład: wzornik kolorów RGB (stan liczbowy ↔ wygląd)

Wzornik RGB to ekran z trzema suwakami (czerwony, zielony, niebieski), które razem składają kolor. Stanem są trzy liczby (0–255), a wyglądem - podgląd koloru. To czytelny przykład zależności **stan** → **wygląd**.

PRZYKŁAD XAML

```

<VerticalStackLayout Padding="20" Spacing="12">
    <BoxView x:Name="Podglad" HeightRequest="100" Color="Black" />
    <Label x:Name="EtykietaRgb" Text="RGB(0, 0, 0)" HorizontalOptions="Center" />

    <Slider x:Name="SuwakR" Minimum="0" Maximum="255" ValueChanged="OnKolor" />
    <Slider x:Name="SuwakG" Minimum="0" Maximum="255" ValueChanged="OnKolor" />
    <Slider x:Name="SuwakB" Minimum="0" Maximum="255" ValueChanged="OnKolor" />
</VerticalStackLayout>

```

PRZYKŁAD C#

```

private void OnKolor(object sender, ValueChangedEventArgs e)
{
    int r = (int)SuwakR.Value; // stan: składowa czerwona
    int g = (int)SuwakG.Value; // stan: składowa zielona
    int b = (int)SuwakB.Value; // stan: składowa niebieska

    Podglad.Color = Color.FromRgb(r, g, b); // wygląd z liczb
    EtykietaRgb.Text = $"RGB({r}, {g}, {b})";
}

```

Na co uważać:

Każdy suwak zwraca `double` - rzutuj na `int`, bo składowe koloru to liczby całkowite 0–255. `Color.FromRgb(r, g, b)` składa kolor z trzech liczb. To wzorcowy przykład: zmiana stanu (liczby z suwaków) natychmiast aktualizuje wygląd (kolor podglądu).

21.12. Typowe rodzaje stanu - podsumowanie

NAJWAŻNIEJSZE INFORMACJE

RODZAJ STANU	TYP	PRZYKŁAD
Liczbowy	<code>int</code> , <code>double</code>	licznik, wynik, składowa koloru
Tekstowy	<code>string</code>	aktualny komunikat, nazwa
Logiczny	<code>bool</code>	włączone/wyłączone, zaznaczone
Indeks	<code>int</code>	aktualny element listy
Obiekt	klasa	wybrany produkt, rekord
Kolekcja	<code>List</code> / <code>ObservableCollection</code>	lista notatek, kości

Na co uważać:

Dobór typu stanu zależy od tego, co pamiętamy. Pamiętaj o trzech zasadach: stan trzymaj w **polach klasy**, po zmianie stanu **aktualizuj interfejs**, a stan traktuj jako **źródło prawdy** (wygląd to jego odbicie).

Świadome zarządzanie stanem to fundament dobrych aplikacji. Jeśli potrafisz odpowiedzieć na pytanie „co moja aplikacja musi pamiętać?” i konsekwentnie aktualizujesz interfejs po każdej zmianie stanu, większość interakcji staje się prosta i przewidywalna.

Naturalnym odruchem początkującego jest pisanie całej logiki wprost w handlerze przycisku. To działa dla prostych przypadków, ale przy rozbudowie prowadzi do „grubych” handlerów liczących dziesiątki linii - trudnych do czytania, testowania i ponownego użycia.

NAJWAŻNIEJSZE INFORMACJE

- Logika w handlerze jest **trudna do ponownego użycia** (przywiązana do przycisku).
- Jest **trudna do testowania** (wymaga interfejsu).
- Powtarza się, gdy ta sama operacja jest potrzebna w kilku miejscach.
- Miesza **co** robimy (logika) z **jak pokazujemy** (widok).

Na co uważać:

Zasada: **widok pokazuje, logika liczy**. Jeśli handler przycisku zaczyna mieć kilkadziesiąt linii obliczeń, to sygnał, że logikę warto przenieść do osobnej klasy. Handler powinien głównie **wywoływać** logikę i **pokazywać** jej wynik.

22. Logika poza widokiem i organizacja kodu

22.1. Klasy pomocnicze

Klasa pomocnicza (helper) to klasa skupiająca powiązane, wielokrotnie używane funkcje - np. obliczenia, formatowanie, walidację. Metody pomocnicze często są **statyczne**, bo nie potrzebują stanu obiektu. Helpery umieszczamy zwykle w folderze `Helpers`.

PRZYKŁAD C#

```
public static class Validator
{
    public static bool PoprawnyEmail(string email)
        => !string.IsNullOrEmpty(email) && email.Contains('@');

    public static bool WZakresie(int liczba, int min, int max)
        => liczba >= min && liczba <= max;
}
```

```
// Użycie w handlerze - krótko i czytelnie
if (!Validator.PoprawnyEmail(PoleEmail.Text))
{
    Komunikat.Text = "Niepoprawny e-mail";
    return;
}
```

Na co uważać:

Klasy statyczne świetnie pasują do bezstanowych funkcji pomocniczych. Dzięki nim ta sama walidacja czy obliczenie jest dostępne w wielu miejscach bez powielania kodu.

22.2. Modele jako nośnik danych i prostej logiki

Model to klasa danych, która może też zawierać **prostą logikę** dotyczącą tych danych - np. obliczenie wartości czy sprawdzenie warunku. Logika ściśle związana z danymi naturalnie należy do modelu.

PRZYKŁAD C#

```
public class Zamowienie
{
    public List<double> Ceny { get; set; } = new();

    // Logika należąca do modelu
    public double Suma() => Ceny.Sum();
    public double SumaZRabatem(double rabatProcent)
        => Suma() * (1 - rabatProcent / 100.0);
}
```

Na co uważać:

Do modelu wrzucaj logikę **ściśle związaną z jego danymi** (np. suma pozycji zamówienia). Logikę dotyczącą zewnętrznych zasobów (baza, sieć) trzymaj w osobnych klasach pomocniczych, nie w modelu.

22.3. Klasa do pracy na danych

PRZYKŁAD C#

```
public class NotatkiMagazyn
{
    private readonly List<string> notatki = new();

    public IReadOnlyList<string> Pobierz() => notatki;
    public void Dodaj(string tresc)
    {
        if (!string.IsNullOrWhiteSpace(tresc))
            notatki.Add(tresc);
    }
    public void Usun(string tresc) => notatki.Remove(tresc);
}
```

Na co uważać:

Taka klasa powinna mieć **jedną odpowiedzialność** (np. tylko notatki, tylko API). Dzięki temu kod jest uporządkowany, a klasy pomocnicze można łatwo testować i podmieniać. Klasy pomocnicze są pomostem między widokiem a danymi.

22.4. Logika walidacji

Reguły walidacji warto wydzielić do klasy, by używać ich spójnie w wielu miejscach i testować niezależnie. Zamiast powtarzać te same

`if`-y w każdym formularzu, wołamy metody walidatora.

PRZYKŁAD C#

```
public static class RegulyHasla
{
    public static bool Wystarczajaco(string haslo) => (haslo?.Length ?? 0) ≥ 6;
    public static bool MaCyfre(string haslo) => haslo?.Any(char.IsDigit) ?? false;
    public static (bool ok, string komunikat) Sprawdz(string haslo)
    {
        if (!Wystarczajaco(haslo)) return (false, "Hasło min. 6 znaków.");
        if (!MaCyfre(haslo)) return (false, "Hasło musi zawierać cyfrę.");
        return (true, "OK");
    }
}
```

Na co uważać:

Zwracanie krotki `(bool ok, string komunikat)` to wygodny sposób przekazania wyniku walidacji i komunikatu naraz. Widok jedynie pokazuje komunikat, a reguły żyją w jednym, testowalnym miejscu.

22.5. Logika losowania

Losowanie (gry, doborów) warto zamknąć w klasie, zwłaszcza gdy ma reguły (np. losuj tylko dostępne elementy). Klasa losująca jest niezależna od widoku.

PRZYKŁAD C#

```
public class Kostka
{
    public int Wartosc { get; private set; } = 1;
    public bool Zablockowana { get; set; }

    private static readonly Random los = new Random();

    public void Rzuc()
    {
        if (!Zablockowana)
            Wartosc = los.Next(1, 7); // 1..6
    }
}
```

```
// Widok tylko steruje modelem i pokazuje stan
private List<Kostka> kosci = new() { new(), new(), new(), new(), new() };

private void OnRzut(object sender, EventArgs e)
{
    foreach (var k in kosci) k.Rzuc(); // logika
    PokazKosci(); // widok
}
```

Na co uważać:

Klasa `Kostka` zna tylko swoją logikę (wartość, blokada, rzut) - nie wie nic o obrazach. Widok odzwierciedla jej stan. To czysty podział „model liczy, widok pokazuje”.

22.6. Logika generowania hasła

Generowanie losowego hasła to klasyczny przykład logiki, którą warto wydzielić. Klasa pomocnicza składa hasło z dozwolonych znaków o zadanej długości.

PRZYKŁAD C#

```
public static class GeneratorHasla
{
    const string Znaki = "abcdefghijklmnopqrstuvwxyzABCDEFGHIJKLMNOPQRSTUVWXYZ0123456789!@#\$";
    private static readonly Random los = new Random();

    public static string Generuj(int dlugosc)
    {
        var sb = new System.Text.StringBuilder();
        for (int i = 0; i < dlugosc; i++)
            sb.Append(Znaki[los.Next(Znaki.Length)]);
        return sb.ToString();
    }
}
```

```
// Użycie
PoleHaslo.Text = GeneratorHasla.Generuj(12);
```

Na co uważać:

`StringBuilder` jest wydajniejszy niż wielokrotne sklejanie napisów w pętli. Logika generatora jest w pełni niezależna od interfejsu - można ją przetestować osobno.

22.7. Logika obliczeń

Obliczenia (BMI, podatek, wynik gry) wydzielamy do metod/klas, by oddzielić **liczenie** od **wyświetlania**. Widok przekazuje dane wejściowe i pokazuje wynik.

PRZYKŁAD C#

```
public static class Kalkulator
{
    // BMI = masa / (wzrost_m)^2
    public static double Bmi(double masaKg, double wzrostM)
        => wzrostM <= 0 ? 0 : masaKg / (wzrostM * wzrostM);

    public static string Kategoria(double bmi) => bmi switch
    {
        < 18.5 => "Niedowaga",
        < 25  => "Waga prawidłowa",
        < 30  => "Nadwaga",
        _    => "Otyłość"
    };
}
```

Na co uważać:

Zabezpiecz obliczenia przed nieprawidłowymi danymi (np. dzielenie przez zero przy wzroście 0). Logika obliczeniowa w osobnej klasie jest łatwa do przetestowania bez uruchamiania aplikacji.

22.8. Logika zmiany stanu i pracy na liście obiektów

Operacje na kolekcji obiektów (dodawanie, filtrowanie, sortowanie, zmiana stanu elementów) warto trzymać w klasie pomocniczej, a nie rozsiewać po handlerach. To porządkuje kod i ułatwia ponowne użycie.

PRZYKŁAD C#

```
public class ZadaniaMagazyn
{
    private readonly List<Zadanie> zadania = new();

    public void Dodaj(string tytul) => zadania.Add(new Zadanie { Tytul = tytul });
    public void Wykonaj(Zadanie z) => z.Wykonane = true;

    public List<Zadanie> Niewykonane()
        => zadania.Where(z => !z.Wykonane).ToList();

    public List<Zadanie> PosortowaneWgDaty()
        => zadania.OrderBy(z => z.DataUtworzenia).ToList();
}
```

Na co uważać:

Metody LINQ (`Where`, `OrderBy`, `Sum`) bardzo upraszczają pracę na listach. Trzymanie tych operacji w klasie pomocniczej sprawia, że widok jedynie wywołuje gotowe metody i pokazuje wynik.

22.9. Logika szyfrowania (prosty przykład)

Prosta transformacja tekstu (np. szyfr przesuwający litery) to dobry przykład logiki czysto algorytmicznej, idealnej do wydzielenia. Klasa szyfrująca nie ma nic wspólnego z interfejsem.

PRZYKŁAD C#

```
public static class SzyfrPrzesuwajacy
{

```

```
// Przesuwa każdą literę o zadaną liczbę pozycji
public static string Zaszyfruj(string tekst, int przesuniecie)
{
    var sb = new System.Text.StringBuilder();
    foreach (char c in tekst)
    {
        if (char.IsLetter(c))
        {
            char baza = char.IsUpper(c) ? 'A' : 'a';
            char nowy = (char)(baza + (c - baza + przesuniecie) % 26);
            sb.Append(nowy);
        }
        else sb.Append(c); // znaki niebędące literami bez zmian
    }
    return sb.ToString();
}

public static string Odszyfruj(string tekst, int przesuniecie)
    => Zaszyfruj(tekst, 26 - (przesuniecie % 26));
}
```

Na co uważać:

Do **prosty** szyfru do celów edukacyjnych - **nie** nadaje się do realnego zabezpieczania danych. Do prawdziwego szyfrowania używa się sprawdzonych bibliotek kryptograficznych. Przykład pokazuje jednak ideę wydzielenia algorytmu od widoku.

22.10. Testowanie logiki bez interfejsu

Największa zaleta wydzielonej logiki to **testowalność**. Klasę pomocniczą, model albo algorytm można przetestować w osobnym projekcie testowym, bez uruchamiania interfejsu. Sprawdzamy, czy dla danych wejściowych zwracają oczekiwany wynik.

PRZYKŁAD C# (IDEA TESTU)

```
// Przykładowy test logiki (w projekcie testowym)
var bmi = Kalkulator.Bmi(70, 1.75); // ~22.86
bool ok = Math.Abs(bmi - 22.86) < 0.01;
// 'ok' powinno być true - logika działa poprawnie
```

Na co uważać:

Logika „wmieszana” w handlersy przycisków jest praktycznie nietestowalna bez uruchamiania aplikacji. Wydzielona - testuje się łatwo i szybko. To jeden z głównych powodów, dla których warto oddzielać logikę od widoku.

22.11. Typowe błędy

NAJWAŻNIEJSZE INFORMACJE

BŁĄD	SKUTEK	ROZWIĄZANIE
Logika sieci/bazy w modelu	mieszanie odpowiedzialności	przenieś do klasy pomocniczej
Powielanie tej samej logiki	trudne utrzymanie	klasa pomocnicza
Brak walidacji w wydzielonej logice	błędne dane	waliduj w jednym miejscu
Statyczny stan tam, gdzie potrzebny obiekt	błędy współdzielenia	użyj instancji

Na co uważać:

Złota zasada: **widok pokazuje, logika liczy**. Wydzielaj walidację, obliczenia, losowanie i operacje na danych do klas pomocniczych, modeli i klas pomocniczych. Kod staje się czytelniejszy, łatwiejszy do ponownego użycia i testowalny.

Oddzielanie logiki od widoku to krok w stronę czytelnej architektury. Nawet w prostej aplikacji wydzielenie walidacji, obliczeń i operacji na danych do osobnych klas znacząco poprawia jakość kodu.

23. Formularze i pobieranie danych

Formularz to jeden z najczęstszych elementów aplikacji – ekran, na którym użytkownik wprowadza dane, a następnie je zatwierdza. Logowanie, rejestracja, dodawanie produktu, rezerwacja – to wszystko formularze. Ten rozdział jest jednym z najobszerniejszych w podręczniku: pokazujemy budowę formularza krok po kroku, pobieranie danych z różnych kontroltek, składanie podsumowania, czyszczenie, blokowanie przycisku oraz kompletne, działające przykłady – od logowania po rozbudowany formularz z wieloma typami pól.

23.1. Budowa prostego formularza

Formularz to zestaw **pól wejściowych** (kontroltek) i **przycisku zatwierdzającego**, zwykle uzupełniony o **obszar komunikatów**. Budujemy go według powtarzalnego schematu: dla każdej informacji dajemy etykietę i odpowiednie pole, a na końcu przycisk i miejsce na wynik/komunikat.

Formularz służy do **zebrania danych od użytkownika** i przekazania ich do dalszej logiki: zapisania, wysłania, obliczenia. To podstawowy sposób komunikacji „od użytkownika do aplikacji”.

NAJWAŻNIEJSZE INFORMACJE

- Każde pole zwykle ma **etykietę** (`Label`) i **kontrolkę wejściową**.
- Na końcu jest **przycisk** zatwierdzający i **obszar komunikatów**.
- Pola układamy w `VerticalStackLayout` lub `Grid`.
- Dłuższe formularze owijamy w `ScrollView`.

PRZYKŁAD XAML

```
<ScrollView>
    <VerticalStackLayout Padding="24" Spacing="14">
        <Label Text="Formularz kontaktowy" FontSize="24" FontAttributes="Bold" />

        <Label Text="Imię" />
        <Entry x:Name="PoleImie" Placeholder="Wpisz imię" />

        <Label Text="E-mail" />
        <Entry x:Name="PoleEmail" Placeholder="adres e-mail" Keyboard="Email" />

        <Button Text="Wyślij" Clicked="OnWyślij" />
        <Label x:Name="Komunikat" />
    </VerticalStackLayout>
</ScrollView>
```

PRZYKŁAD C#

```
private void OnWyślij(object sender, EventArgs e)
{
    string imie = PoleImie.Text;
    string email = PoleEmail.Text;

    if (string.IsNullOrEmpty(imie) || string.IsNullOrEmpty(email))
    {
```

```

        Komunikat.Text = "Uzupełnij wszystkie pola";
        Komunikat.TextColor = Colors.Red;
        return;
    }

    Komunikat.Text = $"Dziękujemy, {imie}!";
    Komunikat.TextColor = Colors.Green;
}

```

TYPOWE BŁĘDY

- Brak `ScrollView` przy dłuższym formularzu (część pól chowa się za klawiaturą).
- Brak obszaru komunikatów (użytkownik nie wie, co poszło nie tak).

Na co uważać:

Zawsze przewiduj miejsce na **komunikat** (np. `Label`), aby informować użytkownika o wyniku. Dłuższe formularze owijaj w `ScrollView`, by zmieściły się na małych ekranach i nie były zasłanianie przez klawiaturę.

23.2. Układ formularza w `VerticalStackLayout` i w `Grid`

Pola formularza można rozmieścić na dwa sposoby. `VerticalStackLayout` układa etykiety nad polem (jeden pod drugim) - proste i czytelne na telefonie. `Grid` pozwala umieścić etykiety po lewej, a pole po prawej (w jednym wierszu) - kompaktowo i elegancko, zwłaszcza na szerszych ekranach.

PRZYKŁAD XAML (GRID)

```

<Grid ColumnDefinitions="110,*" RowDefinitions="Auto,Auto,Auto"
      ColumnSpacing="10" RowSpacing="14" Padding="20">

    <Label Text="Imię:" Grid.Row="0" Grid.Column="0" VerticalOptions="Center" />
    <Entry x:Name="PoleImie" Grid.Row="0" Grid.Column="1" />

    <Label Text="E-mail:" Grid.Row="1" Grid.Column="0" VerticalOptions="Center" />
    <Entry x:Name="PoleEmail" Grid.Row="1" Grid.Column="1" Keyboard="Email" />

    <Label Text="Telefon:" Grid.Row="2" Grid.Column="0" VerticalOptions="Center" />
    <Entry x:Name="PoleTelefon" Grid.Row="2" Grid.Column="1" Keyboard="Telephone" />

</Grid>

```

Na co uważać:

Na telefonie często wygodniejszy jest `VerticalStackLayout` (etykieta nad polem), bo etykiety obok pól zabierają cenną szerokość. `Grid` z etykietą obok pola lepiej sprawdza się na szerszych ekranach. Wybierz układ pod docelowe urządzenie.

23.3. Pobieranie danych z różnych kontroltek

Aby przetworzyć formularz, trzeba **odczytać wartości** ze wszystkich pól. Każdy typ kontrolki przechowuje dane w innej właściwości. Poniższa tabela to praktyczna ściągą.

NAJWAŻNIEJSZE INFORMACJE

KONTROLKA	WŁAŚCIWOŚĆ Z DANYMI	TYP	UWAGI
Entry	Text	string	sprawdź null /pustość
Editor	Text	string	tekst wieloliniowy
Switch	IsToggled	bool	
Slider	Value	double	rzutuj na int w razie potrzeby
Stepper	Value	double	jw.

PRZYKŁAD C#

```
private void OnPobierz(object sender, EventArgs e)
{
    string imie = PoleImie.Text; // Entry
    string opis = PoleOpis.Text; // Editor
    string kategoria = ListaKategorii.SelectedItem?.ToString(); // Picker
    DateTime data = WyborDaty.Date; // DatePicker
    bool zgoda = PoleZgoda.IsChecked; // CheckBox
    bool powiadomienia = Przełącznik.IsToggled; // Switch
    int ocena = (int)Suwak.Value; // Slider
}
```

TYPOWE BŁĘDY

- Brak TryParse przy danych liczbowych z Entry .

Na co uważać:

23.4. Składanie podsumowania z wielu pól

Częstym zadaniem jest zebranie danych z wielu kontrolk i złożenie ich w jeden czytelny tekst - np. po naciśnięciu „Zatwierdź” pokazujemy wszystkie wprowadzone informacje w jednym Label . Najwygodniej użyć interpolacji stringów i znaków nowej linii \n .

PRZYKŁAD C#

```
private void OnPodsumuj(object sender, EventArgs e)
{
    string imie = string.IsNullOrWhiteSpace(PoleImie.Text) ? "-" : PoleImie.Text;
    string kategoria = ListaKategorii.SelectedItem?.ToString() ?? "nie wybrano";
    int ilosc = (int)Suwak.Value;
    string zgoda = PoleZgoda.IsChecked ? "tak" : "nie";

    // Składanie podsumowania z wielu zmiennych
    Podsumowanie.Text =
        $"Imię: {imie}\n" +
        $"Kategoria: {kategoria}\n" +
        $"Ilość: {ilosc}\n" +
        $"Zgoda: {zgoda}";
}
```

Na co uważać:

23.5. Aktualizacja etykiety po kliknięciu przycisku

Najprostsza reakcja formularza to **pokazanie wyniku w etykiecie** po naciśnięciu przycisku. To realizacja schematu „akcja → stan → UI”: odczytujemy dane, przetwarzamy i wpisujemy wynik do `Label`.

PRZYKŁAD C#

```
private void OnPowitaj(object sender, EventArgs e)
{
    string imie = PoleImie.Text;
    if (string.IsNullOrEmpty(imie))
    {
        Powitanie.Text = "Podaj imię, aby otrzymać powitanie.";
        return;
    }
    Powitanie.Text = $"Witaj, {imie}! Miło Cię widzieć.";
}
```

Na co uważać:

To najczęstszy wzorec formularza. Zawsze rozważ przypadek pustych danych - pokaż wtedy komunikat zachęcający do uzupełnienia, zamiast wyświetlać niekompletny wynik.

23.6. Czyszczenie formularza

Po zatwierdzeniu lub na życzenie użytkownika często chcemy **wyczyścić formularz** - przywrócić pola do stanu początkowego. Robimy to, ustawiając właściwości kontrolek na wartości puste/domyślne.

PRZYKŁAD C#

```
private void OnWyczyszc(object sender, EventArgs e)
{
    PoleImie.Text = string.Empty;
    PoleEmail.Text = string.Empty;
    PoleOpis.Text = string.Empty;
    ListaKategorii.SelectedIndex = -1; // brak wyboru
    PoleZgoda.IsChecked = false;
    Przełącznik.IsToggled = false;
    Suwak.Value = Suwak.Minimum;
    Komunikat.Text = string.Empty;
}
```

Na co uważać:

23.7. Blokowanie przycisku do czasu poprawnego uzupełnienia

Dobry formularz **nie pozwala zatwierdzić** niekompletnych danych. Realizujemy to, blokując przycisk (`IsEnabled = false`), dopóki dane nie są poprawne, i odblokowując go, gdy są. Sprawdzanie wykonujemy na bieżąco w zdarzeniu `TextChanged` pól.

PRZYKŁAD XAML I C#

```
<Entry x:Name="PoleEmail" Placeholder="e-mail" TextChanged="OnSprawdz" />
<Entry x:Name="PoleHaslo" Placeholder="hasło" IsPassword="True" TextChanged="OnSprawdz" />
<Button x:Name="PrzyciskZaloguj" Text="Zaloguj" IsEnabled="False" Clicked="OnZaloguj" />
```

```
private void OnSprawdz(object sender, TextChangedEventArgs e)
{
    bool poprawne =
        !string.IsNullOrEmpty(PoleEmail.Text) &&
        PoleEmail.Text.Contains('@') &&
        !string.IsNullOrEmpty(PoleHaslo.Text) &&
        PoleHaslo.Text.Length ≥ 6;

    PrzyciskZaloguj.IsEnabled = poprawne; // aktywny tylko dla poprawnych danych
}
```

Na co uważać:

Blokowanie przycisku daje użytkownikowi natychmiastową informację, czy może już zatwierdzić. To lepsze doświadczenie niż komunikat błędu dopiero po kliknięciu. Pamiętaj, by sprawdzać **wszystkie** wymagane warunki naraz.

23.8. Formularz logowania - kompletny przykład

Formularz logowania to klasyczny przykład: dwa pola (e-mail, hasło z ukrytymi znakami), przycisk i komunikat. Poniżej kompletny widok i logika.

PRZYKŁAD XAML

```
<?xml version="1.0" encoding="utf-8" ?>
<ContentPage xmlns="http://schemas.microsoft.com/dotnet/2021/maui"
              xmlns:x="http://schemas.microsoft.com/winfx/2009/xaml"
              x:Class="MojaApp.LogowaniePage"
              Title="Logowanie">
    <VerticalStackLayout Padding="24" Spacing="14" VerticalOptions="Center">
        <Label Text="Zaloguj się" FontSize="26" FontAttributes="Bold"
              HorizontalOptions="Center" />

        <Entry x:Name="PoleEmail" Placeholder="adres e-mail" Keyboard="Email" />
        <Entry x:Name="PoleHaslo" Placeholder="hasło" IsPassword="True" />

        <Button Text="ZALOGUJ" Clicked="OnZaloguj"
              BackgroundColor="#2196F3" TextColor="White" CornerRadius="8" />

        <Label x:Name="Komunikat" HorizontalOptions="Center" FontAttributes="Bold" />
    </VerticalStackLayout>
</ContentPage>
```

PRZYKŁAD C#

```
namespace MojaApp;

public partial class LogowaniePage : ContentPage
{
    public LogowaniePage() ⇒ InitializeComponent();

    private void OnZaloguj(object sender, EventArgs e)
    {
        string email = PoleEmail.Text;
```

```

string haslo = PoleHaslo.Text;

if (string.IsNullOrEmpty(email) || string.IsNullOrEmpty(haslo))
{
    Pokaz("Uzupełnij e-mail i hasło.", false);
    return;
}
if (!email.Contains('@'))
{
    Pokaz("Podaj poprawny adres e-mail.", false);
    return;
}

// Tu w prawdziwej aplikacji następuje weryfikacja danych logowania
Pokaz("Zalogowano pomyślnie!", true);
}

private void Pokaz(string tekst, bool sukces)
{
    Komunikat.Text = tekst;
    Komunikat.TextColor = sukces ? Colors.Green : Colors.Red;
}
}

```

Na co uważać:

Pole hasła zawsze ustawiaj `IsPassword="True"`, by ukryć znaki. Wzorzec wczesnego wyjścia (`return` po pierwszym błędzie) sprawia, że logika walidacji jest czytelna i liniowa.

23.9. Formularz rejestracji - kompletny przykład

Rejestracja rozszerza logowanie o **potwierdzenie hasła** i sprawdzenie zgodności obu pól. To częsty, praktyczny przykład walidacji.

PRZYKŁAD XAML

```

<VerticalStackLayout Padding="24" Spacing="14">
    <Label Text="Założ konto" FontSize="26" FontAttributes="Bold"
        HorizontalOptions="Center" />

    <Entry x:Name="PoleEmail" Placeholder="adres e-mail" Keyboard="Email" />
    <Entry x:Name="PoleHaslo1" Placeholder="hasło" IsPassword="True" />
    <Entry x:Name="PoleHaslo2" Placeholder="powtórz hasło" IsPassword="True" />

    <Button Text="ZAREJESTRUJ" Clicked="OnZarejestruj" />
    <Label x:Name="Komunikat" HorizontalOptions="Center" FontAttributes="Bold" />
</VerticalStackLayout>

```

PRZYKŁAD C#

```

private void OnZarejestruj(object sender, EventArgs e)
{
    string email = PoleEmail.Text;
    string h1 = PoleHaslo1.Text;
    string h2 = PoleHaslo2.Text;
}

```



```

if (string.IsNullOrEmpty(email) ||
    string.IsNullOrEmpty(h1) ||
    string.IsNullOrEmpty(h2))
{
    Pokaz("Wszystkie pola są wymagane.", false);
    return;
}

if (!email.Contains('@'))
{
    Pokaz("Adres e-mail musi zawierać znak '@'.", false);
    return;
}

if (h1.Length < 6)
{
    Pokaz("Hasło musi mieć co najmniej 6 znaków.", false);
    return;
}

if (h1 != h2)
{
    Pokaz("Hasła nie są takie same.", false);
    return;
}

Pokaz("Konto zostało utworzone!", true);
}

private void Pokaz(string tekst, bool sukces)
{
    Komunikat.Text = tekst;
    Komunikat.TextColor = sukces ? Colors.Green : Colors.Red;
}

```

Na co uważać:

Porównanie haseł to zwykłe `h1 != h2`. Sprawdzaj warunki w sensownej kolejności (najpierw pustość, potem format, potem zgodność), używając wczesnego wyjścia.

23.10. Formularz z wieloma typami kontrolek - kompletny przykład

Rozbudowany formularz łączy wiele typów pól: tekst, listę, datę, suwak, przełącznik i pole wyboru. Po zatwierdzeniu składamy z nich podsumowanie. To realistyczny przykład „dodawania danych”.

PRZYKŁAD XAML

```

<ScrollView>
    <VerticalStackLayout Padding="24" Spacing="14">
        <Label Text="Nowe zgłoszenie" FontSize="24" FontAttributes="Bold" />

        <Label Text="Imię i nazwisko" />
        <Entry x:Name="PoleNazwa" Placeholder="np. Jan Kowalski" />

        <Label Text="Kategoria" />
        <Picker x:Name="ListaKategorii" Title="Wybierz kategorię" />
    </VerticalStackLayout>
</ScrollView>

```

```

<Label Text="Data" />
<DatePicker x:Name="WyborDaty" Format="dd.MM.yyyy" />

<Label x:Name="EtykietaPriorytetu" Text="Priorytet: 1" />
<Slider x:Name="SuwakPriorytetu" Minimum="1" Maximum="5" Value="1"
        ValueChanged="OnPriorytet" />

<HorizontalStackLayout Spacing="8">
    <Switch x:Name="Pilne" />
    <Label Text="Sprawa pilna" VerticalOptions="Center" />
</HorizontalStackLayout>

<Button Text="Zatwierdź" Clicked="OnZatwierdz" />
<Button Text="Wyczyść" Clicked="OnWyczysc" />
<Label x:Name="Podsumowanie" />
</VerticalStackLayout>
</ScrollView>

```

PRZYKŁAD C#

```

public partial class ZgloszeniePage : ContentPage
{
    public ZgloszeniePage()
    {
        InitializeComponent();
        ListaKategorii.ItemsSource = new List<string> { "Usterka", "Pytanie", "Wniosek" };
    }

    private void OnPriorytet(object sender, ValueChangedEventArgs e)
    {
        EtykietaPriorytetu.Text = $"Priorytet: {(int)e.NewValue}";
    }

    private void OnZatwierdz(object sender, EventArgs e)
    {
        if (string.IsNullOrEmpty(PoleNazwa.Text))
        {
            Podsumowanie.Text = "Podaj imię i nazwisko.";
            Podsumowanie.TextColor = Colors.Red;
            return;
        }

        string kategoria = ListaKategorii.SelectedItem?.ToString() ?? "nie wybrano";
        string data = WyborDaty.Date.ToString("dd.MM.yyyy");
        int priorytet = (int)SuwakPriorytetu.Value;
        string pilne = Pilne.IsToggled ? "tak" : "nie";

        Podsumowanie.TextColor = Colors.Black;
        Podsumowanie.Text =
            $"Zgłaszający: {PoleNazwa.Text}\n" +
            $"Kategoria: {kategoria}\n" +
            $"Data: {data}\n" +
            $"Priorytet: {priorytet}\n" +

```

```

        $"Pilne: {pilne}";
    }

    private void OnWyczysc(object sender, EventArgs e)
    {
        PoleNazwa.Text = string.Empty;
        ListaKategorii.SelectedIndex = -1;
        WyboryDaty.Date = DateTime.Today;
        SuwakPriorytetu.Value = 1;
        Pilne.IsToggled = false;
        Podsumowanie.Text = string.Empty;
    }
}

```

Na co uważać:

23.11. Typowe błędy przy formularzach

NAJWAŻNIEJSZE INFORMACJE

BŁĄD	SKUTEK	ROZWIĄZANIE
Brak walidacji	błędne dane trafiają dalej	sprawdzaj przed przetworzeniem
<code>Entry.Text</code> może być <code>null</code>	wyjątek przy porównaniu	<code>string.IsNullOrEmpty</code>
<code>int.Parse</code> na danych usera	wyjątek przy złym wpisie	użyj <code>int.TryParse</code>
Brak <code>ScrollView</code>	pola zastonięte klawiaturą	owiń w <code>ScrollView</code>
Brak komunikatu	użytkownik nie wie, co źle	dodaj obszar komunikatów

Na co uważać:

Dobry formularz prowadzi użytkownika za rękę: ma czytelne etykiety, sensowne podpowiedzi (`Placeholder`), walidację z jasnym komunikatem i - opcjonalnie - blokadę przycisku do czasu poprawnego wypełnienia. Walidacji poświęcamy w całości następny rozdział.

23.12. Formularz logowania - kompletny przykład

Najprostszy formularz: e-mail i hasło, przycisk oraz komunikat. Pokazujemy pełny widok i logikę z walidacją i blokadą przycisku.

```

<?xml version="1.0" encoding="utf-8" ?>
<ContentPage xmlns="http://schemas.microsoft.com/dotnet/2021/maui"
              xmlns:x="http://schemas.microsoft.com/winfx/2009/xaml"
              x:Class="Formularze.LogowaniePage"
              Title="Logowanie">
    <VerticalStackLayout Padding="24" Spacing="14" VerticalOptions="Center">
        <Label Text="Zaloguj się" FontSize="26" FontAttributes="Bold" HorizontalOptions="Center" />
        <Entry x:Name="PoleEmail" Placeholder="adres e-mail" Keyboard="Email" TextChanged="OnZmiana" />
        <Entry x:Name="PoleHaslo" Placeholder="hasło" IsPassword="True" TextChanged="OnZmiana" />
        <Button x:Name="PrzyciskZaloguj" Text="ZALOGUJ" IsEnabled="False" Clicked="OnZaloguj"
              BackgroundColor="#2196F3" TextColor="White" CornerRadius="8" />
        <Label x:Name="Komunikat" HorizontalOptions="Center" FontAttributes="Bold" />
    </VerticalStackLayout>
</ContentPage>

```

```

namespace Formularze;

public partial class LogowaniePage : ContentPage
{
    public LogowaniePage() => InitializeComponent();

    // Walidacja na żywo - blokuje przycisk do czasu poprawnych danych
    private void OnZmiana(object sender, TextChangedEventArgs e)
    {
        bool ok = !string.IsNullOrEmpty(PoleEmail.Text)
            && PoleEmail.Text.Contains('@')
            && !string.IsNullOrEmpty(PoleHaslo.Text)
            && PoleHaslo.Text.Length >= 6;

        PrzyciskZaloguj.IsEnabled = ok;
    }

    private void OnZaloguj(object sender, EventArgs e)
    {
        Komunikat.Text = "Zalogowano pomyślnie!";
        Komunikat.TextColor = Colors.Green;
    }
}

```

23.13. Formularz z każdym typem pola - kompletny przykład

```

<?xml version="1.0" encoding="utf-8" ?>
<ContentPage xmlns="http://schemas.microsoft.com/dotnet/2021/maui"
    xmlns:x="http://schemas.microsoft.com/winfx/2009/xaml"
    x:Class="Formularze.PelnyFormularzPage"
    Title="Zgłoszenie">
    <ScrollView>
        <VerticalStackLayout Padding="24" Spacing="14">
            <Label Text="Formularz zgłoszenia" FontSize="24" FontAttributes="Bold" />

            <Label Text="Imię i nazwisko" />
            <Entry x:Name="PoleNazwa" Placeholder="np. Jan Kowalski" />

            <Label Text="Opis sprawy" />
            <Editor x:Name="PoleOpis" Placeholder="Opisz problem..." AutoSize="TextChanges" HeightRequest="100" />

            <Label Text="Kategoria" />
            <Picker x:Name="ListaKategorii" Title="Wybierz kategorię" />

            <Label Text="Data zdarzenia" />
            <DatePicker x:Name="WyborDaty" Format="dd.MM.yyyy" />

            <Label Text="Godzina" />
            <TimePicker x:Name="WyborGodziny" Format="HH:mm" />

            <Label x:Name="EtykietaPriorytetu" Text="Priorytet: 1" />
            <Slider x:Name="SuwakPriorytetu" Minimum="1" Maximum="5" Value="1" ValueChanged="OnPriorytet" />
        </VerticalStackLayout>
    </ScrollView>
</ContentPage>

```

```

<HorizontalStackLayout Spacing="10">
    <Label Text="Liczba osób:" VerticalOptions="Center" />
    <Stepper x:Name="LiczbaOsob" Minimum="1" Maximum="20" Value="1" ValueChanged="OnOsoby" />
    <Label x:Name="EtykietaOsob" Text="1" VerticalOptions="Center" />
</HorizontalStackLayout>

<HorizontalStackLayout Spacing="8">
    <Switch x:Name="Pilne" />
    <Label Text="Sprawa pilna" VerticalOptions="Center" />
</HorizontalStackLayout>

<Label Text="Sposób kontaktu:" FontAttributes="Bold" />
<RadioButton Content="E-mail" GroupName="kontakt" Value="email" IsChecked="True" />
<RadioButton Content="Telefon" GroupName="kontakt" Value="telefon" />

<HorizontalStackLayout Spacing="8">
    <CheckBox x:Name="Zgoda" />
    <Label Text="Akceptuję regulamin" VerticalOptions="Center" />
</HorizontalStackLayout>

<Button Text="Zatwierdź" Clicked="OnZatwierdz" />
<Button Text="Wyczyść" Clicked="OnWyczysc" />
<Label x:Name="Podsumowanie" />
</VerticalStackLayout>
</ScrollView>
</ContentPage>

```

```

namespace Formularze;

public partial class PełnyFormularzPage : ContentPage
{
    public PełnyFormularzPage()
    {
        InitializeComponent();
        ListaKategorii.ItemsSource = new List<string> { "Usterka", "Pytanie", "Wniosek" };
        WyborDaty.Date = DateTime.Today;
    }

    private void OnPriorytet(object sender, ValueChangedEventArgs e)
    => EtykietaPriorytetu.Text = $"Priorytet: {(int)e.NewValue}";

    private void OnOsoby(object sender, ValueChangedEventArgs e)
    => EtykietaOsob.Text = ((int)e.NewValue).ToString();

    private void OnZatwierdz(object sender, EventArgs e)
    {
        if (string.IsNullOrEmpty(PoleNazwa.Text))
        { Bład("Podaj imię i nazwisko."); return; }
        if (ListaKategorii.SelectedIndex == -1)
        { Bład("Wybierz kategorię."); return; }
        if (!Zgoda.IsChecked)

```

```

{ Blad("Musisz zaakceptować regulamin."); return; }

// Odczyt wszystkich pól i złożenie podsumowania
string kategoria = ListaKategorii.SelectedItem.ToString();
string data = WyboryDate.Date.ToString("dd.MM.yyyy");
string godzina = WyboryGodziny.Time.ToString(@"hh:mm");
int priorytet = (int)SuwakPriorytetu.Value;
int osoby = (int)LiczbaOsob.Value;
string pilne = Pilne.IsToggled ? "tak" : "nie";

Podsumowanie.TextColor = Colors.Black;
Podsumowanie.Text =
    $"Zgłaszający: {PoleNazwa.Text}\n" +
    $"Opis: {PoleOpis.Text}\n" +
    $"Kategoria: {kategoria}\n" +
    $"Data: {data}, godz. {godzina}\n" +
    $"Priorytet: {priorytet}, osób: {osoby}\n" +
    $"Pilne: {pilne}";
}

private void OnWyczyszc(object sender, EventArgs e)
{
    PoleNazwa.Text = PoleOpis.Text = string.Empty;
    ListaKategorii.SelectedIndex = -1;
    WyboryDate.Date = DateTime.Today;
    SuwakPriorytetu.Value = 1;
    LiczbaOsob.Value = 1;
    Pilne.IsToggled = false;
    Zgoda.IsChecked = false;
    Podsumowanie.Text = string.Empty;
}

private void Blad(string tekst)
{
    Podsumowanie.Text = tekst;
    Podsumowanie.TextColor = Colors.Red;
}
}

```

23.14. Formularz wielosekcyjny z Border

Większe formularze warto podzielić na **sekcje** wizualnie oddzielone kartami (**Border**) z nagłówkami. To poprawia czytelność przy wielu polach. Poniżej formularz z dwiema sekcjami: dane osobowe i adres.

```

<ScrollView>
    <VerticalStackLayout Padding="20" Spacing="16">

        <Border Stroke="#DDDDDD" StrokeThickness="1" Padding="16">
            <Border.StrokeShape><RoundRectangle CornerRadius="10" /></Border.StrokeShape>
            <VerticalStackLayout Spacing="10">
                <Label Text="Dane osobowe" FontAttributes="Bold" FontSize="18" />
                <Entry x:Name="Imie" Placeholder="Imię" />
                <Entry x:Name="Nazwisko" Placeholder="Nazwisko" />
            </VerticalStackLayout>
        </Border>
    </VerticalStackLayout>
</ScrollView>

```

```

        <Entry x:Name="Email" Placeholder="E-mail" Keyboard="Email" />
    </VerticalStackLayout>
</Border>

<Border Stroke="#DDDDDD" StrokeThickness="1" Padding="16">
    <Border.StrokeShape><RoundRectangle CornerRadius="10" /></Border.StrokeShape>
    <VerticalStackLayout Spacing="10">
        <Label Text="Adres" FontAttributes="Bold" FontSize="18" />
        <Entry x:Name="Ulica" Placeholder="Ulica i numer" />
        <Grid ColumnDefinitions="*,2*" ColumnSpacing="10">
            <Entry x:Name="Kod" Placeholder="Kod" Grid.Column="0" Keyboard="Numeric" />
            <Entry x:Name="Miasto" Placeholder="Miasto" Grid.Column="1" />
        </Grid>
    </VerticalStackLayout>
</Border>

<Button Text="Zapisz" Clicked="OnZapisz" />
<Label x:Name="Komunikat" />
</VerticalStackLayout>
</ScrollView>

```

```

private void OnZapisz(object sender, EventArgs e)
{
    if (string.IsNullOrEmpty(Imie.Text) || string.IsNullOrEmpty(Nazwisko.Text))
    { Komunikat.Text = "Uzupełnij imię i nazwisko."; Komunikat.TextColor = Colors.Red; return; }

    Komunikat.Text = $"Zapisano: {Imie.Text} {Nazwisko.Text}, {Miasto.Text}";
    Komunikat.TextColor = Colors.Green;
}

```

Na co uważać: dziel długie formularze na sekcje w `Border` z nagłówkami - to znacząco poprawia czytelność. Owijaj całość w `ScrollView`. Zauważ użycie `Grid` z proporcjami `*,2*` dla pary „kod + miasto” w jednym wierszu. Walidację prowadź wg schematu z wczesnym wyjściem.

Walidacja to sprawdzanie, czy dane wprowadzone przez użytkownika są **poprawne i kompletne**, zanim je przetworzymy. To jeden z najważniejszych elementów każdej aplikacji z formularzem - chroni przed błędnymi danymi i prowadzi użytkownika za rękę. Ten rozdział jest bardzo praktyczny: pokazujemy konkretne techniki sprawdzania danych, gotowe fragmenty kodu, sposoby prezentacji błędów i kompletne przykłady.

23.15. Receptury formularzy i walidacji

Ten dział zawiera kompletne, gotowe do użycia formularze .NET MAUI z walidacją danych. Każdy przykład składa się z pełnego kodu XAML oraz pełnego kodu C# code-behind z komentarzami.

23.16. Formularz logowania

Prosty formularz logowania z walidacją pól (czy nie są puste) oraz blokadą przycisku dopóki oba pola nie zostaną wypełnione.

XAML

```

<?xml version="1.0" encoding="utf-8" ?>

<ContentPage xmlns="http://schemas.microsoft.com/dotnet/2021/maui"

```

```

        xmlns:x="http://schemas.microsoft.com/winfx/2009/xaml"
        x:Class="MojaAplikacja.LoginPage"
        Title="Logowanie">
<VerticalStackLayout Padding="30" Spacing="15">

    <Label Text="Logowanie" FontSize="28" FontAttributes="Bold" HorizontalOptions="Center"/>

    <Entry x:Name="EntryLogin"
        Placeholder="Nazwa użytkownika"
        TextChanged="OnTextChanged" />

    <Entry x:Name="EntryHaslo"
        Placeholder="Hasło"
        IsPassword="True"
        TextChanged="OnTextChanged" />

    <Label x:Name="LabelKomunikat"
        Text=""
        FontSize="14"
        HorizontalOptions="Center" />

    <Button x:Name="BtnZaloguj"
        Text="Zaloguj się"
        IsEnabled="False"
        Clicked="OnZalogujClicked" />

</VerticalStackLayout>
</ContentPage>

```

C# CODE-BEHIND

```

namespace MojaAplikacja;

public partial class LoginPage : ContentPage
{
    public LoginPage()
    {
        InitializeComponent();

        // Walidacja na żywo – blokowanie przycisku gdy pola puste
        private void OnTextChanged(object sender, TextChangedEventArgs e)
        {
            bool loginOk = !string.IsNullOrEmpty(EntryLogin.Text);
            bool hasloOk = !string.IsNullOrEmpty(EntryHaslo.Text);

            BtnZaloguj.IsEnabled = loginOk && hasloOk;

            // Zmiana koloru komunikatu
            if (!loginOk || !hasloOk)
            {
                LabelKomunikat.Text = "Wypełnij oba pola";
                LabelKomunikat.TextColor = Colors.Red;
            }
        }
    }
}

```



```

    }
    else
    {
        LabelKomunikat.Text = "Formularz gotowy";
        LabelKomunikat.TextColor = Colors.Green;
    }
}

// Obsługa kliknięcia przycisku logowania
private async void OnZalogujClicked(object sender, EventArgs e)
{
    string login = EntryLogin.Text.Trim();
    string haslo = EntryHaslo.Text.Trim();

    // Przykładowa weryfikacja (w rzeczywistości – baza danych)
    if (login == "admin" && haslo == "1234")
    {
        await DisplayAlert("Sukces", "Zalogowano pomyślnie!", "OK");
    }
    else
    {
        await DisplayAlert("Błąd", "Nieprawidłowy login lub hasło.", "OK");
    }
}
}

```

23.17. Formularz rejestracji (zgodność haseł, walidacja e-mail z @)

Formularz rejestracji nowego użytkownika z walidacją: e-mail musi zawierać znak , hasła muszą się zgadzać, minimalna długość hasła to 6 znaków.

XAML

```

<?xml version="1.0" encoding="utf-8" ?>
<ContentPage xmlns="http://schemas.microsoft.com/dotnet/2021/maui"
    xmlns:x="http://schemas.microsoft.com/winfx/2009/xaml"
    x:Class="MojaAplikacja.RegisterPage"
    Title="Rejestracja">
    <ScrollView>
        <VerticalStackLayout Padding="30" Spacing="15">

            <Label Text="Rejestracja" FontSize="28" FontAttributes="Bold" HorizontalOptions="Center" />

            <Entry x:Name="EntryImie"
                Placeholder="Imię"
                TextChanged="OnFieldChanged" />

            <Entry x:Name="EntryEmail"
                Placeholder="Adres e-mail"
                Keyboard="Email"
                TextChanged="OnFieldChanged" />

            <Label x:Name="LabelEmail" Text="" FontSize="12" />

```

```

        <Entry x:Name="EntryHaslo1"
            Placeholder="Hasło (min. 6 znaków)"
            IsPassword="True"
            TextChanged="OnFieldChanged" />

        <Entry x:Name="EntryHaslo2"
            Placeholder="Powtórz hasło"
            IsPassword="True"
            TextChanged="OnFieldChanged" />

        <Label x:Name="LabelHasla" Text="" FontSize="12" />

        <Label x:Name="LabelStatus" Text="" FontSize="14" HorizontalOptions="Center" />

        <Button x:Name="BtnRejestruj"
            Text="Zarejestruj"
            IsEnabled="False"
            Clicked="OnRejestrujClicked" />

    </VerticalStackLayout>
</ScrollView>
</ContentPage>

```

C# CODE-BEHIND

```

namespace MojaAplikacja;

public partial class RegisterPage : ContentPage
{
    public RegisterPage()
    {
        InitializeComponent();

        // Walidacja w czasie rzeczywistym przy każdej zmianie tekstu
        private void OnFieldChanged(object sender, TextChangedEventArgs e)
        {
            bool imieOk = !string.IsNullOrWhiteSpace(EntryImie.Text);

            // Walidacja e-mail - musi zawierać @
            bool emailOk = !string.IsNullOrWhiteSpace(EntryEmail.Text)
                && EntryEmail.Text.Contains("@");

            if (!string.IsNullOrWhiteSpace(EntryEmail.Text) && !emailOk)
            {
                LabelEmail.Text = "E-mail musi zawierać znak @";
                LabelEmail.TextColor = Colors.Red;
            }
            else if (emailOk)
            {
                LabelEmail.Text = "E-mail poprawny";
                LabelEmail.TextColor = Colors.Green;
            }
        }
    }
}

```

```

    }
    else
    {
        LabelEmail.Text = "";
    }

    // Walidacja hasła – minimum 6 znaków
    bool hasloOk = !string.IsNullOrEmpty(EntryHaslo1.Text)
        && EntryHaslo1.Text.Length ≥ 6;

    // Walidacja zgodności haseł
    bool haslaZgodne = hasloOk
        && EntryHaslo1.Text == EntryHaslo2.Text;

    if (!string.IsNullOrEmpty(EntryHaslo2.Text) && !haslaZgodne)
    {
        LabelHasla.Text = "Hasła nie są zgodne lub są za krótkie";
        LabelHasla.TextColor = Colors.Red;
    }
    else if (haslaZgodne)
    {
        LabelHasla.Text = "Hasła zgodne ✓";
        LabelHasla.TextColor = Colors.Green;
    }
    else
    {
        LabelHasla.Text = "";
    }

    // Aktywacja przycisku tylko gdy wszystko jest poprawne
    BtnRejestruj.IsEnabled = imieOk && emailOk && haslaZgodne;

    // Komunikat statusu
    if (BtnRejestruj.IsEnabled)
    {
        LabelStatus.Text = "Wszystkie pola poprawne";
        LabelStatus.TextColor = Colors.Green;
    }
    else
    {
        LabelStatus.Text = "Uzupełnij formularz poprawnie";
        LabelStatus.TextColor = Colors.OrangeRed;
    }
}

private async void OnRejestrujClicked(object sender, EventArgs e)
{
    await DisplayAlert("Sukces",
        $"Konto dla {EntryImie.Text.Trim()} zostało utworzone!", "OK");
}
}

```

23.18. Formularz kontaktowy

Formularz do wysyłania wiadomości kontaktowej. Walidacja: imię i treść nie mogą być puste, e-mail musi zawierać , treść musi mieć minimum 10 znaków.

XAML

```
<?xml version="1.0" encoding="utf-8" ?>
<ContentPage xmlns="http://schemas.microsoft.com/dotnet/2021/maui"
              xmlns:x="http://schemas.microsoft.com/winfx/2009/xaml"
              x:Class="MojaAplikacja.ContactPage"
              Title="Kontakt">
    <ScrollView>
        <VerticalStackLayout Padding="30" Spacing="15">

            <Label Text="Formularz kontaktowy" FontSize="24" FontAttributes="Bold"/>

            <Entry x:Name="EntryImie"
                  Placeholder="Twoje imię"
                  TextChanged="OnTextChanged" />

            <Entry x:Name="EntryEmail"
                  Placeholder="Twój e-mail"
                  Keyboard="Email"
                  TextChanged="OnTextChanged" />

            <Entry x:Name="EntryTemat"
                  Placeholder="Temat wiadomości"
                  TextChanged="OnTextChanged" />

            <Editor x:Name="EditorTresc"
                  Placeholder="Treść wiadomości (min. 10 znaków)"
                  HeightRequest="150"
                  TextChanged="OnEditorChanged" />

            <Label x:Name="LabelLicznik" Text="0/10 znaków" FontSize="12" TextColor="Gray"/>

            <Label x:Name="LabelWalidacja" Text="" FontSize="14"/>

            <Button x:Name="BtnWyslij"
                   Text="Wyślij wiadomość"
                   IsEnabled="False"
                   Clicked="OnWyslijClicked" />

        </VerticalStackLayout>
    </ScrollView>
</ContentPage>
```

C# CODE-BEHIND

```
namespace MojaAplikacja;

public partial class ContactPage : ContentPage
{
```

```
public ContactPage()
{
    InitializeComponent();
}

private void OnTextChanged(object sender, TextChangedEventArgs e)
{
    SprawdzFormularz();
}

private void OnEditorChanged(object sender, TextChangedEventArgs e)
{
    // Licznik znaków w treści
    int dlugosc = EditorTresc.Text?.Length ?? 0;
    LabelLicznik.Text = $"{dlugosc}/10 znaków";
    LabelLicznik.TextColor = dlugosc ≥ 10 ? Colors.Green : Colors.Red;

    SprawdzFormularz();
}

// Centralna metoda walidacji
private void SprawdzFormularz()
{
    bool imieOk = !string.IsNullOrEmpty(EntryImie.Text);
    bool emailOk = !string.IsNullOrEmpty(EntryEmail.Text)
        && EntryEmail.Text.Contains("@");
    bool tematOk = !string.IsNullOrEmpty(EntryTemat.Text);
    bool trescOk = !string.IsNullOrEmpty(EditorTresc.Text)
        && EditorTresc.Text.Length ≥ 10;

    BtnWyslij.IsEnabled = imieOk && emailOk && tematOk && trescOk;

    // Komunikat walidacji z kolorem
    if (BtnWyslij.IsEnabled)
    {
        LabelWalidacja.Text = "Formularz gotowy do wysłania";
        LabelWalidacja.TextColor = Colors.Green;
    }
    else
    {
        LabelWalidacja.Text = "Wypełnij wszystkie pola poprawnie";
        LabelWalidacja.TextColor = Colors.Red;
    }
}

private async void OnWyslijClicked(object sender, EventArgs e)
{
    await DisplayAlert("Wysłano",
        $"Wiadomość od {EntryImie.Text.Trim()} została wysłana.", "OK");

    // Czyszczenie formularza po wysłaniu
    EntryImie.Text = "";
    EntryEmail.Text = "";
    EntryTemat.Text = "";
}
```

```
        EditorTresc.Text = "";
    }
}
```

23.19. Formularz dodawania produktu

Formularz do dodawania produktu do sklepu. Walidacja: nazwa niepusta, cena musi być liczbą (`double.TryParse`) większą od 0, ilość musi być liczbą całkowitą (`int.TryParse`) z zakresu 1–9999.

XAML

```
<?xml version="1.0" encoding="utf-8" ?>
<ContentPage xmlns="http://schemas.microsoft.com/dotnet/2021/maui"
              xmlns:x="http://schemas.microsoft.com/winfx/2009/xaml"
              x:Class="MojaAplikacja.ProductPage"
              Title="Dodaj produkt">
    <ScrollView>
        <VerticalStackLayout Padding="30" Spacing="15">

            <Label Text="Dodawanie produktu" FontSize="24" FontAttributes="Bold" />

            <Entry x:Name="EntryNazwa"
                  Placeholder="Nazwa produktu"
                  TextChanged="OnFieldChanged" />

            <Entry x:Name="EntryCena"
                  Placeholder="Cena (np. 29.99)"
                  Keyboard="Numeric"
                  TextChanged="OnFieldChanged" />

            <Label x:Name="LabelCena" Text="" FontSize="12" />

            <Entry x:Name="EntryIlosc"
                  Placeholder="Ilość (1-9999)"
                  Keyboard="Numeric"
                  TextChanged="OnFieldChanged" />

            <Label x:Name="LabelIlosc" Text="" FontSize="12" />

            <Entry x:Name="EntryOpis"
                  Placeholder="Opis produktu (opcjonalnie)" />

            <Picker x:Name="PickerKategoria" Title="Wybierz kategorię">
                <Picker.ItemsSource>
                    <x:Array Type="{x:Type x:String}">
                        <x:String>Elektronika</x:String>
                        <x:String>Odzież</x:String>
                        <x:String>Żywność</x:String>
                        <x:String>Sport</x:String>
                        <x:String>Inne</x:String>
                    </x:Array>
                </Picker.ItemsSource>
            </Picker>
        </VerticalStackLayout>
    </ScrollView>
</ContentPage>
```

```

        <Label x:Name="LabelStatus" Text="" FontSize="14" HorizontalOptions="Center"/>

        <Button x:Name="BtnDodaj"
            Text="Dodaj produkt"
            IsEnabled="False"
            Clicked="OnDodajClicked"/>

    </VerticalStackLayout>
</ScrollView>
</ContentPage>

```

C# CODE-BEHIND

```

namespace MojaAplikacja;

public partial class ProductPage : ContentPage
{
    public ProductPage()
    {
        InitializeComponent();

        private void OnFieldChanged(object sender, TextChangedEventArgs e)
        {
            // Walidacja nazwy – nie może być pusta
            bool nazwaOk = !string.IsNullOrEmpty(EntryNazwa.Text);

            // Walidacja ceny – double.TryParse, większa od 0
            bool cenaOk = double.TryParse(EntryCena.Text, out double cena) && cena > 0;
            if (!string.IsNullOrEmpty(EntryCena.Text) && !cenaOk)
            {
                LabelCena.Text = "Podaj prawidłową cenę (liczbę > 0)";
                LabelCena.TextColor = Colors.Red;
            }
            else if (cenaOk)
            {
                LabelCena.Text = $"Cena: {cena:F2} zł ✓";
                LabelCena.TextColor = Colors.Green;
            }
            else
            {
                LabelCena.Text = "";
            }

            // Walidacja ilości – int.TryParse, zakres 1-9999
            bool iloscOk = int.TryParse(EntryIlosc.Text, out int ilosc)
                && ilosc ≥ 1 && ilosc ≤ 9999;
            if (!string.IsNullOrEmpty(EntryIlosc.Text) && !iloscOk)
            {
                LabelIlosc.Text = "Podaj liczbę całkowitą z zakresu 1-9999";
                LabelIlosc.TextColor = Colors.Red;
            }
        }
    }
}

```

```

        else if (iloscOk)
        {
            LabelIlosc.Text = $"Ilość: {ilosc} szt. ✓";
            LabelIlosc.TextColor = Colors.Green;
        }
        else
        {
            LabelIlosc.Text = "";
        }

        // Aktywacja przycisku
        BtnDodaj.IsEnabled = nazwaOk && cenaOk && iloscOk;

        LabelStatus.Text = BtnDodaj.IsEnabled ? "Gotowe do dodania" : "";
        LabelStatus.TextColor = Colors.Green;
    }

    private async void OnDodajClicked(object sender, EventArgs e)
    {
        string kategoria = PickerKategoria.SelectedItem?.ToString() ?? "Brak kategorii";
        double.TryParse(EntryCena.Text, out double cena);
        int.TryParse(EntryIlosc.Text, out int ilosc);

        await DisplayAlert("Produkt dodany",
            $"Nazwa: {EntryNazwa.Text.Trim()}\n" +
            $"Cena: {cena:F2} zł\n" +
            $"Ilość: {ilosc} szt.\n" +
            $"Kategoria: {kategoria}", "OK");
    }
}

```

23.20. Ankieta (RadioButton i CheckBox)

Formularz ankiety wykorzystujący RadioButton do pytań jednokrotnego wyboru i CheckBox do pytań wielokrotnego wyboru.

XAML

```

<?xml version="1.0" encoding="utf-8" ?>
<ContentPage xmlns="http://schemas.microsoft.com/dotnet/2021/maui"
    xmlns:x="http://schemas.microsoft.com/winfx/2009/xaml"
    x:Class="MojaAplikacja.SurveyPage"
    Title="Ankieta">
    <ScrollView>
        <VerticalStackLayout Padding="30" Spacing="15">

            <Label Text="Ankieta satysfakcji" FontSize="24" FontAttributes="Bold"/>

            <!-- Pytanie 1 - RadioButton (jednokrotny wybór) -->
            <Label Text="1. Jak oceniasz naszą usługę?" FontSize="16" FontAttributes="Bold"/>
            <RadioButton x:Name="RadioSwietna" Content="Świetna" GroupName="Ocena"/>
            <RadioButton x:Name="RadioDobra" Content="Dobra" GroupName="Ocena"/>
            <RadioButton x:Name="RadioPrzecietna" Content="Przeciętna" GroupName="Ocena"/>
            <RadioButton x:Name="RadioSlaba" Content="Słaba" GroupName="Ocena"/>

```


←!— Pytanie 2 – RadioButton →

```
<Label Text="2. Czy polecisz nas znajomym?" FontSize="16" FontAttributes="Bold" Margin="0,15,0,0" />
<RadioButton x:Name="RadioTak" Content="Tak" GroupName="Polecenie" />
<RadioButton x:Name="RadioNie" Content="Nie" GroupName="Polecenie" />
<RadioButton x:Name="RadioNieWiem" Content="Nie wiem" GroupName="Polecenie" />
```

←!— Pytanie 3 – CheckBox (wielokrotny wybór) →

```
<Label Text="3. Co Ci się podobało? (możesz wybrać kilka)" FontSize="16" FontAttributes="Bold" Margin="0,15,0,0" />

<HorizontalStackLayout Spacing="10">
    <CheckBox x:Name="ChkObsluga" />
    <Label Text="Obsługa klienta" VerticalOptions="Center" />
</HorizontalStackLayout>
```

```
<HorizontalStackLayout Spacing="10">
    <CheckBox x:Name="ChkJakosc" />
    <Label Text="Jakość produktu" VerticalOptions="Center" />
</HorizontalStackLayout>
```

```
<HorizontalStackLayout Spacing="10">
    <CheckBox x:Name="ChkCena" />
    <Label Text="Cena" VerticalOptions="Center" />
</HorizontalStackLayout>
```

```
<HorizontalStackLayout Spacing="10">
    <CheckBox x:Name="ChkSzybkosc" />
    <Label Text="Szybkość realizacji" VerticalOptions="Center" />
</HorizontalStackLayout>
```

←!— Uwagi dodatkowe →

```
<Label Text="4. Uwagi dodatkowe:" FontSize="16" FontAttributes="Bold" Margin="0,15,0,0" />
<Editor x:Name="EditorUwagi" Placeholder="Wpisz swoje uwagi..." HeightRequest="100" />
```

```
<Label x:Name="LabelWalidacja" Text="" FontSize="14" />
```

```
<Button x:Name="BtnWyslij"
        Text="Wyślij ankietę"
        Clicked="OnWyslijClicked" />
```

```
</VerticalStackLayout>
```

```
</ScrollView>
```

```
</ContentPage>
```

C# CODE-BEHIND

```
namespace MojaAplikacja;

public partial class SurveyPage : ContentPage
{
    public SurveyPage()
    {
        InitializeComponent();
    }
}
```

```

}

private async void OnWyslijClicked(object sender, EventArgs e)
{
    // Walidacja – sprawdzenie czy wybrano odpowiedzi na pytania z RadioButton
    string ocena = GetSelectedRadio("Ocena");
    string polecenie = GetSelectedRadio("Polecenie");

    if (string.IsNullOrEmpty(ocena))
    {
        LabelWalidacja.Text = "Wybierz ocenę w pytaniu 1!";
        LabelWalidacja.TextColor = Colors.Red;
        return;
    }

    if (string.IsNullOrEmpty(polecenie))
    {
        LabelWalidacja.Text = "Odpowiedz na pytanie 2!";
        LabelWalidacja.TextColor = Colors.Red;
        return;
    }

    // Zbieranie zaznaczonych CheckBox-ów
    List<string> podobało = new();
    if (ChkObsluga.IsChecked) podobało.Add("Obsługa klienta");
    if (ChkJakosc.IsChecked) podobało.Add("Jakość produktu");
    if (ChkCena.IsChecked) podobało.Add("Cena");
    if (ChkSzybkosc.IsChecked) podobało.Add("Szybkość realizacji");

    string wybraneStr = podobało.Count > 0
        ? string.Join(", ", podobało)
        : "Brak wyboru";

    LabelWalidacja.Text = "Ankieta wysłana ✓";
    LabelWalidacja.TextColor = Colors.Green;

    await DisplayAlert("Wyniki ankiety",
        $"Ocena: {ocena}\n" +
        $"Polecenie: {polecenie}\n" +
        $"Podobało się: {wybraneStr}\n" +
        $"Uwagi: {EditorUwagi.Text ?? "(brak)"}", "OK");
}

// Pomocnicza metoda do odczytu zaznaczonego RadioButton
private string GetSelectedRadio(string groupName)
{
    // Przeszukujemy wszystkie RadioButton na stronie
    if (groupName == "Ocena")
    {
        if (RadioSwietna.IsChecked) return "Świetna";
        if (RadioDobra.IsChecked) return "Dobra";
        if (RadioPrzecietna.IsChecked) return "Przeciętna";
        if (RadioSlaba.IsChecked) return "Słaba";
    }
}

```

```

        else if (groupName == "PoIecenie")
        {
            if (RadioTak.IsChecked) return "Tak";
            if (RadioNie.IsChecked) return "Nie";
            if (RadioNieWiem.IsChecked) return "Nie wiem";
        }
        return null;
    }
}

```

XAML

```

<?xml version="1.0" encoding="utf-8" ?>
<ContentPage xmlns="http://schemas.microsoft.com/dotnet/2021/maui"
              xmlns:x="http://schemas.microsoft.com/winfx/2009/xaml"
              x:Class="MojaAplikacja.BookingPage"
              Title="Rezerwacja wizyty">
    <ScrollView>
        <VerticalStackLayout Padding="30" Spacing="15">

            <Label Text="Rezerwacja wizyty" FontSize="24" FontAttributes="Bold"/>

            <Entry x:Name="EntryImie"
                  Placeholder="Imię i nazwisko"
                  TextChanged="OnFormChanged" />

            <Label Text="Wybierz specjalistę:" FontSize="14" />
            <Picker x:Name="PickerSpecjalista"
                  Title="Specjalista"
                  SelectedIndexChanged="OnFormChanged">
                <Picker.ItemsSource>
                    <x:Array Type="{x:Type x:String}">
                        <x:String>Dr Kowalski – stomatolog</x:String>
                        <x:String>Dr Nowak – okulista</x:String>
                        <x:String>Dr Wiśniewska – dermatolog</x:String>
                        <x:String>Dr Zieliński – ortopeda</x:String>
                    </x:Array>
                </Picker.ItemsSource>
            </Picker>

            <Label Text="Data wizyty:" FontSize="14" />
            <DatePicker x:Name="DpData"
                      MinimumDate="{x:Static sys:DateTime.Today}"
                      xmlns:sys="clr-namespace:System;assembly=netstandard"
                      DateSelected="OnDateSelected" />

            <Label Text="Godzina wizyty:" FontSize="14" />
            <TimePicker x:Name="TpGodzina" Time="09:00:00" />

            <Label Text="Liczba osób:" FontSize="14" />
            <Slider x:Name="SliderOsoby"

```

```

        Minimum="1" Maximum="5"
        Value="1"
        ValueChanged="OnSliderChanged" />
<Label x:Name="LabelOsoby" Text="Liczba osób: 1" FontSize="14"/>

<Label x:Name="LabelPodsumowanie" Text="" FontSize="14" TextColor="Gray"/>

<Button x:Name="BtnRezerwuj"
        Text="Zarezerwuj wizytę"
        IsEnabled="False"
        Clicked="OnRezerwujClicked" />

</VerticalStackLayout>
</ScrollView>
</ContentPage>

```

C# CODE-BEHIND

```

namespace MojaAplikacja;

public partial class BookingPage : ContentPage
{
    public BookingPage()
    {
        InitializeComponent();

        // Wywoływane przy zmianie tekstu w Entry
        private void OnFormChanged(object sender, EventArgs e)
        {
            SprawdzFormularz();
        }

        // Obsługa zmiany daty
        private void OnDateSelected(object sender, DateChangedEventArgs e)
        {
            SprawdzFormularz();
        }

        // Obsługa zmiany Slidera – wyświetla aktualną wartość
        private void OnSliderChanged(object sender, ValueChangedEventArgs e)
        {
            int osoby = (int)Math.Round(e.NewValue);
            SliderOsoby.Value = osoby; // Przyciąganie do wartości całkowitych
            LabelOsoby.Text = $"Liczba osób: {osoby}";
            SprawdzFormularz();
        }

        // Centralna walidacja formularza
        private void SprawdzFormularz()
        {
            bool imieOk = !string.IsNullOrWhiteSpace(EntryImie.Text);
            bool specjalistaOk = PickerSpecjalista.SelectedIndex ≥ 0;

```

```

        BtnRezerwuj.IsEnabled = imieOk && specjalistaOk;

        // Podsumowanie rezerwacji
        if (BtnRezerwuj.IsEnabled)
        {
            int osoby = (int)Math.Round(SliderOsoby.Value);
            LabelPodsumowanie.Text =
                $"Rezerwacja: {PickerSpecjalista.SelectedItem}, " +
                $"{DpData.Date:dd.MM.yyyy} o {TpGodzina.Time}, " +
                $"{osoby} os.";
        }
        else
        {
            LabelPodsumowanie.Text = "";
        }
    }

    private async void OnRezerwujClicked(object sender, EventArgs e)
    {
        int osoby = (int)Math.Round(SliderOsoby.Value);

        await DisplayAlert("Rezerwacja potwierdzona",
            $"Pacjent: {EntryImie.Text.Trim()}\n" +
            $"Specjalista: {PickerSpecjalista.SelectedItem}\n" +
            $"Data: {DpData.Date:dd.MM.yyyy}\n" +
            $"Godzina: {TpGodzina.Time}\n" +
            $"Liczba osób: {osoby}", "OK");
    }
}

```

23.21. Formularz ustawień (Switch)

Formularz ustawień aplikacji z przełącznikami Switch. Wynik wyświetlany dynamicznie.

XAML

```

<?xml version="1.0" encoding="utf-8" ?>
<ContentPage xmlns="http://schemas.microsoft.com/dotnet/2021/maui"
    xmlns:x="http://schemas.microsoft.com/winfx/2009/xaml"
    x:Class="MojAplikacja.SettingsPage"
    Title="Ustawienia">
    <ScrollView>
        <VerticalStackLayout Padding="30" Spacing="20">

            <Label Text="Ustawienia aplikacji" FontSize="24" FontAttributes="Bold"/>

            <!-- Powiadomienia -->
            <HorizontalStackLayout Spacing="15">
                <Switch x:Name="SwitchPowiadomienia" Toggled="OnSwitchToggled"/>
                <Label Text="Powiadomienia push" VerticalOptions="Center" FontSize="16"/>
            </HorizontalStackLayout>

```

```

<!-- Tryb ciemny -->
<HorizontalStackLayout Spacing="15">
    <Switch x:Name="SwitchTrybCiemny" Toggled="OnSwitchToggled" />
    <Label Text="Tryb ciemny" VerticalOptions="Center" FontSize="16" />
</HorizontalStackLayout>

<!-- Lokalizacja -->
<HorizontalStackLayout Spacing="15">
    <Switch x:Name="SwitchLokalizacja" Toggled="OnSwitchToggled" />
    <Label Text="Udostępnianie lokalizacji" VerticalOptions="Center" FontSize="16" />
</HorizontalStackLayout>

<!-- Automatyczne aktualizacje -->
<HorizontalStackLayout Spacing="15">
    <Switch x:Name="SwitchAktualizacje" Toggled="OnSwitchToggled" />
    <Label Text="Automatyczne aktualizacje" VerticalOptions="Center" FontSize="16" />
</HorizontalStackLayout>

<!-- Dźwięki -->
<HorizontalStackLayout Spacing="15">
    <Switch x:Name="SwitchDzwieki" Toggled="OnSwitchToggled" />
    <Label Text="Dźwięki aplikacji" VerticalOptions="Center" FontSize="16" />
</HorizontalStackLayout>

<!-- Podsumowanie aktywnych ustawień -->
<Label x:Name="LabelPodsumowanie"
    Text="Aktywne ustawienia: brak"
    FontSize="14"
    TextColor="Gray"
    Margin="0,20,0,0" />

<Button Text="Zapisz ustawienia"
    Clicked="OnZapiszClicked" />

</VerticalStackLayout>
</ScrollView>
</ContentPage>

```

C# CODE-BEHIND

```

namespace MojaAplikacja;

public partial class SettingsPage : ContentPage
{
    public SettingsPage()
    {
        InitializeComponent();

        // Wywoływane przy każdej zmianie dowolnego Switch
        private void OnSwitchToggled(object sender, ToggledEventArgs e)
        {
            // Zbieranie aktywnych ustawień

```

```

List<string> aktywne = new();

if (SwitchPowiadomienia.IsToggled) aktywne.Add("Powiadomienia");
if (SwitchTrybCiemny.IsToggled) aktywne.Add("Tryb ciemny");
if (SwitchLokalizacja.IsToggled) aktywne.Add("Lokalizacja");
if (SwitchAktualizacje.IsToggled) aktywne.Add("Aktualizacje");
if (SwitchDzwieki.IsToggled) aktywne.Add("Dźwięki");

// Wyświetlanie podsumowania
if (aktywne.Count > 0)
{
    LabelPodsumowanie.Text = $"Aktywne ({aktywne.Count}): {string.Join(", ", aktywne)}";
    LabelPodsumowanie.TextColor = Colors.DarkGreen;
}
else
{
    LabelPodsumowanie.Text = "Aktywne ustawienia: brak";
    LabelPodsumowanie.TextColor = Colors.Gray;
}
}

private async void OnZapiszClicked(object sender, EventArgs e)
{
    // Zapisanie ustawień (np. w Preferences)
    Preferences.Set("Powiadomienia", SwitchPowiadomienia.IsToggled);
    Preferences.Set("TrybCiemny", SwitchTrybCiemny.IsToggled);
    Preferences.Set("Lokalizacja", SwitchLokalizacja.IsToggled);
    Preferences.Set("Aktualizacje", SwitchAktualizacje.IsToggled);
    Preferences.Set("Dzwieki", SwitchDzwieki.IsToggled);

    await DisplayAlert("Zapisano", "Ustawienia zostały zapisane.", "OK");
}
}

```

23.22. Formularz wielosekcyjny (Border)

Formularz podzielony na sekcje wizualne przy użyciu kontrolki **Border**. Każda sekcja jest oddzielona ramką z zaokrąglonymi rogami.

XAML

```

<?xml version="1.0" encoding="utf-8" ?>
<ContentPage xmlns="http://schemas.microsoft.com/dotnet/2021/maui"
    xmlns:x="http://schemas.microsoft.com/winfx/2009/xaml"
    x:Class="MojaAplikacja.MultiSectionPage"
    Title="Formularz zamówienia">
    <ScrollView>
        <VerticalStackLayout Padding="20" Spacing="20">

            <Label Text="Formularz zamówienia" FontSize="24" FontAttributes="Bold" HorizontalOptions="Center" />

            <!-- Sekcja 1: Dane osobowe -->
            <Border Stroke="Gray" StrokeThickness="1" Padding="15"
                StrokeShape="RoundRectangle 10">

```

```

        <VerticalStackLayout Spacing="10">
            <Label Text="👤 Dane osobowe" FontSize="18" FontAttributes="Bold"/>
            <Entry x:Name="EntryImie" Placeholder="Imię" TextChanged="OnFieldChanged"/>
            <Entry x:Name="EntryNazwisko" Placeholder="Nazwisko" TextChanged="OnFieldChanged"/>
            <Entry x:Name="EntryTelefon" Placeholder="Telefon" Keyboard="Telephone" TextChanged="OnFieldChanged"/>
        </VerticalStackLayout>
    </Border>

    <!-- Sekcja 2: Adres dostawy -->
    <Border Stroke="Gray" StrokeThickness="1" Padding="15"
        StrokeShape="RoundRectangle 10">
        <VerticalStackLayout Spacing="10">
            <Label Text="📍 Adres dostawy" FontSize="18" FontAttributes="Bold"/>
            <Entry x:Name="EntryUlica" Placeholder="Ulica i numer" TextChanged="OnFieldChanged"/>
            <Entry x:Name="EntryMiasto" Placeholder="Miasto" TextChanged="OnFieldChanged"/>
            <Entry x:Name="EntryKod" Placeholder="Kod pocztowy (np. 00-000)" TextChanged="OnFieldChanged"/>
        </VerticalStackLayout>
    </Border>

    <!-- Sekcja 3: Szczegóły zamówienia -->
    <Border Stroke="Gray" StrokeThickness="1" Padding="15"
        StrokeShape="RoundRectangle 10">
        <VerticalStackLayout Spacing="10">
            <Label Text="📦 Szczegóły zamówienia" FontSize="18" FontAttributes="Bold"/>
            <Entry x:Name="EntryProdukt" Placeholder="Nazwa produktu" TextChanged="OnFieldChanged"/>
            <Entry x:Name="EntryIloscSzt" Placeholder="Ilość (liczba)" Keyboard="Numeric" TextChanged="OnFieldChanged"/>
            <Editor x:Name="EditorUwagi" Placeholder="Uwagi do zamówienia" HeightRequest="80"/>
        </VerticalStackLayout>
    </Border>

    <!-- Status -->
    <Label x:Name="LabelStatus" Text="" FontSize="14" HorizontalOptions="Center"/>

    <Button x:Name="BtnZloz"
        Text="Złóż zamówienie"
        IsEnabled="False"
        Clicked="OnZlozClicked"/>

</VerticalStackLayout>
</ScrollView>
</ContentPage>

```

C# CODE-BEHIND

```

namespace MojaAplikacja;

public partial class MultiSectionPage : ContentPage
{
    public MultiSectionPage()
    {
        InitializeComponent();
    }
}

```



```

private void OnFieldChanged(object sender, TextChangedEventArgs e)
{
    // Walidacja sekcji 1
    bool imieOk = !string.IsNullOrEmpty(EntryImie.Text);
    bool nazwiskoOk = !string.IsNullOrEmpty(EntryNazwisko.Text);
    bool telefonOk = !string.IsNullOrEmpty(EntryTelefon.Text)
        && EntryTelefon.Text.Length ≥ 9;

    // Walidacja sekcji 2
    bool ulicaOk = !string.IsNullOrEmpty(EntryUlica.Text);
    bool miastoOk = !string.IsNullOrEmpty(EntryMiasto.Text);
    bool kodOk = !string.IsNullOrEmpty(EntryKod.Text);

    // Walidacja sekcji 3
    bool produktOk = !string.IsNullOrEmpty(EntryProdukt.Text);
    bool iloscOk = int.TryParse(EntryIloscSzt.Text, out int ilosc)
        && ilosc ≥ 1 && ilosc ≤ 100;

    // łączna walidacja
    bool wszystkoOk = imieOk && nazwiskoOk && telefonOk
        && ulicaOk && miastoOk && kodOk
        && produktOk && iloscOk;

    BtnZloz.IsEnabled = wszystkoOk;

    // Komunikat statusu z kolorem
    if (wszystkoOk)
    {
        LabelStatus.Text = "✓ Formularz kompletny – możesz złożyć zamówienie";
        LabelStatus.TextColor = Colors.Green;
    }
    else
    {
        LabelStatus.Text = "Wypełnij wszystkie wymagane pola";
        LabelStatus.TextColor = Colors.OrangeRed;
    }
}

private async void OnZlozClicked(object sender, EventArgs e)
{
    int.TryParse(EntryIloscSzt.Text, out int ilosc);

    await DisplayAlert("Zamówienie złożone",
        $"Klient: {EntryImie.Text} {EntryNazwisko.Text}\n" +
        $"Adres: {EntryUlica.Text}, {EntryKod.Text} {EntryMiasto.Text}\n" +
        $"Produkt: {EntryProdukt.Text} x{ilosc}\n" +
        $"Uwagi: {EditorUwagi.Text ?? "(brak)"}", "OK");
}
}

```

23.23. Kalkulator BMI

Kalkulator wskaźnika masy ciała. Użytkownik podaje wagę (kg) i wzrost (cm). Walidacja z `double.TryParse` i zakresami fizjologicznymi.

XAML

```
<?xml version="1.0" encoding="utf-8" ?>
<ContentPage xmlns="http://schemas.microsoft.com/dotnet/2021/maui"
              xmlns:x="http://schemas.microsoft.com/winfx/2009/xaml"
              x:Class="MojaAplikacja.BmiPage"
              Title="Kalkulator BMI">
    <VerticalStackLayout Padding="30" Spacing="15" VerticalOptions="Center">

        <Label Text="Kalkulator BMI" FontSize="28" FontAttributes="Bold" HorizontalOptions="Center"/>

        <Entry x:Name="EntryWaga"
              Placeholder="Waga (kg), np. 70.5"
              Keyboard="Numeric"
              TextChanged="OnValueChanged"/>

        <Label x:Name="LabelWaga" Text="" FontSize="12"/>

        <Entry x:Name="EntryWzrost"
              Placeholder="Wzrost (cm), np. 175"
              Keyboard="Numeric"
              TextChanged="OnValueChanged"/>

        <Label x:Name="LabelWzrost" Text="" FontSize="12"/>

        <Button x:Name="BtnOblicz"
              Text="Oblicz BMI"
              IsEnabled="False"
              Clicked="OnObliczClicked"/>

        <Label x:Name="LabelWynik"
              Text=""
              FontSize="22"
              FontAttributes="Bold"
              HorizontalOptions="Center"/>

        <Label x:Name="LabelKategoria"
              Text=""
              FontSize="16"
              HorizontalOptions="Center"/>

    </VerticalStackLayout>
</ContentPage>
```

C# CODE-BEHIND

```
namespace MojaAplikacja;

public partial class BmiPage : ContentPage
{
```

```
public BmiPage()
{
    InitializeComponent();
}

private void OnValueChanged(object sender, TextChangedEventArgs e)
{
    // Walidacja wagi - double.TryParse, zakres 20-300 kg
    bool wagaOk = double.TryParse(EntryWaga.Text, out double waga)
        && waga ≥ 20 && waga ≤ 300;
    if (!string.IsNullOrWhiteSpace(EntryWaga.Text) && !wagaOk)
    {
        LabelWaga.Text = "Waga musi być liczbą z zakresu 20-300 kg";
        LabelWaga.TextColor = Colors.Red;
    }
    else if (wagaOk)
    {
        LabelWaga.Text = $"{waga} kg ✓";
        LabelWaga.TextColor = Colors.Green;
    }
    else
    {
        LabelWaga.Text = "";
    }

    // Walidacja wzrostu - double.TryParse, zakres 50-250 cm
    bool wzrostOk = double.TryParse(EntryWzrost.Text, out double wzrost)
        && wzrost ≥ 50 && wzrost ≤ 250;
    if (!string.IsNullOrWhiteSpace(EntryWzrost.Text) && !wzrostOk)
    {
        LabelWzrost.Text = "Wzrost musi być liczbą z zakresu 50-250 cm";
        LabelWzrost.TextColor = Colors.Red;
    }
    else if (wzrostOk)
    {
        LabelWzrost.Text = $"{wzrost} cm ✓";
        LabelWzrost.TextColor = Colors.Green;
    }
    else
    {
        LabelWzrost.Text = "";
    }

    // Aktywacja przycisku gdy oba pola poprawne
    BtnOblicz.IsEnabled = wagaOk && wzrostOk;
}

private void OnObliczClicked(object sender, EventArgs e)
{
    double.TryParse(EntryWaga.Text, out double waga);
    double.TryParse(EntryWzrost.Text, out double wzrost);

    // Wzór BMI: waga(kg) / wzrost(m)^2
    double wzrostM = wzrost / 100.0;
```

```

double bmi = waga / (wzrostM * wzrostM);

LabelWynik.Text = $"BMI: {bmi:F1}";

// Kategoria BMI z kolorami
if (bmi < 18.5)
{
    LabelKategoria.Text = "Niedowaga";
    LabelKategoria.TextColor = Colors.Orange;
    LabelWynik.TextColor = Colors.Orange;
}
else if (bmi < 25)
{
    LabelKategoria.Text = "Waga prawidłowa";
    LabelKategoria.TextColor = Colors.Green;
    LabelWynik.TextColor = Colors.Green;
}
else if (bmi < 30)
{
    LabelKategoria.Text = "Nadwaga";
    LabelKategoria.TextColor = Colors.OrangeRed;
    LabelWynik.TextColor = Colors.OrangeRed;
}
else
{
    LabelKategoria.Text = "Otyłość";
    LabelKategoria.TextColor = Colors.Red;
    LabelWynik.TextColor = Colors.Red;
}
}
}

```

23.24. Formularz z dynamicznym podsumowaniem

Formularz zamówienia pizzy z dynamicznie aktualizowanym podsumowaniem. Każda zmiana jest natychmiast widoczna w sekcji podsumowania.

XAML

```

<?xml version="1.0" encoding="utf-8" ?>
<ContentPage xmlns="http://schemas.microsoft.com/dotnet/2021/maui"
    xmlns:x="http://schemas.microsoft.com/winfx/2009/xaml"
    x:Class="MojaAplikacja.PizzaOrderPage"
    Title="Zamów pizzę">
    <ScrollView>
        <VerticalStackLayout Padding="30" Spacing="15">

            <Label Text="🍕 Zamów pizzę" FontSize="28" FontAttributes="Bold" HorizontalOptions="Center" />

            <!-- Wybór rozmiaru -->
            <Label Text="Rozmiar:" FontSize="16" FontAttributes="Bold" />
            <RadioButton x:Name="RadioMała" Content="Mała (25 cm) – 20 zł"
                GroupName="Rozmiar" CheckedChanged="OnFormUpdated" />

```

```

<RadioButton x:Name="RadioSrednia" Content="Średnia (32 cm) – 28 zł"
              GroupName="Rozmiar" IsChecked="True" CheckedChanged="OnFormUpdated" />
<RadioButton x:Name="RadioDuza" Content="Duża (40 cm) – 35 zł"
              GroupName="Rozmiar" CheckedChanged="OnFormUpdated" />

<!-- Dodatki -->
<Label Text="Dodatki (po 4 zł):" FontSize="16" FontAttributes="Bold" Margin="0,10,0,0" />

<HorizontalStackLayout Spacing="10">
    <CheckBox x:Name="ChkSer" CheckedChanged="OnCheckChanged" />
    <Label Text="Dodatkowy ser" VerticalOptions="Center" />
</HorizontalStackLayout>

<HorizontalStackLayout Spacing="10">
    <CheckBox x:Name="ChkSzynka" CheckedChanged="OnCheckChanged" />
    <Label Text="Szynka" VerticalOptions="Center" />
</HorizontalStackLayout>

<HorizontalStackLayout Spacing="10">
    <CheckBox x:Name="ChkPieczarki" CheckedChanged="OnCheckChanged" />
    <Label Text="Pieczarki" VerticalOptions="Center" />
</HorizontalStackLayout>

<HorizontalStackLayout Spacing="10">
    <CheckBox x:Name="ChkOliwki" CheckedChanged="OnCheckChanged" />
    <Label Text="Oliwki" VerticalOptions="Center" />
</HorizontalStackLayout>

<!-- Ilość -->
<Label Text="Ilość sztuk:" FontSize="16" FontAttributes="Bold" Margin="0,10,0,0" />
<Slider x:Name="SliderIlosc" Minimum="1" Maximum="5" Value="1"
        ValueChanged="OnIloscChanged" />
<Label x:Name="LabelIlosc" Text="1 szt." FontSize="14" HorizontalOptions="Center" />

<!-- Dane dostawy -->
<Label Text="Adres dostawy:" FontSize="16" FontAttributes="Bold" Margin="0,10,0,0" />
<Entry x:Name="EntryAdres" Placeholder="Ulica, numer, mieszkanie" TextChanged="OnAdresChanged" />

<!-- Dynamiczne podsumowanie -->
<Border Stroke="DarkGreen" StrokeThickness="2" Padding="15"
        StrokeShape="RoundRectangle 10" Margin="0,15,0,0">
    <VerticalStackLayout Spacing="5">
        <Label Text="📦 Podsumowanie:" FontSize="18" FontAttributes="Bold" />
        <Label x:Name="LabelPodRozmiar" Text="Rozmiar: Średnia" FontSize="14" />
        <Label x:Name="LabelPodDodatki" Text="Dodatki: brak" FontSize="14" />
        <Label x:Name="LabelPodIlosc" Text="Ilość: 1 szt." FontSize="14" />
        <Label x:Name="LabelPodAdres" Text="Adres: –" FontSize="14" />
        <Label x:Name="LabelPodCena" Text="RAZEM: 28,00 zł"
              FontSize="20" FontAttributes="Bold" TextColor="DarkGreen" />
    </VerticalStackLayout>
</Border>

<Button x:Name="BtnZamow"
        Text="Złóż zamówienie"

```

```

        IsEnabled="False"
        Clicked="OnZamowClicked"/>

    </VerticalStackLayout>

</ScrollView>

</ContentPage>

```

C# CODE-BEHIND

```

namespace MojaAplikacja;

public partial class PizzaOrderPage : ContentPage
{
    public PizzaOrderPage()
    {
        InitializeComponent();
        AktualizujPodsumowanie();
    }

    private void OnFormUpdated(object sender, CheckedChangedEventArgs e)
    {
        AktualizujPodsumowanie();
    }

    private void OnCheckChanged(object sender, CheckedChangedEventArgs e)
    {
        AktualizujPodsumowanie();
    }

    private void OnIloscChanged(object sender, ValueChangedEventArgs e)
    {
        int ilosc = (int)Math.Round(e.NewValue);
        SliderIlosc.Value = ilosc;
        LabelIlosc.Text = $"{ilosc} szt.";
        AktualizujPodsumowanie();
    }

    private void OnAdresChanged(object sender, TextChangedEventArgs e)
    {
        AktualizujPodsumowanie();
    }

    // Dynamiczna aktualizacja podsumowania przy każdej zmianie
    private void AktualizujPodsumowanie()
    {
        // Oblicz cenę bazową wg rozmiaru
        string rozmiar;
        double cenaBase;

        if (RadioMała.IsChecked) { rozmiar = "Mała (25 cm)"; cenaBase = 20; }
        else if (RadioDuża.IsChecked) { rozmiar = "Duża (40 cm)"; cenaBase = 35; }
        else { rozmiar = "Średnia (32 cm)"; cenaBase = 28; }

        LabelPodRozmiar.Text = $"Rozmiar: {rozmiar}";
    }
}

```

```

// Oblicz dodatki
List<string> dodatki = new();
int liczbaDodatkow = 0;
if (ChkSer.IsChecked) { dodatki.Add("Ser"); liczbaDodatkow++; }
if (ChkSzynka.IsChecked) { dodatki.Add("Szynka"); liczbaDodatkow++; }
if (ChkPieczarki.IsChecked) { dodatki.Add("Pieczarki"); liczbaDodatkow++; }
if (ChkOliwki.IsChecked) { dodatki.Add("Oliwki"); liczbaDodatkow++; }

LabelPodDodatki.Text = dodatki.Count > 0
    ? $"Dodatki: {string.Join(", ", dodatki)}"
    : "Dodatki: brak";

// Ilość
int ilosc = (int)Math.Round(SliderIlosc.Value);
LabelPodIlosc.Text = $"Ilość: {ilosc} szt.";

// Adres
string adres = EntryAdres.Text;
bool adresOk = !string.IsNullOrWhiteSpace(adres);
LabelPodAdres.Text = adresOk ? $"Adres: {adres}" : "Adres: -";

// Cena końcowa
double cenaJednostkowa = cenaBase + (liczbaDodatkow * 4);
double cenaCalkowita = cenaJednostkowa * ilosc;
LabelPodCena.Text = $"RAZEM: {cenaCalkowita:F2} zł";

// Aktywacja przycisku – wymagany adres
BtnZamow.IsEnabled = adresOk;
}

private async void OnZamowClicked(object sender, EventArgs e)
{
    await DisplayAlert("Zamówienie przyjęte",
        $"Twoja pizza jest w drodze na adres:\n{EntryAdres.Text.Trim()}", "OK");
}
}

```

23.25. Formularz rejestracji na wydarzenie (walidacja długości tekstu i zakresu liczbowego)

Formularz rejestracji uczestnika na wydarzenie. Walidacja: imię min. 2 znaki, nazwisko min. 2 znaki, wiek (int.TryParse) w zakresie 16–120, e-mail z @, telefon min. 9 cyfr.

XAML

```

<?xml version="1.0" encoding="utf-8" ?>
<ContentPage xmlns="http://schemas.microsoft.com/dotnet/2021/maui"
    xmlns:x="http://schemas.microsoft.com/winfx/2009/xaml"
    x:Class="MojaAplikacja.EventRegisterPage"
    Title="Rejestracja na wydarzenie">
    <ScrollView>
        <VerticalStackLayout Padding="30" Spacing="12">

```

```
<Label Text="Rejestracja na wydarzenie" FontSize="24" FontAttributes="Bold" HorizontalOptions="Center"/>

<Entry x:Name="EntryImie"
    Placeholder="Imię (min. 2 znaki)"
    TextChanged="OnFieldChanged"/>
<Label x:Name="LblImie" Text="" FontSize="11"/>

<Entry x:Name="EntryNazwisko"
    Placeholder="Nazwisko (min. 2 znaki)"
    TextChanged="OnFieldChanged"/>
<Label x:Name="LblNazwisko" Text="" FontSize="11"/>

<Entry x:Name="EntryWiek"
    Placeholder="Wiek (16-120)"
    Keyboard="Numeric"
    TextChanged="OnFieldChanged"/>
<Label x:Name="LblWiek" Text="" FontSize="11"/>

<Entry x:Name="EntryEmail"
    Placeholder="E-mail"
    Keyboard="Email"
    TextChanged="OnFieldChanged"/>
<Label x:Name="LblEmail" Text="" FontSize="11"/>

<Entry x:Name="EntryTelefon"
    Placeholder="Telefon (min. 9 cyfr)"
    Keyboard="Telephone"
    TextChanged="OnFieldChanged"/>
<Label x:Name="LblTelefon" Text="" FontSize="11"/>

<Picker x:Name="PickerBilet" Title="Rodzaj biletu">
    <Picker.ItemsSource>
        <x:Array Type="{x:Type x:String}">
            <x:String>Standard – 50 zł</x:String>
            <x:String>Premium – 100 zł</x:String>
            <x:String>VIP – 200 zł</x:String>
        </x:Array>
    </Picker.ItemsSource>
</Picker>

<HorizontalStackLayout Spacing="10">
    <CheckBox x:Name="ChkRegulamin"/>
    <Label Text="Akceptuję regulamin wydarzenia" VerticalOptions="Center"/>
</HorizontalStackLayout>

<Label x:Name="LabelStatus" Text="" FontSize="14" HorizontalOptions="Center"/>

<Button x:Name="BtnRejestruj"
    Text="Zarejestruj się"
    IsEnabled="False"
    Clicked="OnRejestrujClicked"/>

</VerticalStackLayout>
```



```
</ScrollView>

</ContentPage>
```

C# CODE-BEHIND

```
namespace MojaAplikacja;

public partial class EventRegisterPage : ContentPage
{
    public EventRegisterPage()
    {
        InitializeComponent();
        // Następowanie zmiany CheckBoxa
        ChkRegulamin.CheckedChanged += (s, e) => Waliduj();
    }

    private void OnFieldChanged(object sender, TextChangedEventArgs e)
    {
        Waliduj();
    }

    private void Waliduj()
    {
        // Walidacja imienia – min. 2 znaki
        bool imieOk = !string.IsNullOrEmpty(EntryImie.Text)
            && EntryImie.Text.Trim().Length ≥ 2;
        UstawLabel(LblImie, imieOk, "Imię min. 2 znaki", EntryImie.Text);

        // Walidacja nazwiska – min. 2 znaki
        bool nazwiskoOk = !string.IsNullOrEmpty(EntryNazwisko.Text)
            && EntryNazwisko.Text.Trim().Length ≥ 2;
        UstawLabel(LblNazwisko, nazwiskoOk, "Nazwisko min. 2 znaki", EntryNazwisko.Text);

        // Walidacja wieku – int.TryParse, zakres 16-120
        bool wiekOk = int.TryParse(EntryWiek.Text, out int wiek)
            && wiek ≥ 16 && wiek ≤ 120;
        UstawLabel(LblWiek, wiekOk, "Wiek: liczba 16-120", EntryWiek.Text);

        // Walidacja e-mail – zawiera @
        bool emailOk = !string.IsNullOrEmpty(EntryEmail.Text)
            && EntryEmail.Text.Contains("@");
        UstawLabel(LblEmail, emailOk, "E-mail musi zawierać @", EntryEmail.Text);

        // Walidacja telefonu – min. 9 znaków
        bool telefonOk = !string.IsNullOrEmpty(EntryTelefon.Text)
            && EntryTelefon.Text.Trim().Length ≥ 9;
        UstawLabel(LblTelefon, telefonOk, "Min. 9 cyfr", EntryTelefon.Text);

        // Regulamin zaakceptowany
        bool regulaminOk = ChkRegulamin.IsChecked;

        // łączna walidacja
        bool ok = imieOk && nazwiskoOk && wiekOk && emailOk && telefonOk && regulaminOk;
```

```

        BtnRejestruj.IsEnabled = ok;

        if (ok)
        {
            LabelStatus.Text = "Formularz gotowy ✓";
            LabelStatus.TextColor = Colors.Green;
        }
        else
        {
            LabelStatus.Text = "Uzupełnij wymagane pola i zaakceptuj regulamin";
            LabelStatus.TextColor = Colors.Red;
        }
    }

    // Pomocnicza metoda do ustawiania etykiet walidacji
    private void UstawLabel(Label lbl, bool ok, string msgError, string fieldText)
    {
        if (string.IsNullOrEmpty(fieldText))
        {
            lbl.Text = "";
            return;
        }
        if (ok)
        {
            lbl.Text = "✓";
            lbl.TextColor = Colors.Green;
        }
        else
        {
            lbl.Text = msgError;
            lbl.TextColor = Colors.Red;
        }
    }

    private async void OnRejestrujClicked(object sender, EventArgs e)
    {
        string bilet = PickerBilet.SelectedItem?.ToString() ?? "Standard – 50 zł";

        await DisplayAlert("Rejestracja udana",
            $"Uczestnik: {EntryImie.Text.Trim()} {EntryNazwisko.Text.Trim()}\n" +
            $"Wiek: {EntryWiek.Text}\n" +
            $"E-mail: {EntryEmail.Text.Trim()}\n" +
            $"Bilet: {bilet}", "OK");
    }
}

```

23.26. Formularz zmiany hasła (porównanie starego i nowego)

Formularz umożliwiający zmianę hasła. Walidacja: stare hasło niepuste, nowe hasło min. 8 znaków, potwierdzenie musi się zgadzać z nowym hasłem, nowe hasło nie może być takie samo jak stare.

XAML

```
<?xml version="1.0" encoding="utf-8" ?>
<ContentPage xmlns="http://schemas.microsoft.com/dotnet/2021/maui"
              xmlns:x="http://schemas.microsoft.com/winfx/2009/xaml"
              x:Class="MojaAplikacja.ChangePasswordPage"
              Title="Zmiana hasła">
    <VerticalStackLayout Padding="30" Spacing="15" VerticalOptions="Center">

        <Label Text="Zmiana hasła" FontSize="24" FontAttributes="Bold" HorizontalOptions="Center"/>

        <Entry x:Name="EntryStareHaslo"
              Placeholder="Aktualne hasło"
              IsPassword="True"
              TextChanged="OnFieldChanged" />

        <Entry x:Name="EntryNoweHaslo"
              Placeholder="Nowe hasło (min. 8 znaków)"
              IsPassword="True"
              TextChanged="OnFieldChanged" />

        <Label x:Name="LblSilHasla" Text="" FontSize="12"/>

        <Entry x:Name="EntryPotwierdz"
              Placeholder="Potwierdź nowe hasło"
              IsPassword="True"
              TextChanged="OnFieldChanged" />

        <Label x:Name="LblZgodnosc" Text="" FontSize="12"/>

        <Label x:Name="LblStatus" Text="" FontSize="14" HorizontalOptions="Center"/>

        <Button x:Name="BtnZmien"
              Text="Zmień hasło"
              IsEnabled="False"
              Clicked="OnZmienClicked" />

    </VerticalStackLayout>
</ContentPage>
```

C# CODE-BEHIND

```
namespace MojaAplikacja;

public partial class ChangePasswordPage : ContentPage
{
    public ChangePasswordPage()
    {
        InitializeComponent();
    }

    private void OnFieldChanged(object sender, TextChangedEventArgs e)
    {
    }
}
```

```
string stare = EntryStareHaslo.Text ?? "";
string nowe = EntryNoweHaslo.Text ?? "";
string potwierdz = EntryPotwierdz.Text ?? "";

// Walidacja starego hasła – nie może być puste
bool stareOk = !string.IsNullOrEmpty(stare);

// Walidacja nowego hasła – min. 8 znaków
bool noweOk = nowe.Length ≥ 8;

// Siła hasła (prosty wskaźnik)
if (nowe.Length > 0)
{
    if (nowe.Length < 8)
    {
        LblSilaHasla.Text = "Za krótkie (min. 8 znaków)";
        LblSilaHasla.TextColor = Colors.Red;
    }
    else if (nowe.Length < 12)
    {
        LblSilaHasla.Text = "Siła: średnia";
        LblSilaHasla.TextColor = Colors.Orange;
    }
    else
    {
        LblSilaHasla.Text = "Siła: mocne ✓";
        LblSilaHasla.TextColor = Colors.Green;
    }
}
else
{
    LblSilaHasla.Text = "";
}

// Walidacja zgodności haseł
bool zgodne = noweOk && nowe == potwierdz;
if (potwierdz.Length > 0)
{
    if (zgodne)
    {
        LblZgodnosc.Text = "Hasła zgodne ✓";
        LblZgodnosc.TextColor = Colors.Green;
    }
    else
    {
        LblZgodnosc.Text = "Hasła się nie zgadzają";
        LblZgodnosc.TextColor = Colors.Red;
    }
}
else
{
    LblZgodnosc.Text = "";
}
```

```

// Nowe hasło nie może być takie samo jak stare
bool inneThanStare = stareOk && noweOk && stare != nowe;

if (stareOk && noweOk && stare == nowe)
{
    LblStatus.Text = "Nowe hasło musi być inne niż aktualne!";
    LblStatus.TextColor = Colors.Red;
}
else if (stareOk && zgodne && inneThanStare)
{
    LblStatus.Text = "Wszystko gotowe – możesz zmienić hasło";
    LblStatus.TextColor = Colors.Green;
}
else
{
    LblStatus.Text = "";
}

// Aktywacja przycisku
BtnZmien.IsEnabled = stareOk && zgodne && inneThanStare;
}

private async void OnZmienClicked(object sender, EventArgs e)
{
    await DisplayAlert("Sukces", "Hasło zostało zmienione.", "OK");

    // Czyszczenie pól
    EntryStareHaslo.Text = "";
    EntryNoweHaslo.Text = "";
    EntryPotwierdz.Text = "";
}
}

```

23.27. Formularz opinii z oceną gwiazdkową (Slider jako rating)

Formularz dodawania opinii o produkcie. Slider służy jako ocena w skali 1–5, dynamicznie wyświetlana jako gwiazdki.

XAML

```

<?xml version="1.0" encoding="utf-8" ?>
<ContentPage xmlns="http://schemas.microsoft.com/dotnet/2021/maui"
    xmlns:x="http://schemas.microsoft.com/winfx/2009/xaml"
    x:Class="MojaAplikacja.ReviewPage"
    Title="Dodaj opinię">
    <ScrollView>
        <VerticalStackLayout Padding="30" Spacing="15">

            <Label Text="Dodaj opinię" FontSize="24" FontAttributes="Bold" HorizontalOptions="Center"/>

            <Entry x:Name="EntryAutor"
                Placeholder="Twoje imię / nick"
                TextChanged="OnFieldChanged"/>

```

```

<Label Text="Ocena:" FontSize="16"/>
<Slider x:Name="SliderOcena" Minimum="1" Maximum="5" Value="3"
        ValueChanged="OnOcenaChanged" />
<Label x:Name="LabelGwiazdki" Text="☆☆☆ (3/5)"
        FontSize="20" HorizontalOptions="Center"/>

<Label Text="Tytuł opinii:" FontSize="14"/>
<Entry x:Name="EntryTytuł"
        Placeholder="Krótki tytuł (min. 3 znaki)"
        TextChanged="OnFieldChanged" />

<Label Text="Treść opinii:" FontSize="14"/>
<Editor x:Name="EditorTresc"
        Placeholder="Napisz swoją opinię (min. 20 znaków)"
        HeightRequest="120"
        TextChanged="OnEditorChanged" />

<Label x:Name="LabelLicznik" Text="0/20 znaków" FontSize="12" TextColor="Gray"/>

<HorizontalStackLayout Spacing="10">
    <CheckBox x:Name="ChkPolecam" />
    <Label Text="Polecam ten produkt" VerticalOptions="Center" />
</HorizontalStackLayout>

<Label x:Name="LabelStatus" Text="" FontSize="14" HorizontalOptions="Center" />

<Button x:Name="BtnDodajOpinie"
        Text="Opublikuj opinię"
        IsEnabled="False"
        Clicked="OnDodajClicked" />

</VerticalStackLayout>
</ScrollView>
</ContentPage>

```

C# CODE-BEHIND

```

namespace MojaAplikacja;

public partial class ReviewPage : ContentPage
{
    public ReviewPage()
    {
        InitializeComponent();
    }

    private void OnOcenaChanged(object sender, ValueChangedEventArgs e)
    {
        int ocena = (int)Math.Round(e.NewValue);
        SliderOcena.Value = ocena;

        // Generowanie gwiazdek
        string gwiazdki = new string('☆', ocena);
    }
}

```

```

        LabelGwiazdki.Text = $"{gwiazdki} ({ocena}/5)";

        Waliduj();
    }

    private void OnFieldChanged(object sender, TextChangedEventArgs e)
    {
        Waliduj();
    }

    private void OnEditorChanged(object sender, TextChangedEventArgs e)
    {
        int dlugosc = EditorTresc.Text?.Length ?? 0;
        LabelLicznik.Text = $"{dlugosc}/20 znaków";
        LabelLicznik.TextColor = dlugosc ≥ 20 ? Colors.Green : Colors.Gray;
        Waliduj();
    }

    private void Waliduj()
    {
        bool autorOk = !string.IsNullOrWhiteSpace(EntryAutor.Text);
        bool tytulOk = !string.IsNullOrWhiteSpace(EntryTytul.Text)
            && EntryTytul.Text.Trim().Length ≥ 3;
        bool trescOk = !string.IsNullOrWhiteSpace(EditorTresc.Text)
            && EditorTresc.Text.Length ≥ 20;

        BtnDodajOpinie.IsEnabled = autorOk && tytulOk && trescOk;

        if (BtnDodajOpinie.IsEnabled)
        {
            LabelStatus.Text = "Opinia gotowa do publikacji ✓";
            LabelStatus.TextColor = Colors.Green;
        }
        else
        {
            LabelStatus.Text = "Uzupełnij wszystkie pola";
            LabelStatus.TextColor = Colors.OrangeRed;
        }
    }

    private async void OnDodajClicked(object sender, EventArgs e)
    {
        int ocena = (int)Math.Round(SliderOcena.Value);
        string polecam = ChkPolecam.IsChecked ? "Tak" : "Nie";

        await DisplayAlert("Opinia opublikowana",
            $"Autor: {EntryAutor.Text.Trim()}\n" +
            $"Ocena: {ocena}/5\n" +
            $"Tytuł: {EntryTytul.Text.Trim()}\n" +
            $"Polecam: {polecam}", "OK");
    }
}

```

24. Walidacja i komunikaty dla użytkownika

24.1. Po co walidować dane

Walidacja to zestaw **sprawdzeń**, które dane muszą przejść, by uznać je za poprawne: pole nie może być puste, e-mail musi zawierać @, liczba musi być w zakresie, hasła muszą być zgodne. Walidację wykonujemy **przed** zapisaniem czy wysłaniem danych.

Walidacja **zapobiega błędom**: chroni aplikację przed nieprawidłowymi danymi (np. tekstem tam, gdzie ma być liczba), a użytkownika informuje, co poprawić. Bez niej aplikacja może się „wysypać” lub zapisać bezsensowne dane.

NAJWAŻNIEJSZE INFORMACJE

- Waliduj **zawsze przed** przetworzeniem danych.
- Najpierw sprawdzaj **pustość**, potem **format**, na końcu **logikę** (np. zgodność haseł).
- Wynik walidacji pokazuj użytkownikowi (komunikat, kolor).
- Stosuj wzorec **wczesnego wyjścia** (`return` po pierwszym błędzie).

Na co uważać:

Walidacja po stronie aplikacji poprawia wygodę, ale w aplikacjach komunikujących się z serwerem dane trzeba walidować **także na serwerze** - nie ufaj wyłącznie sprawdzeniom na urządzeniu.

24.2. Sprawdzanie pustych pól - `IsNullOrEmpty`

Najczęstsze sprawdzenie to **czy pole nie jest puste**. Używamy metody `string.IsNullOrEmpty`, która zwraca `true`, gdy tekst jest `null`, pusty (`""`) lub zawiera tylko białe znaki (spacje). To bezpieczniejsze niż samo porównanie z `""`, bo obsługuje też `null` i spacje.

PRZYKŁAD C#

```
if (string.IsNullOrEmpty(PoleImie.Text))
{
    Komunikat.Text = "Pole „Imię” jest wymagane.";
    Komunikat.TextColor = Colors.Red;
    return;
}
```

NAJWAŻNIEJSZE INFORMACJE

METODA	ZWRACA <code>TRUE</code> , GDY
<code>string.IsNullOrEmpty(t)</code>	<code>t</code> jest <code>null</code> lub <code>""</code>
<code>string.IsNullOrWhiteSpace(t)</code>	<code>t</code> jest <code>null</code> , <code>""</code> lub same spacje

Na co uważać:

Preferuj `IsNullOrEmpty` - łapie też pola wypełnione samymi spacjami, które użytkownik traktuje jako „puste”. `Entry.Text` dla pustego pola bywa `null`, więc to sprawdzenie jest też zabezpieczeniem przed błędem.

24.3. Sprawdzanie długości tekstu

Czasem wymagamy, by tekst miał **określoną długość** - np. hasło co najmniej 6 znaków, kod dokładnie 4 znaki. Długość tekstu odczytujemy właściwością `.Length`.

PRZYKŁAD C#

```
string haslo = PoleHaslo.Text ?? "";

if (haslo.Length < 6)
{
```



```
Komunikat.Text = "Hasło musi mieć co najmniej 6 znaków.";
Komunikat.TextColor = Colors.Red;

return;
}
```

Na co uważać:

Zabezpiecz się przed `null` przed odczytem `.Length` (np. `PoleHaslo.Text ?? ""`). Możesz też ograniczyć długość już na poziomie kontrolki przez `MaxLength`, ale walidację w kodzie i tak warto zachować.

24.4. Sprawdzanie, czy tekst zawiera znak (np. e-mail z „@“)

Proste sprawdzenie formatu to **obecność konkretnego znaku** - np. czy adres e-mail zawiera `@`. Używamy metody `.Contains`.

PRZYKŁAD C#

```
string email = PoleEmail.Text ?? "";

if (!email.Contains('@'))
{
    Komunikat.Text = "Adres e-mail musi zawierać znak „@”.";
    Komunikat.TextColor = Colors.Red;
    return;
}
```

Na co uważać:

Sprawdzenie obecności `@` to **uproszczona** walidacja e-maila - wystarcza w wielu prostych aplikacjach. Pełniejszą weryfikację formatu omawiamy w 16.8 (wyrażenia regularne).

24.5. Porównywanie dwóch pól (zgodność haseł)

Przy rejestracji często wymagamy **dwukrotnego wpisania hasła** i sprawdzamy ich **zgodność**. Porównujemy dwa napisy operatorem `≠`.

PRZYKŁAD C#

```
if (PoleHaslo1.Text ≠ PoleHaslo2.Text)
{
    Komunikat.Text = "Hasła nie są takie same.";
    Komunikat.TextColor = Colors.Red;
    return;
}
```

Na co uważać:

Porównanie napisów operatorem `=` / `≠` w C# porównuje ich **zawartość** (a nie referencje), więc działa zgodnie z oczekiwaniem. Zwróć uwagę, że porównanie **rozdziela wielkość liter** - co dla haseł jest pożądane.

24.6. Sprawdzanie, czy tekst składa się tylko z cyfr

Czasem pole powinno zawierać **wyłącznie cyfry** (np. PIN, numer). Można to sprawdzić metodą `All` z LINQ i `char.IsDigit`, albo prościej - próbą konwersji na liczbę.

PRZYKŁAD C#

```
string kod = PoleKod.Text ?? "";

// Sposób 1: każdy znak jest cyfrą
```

```

bool tylkoCyfry = kod.Length > 0 && kod.All(char.IsDigit);

// Sposób 2: udana konwersja na liczbę
bool jestLiczba = int.TryParse(kod, out _);

if (!tylkoCyfry)
{
    Komunikat.Text = "Pole może zawierać tylko cyfry.";
    return;
}

```

Na co uważać:

`kod.All(char.IsDigit)` zwraca `true` także dla **pustego** tekstu - dlatego dodaj warunek `kod.Length > 0`. Dla pól liczbowych warto też ustawić `Keyboard="Numeric"`, by ograniczyć wprowadzanie liter.

24.7. Konwersja tekstu na liczbę: Parse, TryParse, zakres

Dane z `Entry` to **tekst** - by sprawdzić zakres liczbowy, trzeba je najpierw przekonwertować. `int.TryParse` robi to **bezpiecznie**: zwraca `true/false` i nie rzuca wyjątku przy błędnych danych. Po udanej konwersji sprawdzamy zakres (np. wiek 1–120).

PRZYKŁAD C#

```

private void OnSprawdzWiek(object sender, EventArgs e)
{
    // 1. Pustość
    if (string.IsNullOrEmpty(PoleWiek.Text))
    {
        Pokaz("Podaj wiek.", false);
        return;
    }

    // 2. Konwersja na liczbę (bezpieczna)
    if (!int.TryParse(PoleWiek.Text, out int wiek))
    {
        Pokaz("Wiek musi być liczbą.", false);
        return;
    }

    // 3. Zakres
    if (wiek < 1 || wiek > 120)
    {
        Pokaz("Wiek musi być z zakresu 1-120.", false);
        return;
    }

    Pokaz($"Poprawny wiek: {wiek}.", true);
}

private void Pokaz(string tekst, bool ok)
{
    Komunikat.Text = tekst;
    Komunikat.TextColor = ok ? Colors.Green : Colors.Red;
}

```

PORÓWNANIE: PARSE A TRYPARSE

CECHA	INT.PARSE	INT.TRYPARSE
Błędne dane	rzuca wyjątek	zwraca <code>false</code>
Bezpieczne dla danych usera	nie	tak
Zwraca wynik przez	wartość	parametr <code>out</code>

Na co uważać:

Na danych od użytkownika **zawsze** używaj `TryParse`. `int.Parse` przy pustym lub błędnym polu zatrzyma aplikację wyjątkiem. Dla liczb dziesiętnych używaj `double.TryParse`; pamiętaj, że separator (kropka/przecinek) może zależeć od ustawień regionalnych.

24.8. Pełniejsza walidacja e-maila (wyrażenia regularne)

Do dokładniejszego sprawdzenia formatu (e-mail, kod pocztowy, telefon) używamy **wyrażeń regularnych** (`Regex`) - wzorców opisujących dozwoloną postać tekstu. To bardziej zaawansowane niż samo `Contains`, ale daje pewniejszą walidację.

PRZYKŁAD C#

```
using System.Text.RegularExpressions;

private bool PoprawnyEmail(string email)
{
    if (string.IsNullOrEmpty(email)) return false;
    // Uproszczony wzorzec: tekst@tekst.tekst
    return Regex.IsMatch(email, @"^[^@\s]+@[^\s]+\.[^\s]+$");
}
```

Na co uważać:

Wzorce e-maila bywają skomplikowane - dla większości aplikacji wystarczy prosty wzorzec jak powyżej. Nie próbuj tworzyć „idealnego” wzorca obejmującego wszystkie możliwe adresy; uproszczony wzorzec wyłapuje większość pomyłek.

24.9. Wartość domyślna przy błędnych danych

Zamiast blokować działanie przy błędnym wpisie, czasem wygodnie jest **przyjąć wartość domyślną**. Np. jeśli pole liczby jest puste lub błędne, użyj `0`. Realizujemy to przez `TryParse` z przygotowaną wartością zastępczą.

PRZYKŁAD C#

```
// Jeśli konwersja się nie powiedzie, przyjmij 0
if (!int.TryParse(PoleIlosc.Text, out int ilosc))
    ilosc = 0; // wartość domyślna

// Wariant z operatorem ??
int rozmiar = int.TryParse(PoleRozmiar.Text, out int r) ? r : 14;
```

Na co uważać:

Wartość domyślna jest wygodna, ale stosuj ją świadomie - czasem lepiej **poinformować** użytkownika o błędzie niż po cichu przyjąć zastępczą wartość. Dla danych krytycznych (np. cena) preferuj jawny komunikat.

24.10. Prezentacja błędów: Label a DisplayAlert

Wynik walidacji pokazujemy użytkownikowi na dwa główne sposoby. `Label` wyświetla komunikat „na stałe” na ekranie (często czerwony przy błędzie) - dyskretny, nie przerywa pracy. `DisplayAlert` pokazuje **okno dialogowe**, które trzeba zamknąć - bardziej zwraca uwagę, dobre dla ważnych komunikatów.

PRZYKŁAD C#

```
// Komunikat w etykiecie (dyskretny)
Komunikat.Text = "Uzupełnij wszystkie pola.";
Komunikat.TextColor = Colors.Red;

// Okno dialogowe (zwraca uwagę)
await DisplayAlert("Błąd", "Uzupełnij wszystkie pola.", "OK");
```

NAJWAŻNIEJSZE INFORMACJE

SPOSÓB	ZALETY	WADY
Label	nie przerywa pracy, zawsze widoczny	łatwo przeoczyć
DisplayAlert	mocno zwraca uwagę	przerywa pracę, wymaga zamknięcia

Na co uważać:

Do drobnych błędów walidacji w formularzu lepszy jest **Label** (np. pod polem lub na dole). **DisplayAlert** rezerwuj na ważne komunikaty wymagające reakcji (np. „Czy na pewno usunąć?"). Nie zasypuj użytkownika oknami dialogowymi przy każdym drobiazgu.

24.11. Walidacja na żywo (TextChanged) i blokowanie przycisku

Bardziej dopracowane formularze walidują **na bieżąco**, w trakcie wpisywania, korzystając ze zdarzenia **TextChanged**. Po każdej zmianie sprawdzamy poprawność i odpowiednio **blokujemy lub odblokowujemy** przycisk zatwierdzający.

PRZYKŁAD C#

```
private void OnPoleZmienione(object sender, TextChangedEventArgs e)
{
    bool poprawne =
        !string.IsNullOrWhiteSpace(PoleEmail.Text) &&
        PoleEmail.Text.Contains('@') &&
        !string.IsNullOrWhiteSpace(PoleHaslo.Text) &&
        PoleHaslo.Text.Length >= 6;

    PrzyciskZapisz.IsEnabled = poprawne;
}
```

Na co uważać:

Walidacja na żywo daje natychmiastową informację zwrotną i jest przyjazna dla użytkownika. Łącz ją z blokowaniem przycisku (**IsEnabled**), by uniemożliwić zatwierdzenie błędnych danych. Pamiętaj o sprawdzeniu wszystkich warunków naraz.

24.12. Kompletny przykład walidacji formularza

PRZYKŁAD XAML

```
<VerticalStackLayout Padding="24" Spacing="12">
    <Entry x:Name="PoleEmail" Placeholder="e-mail" Keyboard="Email" />
    <Entry x:Name="PoleHaslo1" Placeholder="hasło" IsPassword="True" />
    <Entry x:Name="PoleHaslo2" Placeholder="powtórz hasło" IsPassword="True" />
    <Entry x:Name="PoleWiek" Placeholder="wiek" Keyboard="Numeric" />
    <Button Text="Zarejestruj" Clicked="OnZarejestruj" />
    <Label x:Name="Komunikat" FontAttributes="Bold" />
</VerticalStackLayout>
```

PRZYKŁAD C#

```
private void OnZarejestruj(object sender, EventArgs e)
{
    string email = PoleEmail.Text;
    string h1 = PoleHaslo1.Text;
    string h2 = PoleHaslo2.Text;

    if (string.IsNullOrEmpty(email) ||
        string.IsNullOrEmpty(h1) ||
        string.IsNullOrEmpty(h2) ||
        string.IsNullOrEmpty(PoleWiek.Text))
    { Pokaz("Wszystkie pola są wymagane.", false); return; }

    if (!email.Contains('@'))
    { Pokaz("Niepoprawny e-mail.", false); return; }

    if (h1.Length < 6)
    { Pokaz("Hasło min. 6 znaków.", false); return; }

    if (h1 != h2)
    { Pokaz("Hasła nie są zgodne.", false); return; }

    if (!int.TryParse(PoleWiek.Text, out int wiek) || wiek < 1 || wiek > 120)
    { Pokaz("Wiek musi być liczbą 1-120.", false); return; }

    Pokaz("Rejestracja zakończona sukcesem!", true);
}

private void Pokaz(string tekst, bool ok)
{
    Komunikat.Text = tekst;
    Komunikat.TextColor = ok ? Colors.Green : Colors.Red;
}
```

Na co uważać:

Zauważ kolejność: pustość → format e-maila → długość hasła → zgodność haseł → liczba i zakres wieku. Wzorzec wczesnego wyjścia (`return`) sprawia, że pokazujemy **pierwszy** napotkany błąd, a kod pozostaje czytelny. Pomocnicza metoda `Pokaz` eliminuje powtarzanie.

24.13. Typowe błędy walidacji

NAJWAŻNIEJSZE INFORMACJE

BŁĄD	SKUTEK	ROZWIĄZANIE
<code>int.Parse</code> zamiast <code>TryParse</code>	wyjątek przy złym wpisie	użyj <code>TryParse</code>
Brak sprawdzenia <code>null</code>	wyjątek na <code>.Length</code> / <code>.Contains</code>	<code>IsNullOrEmpty</code> , <code>?? ""</code>
Walidacja po przetworzeniu	błędne dane już zapisane	waliduj przed przetworzeniem
Brak komunikatu	użytkownik nie wie, co źle	pokaż błąd (Label/Alert)
<code>All(char.IsDigit)</code> na pustym	uznaje pusty za poprawny	dodaj warunek długości
Walidacja tylko po kliknięciu	późna informacja	rozważ walidację na żywo

Na co uważać:

Najczęstsze pułapki to `Parse` zamiast `TryParse` oraz brak zabezpieczenia przed `null`. Waliduj zawsze **przed** użyciem danych, pokazuj czytelny komunikat i sprawdzaj warunki w sensownej kolejności (pustość → format → logika).

Walidacja to nie „dodatek”, lecz integralna część każdego formularza. Trzy filary dobrej walidacji to: sprawdzaj **przed** przetworzeniem, używaj `TryParse` dla liczb i `IsNullOrWhiteSpace` dla tekstu, oraz zawsze **informuj** użytkownika o wyniku.

Okna dialogowe to wbudowany, natywny sposób komunikacji z użytkownikiem: pokazania komunikatu, zadania pytania, pobrania krótkiej informacji lub menu wyboru. MAUI udostępnia trzy główne metody: `DisplayAlert`, `DisplayActionSheet` i `DisplayPromptAsync`. Wszystkie są **asynchroniczne** i wywoływane jako metody strony. Ten rozdział omawia każdą z nich oraz wyjaśnia, kiedy użyć dialogu, a kiedy zwykłej etykiety.

24.14. Przegląd okien dialogowych

Okno dialogowe to systemowy „pop-up”, który pojawia się nad ekranem i wstrzymuje interakcję do czasu reakcji użytkownika. MAUI rysuje je w stylu danej platformy. Trzy podstawowe rodzaje to: komunikat/pytanie (`DisplayAlert`), menu wyboru (`DisplayActionSheet`) i pole wpisania tekstu (`DisplayPromptAsync`).

NAJWAŻNIEJSZE INFORMACJE

METODA	ZASTOSOWANIE	ZWRACA
<code>DisplayAlert</code>	komunikat / pytanie tak-nie	nic lub <code>bool</code>
<code>DisplayActionSheet</code>	menu wyboru opcji	wybrany napis
<code>DisplayPromptAsync</code>	pobranie tekstu	wpisany napis lub <code>null</code>

Na co uważać:

Wszystkie trzy metody są **asynchroniczne** - wywołuj je z `await`, a metodę obsługi oznacz `async`. Dialog **przerywa** pracę użytkownika, więc używaj go do rzeczy wymagających uwagi, nie do drobnych komunikatów.

24.15. DisplayAlert - komunikat z jednym przyciskiem

`DisplayAlert` z trzema argumentami (tytuł, treść, przycisk) pokazuje **komunikat informacyjny** z jednym przyciskiem zamykającym. Idealny do potwierdzeń sukcesu, informacji czy komunikatów o błędzie.

PRZYKŁAD C#

```
private async void OnZapisz(object sender, EventArgs e)
{
    // Komunikat informacyjny
    await DisplayAlert("Sukces", "Dane zostały zapisane.", "OK");
}

private async void OnBład()
{
    // Komunikat o błędzie
    await DisplayAlert("Błąd", "Nie udało się połączyć z serwerem.", "OK");
}
```

Na co uważać:

Metoda musi być `async`, a wywołanie poprzedzone `await`. Tytuł powinien być krótki, treść konkretna, a przycisk zwykle to „OK”. Nie nadużywaj - zbyt wiele komunikatów męczy użytkownika.

24.16. DisplayAlert - pytanie z dwoma przyciskami

`DisplayAlert` z czterema argumentami (tytuł, treść, przycisk akceptacji, przycisk anulowania) pokazuje **pytanie** i zwraca `bool`: `true`, gdy użytkownik wybrał pierwszy przycisk, `false` - gdy drugi. Idealne do potwierdzeń typu „Czy na pewno?”.

PRZYKŁAD C#

```
private async void OnUsun(object sender, EventArgs e)
{
    bool potwierdzono = await DisplayAlert(
        "Potwierdzenie",
        "Czy na pewno usunąć element?",
        "Tak", // zwraca true
        "Nie"); // zwraca false

    if (potwierdzono)
    {
        // wykonaj usunięcie
        Status.Text = "Element usunięty.";
    }
}
```

Na co uważać:

Wynik (`bool`) odbieramy przez `await`. Pierwszy przycisk (akceptacji) zwraca `true`. Potwierdzenia stosuj przy akcjach **nieodwracalnych** (usuwanie, wylogowanie), by uchronić użytkownika przed pomyłką.

24.17. DisplayActionSheet - menu akcji

`DisplayActionSheet` pokazuje **menu wyboru** wysuwane od dołu ekranu - kilka opcji do wyboru, przycisk anulowania i opcjonalny przycisk „niszczący” (wyróżniony). Zwraca napis wybranej opcji.

PRZYKŁAD C#

```
private async void OnOpcje(object sender, EventArgs e)
{
    string wybor = await DisplayActionSheet(
        "Wybierz akcję", // tytuł
        "Anuluj",        // przycisk anulowania
        "Usuń",          // przycisk niszczący (może być null)
        "Edytuj", "Udostępnij"); // pozostałe opcje

    switch (wybor)
    {
        case "Edytuj":    Status.Text = "Edycja"; break;
        case "Udostępnij": Status.Text = "Udostępnianie"; break;
        case "Usuń":      Status.Text = "Usuwanie"; break;
    }
}
```

Na co uważać:

Gdy użytkownik anuluje, metoda zwraca napis przycisku anulowania (lub `null`). Sprawdzaj wynik, by nie wykonać akcji po anulowaniu.

`DisplayActionSheet` jest wygodniejszy niż upychanie wielu przycisków w interfejsie.

24.18. DisplayPromptAsync - pobieranie tekstu

`DisplayPromptAsync` pokazuje okno z **polem tekstowym** , pozwalając szybko pobrać krótką informację (nazwę, kod, komentarz) bez budowania osobnego ekranu. Zwraca wpisany tekst lub `null` przy anulowaniu.

PRZYKŁAD C#

```
private async void OnDodajNazwe(object sender, EventArgs e)
{
    string nazwa = await DisplayPromptAsync(
        "Nowa pozycja",
        "Podaj nazwę:",
        accept: "OK",
        cancel: "Anuluj",
        placeholder: "np. Zakupy",
        maxLength: 50,
        keyboard: Keyboard.Text);

    if (string.IsNullOrEmpty(nazwa)) return; // anulowano lub pusto
    notatki.Add(nazwa);
}
```

Na co uważać:

`DisplayPromptAsync` zwraca `null`, gdy użytkownik anuluje - zawsze sprawdzaj wynik (`IsNullOrEmpty`). Dla rozbudowanych danych lepszy jest osobny ekran; prompt jest do prostych, jednowierszowych wpisów.

24.19. Komunikat po walidacji, zapisie i akcji

Dialogi często wieńczą operacje: po walidacji pokazujemy błąd, po zapisie - potwierdzenie, po akcji - informację o wyniku. Dobór między dialogiem a etykietą zależy od wagi komunikatu.

PRZYKŁAD C#

```
private async void OnZatwierdz(object sender, EventArgs e)
{
    if (string.IsNullOrEmpty(PoleNazwa.Text))
    {
        // Drobną błąd walidacji - można pokazać w etykiecie lub alertem
        await DisplayAlert("Uwaga", "Podaj nazwę.", "OK");
        return;
    }

    // ...zapis...
    await DisplayAlert("Gotowe", "Zapisano pomyślnie.", "OK");
}
```

Na co uważać:

Po **udanym zapisie** krótki komunikat potwierdzający buduje zaufanie użytkownika. Po **błędzie** jasno powiedz, co poszło nie tak i jak to naprawić.

24.20. Kiedy etykieta, a kiedy alert

NAJWAŻNIEJSZE INFORMACJE

SYTUACJA	LEPSZY WYBÓR
Drobny błąd walidacji w formularzu	<code>Label</code> (np. pod polem)
Ważne potwierdzenie (usuwanie)	<code>DisplayAlert</code> z dwoma przyciskami
Informacja o sukcesie operacji	krótki <code>DisplayAlert</code> lub <code>Label</code>
Wybór jednej z kilku akcji	<code>DisplayActionSheet</code>
Pobranie krótkiego tekstu	<code>DisplayPromptAsync</code>
Komunikat wyświetlany na stałe	<code>Label</code>

Na co uważać:

Dialog przerywa pracę - używaj go do rzeczy wymagających uwagi (potwierdzenia, ważne błędy). Do dyskretnych, ciągłych komunikatów (np. walidacja na żywo) lepszy jest `Label`, który nie zatrzymuje użytkownika. Równowaga jest kluczowa: zbyt wiele dialogów irytuje.

24.21. Typowe błędy

NAJWAŻNIEJSZE INFORMACJE

BŁĄD	SKUTEK	ROZWIĄZANIE
Brak <code>async</code> / <code>await</code>	błąd kompilacji lub brak wyniku	oznacz metodę <code>async</code> , użyj <code>await</code>
Ignorowanie <code>null</code> z promptu	wyjątek lub pusty wpis	sprawdź <code>IsNullOrWhiteSpace</code>
Akcja po anulowaniu ActionSheet	niechciane działanie	sprawdź wybór
Zbyt wiele dialogów	irytacja użytkownika	używaj etykiet do drobiazgów

Na co uważać:

Najczęstszy błąd to zapomnienie `async` / `await`. Pamiętaj też, że `DisplayActionSheet` i `DisplayPromptAsync` mogą zwrócić wynik anulowania/ `null` - zawsze go sprawdzaj przed podjęciem akcji.

Dialogi to szybki sposób komunikacji bez budowania własnych ekranów. Zasada: potwierdzenia i ważne komunikaty → dialog; dyskretnie, ciągłe informacje (walidacja na żywo) → etykieta. Zawsze obsługuj `async` / `await` i sprawdzaj wyniki anulowania.

24.22. Podsumowanie technik walidacji

TECHNIKA	ZASTOSOWANIE
<code>string.IsNullOrEmpty()</code>	Sprawdzenie czy pole nie jest puste
<code>int.TryParse()</code> z zakresem	Walidacja liczb całkowitych
<code>double.TryParse()</code> z zakresem	Walidacja liczb zmiennoprzecinkowych
<code>.Contains("@")</code>	Prosta walidacja e-mail
<code>.Length >= n</code>	Minimalna długość tekstu
<code>TextChanged</code> + <code>IsEnabled</code>	Blokowanie przycisku na żywo
<code>TextColor = Colors.Red/Green</code>	Komunikaty zmieniające kolor
<code>DisplayAlert()</code>	Wyświetlenie komunikatu po akcji
<code>Slider</code> + <code>Math.Round</code>	Wartości całkowite ze Slidera

25. Listy, kolekcje i praktyczne receptury

Wyświetlanie **list danych** to jedna z najczęstszych potrzeb w aplikacjach: lista notatek, produktów, kontaktów, zadań. Ten rozdział jest bardzo praktyczny - pokazuje, jak przechowywać dane w kolekcjach, jak wyświetlać je w `CollectionView`, jak budować wygląd pojedynczego elementu, a przede wszystkim jak **dodawać, usuwać i edytować** elementy z automatycznym odświeżaniem widoku. Wyjaśniamy też kluczową różnicę między `List` a `ObservableCollection`.

25.1. List a ObservableCollection - najważniejsza różnica

Dane listy przechowujemy w **kolekcji**. Dwie najważniejsze to `List<T>` i `ObservableCollection<T>`. `List<T>` to zwykła kolekcja - świetna do przechowywania danych, ale **nie informuje interfejsu o zmianach**. `ObservableCollection<T>` przy każdym dodaniu/usunięciu elementu **automatycznie powiadamia** powiązany widok, który natychmiast się odświeża.

`ObservableCollection<T>` służy do **list wyświetlanych na ekranie**, które mają się dynamicznie odświeżać. `List<T>` używamy do danych „roboczych” w pamięci, gdzie odświeżanie widoku nie jest potrzebne.

NAJWAŻNIEJSZE INFORMACJE

CECHA	<code>LIST<T></code>	<code>OBSERVABLECOLLECTION<T></code>
Powiadamia widok o zmianach	nie	tak
Odświeżanie po dodaniu	brak/ręczne	automatyczne
Przestrzeń nazw	<code>System.Collections.Generic</code>	<code>System.Collections.ObjectModel</code>
Typowe użycie	dane robocze	listy na ekranie

PRZYKŁAD C#

```
using System.Collections.ObjectModel;

// Lista wyświetlana - ObservableCollection (odświeża widok)
ObservableCollection<string> notatki = new ObservableCollection<string>();

// Dane robocze - zwykła List
List<int> liczby = new List<int> { 1, 2, 3 };
```

TYPOWE BŁĘDY

- Użycie `List<T>` dla listy na ekranie → „lista nie odświeża się” po dodaniu.

Na co uważać:

To **najczęstszy błąd** przy listach: gdy lista na ekranie nie reaguje na dodanie elementu, prawie zawsze przyczyną jest użycie `List<T>` zamiast `ObservableCollection<T>`. Do list wyświetlanych - zawsze `ObservableCollection`.

25.2. Lista tekstów i lista obiektów

Kolekcja może zawierać **proste wartości** (np. napisy) albo **obiekty** (np. produkty z wieloma właściwościami). Lista tekstów jest najprostsza; lista obiektów pozwala wyświetlić bogatszy element (nazwa, cena, opis).

PRZYKŁAD C#

```
// Lista tekstów
ObservableCollection<string> miasta = new() { "Warszawa", "Kraków", "Gdańsk" };

// Model obiektu
public class Produkt
{
    public string Nazwa { get; set; }
```

```
public double Cena { get; set; }  
  
}  
  
// Lista obiektów  
ObservableCollection<Produkt> produkty = new()  
{  
    new Produkt { Nazwa = "Kawa", Cena = 19.99 },  
    new Produkt { Nazwa = "Herbata", Cena = 12.50 }  
};
```

Na co uważać:

Dla listy tekstów w szablonie używamy `{Binding .}` (całość). Dla listy obiektów wiążemy konkretne właściwości: `{Binding Nazwa}`, `{Binding Cena}`. Wybór zależy od tego, co przechowuje kolekcja.

25.3. CollectionView

`CollectionView` to nowoczesna, wydajna kontrolka do **wyświetlania list**. Podpinamy do niej kolekcję przez `ItemsSource`, a wygląd pojedynczego elementu opisujemy w `ItemTemplate` (przez `DataTemplate`). Sama generuje widok dla każdego elementu i wydajnie obsługuje nawet długie listy.

NAJWAŻNIEJSZE WŁAŚCIWOŚCI

WŁAŚCIWOŚĆ	OPIS	PRZYKŁAD
<code>ItemsSource</code>	kolekcja danych	<code>{Binding Produkty}</code> lub w C#
<code>ItemTemplate</code>	sablon pojedynczego elementu	<code>DataTemplate</code>
<code>SelectionMode</code>	tryb zaznaczania	<code>Single</code> / <code>None</code>
<code>SelectedItem</code>	zaznaczony element	odczyt w C#
<code>EmptyView</code>	widok dla pustej listy	„Brak danych”

NAJWAŻNIEJSZE ZDARZENIA

ZDARZENIE	KIEDY WYSTĘPUJE	PRZYKŁAD UŻYCIA
<code>SelectionChanged</code>	po zmianie zaznaczenia	otwarcie szczegółów

PRZYKŁAD PODSTAWOWY

```
<CollectionView x:Name="ListaMiast">  
    <CollectionView.ItemTemplate>  
        <DataTemplate>  
            <Label Text="{Binding .}" FontSize="18" Padding="10" />  
        </DataTemplate>  
    </CollectionView.ItemTemplate>  
</CollectionView>
```

PRZYKŁAD W C#

```
ListaMiast.ItemsSource = new ObservableCollection<string>  
{ "Warszawa", "Kraków", "Gdańsk" };
```

TYPOWE ZASTOSOWANIA

- Lista notatek, produktów, kontaktów, zadań.

TYPOWE BŁĘDY

- Brak `ItemTemplate` (lista pokazuje nazwy typów zamiast danych).
- Użycie `List<T>` zamiast `ObservableCollection<T>`.

25.4. DataTemplate i ItemTemplate - wygląd elementu

`DataTemplate` to **szablon pojedynczego elementu** listy. Definiujemy w nim, jak ma wyglądać każdy wiersz: zwykły tekst, czy bogaty układ z obrazem, tytułem i opisem. Wewnątrz używamy **wiązań** (`{Binding Właściwość}`), które łączą właściwości obiektu z kontrolkami.

PRZYKŁAD XAML (LISTA OBIEKTÓW)

```
<CollectionView x:Name="ListaProduktow">
    <CollectionView.ItemTemplate>
        <DataTemplate>
            <Grid Padding="12" ColumnDefinitions="*,Auto">
                <VerticalStackLayout Grid.Column="0">
                    <Label Text="{Binding Nazwa}" FontSize="18" FontAttributes="Bold" />
                    <Label Text="{Binding Opis}" TextColor="Gray" FontSize="13" />
                </VerticalStackLayout>
                <Label Text="{Binding Cena, StringFormat='{0:0.00} zł'}"
                    Grid.Column="1" TextColor="Green" VerticalOptions="Center" />
            </Grid>
        </DataTemplate>
    </CollectionView.ItemTemplate>
</CollectionView>
```

Na co uważać:

W szablonie `BindingContext` to **pojedynczy element** kolekcji, więc `{Binding Nazwa}` odnosi się do właściwości tego elementu. `StringFormat` pozwala sformatować wartość (np. cenę z dwoma miejscami i „zł”). Dla listy napisów używamy `{Binding .}`.

25.5. ListView - alternatywa klasyczna

`ListView` to starsza kontrolka listy (z Xamarin.Forms), wciąż dostępna w MAUI. Działa podobnie do `CollectionView`, ale każdy element owija w `ViewCell` i ma wbudowane funkcje jak nagłówki sekcji czy „pociągnij, by odświeżyć”. W nowych projektach zaleca się `CollectionView`.

PRZYKŁAD PODSTAWOWY

```
<ListView x:Name="ListaLV">
    <ListView.ItemTemplate>
        <DataTemplate>
            <ViewCell>
                <Label Text="{Binding .}" Padding="10" />
            </ViewCell>
        </DataTemplate>
    </ListView.ItemTemplate>
</ListView>
```

PORÓWNIANIE: LISTVIEW A COLLECTIONVIEW

CECHA	LISTVIEW	COLLECTIONVIEW
Wiek	starsza	nowsza
Wydajność	dobra	lepszta
ViewCell	wymagany	niepotrzebny
Zalecenie	starszy kod	nowe projekty

Na co uważać:

Wybierz `CollectionView` w nowych projektach. `ListView` warto znać, bo pojawia się w starszych przykładach i ma kilka wbudowanych funkcji, których `CollectionView` nie ma od ręki.

25.6. Dodawanie elementów i automatyczne odświeżanie

Największa zaleta `CollectionView` + `ObservableCollection` ujawnia się przy **dobawaniu elementów w trakcie działania**. Wystarczy wywołać `Add` na kolekcji, a element **natychmiast** pojawia się na liście - bez żadnego ręcznego odświeżania.

PRZYKŁAD XAML

```
<Grid Padding="20" RowSpacing="12" RowDefinitions="Auto,*">
    <HorizontalStackLayout Grid.Row="0" Spacing="10">
        <Entry x:Name="PoleNotatki" Placeholder="Nowa notatka..." WidthRequest="240" />
        <Button Text="Dodaj" Clicked="OnDodaj" />
    </HorizontalStackLayout>

    <CollectionView x:Name="ListaNotatek" Grid.Row="1">
        <CollectionView.ItemTemplate>
            <DataTemplate>
                <Label Text="{Binding .}" FontSize="18" Padding="8" />
            </DataTemplate>
        </CollectionView.ItemTemplate>
    </CollectionView>
</Grid>
```

PRZYKŁAD C#

```
ObservableCollection<string> notatki = new ObservableCollection<string>();

public MainPage()
{
    InitializeComponent();
    ListaNotatek.ItemsSource = notatki; // podpięcie RAZ, na starcie
}

private void OnDodaj(object sender, EventArgs e)
{
    if (string.IsNullOrEmpty(PoleNotatki.Text)) return;
    notatki.Add(PoleNotatki.Text); // element od razu widoczny
    PoleNotatki.Text = string.Empty; // wyczyść pole
}
```

Na co uważać:

`ItemsSource` ustawiamy **tylko raz** (w konstruktorze). Później operujemy na samej kolekcji (`Add` , `Remove`), a widok odświeża się sam. Nie trzeba ponownie przypisywać `ItemsSource` .

25.7. Usuwanie elementów

Usuwanie działa analogicznie do dodawania - wywołujemy `Remove` na kolekcji, a element znika z listy. Często przycisk usuwania umieszczamy w szablonie elementu i przekazujemy dany element jako parametr.

PRZYKŁAD C#

```
private void OnUsun(object sender, EventArgs e)
{
    // Element do usunięcia można odczytać z BindingContext przycisku
    if (sender is Button btn && btn.BindingContext is string notatka)
        notatki.Remove(notatka);
}

// Usuwanie zaznaczonego elementu
private void OnUsunZaznaczony(object sender, EventArgs e)
{
    if (ListaNotatek.SelectedItem is string wybrana)
        notatki.Remove(wybrana);
}

// Wyczyszczenie całej listy
private void OnWyczyszc(object sender, EventArgs e) => notatki.Clear();
```

Na co uważać:

`Remove` usuwa pierwszy pasujący element. `Clear` czyści całą kolekcję. Oba natychmiast odświeżają widok (dla `ObservableCollection`). Przy usuwaniu z szablonu wygodnie odczytać element z `BindingContext` przycisku.

25.8. Edycja i wybór elementu

Wybór elementu obsługujemy zdarzeniem `SelectionChanged` (lub `SelectedItem`). **Edycja** zwykle polega na przejściu do osobnego ekranu z danymi wybranego elementu, zmianie i zapisaniu. Po edycji odświeżamy listę.

PRZYKŁAD C#

```
private async void OnWybor(object sender, SelectionChangedEventArgs e)
{
    if (e.CurrentSelection.FirstOrDefault() is Produkt p)
    {
        // przejście do ekranu szczegółów/edycji
        await Navigation.PushAsync(new EdycjaPage(p));
    }
    ((CollectionView)sender).SelectedItem = null; // odznaczyć po wejściu
}
```

Na co uważać:

Po obsłudze wyboru warto **odznaczyć** element (`SelectedItem = null`), by po powrocie można było go znów wybrać. Edycja obiektu w kolekcji wymaga, by widok wiedział o zmianie właściwości - przy prostych typach wystarcza podmiana elementu lub ponowne odświeżenie listy.

25.9. Przechodzenie po elementach: Poprzedni / Następny

Czasem nie pokazujemy całej listy, lecz **jeden element naraz**, z przyciskami „Poprzedni” i „Następny”. Stanem jest wtedy **indeks** aktualnego elementu, a przyciski go zmieniają. Popularne jest **zawijanie** indeksu na końcach.

PRZYKŁAD C#

```
List<string> elementy = new() { "Pierwszy", "Drugi", "Trzeci" };
int indeks = 0;

private void OnNastepny(object sender, EventArgs e)
{
    indeks++;
    if (indeks ≥ elementy.Count) indeks = 0; // zawinięcie na koniec
    Pokaz();
}

private void OnPoprzedni(object sender, EventArgs e)
{
    indeks--;
    if (indeks < 0) indeks = elementy.Count - 1; // zawinięcie na początek
    Pokaz();
}

private void Pokaz() ⇒ Etykieta.Text = elementy[indeks];
```

Na co uważać:

Pilnuj zakresu indeksu. **Zawijanie:** po ostatnim wracamy do `0`, przed pierwszym przechodzimy do `Count - 1`. Bez tego wyjdiesz poza zakres listy i otrzymasz błąd. To częsty wzorec w galeriach i przeglądarkach.

25.10. Lista obiektów wczytana z pliku

Listę można wypełnić danymi z pliku (np. `dane.txt`) - każdy wiersz to jeden element, który parsujemy na obiekt. Łączymy tu pracę z plikami z kolekcjami.

PRZYKŁAD C#

```
// Plik "produkty.txt": każda linia w formacie "Nazwa;Cena"
private async Task WczytajZPliku()
{
    string sciezka = Path.Combine(FileSystem.AppDataDirectory, "produkty.txt");
    if (!File.Exists(sciezka)) return;

    string[] linie = await File.ReadAllLinesAsync(sciezka);
    foreach (string linia in linie)
    {
        string[] czesci = linia.Split(';');
        if (czesci.Length == 2 && double.TryParse(czesci[1], out double cena))
            produkty.Add(new Produkt { Nazwa = czesci[0], Cena = cena });
    }
}
```

Na co uważać:

Przy parsowaniu pliku **waliduj** każdą linię (sprawdź liczbę części, użyj `TryParse`), bo plik może zawierać błędne dane. Pomijaj puste lub niepoprawne wiersze, zamiast pozwolić aplikacji na błąd.

25.11. Pusta lista - EmptyView

Gdy lista jest pusta, warto pokazać **komunikat zastępczy** zamiast pustego ekranu. Służy do tego właściwość `EmptyView` w `CollectionView`.

PRZYKŁAD XAML

```
<CollectionView x:Name="ListaNotatek">
    <CollectionView.EmptyView>
        <Label Text="Brak notatek. Dodaj pierwszą!"
            HorizontalOptions="Center" VerticalOptions="Center"
            TextColor="Gray" />
    </CollectionView.EmptyView>
    <CollectionView.ItemTemplate>
        <DataTemplate>
            <Label Text="{Binding .}" Padding="8" />
        </DataTemplate>
    </CollectionView.ItemTemplate>
</CollectionView>
```

Na co uważać:

`EmptyView` poprawia odbiór aplikacji - pusty ekran bywa mylący, a komunikat „Brak danych” jasno informuje o sytuacji i zachęca do działania.

25.12. Typowe błędy przy listach

NAJWAŻNIEJSZE INFORMACJE

BŁĄD	SKUTEK	ROZWIĄZANIE
<code>List<T></code> zamiast <code>ObservableCollection</code>	lista się nie odświeża	użyj <code>ObservableCollection</code>
Ponowne przypisanie <code>ItemsSource</code> po każdej zmianie	zbędne, czasem miganie	podepnij raz, operuj na kolekcji
Brak <code>ItemTemplate</code>	widać nazwy typów	dodaj <code>DataTemplate</code>
Wyjście poza zakres indeksu	błąd przy Poprzedni/Następny	zawijaj indeks
Brak walidacji danych z pliku	błąd parsowania	<code>TryParse</code> , sprawdzaj linie
Lista w <code>ScrollView</code>	konflikt przewijania	nie zagnieżdżaj

Na co uważać:

Najważniejsza zasada: do list wyświetlanych używaj `ObservableCollection`, podepnij `ItemsSource` raz i operuj na kolekcji (`Add` / `Remove` / `Clear`). To rozwiązuje większość problemów z odświeżaniem.

Para `CollectionView` + `ObservableCollection` to przepis na dynamiczną listę. Dodajesz element przez `Add`, usuwasz przez `Remove`, a widok aktualizuje się sam. To jeden z najważniejszych wzorców w aplikacjach MAUI - opanuj go dobrze.

Data Binding (wiązanie danych) to mechanizm, który **automatycznie łączy** właściwość kontrolki z właściwością obiektu danych. Zamiast ręcznie przepisywać wartości w code-behind, deklarujemy powiązanie, a framework dba o synchronizację. W prostych aplikacjach najczęściej użyjesz go przy `CollectionView`, `Picker` i formularzach.

25.13. Receptury list i kolekcji

Ten dział zawiera praktyczne przykłady pracy z listami i kolekcjami w .NET MAUI. Każdy przykład składa się z kompletnego kodu XAML oraz C# code-behind.

25.14. ObservableCollection - dodawanie elementów

`ObservableCollection<T>` automatycznie powiadamia widok o zmianach. Poniżej dodawanie elementów do listy.

XAML

```
<?xml version="1.0" encoding="utf-8" ?>
<ContentPage xmlns="http://schemas.microsoft.com/dotnet/2021/maui"
              xmlns:x="http://schemas.microsoft.com/winfx/2009/xaml"
              x:Class="MauiApp.DodawaniePage">

    <VerticalStackLayout Padding="20" Spacing="10">
        <Entry x:Name="NowyElementEntry" Placeholder="Wpisz element" />
        <Button Text="Dodaj" Clicked="DodajClicked" />
        <CollectionView x:Name="ListaView" />
    </VerticalStackLayout>

</ContentPage>
```

C#

```
using System.Collections.ObjectModel;

namespace MauiApp;

public partial class DodawaniePage : ContentPage
{
    // Kolekcja automatycznie odświeża widok
    private ObservableCollection<string> _elementy = new();

    public DodawaniePage()
    {
        InitializeComponent();
        ListaView.ItemsSource = _elementy;
    }

    private void DodajClicked(object sender, EventArgs e)
    {
        // Dodanie nowego elementu do kolekcji
        if (!string.IsNullOrEmpty(NowyElementEntry.Text))
        {
            _elementy.Add(NowyElementEntry.Text);
            NowyElementEntry.Text = string.Empty;
        }
    }
}
```

25.15. ObservableCollection - usuwanie elementów

Usuwanie wybranego elementu z kolekcji z natychmiastowym odświeżeniem listy.

XAML

```
<?xml version="1.0" encoding="utf-8" ?>
<ContentPage xmlns="http://schemas.microsoft.com/dotnet/2021/maui"
              xmlns:x="http://schemas.microsoft.com/winfx/2009/xaml"
              x:Class="MauApp.UsuwaniePage">

    <VerticalStackLayout Padding="20" Spacing="10">
        <CollectionView x:Name="ListView"
                        SelectionMode="Single"
                        SelectionChanged="ListaSelectionChanged" />
        <Button x:Name="UsunBtn" Text="Usuń zaznaczony" Clicked="UsunClicked" IsEnabled="False" />
    </VerticalStackLayout>

</ContentPage>
```

C#

```
using System.Collections.ObjectModel;

namespace MauApp;

public partial class UsuwaniePage : ContentPage
{
    private ObservableCollection<string> _elementy = new()
    {
        "Jabłko", "Gruszka", "Banan", "Pomarańcza"
    };

    public UsuwaniePage()
    {
        InitializeComponent();
        ListView.ItemsSource = _elementy;
    }

    private void ListaSelectionChanged(object sender, SelectionChangedEventArgs e)
    {
        UsunBtn.IsEnabled = e.CurrentSelection.Count > 0;
    }

    private void UsunClicked(object sender, EventArgs e)
    {
        // Usunięcie zaznaczonego elementu
        if (ListView.SelectedItem is string wybrany)
        {
            _elementy.Remove(wybrany);
            ListView.SelectedItem = null;
        }
    }
}
```

```
}  
  
}
```

25.16. ObservableCollection - edycja elementu

Edycja elementu wymaga zastąpienia go nową wartością na danym indeksie.

XAML

```
<?xml version="1.0" encoding="utf-8" ?>  
<ContentPage xmlns="http://schemas.microsoft.com/dotnet/2021/maui"  
             xmlns:x="http://schemas.microsoft.com/winfx/2009/xaml"  
             x:Class="MauiApp.EdycjaPage">  
  
    <VerticalStackLayout Padding="20" Spacing="10">  
        <CollectionView x:Name="ListView" SelectionMode="Single"  
                        SelectionChanged="OnSelectionChanged" />  
        <Entry x:Name="EdycjaEntry" Placeholder="Nowa wartość" />  
        <Button Text="Zapisz zmiany" Clicked="ZapiszClicked" />  
    </VerticalStackLayout>  
  
</ContentPage>
```

C#

```
using System.Collections.ObjectModel;  
  
namespace MauiApp;  
  
public partial class EdycjaPage : ContentPage  
{  
    private ObservableCollection<string> _elementy = new()  
    {  
        "Element 1", "Element 2", "Element 3"  
    };  
  
    public EdycjaPage()  
    {  
        InitializeComponent();  
        ListView.ItemsSource = _elementy;  
    }  
  
    private void OnSelectionChanged(object sender, SelectionChangedEventArgs e)  
    {  
        if (e.CurrentSelection.FirstOrDefault() is string wybrany)  
            EdycjaEntry.Text = wybrany;  
    }  
  
    private void ZapiszClicked(object sender, EventArgs e)  
    {  
        if (ListView.SelectedItem is string wybrany)  
        {  

```

```

        // Zamiana elementu na danym indeksie
        int index = _elementy.IndexOf(wybrany);
        if (index ≥ 0)
        {
            _elementy[index] = EdycjaEntry.Text;
        }
    }
}
}

```

25.17. ObservableCollection - czyszczenie kolekcji

Metoda `Clear()` usuwa wszystkie elementy i odświeża widok.

XAML

```

<?xml version="1.0" encoding="utf-8" ?>
<ContentPage xmlns="http://schemas.microsoft.com/dotnet/2021/maui"
              xmlns:x="http://schemas.microsoft.com/winfx/2009/xaml"
              x:Class="MauApp.CzyszczeniePage">

    <VerticalStackLayout Padding="20" Spacing="10">
        <Button Text="Dodaj 5 elementów" Clicked="DodajClicked" />
        <Button Text="Wyczyść listę" Clicked="WyczyscClicked" />
        <Label x:Name="LicznikLabel" Text="Elementów: 0" />
        <CollectionView x:Name="ListaView" />
    </VerticalStackLayout>

</ContentPage>

```

C#

```

using System.Collections.ObjectModel;

namespace MauApp;

public partial class CzyszczeniePage : ContentPage
{
    private ObservableCollection<string> _elementy = new();

    public CzyszczeniePage()
    {
        InitializeComponent();
        ListaView.ItemsSource = _elementy;
    }

    private void DodajClicked(object sender, EventArgs e)
    {
        for (int i = 1; i ≤ 5; i++)
            _elementy.Add($"Element {_elementy.Count + 1}");
        LicznikLabel.Text = $"Elementów: {_elementy.Count}";
    }
}

```

```
private void WyczyscClickled(object sender, EventArgs e)
{
    // Usunięcie wszystkich elementów jednym wywołaniem
    _elementy.Clear();
    LicznikLabel.Text = $"Elementów: {_elementy.Count}";
}
}
```

25.18. CollectionView - prosty DataTemplate

CollectionView to nowoczesny widok listy. Prosty **DataTemplate** wyświetla tekst.

XAML

```
<?xml version="1.0" encoding="utf-8" ?>
<ContentPage xmlns="http://schemas.microsoft.com/dotnet/2021/maui"
              xmlns:x="http://schemas.microsoft.com/winfx/2009/xaml"
              x:Class="MauiApp.ProstyTemplatePage">

    <CollectionView x:Name="ListView">
        <CollectionView.ItemTemplate>
            <DataTemplate>
                <!-- Prosty szablon: sam Label -->
                <Label Text="{Binding .}"
                      FontSize="18"
                      Padding="10" />
            </DataTemplate>
        </CollectionView.ItemTemplate>
    </CollectionView>

</ContentPage>
```

C#

```
using System.Collections.ObjectModel;

namespace MauiApp;

public partial class ProstyTemplatePage : ContentPage
{
    public ProstyTemplatePage()
    {
        InitializeComponent();

        // Prosta lista stringów
        var miasta = new ObservableCollection<string>
        {
            "Warszawa", "Kraków", "Gdańsk", "Wrocław", "Poznań"
        };
        ListView.ItemsSource = miasta;
    }
}
```

```
}  
}
```

25.19. CollectionView - złożony DataTemplate z obiektem

Szablon wyświetlający wiele właściwości obiektu w karcie.

XAML

```
<?xml version="1.0" encoding="utf-8" ?>  
<ContentPage xmlns="http://schemas.microsoft.com/dotnet/2021/maui"  
             xmlns:x="http://schemas.microsoft.com/winfx/2009/xaml"  
             x:Class="MauiApp.ZlozonyTemplatePage">  
  
    <CollectionView x:Name="ListView">  
        <CollectionView.ItemTemplate>  
            <DataTemplate>  
                <Frame Margin="5" Padding="10" CornerRadius="8" BorderColor="Gray">  
                    <VerticalStackLayout Spacing="4">  
                        <Label Text="{Binding Nazwa}" FontSize="16" FontAttributes="Bold" />  
                        <Label Text="{Binding Opis}" FontSize="13" TextColor="Gray" />  
                        <Label Text="{Binding Cena, StringFormat='Cena: {0:F2} zł'}" FontSize="14" />  
                    </VerticalStackLayout>  
                </Frame>  
            </DataTemplate>  
        </CollectionView.ItemTemplate>  
    </CollectionView>  
  
</ContentPage>
```

C#

```
using System.Collections.ObjectModel;  
  
namespace MauiApp;  
  
// Model danych  
public class Produkt  
{  
    public string Nazwa { get; set; }  
    public string Opis { get; set; }  
    public double Cena { get; set; }  
}  
  
public partial class ZlozonyTemplatePage : ContentPage  
{  
    public ZlozonyTemplatePage()  
    {  
        InitializeComponent();  
  
        var produkty = new ObservableCollection<Produkt>  
        {
```

```

        new Produkt { Nazwa = "Laptop", Opis = "16 GB RAM, SSD 512 GB", Cena = 3499.99 },
        new Produkt { Nazwa = "Myszka", Opis = "Bezprzewodowa, ergonomiczna", Cena = 129.00 },
        new Produkt { Nazwa = "Klawiatura", Opis = "Mechaniczna, podświetlana", Cena = 299.50 }
    };

    ListView.ItemsSource = produkty;
}
}

```

25.20. CollectionView - GridItemsLayout (siatka)

Wyświetlanie elementów w siatce zamiast listy pionowej.

XAML

```

<?xml version="1.0" encoding="utf-8" ?>
<ContentPage xmlns="http://schemas.microsoft.com/dotnet/2021/maui"
              xmlns:x="http://schemas.microsoft.com/winfx/2009/xaml"
              x:Class="MauiApp.GridLayoutPage">

    <CollectionView x:Name="ListView">
        <!-- Siatka z 2 kolumnami, przewijana pionowo -->
        <CollectionView.ItemsLayout>
            <GridItemsLayout Orientation="Vertical"
                             Span="2"
                             HorizontalItemSpacing="10"
                             VerticalItemSpacing="10" />
        </CollectionView.ItemsLayout>
        <CollectionView.ItemTemplate>
            <DataTemplate>
                <Frame Padding="15" BackgroundColor="LightBlue" CornerRadius="10">
                    <Label Text="{Binding .}" HorizontalTextAlignment="Center" FontSize="16" />
                </Frame>
            </DataTemplate>
        </CollectionView.ItemTemplate>
    </CollectionView>

</ContentPage>

```

C#

```

using System.Collections.ObjectModel;

namespace MauiApp;

public partial class GridLayoutPage : ContentPage
{
    public GridLayoutPage()
    {
        InitializeComponent();

        var kolory = new ObservableCollection<string>
        {

```

```

        "Czerwony", "Zielony", "Niebieski", "Żółty",
        "Fioletowy", "Pomarańczowy", "Różowy", "Brązowy"
    };

    ListView.ItemsSource = kolory;
}
}

```

25.21. CollectionView - EmptyView

Widok wyświetlany, gdy kolekcja jest pusta.

XAML

```

<?xml version="1.0" encoding="utf-8" ?>
<ContentPage xmlns="http://schemas.microsoft.com/dotnet/2021/maui"
              xmlns:x="http://schemas.microsoft.com/winfx/2009/xaml"
              x:Class="MauiApp.EmptyViewPage">

    <VerticalStackLayout Padding="20" Spacing="10">
        <Button Text="Wyczyść listę" Clicked="WyczyscClicked" />
        <Button Text="Załaduj dane" Clicked="ZaladujClicked" />

        <CollectionView x:Name="ListView">
            <!-- Widok gdy lista jest pusta -->
            <CollectionView.EmptyView>
                <VerticalStackLayout HorizontalOptions="Center" VerticalOptions="Center">
                    <Label Text="❌" FontSize="48" HorizontalTextAlignment="Center" />
                    <Label Text="Brak elementów do wyświetlenia"
                          FontSize="18" HorizontalTextAlignment="Center" TextColor="Gray" />
                </VerticalStackLayout>
            </CollectionView.EmptyView>
            <CollectionView.ItemTemplate>
                <DataTemplate>
                    <Label Text="{Binding .}" Padding="10" FontSize="16" />
                </DataTemplate>
            </CollectionView.ItemTemplate>
        </CollectionView>
    </VerticalStackLayout>

</ContentPage>

```

C#

```

using System.Collections.ObjectModel;

namespace MauiApp;

public partial class EmptyViewPage : ContentPage
{
    private ObservableCollection<string> _elementy = new();

    public EmptyViewPage()
    {
    }
}

```



```

{
    InitializeComponent();
    ListView.ItemsSource = _elementy;
}

private void WyczyscClicked(object sender, EventArgs e)
{
    _elementy.Clear();
}

private void ZaladujClicked(object sender, EventArgs e)
{
    _elementy.Add("Pierwszy element");
    _elementy.Add("Drugi element");
    _elementy.Add("Trzeci element");
}
}

```

25.22. CollectionView - SelectionChanged

Reagowanie na wybór elementu z listy.

XAML

```

<?xml version="1.0" encoding="utf-8" ?>
<ContentPage xmlns="http://schemas.microsoft.com/dotnet/2021/maui"
              xmlns:x="http://schemas.microsoft.com/winfx/2009/xaml"
              x:Class="MauiApp.SelectionPage">

    <VerticalStackLayout Padding="20" Spacing="10">
        <Label x:Name="WyborLabel" Text="Wybierz element z listy" FontSize="18" />

        <CollectionView x:Name="ListView"
                       SelectionMode="Single"
                       SelectionChanged="OnSelectionChanged">
            <CollectionView.ItemTemplate>
                <DataTemplate>
                    <Label Text="{Binding .}" Padding="12" FontSize="16" />
                </DataTemplate>
            </CollectionView.ItemTemplate>
        </CollectionView>
    </VerticalStackLayout>

</ContentPage>

```

C#

```

using System.Collections.ObjectModel;

namespace MauiApp;

public partial class SelectionPage : ContentPage

```

```

{
    public SelectionPage()
    {
        InitializeComponent();

        var jezyki = new ObservableCollection<string>
        {
            "C#", "Python", "Java", "JavaScript", "TypeScript", "Go"
        };
        ListView.ItemsSource = jezyki;
    }

    private void OnSelectionChanged(object sender, SelectionChangedEventArgs e)
    {
        // Pobranie wybranego elementu
        if (e.CurrentSelection.FirstOrDefault() is string wybrany)
        {
            WyborLabel.Text = $"Wybrałeś: {wybrany}";
        }
    }
}

```

25.23. CollectionView - Header i Footer

Nagłówek i stopka listy wyświetlane nad/pod elementami.

XAML

```

<?xml version="1.0" encoding="utf-8" ?>
<ContentPage xmlns="http://schemas.microsoft.com/dotnet/2021/maui"
              xmlns:x="http://schemas.microsoft.com/winfx/2009/xaml"
              x:Class="MauiApp.HeaderFooterPage">

    <CollectionView x:Name="ListView">
        <!-- Nagłówek listy -->
        <CollectionView.Header>
            <VerticalStackLayout BackgroundColor="DarkBlue" Padding="15">
                <Label Text="🛒 Lista zakupów" FontSize="22" TextColor="White" />
            </VerticalStackLayout>
        </CollectionView.Header>

        <!-- Stopka listy -->
        <CollectionView.Footer>
            <VerticalStackLayout BackgroundColor="LightGray" Padding="10">
                <Label Text="Koniec listy" FontSize="14" HorizontalTextAlignment="Center" />
            </VerticalStackLayout>
        </CollectionView.Footer>

        <CollectionView.ItemTemplate>
            <DataTemplate>
                <Label Text="{Binding .}" Padding="10" FontSize="16" />
            </DataTemplate>
        </CollectionView.ItemTemplate>
    </CollectionView>

```

```
</CollectionView>
```

```
</ContentPage>
```

C#

```
using System.Collections.ObjectModel;

namespace MauiApp;

public partial class HeaderFooterPage : ContentPage
{
    public HeaderFooterPage()
    {
        InitializeComponent();

        var zakupy = new ObservableCollection<string>
        {
            "Chleb", "Masło", "Mleko", "Jajka", "Ser", "Szynka"
        };
        ListView.ItemsSource = zakupy;
    }
}
```

25.24. CollectionView - lista pozioma

Wyświetlanie elementów poziomo z przewijaniem w bok.

XAML

```
<?xml version="1.0" encoding="utf-8" ?>
<ContentPage xmlns="http://schemas.microsoft.com/dotnet/2021/maui"
              xmlns:x="http://schemas.microsoft.com/winfx/2009/xaml"
              x:Class="MauiApp.HorizontalListPage">

    <CollectionView x:Name="ListView" HeightRequest="120">
        <!-- Layout poziomy -->
        <CollectionView.ItemsLayout>
            <LinearItemsLayout Orientation="Horizontal" ItemSpacing="10" />
        </CollectionView.ItemsLayout>
        <CollectionView.ItemTemplate>
            <DataTemplate>
                <Frame WidthRequest="100" HeightRequest="100"
                      CornerRadius="50" BackgroundColor="Coral" Padding="0">
                    <Label Text="{Binding .}"
                          HorizontalTextAlignment="Center"
                          VerticalTextAlignment="Center"
                          TextColor="White" FontSize="14" />
                </Frame>
            </DataTemplate>
        </CollectionView.ItemTemplate>
    </CollectionView>
```

```
</ContentPage>
```

C#

```
using System.Collections.ObjectModel;

namespace MauiApp;

public partial class HorizontalListPage : ContentPage
{
    public HorizontalListPage()
    {
        InitializeComponent();

        var inicjaly = new ObservableCollection<string>
        {
            "AB", "CD", "EF", "GH", "IJ", "KL", "MN", "OP"
        };
        ListaView.ItemsSource = inicjaly;
    }
}
```

25.25. ListView - ViewCell z wieloma elementami

`ListView` z niestandardową komórką `ViewCell` .

XAML

```
<?xml version="1.0" encoding="utf-8" ?>
<ContentPage xmlns="http://schemas.microsoft.com/dotnet/2021/maui"
              xmlns:x="http://schemas.microsoft.com/winfx/2009/xaml"
              x:Class="MauiApp.ViewCellPage">

    <ListView x:Name="MojaLista" HasUnevenRows="True">
        <ListView.ItemTemplate>
            <DataTemplate>
                <ViewCell>
                    <HorizontalStackLayout Padding="10" Spacing="10">
                        <BoxView Color="{Binding Kolor}" WidthRequest="40" HeightRequest="40" />
                        <VerticalStackLayout VerticalOptions="Center">
                            <Label Text="{Binding Tytuł}" FontSize="16" FontAttributes="Bold" />
                            <Label Text="{Binding Podtytuł}" FontSize="13" TextColor="Gray" />
                        </VerticalStackLayout>
                    </HorizontalStackLayout>
                </ViewCell>
            </DataTemplate>
        </ListView.ItemTemplate>
    </ListView>

</ContentPage>
```

```

using System.Collections.ObjectModel;

namespace MauiApp;

public class ElementListy
{
    public string Tytuł { get; set; }
    public string Podtytuł { get; set; }
    public Color Kolor { get; set; }
}

public partial class ViewCellPage : ContentPage
{
    public ViewCellPage()
    {
        InitializeComponent();

        var dane = new ObservableCollection<ElementListy>
        {
            new() { Tytuł = "Błąd krytyczny", Podtytuł = "Serwer nie odpowiada", Kolor = Colors.Red },
            new() { Tytuł = "Ostrzeżenie", Podtytuł = "Mało pamięci", Kolor = Colors.Orange },
            new() { Tytuł = "Informacja", Podtytuł = "Backup zakończony", Kolor = Colors.Green }
        };
        MojaLista.ItemsSource = dane;
    }
}

```

25.26. ListView - ItemTapped

Obsługa zdarzenia dotknięcia elementu w `ListView`.

XAML

```

<?xml version="1.0" encoding="utf-8" ?>
<ContentPage xmlns="http://schemas.microsoft.com/dotnet/2021/maui"
    xmlns:x="http://schemas.microsoft.com/winfx/2009/xaml"
    x:Class="MauiApp.ItemTappedPage">

    <VerticalStackLayout Padding="20">
        <Label x:Name="InfoLabel" Text="Dotknij element" FontSize="18" />
        <ListView x:Name="MojaLista" ItemTapped="OnItemTapped">
            <ListView.ItemTemplate>
                <DataTemplate>
                    <TextCell Text="{Binding .}" />
                </DataTemplate>
            </ListView.ItemTemplate>
        </ListView>
    </VerticalStackLayout>

</ContentPage>

```

```

using System.Collections.ObjectModel;

namespace MauiApp;

public partial class ItemTappedPage : ContentPage
{
    public ItemTappedPage()
    {
        InitializeComponent();

        var owoce = new ObservableCollection<string>
        {
            "Jabłko", "Gruszka", "Śliwka", "Wiśnia", "Malina"
        };
        MojaLista.ItemsSource = owoce;
    }

    private async void OnItemTapped(object sender, ItemTappedEventArgs e)
    {
        // Pobranie dotkniętego elementu
        if (e.Item is string owoc)
        {
            InfoLabel.Text = $"Wybrałeś: {owoc}";
            await DisplayAlert("Wybór", $"Dotknięto: {owoc}", "OK");
        }

        // Resetowanie zaznaczenia
        MojaLista.SelectedItem = null;
    }
}

```

25.27. ListView - grupowanie

Grupowanie elementów w sekcje z nagłówkami.

XAML

```

<?xml version="1.0" encoding="utf-8" ?>
<ContentPage xmlns="http://schemas.microsoft.com/dotnet/2021/maui"
              xmlns:x="http://schemas.microsoft.com/winfx/2009/xaml"
              x:Class="MauiApp.GrupowaniePage">

    <ListView x:Name="MojaLista"
              IsGroupingEnabled="True"
              HasUnevenRows="True">
        <ListView.GroupHeaderTemplate>
            <DataTemplate>
                <ViewCell>
                    <Label Text="{Binding Nazwa}"
                          FontSize="18" FontAttributes="Bold"
                          BackgroundColor="LightGray" Padding="10" />
                </ViewCell>
            </DataTemplate>
        </ListView.GroupHeaderTemplate>
    </ListView>
</ContentPage>

```

```

        </ViewCell>
    </DataTemplate>
</ListView.GroupHeaderTemplate>
<ListView.ItemTemplate>
    <DataTemplate>
        <TextCell Text="{Binding .}" />
    </DataTemplate>
</ListView.ItemTemplate>
</ListView>

</ContentPage>

```

C#

```

using System.Collections.ObjectModel;

namespace MauiApp;

// Grupa dziedziczy po ObservableCollection
public class GrupaElementow : ObservableCollection<string>
{
    public string Nazwa { get; set; }

    public GrupaElementow(string nazwa, List<string> elementy) : base(elementy)
    {
        Nazwa = nazwa;
    }
}

public partial class GrupowaniePage : ContentPage
{
    public GrupowaniePage()
    {
        InitializeComponent();

        var grupy = new ObservableCollection<GrupaElementow>
        {
            new GrupaElementow("Owoce", new List<string> { "Jabłko", "Gruszka", "Banan" }),
            new GrupaElementow("Warzywa", new List<string> { "Marchew", "Ziemniak", "Pomidor" }),
            new GrupaElementow("Napoje", new List<string> { "Woda", "Sok", "Herbata" })
        };
        MojaLista.ItemsSource = grupy;
    }
}

```

25.28. ListView - Pull-to-Refresh

Odświeżanie listy przez pociągnięcie w dół.

XAML

```
<?xml version="1.0" encoding="utf-8" ?>
<ContentPage xmlns="http://schemas.microsoft.com/dotnet/2021/maui"
              xmlns:x="http://schemas.microsoft.com/winfx/2009/xaml"
              x:Class="MauiApp.PullRefreshPage">

    <ListView x:Name="MojaLista"
              IsPullToRefreshEnabled="True"
              Refreshing="OnRefreshing">
        <ListView.ItemTemplate>
            <DataTemplate>
                <TextCell Text="{Binding .}" />
            </DataTemplate>
        </ListView.ItemTemplate>
    </ListView>

</ContentPage>
```

C#

```
using System.Collections.ObjectModel;

namespace MauiApp;

public partial class PullRefreshPage : ContentPage
{
    private ObservableCollection<string> _elementy = new();
    private int _licznik = 0;

    public PullRefreshPage()
    {
        InitializeComponent();
        DodajElementy();
        MojaLista.ItemsSource = _elementy;
    }

    private void DodajElementy()
    {
        for (int i = 0; i < 3; i++)
        {
            _licznik++;
            _elementy.Add($"Element {_licznik} - {DateTime.Now:HH:mm:ss}");
        }
    }

    private async void OnRefreshing(object sender, EventArgs e)
    {
        // Symulacja pobierania danych z serwera
        await Task.Delay(1000);
        DodajElementy();

        // Zakończenie animacji odświeżania
    }
}
```



```
        MojaLista.IsRefreshing = false;
    }
}
```

25.29. CarouselView + IndicatorView

Karuzela z wskaźnikiem bieżącej strony (kropki).

XAML

```
<?xml version="1.0" encoding="utf-8" ?>
<ContentPage xmlns="http://schemas.microsoft.com/dotnet/2021/maui"
              xmlns:x="http://schemas.microsoft.com/winfx/2009/xaml"
              x:Class="MauiApp.CarouselPage">

    <VerticalStackLayout Padding="20" Spacing="20">
        <CarouselView x:Name="Karuzela"
                      IndicatorView="Wskaznik"
                      HeightRequest="200">
            <CarouselView.ItemTemplate>
                <DataTemplate>
                    <Frame BackgroundColor="{Binding Kolor}"
                          CornerRadius="15" Padding="20"
                          HorizontalOptions="Fill" VerticalOptions="Fill">
                        <Label Text="{Binding Tekst}"
                              FontSize="24" TextColor="White"
                              HorizontalTextAlignment="Center"
                              VerticalTextAlignment="Center" />
                    </Frame>
                </DataTemplate>
            </CarouselView.ItemTemplate>
        </CarouselView>

        <!-- Wskaźnik kropkowy -->
        <IndicatorView x:Name="Wskaznik"
                      IndicatorColor="LightGray"
                      SelectedIndicatorColor="DarkBlue"
                      HorizontalOptions="Center" />
    </VerticalStackLayout>

</ContentPage>
```

C#

```
using System.Collections.ObjectModel;

namespace MauiApp;

public class SlajdModel
{
    public string Tekst { get; set; }
    public Color Kolor { get; set; }
}
```

```

}

public partial class CarouselPage : ContentPage
{
    public CarouselPage()
    {
        InitializeComponent();

        var slajdy = new ObservableCollection<SlajdModel>
        {
            new() { Tekst = "Slajd 1: Witaj!", Kolor = Colors.CornflowerBlue },
            new() { Tekst = "Slajd 2: .NET MAUI", Kolor = Colors.MediumSeaGreen },
            new() { Tekst = "Slajd 3: Kolekcje", Kolor = Colors.Coral },
            new() { Tekst = "Slajd 4: Koniec", Kolor = Colors.MediumPurple }
        };

        Karuzela.ItemsSource = slajdy;
    }
}

```

25.30. SwipeView w CollectionView

Elementy z akcjami przy przesunięciu (usuwanie, edycja).

XAML

```
<?xml version="1.0" encoding="utf-8" ?>
<ContentPage xmlns="http://schemas.microsoft.com/dotnet/2021/maui"
    xmlns:x="http://schemas.microsoft.com/winfx/2009/xaml"
    x:Class="MauApp.SwipePage">

    <CollectionView x:Name="ListaView">
        <CollectionView.ItemTemplate>
            <DataTemplate>
                <SwipeView>
                    <!-- Akcja po przesunięciu w lewo -->
                    <SwipeView.RightItems>
                        <SwipeItems>
                            <SwipeItem Text="Usuń"
                                BackgroundColor="Red"
                                Invoked="UsunSwipe" />
                        </SwipeItems>
                    </SwipeView.RightItems>

                    <!-- Akcja po przesunięciu w prawo -->
                    <SwipeView.LeftItems>
                        <SwipeItems>
                            <SwipeItem Text="Ulubione"
                                BackgroundColor="Gold"
                                Invoked="UlubioneSwipe" />
                        </SwipeItems>
                    </SwipeView.LeftItems>

                    <!-- Zawartość elementu -->
```

```

        <Frame Padding="15" Margin="5">
            <Label Text="{Binding .}" FontSize="16" />
        </Frame>
    </SwipeView>
</DataTemplate>
</CollectionView.ItemTemplate>
</CollectionView>

</ContentPage>

```

C#

```

using System.Collections.ObjectModel;

namespace MauiApp;

public partial class SwipePage : ContentPage
{
    private ObservableCollection<string> _elementy = new()
    {
        "Notatka 1", "Notatka 2", "Notatka 3", "Notatka 4", "Notatka 5"
    };

    public SwipePage()
    {
        InitializeComponent();
        ListaView.ItemsSource = _elementy;
    }

    private void UsunSwipe(object sender, EventArgs e)
    {
        // Usunięcie elementu po przesunięciu
        if (sender is SwipeItem swipeItem && swipeItem.BindingContext is string element)
        {
            _elementy.Remove(element);
        }
    }

    private async void UlubioneSwipe(object sender, EventArgs e)
    {
        if (sender is SwipeItem swipeItem && swipeItem.BindingContext is string element)
        {
            await DisplayAlert("Ulubione", $"Dodano: {element}", "OK");
        }
    }
}

```

25.31. Wyszukiwanie - SearchBar + filtrowanie LINQ

Filtrowanie listy w czasie rzeczywistym przy użyciu SearchBar.

XAML

```
<?xml version="1.0" encoding="utf-8" ?>
<ContentPage xmlns="http://schemas.microsoft.com/dotnet/2021/maui"
              xmlns:x="http://schemas.microsoft.com/winfx/2009/xaml"
              x:Class="MauiApp.WyszukiwaniePage">

    <VerticalStackLayout Padding="10" Spacing="10">
        <SearchBar x:Name="Szukaj"
                  Placeholder="Szukaj..."
                  TextChanged="OnTextChanged" />
        <CollectionView x:Name="ListView">
            <CollectionView.ItemTemplate>
                <DataTemplate>
                    <Label Text="{Binding .}" Padding="10" FontSize="16" />
                </DataTemplate>
            </CollectionView.ItemTemplate>
        </CollectionView>
    </VerticalStackLayout>

</ContentPage>
```

C#

```
using System.Collections.ObjectModel;

namespace MauiApp;

public partial class WyszukiwaniePage : ContentPage
{
    private List<string> _wszystkie = new()
    {
        "Algorytm", "Baza danych", "Chmura", "Docker", "Ethernet",
        "Framework", "Git", "HTML", "Internet", "JSON"
    };

    private ObservableCollection<string> _wyswietlane = new();

    public WyszukiwaniePage()
    {
        InitializeComponent();
        _wyswietlane = new ObservableCollection<string>(_wszystkie);
        ListView.ItemsSource = _wyswietlane;
    }

    private void OnTextChanged(object sender, TextChangedEventArgs e)
    {
        // Filtrowanie LINQ - szukanie bez rozróżniania wielkości liter
        var fraza = e.NewTextValue?.ToLower() ?? "";
        var wyniki = _wszystkie
            .Where(x => x.ToLower().Contains(fraza))
            .ToList();
    }
}
```

```

        _wyswietlane.Clear();
        foreach (var w in wyniki)
            _wyswietlane.Add(w);
    }
}

```

25.32. Sortowanie kolekcji

Sortowanie listy rosnąco i malejąco z aktualizacją widoku.

XAML

```

<?xml version="1.0" encoding="utf-8" ?>
<ContentPage xmlns="http://schemas.microsoft.com/dotnet/2021/maui"
              xmlns:x="http://schemas.microsoft.com/winfx/2009/xaml"
              x:Class="MauiApp.SortowaniePage">

    <VerticalStackLayout Padding="20" Spacing="10">
        <HorizontalStackLayout Spacing="10">
            <Button Text="A → Z" Clicked="SortujRosnaco" />
            <Button Text="Z → A" Clicked="SortujMalejaco" />
        </HorizontalStackLayout>
        <CollectionView x:Name="ListView">
            <CollectionView.ItemTemplate>
                <DataTemplate>
                    <Label Text="{Binding .}" Padding="10" FontSize="16" />
                </DataTemplate>
            </CollectionView.ItemTemplate>
        </CollectionView>
    </VerticalStackLayout>

</ContentPage>

```

C#

```

using System.Collections.ObjectModel;

namespace MauiApp;

public partial class SortowaniePage : ContentPage
{
    private List<string> _dane = new()
    {
        "Zebra", "Antylopa", "Kangur", "Delfin", "Orzeł", "Bóbr"
    };

    private ObservableCollection<string> _wyswietlane;

    public SortowaniePage()
    {
        InitializeComponent();
        _wyswietlane = new ObservableCollection<string>(_dane);
    }
}

```

```

        ListView.ItemsSource = _wyswietlane;
    }

    private void SortujRosnaco(object sender, EventArgs e)
    {
        // Sortowanie LINQ rosnaco
        var posortowane = _wyswietlane.OrderBy(x => x).ToList();
        _wyswietlane.Clear();
        foreach (var el in posortowane)
            _wyswietlane.Add(el);
    }

    private void SortujMalejaco(object sender, EventArgs e)
    {
        // Sortowanie LINQ malejaco
        var posortowane = _wyswietlane.OrderByDescending(x => x).ToList();
        _wyswietlane.Clear();
        foreach (var el in posortowane)
            _wyswietlane.Add(el);
    }
}

```

25.33. Lista obiektów z sortowaniem po właściwości

Sortowanie listy obiektów po wybranym polu (np. cena, nazwa).

XAML

```

<?xml version="1.0" encoding="utf-8" ?>
<ContentPage xmlns="http://schemas.microsoft.com/dotnet/2021/maui"
              xmlns:x="http://schemas.microsoft.com/winfx/2009/xaml"
              x:Class="MauiApp.SortowanieObiektowPage">

    <VerticalStackLayout Padding="20" Spacing="10">
        <HorizontalStackLayout Spacing="10">
            <Button Text="Sortuj po nazwie" Clicked="SortujNazwa" />
            <Button Text="Sortuj po cenie" Clicked="SortujCena" />
        </HorizontalStackLayout>
        <CollectionView x:Name="ListView">
            <CollectionView.ItemTemplate>
                <DataTemplate>
                    <HorizontalStackLayout Padding="10" Spacing="15">
                        <Label Text="{Binding Nazwa}" FontSize="16" FontAttributes="Bold" />
                        <Label Text="{Binding Cena, StringFormat='{0:F2} zł'}" FontSize="14" TextColor="Gray" />
                    </HorizontalStackLayout>
                </DataTemplate>
            </CollectionView.ItemTemplate>
        </CollectionView>
    </VerticalStackLayout>

</ContentPage>

```

```
using System.Collections.ObjectModel;

namespace MauiApp;

public class Towar
{
    public string Nazwa { get; set; }
    public double Cena { get; set; }
}

public partial class SortowanieObiektowPage : ContentPage
{
    private ObservableCollection<Towar> _towary = new()
    {
        new() { Nazwa = "Monitor", Cena = 1200.00 },
        new() { Nazwa = "Kabel HDMI", Cena = 35.50 },
        new() { Nazwa = "Pendrive 64GB", Cena = 49.99 },
        new() { Nazwa = "Słuchawki", Cena = 189.00 }
    };

    public SortowanieObiektowPage()
    {
        InitializeComponent();
        ListView.ItemsSource = _towary;
    }

    private void SortujNazwa(object sender, EventArgs e)
    {
        var sorted = _towary.OrderBy(t => t.Nazwa).ToList();
        _towary.Clear();
        foreach (var t in sorted) _towary.Add(t);
    }

    private void SortujCena(object sender, EventArgs e)
    {
        var sorted = _towary.OrderBy(t => t.Cena).ToList();
        _towary.Clear();
        foreach (var t in sorted) _towary.Add(t);
    }
}
```

25.34. Lista z pliku tekstowego

Wczytywanie elementów listy z pliku tekstowego zapisanego w zasobach aplikacji.

XAML

```
<?xml version="1.0" encoding="utf-8" ?>
<ContentPage xmlns="http://schemas.microsoft.com/dotnet/2021/maui"
              xmlns:x="http://schemas.microsoft.com/winfx/2009/xaml"
              x:Class="MauiApp.ZPlikuPage">
```

```

<VerticalStackLayout Padding="20" Spacing="10">
    <Button Text="Wczytaj z pliku" Clicked="WczytajClicked" />
    <Label x:Name="StatusLabel" Text="" TextColor="Gray" />
    <CollectionView x:Name="ListView">
        <CollectionView.ItemTemplate>
            <DataTemplate>
                <Label Text="{Binding .}" Padding="8" FontSize="16" />
            </DataTemplate>
        </CollectionView.ItemTemplate>
    </CollectionView>
</VerticalStackLayout>

</ContentPage>

```

C#

```

using System.Collections.ObjectModel;

namespace MauiApp;

public partial class ZPlikuPage : ContentPage
{
    private ObservableCollection<string> _elementy = new();

    public ZPlikuPage()
    {
        InitializeComponent();
        ListView.ItemsSource = _elementy;
    }

    private async void WczytajClicked(object sender, EventArgs e)
    {
        try
        {
            // Wczytanie pliku z folderu Raw (MauiAsset)
            using var stream = await FileSystem.OpenAppPackageFileAsync("dane.txt");
            using var reader = new StreamReader(stream);

            _elementy.Clear();
            string linia;
            while ((linia = await reader.ReadLineAsync()) != null)
            {
                if (!string.IsNullOrEmpty(linia))
                {
                    _elementy.Add(linia.Trim());
                }
            }

            StatusLabel.Text = $"Wczytano {_elementy.Count} elementów";
        }
        catch (Exception ex)
        {
            StatusLabel.Text = $"Błąd: {ex.Message}";
        }
    }
}

```



```
}  
}
```

25.35. Lista z pliku CSV (obiekty)

Wczytywanie obiektów z pliku CSV i wyświetlanie ich na liście.

XAML

```
<?xml version="1.0" encoding="utf-8" ?>  
<ContentPage xmlns="http://schemas.microsoft.com/dotnet/2021/maui"  
             xmlns:x="http://schemas.microsoft.com/winfx/2009/xaml"  
             x:Class="MauiApp.CsvPage">  
  
    <VerticalStackLayout Padding="20" Spacing="10">  
        <Button Text="Wczytaj CSV" Clicked="WczytajCsv" />  
        <CollectionView x:Name="ListaView">  
            <CollectionView.ItemTemplate>  
                <DataTemplate>  
                    <HorizontalStackLayout Padding="8" Spacing="10">  
                        <Label Text="{Binding Imie}" FontSize="16" FontAttributes="Bold" />  
                        <Label Text="{Binding Wiek, StringFormat='{0} \tlat)'}" FontSize="14" TextColor="Gray" />  
                    </HorizontalStackLayout>  
                </DataTemplate>  
            </CollectionView.ItemTemplate>  
        </CollectionView>  
    </VerticalStackLayout>  
  
</ContentPage>
```

C#

```
using System.Collections.ObjectModel;  
  
namespace MauiApp;  
  
public class Osoba  
{  
    public string Imie { get; set; }  
    public int Wiek { get; set; }  
}  
  
public partial class CsvPage : ContentPage  
{  
    private ObservableCollection<Osoba> _osoby = new();  
  
    public CsvPage()  
    {  
        InitializeComponent();  
        ListaView.ItemsSource = _osoby;  
    }  
}
```

```

private async void WczytajCsv(object sender, EventArgs e)
{
    // Plik CSV: imie;wiek (w Resources/Raw)
    using var stream = await FileSystem.OpenAppPackageFileAsync("osoby.csv");
    using var reader = new StreamReader(stream);

    _osoby.Clear();
    string linia;
    while ((linia = await reader.ReadLineAsync()) != null)
    {
        var czesci = linia.Split(';');
        if (czesci.Length == 2 && int.TryParse(czesci[1], out int wiek))
        {
            _osoby.Add(new Osoba { Imie = czesci[0], Wiek = wiek });
        }
    }
}

```

25.36. Nawigacja Poprzedni/Następny z zawijaniem

Przyciski „Poprzedni” i „Następny” przechodzą między elementami listy z zawijaniem (po ostatnim wraca pierwszy).

XAML

```

<?xml version="1.0" encoding="utf-8" ?>
<ContentPage xmlns="http://schemas.microsoft.com/dotnet/2021/maui"
    xmlns:x="http://schemas.microsoft.com/winfx/2009/xaml"
    x:Class="MauApp.NawigacjaPage">

    <VerticalStackLayout Padding="30" Spacing="20" VerticalOptions="Center">
        <Label x:Name="IndeksLabel" Text="1 / 5" HorizontalTextAlignment="Center" FontSize="14" TextColor="Gray" />

        <Frame Padding="20" CornerRadius="10" BackgroundColor="AliceBlue">
            <Label x:Name="ElementLabel" Text="" FontSize="22" HorizontalTextAlignment="Center" />
        </Frame>

        <HorizontalStackLayout HorizontalOptions="Center" Spacing="20">
            <Button Text="◀ Poprzedni" Clicked="PoprzedniClicked" />
            <Button Text="Następny ▶" Clicked="NastepnyClicked" />
        </HorizontalStackLayout>
    </VerticalStackLayout>

</ContentPage>

```

C#

```

namespace MauApp;

public partial class NawigacjaPage : ContentPage
{
    private List<string> _elementy = new()

```

```

{
    "Poniedziałek", "Wtorek", "Środa", "Czwartek", "Piątek"
};

private int _aktualnyIndex = 0;

public NawigacjaPage()
{
    InitializeComponent();
    WyświetlAktualny();
}

private void WyświetlAktualny()
{
    ElementLabel.Text = _elementy[_aktualnyIndex];
    IndeksLabel.Text = $"{_aktualnyIndex + 1} / {_elementy.Count}";
}

private void PoprzedniClicked(object sender, EventArgs e)
{
    // Zawijanie – po pierwszym przechodzi do ostatniego
    _aktualnyIndex--;
    if (_aktualnyIndex < 0)
        _aktualnyIndex = _elementy.Count - 1;
    WyświetlAktualny();
}

private void NastepnyClicked(object sender, EventArgs e)
{
    // Zawijanie – po ostatnim przechodzi do pierwszego
    _aktualnyIndex++;
    if (_aktualnyIndex ≥ _elementy.Count)
        _aktualnyIndex = 0;
    WyświetlAktualny();
}
}

```

25.37. CollectionView - wielokrotny wybór (Multi-Select)

Umożliwienie wyboru wielu elementów jednocześnie.

XAML

```

<?xml version="1.0" encoding="utf-8" ?>
<ContentPage xmlns="http://schemas.microsoft.com/dotnet/2021/maui"
    xmlns:x="http://schemas.microsoft.com/winfx/2009/xaml"
    x:Class="MauiApp.MultiSelectPage">

    <VerticalStackLayout Padding="20" Spacing="10">
        <Label Text="Zaznacz kilka elementów:" FontSize="16" />
        <CollectionView x:Name="ListView"
            SelectionMode="Multiple"
            SelectionChanged="OnMultiSelect">

```

```

        <CollectionView.ItemTemplate>
            <DataTemplate>
                <Label Text="{Binding .}" Padding="12" FontSize="16" />
            </DataTemplate>
        </CollectionView.ItemTemplate>
    </CollectionView>

    <Label x:Name="WynikLabel" Text="" FontSize="14" TextColor="DarkGreen" />
    <Button Text="Pokaż zaznaczone" Clicked="PokazClicked" />
</VerticalStackLayout>

</ContentPage>

```

C#

```

using System.Collections.ObjectModel;

namespace MauiApp;

public partial class MultiSelectPage : ContentPage
{
    private List<string> _zaznaczone = new();

    public MultiSelectPage()
    {
        InitializeComponent();

        var technologie = new ObservableCollection<string>
        {
            "XAML", "C#", "Binding", "REST API", "SQLite", "Blazor"
        };
        ListView.ItemsSource = technologie;
    }

    private void OnMultiSelect(object sender, SelectionChangedEventArgs e)
    {
        // Zapisanie zaznaczonych elementów
        _zaznaczone = e.CurrentSelection.Cast<string>().ToList();
    }

    private async void PokazClicked(object sender, EventArgs e)
    {
        var tekst = string.Join(", ", _zaznaczone);
        WynikLabel.Text = $"Wybrano: {tekst}";
        await DisplayAlert("Zaznaczono", tekst, "OK");
    }
}

```

25.38. Filtrowanie obiektów z wieloma kryteriami

XAML

```
<?xml version="1.0" encoding="utf-8" ?>
<ContentPage xmlns="http://schemas.microsoft.com/dotnet/2021/maui"
              xmlns:x="http://schemas.microsoft.com/winfx/2009/xaml"
              x:Class="MauApp.FiltrowanieZlozonePage">

    <VerticalStackLayout Padding="15" Spacing="10">

        <SearchBar x:Name="Szukaj" Placeholder="Szukaj po nazwie..." TextChanged="FiltrujListe" />
        <Picker x:Name="KategoriaPicker" Title="Kategoria"
              SelectedIndexChanged="FiltrujListe">
            <Picker.Items>
                <x:String>Wszystkie</x:String>
                <x:String>Elektronika</x:String>
                <x:String>Odzież</x:String>
                <x:String>Jedzenie</x:String>
            </Picker.Items>
        </Picker>
        <CollectionView x:Name="ListaView">
            <CollectionView.ItemTemplate>
                <DataTemplate>
                    <HorizontalStackLayout Padding="10" Spacing="10">
                        <Label Text="{Binding Nazwa}" FontSize="16" />
                        <Label Text="{Binding Kategoria, StringFormat='{0}}'" TextColor="Gray" FontSize="13" />
                    </HorizontalStackLayout>
                </DataTemplate>
            </CollectionView.ItemTemplate>
        </CollectionView>
    </VerticalStackLayout>

</ContentPage>
```

C#

```
using System.Collections.ObjectModel;

namespace MauApp;

public class ProduktFiltr
{
    public string Nazwa { get; set; }
    public string Kategoria { get; set; }
}

public partial class FiltrowanieZlozonePage : ContentPage
{
    private List<ProduktFiltr> _wszystkie = new()
    {
        new() { Nazwa = "Laptop", Kategoria = "Elektronika" },
        new() { Nazwa = "Koszulka", Kategoria = "Odzież" },
        new() { Nazwa = "Telefon", Kategoria = "Elektronika" },
    };
}
```

```

        new() { Nazwa = "Chleb", Kategoria = "Jedzenie" },
        new() { Nazwa = "Spodnie", Kategoria = "Odzież" },
        new() { Nazwa = "Masło", Kategoria = "Jedzenie" },
        new() { Nazwa = "Tablet", Kategoria = "Elektronika" }
    };

    private ObservableCollection<ProduktFiltr> _wyswietlane = new();

    public FiltrowanieZłożonePage()
    {
        InitializeComponent();
        KategoriaPicker.SelectedIndex = 0;
        _wyswietlane = new ObservableCollection<ProduktFiltr>(_wszystkie);
        ListView.ItemsSource = _wyswietlane;
    }

    private void FiltrujListe(object sender, EventArgs e)
    {
        var fraza = Szukaj.Text?.ToLower() ?? "";
        var kategoria = KategoriaPicker.SelectedItem?.ToString() ?? "Wszystkie";

        // Połączenie filtrów LINQ
        var wyniki = _wszystkie
            .Where(p => p.Nazwa.ToLower().Contains(fraza))
            .Where(p => kategoria == "Wszystkie" || p.Kategoria == kategoria)
            .ToList();

        _wyswietlane.Clear();
        foreach (var p in wyniki)
            _wyswietlane.Add(p);
    }
}

```

25.39. CollectionView - RefreshView (pull-to-refresh)

Nowoczesny sposób odświeżania z `RefreshView` wokół `CollectionView`.

XAML

```

<?xml version="1.0" encoding="utf-8" ?>
<ContentPage xmlns="http://schemas.microsoft.com/dotnet/2021/maui"
              xmlns:x="http://schemas.microsoft.com/winfx/2009/xaml"
              x:Class="MauiApp.RefreshViewPage">

    <RefreshView x:Name="Refresher" Refreshing="OnRefreshing">
        <CollectionView x:Name="ListView">
            <CollectionView.ItemTemplate>
                <DataTemplate>
                    <Label Text="{Binding .}" Padding="12" FontSize="16" />
                </DataTemplate>
            </CollectionView.ItemTemplate>
        </CollectionView>
    </RefreshView>

```

```
</ContentPage>
```

C#

```
using System.Collections.ObjectModel;

namespace MauiApp;

public partial class RefreshViewPage : ContentPage
{
    private ObservableCollection<string> _wiadomosci = new()
    {
        "Wiadomość 1", "Wiadomość 2", "Wiadomość 3"
    };
    private int _licznik = 3;

    public RefreshViewPage()
    {
        InitializeComponent();
        ListView.ItemsSource = _wiadomosci;
    }

    private async void OnRefreshing(object sender, EventArgs e)
    {
        // Symulacja pobrania nowych danych
        await Task.Delay(800);
        _licznik++;
        _wiadomosci.Insert(0, $"Nowa wiadomość {_licznik} - {DateTime.Now:HH:mm:ss}");

        // Zatrzymanie animacji
        Refresher.IsRefreshing = false;
    }
}
```

25.40. ObservableCollection - przenoszenie elementów (Move)

Zmiana kolejności elementów na liście za pomocą metody `Move`.

XAML

```
<?xml version="1.0" encoding="utf-8" ?>
<ContentPage xmlns="http://schemas.microsoft.com/dotnet/2021/maui"
              xmlns:x="http://schemas.microsoft.com/winfx/2009/xaml"
              x:Class="MauiApp.PrzenoszeniePage">

    <VerticalStackLayout Padding="20" Spacing="10">
        <CollectionView x:Name="ListView" SelectionMode="Single" />
        <HorizontalStackLayout HorizontalOptions="Center" Spacing="15">
            <Button Text="⬆ Do góry" Clicked="DoGoryClicked" />
            <Button Text="⬇ W dół" Clicked="WDolClicked" />
        </HorizontalStackLayout>
    </VerticalStackLayout>
</ContentPage>
```

```
</VerticalStackLayout>
```

```
</ContentPage>
```

C#

```
using System.Collections.ObjectModel;

namespace MauiApp;

public partial class PrzenoszeniePage : ContentPage
{
    private ObservableCollection<string> _elementy = new()
    {
        "Pierwszy", "Drugi", "Trzeci", "Czwarty", "Piąty"
    };

    public PrzenoszeniePage()
    {
        InitializeComponent();
        ListView.ItemsSource = _elementy;
    }

    private void DoGoryClicked(object sender, EventArgs e)
    {
        if (ListView.SelectedItem is string wybrany)
        {
            int index = _elementy.IndexOf(wybrany);
            if (index > 0)
            {
                // Przesunięcie elementu o jedną pozycję w górę
                _elementy.Move(index, index - 1);
            }
        }
    }

    private void WDoLClicked(object sender, EventArgs e)
    {
        if (ListView.SelectedItem is string wybrany)
        {
            int index = _elementy.IndexOf(wybrany);
            if (index < _elementy.Count - 1)
            {
                // Przesunięcie elementu o jedną pozycję w dół
                _elementy.Move(index, index + 1);
            }
        }
    }
}
```


25.41. CollectionView - SelectionMode None z TapGestureRecognizer

Alternatywna obsługa dotyku bez zaznaczenia elementu.

XAML

```
<?xml version="1.0" encoding="utf-8" ?>
<ContentPage xmlns="http://schemas.microsoft.com/dotnet/2021/maui"
              xmlns:x="http://schemas.microsoft.com/winfx/2009/xaml"
              x:Class="MauiApp.TapGesturePage">

    <VerticalStackLayout Padding="20" Spacing="10">
        <Label x:Name="InfoLabel" Text="Dotknij element" FontSize="18" />
        <CollectionView x:Name="ListView" SelectionMode="None">
            <CollectionView.ItemTemplate>
                <DataTemplate>
                    <Frame Padding="12" Margin="3">
                        <Frame.GestureRecognizers>
                            <TapGestureRecognizer Tapped="OnItemTapped"
                                                    BindingContext="{Binding .}" />
                        </Frame.GestureRecognizers>
                        <Label Text="{Binding .}" FontSize="16" />
                    </Frame>
                </DataTemplate>
            </CollectionView.ItemTemplate>
        </CollectionView>
    </VerticalStackLayout>

</ContentPage>
```

C#

```
using System.Collections.ObjectModel;

namespace MauiApp;

public partial class TapGesturePage : ContentPage
{
    public TapGesturePage()
    {
        InitializeComponent();

        var elementy = new ObservableCollection<string>
        {
            "Element A", "Element B", "Element C", "Element D"
        };
        ListView.ItemsSource = elementy;
    }

    private void OnItemTapped(object sender, TappedEventArgs e)
    {
        // Pobranie elementu z BindingContext gestu
        if (sender is TapGestureRecognizer tap && tap.BindingContext is string wartosc)
        {
        }
    }
}
```

```

        InfoLabel.Text = $"Dotknięto: {wartosc}";
    }
}
}

```

25.42. Lista z licznikiem - dodawanie i usuwanie z podsumowaniem

Praktyczny przykład listy zadań z licznikiem elementów.

XAML

```

<?xml version="1.0" encoding="utf-8" ?>
<ContentPage xmlns="http://schemas.microsoft.com/dotnet/2021/maui"
              xmlns:x="http://schemas.microsoft.com/winfx/2009/xaml"
              x:Class="MauApp.LicznikPage">

    <VerticalStackLayout Padding="20" Spacing="10">
        <Label x:Name="LicznikLabel" Text="Zadań: 0" FontSize="20" FontAttributes="Bold" />
        <HorizontalStackLayout Spacing="10">
            <Entry x:Name="ZadanieEntry" Placeholder="Nowe zadanie" HorizontalOptions="FillAndExpand" />
            <Button Text="+" Clicked="DodajClicked" WidthRequest="50" />
        </HorizontalStackLayout>
        <CollectionView x:Name="ListView" SelectionMode="Single">
            <CollectionView.ItemTemplate>
                <DataTemplate>
                    <HorizontalStackLayout Padding="10" Spacing="10">
                        <Label Text="{Binding .}" FontSize="16" HorizontalOptions="FillAndExpand" />
                    </HorizontalStackLayout>
                </DataTemplate>
            </CollectionView.ItemTemplate>
        </CollectionView>
        <Button Text="Usuń zaznaczone" Clicked="UsunClicked" BackgroundColor="Red" TextColor="White" />
    </VerticalStackLayout>

</ContentPage>

```

C#

```

using System.Collections.ObjectModel;

namespace MauApp;

public partial class LicznikPage : ContentPage
{
    private ObservableCollection<string> _zadania = new();

    public LicznikPage()
    {
        InitializeComponent();
        ListView.ItemsSource = _zadania;
        // Reagowanie na zmiany w kolekcji
        _zadania.CollectionChanged += (s, e) => AktualizujLicznik();
    }
}

```

```

    }

    private void AktualizujLicznik()
    {
        LicznikLabel.Text = $"Zadań: {_zadania.Count}";
    }

    private void DodajClickled(object sender, EventArgs e)
    {
        if (!string.IsNullOrWhiteSpace(ZadanieEntry.Text))
        {
            _zadania.Add(ZadanieEntry.Text);
            ZadanieEntry.Text = string.Empty;
        }
    }

    private void UsunClickled(object sender, EventArgs e)
    {
        if (ListView.SelectedItem is string zadanie)
        {
            _zadania.Remove(zadanie);
            ListView.SelectedItem = null;
        }
    }
}

```

25.43. Nawigacja Poprzedni/Następny z obiektami (szczegóły)

Przeglądanie szczegółów obiektów z nawigacją i zawijaniem.

XAML

```

<?xml version="1.0" encoding="utf-8" ?>
<ContentPage xmlns="http://schemas.microsoft.com/dotnet/2021/maui"
    xmlns:x="http://schemas.microsoft.com/winfx/2009/xaml"
    x:Class="MauiApp.SzczegolyNawigacjaPage">

    <VerticalStackLayout Padding="30" Spacing="15" VerticalOptions="Center">
        <Label x:Name="PozycjaLabel" HorizontalTextAlignment="Center" FontSize="13" TextColor="Gray" />

        <Frame Padding="25" CornerRadius="12" BackgroundColor="Lavender">
            <VerticalStackLayout Spacing="8">
                <Label x:Name="NazwaLabel" FontSize="22" FontAttributes="Bold" />
                <Label x:Name="OpisLabel" FontSize="16" TextColor="DimGray" />
                <Label x:Name="CenaLabel" FontSize="18" TextColor="DarkGreen" />
            </VerticalStackLayout>
        </Frame>

        <HorizontalStackLayout HorizontalOptions="Center" Spacing="20">
            <Button Text="◀ Poprzedni" Clicked="PoprzedniClickled" />
            <Button Text="Następny ▶" Clicked="NastepnyClickled" />
        </HorizontalStackLayout>
    </VerticalStackLayout>

```

</ContentPage>

C#

```
namespace MauiApp;

public class ProduktNav
{
    public string Nazwa { get; set; }
    public string Opis { get; set; }
    public double Cena { get; set; }
}

public partial class SzczegolyNawigacjaPage : ContentPage
{
    private List<ProduktNav> _produkty = new()
    {
        new() { Nazwa = "Laptop", Opis = "Wydajny notebook do pracy", Cena = 4299.00 },
        new() { Nazwa = "Tablet", Opis = "Lekki, z rysikiem", Cena = 2199.00 },
        new() { Nazwa = "Smartfon", Opis = "Flagowy model 2024", Cena = 3599.00 },
        new() { Nazwa = "Smartwatch", Opis = "Z pomiarem tętna", Cena = 899.00 }
    };

    private int _index = 0;

    public SzczegolyNawigacjaPage()
    {
        InitializeComponent();
        Wyświetl();
    }

    private void Wyświetl()
    {
        var p = _produkty[_index];
        NazwaLabel.Text = p.Nazwa;
        OpisLabel.Text = p.Opis;
        CenaLabel.Text = $"{p.Cena:F2} zł";
        PozycjaLabel.Text = $"{_index + 1} / {_produkty.Count}";
    }

    private void PoprzedniClicked(object sender, EventArgs e)
    {
        // Zawijanie: po pierwszym wraca ostatni
        _index = (_index - 1 + _produkty.Count) % _produkty.Count;
        Wyświetl();
    }

    private void NastepnyClicked(object sender, EventArgs e)
    {
        // Zawijanie: po ostatnim wraca pierwszy
        _index = (_index + 1) % _produkty.Count;
        Wyświetl();
    }
}
```

```
}  
  
}
```

25.44. Podsumowanie

W tej części przedstawiono **30 kompletnych przykładów** obejmujących:

- `ObservableCollection` - dodawanie, usuwanie, edycja, czyszczenie, przenoszenie
- `CollectionView` - DataTemplate prosty i złożony, GridItemsLayout, EmptyView, SelectionChanged, Header/Footer, lista pozioma, multi-select, RefreshView, TapGesture
- `ListView` - ViewCell, ItemTapped, grupowanie, pull-to-refresh
- `CarouselView` + `IndicatorView`
- `SwipeView` w liście
- Wyszukiwanie/filtrowanie z `SearchBar` + LINQ (proste i złożone)
- Sortowanie (stringi i obiekty)
- Lista obiektów, lista z pliku tekstowego, lista z CSV
- Nawigacja Poprzedni/Następny z zawijaniem (stringi i obiekty)

Każdy przykład zawiera kompletny kod XAML oraz C# gotowy do użycia w projekcie .NET MAUI.

26. Binding danych w praktyce

26.1. Czym jest binding

Binding to automatyczne połączenie właściwości kontrolki (np. `Label.Text`) z właściwością obiektu danych (np. `Uzytkownik.Imie`). Gdy zmieni się dana, kontrolka sama się aktualizuje (i odwrotnie, w trybie dwukierunkowym). Zamiast `Etykieta.Text = obiekt.Imie;` piszemy w XAML `Text="{Binding Imie}"`.

Binding pozwala **oddzielić dane od widoku** i uniknąć ręcznego przepisywania wartości. Widok „sam się rysuje” na podstawie danych, a code-behind może skupić się na obsłudze zdarzeń.

NAJWAŻNIEJSZE INFORMACJE

- Binding łączy **źródło** (obiekt danych) z **celem** (właściwość kontrolki).
- Źródło ustawiamy przez `BindingContext`.
- W XAML: `Text="{Binding Nazwa}"`.
- Do automatycznego odświeżania potrzebny jest `INotifyPropertyChanged`.

PRZYKŁAD

```
public class Uzytkownik  
{  
    public string Imie { get; set; }  
    public int Wiek { get; set; }  
}
```

```
<VerticalStackLayout>  
    <Label Text="{Binding Imie}" FontSize="22" />  
    <Label Text="{Binding Wiek}" />  
</VerticalStackLayout>
```

```
// Ustawienie źródła danych
BindingContext = new Uzytkownik { Imie = "Anna", Wiek = 28 };
```

Na co uważać:

Binding to inny sposób myślenia niż code-behind: nie ustawiasz wartości „ręcznie”, lecz deklarujesz powiązanie. Na początek bywa mniej intuicyjny, ale przy większych aplikacjach znacząco upraszcza kod.

26.2. Po co stosuje się binding

Binding **eliminuje powtarzalny kod** synchronizacji widoku z danymi. Bez niego po każdej zmianie danych musielibyśmy ręcznie aktualizować kontrolki; z bindingiem dzieje się to automatycznie.

NAJWAŻNIEJSZE INFORMACJE

- Mniej kodu (brak ręcznego `Etykieta.Text = ...`).
- Czytelny podział: dane osobno, widok osobno.
- Wygodne połączenie XAML z obiektami danych.
- Automatyczne odświeżanie widoku po zmianie danych.

Na co uważać:

Dla bardzo prostych, jednoekranowych aplikacji code-behind bywa szybszy. Binding opłaca się przy listach, szablonach elementów i formularzach z wieloma polami. Oba podejścia są poprawne.

26.3. BindingContext

BindingContext to właściwość określająca **obiekt-źródło danych** dla kontrolki i jej dzieci. Ustawiamy go zwykle raz, na poziomie strony, a wszystkie kontrolki w środku **dziedziczą** go w dół drzewa wizualnego.

PRZYKŁAD C#

```
public MainPage()
{
    InitializeComponent();
    BindingContext = new Uzytkownik { Imie = "Anna", Wiek = 28 }; // źródło dla całej strony
}
```

PRZYKŁAD XAML (USTAWIENIE KONTEKSTU W XAML)

```
<ContentPage.BindingContext>
    <local:Uzytkownik />
</ContentPage.BindingContext>
```

Na co uważać:

Jeśli powiązania „nic nie pokazują”, **najpierw sprawdź** **BindingContext**. Bez niego ścieżki bindingu nie mają do czego się odnosić, a kontrolki pozostają puste. To najczęstszy błąd przy bindingu.

26.4. Binding w XAML i w C#

Binding deklarujemy najczęściej w **XAML** wyrażeniem `{Binding Ścieżka}`. Można go też utworzyć w **C#** metodą `SetBinding`, ale to rzadsze. W XAML jest czytelniej.

PRZYKŁAD XAML

```
<Entry Text="{Binding Imie}" Placeholder="imię" />
```

```
<Label Text="{Binding Wiek}" />
```

PRZYKŁAD W C#

```
// Binding utworzony w kodzie (rzadziej używane)
var etykieta = new Label();
etykieta.SetBinding(Label.TextProperty, "Imie");
etykieta.BindingContext = uzytkownik;
```

Na co uważać:

W codziennej pracy używamy bindingu w XAML - jest czytelny i zwięzły. Binding w C# przydaje się przy kontrolkach tworzonych dynamicznie.

26.5. Ścieżka i StringFormat

W wyrażeniu `{Binding Imie}` słowo `Imie` to **ścieżka** - nazwa właściwości źródła. Ścieżka może sięgać głębiej (`{Binding Adres.Miasto}`), a `{Binding .}` oznacza całe źródło. `StringFormat` pozwala sformatować wyświetlaną wartość.

PRZYKŁAD XAML

```
←!— Formatowanie liczby jako ceny →
<Label Text="{Binding Cena, StringFormat='Cena: {0:0.00} zł'}" />

←!— Zagnieżdżona ścieżka →
<Label Text="{Binding Adres.Miasto}" />

←!— Całe źródło (dla list napisów) →
<Label Text="{Binding .}" />
```

Na co uważać:

`StringFormat` używa składni jak `string.Format` (`{0}` to wartość). Dla list zawierających proste wartości (napisy) używamy `{Binding .}` - kropka oznacza bieżący element.

26.6. Tryby bindingu: OneWay, TwoWay, OneTime

Tryb (`Mode`) określa **kierunek przepływu danych**. `OneWay` (domyślny dla większości) - źródło → widok. `TwoWay` - w obie strony (dla pól edycji). `OneTime` - raz, przy załadowaniu. `OneWayToSource` - tylko widok → źródło.

NAJWAŻNIEJSZE INFORMACJE

TRYB	KIERUNEK	TYPOWE UŻYCIE
<code>OneWay</code>	źródło → widok	etykiety, wyświetlanie
<code>TwoWay</code>	źródło ↔ widok	<code>Entry</code> , <code>Switch</code> , <code>Slider</code>
<code>OneTime</code>	źródło → widok (raz)	dane niezmiennie
<code>OneWayToSource</code>	widok → źródło	rzadkie

PRZYKŁAD XAML

```
←!— Pole edycji - dwukierunkowo, by zmiany wracały do danych →
<Entry Text="{Binding Imie, Mode=TwoWay}" />
```

←!— Etykieta - jednokierunkowo wystarczy →

```
<Label Text="{Binding Imie}" />
```

Na co uważać:

Dla pól, w których użytkownik **wpisuje** dane (**Entry**, **Switch**, **Slider**), używaj **TwoWay**, by zmiany wracały do źródła. Wiele kontrolki ma sensowny tryb domyślny, ale przy polach edycji warto ustawić go świadomie.

26.7. Binding do tekstu, liczby, wartości logicznej i listy

Binding działa z różnymi typami danych. Tekst wiążemy z **Label.Text** / **Entry.Text**, liczbę z **Slider.Value**, wartość logiczną z **Switch.IsToggled** / **IsVisible**, a kolekcję z **CollectionView.ItemsSource**.

PRZYKŁAD XAML

```
←!— Tekst →
<Label Text="{Binding Tytuł}" />

←!— Liczba →
<Slider Value="{Binding Głosnosc, Mode=TwoWay}" Maximum="100" />

←!— Bool →
<Switch IsToggled="{Binding Powiadomienia, Mode=TwoWay}" />
<Label IsVisible="{Binding CzyBłąd}" Text="Wystąpił błąd" />

←!— Lista →
<CollectionView ItemsSource="{Binding Produkty}" />
```

Na co uważać:

Binding wartości logicznej do **IsVisible** to wygodny sposób pokazywania/ukrywania elementów zależnie od stanu - bez pisania kodu. Liczby i wartości logiczne wiążemy **TwoWay**, gdy użytkownik je zmienia.

26.8. Binding w CollectionView

W **CollectionView** wiążemy kolekcję z **ItemsSource**, a w **ItemTemplate** wiążemy właściwości pojedynczego elementu. To najczęstsze zastosowanie bindingu w praktyce.

PRZYKŁAD XAML

```
<CollectionView ItemsSource="{Binding Produkty}">
  <CollectionView.ItemTemplate>
    <DataTemplate>
      <Grid Padding="10" ColumnDefinitions="*,Auto">
        <Label Text="{Binding Nazwa}" Grid.Column="0" />
        <Label Text="{Binding Cena, StringFormat='{0:0.00} zł'}" Grid.Column="1" />
      </Grid>
    </DataTemplate>
  </CollectionView.ItemTemplate>
</CollectionView>
```

Na co uważać:

W szablonie **BindingContext** to **pojedynczy element** kolekcji, więc **{Binding Nazwa}** odnosi się do właściwości tego elementu. Gdy w przycisku wewnątrz szablonu chcesz pobrać bieżący element, najprościej w code-behind odczytać **BindingContext** przycisku.

26.9. INotifyPropertyChanged - temat opcjonalny

Sam binding nie wystarczy, by widok **reagował na zmiany danych w trakcie działania**. Obiekt-źródło musi **powiadomić** o zmianie - służy do tego interfejs **INotifyPropertyChanged** ze zdarzeniem **PropertyChanged**. Wywołujemy je w setterze właściwości, by zasygnalizować „ta wartość się zmieniła”.

PRZYKŁAD C#

```
using System.ComponentModel;
using System.Runtime.CompilerServices;

public class ObiektPowiadamiajacy : INotifyPropertyChanged
{
    public event PropertyChangedEventHandler PropertyChanged;

    protected void OnPropertyChanged([CallerMemberName] string nazwa = null)
        => PropertyChanged?.Invoke(this, new PropertyChangedEventArgs(nazwa));

    protected bool SetProperty<T>(ref T pole, T wartosc, [CallerMemberName] string nazwa = null)
    {
        if (Equals(pole, wartosc)) return false;
        pole = wartosc;
        OnPropertyChanged(nazwa);
        return true;
    }
}

public class LicznikDanych : ObiektPowiadamiajacy
{
    int liczba;
    public int Liczba
    {
        get => liczba;
        set => SetProperty(ref liczba, value); // powiadamia widok
    }
}
```

Na co uważać:

To temat przydatny przy bardziej rozbudowanym bindingu. Jeśli właściwość zmienia się w kodzie, ale widok **się nie odświeża**, może brakować `INotifyPropertyChanged` albo wywołania `OnPropertyChanged` w setterze. Atrybut `[CallerMemberName]` pozwala nie podawać nazwy właściwości ręcznie.

26.10. Odświeżanie listy - ObservableCollection

Dla list rolę powiadamiania pełni `ObservableCollection<T>` - przy dodaniu/usunięciu elementu sama informuje `CollectionView`, który się odświeża. To „listowy odpowiednik” `INotifyPropertyChanged`.

Na co uważać:

Pamiętaj o dwóch poziomach powiadamiania: `INotifyPropertyChanged` dla **właściwości** (np. tekst, liczba) i `ObservableCollection` dla **kolekcji**. Zmiana właściwości obiektu wewnątrz listy wymaga, by sam obiekt implementował `INotifyPropertyChanged`.

26.11. Typowe błędy z bindingiem

NAJWAŻNIEJSZE INFORMACJE

BŁĄD	SKUTEK	ROZWIĄZANIE
Brak <code>BindingContext</code>	puste kontrolki	ustaw <code>BindingContext</code>
Literówka w ścieżce <code>{Binding ...}</code>	brak wartości	sprawdź nazwę właściwości
Brak <code>INotifyPropertyChanged</code>	widok się nie odświeża	implementuj interfejs
<code>List</code> zamiast <code>ObservableCollection</code>	lista się nie odświeża	użyj <code>ObservableCollection</code>
Brak <code>Mode=TwoWay</code> w polu edycji	zmiany nie wracają do danych	ustaw <code>TwoWay</code>
Handler nie wie, którego elementu dotyczy przycisk w liście	nie działa usuwanie/edycja	odczytaj <code>BindingContext</code> przycisku

Na co uważać:

Trzy najczęstsze problemy: brak `BindingContext`, brak `INotifyPropertyChanged` i użycie `List` zamiast `ObservableCollection`. Gdy „binding nie działa”, sprawdź je w tej kolejności - to rozwiązuje większość przypadków.

Binding to most między danymi a widokiem. Zapamiętaj: ustaw `BindingContext`, dla pól edycji używaj `TwoWay`, a do dynamicznych list używaj `ObservableCollection`. `INotifyPropertyChanged` potraktuj jako kolejny krok, gdy chcesz automatycznie odświeżać pojedyncze właściwości obiektu.

27. Obrazy i zasoby graficzne

Obrazy są wszechobecne w aplikacjach: logo, zdjęcia, ikony, grafiki stanu. Co więcej, w wielu aplikacjach obraz **zmienia się dynamicznie** w zależności od stanu - to częsty i ważny wzorec. Ten rozdział szczegółowo omawia dodawanie obrazów do projektu, zasady nazewnictwa, źródła obrazów, dynamiczną podmianę, klikalne obrazy oraz pracę z wieloma obrazami naraz, z praktycznymi przykładami (galeria, zdjęcie użytkownika, obrazy kości, ikony).

27.1. Dodawanie obrazów do projektu

Obrazy aplikacji umieszczamy w folderze `Resources/Images`. MAUI traktuje je jako zasoby (`MauiImage`) i automatycznie przygotowuje w odpowiednich rozdzielczościach dla różnych ekranów. Aby dodać grafikę, wystarczy umieścić plik (np. `logo.png`) w tym folderze.

Folder `Resources/Images` to **centralne miejsce na wszystkie obrazy** dołączone do aplikacji. Dzięki temu nie martwimy się o ścieżki specyficzne dla platform ani o różne rozdzielczości - MAUI robi to za nas.

NAJWAŻNIEJSZE INFORMACJE

- Obrazy umieszczamy w `Resources/Images`.
- Obsługiwane formaty to m.in. **PNG** i **SVG** (wektorowe, skalowalne).
- Po dodaniu obrazu często trzeba **przebudować** projekt.
- Nazwa pliku staje się jego identyfikatorem w kodzie.

Na co uważać:

SVG jest szczególnie wygodny dla ikon, bo skaluje się bez utraty jakości. MAUI sam przekształca SVG na odpowiednie rozmiary. Po dodaniu nowego obrazu wykonaj Rebuild, by został zarejestrowany.

27.2. Nazewnictwo plików graficznych

MAUI ma **rygorystyczne wymagania** co do nazw plików obrazów. Nazwa musi składać się z **małych liter**, cyfr i podkreślników - bez spacji, myślników i wielkich liter. Złamanie tej zasady powoduje błąd budowania.

NAJWAŻNIEJSZE INFORMACJE

POPRAWNA NAZWA	NIEPOPRAWNA NAZWA	DLACZEGO ŹŁE
Logo.png	Logo.png	wielka litera
kostka1.png	kostka 1.png	spacja
ikona_kosz.png	ikona-kosz.png	myślnik
zdjecie_1.png	Zdjęcie 1.PNG	wielkie litery, spacja

Na co uważać:

To **bardzo częsty błąd początkujących**. Trzymaj się schematu: małe litery, cyfry, podkreślniki. `Moja Grafika.png` → zmień na `moja_grafika.png`. Po zmianie nazwy przebuduj projekt.

Plik o nazwie z wielką literą, spacją lub myślnikiem **uniemożliwi zbudowanie** aplikacji. To jedna z najczęstszych przyczyn błędów budowania związanych z zasobami.

27.3. Image.Source - wyświetlanie obrazu

Obraz wyświetla kontrolka `Image`, a jego źródło wskazuje właściwość `Source`. Najczęściej podajemy nazwę pliku z `Resources/Images` (np. `Source="logo.png"`).

PRZYKŁAD XAML

```
<Image Source="logo.png"
        Aspect="AspectFit"
        HeightRequest="120" />
```

PRZYKŁAD W C#

```
// Ustawienie obrazu w kodzie (nazwa pliku jako string)
Logo.Source = "logo.png";
```

Na co uważać:

W C# `Source` ustawiamy napisem z nazwą pliku. Pamiętaj o poprawnej nazwie (małe litery) i o tym, że plik musi być w `Resources/Images`.

27.4. Źródła obrazu: zasób, internet, plik lokalny

Obraz może pochodzić z trzech źródeł. **Zasób aplikacji** - plik dołączony do aplikacji (`Source="logo.png"`). **Internet** - obraz pobierany z adresu URL (`Source="https://..."`). **Plik lokalny** - obraz zapisany na urządzeniu, podawany przez ścieżkę.

NAJWAŻNIEJSZE INFORMACJE

ŹRÓDŁO	ZAPIS	UWAGI
Zasób aplikacji	<code>Source="logo.png"</code>	małe litery, w <code>Resources/Images</code>
Internet (URL)	<code>Source="https://example.com/foto.png"</code>	wymaga sieci
Plik lokalny	<code>ImageSource.FromFile(ścieżka)</code>	np. z <code>AppDataDirectory</code>

PRZYKŁAD C#

```
// Zasób aplikacji
Obraz.Source = "logo.png";
```

```
// Z internetu (URL)
Obraz.Source = "https://example.com/foto.png";

// Z pliku na urządzeniu
Obraz.Source = ImageSource.FromFile(sciezkaDoPliku);
```

Na co uważać:

Obraz z internetu wymaga połączenia i może się nie załadować - warto przewidzieć obraz zastępczy. Obraz z pliku lokalnego (np. zdjęcie zrobione aparatem) podajemy przez `ImageSource.FromFile` z pełną ścieżką.

27.5. Aspect - dopasowanie obrazu

Właściwość `Aspect` decyduje, jak obraz dopasuje się do przydzielonego miejsca. Najważniejsze wartości to `AspectFit` (mieści cały obraz, zachowuje proporcje), `AspectFill` (wypełnia obszar, przycina nadmiar) i `Fill` (rozciąga, może zniekształcić).

NAJWAŻNIEJSZE INFORMACJE

WARTOŚĆ	DZIAŁANIE	KIEDY UŻYWAĆ
<code>AspectFit</code>	cały obraz, proporcje zachowane	gdy ważne, by widać było całość
<code>AspectFill</code>	wypełnia obszar, przycina	tła, miniatury
<code>Fill</code>	rozciąga bez proporcji	rzadko (może zniekształcić)
<code>Center</code>	oryginalny rozmiar, wyśrodkowany	małe ikony

PRZYKŁAD XAML

```
<Image Source="zdjecie.png" Aspect="AspectFit" HeightRequest="200" />
<Image Source="tlo.png" Aspect="AspectFill" HeightRequest="200" />
```

Na co uważać:

`AspectFit` może zostawić puste pasy (bo zachowuje proporcje), a `AspectFill` przycina obraz (bo wypełnia obszar). Wybierz zależnie od tego, czy ważniejsze jest pokazanie całości, czy wypełnienie obszaru. `Fill` używaj rzadko - zniekształca obraz.

27.6. Opacity - przezroczystość

Właściwość `Opacity` ustawia **przezroczystość** obrazu w zakresie od `0.0` (całkowicie niewidoczny) do `1.0` (pełne krycie). Wartość pośrednia (np. `0.4`) daje efekt „przygaszenia” - przydatny do oznaczania elementów nieaktywnych lub zablokowanych.

PRZYKŁAD C#

```
ObrazKostki.Opacity = 1.0; // pełna widoczność (aktywny)
ObrazKostki.Opacity = 0.4; // przygaszony (np. zablokowany)
```

Na co uważać:

`Opacity` to świetny sposób na wizualne odróżnienie elementów aktywnych od nieaktywnych bez ich ukrywania. W grze z kośćmi przygaszenie (`0.4`) jasno pokazuje, które kości są zablokowane.

27.7. Zmiana obrazu w czasie działania aplikacji

Bardzo często obraz **zmienia się dynamicznie** w reakcji na stan lub akcję - podmieniamy `Source`, by pokazać inną grafikę. To wzorec używany w grach (kości, karty), panelach (ikona stanu) i galeriach.

PRZYKŁAD C#

```
private bool wlaczone = false;

private void OnPrzełącz(object sender, EventArgs e)
{
    wlaczone = !wlaczone;
    // Podmiana obrazu zależnie od stanu
    Ikona.Source = wlaczone ? "wlaczone.png" : "wywlaczone.png";
}
```

Na co uważać:

Podmiana `Source` natychmiast przerysowuje obraz. Upewnij się, że oba pliki (`wlaczone.png`, `wywlaczone.png`) istnieją w `Resources/Images` i mają poprawne nazwy.

27.8. Budowanie nazwy obrazka na podstawie danych

Gdy mamy serię obrazów o regularnych nazwach (np. `kostka1.png` ... `kostka6.png`), możemy zbudować nazwę dynamicznie z danych za pomocą interpolacji stringów. To eliminuje długie instrukcje `switch`.

PRZYKŁAD C#

```
int wartosc = los.Next(1, 7); // 1..6
// Zamiast switch - budujemy nazwę pliku z liczby
ObrazKostki.Source = $"kostka{wartosc}.png"; // np. "kostka3.png"
```

Na co uważać:

To bardzo wygodny wzorzec, ale wymaga **konsekwentnego nazewnictwa** plików (`kostka1.png`, `kostka2.png` ...). Jeśli któregoś pliku brakuje, obraz się nie pokaże. Upewnij się, że wszystkie potrzebne pliki istnieją.

27.9. Obsługa braku obrazka

Czasem obraz może się nie załadować - brak pliku, błędna nazwa, brak sieci (przy URL). Warto **przewidzieć** taką sytuację, np. ustawiając obraz zastępczy lub sprawdzając dane przed podmianą.

PRZYKŁAD C#

```
private void UstawObraz(string nazwa)
{
    // Prosty mechanizm zabezpieczający - obraz zastępczy przy pustej nazwie
    Obraz.Source = string.IsNullOrEmpty(nazwa) ? "placeholder.png" : nazwa;
}
```

Na co uważać:

Dla obrazów z internetu rozważ pokazanie `ActivityIndicator` podczas ładowania oraz obrazu zastępczego, gdy pobranie się nie powiedzie. Dla obrazów z zasobów najważniejsze to pewność, że plik istnieje i ma poprawną nazwę.

27.10. Klikalny obraz - TapGestureRecognizer

PRZYKŁAD XAML

```
<Image Source="serce_puste.png" x:Name="PoLubienie" HeightRequest="50">
    <Image.GestureRecognizers>
        <TapGestureRecognizer Tapped="OnPoLub" />
    </Image.GestureRecognizers>
</Image>
```

```
</Image.GestureRecognizers>
</Image>
```

PRZYKŁAD C#

```
private bool polubione = false;

private void OnPolub(object sender, EventArgs e)
{
    polubione = !polubione;
    Polubienie.Source = polubione ? "serce_pełne.png" : "serce_puste.png";
}
```

Na co uważać:

27.11. Wiele obrazów na ekranie i rozpoznawanie kliknięcia

Gdy na ekranie jest **wiele obrazów** obsługiwanych jednym handlerem (np. siatka kart, kości), musimy rozpoznać, **który** obraz dotknięto. Najprościej nadać każdemu unikalne `ClassId` (np. numer) i odczytać je z `sender`.

PRZYKŁAD XAML

```
<HorizontalStackLayout Spacing="8" HorizontalOptions="Center">
    <Image Source="kostka1.png" ClassId="0" HeightRequest="60">
        <Image.GestureRecognizers>
            <TapGestureRecognizer Tapped="OnKostka" />
        </Image.GestureRecognizers>
    </Image>
    <Image Source="kostka1.png" ClassId="1" HeightRequest="60">
        <Image.GestureRecognizers>
            <TapGestureRecognizer Tapped="OnKostka" />
        </Image.GestureRecognizers>
    </Image>
    ←!— kolejne obrazy z ClassId="2", "3", "4" →
</HorizontalStackLayout>
```

PRZYKŁAD C#

```
private void OnKostka(object sender, EventArgs e)
{
    var obraz = (Image)sender;
    int indeks = int.Parse(obraz.ClassId); // który obraz
    // np. przełączenie przezroczystości (blokada)
    obraz.Opacity = obraz.Opacity == 1.0 ? 0.4 : 1.0;
}
```

Na co uważać:

`ClassId` to prosty sposób rozróżnienia wielu obrazów jednym handlerem. Możesz też trzymać obrazy w **tablicy** (`Image[] kostki = { Kostka0, Kostka1, ... }`) i operować na nich w pętli - to wygodne w grach.

27.12. Obrazy jako element stanu - przykład gry w kości

W grach obraz często **odzwierciedla stan**: wartość kości to obraz `kostkaN.png`, blokada to `Opacity`. Łączymy tu obrazy, stan (tablice) i interakcje (gesty).

PRZYKŁAD C#

```
Image[] kostki;
int[] wartosci = new int[5];
bool[] zablokowana = new bool[5];
readonly Random los = new Random();

public MainPage()
{
    InitializeComponent();
    kostki = new[] { Kostka0, Kostka1, Kostka2, Kostka3, Kostka4 };
}

private void OnRzut(object sender, EventArgs e)
{
    for (int i = 0; i < kostki.Length; i++)
    {
        if (zablokowana[i]) continue; // pomiń zablokowane
        wartosci[i] = los.Next(1, 7);
        kostki[i].Source = $"kostka{wartosci[i]}.png"; // obraz ze stanu
    }
}

private void OnKostkaDotkniete(object sender, EventArgs e)
{
    var obraz = (Image)sender;
    int i = int.Parse(obraz.ClassId);
    zablokowana[i] = !zablokowana[i]; // przełącz blokadę
    obraz.Opacity = zablokowana[i] ? 0.4 : 1.0; // pokaż blokadę
}
```

Na co uważać:

To wzorcowy przykład „obraz = odbicie stanu”. Wartości i blokady trzymamy w tablicach, a obrazy tylko je wyświetlają. `continue` pomija zablokowane kości przy losowaniu. Zebranie obrazów w tablicę upraszcza obsługę w pętli.

27.13. Inne praktyczne zastosowania obrazów

NAJWAŻNIEJSZE INFORMACJE

- **Galeria** - `CollectionView` z `Image` w szablonie, dane to lista ścieżek/URL.
- **Zdjęcie użytkownika** - `Image` z `ImageSource.FromFile` (zdjęcie z aparatu albo galerii).
- **Ikony przycisków** - `ImageButton` lub `Button` z `ImageSource`.
- **Obrazy stanu** - podmiana `Source` zależnie od danych (włączone/wyłączone, kości).

PRZYKŁAD XAML (GALERIA)

```
<CollectionView ItemsSource="{Binding Zdjecia}">
    <CollectionView.ItemsLayout>
        <GridItemsLayout Orientation="Vertical" Span="3" />
    </CollectionView.ItemsLayout>
    <CollectionView.ItemTemplate>
```

```

        <DataTemplate>
            <Image Source="{Binding .}" Aspect="AspectFill" HeightRequest="120" />
        </DataTemplate>
    </CollectionView.ItemTemplate>
</CollectionView>

```

Na co uważać:

`GridItemsLayout` z `Span="3"` tworzy siatkę 3 kolumn - idealną dla galerii miniatur. Dla miniatur dobrze pasuje `AspectFill` (wypełnia kafelek). Dla zdjęć z urządzenia używaj `ImageSource.FromFile`.

27.14. Typowe błędy z obrazami

NAJWAŻNIEJSZE INFORMACJE

BŁĄD	SKUTEK	ROZWIĄZANIE
Wielka litera/spacja w nazwie	błąd budowania	małe litery, podkreślniki
Brak pliku w <code>Resources/Images</code>	obraz się nie pokazuje	dodaj plik, przebuduj
Brak przebudowania po dodaniu	obraz nieznany	Rebuild
Złe <code>Aspect</code> (zniekształcenie)	obraz rozciągnięty	<code>AspectFit</code> / <code>AspectFill</code>
Niespójne nazwy serii	brak obrazu przy interpolacji	ujednolić nazwy
Obraz z URL bez sieci	pusty obraz	obraz zastępczy, obsługa błędu

Na co uważać:

Najczęstsze problemy z obrazami to **nazwy plików** (wielkie litery, spacje) i **brak przebudowania** po dodaniu. Dwie złote zasady: nazywaj pliki małymi literami bez spacji, a po dodaniu obrazu wykonuj Rebuild.

Obrazy to często najbardziej „dynamiczna” część interfejsu - podmieniasz je w reakcji na stan. Opanuj trzy rzeczy: poprawne nazewnictwo, podmianę `Source` w kodzie (z interpolacją nazwy) oraz `Opacity` do oznaczania stanu. To pokrywa większość zastosowań.

27.15. Obraz z zasobu (Resources/Images)

Pliki obrazów umieszczone w `Resources/Images` są automatycznie dostępne po nazwie (bez rozszerzenia w XAML).

```

<?xml version="1.0" encoding="utf-8" ?>
<ContentPage xmlns="http://schemas.microsoft.com/dotnet/2021/maui"
    xmlns:x="http://schemas.microsoft.com/winfx/2009/xaml"
    x:Class="MojaAplikacja.ObrazZasobPage"
    Title="Obraz z zasobu">
    <VerticalStackLayout Padding="20" Spacing="15">
        <Label Text="Obrazy z Resources/Images:" FontSize="18" FontAttributes="Bold"/>

        <!-- Obraz z zasobu - podajemy nazwę pliku bez rozszerzenia -->
        <Image Source="logo" HeightRequest="100" Aspect="AspectFit"/>

        <!-- Obraz PNG z zasobu -->
        <Image Source="ikona_start.png" HeightRequest="80" Aspect="AspectFit"/>

        <!-- Obraz SVG z zasobu (MAUI konwertuje automatycznie) -->
        <Image Source="strzałka.png" HeightRequest="60" Aspect="AspectFit"/>
    </VerticalStackLayout>
</ContentPage>

```



```

        <Label Text="Obrazy powyżej pochodzą z folderu Resources/Images projektu."
              FontSize="12" TextColor="Gray"/>
    </VerticalStackLayout>
</ContentPage>

```

```

namespace MojaAplikacja;

public partial class ObrazZasobPage : ContentPage
{
    public ObrazZasobPage()
    {
        InitializeComponent();

        // Obrazy z zasobów nie wymagają kodu C# – wystarczy XAML.
        // Można też ustawić źródło programowo:
        private void UstawObrazProgramowo()
        {
            // Sposób 1: z nazwy pliku zasobu
            var image = new Image
            {
                Source = ImageSource.FromFile("logo.png"),
                HeightRequest = 100,
                Aspect = Aspect.AspectFit
            };
        }
    }
}

```

27.16. Obraz z URL

Ładujemy obraz bezpośrednio z adresu internetowego. MAUI automatycznie pobiera i cache'uje obraz.

```

<?xml version="1.0" encoding="utf-8" ?>
<ContentPage xmlns="http://schemas.microsoft.com/dotnet/2021/maui"
              xmlns:x="http://schemas.microsoft.com/winfx/2009/xaml"
              x:Class="MojaAplikacja.ObrazUrlPage"
              Title="Obraz z URL">
    <VerticalStackLayout Padding="20" Spacing="15">
        <Label Text="Obraz pobrany z Internetu:" FontSize="18"/>
        <Entry x:Name="EntryUrl" Placeholder="Wklej URL obrazu"
              Text="https://picsum.photos/400/300"/>
        <Button Text="Załaduj obraz" Clicked="OnZaładujClicked"/>

        <!-- Obraz z URL bezpośrednio w XAML -->
        <Image Source="https://picsum.photos/400/200"
              HeightRequest="200" Aspect="AspectFit"/>

        <!-- Obraz ładowany dynamicznie -->
        <Image x:Name="ImageDynamiczny" HeightRequest="200" Aspect="AspectFit"/>
    </VerticalStackLayout>
</ContentPage>

```

```
</VerticalStackLayout>

</ContentPage>
```

```
namespace MojaAplikacja;

public partial class ObrazUrlPage : ContentPage
{
    public ObrazUrlPage()
    {
        InitializeComponent();

        private void OnZaladujClicked(object sender, EventArgs e)
        {
            string url = EntryUrl.Text?.Trim();
            if (!string.IsNullOrEmpty(url))
            {
                // Ustawiamy źródło obrazu z URI
                ImageDynamiczny.Source = new UriImageSource
                {
                    Uri = new Uri(url),
                    // Czas cache'owania (opcjonalnie)
                    CacheValidity = TimeSpan.FromHours(24)
                };
            }
        }
    }
}
```

27.17. Obraz z pliku lokalnego

Wyświetlamy obraz zapisany na dysku urządzenia (np. wcześniej pobrany lub zrobiony aparatem).

```
<?xml version="1.0" encoding="utf-8" ?>
<ContentPage xmlns="http://schemas.microsoft.com/dotnet/2021/maui"
    xmlns:x="http://schemas.microsoft.com/winfx/2009/xaml"
    x:Class="MojaAplikacja.ObrazLokalnyPage"
    Title="Obraz z pliku lokalnego">
    <VerticalStackLayout Padding="20" Spacing="15">
        <Button Text="Wybierz obraz z dysku" Clicked="OnWybierzClicked" />
        <Image x:Name="ImageLokalny" HeightRequest="300" Aspect="AspectFit" />
        <Label x:Name="LabelSciezka" FontSize="11" TextColor="Gray" />
    </VerticalStackLayout>
</ContentPage>
```

```
namespace MojaAplikacja;

public partial class ObrazLokalnyPage : ContentPage
{

```

```

public ObrazLokalnyPage()
{
    InitializeComponent();
}

private async void OnWybierzCliked(object sender, EventArgs e)
{
    try
    {
        var wynik = await FilePicker.Default.PickAsync(new PickOptions
        {
            FileTypes = FilePickerFileType.Images,
            PickerTitle = "Wybierz obraz"
        });

        if (wynik != null)
        {
            LabelSciezka.Text = wynik.FullPath;

            // Sposób 1: z pełnej ścieżki pliku
            ImageLokalny.Source = ImageSource.FromFile(wynik.FullPath);

            // Sposób 2: ze strumienia (alternatywa)
            // var stream = await wynik.OpenReadAsync();
            // ImageLokalny.Source = ImageSource.FromStream(() => stream);
        }
    }
    catch (Exception ex)
    {
        await DisplayAlert("Błąd", ex.Message, "OK");
    }
}
}

```

27.18. Podmiana obrazu w runtime + budowanie nazwy

Zmieniamy wyświetlany obraz dynamicznie, budując nazwę pliku na podstawie logiki aplikacji.

```

<?xml version="1.0" encoding="utf-8" ?>
<ContentPage xmlns="http://schemas.microsoft.com/dotnet/2021/maui"
    xmlns:x="http://schemas.microsoft.com/winfx/2009/xaml"
    x:Class="MojaAplikacja.PodmianaObrazuPage"
    Title="Podmiana obrazu">
    <VerticalStackLayout Padding="20" Spacing="15">
        <Label Text="Wybierz kategorię:" FontSize="16"/>
        <Picker x:Name="PickerKategoria" SelectedIndexChanged="OnKategoriaChanged">
            <Picker.ItemsSource>
                <x:Array Type="{x:Type x:String}">
                    <x:String>słońce</x:String>
                    <x:String>deszcz</x:String>
                    <x:String>śnieg</x:String>
                    <x:String>wiatr</x:String>
                </x:Array>
            </Picker.ItemsSource>
        </Picker>
    </VerticalStackLayout>
</ContentPage>

```

```

        </x:Array>
    </Picker.ItemsSource>
</Picker>

<Image x:Name="ImagePogoda" HeightRequest="200" Aspect="AspectFit"/>
<Label x:Name="LabelNazwaPliku" FontSize="12" TextColor="Gray"/>

<Label Text="Numer wariantu (1-3):" FontSize="14"/>
<Stepper x:Name="StepperWariant" Minimum="1" Maximum="3" Increment="1"
    ValueChanged="OnWariantChanged"/>
<Label x:Name="LabelWariant" Text="1"/>
</VerticalStackLayout>
</ContentPage>

```

```

namespace MojaAplikacja;

public partial class PodmianaObrazuPage : ContentPage
{
    public PodmianaObrazuPage()
    {
        InitializeComponent();
    }

    private void OnKategoriaChanged(object sender, EventArgs e)
    {
        AktualizujObraz();
    }

    private void OnWariantChanged(object sender, ValueChangedEventArgs e)
    {
        LabelWariant.Text = ((int)e.NewValue).ToString();
        AktualizujObraz();
    }

    private void AktualizujObraz()
    {
        // Pobieramy wybraną kategorię
        string kategoria = PickerKategoria.SelectedItem as string;
        if (string.IsNullOrEmpty(kategoria)) return;

        int wariant = (int)StepperWariant.Value;

        // Budujemy nazwę pliku dynamicznie: np. "pogoda_slonce_2.png"
        string nazwaPliku = $"pogoda_{kategoria}_{wariant}.png";

        // Podmieniamy źródło obrazu
        ImagePogoda.Source = ImageSource.FromFile(nazwaPliku);
        LabelNazwaPliku.Text = $"Plik: {nazwaPliku}";
    }
}

```

27.19. Klikalny obraz - TapGestureRecognizer

Dodajemy obsługę kliknięcia/tapnięcia na obrazie za pomocą gestu.

```
<?xml version="1.0" encoding="utf-8" ?>
<ContentPage xmlns="http://schemas.microsoft.com/dotnet/2021/maui"
              xmlns:x="http://schemas.microsoft.com/winfx/2009/xaml"
              x:Class="MojaAplikacja.KlikalnyObrazPage"
              Title="Klikalny obraz">
    <VerticalStackLayout Padding="20" Spacing="15">
        <Label Text="Kliknij na obraz poniżej:" FontSize="16"/>

        <!-- Obraz z obsługą tapnięcia -->
        <Image x:Name="ImageKlikalny" Source="przycisk_start.png"
              HeightRequest="150" Aspect="AspectFit">
            <Image.GestureRecognizers>
                <TapGestureRecognizer Tapped="OnObrazKlikniety" NumberOfTapsRequired="1"/>
            </Image.GestureRecognizers>
        </Image>

        <Label x:Name="LabelLicznik" Text="Kliknięcia: 0" FontSize="18"/>

        <!-- Drugi obraz - podwójne kliknięcie -->
        <Label Text="Kliknij dwukrotnie poniższy obraz:" FontSize="14"/>
        <Image Source="ikona_serce.png" HeightRequest="100" Aspect="AspectFit">
            <Image.GestureRecognizers>
                <TapGestureRecognizer Tapped="OnPodwojneKlikniecie" NumberOfTapsRequired="2"/>
            </Image.GestureRecognizers>
        </Image>

        <Label x:Name="LabelInfo" TextColor="HotPink" FontSize="16"/>
    </VerticalStackLayout>
</ContentPage>
```

```
namespace MojaAplikacja;

public partial class KlikalnyObrazPage : ContentPage
{
    private int _licznik = 0;

    public KlikalnyObrazPage()
    {
        InitializeComponent();
    }

    private void OnObrazKlikniety(object sender, TappedEventArgs e)
    {
        // Zwiększamy licznik kliknięć
        _licznik++;
        LabelLicznik.Text = $"Kliknięcia: {_licznik}";

        // Podmieniamy obraz po kliknięciu
```

```

        ImageKlikalny.Source = _licznik % 2 == 0
            ? "przycisk_start.png"
            : "przycisk_aktywny.png";
    }

    private async void OnPodwojneKlikniecie(object sender, TappedEventArgs e)
    {
        // Animacja + komunikat przy podwójnym kliknięciu
        LabelInfo.Text = "❤️ Polubiono!";

        // Prosta animacja skalowania
        var obraz = sender as Image;
        if (obraz != null)
        {
            await obraz.ScaleTo(1.3, 150);
            await obraz.ScaleTo(1.0, 150);
        }
    }
}

```

28. Pliki lokalne i zasoby Raw

28.1. Czym jest plik lokalny

Plik lokalny to plik zapisany na urządzeniu, na którym działa aplikacja. Może to być plik tekstowy z danymi, JSON, zdjęcie czy dowolny inny zasób. Aplikacja może **odczytywać** i **zapisywać** pliki w swoim prywatnym katalogu, dzięki czemu dane przetrwają zamknięcie aplikacji.

Pliki lokalne służą do **trwałego przechowywania** danych: zapisanej notatki, eksportu listy, cache danych z sieci, ustawień w formacie tekstowym. To sposób na zapamiętanie większych danych niż drobne ustawienia, do których zwykle wystarczy **Preferences**.

NAJWAŻNIEJSZE INFORMACJE

- Pliki lokalne są zapisane **na urządzeniu**, w katalogu aplikacji.
- Dane w plikach **przetrwają** zamknięcie aplikacji.
- Operacje na plikach wykonujemy **asynchronicznie** (**await**).
- Ścieżki budujemy bezpiecznie przez **Path.Combine** i **FileSystem.AppDataDirectory**.

Na co uważać:

Każda platforma ma inną strukturę katalogów i ograniczenia dostępu. Dlatego nie wpisujemy ścieżek „na sztywno” (jak **C:\dane**), lecz używamy katalogów udostępnionych przez MAUI (**FileSystem.AppDataDirectory**).

28.2. Plik projektu a plik użytkownika

To kluczowe rozróżnienie. **Plik projektu** (zasób) jest **dołączony do aplikacji** podczas budowania - np. obraz w **Resources/Images** lub dane startowe w **Resources/Raw**. Jest **tylko do odczytu** i ten sam dla wszystkich użytkowników. **Plik użytkownika** powstaje **w trakcie działania** aplikacji (np. zapisana notatka) i jest zapisywany w prywatnym katalogu aplikacji.

NAJWAŻNIEJSZE INFORMACJE

CECHA	PLIK PROJEKTU (ZASÓB)	PLIK UŻYTKOWNIKA
Powstaje	przy budowaniu	w trakcie działania
Dostęp	tylko odczyt	odczyt i zapis
Lokalizacja	<code>Resources/Raw</code> , <code>Resources/Images</code>	<code>FileSystem.AppDataDirectory</code>
Zawartość	ta sama dla wszystkich	indywidualna

Na co uważać:

Nie da się **nadpisać** pliku-zasobu dołączonego do aplikacji - jest tylko do odczytu. Aby zapisać zmienione dane, skopiuj je do pliku w `AppDataDirectory` i tam zapisuj. To częste nieporozumienie: „zapisuję do `Resources/Raw`” - tak się nie da.

28.3. Zasób a plik zapisywany przez aplikację

Zasób (`Resources/Raw`, `Resources/Images`) to dane wbudowane w aplikację - czytamy je przez specjalny mechanizm (`FileSystem.OpenAppPackageFileAsync`). Plik zapisywany przez aplikację to plik w `AppDataDirectory` - czytamy i zapisujemy go zwykłymi metodami klasy `File`.

PRZYKŁAD C# (ODCZYT ZASOBU Z RESOURCES/RAW)

```
// Odczyt pliku dołączonego do aplikacji (tylko do odczytu)
using var stream = await FileSystem.OpenAppPackageFileAsync("dane.txt");
using var reader = new StreamReader(stream);
string tresc = await reader.ReadToEndAsync();
```

Na co uważać:

Plik z `Resources/Raw` czytamy przez `FileSystem.OpenAppPackageFileAsync("nazwa")` - nie przez `File.ReadAllText` ze ścieżką, bo zasób jest „spakowany” w aplikacji. To częsty błąd: próba czytania zasobu jak zwykłego pliku ze ścieżki.

28.4. Foldery Resources/Raw i Resources/Images

`Resources/Raw` to folder na **dowolne pliki surowe** dołączone do aplikacji: dane startowe (`dane.txt`), pliki JSON, konfiguracje. `Resources/Images` to folder na obrazy. Oba to zasoby tylko do odczytu.

NAJWAŻNIEJSZE INFORMACJE

- `Resources/Raw` - dane tekstowe/binarne dołączone do aplikacji.
- `Resources/Images` - obrazy (PNG, SVG).
- Oba są **tylko do odczytu** (część pakietu aplikacji).
- Odczyt: `FileSystem.OpenAppPackageFileAsync("nazwa")`.

Na co uważać:

`Resources/Raw` to idealne miejsce na **dane startowe** wczytywane przy pierwszym uruchomieniu. Jeśli aplikacja ma je później modyfikować, skopiuj je raz do `AppDataDirectory` i pracuj na kopii.

28.5. FileSystem.AppDataDirectory i budowanie ścieżek

`FileSystem.AppDataDirectory` zwraca ścieżkę do **prywatnego katalogu aplikacji**, w którym możemy bezpiecznie zapisywać pliki. Jest niewidoczny dla innych aplikacji i działa na każdej platformie. Ścieżkę do konkretnego pliku budujemy przez `Path.Combine`, które poprawnie łączy fragmenty.

PRZYKŁAD C#

```
// Bezpieczne zbudowanie ścieżki do pliku w katalogu aplikacji
string katalog = FileSystem.AppDataDirectory;
string sciezka = Path.Combine(katalog, "notatka.txt");
```

NAJWAŻNIEJSZE INFORMACJE

ELEMENT	ROLA
<code>FileSystem.AppDataDirectory</code>	prywatny katalog aplikacji (zapis/odczyt)
<code>FileSystem.CacheDirectory</code>	katalog na dane tymczasowe (cache)
<code>Path.Combine(a, b)</code>	bezpieczne łączenie fragmentów ścieżki

Na co uważać:

Zawsze buduj ścieżki przez `Path.Combine` i `AppDataDirectory`. Ręczne sklejanie ścieżek (`katalog + "\\ " + nazwa`) jest błędne - separatory różnią się między systemami (`/` vs `\`). `Path.Combine` dobiera właściwy separator automatycznie.

28.6. Ścieżki względne i bezwzględne

Ścieżka **bezwzględna** to pełna ścieżka od korzenia systemu (np. zwrócona przez `AppDataDirectory`). Ścieżka **względna** określa położenie względem innego katalogu. W MAUI do plików użytkownika używamy ścieżek bezwzględnych budowanych z `AppDataDirectory`; nie wpisujemy własnych ścieżek bezwzględnych „na sztywno”.

Na co uważać:

Nigdy nie wpisuj ścieżek typu `C:\Users\...` ani `/sdcard/...` - nie zadziałają wieloplatformowo i aplikacja może nie mieć do nich dostępu. Jedyne bezpieczne źródło ścieżki to katalogi z `FileSystem`.

28.7. Odczyt pliku tekstowego

Odczyt pliku to wczytanie jego zawartości do zmiennej `string`. Służy do tego asynchroniczna metoda `File.ReadAllTextAsync` (cała treść) lub `File.ReadAllLinesAsync` (tablica linii). Przed odczytem sprawdzamy, czy plik istnieje.

PRZYKŁAD C#

```
private async Task<string> WczytajTekst()
{
    string sciezka = Path.Combine(FileSystem.AppDataDirectory, "notatka.txt");

    if (!File.Exists(sciezka))
        return ""; // plik jeszcze nie istnieje

    return await File.ReadAllTextAsync(sciezka);
}

// Odczyt linia po linii
private async Task<string[]> WczytajLinie()
{
    string sciezka = Path.Combine(FileSystem.AppDataDirectory, "lista.txt");
    if (!File.Exists(sciezka)) return Array.Empty<string>();
    return await File.ReadAllLinesAsync(sciezka);
}
```

Na co uważać:

Zawsze sprawdzaj `File.Exists` przed odczytem - próba odczytu nieistniejącego pliku rzuca wyjątek. Operacje plikowe są asynchroniczne, więc wywołuj je z `await` w metodzie `async`.

28.8. Zapis pliku tekstowego

Zapis to umieszczenie tekstu w pliku. `File.WriteAllTextAsync` zapisuje treść, **nadpisując** istniejący plik (lub tworząc nowy). Jeśli katalog istnieje (a `AppDataDirectory` istnieje zawsze), zapis się powiedzie.

PRZYKŁAD C#

```
private async Task ZapiszTekst(string tresc)
{
    string sciezka = Path.Combine(FileSystem.AppDataDirectory, "notatka.txt");
    await File.WriteAllTextAsync(sciezka, tresc); // nadpisuje plik
}
```

Na co uważać:

`WriteAllTextAsync` nadpisuje cały plik. Jeśli chcesz dodać treść na koniec istniejącego pliku (a nie zastąpić), użyj `AppendAllTextAsync` (następny podrozdział).

28.9. Dopisywanie do pliku

Dopisywanie (`File.AppendAllTextAsync`) dodaje tekst **na koniec** istniejącego pliku, zachowując dotychczasową treść. Przydatne np. do logów lub dopisywania kolejnych wpisów.

PRZYKŁAD C#

```
private async Task DopiszWpis(string wpis)
{
    string sciezka = Path.Combine(FileSystem.AppDataDirectory, "log.txt");
    // Dodanie linii na koniec pliku (z nową linią)
    await File.AppendAllTextAsync(sciezka, wpis + Environment.NewLine);
}
```

Na co uważać:

`AppendAllTextAsync` tworzy plik, jeśli nie istnieje, więc nie trzeba go wcześniej tworzyć. Pamiętaj o dodaniu znaku nowej linii (`Environment.NewLine` lub `\n`), jeśli chcesz, by wpisy były w osobnych liniach.

28.10. Sprawdzanie istnienia i usuwanie pliku

`File.Exists(sciezka)` zwraca `true`, gdy plik istnieje. `File.Delete(sciezka)` usuwa plik. Przed usunięciem warto sprawdzić istnienie, by uniknąć wyjątku.

PRZYKŁAD C#

```
string sciezka = Path.Combine(FileSystem.AppDataDirectory, "notatka.txt");

// Sprawdzenie
bool istnieje = File.Exists(sciezka);

// Bezpieczne usunięcie
if (File.Exists(sciezka))
    File.Delete(sciezka);
```

Na co uważać:

`File.Delete` nieistniejącego pliku może rzucić wyjątek - dlatego poprzedzamy je sprawdzeniem `File.Exists`. Usunięcie jest nieodwracalne, więc przy danych użytkownika warto poprosić o potwierdzenie (`DisplayAlert`).

28.11. Odczyt danych z pliku „dane.txt” i parsowanie

Częsty scenariusz: aplikacja ma **plik z danymi startowymi** (np. `dane.txt` w `Resources/Raw`), który odczytuje przy uruchomieniu i **parsuje** na listę. Każda linia to jeden wpis, który dzielimy (`Split`) i konwertujemy.

PRZYKŁAD C#

```
// Plik "dane.txt" w Resources/Raw, linie w formacie "Nazwa;Cena"
private async Task<List<Produkt>> WczytajProdukty()
{
    var lista = new List<Produkt>();

    using var stream = await FileSystem.OpenAppPackageFileAsync("dane.txt");
    using var reader = new StreamReader(stream);

    string linia;
    while ((linia = await reader.ReadLineAsync()) != null)
    {
        if (string.IsNullOrEmpty(linia)) continue; // pomiń puste

        string[] czesci = linia.Split(';');
        if (czesci.Length == 2 && double.TryParse(czesci[1], out double cena))
        {
            lista.Add(new Produkt { Nazwa = czesci[0].Trim(), Cena = cena });
        }
    }
    return lista;
}
```

Na co uważać:

Przy parsowaniu **waliduj każdą linię**: sprawdź liczbę części po `Split` i użyj `TryParse` dla liczb. Pomijaj puste lub błędne linie (`continue`), zamiast pozwolić aplikacji na wyjątek. Dane startowe czytamy z `Resources/Raw` przez `OpenAppPackageFileAsync`.

28.12. Tworzenie listy obiektów na podstawie pliku

To rozwinięcie poprzedniego punktu: po wczytaniu i sparsowaniu pliku tworzymy **kolekcję obiektów** i wyświetlamy ją w `CollectionView`. Łączymy pracę z plikami, parsowanie i listy.

PRZYKŁAD C#

```
ObservableCollection<Produkt> produkty = new();

protected override async void OnAppearing()
{
    base.OnAppearing();
    if (produkty.Count == 0) // wczytaj tylko raz
    {
        var wczytane = await WczytajProdukty(); // metoda z 21.11
        foreach (var p in wczytane)
            produkty.Add(p);
        ListaProduktow.ItemsSource = produkty;
    }
}
```

Na co uważać:

Wczytuj dane raz (np. sprawdzając, czy kolekcja jest pusta), by nie duplikować ich przy każdym wejściu na stronę. Dane startowe z pliku to dobry punkt wyjścia dla aplikacji listowych.

28.13. Zapis wyniku działania aplikacji i dużego tekstu

Aplikacja może **zapisać wynik** swojej pracy do pliku - np. zaszyfrowany tekst, raport, eksport listy. Dla dużego tekstu używamy tej samej metody `WriteAllTextAsync`; dla listy obiektów składamy tekst z elementów lub serializujemy do JSON.

PRZYKŁAD C#

```
// Zapis listy notatek do pliku (każda w osobnej linii)
private async Task EksportujNotatki(IEnumerable<string> notatki)
{
    string sciezka = Path.Combine(FileSystem.AppDataDirectory, "eksport.txt");
    string tresc = string.Join(Environment.NewLine, notatki);
    await File.WriteAllTextAsync(sciezka, tresc);
}

// Zapis dużego tekstu (np. wynik przetwarzania)
private async Task ZapiszWynik(string duzyTekst)
{
    string sciezka = Path.Combine(FileSystem.AppDataDirectory, "wynik.txt");
    await File.WriteAllTextAsync(sciezka, duzyTekst);
}
```

Na co uważać:

`string.Join` z `Environment.NewLine` to wygodny sposób zapisania kolekcji do pliku linia po linii. Dla danych strukturalnych (obiekty) lepszy jest format JSON - wtedy łatwo je z powrotem odczytać.

28.14. Obsługa błędów przy pracy z plikami

Operacje na plikach mogą się nie udać - brak pliku, brak uprawnień, błąd zapisu. Dlatego otaczamy je blokiem `try/catch`, by aplikacja nie „wysypała się”, a użytkownik dostał czytelny komunikat.

PRZYKŁAD C#

```
private async Task<string> BezpiecznyOdczyt(string nazwaPliku)
{
    try
    {
        string sciezka = Path.Combine(FileSystem.AppDataDirectory, nazwaPliku);
        if (!File.Exists(sciezka)) return "";
        return await File.ReadAllTextAsync(sciezka);
    }
    catch (Exception ex)
    {
        await DisplayAlert("Błąd", $"Nie udało się odczytać pliku: {ex.Message}", "OK");
        return "";
    }
}
```

Na co uważać:

Zawsze otaczaj operacje plikowe `try/catch`. Najczęstsze wyjątki to brak pliku (`FileNotFoundException`) i brak uprawnień (`UnauthorizedAccessException`). Informuj użytkownika, zamiast pozostawiać go z zamkniętą aplikacją.

PRZYKŁAD C#

```
private async void OnWybierzPlik(object sender, EventArgs e)
{
    try
    {
        var plik = await FilePicker.PickAsync(new PickOptions
        {
            PickerTitle = "Wybierz plik tekstowy"
        });

        if (plik == null) return; // anulowano

        using var stream = await plik.OpenReadAsync();
        using var reader = new StreamReader(stream);
        Podglad.Text = await reader.ReadToEndAsync();
    }
    catch (Exception ex)
    {
        await DisplayAlert("Błąd", ex.Message, "OK");
    }
}
```

Na co uważać:

Zawsze obsługuj przypadek anulowania (`plik == null`). Operację otocz `try/catch` . Plik wybrany przez użytkownika odczytujemy ze strumienia (`OpenReadAsync`), bo może znajdować się poza katalogiem aplikacji.

28.15. Wybór wielu plików i filtrowanie typów

PRZYKŁAD C#

```
// Wybór wielu plików
var pliki = await FilePicker.PickMultipleAsync();
if (pliki != null)
{
    foreach (var p in pliki)
        Lista.Add(p.FileName);
}

// Filtrowanie typów (tylko obrazy)
var opcje = new PickOptions
{
    PickerTitle = "Wybierz obraz",
    FileTypes = FilePickerFileType.Images
};
var obraz = await FilePicker.PickAsync(opcje);
```

Na co uważać:

PRZYKŁAD C#

```
// Zrobienie zdjęcia aparatem
private async void OnZrobZdjecie(object sender, EventArgs e)
```

```

{
    try
    {
        if (!MediaPicker.Default.IsCaptureSupported)
        {
            await DisplayAlert("Uwaga", "Aparat niedostępny", "OK");
            return;
        }

        FileResult zdjecie = await MediaPicker.Default.CapturePhotoAsync();
        if (zdjecie != null)
            PodgladZdjecia.Source = ImageSource.FromFile(zdjecie.FullPath);
    }
    catch (Exception ex)
    {
        await DisplayAlert("Błąd", ex.Message, "OK");
    }
}

// Wybór zdjęcia z galerii
private async void OnWybierzZdjecie(object sender, EventArgs e)
{
    FileResult zdjecie = await MediaPicker.Default.PickPhotoAsync();
    if (zdjecie != null)
        PodgladZdjecia.Source = ImageSource.FromFile(zdjecie.FullPath);
}

```

Na co uważać:

Sprawdź `IsCaptureSupported` przed użyciem aparatu. Wynik (`FileResult`) może być `null` przy anulowaniu. Zdjęcie wyświetlamy przez `ImageSource.FromFile(zdjecie.FullPath)`. Pamiętaj o uprawnieniach do aparatu i galerii.

28.16. Uprawnienia związane z plikami i multimediami

Dostęp do aparatu, galerii czy niektórych lokalizacji wymaga **uprawnień**, które trzeba zadeklarować (w plikach platformowych) i czasem poprosić o nie w czasie działania. MAUI udostępnia do tego API `Permissions`.

NAJWAŻNIEJSZE INFORMACJE

FUNKCJA	UPRAWNIENIE
Aparat	<code>Permissions.Camera</code>
Galeria/zdjęcia	<code>Permissions.Photos</code> / <code>StorageRead</code>
Zapis w pamięci	<code>Permissions.StorageWrite</code> (starsze Androidy)

Na co uważać:

Prywatny katalog aplikacji (`AppDataDirectory`) **nie wymaga** specjalnych uprawnień - to Twoje główne miejsce zapisu. Uprawnienia są potrzebne dopiero przy aparacie, galerii czy dostępie do współdzielonej pamięci. Zawsze obsłuż odmowę uprawnienia.

28.17. Typowe problemy ze ścieżkami i dostępem do plików

NAJWAŻNIEJSZE INFORMACJE

PROBLEM	PRZYCZYNA	ROZWIĄZANIE
Plik niezaleziony	zła ścieżka / brak pliku	<code>Path.Combine</code> + <code>File.Exists</code>
„Nie mogę zapisać do <code>Resources/Raw</code> ”	zasób tylko do odczytu	zapisuj w <code>AppDataDirectory</code>
Ścieżka działa na Windows, nie na Androidzie	sztynna ścieżka	używaj <code>FileSystem</code>
Wyjątek przy odczycie	brak <code>try/catch</code>	otocz operację <code>try/catch</code>
Zasób czytany jak plik	zły sposób odczytu	<code>OpenAppPackageFileAsync</code>
Brak uprawnień (aparat/galeria)	nieobsłużone uprawnienie	poproś i obsłuż odmowę

Na co uważać:

Trzy najważniejsze zasady pracy z plikami: (1) **buduj ścieżki** przez `Path.Combine` + `AppDataDirectory`, (2) **sprawdzaj istnienie** przed odczytem/usunięciem i otaczaj operacje `try/catch`, (3) pamiętaj o różnicy: **zasoby** są tylko do odczytu (czytane przez `OpenAppPackageFileAsync`), a **pliki użytkownika** zapisujesz w `AppDataDirectory`.

Najczęstsze nieporozumienie początkujących: próba zapisu do `Resources/Raw` lub `Resources/Images`. Te foldery są **tylko do odczytu** (część pakietu aplikacji). Wszystko, co aplikacja tworzy lub zmienia, zapisuj w `FileSystem.AppDataDirectory`.

28.18. Receptury plików, obrazów, danych i API

Ten dział zawiera kompletne, gotowe do użycia receptury dla najczęstszych operacji na plikach, obrazach, danych lokalnych i zdalnych API w aplikacjach .NET MAUI. Każdy przykład zawiera pełny kod XAML oraz C# code-behind z komentarzami.

28.19. Zapis pliku tekstowego

Zapisujemy tekst do pliku w katalogu danych aplikacji (`FileSystem.AppDataDirectory`). Ten katalog jest prywatny dla aplikacji i nie wymaga dodatkowych uprawnień.

```
<?xml version="1.0" encoding="utf-8" ?>
<ContentPage xmlns="http://schemas.microsoft.com/dotnet/2021/maui"
              xmlns:x="http://schemas.microsoft.com/winfx/2009/xaml"
              x:Class="MojaAplikacja.ZapiszPlikPage"
              Title="Zapis pliku">
  <VerticalStackLayout Padding="20" Spacing="15">
    <Label Text="Wpisz tekst do zapisania:" FontSize="18"/>
    <Editor x:Name="EditorTresc" HeightRequest="150" Placeholder="Treść pliku..." />
    <Entry x:Name="EntryNazwa" Placeholder="Nazwa pliku (np. notatka.txt)" />
    <Button Text="Zapisz plik" Clicked="OnZapiszClicked" />
    <Label x:Name="LabelStatus" TextColor="Green" />
  </VerticalStackLayout>
</ContentPage>
```

```
using System.IO;

namespace MojaAplikacja;

public partial class ZapiszPlikPage : ContentPage
```

```

{
    public ZapiszPlikPage()
    {
        InitializeComponent();
    }

    private async void OnZapiszClicked(object sender, EventArgs e)
    {
        // Pobieramy nazwę pliku z kontrolki
        string nazwaPliku = EntryNazwa.Text?.Trim();
        if (string.IsNullOrEmpty(nazwaPliku))
        {
            LabelStatus.Text = "Podaj nazwę pliku!";
            LabelStatus.TextColor = Colors.Red;
            return;
        }

        // Budujemy pełną ścieżkę w katalogu danych aplikacji
        string sciezka = Path.Combine(FileSystem.AppDataDirectory, nazwaPliku);

        // Zapisujemy tekst asynchronicznie
        await File.WriteAllTextAsync(sciezka, EditorTresc.Text ?? "");

        LabelStatus.TextColor = Colors.Green;
        LabelStatus.Text = $"Zapisano: {sciezka}";
    }
}

```

28.20. Odczyt pliku tekstowego

Przed odczytem sprawdzamy, czy plik istnieje (`File.Exists`), aby uniknąć wyjątku.

```

<?xml version="1.0" encoding="utf-8" ?>
<ContentPage xmlns="http://schemas.microsoft.com/dotnet/2021/maui"
    xmlns:x="http://schemas.microsoft.com/winfx/2009/xaml"
    x:Class="MojaAplikacja.OdczytPlikPage"
    Title="Odczyt pliku">
    <VerticalStackLayout Padding="20" Spacing="15">
        <Entry x:Name="EntryNazwa" Placeholder="Nazwa pliku do odczytu"/>
        <Button Text="Odczytaj plik" Clicked="OnOdczytajClicked"/>
        <Label x:Name="LabelZawartosc" FontSize="14"/>
        <Label x:Name="LabelBład" TextColor="Red" />
    </VerticalStackLayout>
</ContentPage>

```

```

using System.IO;

namespace MojaAplikacja;

public partial class OdczytPlikPage : ContentPage

```

```

{
    public OdczytPlikPage()
    {
        InitializeComponent();
    }

    private async void OnOdczytajClick(object sender, EventArgs e)
    {
        string nazwaPliku = EntryNazwa.Text?.Trim();
        if (string.IsNullOrEmpty(nazwaPliku))
        {
            LabelBlad.Text = "Podaj nazwę pliku!";
            return;
        }

        // Pełna ścieżka
        string sciezka = Path.Combine(FileSystem.AppDataDirectory, nazwaPliku);

        // Sprawdzamy istnienie pliku
        if (!File.Exists(sciezka))
        {
            LabelBlad.Text = "Plik nie istnieje!";
            LabelZawartosc.Text = "";
            return;
        }

        // Odczytujemy zawartość asynchronicznie
        string zawartosc = await File.ReadAllTextAsync(sciezka);

        LabelBlad.Text = "";
        LabelZawartosc.Text = zawartosc;
    }
}

```

28.21. Dopisywanie do pliku

Metoda `File.AppendAllTextAsync` dodaje tekst na końcu istniejącego pliku (lub tworzy nowy, jeśli nie istnieje).

```

<?xml version="1.0" encoding="utf-8" ?>
<ContentPage xmlns="http://schemas.microsoft.com/dotnet/2021/maui"
              xmlns:x="http://schemas.microsoft.com/winfx/2009/xaml"
              x:Class="MojaAplikacja.DopisywaniePage"
              Title="Dopisywanie do pliku">
    <VerticalStackLayout Padding="20" Spacing="15">
        <Entry x:Name="EntryNazwa" Placeholder="Nazwa pliku" Text="dziennik.txt" />
        <Entry x:Name="EntryLinia" Placeholder="Tekst do dopisania" />
        <Button Text="Dopisz linię" Clicked="OnDopiszClick" />
        <Button Text="Pokaż zawartość" Clicked="OnPokazClick" />
        <Label x:Name="LabelZawartosc" FontSize="12" />
    </VerticalStackLayout>
</ContentPage>

```



```

using System.IO;

namespace MojaAplikacja;

public partial class DopisywaniePage : ContentPage
{
    public DopisywaniePage()
    {
        InitializeComponent();
    }

    private async void OnDopiszClicked(object sender, EventArgs e)
    {
        string sciezka = Path.Combine(FileSystem.AppDataDirectory, EntryNazwa.Text);

        // Dopisujemy linię z nową linią na końcu
        string linia = EntryLinia.Text + Environment.NewLine;
        await File.AppendAllTextAsync(sciezka, linia);

        EntryLinia.Text = "";
        await DisplayAlert("OK", "Dopisano linię do pliku.", "Zamknij");
    }

    private async void OnPokazClicked(object sender, EventArgs e)
    {
        string sciezka = Path.Combine(FileSystem.AppDataDirectory, EntryNazwa.Text);

        if (File.Exists(sciezka))
        {
            string zawartosc = await File.ReadAllTextAsync(sciezka);
            LabelZawartosc.Text = zawartosc;
        }
        else
        {
            LabelZawartosc.Text = "(plik nie istnieje)";
        }
    }
}

```

28.22. Usuwanie pliku

Usuujemy plik po potwierdzeniu przez użytkownika. Sprawdzamy najpierw, czy plik istnieje.

```

<?xml version="1.0" encoding="utf-8" ?>
<ContentPage xmlns="http://schemas.microsoft.com/dotnet/2021/maui"
    xmlns:x="http://schemas.microsoft.com/winfx/2009/xaml"
    x:Class="MojaAplikacja.UsunPlikPage"
    Title="Usuwanie pliku">
    <VerticalStackLayout Padding="20" Spacing="15">
        <Entry x:Name="EntryNazwa" Placeholder="Nazwa pliku do usunięcia"/>
        <Button Text="Usuń plik" Clicked="OnUsunClicked" BackgroundColor="Red" TextColor="White"/>
    </VerticalStackLayout>
</ContentPage>

```

```

        <Label x:Name="LabelStatus" />
    </VerticalStackLayout>
</ContentPage>

```

```

using System.IO;

namespace MojaAplikacja;

public partial class UsunPlikPage : ContentPage
{
    public UsunPlikPage()
    {
        InitializeComponent();
    }

    private async void OnUsunClicked(object sender, EventArgs e)
    {
        string nazwaPliku = EntryNazwa.Text?.Trim();
        if (string.IsNullOrEmpty(nazwaPliku))
        {
            LabelStatus.Text = "Podaj nazwę pliku!";
            return;
        }

        string sciezka = Path.Combine(FileSystem.AppDataDirectory, nazwaPliku);

        // Sprawdzamy czy plik istnieje
        if (!File.Exists(sciezka))
        {
            LabelStatus.Text = "Plik nie istnieje.";
            return;
        }

        // Potwierdzenie od użytkownika
        bool potwierdz = await DisplayAlert("Potwierdzenie",
            $"Czy na pewno usunąć plik '{nazwaPliku}'?", "Tak", "Nie");

        if (potwierdz)
        {
            // Usuwamy plik
            File.Delete(sciezka);
            LabelStatus.Text = $"Plik '{nazwaPliku}' został usunięty.";
        }
    }
}

```

28.23. Odczyt zasobu z Resources/Raw

Pliki umieszczone w folderze `Resources/Raw` projektu (z Build Action = MauiAsset) można odczytać przez `FileSystem.OpenAppPackageFileAsync`.

```

<?xml version="1.0" encoding="utf-8" ?>
<ContentPage xmlns="http://schemas.microsoft.com/dotnet/2021/maui"
              xmlns:x="http://schemas.microsoft.com/winfx/2009/xaml"
              x:Class="MojaAplikacja.ZasobRawPage"
              Title="Odczyt zasobu Raw">
    <VerticalStackLayout Padding="20" Spacing="15">
        <Label Text="Zawartość pliku dane.txt z Resources/Raw:" FontSize="16" FontAttributes="Bold"/>
        <Button Text="Wczytaj zasób" Clicked="OnWczytajClicked"/>
        <Label x:Name="LabelZawartosc" FontSize="14"/>
    </VerticalStackLayout>
</ContentPage>

```

```

using System.IO;

namespace MojaAplikacja;

public partial class ZasobRawPage : ContentPage
{
    public ZasobRawPage()
    {
        InitializeComponent();
    }

    private async void OnWczytajClicked(object sender, EventArgs e)
    {
        try
        {
            // Otwieramy strumień do pliku w Resources/Raw
            using Stream stream = await FileSystem.OpenAppPackageFileAsync("dane.txt");

            // Czytamy zawartość strumienia
            using StreamReader reader = new StreamReader(stream);
            string zawartosc = await reader.ReadToEndAsync();

            LabelZawartosc.Text = zawartosc;
        }
        catch (FileNotFoundException)
        {
            LabelZawartosc.Text = "Nie znaleziono pliku dane.txt w zasobach.";
        }
    }
}

```

28.24. Parsowanie linii na listę obiektów

Wczytujemy plik CSV/tekstowy z liniami w formacie `nazwa;cena;ilość` i parsujemy je na obiekty.

```

<?xml version="1.0" encoding="utf-8" ?>
<ContentPage xmlns="http://schemas.microsoft.com/dotnet/2021/maui"

```

```

        xmlns:x="http://schemas.microsoft.com/winfx/2009/xaml"
        x:Class="MojaAplikacja.ParsowaniePage"
        Title="Parsowanie danych">
<VerticalStackLayout Padding="20" Spacing="10">
    <Button Text="Wczytaj i parsuj produkty.txt" Clicked="OnParsujClicked"/>
    <CollectionView x:Name="ListaProdukty">
        <CollectionView.ItemTemplate>
            <DataTemplate>
                <HorizontalStackLayout Spacing="10" Padding="5">
                    <Label Text="{Binding Nazwa}" FontAttributes="Bold" WidthRequest="120"/>
                    <Label Text="{Binding Cena, StringFormat='{0:F2} zł}'"/>
                    <Label Text="{Binding Ilosc, StringFormat='szt: {0}'}"/>
                </HorizontalStackLayout>
            </DataTemplate>
        </CollectionView.ItemTemplate>
    </CollectionView>
</VerticalStackLayout>
</ContentPage>

```

```

using System.IO;

namespace MojaAplikacja;

// Model danych
public class Produkt
{
    public string Nazwa { get; set; }
    public double Cena { get; set; }
    public int Ilosc { get; set; }
}

public partial class ParsowaniePage : ContentPage
{
    public ParsowaniePage()
    {
        InitializeComponent();
    }

    private async void OnParsujClicked(object sender, EventArgs e)
    {
        // Ścieżka do pliku w danych aplikacji
        string sciezka = Path.Combine(FileSystem.AppDataDirectory, "produkty.txt");

        if (!File.Exists(sciezka))
        {
            await DisplayAlert("Błąd", "Plik produkty.txt nie istnieje.", "OK");
            return;
        }

        // Odczytujemy wszystkie linie
        string zawartosc = await File.ReadAllTextAsync(sciezka);
        string[] linie = zawartosc.Split(Environment.NewLine, StringSplitOptions.RemoveEmptyEntries);
    }
}

```

```

List<Produkt> produkty = new List<Produkt>();

foreach (string linia in linie)
{
    // Format: nazwa;cena;ilość
    string[] czesci = linia.Split(';');

    if (czesci.Length ≥ 3)
    {
        // Parsujemy cenę i ilość z walidacją
        if (double.TryParse(czesci[1], out double cena) &&
            int.TryParse(czesci[2], out int ilosc))
        {
            produkty.Add(new Produkt
            {
                Nazwa = czesci[0].Trim(),
                Cena = cena,
                Ilosc = ilosc
            });
        }
    }
}

// Wyświetlamy w CollectionView
ListaProdukty.ItemsSource = produkty;
}
}

```

Użytkownik wybiera pojedynczy plik z systemu. Wynik zawiera ścieżkę i nazwę.

```

<?xml version="1.0" encoding="utf-8" ?>
<ContentPage xmlns="http://schemas.microsoft.com/dotnet/2021/maui"
    xmlns:x="http://schemas.microsoft.com/winfx/2009/xaml"
    x:Class="MojaAplikacja.FilePickerJedenPage"
    Title="FilePicker - jeden plik">
    <VerticalStackLayout Padding="20" Spacing="15">
        <Button Text="Wybierz plik" Clicked="OnWybierzClicked"/>
        <Label x:Name="LabelNazwa" FontSize="16"/>
        <Label x:Name="LabelSciezka" FontSize="12" TextColor="Gray"/>
        <Label x:Name="LabelZawartosc" FontSize="14"/>
    </VerticalStackLayout>
</ContentPage>

```

```

using System.IO;

namespace MojaAplikacja;

public partial class FilePickerJedenPage : ContentPage

```

```

{
    public FilePickerJedenPage()
    {
        InitializeComponent();
    }

    private async void OnWybierzClicked(object sender, EventArgs e)
    {
        try
        {
            // Otwieramy systemowy dialog wyboru pliku
            var wynik = await FilePicker.Default.PickAsync();

            if (wynik != null)
            {
                // Wyświetlamy informacje o pliku
                LabelNazwa.Text = $"Nazwa: {wynik.FileName}";
                LabelSieczka.Text = $"Ścieżka: {wynik.FullPath}";

                // Odczytujemy zawartość wybranego pliku
                using var stream = await wynik.OpenReadAsync();
                using var reader = new StreamReader(stream);
                string zawartosc = await reader.ReadToEndAsync();

                LabelZawartosc.Text = zawartosc.Length > 500
                    ? zawartosc.Substring(0, 500) + "... "
                    : zawartosc;
            }
        }
        catch (Exception ex)
        {
            await DisplayAlert("Błąd", ex.Message, "OK");
        }
    }
}

```

Wybieramy wiele plików jednocześnie, ograniczając do konkretnych typów (np. tylko obrazy).

```

<?xml version="1.0" encoding="utf-8" ?>
<ContentPage xmlns="http://schemas.microsoft.com/dotnet/2021/maui"
    xmlns:x="http://schemas.microsoft.com/winfx/2009/xaml"
    x:Class="MojaAplikacja.FilePickerWielePage"
    Title="FilePicker - wiele plików">
    <VerticalStackLayout Padding="20" Spacing="15">
        <Button Text="Wybierz pliki tekstowe" Clicked="OnWybierzTxtClicked"/>
        <Button Text="Wybierz obrazy" Clicked="OnWybierzObrazyClicked"/>
        <Label x:Name="LabelIlosc" FontSize="16" FontAttributes="Bold"/>
        <CollectionView x:Name="ListaPliki">
            <CollectionView.ItemTemplate>
                <DataTemplate>
                    <Label Text="{Binding}" Padding="5" FontSize="13"/>
                </DataTemplate>
            </CollectionView.ItemTemplate>
        </CollectionView>
    </VerticalStackLayout>
</ContentPage>

```

```
</DataTemplate>
</CollectionView.ItemTemplate>
</CollectionView>
</VerticalStackLayout>
</ContentPage>
```

```
namespace MojaAplikacja;

public partial class FilePickerWielePage : ContentPage
{
    public FilePickerWielePage()
    {
        InitializeComponent();
    }

    private async void OnWybierzTxtClicked(object sender, EventArgs e)
    {
        // Filtr dla plików tekstowych
        var opcje = new PickOptions
        {
            PickerTitle = "Wybierz pliki tekstowe",
            FileTypes = new FilePickerFileType(new Dictionary<DevicePlatform, IEnumerable<string>>
            {
                { DevicePlatform.WinUI, new[] { ".txt", ".csv", ".log" } },
                { DevicePlatform.Android, new[] { "text/plain", "text/csv" } },
                { DevicePlatform.iOS, new[] { "public.plain-text" } }
            })
        };

        await WybierzWielePlików(opcje);
    }

    private async void OnWybierzObrazyClicked(object sender, EventArgs e)
    {
        // Filtr dla obrazów
        var opcje = new PickOptions
        {
            PickerTitle = "Wybierz obrazy",
            FileTypes = FilePickerFileType.Images // Wbudowany filtr
        };

        await WybierzWielePlików(opcje);
    }

    private async Task WybierzWielePlików(PickOptions opcje)
    {
        try
        {
            // PickMultipleAsync pozwala wybrać wiele plików
            var wyniki = await FilePicker.Default.PickMultipleAsync(opcje);

            if (wyniki != null && wyniki.Any())
```

```

        {
            var nazwy = wyniki.Select(f => $"{f.FileName} ({f.FullPath})").ToList();
            LabelIlosc.Text = $"Wybrano {nazwy.Count} plików:";
            ListaPliki.ItemsSource = nazwy;
        }
    }
    catch (Exception ex)
    {
        await DisplayAlert("Błąd", ex.Message, "OK");
    }
}
}

```

Robimy zdjęcie aparatem urządzenia i wyświetlamy je w aplikacji.

```

<?xml version="1.0" encoding="utf-8" ?>
<ContentPage xmlns="http://schemas.microsoft.com/dotnet/2021/maui"
    xmlns:x="http://schemas.microsoft.com/winfx/2009/xaml"
    x:Class="MojaAplikacja.MediaPickerAparatPage"
    Title="Zdjęcie z aparatu">
    <VerticalStackLayout Padding="20" Spacing="15">
        <Button Text="Zrób zdjęcie" Clicked="OnZrobZdjecieClicked" />
        <Image x:Name="ImageZdjecie" HeightRequest="300" Aspect="AspectFit" />
        <Label x:Name="LabelSciezka" FontSize="12" TextColor="Gray" />
    </VerticalStackLayout>
</ContentPage>

```

```

namespace MojaAplikacja;

public partial class MediaPickerAparatPage : ContentPage
{
    public MediaPickerAparatPage()
    {
        InitializeComponent();
    }

    private async void OnZrobZdjecieClicked(object sender, EventArgs e)
    {
        try
        {
            // Sprawdzamy czy aparat jest dostępny
            if (!MediaPicker.Default.IsCaptureSupported)
            {
                await DisplayAlert("Błąd", "Aparat nie jest dostępny na tym urządzeniu.", "OK");
                return;
            }

            // Robimy zdjęcie
            var zdjecie = await MediaPicker.Default.CapturePhotoAsync();

```



```

        if (zdjecie != null)
        {
            // Wyświetlamy ścieżkę
            LabelSciezka.Text = zdjecie.FullPath;

            // Ładujemy zdjęcie do kontrolki Image
            var stream = await zdjecie.OpenReadAsync();
            ImageZdjecie.Source = ImageSource.FromStream(() => stream);
        }
    }
    catch (PermissionException)
    {
        await DisplayAlert("Brak uprawnień", "Przyznaj uprawnienia do aparatu.", "OK");
    }
    catch (Exception ex)
    {
        await DisplayAlert("Błąd", ex.Message, "OK");
    }
}
}

```

Wybieramy istniejące zdjęcie z galerii urządzenia.

```

<?xml version="1.0" encoding="utf-8" ?>
<ContentPage xmlns="http://schemas.microsoft.com/dotnet/2021/maui"
              xmlns:x="http://schemas.microsoft.com/winfx/2009/xaml"
              x:Class="MojaAplikacja.MediaPickerGaleriaPage"
              Title="Wybór z galerii">
    <VerticalStackLayout Padding="20" Spacing="15">
        <Button Text="Wybierz zdjęcie z galerii" Clicked="OnWybierzClicked"/>
        <Button Text="Wybierz wideo z galerii" Clicked="OnWybierzWideoClicked"/>
        <Image x:Name="ImageWybrane" HeightRequest="300" Aspect="AspectFit"/>
        <Label x:Name="LabelInfo" FontSize="12"/>
    </VerticalStackLayout>
</ContentPage>

```

```

namespace MojaAplikacja;

public partial class MediaPickerGaleriaPage : ContentPage
{
    public MediaPickerGaleriaPage()
    {
        InitializeComponent();
    }

    private async void OnWybierzClicked(object sender, EventArgs e)
    {
        try

```

```

{
    // Wybieramy zdjęcie z galerii
    var zdjecie = await MediaPicker.Default.PickPhotoAsync();

    if (zdjecie != null)
    {
        // Wyświetlamy informacje
        LabelInfo.Text = $"Plik: {zdjecie.FileName}\nŚcieżka: {zdjecie.FullPath}";

        // Ustawiamy źródło obrazu
        var stream = await zdjecie.OpenReadAsync();
        ImageWybrane.Source = ImageSource.FromStream(() => stream);
    }
}
catch (Exception ex)
{
    await DisplayAlert("Błąd", ex.Message, "OK");
}
}

private async void OnWybierzWideoClicked(object sender, EventArgs e)
{
    try
    {
        // Wybieramy wideo z galerii
        var wideo = await MediaPicker.Default.PickVideoAsync();

        if (wideo != null)
        {
            LabelInfo.Text = $"Wideo: {wideo.FileName}\nŚcieżka: {wideo.FullPath}";
        }
    }
    catch (Exception ex)
    {
        await DisplayAlert("Błąd", ex.Message, "OK");
    }
}
}
}

```

29. Preferences i ustawienia aplikacji

Często chcemy zapamiętać **drobne ustawienia** użytkownika: wybrany motyw, ostatni login, rozmiar czcionki czy stan przełącznika. Do takich małych danych typu klucz-wartość idealnie nadaje się mechanizm **Preferences** - najprostszy sposób trwałego przechowywania w MAUI. Ten rozdział omawia, czym są preferencje, jak je zapisywać i odczytywać oraz pokazuje praktyczne zastosowania.

29.1. Czym są Preferences

Preferences to wbudowany magazyn **par klucz-wartość** prostych typów (**string**, **int**, **bool**, **double**, **DateTime**). Działa jak słownik, który przetrwa zamknięcie aplikacji. Zapis to **Preferences.Set(klucz, wartosc)**, odczyt - **Preferences.Get(klucz, domyslna)**.

Preferences służy do **zapamiętywania drobnych ustawień**: motywu, loginu, flag, ostatnich wyborów. To najszybszy sposób na trwałe przechowanie niewielkiej informacji bez bazy ani plików.

NAJWAŻNIEJSZE INFORMACJE

METODA	DZIAŁANIE
<code>Preferences.Set("klucz", wartosc)</code>	zapis wartości
<code>Preferences.Get("klucz", domyslna)</code>	odczyt z wartością domyślną
<code>Preferences.ContainsKey("klucz")</code>	sprawdzenie istnienia
<code>Preferences.Remove("klucz")</code>	usunięcie klucza
<code>Preferences.Clear()</code>	wyczyszczenie wszystkich

Na co uważać:

`Preferences` przechowuje dane **jawnie** - nie używaj go do haseł czy tokenów. Do danych wrażliwych służy `SecureStorage`, które je szyfruje.

29.2. Kiedy używać Preferences

Kiedy używać?

- Drobne ustawienia: motyw, język, rozmiar czcionki.
- Zapamiętanie ostatniego loginu lub wyboru.
- Flagi: „pierwsze uruchomienie”, „pokazano samouczek”.
- Stan przełącznika (powiadomienia włączone/wyłączone).

Kiedy nie używać?

- Duże lub złożone dane → użyj **plików** lub **SQLite**.
- Dane wrażliwe (hasła, tokeny) → użyj **SecureStorage**.
- Listy obiektów → użyj **SQLite** (lub JSON w pliku).

Na co uważać:

`Preferences` jest do **małych** danych. Próba przechowywania w nim dużych struktur (np. całej listy) jest złą praktyką - od tego są pliki i baza.

29.3. Zapis i odczyt różnych typów

`Preferences` obsługuje proste typy. Przy odczycie podajemy **wartość domyślną** zwracaną, gdy klucz jeszcze nie istnieje - to elegancko obsługuje pierwsze uruchomienie.

PRZYKŁAD C#

```
// Zapis różnych typów
Preferences.Set("login", "anna");
Preferences.Set("rozmiar_czcionki", 18);
Preferences.Set("tryb_ciemny", true);
Preferences.Set("ostatnie_logowanie", DateTime.Now);

// Odczyt z wartością domyślną (gdy klucz nie istnieje)
string login = Preferences.Get("login", "");
int rozmiar = Preferences.Get("rozmiar_czcionki", 14);
bool ciemny = Preferences.Get("tryb_ciemny", false);
DateTime ostatnie = Preferences.Get("ostatnie_logowanie", DateTime.MinValue);
```

Na co uważać:

Zawsze podawaj sensowną **wartość domyślną** przy odczycie - to ona zostanie użyta przy pierwszym uruchomieniu, gdy klucza jeszcze nie ma. Typ wartości przy odczycie musi pasować do typu przy zapisie.

29.4. Sprawdzanie, usuwanie i czyszczenie

Możemy sprawdzić, czy klucz istnieje (`ContainsKey`), usunąć pojedynczy klucz (`Remove`) lub wyczyścić wszystkie ustawienia (`Clear`).

PRZYKŁAD C#

```
if (Preferences.ContainsKey("login"))
{
    string login = Preferences.Get("login", "");
}

Preferences.Remove("login"); // usuń jeden klucz
Preferences.Clear();         // usuń wszystkie ustawienia
```

Na co uważać:

`Clear()` usuwa **wszystkie** zapisane ustawienia - używaj go ostrożnie (np. przy wylogowaniu lub resecie aplikacji). Do usunięcia jednej wartości użyj `Remove`.

29.5. Przykład: zapamiętanie loginu

PRZYKŁAD C#

```
// Przy starcie - podpowiedz ostatni login
public LogowaniePage()
{
    InitializeComponent();
    PoleLogin.Text = Preferences.Get("ostatni_login", "");
}

// Po udanym logowaniu - zapamiętaj login
private void OnZaloguj(object sender, EventArgs e)
{
    // ...weryfikacja...
    Preferences.Set("ostatni_login", PoleLogin.Text);
}
```

Na co uważać:

Zapamiętuj **login**, ale **nigdy hasło** w `Preferences`. Dla danych logowania (tokeny) używaj `SecureStorage`.

29.6. Przykład: zapamiętanie motywu i rozmiaru czcionki

PRZYKŁAD C#

```
// Motyw - odczyt przy starcie i zapis po zmianie
public App()
{
    InitializeComponent();

    bool ciemny = Preferences.Get("tryb_ciemny", false);
    UserAppTheme = ciemny ? AppTheme.Dark : AppTheme.Light;
    MainPage = new AppShell();
}

private void OnZmienMotyw(object sender, ToggledEventArgs e)
```

```

{
    Application.Current.UserAppTheme = e.Value ? AppTheme.Dark : AppTheme.Light;
    Preferences.Set("tryb_ciemny", e.Value); // zapamiętaj wybór
}

```

```

// Rozmiar czcionki - przywrócenie i zapis
int rozmiar = Preferences.Get("rozmiar", 16);
Podglad.FontSize = rozmiar;

private void OnRozmiar(object sender, ValueChangedEventArgs e)
{
    int r = (int)e.NewValue;
    Podglad.FontSize = r;
    Preferences.Set("rozmiar", r);
}

```

Na co uważać:

Ustawienia wpływające na cały wygląd (motyw) najlepiej odczytać **wcześniej** - np. w konstruktorze `App`. Dzięki temu aplikacja od razu startuje z zapamiętanym motywem.

29.7. Przykład: zapamiętanie ostatniej wartości i ustawień użytkownika

PRZYKŁAD C#

```

// Zapamiętanie ostatnio wpisanej wartości i preferencji
private void OnZapiszUstawienia(object sender, EventArgs e)
{
    Preferences.Set("powiadomienia", PrzełącznikPowiadomien.IsToggled);
    Preferences.Set("ostatnie_miasto", PoleMiasto.Text);
    Preferences.Set("liczba_wynikow", (int)SuwakWynikow.Value);
}

protected override void OnAppearing()
{
    base.OnAppearing();
    PrzełącznikPowiadomien.IsToggled = Preferences.Get("powiadomienia", true);
    PoleMiasto.Text = Preferences.Get("ostatnie_miasto", "");
    SuwakWynikow.Value = Preferences.Get("liczba_wynikow", 10);
}

```

Na co uważać:

Wczytaj zapamiętane ustawienia w `OnAppearing` lub w konstruktorze, a zapisuj po zmianie. To typowy wzorec „aplikacja pamięta moje preferencje”.

29.8. Typowe błędy

NAJWAŻNIEJSZE INFORMACJE

BŁĄD	SKUTEK	ROZWIĄZANIE
Hasło/token w <code>Preferences</code>	brak bezpieczeństwa	użyj <code>SecureStorage</code>
Brak wartości domyślnej	nieprzewidywalny wynik	podaj domyślną w <code>Get</code>
Niezgodny typ przy odczycie	błąd/zła wartość	czytaj tym samym typem co zapis
Duże dane w <code>Preferences</code>	zła praktyka	użyj plików/SQLite
<code>Clear</code> zamiast <code>Remove</code>	utrata wszystkich ustawień	usuwać pojedynczy klucz

Na co uważać:

`Preferences` to narzędzie do **małych** ustawień. Pamiętaj o wartościach domyślnych, zgodności typów i o tym, by nie przechowywać tam danych wrażliwych ani dużych struktur.

`Preferences` to najprostszy sposób, by aplikacja „pamiętała” wybory użytkownika między uruchomieniami. Zapisuj drobne ustawienia (motyw, login, rozmiar czcionki), odczytuj je przy starcie z sensowną wartością domyślną, a wrażliwe dane trzymaj w `SecureStorage`.

Gdy aplikacja przechowuje **dużo ustrukturyzowanych danych** (lista zadań, produktów, kontaktów), pliki tekstowe i `Preferences` przestają wystarczać. Z pomocą przychodzi **SQLite** – lekka, lokalna baza danych wbudowana w urządzenia mobilne. Ten rozdział tłumaczy od zera, czym jest baza danych, kiedy jej użyć, jak skonfigurować SQLite w MAUI i jak wykonywać operacje **CRUD** (dodawanie, odczyt, edycja, usuwanie).

29.9. Preferences - zapis i odczyt ustawień

`Preferences` przechowuje proste pary klucz-wartość (string, int, bool, double) trwale na urządzeniu.

```
<?xml version="1.0" encoding="utf-8" ?>
<ContentPage xmlns="http://schemas.microsoft.com/dotnet/2021/maui"
              xmlns:x="http://schemas.microsoft.com/winfx/2009/xaml"
              x:Class="MojaAplikacja.PreferencjePage"
              Title="Preferences">
  <VerticalStackLayout Padding="20" Spacing="15">
    <Label Text="Ustawienia użytkownika" FontSize="20" FontAttributes="Bold"/>

    <Label Text="Imię:"/>
    <Entry x:Name="EntryImie" Placeholder="Twoje imię"/>

    <Label Text="Wiek:"/>
    <Entry x:Name="EntryWiek" Placeholder="Wiek" Keyboard="Numeric"/>

    <HorizontalStackLayout Spacing="10">
      <Label Text="Powiadomienia:" VerticalOptions="Center"/>
      <Switch x:Name="SwitchPowiadomienia"/>
    </HorizontalStackLayout>

    <Button Text="Zapisz ustawienia" Clicked="OnZapiszClicked"/>
    <Button Text="Wczytaj ustawienia" Clicked="OnWczytajClicked"/>
    <Button Text="Wyczyść wszystko" Clicked="OnWyczyscClicked"/>

    <Label x:Name="LabelStatus" FontSize="14" TextColor="Green"/>
  </VerticalStackLayout>
</ContentPage>
```

```
</VerticalStackLayout>

</ContentPage>
```

```
namespace MojaAplikacja;

public partial class PreferencjePage : ContentPage
{
    public PreferencjePage()
    {
        InitializeComponent();
        // Wczytujemy ustawienia przy starcie strony
        WczytajUstawienia();
    }

    private void OnZapiszClick(object sender, EventArgs e)
    {
        // Zapisujemy string
        Preferences.Set("imie", EntryImie.Text ?? "");

        // Zapisujemy int
        if (int.TryParse(EntryWiek.Text, out int wiek))
            Preferences.Set("wiek", wiek);

        // Zapisujemy bool
        Preferences.Set("powiadomienia", SwitchPowiadomienia.IsToggled);

        LabelStatus.Text = "Ustawienia zapisane!";
    }

    private void OnWczytajClick(object sender, EventArgs e)
    {
        WczytajUstawienia();
        LabelStatus.Text = "Ustawienia wczytane.";
    }

    private void WczytajUstawienia()
    {
        // Odczytujemy z wartościami domyślnymi
        EntryImie.Text = Preferences.Get("imie", "");
        EntryWiek.Text = Preferences.Get("wiek", 0).ToString();
        SwitchPowiadomienia.IsToggled = Preferences.Get("powiadomienia", true);
    }

    private void OnWyczyscClick(object sender, EventArgs e)
    {
        // Usuwamy wszystkie zapisane preferencje
        Preferences.Clear();
        EntryImie.Text = "";
        EntryWiek.Text = "";
        SwitchPowiadomienia.IsToggled = false;
        LabelStatus.Text = "Ustawienia wyczyszczone.";
    }
}
```

```
}  
}
```

29.10. Preferences - zapamiętanie motywu

Aplikacja zapamiętuje wybrany motyw (jasny/ciemny) i przywraca go przy uruchomieniu.

```
<?xml version="1.0" encoding="utf-8" ?>  
<ContentPage xmlns="http://schemas.microsoft.com/dotnet/2021/maui"  
    xmlns:x="http://schemas.microsoft.com/winfx/2009/xaml"  
    x:Class="MojaAplikacja.MotywPage"  
    Title="Motyw aplikacji">  
    <VerticalStackLayout Padding="20" Spacing="20">  
        <Label Text="Wybierz motyw:" FontSize="20" FontAttributes="Bold"/>  
  
        <RadioButton x:Name="RadioJasny" Content="Jasny" GroupName="motyw"  
            CheckedChanged="OnMotywChanged" />  
        <RadioButton x:Name="RadioCiemny" Content="Ciemny" GroupName="motyw"  
            CheckedChanged="OnMotywChanged" />  
        <RadioButton x:Name="RadioSystemowy" Content="Systemowy" GroupName="motyw"  
            CheckedChanged="OnMotywChanged" />  
  
        <Label x:Name="LabelAktualny" FontSize="14" TextColor="Gray" />  
    </VerticalStackLayout>  
</ContentPage>
```

```
namespace MojaAplikacja;  
  
public partial class MotywPage : ContentPage  
{  
    public MotywPage()  
    {  
        InitializeComponent();  
        // Wczytujemy zapisany motyw przy starcie  
        WczytajMotyw();  
    }  
  
    private void OnMotywChanged(object sender, CheckedChangedEventArgs e)  
    {  
        if (!e.Value) return; // Ignorujemy odznaczenie  
  
        string motyw = "systemowy";  
  
        if (RadioJasny.IsChecked)  
        {  
            motyw = "jasny";  
            Application.Current.UserAppTheme = AppTheme.Light;  
        }  
        else if (RadioCiemny.IsChecked)  
        {
```



```

        motyw = "ciemny";
        Application.Current.UserAppTheme = AppTheme.Dark;
    }
    else
    {
        motyw = "systemowy";
        Application.Current.UserAppTheme = AppTheme.Unspecified;
    }

    // Zapisujemy wybór w Preferences
    Preferences.Set("motyw", motyw);
    LabelAktualny.Text = $"Zapisany motyw: {motyw}";
}

private void WczytajMotyw()
{
    // Odczytujemy zapisany motyw
    string motyw = Preferences.Get("motyw", "systemowy");

    switch (motyw)
    {
        case "jasny":
            RadioJasny.IsChecked = true;
            Application.Current.UserAppTheme = AppTheme.Light;
            break;
        case "ciemny":
            RadioCiemny.IsChecked = true;
            Application.Current.UserAppTheme = AppTheme.Dark;
            break;
        default:
            RadioSystemowy.IsChecked = true;
            Application.Current.UserAppTheme = AppTheme.Unspecified;
            break;
    }

    LabelAktualny.Text = $"Aktualny motyw: {motyw}";
}
}

```

30. SQLite i dane lokalne

30.1. Kiedy wystarczy plik, a kiedy potrzeba bazy

Do przechowywania danych mamy kilka narzędzi o różnej „pojemności”. **Plik** nadaje się do prostych, niewielkich danych tekstowych.

Preferences do drobnych ustawień (klucz-wartość). **SQLite** do **dużych zbiorów rekordów**, na których trzeba wyszukiwać, filtrować i sortować.

NAJWAŻNIEJSZE INFORMACJE

DANE	NARZĘDZIE
Drobne ustawienie (motyw, login)	Preferences
Mały tekst / eksport	plik tekstowy
Dużo rekordów (lista zadań, produktów)	SQLite
Dane wrażliwe (token)	SecureStorage

Na co uważać:

Nie używaj plików tekstowych do dużych, złożonych danych - szybko stają się trudne w obsłudze. Gdy potrzebujesz wyszukiwania, sortowania czy relacji między danymi, wybierz SQLite.

30.2. Czym jest SQLite

SQLite to lekka baza danych SQL działająca bez osobnego serwera - cała baza to jeden plik w katalogu aplikacji. Jest szybka, niezawodna i wbudowana w Android oraz iOS. Dane organizuje w **tabele** (zbiory rekordów), złożone z **rekordów** (wierszy) i **kolumn** (właściwości).

NAJWAŻNIEJSZE INFORMACJE

POJĘCIE	ZNACZENIE
Tabela	zbiór rekordów jednego typu
Rekord (wiersz)	pojedynczy obiekt
Kolumna	pojedyncza właściwość
Klucz główny	unikalny identyfikator rekordu (Id)
CRUD	Create, Read, Update, Delete

Na co uważać:

SQLite to **baza lokalna** - działa na urządzeniu, offline. To nie to samo co baza serwerowa (np. SQL Server). Idealnie nadaje się do danych aplikacji mobilnej, które mają być dostępne bez połączenia z siecią.

30.3. Instalacja pakietu i model danych

Aby używać SQLite w MAUI, instalujemy pakiet NuGet `sqlite-net-pcl`, który mapuje klasy C# na tabele. **Model danych** to zwykła klasa, w której **klucz główny** oznaczamy atrybutem `[PrimaryKey]`, a automatyczną numerację - `[AutoIncrement]`.

PRZYKŁAD (INSTALACJA)

```
dotnet add package sqlite-net-pcl
```

PRZYKŁAD C#

```
using SQLite;

public class Zadanie
{
    [PrimaryKey, AutoIncrement]
    public int Id { get; set; } // klucz główny, nadawany automatycznie

    public string Tytuł { get; set; }
    public bool Wykonane { get; set; }
    public DateTime DataUtworzenia { get; set; } = DateTime.Now;
}
```

Na co uważać:

Klucz z `[AutoIncrement]` nadaje **baza** przy dodawaniu - nie ustawiaj go ręcznie. Po wstawieniu obiektu jego `Id` zostanie automatycznie wypełnione. Każda publiczna właściwość staje się kolumną.

30.4. Klasa bazy danych i tworzenie tabeli

Dostęp do bazy zamykamy w **osobnej klasie**, która tworzy połączenie (`SQLiteAsyncConnection`) i tabelę, a także udostępnia metody CRUD. Połączenie inicjalizujemy **leniwie** (przy pierwszym użyciu), wskazując ścieżkę pliku bazy w `AppDataDirectory`.

PRZYKŁAD C#

```
using SQLite;

public class ZadaniaBaza
{
    SQLiteAsyncConnection db;

    async Task Init()
    {
        if (db != null) return; // już zainicjalizowane
        string sciezka = Path.Combine(FileSystem.AppDataDirectory, "zadania.db3");
        db = new SQLiteAsyncConnection(sciezka);
        await db.CreateTableAsync<Zadanie>(); // utworzenie tabeli (raz)
    }
}
```

Na co uważać:

Operacje SQLite są **asynchroniczne** (`await`), by nie blokować interfejsu. Ścieżkę bazy buduj przez `Path.Combine` + `AppDataDirectory`. `CreateTableAsync` jest bezpieczne do wielokrotnego wywołania - tworzy tabelę tylko, jeśli nie istnieje.

30.5. Dodawanie rekordu (Create)

Dodanie rekordu to wstawienie nowego obiektu do tabeli metodą `InsertAsync`. Baza automatycznie nada mu `Id`.

PRZYKŁAD C#

```
public async Task<int> Dodaj(Zadanie z)
{
    await Init();
```

```
return await db.InsertAsync(z); // zwraca liczbę dodanych rekordów
}
```

```
// Użycie
var zadanie = new Zadanie { Tytuł = "Kup mleko" };
await baza.Dodaj(zadanie);
// teraz zadanie.Id ma nadaną wartość
```

Na co uważać:

Po `InsertAsync` obiekt ma już nadane `Id`. Nie ustawiaj `Id` ręcznie przy dodawaniu - pozwól bazie to zrobić.

30.6. Odczyt rekordów (Read)

Odczyt to pobranie rekordów z tabeli. `ToListAsync` zwraca wszystkie, a zapytania z `Where` - wybrane (filtrowanie). Możemy też pobrać rekord po `Id`.

PRZYKŁAD C#

```
// Wszystkie rekordy
public async Task<List<Zadanie>> PobierzWszystkie()
{
    await Init();
    return await db.Table<Zadanie>().ToListAsync();
}

// Rekord po Id
public async Task<Zadanie> PobierzPoId(int id)
{
    await Init();
    return await db.Table<Zadanie>().Where(z => z.Id == id).FirstOrDefaultAsync();
}

// Filtrowanie (tylko niewykonane)
public async Task<List<Zadanie>> Niewykonane()
{
    await Init();
    return await db.Table<Zadanie>().Where(z => !z.Wykonane).ToListAsync();
}
```

Na co uważać:

`FirstOrDefaultAsync` zwraca `null`, gdy nie znaleziono rekordu - sprawdzaj wynik. Filtrowanie i sortowanie (`Where`, `OrderBy`) wykonujemy na `db.Table<T>()`, co jest wydajne i czytelne.

30.7. Aktualizacja rekordu (Update)

Aktualizacja to zapisanie zmian w istniejącym rekordzie metodą `UpdateAsync`. Rekord musi mieć ustawione `Id` (czyli pochodzić z bazy lub mieć znany identyfikator).

PRZYKŁAD C#

```
public async Task<int> Aktualizuj(Zadanie z)
{
    await Init();
```

```
return await db.UpdateAsync(z); // aktualizuje rekord o danym Id
}
```

```
// Użycie: oznacz zadanie jako wykonane
zadanie.Wykonane = true;
await baza.Aktualizuj(zadanie);
```

Na co uważać:

`UpdateAsync` działa na podstawie `Id` - jeśli `Id` wynosi 0 (nowy obiekt), aktualizacja nie znajdzie rekordu. Często stosuje się jedną metodę „Zapisz”, która dodaje (gdy `Id == 0`) lub aktualizuje (gdy `Id != 0`).

30.8. Usuwanie rekordu (Delete)

Usunięcie to skasowanie rekordu z tabeli metodą `DeleteAsync`. Można też usunąć po `Id` lub wyczyścić całą tabelę.

PRZYKŁAD C#

```
public async Task<int> Usun(Zadanie z)
{
    await Init();
    return await db.DeleteAsync(z);
}

// Usunięcie wszystkich rekordów danego typu
public async Task Wyczyszc()
{
    await Init();
    await db.DeleteAllAsync<Zadanie>();
}
```

Na co uważać:

Usunięcie jest nieodwracalne - przy danych użytkownika warto poprosić o potwierdzenie (`DisplayAlert`). `DeleteAllAsync` czyści całą tabelę, więc używaj go ostrożnie.

30.9. Wzorzec „Zapisz” (dodaj lub aktualizuj)

Wygodnym wzorcem jest jedna metoda `Zapisz`, która **dodaje** nowy rekord (gdy `Id == 0`) lub **aktualizuje** istniejący (gdy `Id != 0`). Upraszcza to obsługę formularzy dodawania i edycji.

PRZYKŁAD C#

```
public async Task<int> Zapisz(Zadanie z)
{
    await Init();
    if (z.Id != 0)
        return await db.UpdateAsync(z); // istnieje → aktualizuj
    return await db.InsertAsync(z);     // nowy → dodaj
}
```

Na co uważać:

Ten wzorzec pozwala użyć tego samego ekranu do dodawania i edycji: dla nowego elementu `Id` to 0 (dodanie), dla edytowanego - istniejące `Id` (aktualizacja).

30.10. Połączenie SQLite z CollectionView

Dane z bazy wyświetlamy w `CollectionView`. Po każdej operacji (dodanie, usunięcie, edycja) ponownie wczytujemy listę z bazy i przypisujemy do widoku - albo aktualizujemy `ObservableCollection`.

PRZYKŁAD C#

```
readonly ZadaniaBaza baza = new ZadaniaBaza();

protected override async void OnAppearing()
{
    base.OnAppearing();
    await OdswiezListe();
}

private async void OnDodaj(object sender, EventArgs e)
{
    if (string.IsNullOrEmpty(PoleTytuL.Text)) return;
    await baza.Zapisz(new Zadanie { TytuL = PoleTytuL.Text });
    PoleTytuL.Text = string.Empty;
    await OdswiezListe();
}

private async Task OdswiezListe()
{
    Lista.ItemsSource = await baza.PobierzWszystkie();
}
```

Na co uważać:

Po każdej zmianie w bazie odśwież widok danymi z bazy - wtedy interfejs zawsze odzwierciedla rzeczywisty, zapisany stan. Odświeżanie w `OnAppearing` zapewnia aktualność po powrocie z innego ekranu.

30.11. Kompletny przykład: aplikacja zadań z SQLite

PRZYKŁAD C# (PEŁNA KLASA BAZY)

```
using SQLite;

public class ZadaniaBaza
{
    SQLiteAsyncConnection db;

    async Task Init()
    {
        if (db != null) return;
        string sciezka = Path.Combine(FileSystem.AppDataDirectory, "zadania.db3");
        db = new SQLiteAsyncConnection(sciezka);
        await db.CreateTableAsync<Zadanie>();
    }

    public async Task<List<Zadanie>> Pobierz()
    { await Init(); return await db.Table<Zadanie>().ToListAsync(); }

    public async Task Zapisz(Zadanie z)
    {
```

```
        await Init();
        if (z.Id != 0) await db.UpdateAsync(z);
        else await db.InsertAsync(z);
    }

    public async Task Usun(Zadanie z)
    { await Init(); await db.DeleteAsync(z); }
}
```

Na co uważać:

30.12. Typowe błędy

NAJWAŻNIEJSZE INFORMACJE

BŁĄD	SKUTEK	ROZWIĄZANIE
Brak <code>[PrimaryKey]</code> / <code>[AutoIncrement]</code>	błędne Id	dodaj atrybuty
Ręczne ustawianie <code>Id</code> przy dodawaniu	konflikt	pozwól bazie nadać Id
Synchroniczne wywołania	blokada UI	używaj <code>await</code>
Brak <code>CreateTableAsync</code>	brak tabeli	utwórz tabelę w <code>Init</code>
<code>UpdateAsync</code> na obiekcie z <code>Id=0</code>	brak efektu	użyj wzorca Zapisz
Brak odświeżenia widoku po zmianie	nieaktualna lista	wczytaj ponownie po operacji

Na co uważać:

Najczęstsze potknięcia: brak atrybutów klucza, ręczne ustawianie `Id` oraz brak utworzenia tabeli. Pamiętaj, że wszystkie operacje są asynchroniczne (`await`), a po każdej zmianie odświeżaj widok.

SQLite to standardowy sposób trwałego przechowywania większych danych w MAUI. Zapamiętaj przepis: model z `[PrimaryKey, AutoIncrement]`, klasa bazy z leniwym `Init` i `CreateTableAsync`, metody CRUD oparte na `InsertAsync` / `UpdateAsync` / `DeleteAsync` / `Table<T>()`, oraz odświeżanie widoku po każdej operacji.

30.13. SQLite - model + klasa bazy CRUD

Kompletna implementacja bazy SQLite z modelem i klasą obsługującą operacje CRUD. Wymaga pakietu NuGet: `sqlite-net-pcl`.

```
<?xml version="1.0" encoding="utf-8" ?>
<ContentPage xmlns="http://schemas.microsoft.com/dotnet/2021/maui"
              xmlns:x="http://schemas.microsoft.com/winfx/2009/xaml"
              x:Class="MojaAplikacja.SQLiteCrudPage"
              Title="SQLite CRUD">
    <VerticalStackLayout Padding="20" Spacing="10">
        <Label Text="Zarządzanie kontaktami" FontSize="20" FontAttributes="Bold"/>

        <Entry x:Name="EntryImie" Placeholder="Imię"/>
        <Entry x:Name="EntryTelefon" Placeholder="Telefon" Keyboard="Telephone"/>

        <HorizontalStackLayout Spacing="10">
            <Button Text="Dodaj" Clicked="OnDodajClicked"/>
            <Button Text="Aktualizuj" Clicked="OnAktualizujClicked"/>
            <Button Text="Usuń" Clicked="OnUsunClicked" BackgroundColor="Red" TextColor="White"/>
        </HorizontalStackLayout>
    </VerticalStackLayout>
</ContentPage>
```

```

        </HorizontalStackLayout>

        <CollectionView x:Name="ListaKontakty" SelectionMode="Single"
            SelectionChanged="OnWyborKontaktu">
            <CollectionView.ItemTemplate>
                <DataTemplate>
                    <HorizontalStackLayout Padding="10" Spacing="15">
                        <Label Text="{Binding Imie}" FontSize="16" FontAttributes="Bold"/>
                        <Label Text="{Binding Telefon}" FontSize="14" TextColor="Gray"/>
                    </HorizontalStackLayout>
                </DataTemplate>
            </CollectionView.ItemTemplate>
        </CollectionView>
    </VerticalStackLayout>
</ContentPage>

```

```

using SQLite;
using System.IO;

namespace MojaAplikacja;

// ===== MODEL =====
public class Kontakt
{
    [PrimaryKey, AutoIncrement]
    public int Id { get; set; }

    [MaxLength(100)]
    public string Imie { get; set; }

    [MaxLength(20)]
    public string Telefon { get; set; }
}

// ===== KLASA BAZY DANYCH =====
public class KontaktyBaza
{
    private SQLiteAsyncConnection _baza;

    // Inicjalizacja połączenia
    private async Task InicjalizujAsync()
    {
        if (_baza != null) return;

        // Ścieżka do pliku bazy danych
        string sciezka = Path.Combine(FileSystem.AppDataDirectory, "kontakty.db3");
        _baza = new SQLiteAsyncConnection(sciezka);

        // Tworzymy tabelę jeśli nie istnieje
        await _baza.CreateTableAsync<Kontakt>();
    }
}

```



```

// CREATE – dodanie kontaktu
public async Task<int> DodajKontaktAsync(Kontakt kontakt)
{
    await InicjalizujAsync();
    return await _baza.InsertAsync(kontakt);
}

// READ – pobranie wszystkich kontaktów
public async Task<List<Kontakt>> PobierzWszystkieAsync()
{
    await InicjalizujAsync();
    return await _baza.Table<Kontakt>().ToListAsync();
}

// READ – pobranie jednego kontaktu
public async Task<Kontakt> PobierzPoIdAsync(int id)
{
    await InicjalizujAsync();
    return await _baza.Table<Kontakt>().Where(k => k.Id == id).FirstOrDefaultAsync();
}

// UPDATE – aktualizacja kontaktu
public async Task<int> AktualizujKontaktAsync(Kontakt kontakt)
{
    await InicjalizujAsync();
    return await _baza.UpdateAsync(kontakt);
}

// DELETE – usunięcie kontaktu
public async Task<int> UsunKontaktAsync(Kontakt kontakt)
{
    await InicjalizujAsync();
    return await _baza.DeleteAsync(kontakt);
}
}

// ===== CODE-BEHIND STRONY =====
public partial class SqliteCrudPage : ContentPage
{
    private KontaktyBaza kontaktyBaza = new KontaktyBaza();
    private Kontakt _wybranyKontakt;

    public SqliteCrudPage()
    {
        InitializeComponent();
    }

    protected override async void OnAppearing()
    {
        base.OnAppearing();
        await OdswiezListe();
    }

    private async Task OdswiezListe()

```

```

{
    ListaKontakty.ItemsSource = await kontaktyBaza.PobierzWszystkieAsync();
}

private async void OnDodajClickd(object sender, EventArgs e)
{
    if (string.IsNullOrEmpty(EntryImie.Text)) return;

    var kontakt = new Kontakt
    {
        Imie = EntryImie.Text,
        Telefon = EntryTelefon.Text ?? ""
    };

    await kontaktyBaza.DodajKontaktAsync(kontakt);
    EntryImie.Text = "";
    EntryTelefon.Text = "";
    await OdswiezListe();
}

private async void OnAktualizujClickd(object sender, EventArgs e)
{
    if (_wybranyKontakt == null) return;

    _wybranyKontakt.Imie = EntryImie.Text;
    _wybranyKontakt.Telefon = EntryTelefon.Text;

    await kontaktyBaza.AktualizujKontaktAsync(_wybranyKontakt);
    await OdswiezListe();
}

private async void OnUsunClickd(object sender, EventArgs e)
{
    if (_wybranyKontakt == null) return;

    await kontaktyBaza.UsunKontaktAsync(_wybranyKontakt);
    _wybranyKontakt = null;
    EntryImie.Text = "";
    EntryTelefon.Text = "";
    await OdswiezListe();
}

private void OnWyborKontaktu(object sender, SelectionChangedEventArgs e)
{
    _wybranyKontakt = e.CurrentSelection.FirstOrDefault() as Kontakt;
    if (_wybranyKontakt != null)
    {
        EntryImie.Text = _wybranyKontakt.Imie;
        EntryTelefon.Text = _wybranyKontakt.Telefon;
    }
}
}

```

30.14. SQLite - pełna aplikacja zadań z CollectionView

Kompletna aplikacja typu "lista zadań" (to-do) z SQLite i CollectionView, w tym oznaczanie jako wykonane.

```
<?xml version="1.0" encoding="utf-8" ?>
<ContentPage xmlns="http://schemas.microsoft.com/dotnet/2021/maui"
              xmlns:x="http://schemas.microsoft.com/winfx/2009/xaml"
              x:Class="MojaAplikacja.ZadaniaPage"
              Title="Lista zadań">
    <Grid RowDefinitions="Auto,*" Padding="15" RowSpacing="10">

        <!-- Panel dodawania -->
        <HorizontalStackLayout Grid.Row="0" Spacing="10">
            <Entry x:Name="EntryZadanie" Placeholder="Nowe zadanie..."
                  HorizontalOptions="FillAndExpand" WidthRequest="250"/>
            <Button Text="+" Clicked="OnDodajClickedList" WidthRequest="50"
                  FontSize="20" FontAttributes="Bold"/>
        </HorizontalStackLayout>

        <!-- Lista zadań -->
        <CollectionView Grid.Row="1" x:Name="ListaZadan" SelectionMode="None">
            <CollectionView.ItemTemplate>
                <DataTemplate>
                    <SwipeView>
                        <SwipeView.RightItems>
                            <SwipeItems>
                                <SwipeItem Text="Usuń" BackgroundColor="Red"
                                      Invoked="OnUsunSwipe"/>
                            </SwipeItems>
                        </SwipeView.RightItems>

                        <Grid ColumnDefinitions="Auto,*" Padding="10" ColumnSpacing="10">
                            <CheckBox IsChecked="{Binding CzyWykonane}"
                                      CheckedChanged="OnCheckChanged"
                                      BindingContext="{Binding .}"/>
                            <Label Grid.Column="1" Text="{Binding Tytuł}"
                                   VerticalOptions="Center" FontSize="16"
                                   TextDecorations="{Binding Dekoracja}"/>
                        </Grid>
                    </SwipeView>
                </DataTemplate>
            </CollectionView.ItemTemplate>
        </CollectionView>
    </Grid>
</ContentPage>
```

```
using SQLite;
using System.IO;

namespace MojaAplikacja;
```

```
// Model zadania
public class Zadanie
{
    [PrimaryKey, AutoIncrement]
    public int Id { get; set; }

    public string Tytuł { get; set; }

    public bool CzyWykonane { get; set; }

    // Właściwość pomocnicza do przekreślenia tekstu
    [Ignore]
    public TextDecorations Dekoracja =>
        CzyWykonane ? TextDecorations.Strikethrough : TextDecorations.None;
}

// Klasa bazy danych zadań
public class ZadaniaBaza
{
    private SQLiteAsyncConnection _baza;

    private async Task InicjalizujAsync()
    {
        if (_baza != null) return;
        string sciezka = Path.Combine(FileSystem.AppDataDirectory, "zadania.db3");
        _baza = new SQLiteAsyncConnection(sciezka);
        await _baza.CreateTableAsync<Zadanie>();
    }

    public async Task<List<Zadanie>> PobierzWszystkieAsync()
    {
        await InicjalizujAsync();
        return await _baza.Table<Zadanie>().ToListAsync();
    }

    public async Task DodajAsync(Zadanie zadanie)
    {
        await InicjalizujAsync();
        await _baza.InsertAsync(zadanie);
    }

    public async Task AktualizujAsync(Zadanie zadanie)
    {
        await InicjalizujAsync();
        await _baza.UpdateAsync(zadanie);
    }

    public async Task UsunAsync(Zadanie zadanie)
    {
        await InicjalizujAsync();
        await _baza.DeleteAsync(zadanie);
    }
}
```

```
// Code-behind strony
public partial class ZadaniaPage : ContentPage
{
    private ZadaniaBaza zadaniaBaza = new ZadaniaBaza();

    public ZadaniaPage()
    {
        InitializeComponent();
    }

    protected override async void OnAppearing()
    {
        base.OnAppearing();
        await OdswiezListe();
    }

    private async Task OdswiezListe()
    {
        ListaZadan.ItemsSource = await zadaniaBaza.PobierzWszystkieAsync();
    }

    private async void OnDodajCliked(object sender, EventArgs e)
    {
        string tytul = EntryZadanie.Text?.Trim();
        if (string.IsNullOrEmpty(tytul)) return;

        await zadaniaBaza.DodajAsync(new Zadanie { Tytul = tytul, CzyWykonane = false });
        EntryZadanie.Text = "";
        await OdswiezListe();
    }

    private async void OnCheckChanged(object sender, CheckedChangedEventArgs e)
    {
        var checkBox = sender as CheckBox;
        var zadanie = checkBox?.BindingContext as Zadanie;
        if (zadanie == null) return;

        zadanie.CzyWykonane = e.Value;
        await zadaniaBaza.AktualizujAsync(zadanie);
        await OdswiezListe();
    }

    private async void OnUsunSwipe(object sender, EventArgs e)
    {
        var swipeItem = sender as SwipeItem;
        var zadanie = swipeItem?.BindingContext as Zadanie;
        if (zadanie == null) return;

        await zadaniaBaza.UsunAsync(zadanie);
        await OdswiezListe();
    }
}
```

31. API, JSON i HttpClient - temat opcjonalny

Wiele aplikacji pobiera dane z **internetu** - pogodę, wiadomości, produkty ze sklepu. Komunikacja odbywa się zwykle przez **API** (interfejs serwera), a dane przesyłane są w formacie **JSON**. Ten rozdział tłumaczy od zera, czym jest API, REST i JSON, jak pobierać i wysyłać dane przez **HttpClient**, jak zamieniać JSON na obiekty C# (deserializacja) i jak obsługiwać błędy sieci.

31.1. Czym jest API

API (*Application Programming Interface*) to **interfejs**, przez który aplikacja komunikuje się z serwerem. Serwer udostępnia dane i operacje pod określonymi adresami (URL), a aplikacja wysyła do nich żądania i odbiera odpowiedzi. Dzięki API aplikacja mobilna może np. pobrać aktualną pogodę z publicznego serwisu pogodowego.

NAJWAŻNIEJSZE INFORMACJE

- API to „drzwi”, przez które aplikacja rozmawia z serwerem.
- Komunikacja odbywa się przez protokół **HTTP** (jak w przeglądarce).
- Dane przesyłane są najczęściej w formacie **JSON**.
- Aplikacja wysyła **żądanie** i odbiera **odpowiedź**.

Na co uważać:

Komunikacja z API wymaga **połączenia z internetem** i jest **asynchroniczna** (trwa pewien czas). Zawsze obsługuj sytuacje braku sieci i błędów serwera.

31.2. REST API i endpointy

REST API to popularny styl projektowania API, w którym każdy zasób ma swój **adres (endpoint)**, a operacje wyrażamy **metodami HTTP**: **GET** (pobierz), **POST** (utwórz), **PUT** (zaktualizuj), **DELETE** (usuń). **Endpoint** to konkretny adres URL, np. `https://api.przyklad.pl/produkty`.

NAJWAŻNIEJSZE INFORMACJE

METODA HTTP	OPERACJA	PRZYKŁAD
GET	pobranie danych	pobierz listę produktów
POST	utworzenie	dodaj nowy produkt
PUT	aktualizacja	zmień produkt
DELETE	usunięcie	usuń produkt

Na co uważać:

Metoda HTTP określa **rodzaj operacji**, a endpoint - **zasób**. Np. `GET /produkty/5` pobiera produkt o id 5. To spójna, przewidywalna konwencja, którą stosuje większość API.

31.3. Kody statusu HTTP

Serwer w odpowiedzi zwraca **kod statusu** informujący o wyniku. Kody z zakresu 200 oznaczają sukces, 400 - błąd po stronie klienta (np. złe dane), 500 - błąd serwera. Aplikacja powinna sprawdzać ten kod.

NAJWAŻNIEJSZE INFORMACJE

KOD	ZNACZENIE
200 OK	sukces
201 Created	utworzono (po POST)
400 Bad Request	błędne żądanie
401 Unauthorized	brak autoryzacji
404 Not Found	nie znaleziono zasobu
500 Internal Server Error	błąd serwera

Na co uważać:

Nie zakładaj, że żądanie zawsze się powiedzie. Sprawdzaj kod statusu (`response.IsSuccessStatusCode`) i obsługuj błędy - serwer może być niedostępny, dane mogą być błędne, a sieć może paść.

31.4. Czym jest JSON

JSON (*JavaScript Object Notation*) to tekstowy **format danych** - czytelny dla człowieka i łatwy do przetwarzania. Dane zapisuje jako pary „klucz”: wartość, obiekty w nawiasach klamrowych `{ }` i tablice w nawiasach kwadratowych `[ ]`. To najpopularniejszy format wymiany danych między aplikacją a API.

PRZYKŁAD (JSON)

```
{
  "id": 5,
  "nazwa": "Kawa",
  "cena": 19.99,
  "dostepny": true,
  "tagi": ["napoje", "gorące"]
}
```

Na co uważać:

JSON to **tekst** - aby użyć go w C#, trzeba go **zdeserializować** na obiekt (i odwrotnie - **zserializować** obiekt na tekst). JSON rozróżnia typy: tekst w cudzysłowie, liczby bez, wartości logiczne `true` / `false`.

31.5. HttpClient - pobieranie danych (GET)

`HttpClient` to klasa do wysyłania żądań HTTP i odbierania odpowiedzi. Metoda `GetStringAsync` pobiera odpowiedź jako tekst, a `GetFromJsonAsync` (z pakietu) od razu deserializuje JSON na obiekt. Operacje są asynchroniczne (`await`).

PRZYKŁAD C#

```
using System.Net.Http;
using System.Net.Http.Json;

readonly HttpClient http = new HttpClient();

private async Task<List<Produkt>> PobierzProdukty()
{
    // Pobranie i deserializacja JSON w jednym kroku
    var produkty = await http.GetFromJsonAsync<List<Produkt>>(
        "https://api.przyklad.pl/produkty");
}
```

```
return produkty ?? new List<Produkt>();
}
```

Na co uważać:

`GetFromJsonAsync<T>` pobiera i od razu zamienia JSON na obiekty typu `T` - to najwygodniejsza metoda. Zwraca `null` przy braku danych, więc zabezpiecz się `?? new List<...>()`. Wymaga przestrzeni `System.Net.Http.Json`.

31.6. Wysyłanie danych (POST)

Aby **wysłać** dane do serwera (utworzyć rekord), używamy `PostAsJsonAsync`, które serializuje obiekt do JSON i wysyła go metodą POST. Serwer odpowiada kodem statusu i ewentualnie utworzonym obiektem.

PRZYKŁAD C#

```
private async Task<bool> DodajProdukt(Produkt p)
{
    var odpowiedz = await http.PostAsJsonAsync(
        "https://api.przyklad.pl/produkty", p);

    return odpowiedz.IsSuccessStatusCode; // czy się powiodło
}
```

Na co uważać:

Sprawdź `IsSuccessStatusCode`, by wiedzieć, czy operacja się udała. `PostAsJsonAsync` sam serializuje obiekt do JSON - nie musisz robić tego ręcznie. Analogicznie działają `PutAsJsonAsync` (aktualizacja) i `DeleteAsync` (usunięcie).

31.7. Serializacja i deserializacja - System.Text.Json

Serializacja to zamiana obiektu C# na tekst JSON; **deserializacja** - odwrotnie, tekstu JSON na obiekt. W .NET służy do tego klasa `JsonSerializer` z przestrzeni `System.Text.Json`. Metody `GetFromJsonAsync` / `PostAsJsonAsync` robią to automatycznie, ale czasem chcemy zrobić to ręcznie (np. dla danych z pliku).

PRZYKŁAD C#

```
using System.Text.Json;

// Serializacja obiektu → tekst JSON
Produkt p = new Produkt { Nazwa = "Kawa", Cena = 19.99 };
string json = JsonSerializer.Serialize(p);

// Deserializacja tekstu JSON → obiekt
Produkt odczytany = JsonSerializer.Deserialize<Produkt>(json);

// Deserializacja listy
List<Produkt> lista = JsonSerializer.Deserialize<List<Produkt>>(jsonListy);
```

Na co uważać:

Domyślnie `System.Text.Json` dopasowuje nazwy właściwości **rozdzielając wielkość liter** względem JSON, ale można włączyć tryb ignorujący (`PropertyNameCaseInsensitive = true`). Jeśli nazwy w JSON różnią się od właściwości C#, użyj atrybutu `[JsonPropertyName("nazwa_w_json")]`.

31.8. Model danych pod JSON

Aby zdeserializować JSON, potrzebujemy **klasy C#** z właściwościami odpowiadającymi polom JSON. Nazwy właściwości powinny pasować do kluczy JSON (lub mapujemy je atrybutem).

PRZYKŁAD C#

```
using System.Text.Json.Serialization;

public class Produkt
{
    public int Id { get; set; }
    public string Nazwa { get; set; }
    public double Cena { get; set; }
    public bool Dostępny { get; set; }

    // Mapowanie, gdy nazwa w JSON różni się od właściwości
    [JsonPropertyName("data_dodania")]
    public DateTime DataDodania { get; set; }
}
```

Na co uważać:

Model musi pasować do struktury JSON. Gdy klucz JSON ma inną nazwę (np. `snake_case`), użyj `[JsonPropertyName(...)]`. Pola, których nie ma w JSON, pozostaną z wartościami domyślnymi.

31.9. Wyświetlanie danych z API w `CollectionView`

Typowy przepływ: pobieramy dane z API (lista obiektów), przypisujemy do `CollectionView` i pokazujemy. Podczas pobierania warto pokazać wskaźnik ładowania.

PRZYKŁAD C#

```
protected override async void OnAppearing()
{
    base.OnAppearing();
    Loader.IsRunning = Loader.IsVisible = true;
    try
    {
        var produkty = await PobierzProdukty(); // metoda z 28.5
        Lista.ItemsSource = produkty;
    }
    catch (Exception ex)
    {
        await DisplayAlert("Błąd", "Nie udało się pobrać danych.", "OK");
    }
    finally
    {
        Loader.IsRunning = Loader.IsVisible = false;
    }
}
```

Na co uważać:

Pokaż `ActivityIndicator` podczas pobierania i ukryj go w `finally`. Otaczaj pobieranie `try/catch`, bo sieć może zawieść. Po pobraniu przypisz dane do `ItemsSource`.

31.10. Obsługa błędów API

Komunikacja z API może się nie udać na wiele sposobów: brak sieci, timeout, błąd serwera, błędne dane. Dlatego **zawsze** otaczamy ją `try/catch` i sprawdzamy kod statusu, a użytkownika informujemy o problemie.

PRZYKŁAD C#

```
private async Task<List<Produkt>> BezpiecznePobranie()
{
    try
    {
        var odpowiedz = await http.GetAsync("https://api.przyklad.pl/produkty");
        if (!odpowiedz.IsSuccessStatusCode)
        {
            await DisplayAlert("Błąd", $"Serwer zwrócił: {(int)odpowiedz.StatusCode}", "OK");
            return new List<Produkt>();
        }

        string json = await odpowiedz.Content.ReadAsStringAsync();
        return JsonSerializer.Deserialize<List<Produkt>>(json,
            new JsonSerializerOptions { PropertyNameCaseInsensitive = true })
            ?? new List<Produkt>();
    }
    catch (HttpRequestException)
    {
        await DisplayAlert("Błąd sieci", "Sprawdź połączenie z internetem.", "OK");
        return new List<Produkt>();
    }
}
```

Na co uważać:

Rozróżniaj rodzaje błędów: `HttpRequestException` to zwykle problem z siecią, a kod statusu (np. 404, 500) - problem po stronie serwera. W obu przypadkach poinformuj użytkownika zamiast pozwolić aplikacji na awarię. Warto też sprawdzić połączenie przez `Connectivity` przed żądaniem.

31.11. Typowe błędy

NAJWAŻNIEJSZE INFORMACJE

BŁĄD	SKUTEK	ROZWIĄZANIE
Brak <code>try/catch</code>	awaria przy braku sieci	otocz żądanie <code>try/catch</code>
Brak sprawdzenia statusu	błędne dane traktowane jak OK	sprawdź <code>IsSuccessStatusCode</code>
Model niezgodny z JSON	puste/ <code>null</code> właściwości	dopasuj model, <code>[JsonPropertyName]</code>
Brak wskaźnika ładowania	aplikacja wygląda na zawieszoną	pokaż <code>ActivityIndicator</code>
Operacja synchroniczna	blokada UI	używaj <code>await</code>

Na co uważać:

Trzy filary pracy z API: **asynchroniczność** (`await`), **obsługa błędów** (`try/catch` + sprawdzanie statusu) oraz **zgodność modelu z JSON**. Zawsze pokazuj użytkownikowi, że trwa pobieranie, i informuj o problemach.

Praca z API to: pobierz (`GetFromJsonAsync`), pokaż wskaźnik ładowania, obsłuż błędy, zdeserializuj na model i wyświetl w `CollectionView`. Logikę pobierania danych trzymaj w osobnej klasie, a `HttpClient` współdziel. To wzorec aplikacji takich jak pogodynka czy katalog online.

31.12. HttpClient GET + JSON (GetFromJsonAsync)

Pobieramy dane z API REST i deserializujemy JSON do modelu C#.

```

<?xml version="1.0" encoding="utf-8" ?>
<ContentPage xmlns="http://schemas.microsoft.com/dotnet/2021/maui"
              xmlns:x="http://schemas.microsoft.com/winfx/2009/xaml"
              x:Class="MojaAplikacja.HttpGetPage"
              Title="HttpClient GET">
    <VerticalStackLayout Padding="20" Spacing="10">
        <Button Text="Pobierz listę użytkowników" Clicked="OnPobierzClicked"/>
        <ActivityIndicator x:Name="Ladowanie" IsRunning="False" IsVisible="False"/>
        <Label x:Name="LabelBład" TextColor="Red" />

        <CollectionView x:Name="ListaUzytkownicy">
            <CollectionView.ItemTemplate>
                <DataTemplate>
                    <VerticalStackLayout Padding="10" Spacing="3">
                        <Label Text="{Binding Name}" FontSize="16" FontAttributes="Bold"/>
                        <Label Text="{Binding Email}" FontSize="13" TextColor="Gray"/>
                        <Label Text="{Binding Phone}" FontSize="12"/>
                        <BoxView HeightRequest="1" Color="LightGray"/>
                    </VerticalStackLayout>
                </DataTemplate>
            </CollectionView.ItemTemplate>
        </CollectionView>
    </VerticalStackLayout>
</ContentPage>

```

```

using System.Net.Http.Json;

namespace MojaAplikacja;

// Model odpowiadający strukturze JSON z API
public class Uzytkownik
{
    public int Id { get; set; }
    public string Name { get; set; }
    public string Email { get; set; }
    public string Phone { get; set; }
}

public partial class HttpGetPage : ContentPage
{
    // Jeden HttpClient na cały cykl życia (najlepsza praktyka)
    private readonly HttpClient _httpClient = new HttpClient();

    public HttpGetPage()
    {
        InitializeComponent();
    }

    private async void OnPobierzClicked(object sender, EventArgs e)
    {
        try

```

```

{
    // Pokazujemy wskaźnik ładowania
    Ladowanie.IsVisible = true;
    Ladowanie.IsRunning = true;
    LabelBlad.Text = "";

    // Pobieramy i deserializujemy JSON w jednym kroku
    var uzytkownicy = await _httpClient.GetFromJsonAsync<List<Uzytkownik>>(
        "https://jsonplaceholder.typicode.com/users");

    // Wyświetlamy w CollectionView
    ListaUzytkownicy.ItemsSource = uzytkownicy;
}
catch (HttpRequestException ex)
{
    LabelBlad.Text = $"Błąd HTTP: {ex.Message}";
}
catch (Exception ex)
{
    LabelBlad.Text = $"Błąd: {ex.Message}";
}
finally
{
    // Ukrywamy wskaźnik ładowania
    Ladowanie.IsVisible = false;
    Ladowanie.IsRunning = false;
}
}
}

```

31.13. HttpClient POST

Wysyłamy dane do API w formacie JSON metodą POST.

```

<?xml version="1.0" encoding="utf-8" ?>
<ContentPage xmlns="http://schemas.microsoft.com/dotnet/2021/maui"
    xmlns:x="http://schemas.microsoft.com/winfx/2009/xaml"
    x:Class="MojaAplikacja.HttpPostPage"
    Title="HttpClient POST">
    <VerticalStackLayout Padding="20" Spacing="15">
        <Label Text="Wyślij nowy wpis do API" FontSize="18" FontAttributes="Bold"/>

        <Entry x:Name="EntryTytuł" Placeholder="Tytuł"/>
        <Editor x:Name="EditorTresc" Placeholder="Treść wpisu..." HeightRequest="120"/>

        <Button Text="Wyślij (POST)" Clicked="OnWyślijClicked"/>
        <ActivityIndicator x:Name="Ladowanie" IsRunning="False" IsVisible="False"/>

        <Frame x:Name="FrameOdpowiedz" IsVisible="False" BorderColor="Green" Padding="10">
            <VerticalStackLayout Spacing="5">
                <Label Text="Odpowiedź serwera:" FontAttributes="Bold"/>
                <Label x:Name="LabelOdpowiedz" FontSize="13"/>
            </VerticalStackLayout>
        </Frame>
    </VerticalStackLayout>
</ContentPage>

```

```

        </VerticalStackLayout>
    </Frame>

    <Label x:Name="LabelBlad" TextColor="Red" />
</VerticalStackLayout>
</ContentPage>

```

```

using System.Net.Http.Json;
using System.Text.Json;

namespace MojaAplikacja;

// Model danych do wysłania
public class NowyWpis
{
    public string Title { get; set; }
    public string Body { get; set; }
    public int UserId { get; set; }
}

// Model odpowiedzi z serwera
public class OdpowiedzWpis
{
    public int Id { get; set; }
    public string Title { get; set; }
    public string Body { get; set; }
    public int UserId { get; set; }
}

public partial class HttpPostPage : ContentPage
{
    private readonly HttpClient _httpClient = new HttpClient();

    public HttpPostPage()
    {
        InitializeComponent();
    }

    private async void OnWyslijClicked(object sender, EventArgs e)
    {
        if (string.IsNullOrEmpty(EntryTytul.Text))
        {
            LabelBlad.Text = "Podaj tytuł!";
            return;
        }

        try
        {
            Ladowanie.IsVisible = true;
            Ladowanie.IsRunning = true;
            LabelBlad.Text = "";
            FrameOdpowiedz.IsVisible = false;

```

```

        // Tworzymy obiekt do wysłania
        var wpis = new NowyWpis
        {
            Title = EntryTytuł.Text,
            Body = EditorTresc.Text ?? "",
            UserId = 1
        };

        // Wysyłamy POST z automatyczną serializacją do JSON
        var odpowiedz = await _httpClient.PostAsJsonAsync(
            "https://jsonplaceholder.typicode.com/posts", wpis);

        // Sprawdzamy kod statusu
        odpowiedz.EnsureSuccessStatusCode();

        // Deserializujemy odpowiedź
        var wynik = await odpowiedz.Content.ReadFromJsonAsync<OdpowiedzWpis>();

        // Wyświetlamy odpowiedź serwera
        FrameOdpowiedz.IsVisible = true;
        LabelOdpowiedz.Text = $"ID: {wynik.Id}\nTytuł: {wynik.Title}\nTreść: {wynik.Body}";
    }
    catch (HttpRequestException ex)
    {
        LabelBlad.Text = $"Błąd HTTP: {ex.StatusCode} - {ex.Message}";
    }
    catch (Exception ex)
    {
        LabelBlad.Text = $"Błąd: {ex.Message}";
    }
    finally
    {
        Ladowanie.IsVisible = false;
        Ladowanie.IsRunning = false;
    }
}
}

```

31.14. ActivityIndicator + try/catch + Connectivity

Kompletny wzorzec bezpiecznego pobierania danych: sprawdzenie połączenia, wskaźnik ładowania, obsługa błędów i timeout.

```

<?xml version="1.0" encoding="utf-8" ?>
<ContentPage xmlns="http://schemas.microsoft.com/dotnet/2021/maui"
    xmlns:x="http://schemas.microsoft.com/winfx/2009/xaml"
    x:Class="MojaAplikacja.BezpiecznePobieraniePage"
    Title="Bezpieczne pobieranie">
    <Grid RowDefinitions="Auto,Auto,*" Padding="20" RowSpacing="15">

        <!-- Przycisk pobierania -->
        <Button Grid.Row="0" Text="Pobierz dane z API" Clicked="OnPobierzClicked"

```

```

        x:Name="BtnPobierz" />

<!-- Wskaźnik ładowania z komunikatem -->
<VerticalStackLayout Grid.Row="1" x:Name="PanelLadowanie" IsVisible="False"
    HorizontalOptions="Center" Spacing="5">
    <ActivityIndicator IsRunning="True" Color="Blue" HeightRequest="40"/>
    <Label Text="Pobieranie danych..." HorizontalOptions="Center" FontSize="14"/>
</VerticalStackLayout>

<!-- Wyniki lub błąd -->
<VerticalStackLayout Grid.Row="2" Spacing="10">
    <!-- Komunikat o braku internetu -->
    <Frame x:Name="FrameBrakInternetu" IsVisible="False"
        BackgroundColor="#FFF3CD" BorderColor="#FFC107" Padding="15">
        <HorizontalStackLayout Spacing="10">
            <Label Text="⚠" FontSize="24"/>
            <Label Text="Brak połączenia z Internetem. Sprawdź ustawienia sieci."
                VerticalOptions="Center"/>
        </HorizontalStackLayout>
    </Frame>

    <!-- Komunikat o błędzie -->
    <Frame x:Name="FrameBlad" IsVisible="False"
        BackgroundColor="#F8D7DA" BorderColor="#F5C6CB" Padding="15">
        <VerticalStackLayout Spacing="5">
            <Label Text="❌ Wystąpił błąd" FontAttributes="Bold"/>
            <Label x:Name="LabelBladSzczegoly" FontSize="12"/>
            <Button Text="Spróbuj ponownie" Clicked="OnPobierzClicked"
                BackgroundColor="#DC3545" TextColor="White"/>
        </VerticalStackLayout>
    </Frame>

    <!-- Lista wyników -->
    <CollectionView x:Name="ListaWyniki">
        <CollectionView.ItemTemplate>
            <DataTemplate>
                <Frame Margin="0,5" Padding="10" BorderColor="LightGray">
                    <VerticalStackLayout>
                        <Label Text="{Binding Title}" FontSize="15" FontAttributes="Bold"
                            LineBreakMode="WordWrap"/>
                        <Label Text="{Binding Body}" FontSize="12" TextColor="Gray"
                            MaxLines="2"/>
                    </VerticalStackLayout>
                </Frame>
            </DataTemplate>
        </CollectionView.ItemTemplate>
    </CollectionView>
</VerticalStackLayout>
</Grid>
</ContentPage>

```

```
using System.Net.Http.Json;

namespace MojaAplikacja;

// Model danych
public class Post
{
    public int Id { get; set; }
    public string Title { get; set; }
    public string Body { get; set; }
    public int UserId { get; set; }
}

public partial class BezpiecznePobieraniePage : ContentPage
{
    private readonly HttpClient _httpClient;

    public BezpiecznePobieraniePage()
    {
        InitializeComponent();

        // Konfigurujemy HttpClient z timeoutem
        _httpClient = new HttpClient
        {
            Timeout = TimeSpan.FromSeconds(15) // Timeout 15 sekund
        };
    }

    private async void OnPobierzClicked(object sender, EventArgs e)
    {
        // KROK 1: Sprawdzamy połączenie z internetem
        var dostep = Connectivity.Current.NetworkAccess;

        if (dostep != NetworkAccess.Internet)
        {
            // Brak internetu – informujemy użytkownika
            FrameBrakInternetu.IsVisible = true;
            FrameBlad.IsVisible = false;
            ListaWyniki.ItemsSource = null;
            return;
        }

        // KROK 2: Ukrywamy komunikaty, pokazujemy wskaźnik ładowania
        FrameBrakInternetu.IsVisible = false;
        FrameBlad.IsVisible = false;
        PanelLadowanie.IsVisible = true;
        BtnPobierz.IsEnabled = false;

        try
        {
            // KROK 3: Pobieramy dane z API
            var posty = await _httpClient.GetFromJsonAsync<List<Post>>(
                "https://jsonplaceholder.typicode.com/posts");
        }
    }
}
```



```
        // KROK 4: Wyświetlamy wyniki
        ListaWyniki.ItemsSource = posty;
    }
    catch (TaskCanceledException)
    {
        // Timeout - żądanie trwało za długo
        FrameBład.IsVisible = true;
        LabelBładSzczegoly.Text = "Przekroczono czas oczekiwania. Serwer nie odpowiada.";
    }
    catch (HttpRequestException ex)
    {
        // Błąd HTTP (np. 404, 500)
        FrameBład.IsVisible = true;
        LabelBładSzczegoly.Text = $"Błąd serwera: {ex.StatusCode} - {ex.Message}";
    }
    catch (Exception ex)
    {
        // Inny nieoczekiwany błąd
        FrameBład.IsVisible = true;
        LabelBładSzczegoly.Text = $"Nieoczekiwany błąd: {ex.Message}";
    }
    finally
    {
        // KROK 5: Zawsze ukrywamy wskaźnik i odblokowujemy przycisk
        Panelladowanie.IsVisible = false;
        BtnPobierz.IsEnabled = true;
    }
}
}
```

31.15. Podsumowanie

#	TEMAT	KLUCZOWE KLASY/METODY
1	Zapis pliku	<code>File.WriteAllTextAsync</code> , <code>FileSystem.AppDataDirectory</code>
2	Odczyt pliku	<code>File.ReadAllTextAsync</code> , <code>File.Exists</code>
3	Dopisywanie	<code>File.AppendAllTextAsync</code>
4	Usuwanie	<code>File.Delete</code> , <code>File.Exists</code>
5	Zasób Raw	<code>FileSystem.OpenAppPackageFileAsync</code>
6	Parsowanie	<code>Split</code> , <code>TryParse</code> , <code>List<T></code>
11	Obraz zasób	<code>ImageSource.FromFile</code> , atrybut <code>Source</code>
12	Obraz URL	<code>UriImageSource</code> , <code>CacheValidity</code>
13	Obraz lokalny	<code>ImageSource.FromFile</code> , <code>ImageSource.FromStream</code>
14	Podmiana runtime	Dynamiczne budowanie nazwy pliku
15	Klikalny obraz	<code>TapGestureRecognizer</code> , <code>Tapped</code>
16	Preferences	<code>Preferences.Set</code> , <code>Preferences.Get</code>
17	Motyw	<code>Application.Current.UserAppTheme</code> , <code>Preferences</code>
18	SQLite CRUD	<code>SQLiteAsyncConnection</code> , atrybuty <code>[PrimaryKey]</code>
19	SQLite + CV	<code>CollectionView</code> , <code>SwipeView</code> , <code>CheckBox</code>
20	HTTP GET	<code>HttpClient.GetFromJsonAsync<T></code>
21	HTTP POST	<code>HttpClient.PostAsJsonAsync</code>
22	Connectivity	<code>Connectivity.Current.NetworkAccess</code> , <code>ActivityIndicator</code>

GOTOWE: 892 linii.

32. Funkcje urządzenia i uprawnienia - temat opcjonalny

Aplikacje mobilne mogą korzystać z **funkcji urządzenia**: aparatu, lokalizacji, schowka, latarki, połączeń. Dostęp do wrażliwych funkcji wymaga **uprawnień**, o które trzeba poprosić. MAUI udostępnia spójne API (`Permissions` oraz klasy z `Microsoft.Maui.Devices` i `ApplicationModel`). Ten rozdział omawia uprawnienia oraz najważniejsze funkcje urządzenia.

32.1. Czym są uprawnienia

Uprawnienie to zgoda użytkownika (i systemu) na dostęp aplikacji do wrażliwej funkcji - aparatu, lokalizacji, kontaktów. Bez uprawnienia system zablokuje dostęp. Uprawnienia **deklarujemy** w plikach platformowych i czasem **prosimy** o nie w czasie działania.

NAJWAŻNIEJSZE INFORMACJE

- Uprawnienia deklarujemy w `Platforms/Android` (`AndroidManifest.xml`) i `Platforms/iOS` (`Info.plist`).
- W czasie działania prosimy o nie przez `Permissions.RequestAsync` .
- Użytkownik może odmówić - zawsze obsłuż tę sytuację.

Na co uważać:

Sama deklaracja w manifestie nie wystarcza - na nowszych systemach trzeba też **poprosić** o uprawnienie w trakcie działania i obsłużyć odmowę.

32.2. Uprawnienia na Androidzie i iOS

Każda platforma ma własny sposób deklarowania uprawnień. **Android** używa `AndroidManifest.xml`, **iOS** - `Info.plist` (gdzie podajemy też opis, po co aplikacja potrzebuje danej funkcji).

PRZYKŁAD (ANDROID - ANDROIDMANIFEST.XML)

```
<uses-permission android:name="android.permission.CAMERA" />
<uses-permission android:name="android.permission.ACCESS_FINE_LOCATION" />
```

PRZYKŁAD (IOS - INFO.PLIST)

```
<key>NSCameraUsageDescription</key>
<string>Aplikacja używa aparatu do robienia zdjęć.</string>
```

Na co uważać:

iOS wymaga opisu (`... UsageDescription`), dlaczego aplikacja potrzebuje danej funkcji - bez niego aplikacja zostanie odrzucona. Deklaruj tylko te uprawnienia, których naprawdę używasz.

32.3. Sprawdzanie i proszenie o uprawnienie

W kodzie sprawdzamy status uprawnienia (`CheckStatusAsync`) i prosimy o nie (`RequestAsync`). Obie metody są asynchroniczne i zwracają status (`Granted` , `Denied` itd.).

PRZYKŁAD C#

```
private async Task<bool> ZapewnijAparat()
{
    var status = await Permissions.CheckStatusAsync<Permissions.Camera>();
    if (status != PermissionStatus.Granted)
        status = await Permissions.RequestAsync<Permissions.Camera>();

    if (status != PermissionStatus.Granted)
    {
        await DisplayAlert("Brak zgody", "Aparat wymaga uprawnienia.", "OK");
        return false;
    }
    return true;
}
```

Na co uważać:

Najpierw **sprawdź** status, a poproś tylko, gdy uprawnienie nie jest jeszcze przyznane. Zawsze obsług **odmowę** - pokaż komunikat i nie próbuj używać zablokowanej funkcji.

32.4. Najważniejsze funkcje urządzenia

NAJWAŻNIEJSZE INFORMACJE

FUNKCJA	KLASA / API	ZASTOSOWANIE
Lokalizacja	Geolocation	pozycja GPS
Schowek	Clipboard	kopiowanie tekstu
Latarka	Flashlight	włączanie latarki
Telefon	PhoneDialer	wybieranie numeru
SMS	Sms	wysyłanie SMS
E-mail	Email	wysyłanie e-maila
Przeglądarka	Browser	otwieranie stron
Udostępnianie	Share	udostępnianie treści
Połączenie	Connectivity	stan internetu
Informacje	DeviceInfo	model, system

Na co uważać:

Większość tych funkcji jest w przestrzeniach `Microsoft.Maui.ApplicationModel` i `... Devices`. Część (aparat, lokalizacja) wymaga uprawnień; inne (schowek, przeglądarka) zwykle nie.

32.5. Przykłady: schowek, przeglądarka, telefon, e-mail

PRZYKŁAD C#

```
// Schowek - kopiowanie i wklejanie
await Clipboard.Default.SetTextAsync("Tekst do skopiowania");
string tekst = await Clipboard.Default.GetTextAsync();

// Przeglądarka - otwarcie strony
await Browser.Default.OpenAsync("https://example.com");

// Telefon - wybranie numeru
PhoneDialer.Default.Open("123456789");

// E-mail - utworzenie wiadomości
await Email.Default.ComposeAsync(new EmailMessage
{
    Subject = "Temat",
    Body = "Treść wiadomości",
    To = new List<string> { "adres@example.com" }
});
```

Na co uważać:

Te funkcje **otwierają zewnętrzne aplikacje** (telefon, klient poczty, przeglądarkę). Sprawdzaj dostępność (np. `PhoneDialer.Default.IsSupported`) i otaczaj wywołania `try/catch`, bo na niektórych urządzeniach mogą być niedostępne.

32.6. Geolokalizacja, latarka, udostępnianie

PRZYKŁAD C#

```
// Lokalizacja (wymaga uprawnień)
var lokacja = await Geolocation.Default.GetLocationAsync();
if (lokacja != null)
    Wynik.Text = $"{lokacja.Latitude}, {lokacja.Longitude}";

// Latarka
await Flashlight.Default.TurnOnAsync();
await Flashlight.Default.TurnOffAsync();

// Udostępnianie tekstu
await Share.Default.RequestAsync(new ShareTextRequest
{
    Text = "Zobacz tę aplikację!",
    Title = "Udostępnij"
});
```

Na co uważać:

Geolokalizacja wymaga uprawnień i może zwrócić `null` (np. brak GPS). Latarka działa tylko na urządzeniach z aparatem. Zawsze sprawdzaj dostępność funkcji i obsługuj błędy.

32.7. Connectivity i DeviceInfo

`Connectivity` informuje o **stanie połączenia** z internetem - przydatne przed pobieraniem danych z API. `DeviceInfo` podaje informacje o urządzeniu (model, system, typ).

PRZYKŁAD C#

```
// Sprawdzenie połączenia z internetem
if (Connectivity.Current.NetworkAccess != NetworkAccess.Internet)
{
    await DisplayAlert("Brak sieci", "Połącz się z internetem.", "OK");
    return;
}

// Informacje o urządzeniu
string model = DeviceInfo.Current.Model;
string system = DeviceInfo.Current.Platform.ToString();
bool toTelefon = DeviceInfo.Current.Idiom == DeviceIdiom.Phone;
```

Na co uważać:

Sprawdzaj `Connectivity` **przed** żądaniami sieciowymi - to oszczędza użytkownikowi czekania na nieudane pobranie i pozwala pokazać czytelny komunikat. `DeviceInfo.Idiom` pomaga dostosować interfejs (telefon vs komputer).

32.8. Typowe błędy

NAJWAŻNIEJSZE INFORMACJE

BŁĄD	SKUTEK	ROZWIĄZANIE
Brak deklaracji w manifeście	funkcja nie działa	dodaj uprawnienie w <code>Platforms</code>
Brak prośby w runtime	odmowa dostępu	<code>Permissions.RequestAsync</code>
Nieobsłużona odmowa	awaria przy użyciu funkcji	sprawdź status, pokaż komunikat
Brak <code>UsageDescription</code> (iOS)	odrzućenie aplikacji	dodaj opis w <code>Info.plist</code>
Użycie funkcji bez sprawdzenia dostępności	błąd na niektórych urządzeniach	sprawdź <code>IsSupported</code>

Na co uważać:

Praca z funkcjami urządzenia wymaga trzech kroków: **zadeklaruj** uprawnienie, **poproś** o nie w runtime, **obsłuż odmowę** i niedostępność. Zawsze otaczaj wywołania `try/catch`, bo zachowanie różni się między platformami i urządzeniami.

Funkcje urządzenia ożywiają aplikację (aparat, GPS, udostępnianie), ale wiążą się z uprawnieniami i różnicami platform. Zasada: zadeklaruj → poproś → obsłuż odmowę. Przed żądaniami sieciowymi sprawdzaj `Connectivity`.

Choć MAUI pozwala pisać wspólny kod, czasem potrzebujemy **zachowania zależnego od platformy** - innej wartości na telefonie i komputerze, kodu działającego tylko na Androidzie czy dostosowania do różnic systemowych. Ten rozdział pokazuje mechanizmy: folder `Platforms`, dyrektywy kompilacji warunkowej (`#if`), `OnPlatform` i `OnIdiom` oraz omawia typowe różnice między systemami.

33. Kod platformowy i różnice systemowe - temat opcjonalny

33.1. Folder Platforms

Folder `Platforms` zawiera kod i ustawienia **specyficzne dla każdego systemu**, w podkatalogach `Android`, `iOS`, `MacCatalyst`, `Windows`. To tam trafiają pliki startowe platform, manifesty uprawnień i kod natywny.

NAJWAŻNIEJSZE INFORMACJE

- `Platforms/Android` - `AndroidManifest.xml`, kod natywny Androida.
- `Platforms/iOS` - `Info.plist`, kod natywny iOS.
- `Platforms/Windows` - konfiguracja WinUI.
- Większość czasu pracujesz **poza** tym folderem (wspólny kod).

Na co uważać:

Do `Platforms` zaglądasz rzadko - głównie przy uprawnieniach lub integracji z kodem natywnym. Na początek nauki możesz go ignorować.

33.2. Kompilacja warunkowa: #if ANDROID, #if IOS, #if WINDOWS

Dyrektywy kompilacji warunkowej pozwalają zawrzeć w jednym pliku kod, który zostanie skompilowany **tylko dla wybranej platformy**. Używamy `#if ANDROID`, `#if IOS`, `#if WINDOWS`, `#if MACCATALYST`.

PRZYKŁAD C#

```
public string NazwaPlatformy()
{
    #if ANDROID
        return "Android";
    #elif IOS
        return "iOS";
    }
```

```

#elif WINDOWS
    return "Windows";
#else
    return "Inna";
#endif
}

```

Na co uważać:

Kod w bloku `#if ANDROID` istnieje **tylko** w wersji na Android - na innych platformach jest pomijany. To przydatne do drobnych różnic, ale nadużywanie pogarsza czytelność. Do typowych różnic lepiej użyć `OnPlatform`.

33.3. OnPlatform - różne wartości per platforma

`OnPlatform` pozwala podać **różne wartości właściwości** dla różnych platform - wprost w XAML, bez pisania kodu. Np. inny odstęp na iOS i Android.

PRZYKŁAD XAML

```

<!-- Inny padding na każdej platformie -->
<VerticalStackLayout>
    <VerticalStackLayout.Padding>
        <OnPlatform x:TypeArguments="Thickness"
                    Android="16" iOS="20" WinUI="24" />
    </VerticalStackLayout.Padding>
    <Label Text="Treść" />
</VerticalStackLayout>

```

```

<!-- Krótszy zapis inline -->
<Label FontSize="{OnPlatform Android=16, iOS=18, WinUI=20}" Text="Tekst" />

```

Na co uważać:

`OnPlatform` jest czytelniejszy niż `#if` dla różnic w **wartościach** właściwości (rozmiary, kolory, odstępy). Wymaga podania `x:TypeArguments` w wersji rozbudowanej.

33.4. OnIdiom - telefon, tablet, komputer

`OnIdiom` pozwala podać różne wartości w zależności od **typu urządzenia**: `Phone` (telefon), `Tablet`, `Desktop`. Przydatne do dostosowania układu - np. więcej kolumn na komputerze.

PRZYKŁAD XAML

```

<!-- Inny rozmiar czcionki na telefonie i komputerze -->
<Label Text="Nagłówek"
        FontSize="{OnIdiom Phone=20, Tablet=26, Desktop=32}" />

<!-- Inny padding -->
<ContentPage.Padding>
    <OnIdiom x:TypeArguments="Thickness" Phone="16" Desktop="40" />
</ContentPage.Padding>

```

PRZYKŁAD W C#

```
bool toTelefon = DeviceInfo.Current.Idiom == DeviceIdiom.Phone;
int kolumny = toTelefon ? 1 : 3; // więcej kolumn na większym ekranie
```

Na co uważać:

`OnIdiom` to klucz do **responsywności** między telefonem a komputerem. Projektuj pod telefon, a na większych ekranach wykorzystaj dodatkową przestrzeń (więcej kolumn, większe odstępy).

33.5. Partial classes dla kodu natywnego

Aby napisać kod natywny per platforma z jednym wspólnym interfejsem, używamy **klas częściowych** (`partial`): wspólna deklaracja w głównym projekcie, a implementacje w plikach platformowych. To zaawansowany mechanizm dla integracji z natywnymi API.

Na co uważać:

33.6. Typowe różnice między platformami

NAJWAŻNIEJSZE INFORMACJE

OBSZAR	RÓŻNICA MIĘDZY PLATFORMAMI
Wygląd kontrolek	natywny styl każdej platformy
Uprawnienia	inne nazwy i deklaracje
Ścieżki plików	różna struktura katalogów
Klawiatura	różne zachowanie i typy
Pasek statusu / safe area	różne wymiary i obszary bezpieczne
Nawigacja	drobne różnice w gestach

Na co uważać:

Te same kontrolki **wyglądają nieco inaczej** na każdej platformie - to celowe (natywny wygląd), nie błąd. Ścieżki plików zawsze buduj przez `FileSystem`, a nie ręcznie. Uważaj na „safe area” (np. notch w telefonach) - MAUI zwykle radzi sobie z tym automatycznie.

33.7. Testowanie na różnych platformach

Aplikację wieloplatformową warto **testować na każdej** docelowej platformie, bo drobne różnice (wygląd, uprawnienia, ścieżki) ujawniają się dopiero w praktyce. Minimum to test na Androidzie i Windows (jeśli oba są celem).

Na co uważać:

Nie zakładaj, że „skoro działa na Windows, zadziała wszędzie”. Testuj na realnych platformach docelowych. Szczególnie sprawdź uprawnienia (różnią się), pracę z plikami i wygląd na małym ekranie telefonu.

33.8. Typowe błędy

NAJWAŻNIEJSZE INFORMACJE

BŁĄD	SKUTEK	ROZWIĄZANIE
Nadużywanie <code>#if</code>	nieczytelny kod	użyj <code>OnPlatform</code> / <code>OnIdiom</code>
Sztuczne ścieżki plików	błąd na innej platformie	używaj <code>FileSystem</code>
Brak <code>x:TypeArguments</code> w <code>OnPlatform</code>	błąd XAML	dodaj typ
Test tylko na jednej platformie	ukryte błędy	testuj na wszystkich docelowych
Założenie identycznego wyglądu	zaskoczenie różnicami	akceptuj natywny styl

Na co uważać:

Do różnic w **wartościach** używaj `OnPlatform` / `OnIdiom` (czytelne, w XAML), a `#if` rezerwuj na różnice w **kodzie**. Zawsze testuj na docelowych platformach.

Większość aplikacji wymaga niewiele kodu specyficznego dla platformy. Do różnic w wyglądzie i rozmiarach używaj `OnPlatform` i `OnIdiom`, do drobnych różnic w logice - `#if`. Buduj responsywnie i testuj na realnych urządzeniach.

34. Debugowanie

Każdy program ma błędy - kluczowa jest umiejętność ich **znajdowania** (debugowanie) i **obsługi** (try/catch). Ten rozdział omawia narzędzia debugowania (breakpointy, podgląd zmiennych), debugowanie XAML i bindingu oraz obsługę wyjątków w typowych scenariuszach: konwersji, plików, API i SQLite.

34.1. Czym jest debugowanie

Debugowanie to proces **znajdowania i usuwania błędów**. Najważniejsze narzędzie to **breakpoint** (punkt wstrzymania) - zatrzymuje program w wybranej linii, byśmy mogli podejrzec wartości zmiennych i prześledzić działanie krok po kroku. Debugowanie odbywa się w trybie Debug.

NAJWAŻNIEJSZE INFORMACJE

- Breakpoint zatrzymuje program w wybranej linii.
- Po zatrzymaniu podglądamy zmienne i wykonujemy kod krok po kroku.
- Debugowanie działa w trybie **Debug** (pełne informacje diagnostyczne).

Na co uważać:

Debugowanie to podstawowa umiejętność - często szybciej znajdziesz błąd breakpointem niż zgadywaniem. Ustaw breakpoint przed podejrzanym fragmentem i prześledź wartości.

34.2. Breakpointy i kroki: Step Over, Step Into, Step Out

Po zatrzymaniu na breakpointie sterujemy wykonaniem: **Step Over** (wykonaj linię, nie wchodź do metod), **Step Into** (wejdź do wywoływanej metody), **Step Out** (wyjdź z bieżącej metody). Pozwala to dokładnie prześledzić przepływ.

NAJWAŻNIEJSZE INFORMACJE

AKCJA	DZIAŁANIE
Continue (F5)	wznów do kolejnego breakpointu
Step Over (F10)	wykonaj linię bez wchodzenia w metody
Step Into (F11)	wejdź do wywoływanej metody
Step Out	wyjdź z bieżącej metody

Na co uważać:

Step Over używaj, gdy ufasz wywoływanej metodzie; **Step Into**, gdy chcesz sprawdzić jej wnętrze. To pozwala skupić się na podejrzanym fragmencie, nie tracąc czasu na sprawdzony kod.

34.3. Podgląd zmiennych: Watch, Locals, Output

Podczas zatrzymania podglądamy wartości: **Locals** pokazuje zmienne lokalne, **Watch** pozwala obserwować wybrane wyrażenia, a **Output** wyświetla komunikaty diagnostyczne (w tym z `Debug.WriteLine`).

PRZYKŁAD C#

```
// Wypisanie wartości do okna Output (tylko Debug)
System.Diagnostics.Debug.WriteLine($"Licznik = {licznik}");
```

Na co uważać:

`Debug.WriteLine` to prosty sposób śledzenia wartości bez breakpointów - komunikaty trafiają do okna Output. Działa tylko w trybie Debug, więc nie spowalnia wersji Release.

34.4. Debugowanie XAML i bindingu

Błędy w XAML (zła nazwa, niezamknięty znacznik) zwykle zatrzymują budowanie z komunikatem wskazującym linię. Błędy **bindingu** są podstępniejsze - kontrolka jest pusta, choć kod się kompiluje. Pomaga okno Output, gdzie MAUI wypisuje ostrzeżenia o nieudanych powiązaniach.

Na co uważać:

Gdy binding „nic nie pokazuje”, sprawdź okno **Output** - często znajdziesz tam komunikat typu „binding could not be resolved” z nazwą problematycznej właściwości. Najczęstsze przyczyny: brak `BindingContext` lub literówka w ścieżce.

34.5. Obsługa wyjątków: try, catch, finally

Wyjątek to błąd w trakcie działania. Blok `try` zawiera kod mogący się nie udać, `catch` przechwytuje błąd i pozwala zareagować, a `finally` wykonuje się zawsze (np. do sprzątnięcia).

PRZYKŁAD C#

```
try
{
    int liczba = int.Parse(PoleWiek.Text); // może rzucić wyjątek
}
catch (FormatException)
{
    // konkretny rodzaj błędu
    await DisplayAlert("Błąd", "To nie jest liczba.", "OK");
}
catch (Exception ex)
{
    // dowolny inny błąd
    await DisplayAlert("Błąd", ex.Message, "OK");
}
finally
{
    Loader.IsRunning = false; // wykona się zawsze
}
```

Na co uważać:

Łap **konkretne** wyjątki przed ogólnym (`Exception`). Nie zostawiaj pustego `catch` - to ukrywa błędy. `finally` to dobre miejsce na czyszczenie (ukrycie wskaźnika ładowania).

34.6. Wyjątki przy konwersji, plikach, API i SQLite

NAJWAŻNIEJSZE INFORMACJE

OPERACJA	TYPOWY WYJĄTEK	ZABEZPIECZENIE
Konwersja tekstu na liczbę	<code>FormatException</code>	użyj <code>TryParse</code> zamiast <code>Parse</code>
Odczyt pliku	<code>FileNotFoundException</code>	sprawdź <code>File.Exists</code>
Żądanie API	<code>HttpRequestException</code>	<code>try/catch</code> , sprawdź sieć
Operacje SQLite	<code>SQLiteException</code>	<code>try/catch</code> , sprawdź model
Dostęp do <code>null</code>	<code>NullReferenceException</code>	sprawdź <code>null</code> , użyj <code>?.</code>

PRZYKŁAD C#

```
// Najlepiej UNIKAĆ wyjątku zamiast go łapać:
if (int.TryParse(PoleWiek.Text, out int wiek))
{
    // poprawna liczba
}

// zamiast: int wiek = int.Parse(...) w try/catch
```

Na co uważać:

Lepiej **zapobiegać** wyjątkom niż je łapać: `TryParse` zamiast `Parse`, `File.Exists` przed odczytem, sprawdzanie `null`. `try/catch` rezerwuj na sytuacje, których nie da się przewidzieć (sieć, dostęp do pliku).

34.7. Komunikaty dla użytkownika i logowanie błędów

Gdy wystąpi błąd, użytkownik powinien dostać **zrozumiały komunikat** (nie techniczny opis wyjątku), a programista - szczegóły w logu. Komunikat pokazujemy przez `DisplayAlert`, a szczegóły logujemy przez `Debug.WriteLine`.

PRZYKŁAD C#

```
catch (Exception ex)
{
    // Dla programisty - pełny błąd w logu
    System.Diagnostics.Debug.WriteLine($"BŁĄD: {ex}");
    // Dla użytkownika - prosty komunikat
    await DisplayAlert("Ups", "Coś poszło nie tak. Spróbuj ponownie.", "OK");
}
```

Na co uważać:

Nie pokazuj użytkownikowi surowego `ex.Message` z technicznymi szczegółami - bywa niezrozumiały i nieprofesjonalny. Pokaż prosty komunikat, a szczegóły załoguj dla siebie.

34.8. Typowe błędy początkujących

NAJWAŻNIEJSZE INFORMACJE

BŁĄD	OBJAW	ROZWIĄZANIE
Brak <code>InitializeComponent()</code>	kontrolki <code>null</code>	dodaj w konstruktorze
<code>int.Parse</code> na danych usera	wyjątek przy złym wpisie	<code>int.TryParse</code>
Brak <code>BindingContext</code>	puste kontrolki	ustaw kontekst
Pusty <code>catch</code>	ukryte błędy	reaguj lub loguj
<code>NullReferenceException</code>	aplikacja się zamyka	sprawdź <code>null</code> , <code>?.</code>
<code>List</code> zamiast <code>ObservableCollection</code>	lista bez odświeżania	użyj <code>ObservableCollection</code>

Na co uważać:

Najczęstszy wyjątek to `NullReferenceException` - odwołanie do obiektu, który jest `null`. Sprawdzaj `null` (operatory `?.` i `??`), zanim użyjesz obiektu. Drugi częsty błąd to `FormatException` z `Parse` - używaj `TryParse`.

Dobra strategia: **zapobiegaj** błędom (`TryParse`, `File.Exists`, sprawdzanie `null`), **łap** tylko nieprzewidywalne (sieć, pliki), **informuj** użytkownika prostym komunikatem i **loguj** szczegóły dla siebie. Breakpoint i okno Output to Twoi najlepsi pomocnicy przy szukaniu błędów.

35. Gotowe aplikacje - kompletne przykłady

35.1. Jak korzystać z gotowych aplikacji

Ten dział zawiera 30 kompletnych miniaplikacji w .NET MAUI. Każdy przykład ma pełny kod XAML oraz pełny code-behind w C#. Aplikacje są ułożone tematycznie: kalkulatory i przeliczniki, formularze, listy, obrazy i gry, pliki i dane lokalne, API oraz funkcje urządzenia.

35.2. Kalkulatory i przeliczniki

Te przykłady skupiają się na pobieraniu liczb z pól tekstowych, walidacji danych, prostych obliczeniach i czytelnym pokazaniu wyniku.

35.3. Kalkulator BMI

Aplikacja liczy wskaźnik BMI na podstawie masy i wzrostu. Ćwicz `Entry`, `Button`, `Label`, `TryParse`, instrukcje warunkowe i formatowanie wyniku.

```
<?xml version="1.0" encoding="utf-8" ?>
<ContentPage xmlns="http://schemas.microsoft.com/dotnet/2021/maui"
              xmlns:x="http://schemas.microsoft.com/winfx/2009/xaml"
              x:Class="Miniaplikacje.BmiPage"
              Title="Kalkulator BMI">
    <VerticalStackLayout Padding="24" Spacing="14">
        <Label Text="Kalkulator BMI" FontSize="26" FontAttributes="Bold" />

        <Label Text="Masa ciała (kg)" />
        <Entry x:Name="MasaEntry" Placeholder="np. 70" Keyboard="Numeric" />

        <Label Text="Wzrost (m)" />
        <Entry x:Name="WzrostEntry" Placeholder="np. 1.75" Keyboard="Numeric" />

        <Button Text="Oblicz" Clicked="OnOblicz" />

        <Label x:Name="WynikLabel" FontSize="20" FontAttributes="Bold" />
        <Label x:Name="OpisLabel" TextColor="Gray" />
    </VerticalStackLayout>
</ContentPage>
```

```
namespace Miniaplikacje;

public partial class BmiPage : ContentPage
{
    public BmiPage()
    {
        InitializeComponent();
    }

    private void OnOblicz(object sender, EventArgs e)
    {
        if (!double.TryParse(MasaEntry.Text, out double masa) || masa <= 0 ||
            !double.TryParse(WzrostEntry.Text, out double wzrost) || wzrost <= 0)
        {
            WynikLabel.Text = "Podaj poprawną masę i wzrost.";
            WynikLabel.TextColor = Colors.Red;
            OpisLabel.Text = "";
            return;
        }
    }
}
```

```

    }

    double bmi = masa / (wzrost * wzrost);
    string kategoria;
    Color kolor;

    if (bmi < 18.5) { kategoria = "niedowaga"; kolor = Colors.Orange; }
    else if (bmi < 25) { kategoria = "waga prawidłowa"; kolor = Colors.Green; }
    else if (bmi < 30) { kategoria = "nadwaga"; kolor = Colors.OrangeRed; }
    else { kategoria = "otyłość"; kolor = Colors.Red; }

    WynikLabel.Text = $"BMI: {bmi:0.0}";
    WynikLabel.TextColor = kolor;
    OpisLabel.Text = $"Kategoria: {kategoria}";
}
}

```

35.4. Kalkulator napiwku

Aplikacja oblicza napiwek, sumę rachunku i kwotę na osobę. Ćwiczysz **Slider**, dynamiczną aktualizację etykiet i obliczenia z wartościami liczbowymi.

```

<?xml version="1.0" encoding="utf-8" ?>
<ContentPage xmlns="http://schemas.microsoft.com/dotnet/2021/maui"
    xmlns:x="http://schemas.microsoft.com/winfx/2009/xaml"
    x:Class="Miniaplikacje.NapiwekPage"
    Title="Kalkulator napiwku">
    <VerticalStackLayout Padding="24" Spacing="14">
        <Label Text="Kalkulator napiwku" FontSize="26" FontAttributes="Bold" />

        <Entry x:Name="RachunekEntry" Placeholder="Kwota rachunku" Keyboard="Numeric" TextChanged="OnDaneZmienione" />

        <Label x:Name="ProcentLabel" Text="Napiwek: 10%" />
        <Slider x:Name="ProcentSlider" Minimum="0" Maximum="30" Value="10" ValueChanged="OnSuwakZmieniony" />

        <Label x:Name="OsobyLabel" Text="Liczba osób: 1" />
        <Stepper x:Name="OsobyStepper" Minimum="1" Maximum="10" Increment="1" Value="1" ValueChanged="OnSuwakZmieniony" />

        <BoxView HeightRequest="1" Color="LightGray" />
        <Label x:Name="NapiwekLabel" Text="Napiwek: 0,00 zł" />
        <Label x:Name="SumaLabel" Text="Suma: 0,00 zł" FontAttributes="Bold" />
        <Label x:Name="NaOsobeLabel" Text="Na osobę: 0,00 zł" TextColor="DarkBlue" />
    </VerticalStackLayout>
</ContentPage>

```

```

namespace Miniaplikacje;

public partial class NapiwekPage : ContentPage
{
    public NapiwekPage()
    {

```

```

        InitializeComponent();
        Przelicz();
    }

    private void OnDaneZmienione(object sender, TextChangedEventArgs e) => Przelicz();
    private void OnSuwakZmieniony(object sender, ValueChangedEventArgs e) => Przelicz();

    private void Przelicz()
    {
        int procent = (int)Math.Round(ProcentSlider.Value);
        int osoby = (int)OsobyStepper.Value;
        ProcentSlider.Value = procent;

        ProcentLabel.Text = $"Napiwek: {procent}%";
        OsobyLabel.Text = $"Liczba osób: {osoby}";

        if (!double.TryParse(RachunekEntry.Text, out double rachunek) || rachunek <= 0)
        {
            NapiwekLabel.Text = "Napiwek: 0,00 zł";
            SumaLabel.Text = "Suma: 0,00 zł";
            NaOsobeLabel.Text = "Na osobę: 0,00 zł";
            return;
        }

        double napiwek = rachunek * procent / 100.0;
        double suma = rachunek + napiwek;
        double naOsobe = suma / osoby;

        NapiwekLabel.Text = $"Napiwek: {napiwek:0.00} zł";
        SumaLabel.Text = $"Suma: {suma:0.00} zł";
        NaOsobeLabel.Text = $"Na osobę: {naOsobe:0.00} zł";
    }
}

```

35.5. Przelicznik jednostek

Aplikacja przelicza temperaturę, długość i masę. Ćwicz `Picker`, `Entry`, `switch`, walidację liczby i aktualizację wyniku.

```

<?xml version="1.0" encoding="utf-8" ?>
<ContentPage xmlns="http://schemas.microsoft.com/dotnet/2021/maui"
              xmlns:x="http://schemas.microsoft.com/winfx/2009/xaml"
              x:Class="Miniaplikacje.PrzelicznikPage"
              Title="Przelicznik jednostek">
    <VerticalStackLayout Padding="24" Spacing="14">
        <Label Text="Przelicznik jednostek" FontSize="26" FontAttributes="Bold" />

        <Picker x:Name="TrybPicker" Title="Wybierz przelicznik" SelectedIndexChanged="OnPrzelicz">
            <Picker.Items>
                <x:String>Kilometry na mile</x:String>
                <x:String>Mile na kilometry</x:String>
                <x:String>Celsjusz na Fahrenheit</x:String>
                <x:String>Kilogramy na funty</x:String>
            </Picker.Items>

```

```

</Picker>

<Entry x:Name="WartoscEntry" Placeholder="Wpisz wartość" Keyboard="Numeric" TextChanged="OnPrzelicz" />
<Label x:Name="WynikLabel" FontSize="22" FontAttributes="Bold" />
</VerticalStackLayout>
</ContentPage>

```

```

namespace Miniaplikacje;

public partial class PrzelicznikPage : ContentPage
{
    public PrzelicznikPage()
    {
        InitializeComponent();
        TrybPicker.SelectedIndex = 0;
    }

    private void OnPrzelicz(object sender, EventArgs e)
    {
        if (TrybPicker.SelectedIndex < 0 || !double.TryParse(WartoscEntry.Text, out double x))
        {
            WynikLabel.Text = "Wpisz liczbę.";
            WynikLabel.TextColor = Colors.Red;
            return;
        }

        double wynik = TrybPicker.SelectedIndex switch
        {
            0 => x * 0.621371,
            1 => x / 0.621371,
            2 => x * 9 / 5 + 32,
            3 => x * 2.20462,
            _ => x
        };

        WynikLabel.TextColor = Colors.Black;
        WynikLabel.Text = $"Wynik: {wynik:0.##}";
    }
}

```

35.6. Kalkulator zamówienia pizzy

Aplikacja liczy cenę zamówienia na podstawie rozmiaru, dodatków i liczby sztuk. Ćwicz [RadioButton](#), [CheckBox](#), [Stepper](#) i dynamiczne podsumowanie.

```

<?xml version="1.0" encoding="utf-8" ?>
<ContentPage xmlns="http://schemas.microsoft.com/dotnet/2021/maui"
    xmlns:x="http://schemas.microsoft.com/winfx/2009/xaml"
    x:Class="Miniaplikacje.PizzaPage"
    Title="Zamówienie pizzy">
    <ScrollView>

```

```

<VerticalStackLayout Padding="24" Spacing="12">
    <Label Text="Zamówienie pizzzy" FontSize="26" FontAttributes="Bold" />

    <Label Text="Rozmiar" FontAttributes="Bold" />
    <RadioButton x:Name="MalaRadio" Content="Mała - 22 zł" GroupName="rozmiar" CheckedChanged="OnZmieniono" />
    <RadioButton x:Name="SredniaRadio" Content="Średnia - 30 zł" GroupName="rozmiar" IsChecked="True"
CheckedChanged="OnZmieniono" />
    <RadioButton x:Name="DuzaRadio" Content="Duża - 38 zł" GroupName="rozmiar" CheckedChanged="OnZmieniono" />

    <Label Text="Dodatki po 4 zł" FontAttributes="Bold" Margin="0,10,0,0" />
    <CheckBox x:Name="SerCheck" CheckedChanged="OnZmieniono" />
    <Label Text="Dodatkowy ser" />
    <CheckBox x:Name="PieczarkiCheck" CheckedChanged="OnZmieniono" />
    <Label Text="Pieczarki" />
    <CheckBox x:Name="KukurydzaCheck" CheckedChanged="OnZmieniono" />
    <Label Text="Kukurydza" />

    <Label x:Name="IloscLabel" Text="Liczba sztuk: 1" FontAttributes="Bold" />
    <Stepper x:Name="IloscStepper" Minimum="1" Maximum="10" Value="1" ValueChanged="OnZmieniono" />

    <Label x:Name="SumaLabel" FontSize="22" FontAttributes="Bold" TextColor="DarkGreen" />
</VerticalStackLayout>
</ScrollView>
</ContentPage>

```

```

namespace Miniaplikacje;

public partial class PizzaPage : ContentPage
{
    public PizzaPage()
    {
        InitializeComponent();
        Przelicz();
    }

    private void OnZmieniono(object sender, EventArgs e) => Przelicz();

    private void Przelicz()
    {
        int cena = 30;
        if (MalaRadio.IsChecked) cena = 22;
        if (DuzaRadio.IsChecked) cena = 38;

        int dodatki = 0;
        if (SerCheck.IsChecked) dodatki++;
        if (PieczarkiCheck.IsChecked) dodatki++;
        if (KukurydzaCheck.IsChecked) dodatki++;

        int ilosc = (int)IloscStepper.Value;
        IloscLabel.Text = $"Liczba sztuk: {ilosc}";

        int suma = (cena + dodatki * 4) * ilosc;
    }
}

```



```

        SumaLabel.Text = $"Do zapłaty: {suma} zł";
    }
}

```

35.7. Generator hasła

Aplikacja generuje losowe hasło o wybranej długości. Ćwicz [Slider](#), [CheckBox](#), [Random](#), walidację opcji i pracę z tekstem.

```

<?xml version="1.0" encoding="utf-8" ?>
<ContentPage xmlns="http://schemas.microsoft.com/dotnet/2021/maui"
    xmlns:x="http://schemas.microsoft.com/winfx/2009/xaml"
    x:Class="Miniaplikacje.HasloPage"
    Title="Generator hasła">
    <VerticalStackLayout Padding="24" Spacing="14">
        <Label Text="Generator hasła" FontSize="26" FontAttributes="Bold" />
        <Label x:Name="DlugoscLabel" Text="Długość: 12" />
        <Slider x:Name="DlugoscSlider" Minimum="6" Maximum="24" Value="12" ValueChanged="OnDlugosc" />

        <HorizontalStackLayout><CheckBox x:Name="DuzeCheck" IsChecked="True" /><Label Text="Wielkie litery"
VerticalOptions="Center" /></HorizontalStackLayout>
        <HorizontalStackLayout><CheckBox x:Name="CyfryCheck" IsChecked="True" /><Label Text="Cyfry" VerticalOptions="Center" />
</HorizontalStackLayout>
        <HorizontalStackLayout><CheckBox x:Name="ZnakiCheck" /><Label Text="Znaki specjalne" VerticalOptions="Center" />
</HorizontalStackLayout>

        <Button Text="Generuj" Clicked="OnGeneruj" />
        <Label x:Name="HasloLabel" FontSize="22" FontAttributes="Bold" LineBreakMode="CharacterWrap" />
    </VerticalStackLayout>
</ContentPage>

```

```

namespace Miniaplikacje;

public partial class HasloPage : ContentPage
{
    readonly Random los = new Random();

    public HasloPage()
    {
        InitializeComponent();
    }

    private void OnDlugosc(object sender, ValueChangedEventArgs e)
    {
        DlugoscLabel.Text = $"Długość: {(int)e.NewValue}";
    }

    private async void OnGeneruj(object sender, EventArgs e)
    {
        string znaki = "abcdefghijklmnopqrstuvwxyz";
        if (DuzeCheck.IsChecked) znaki += "ABCDEFGHIJKLMNOPQRSTUVWXYZ";
        if (CyfryCheck.IsChecked) znaki += "0123456789";
    }
}

```

```

        if (ZnakiCheck.IsChecked) znaki += "!@#$$%";

        if (string.IsNullOrEmpty(znaki))
        {
            await DisplayAlert("Błąd", "Wybierz przynajmniej jedną grupę znaków.", "OK");
            return;
        }

        int dlugosc = (int)DlugoscSlider.Value;
        char[] haslo = new char[dlugosc];
        for (int i = 0; i < haslo.Length; i++)
            haslo[i] = znaki[los.Next(znaki.Length)];

        HasloLabel.Text = new string(haslo);
    }
}

```

35.8. Formularze i rezerwacje

W tej grupie najważniejsze są dane wpisywane przez użytkownika, wybory z list, pola wyboru, komunikaty i budowanie podsumowania.

35.9. Formularz rejestracji

Aplikacja sprawdza e-mail, hasło, zgodność haseł i akceptację regulaminu. Ćwiczy walidację formularza i komunikaty w etykiecie.

```

<?xml version="1.0" encoding="utf-8" ?>
<ContentPage xmlns="http://schemas.microsoft.com/dotnet/2021/maui"
    xmlns:x="http://schemas.microsoft.com/winfx/2009/xaml"
    x:Class="Miniaplikacje.RejestracjaPage"
    Title="Rejestracja">
    <ScrollView>
        <VerticalStackLayout Padding="24" Spacing="12">
            <Label Text="Rejestracja" FontSize="26" FontAttributes="Bold" />
            <Entry x:Name="EmailEntry" Placeholder="E-mail" Keyboard="Email" />
            <Entry x:Name="HasloEntry" Placeholder="Hasło" IsPassword="True" />
            <Entry x:Name="PowtorzEntry" Placeholder="Powtórz hasło" IsPassword="True" />
            <HorizontalStackLayout>
                <CheckBox x:Name="RegulaminCheck" />
                <Label Text="Akceptuję regulamin" VerticalOptions="Center" />
            </HorizontalStackLayout>
            <Button Text="Zarejestruj" Clicked="OnZarejestruj" />
            <Label x:Name="KomunikatLabel" FontAttributes="Bold" />
        </VerticalStackLayout>
    </ScrollView>
</ContentPage>

```

```

namespace Miniaplikacje;

public partial class RejestracjaPage : ContentPage
{
    public RejestracjaPage()
    {

```

```

        InitializeComponent();
    }

    private void OnZarejestruj(object sender, EventArgs e)
    {
        string email = EmailEntry.Text ?? "";
        string haslo = HasloEntry.Text ?? "";
        string powtorz = PowtorzEntry.Text ?? "";

        if (string.IsNullOrEmpty(email) || string.IsNullOrEmpty(haslo) || string.IsNullOrEmpty(powtorz))
        { Pokaz("Uzupełnij wszystkie pola.", false); return; }

        if (!email.Contains('@'))
        { Pokaz("Adres e-mail musi zawierać znak @.", false); return; }

        if (haslo.Length < 6)
        { Pokaz("Hasło musi mieć co najmniej 6 znaków.", false); return; }

        if (haslo != powtorz)
        { Pokaz("Hasła nie są takie same.", false); return; }

        if (!RegulaminCheck.IsChecked)
        { Pokaz("Zaakceptuj regulamin.", false); return; }

        Pokaz("Konto zostało utworzone.", true);
    }

    private void Pokaz(string tekst, bool ok)
    {
        KomunikatLabel.Text = tekst;
        KomunikatLabel.TextColor = ok ? Colors.Green : Colors.Red;
    }
}

```

35.10. Rezerwacja wizyty

Aplikacja tworzy podsumowanie rezerwacji na podstawie wyboru specjalisty, daty, godziny i danych klienta. Ćwicz [Picker](#), [DatePicker](#), [TimePicker](#) i walidację.

```

<?xml version="1.0" encoding="utf-8" ?>
<ContentPage xmlns="http://schemas.microsoft.com/dotnet/2021/maui"
              xmlns:x="http://schemas.microsoft.com/winfx/2009/xaml"
              x:Class="Miniaplikacje.RezerwacjaPage"
              Title="Rezerwacja wizyty">
    <ScrollView>
        <VerticalStackLayout Padding="24" Spacing="12">
            <Label Text="Rezerwacja wizyty" FontSize="26" FontAttributes="Bold" />
            <Entry x:Name="ImieEntry" Placeholder="Imię i nazwisko" />
            <Picker x:Name="SpecjalistaPicker" Title="Wybierz specjalistę">
                <Picker.Items>
                    <x:String>Anna Nowak</x:String>
                    <x:String>Jan Kowalski</x:String>
                    <x:String>Marta Zielińska</x:String>
                </Picker.Items>
            </Picker>
        </VerticalStackLayout>
    </ScrollView>
</ContentPage>

```

```

        </Picker.Items>
    </Picker>
    <DatePicker x:Name="DatePicker" Format="dd.MM.yyyy" />
    <TimePicker x:Name="GodzinaPicker" Format="HH:mm" />
    <Button Text="Zarezerwuj" Clicked="OnRezerwuj" />
    <Label x:Name="PodsumowanieLabel" LineBreakMode="WordWrap" />
</VerticalStackLayout>
</ScrollView>
</ContentPage>

```

```

namespace Miniaplikacje;

public partial class RezerwacjaPage : ContentPage
{
    public RezerwacjaPage()
    {
        InitializeComponent();
        DatePicker.MinimumDate = DateTime.Today;
    }

    private async void OnRezerwuj(object sender, EventArgs e)
    {
        if (string.IsNullOrWhiteSpace(ImieEntry.Text))
        {
            await DisplayAlert("Błąd", "Podaj imię i nazwisko.", "OK");
            return;
        }

        if (SpecjalistaPicker.SelectedIndex < 0)
        {
            await DisplayAlert("Błąd", "Wybierz specjalistę.", "OK");
            return;
        }

        PodsumowanieLabel.Text =
            $"Rezerwacja dla: {ImieEntry.Text}\n" +
            $"Specjalista: {SpecjalistaPicker.SelectedItem}\n" +
            $"Data: {DatePicker.Date:dd.MM.yyyy}\n" +
            $"Godzina: {GodzinaPicker.Time:hh\\:mm}";
    }
}

```

35.11. Ankieta satysfakcji

Aplikacja zbiera ocenę, zaznaczone zalety i komentarz. Ćwicz `RadioButton`, `CheckBox`, `Editor`, składanie tekstu i `DisplayAlert`.

```

<?xml version="1.0" encoding="utf-8" ?>

<ContentPage xmlns="http://schemas.microsoft.com/dotnet/2021/maui"
              xmlns:x="http://schemas.microsoft.com/winfx/2009/xaml"
              x:Class="Miniaplikacje.AnkietaPage"

```

```

        Title="Ankieta">
<ScrollView>
    <VerticalStackLayout Padding="24" Spacing="10">
        <Label Text="Ankieta satysfakcji" FontSize="26" FontAttributes="Bold" />

        <Label Text="Ocena" FontAttributes="Bold" />
        <RadioButton x:Name="Ocena5" Content="5 - bardzo dobra" GroupName="ocena" IsChecked="True" />
        <RadioButton x:Name="Ocena4" Content="4 - dobra" GroupName="ocena" />
        <RadioButton x:Name="Ocena3" Content="3 - przeciętna" GroupName="ocena" />

        <Label Text="Co było dobre?" FontAttributes="Bold" />
        <HorizontalStackLayout><CheckBox x:Name="JakoscCheck" /><Label Text="Jakość" VerticalOptions="Center" />
    </HorizontalStackLayout>

    <HorizontalStackLayout><CheckBox x:Name="CenaCheck" /><Label Text="Cena" VerticalOptions="Center" />
</HorizontalStackLayout>

    <HorizontalStackLayout><CheckBox x:Name="ObslugaCheck" /><Label Text="Obsługa" VerticalOptions="Center" />
</HorizontalStackLayout>

        <Editor x:Name="KomentarzEditor" Placeholder="Komentarz" HeightRequest="100" />
        <Button Text="Wyślij" Clicked="OnWyslij" />
    </VerticalStackLayout>
</ScrollView>
</ContentPage>

```

```

namespace Miniaplikacje;

public partial class AnkietaPage : ContentPage
{
    public AnkietaPage()
    {
        InitializeComponent();
    }

    private async void OnWyslij(object sender, EventArgs e)
    {
        string ocena = Ocena5.IsChecked ? "5" : Ocena4.IsChecked ? "4" : "3";
        List<string> zalety = new();
        if (JakoscCheck.IsChecked) zalety.Add("jakość");
        if (CenaCheck.IsChecked) zalety.Add("cena");
        if (ObslugaCheck.IsChecked) zalety.Add("obsługa");

        string tekst = $"Ocena: {ocena}\n" +
            $"Zalety: {(zalety.Count == 0 ? "brak" : string.Join(", ", zalety))}\n" +
            $"Komentarz: {KomentarzEditor.Text}";

        await DisplayAlert("Podsumowanie", tekst, "OK");
    }
}

```

35.12. Formularz kontaktowy

Aplikacja sprawdza dane kontaktowe i treść wiadomości. Ćwiczy **Entry**, **Editor**, **Picker**, licznik znaków oraz blokowanie przycisku.

```
<?xml version="1.0" encoding="utf-8" ?>
<ContentPage xmlns="http://schemas.microsoft.com/dotnet/2021/maui"
    xmlns:x="http://schemas.microsoft.com/winfx/2009/xaml"
    x:Class="Miniaplikacje.KontaktPage"
    Title="Kontakt">
    <ScrollView>
        <VerticalStackLayout Padding="24" Spacing="12">
            <Label Text="Formularz kontaktowy" FontSize="26" FontAttributes="Bold" />
            <Entry x:Name="EmailEntry" Placeholder="E-mail" Keyboard="Email" TextChanged="OnZmieniono" />
            <Picker x:Name="TematPicker" Title="Temat" SelectedIndexChanged="OnZmieniono">
                <Picker.Items>
                    <x:String>Pytanie</x:String>
                    <x:String>Reklamacja</x:String>
                    <x:String>Inna sprawa</x:String>
                </Picker.Items>
            </Picker>
            <Editor x:Name="TrescEditor" Placeholder="Treść wiadomości" HeightRequest="120" TextChanged="OnZmieniono" />
            <Label x:Name="LicznikLabel" Text="Znaków: 0" TextColor="Gray" />
            <Button x:Name="WyslijButton" Text="Wyślij" Clicked="OnWyslij" IsEnabled="False" />
            <Label x:Name="KomunikatLabel" />
        </VerticalStackLayout>
    </ScrollView>
</ContentPage>
```

```
namespace Miniaplikacje;

public partial class KontaktPage : ContentPage
{
    public KontaktPage()
    {
        InitializeComponent();
    }

    private void OnZmieniono(object sender, EventArgs e)
    {
        int znaki = TrescEditor.Text?.Length ?? 0;
        LicznikLabel.Text = $"Znaków: {znaki}";

        bool ok = !string.IsNullOrEmpty(EmailEntry.Text) &&
            EmailEntry.Text.Contains('@') &&
            TematPicker.SelectedIndex ≥ 0 &&
            znaki ≥ 10;

        WyslijButton.IsEnabled = ok;
    }

    private void OnWyslij(object sender, EventArgs e)
    {
    }
```

```

        KomunikatLabel.Text = $"Wysłano wiadomość: {TematPicker.SelectedItem}";
        KomunikatLabel.TextColor = Colors.Green;
    }
}

```

35.13. Zamówienie biletu

Aplikacja wylicza koszt biletu na podstawie typu biletu, liczby sztuk i opcji dodatkowych. Ćwicz **Picker**, **Stepper**, **CheckBox** i podsumowanie.

```

<?xml version="1.0" encoding="utf-8" ?>
<ContentPage xmlns="http://schemas.microsoft.com/dotnet/2021/maui"
    xmlns:x="http://schemas.microsoft.com/winfx/2009/xaml"
    x:Class="Miniaplikacje.BiletPage"
    Title="Zamówienie biletu">
    <VerticalStackLayout Padding="24" Spacing="12">
        <Label Text="Zamówienie biletu" FontSize="26" FontAttributes="Bold" />
        <Picker x:Name="BiletPicker" Title="Rodzaj biletu" SelectedIndexChanged="OnZmieniono">
            <Picker.Items>
                <x:String>Normalny - 40 zł</x:String>
                <x:String>Ulgowy - 25 zł</x:String>
                <x:String>VIP - 80 zł</x:String>
            </Picker.Items>
        </Picker>

        <Label x:Name="IloscLabel" Text="Liczba biletów: 1" />
        <Stepper x:Name="IloscStepper" Minimum="1" Maximum="8" Value="1" ValueChanged="OnZmieniono" />

        <HorizontalStackLayout><CheckBox x:Name="ParkingCheck" CheckedChanged="OnZmieniono" /><Label Text="Parking +15 zł"
VerticalOptions="Center" /></HorizontalStackLayout>

        <HorizontalStackLayout><CheckBox x:Name="UbezpieczenieCheck" CheckedChanged="OnZmieniono" /><Label Text="Ubezpieczenie +5
zł" VerticalOptions="Center" /></HorizontalStackLayout>

        <Label x:Name="SumaLabel" FontSize="22" FontAttributes="Bold" />
        <Button Text="Złóż zamówienie" Clicked="OnZamow" />
    </VerticalStackLayout>
</ContentPage>

```

```

namespace Miniaplikacje;

public partial class BiletPage : ContentPage
{
    public BiletPage()
    {
        InitializeComponent();
        BiletPicker.SelectedIndex = 0;
        Przelicz();
    }

    private void OnZmieniono(object sender, EventArgs e) => Przelicz();
}

```

```

private void Przelicz()
{
    int cena = BiletPicker.SelectedIndex switch { 0 => 40, 1 => 25, 2 => 80, _ => 0 };
    int ilosc = (int)IloscStepper.Value;
    int dodatki = (ParkingCheck.IsChecked ? 15 : 0) + (UbezpieczenieCheck.IsChecked ? 5 : 0);
    IloscLabel.Text = $"Liczba biletów: {ilosc}";
    SumaLabel.Text = $"Do zapłaty: {(cena * ilosc) + dodatki} zł";
}

private async void OnZamow(object sender, EventArgs e)
{
    await DisplayAlert("Zamówienie", SumaLabel.Text, "OK");
}
}

```

35.14. Listy i kolekcje

Te aplikacje pokazują pracę z listami obiektów, dodawanie elementów, usuwanie, odświeżanie widoku i proste podsumowania.

35.15. Lista zakupów

Aplikacja pozwala dodawać i usuwać produkty z listy zakupów. Ćwicz `ObservableCollection`, `CollectionView`, `EmptyView` i `BindingContext` przycisku w szablonie.

```

<?xml version="1.0" encoding="utf-8" ?>
<ContentPage xmlns="http://schemas.microsoft.com/dotnet/2021/maui"
    xmlns:x="http://schemas.microsoft.com/winfx/2009/xaml"
    x:Class="Miniaplikacje.ZakupyPage"
    Title="Lista zakupów">
    <Grid Padding="20" RowDefinitions="Auto,Auto,*" RowSpacing="12">
        <HorizontalStackLayout Spacing="10">
            <Entry x:Name="ProduktEntry" Placeholder="Produkt" WidthRequest="220" />
            <Button Text="Dodaj" Clicked="OnDodaj" />
        </HorizontalStackLayout>
        <Label x:Name="LicznikLabel" Grid.Row="1" Text="Produktów: 0" />
        <CollectionView x:Name="ListaView" Grid.Row="2">
            <CollectionView.EmptyView>
                <Label Text="Lista jest pusta" HorizontalOptions="Center" VerticalOptions="Center" />
            </CollectionView.EmptyView>
            <CollectionView.ItemTemplate>
                <DataTemplate>
                    <Grid Padding="8" ColumnDefinitions="*,Auto">
                        <Label Text="{Binding .}" VerticalOptions="Center" />
                        <Button Grid.Column="1" Text="Usuń" Clicked="OnUsun" />
                    </Grid>
                </DataTemplate>
            </CollectionView.ItemTemplate>
        </CollectionView>
    </Grid>
</ContentPage>

```



```

using System.Collections.ObjectModel;

namespace Miniaplikacje;

public partial class ZakupyPage : ContentPage
{
    ObservableCollection<string> produkty = new();

    public ZakupyPage()
    {
        InitializeComponent();
        ListView.ItemsSource = produkty;
        Odswiez();
    }

    private void OnDodaj(object sender, EventArgs e)
    {
        string tekst = ProduktEntry.Text?.Trim();
        if (string.IsNullOrEmpty(tekst)) return;
        produkty.Add(tekst);
        ProduktEntry.Text = "";
        Odswiez();
    }

    private void OnUsun(object sender, EventArgs e)
    {
        if (sender is Button btn && btn.BindingContext is string produkt)
            produkty.Remove(produkt);
        Odswiez();
    }

    private void Odswiez()
    {
        LicznikLabel.Text = $"Produktów: {produkty.Count}";
    }
}

```

35.16. Lista notatek

Aplikacja przechowuje krótkie notatki w pamięci i pokazuje je w `CollectionView`. Ćwiczy dodawanie, czyszczenie pola i usuwanie elementów.

```

<?xml version="1.0" encoding="utf-8" ?>
<ContentPage xmlns="http://schemas.microsoft.com/dotnet/2021/maui"
    xmlns:x="http://schemas.microsoft.com/winfx/2009/xaml"
    x:Class="Miniaplikacje.NotatkiPage"
    Title="Notatki">
    <Grid Padding="20" RowDefinitions="Auto,*" RowSpacing="12">
        <HorizontalStackLayout Spacing="10">
            <Entry x:Name="NotatkaEntry" Placeholder="Treść notatki" WidthRequest="240" />
            <Button Text="Dodaj" Clicked="OnDodaj" />
        </HorizontalStackLayout>
    </Grid>
</ContentPage>

```

```

<CollectionView x:Name="NotatkiView" Grid.Row="1">
    <CollectionView.ItemTemplate>
        <DataTemplate>
            <Grid Padding="8" ColumnDefinitions="*,Auto">
                <Label Text="{Binding .}" />
                <Button Grid.Column="1" Text="Usuń" Clicked="OnUsun" />
            </Grid>
        </DataTemplate>
    </CollectionView.ItemTemplate>
</CollectionView>
</Grid>
</ContentPage>

```

```

using System.Collections.ObjectModel;

namespace Miniaplikacje;

public partial class NotatkiPage : ContentPage
{
    ObservableCollection<string> notatki = new();

    public NotatkiPage()
    {
        InitializeComponent();
        NotatkiView.ItemsSource = notatki;
    }

    private void OnDodaj(object sender, EventArgs e)
    {
        if (string.IsNullOrEmpty(NotatkaEntry.Text)) return;
        notatki.Add(NotatkaEntry.Text.Trim());
        NotatkaEntry.Text = "";
    }

    private void OnUsun(object sender, EventArgs e)
    {
        if (sender is Button btn && btn.BindingContext is string notatka)
            notatki.Remove(notatka);
    }
}

```

35.17. Planer zadań

Aplikacja tworzy listę zadań i oznacza je jako wykonane. Ćwiczy listę obiektów, `CheckBox` w `DataTemplate` i odświeżanie podsumowania.

```

<?xml version="1.0" encoding="utf-8" ?>
<ContentPage xmlns="http://schemas.microsoft.com/dotnet/2021/maui"
    xmlns:x="http://schemas.microsoft.com/winfx/2009/xaml"
    x:Class="Miniaplikacje.ZadaniaPage"
    Title="Planer zadań">

```

```

<Grid Padding="20" RowDefinitions="Auto,Auto,*" RowSpacing="12">
    <HorizontalStackLayout Spacing="10">
        <Entry x:Name="ZadanieEntry" Placeholder="Nowe zadanie" WidthRequest="230" />
        <Button Text="Dodaj" Clicked="OnDodaj" />
    </HorizontalStackLayout>
    <Label x:Name="PodsumowanieLabel" Grid.Row="1" />
    <CollectionView x:Name="ZadaniaView" Grid.Row="2">
        <CollectionView.ItemTemplate>
            <DataTemplate>
                <Grid Padding="8" ColumnDefinitions="Auto,*,Auto">
                    <CheckBox CheckedChanged="OnWykonane" IsChecked="{Binding Wykonane}" />
                    <Label Grid.Column="1" Text="{Binding Tytul}" VerticalOptions="Center" />
                    <Button Grid.Column="2" Text="Usuń" Clicked="OnUsun" />
                </Grid>
            </DataTemplate>
        </CollectionView.ItemTemplate>
    </CollectionView>
</Grid>
</ContentPage>

```

```

using System.Collections.ObjectModel;

namespace Miniaplikacje;

public class ZadanieProste
{
    public string Tytul { get; set; }
    public bool Wykonane { get; set; }
}

public partial class ZadaniaPage : ContentPage
{
    ObservableCollection<ZadanieProste> zadania = new();

    public ZadaniaPage()
    {
        InitializeComponent();
        ZadaniaView.ItemsSource = zadania;
        Odswiez();
    }

    private void OnDodaj(object sender, EventArgs e)
    {
        if (string.IsNullOrEmpty(ZadanieEntry.Text)) return;
        zadania.Add(new ZadanieProste { Tytul = ZadanieEntry.Text.Trim() });
        ZadanieEntry.Text = "";
        Odswiez();
    }

    private void OnWykonane(object sender, CheckedChangedEventArgs e)
    {
        if (sender is CheckBox cb && cb.BindingContext is ZadanieProste z)

```

```

        z.Wykonane = e.Value;
        Odswiez();
    }

    private void OnUsun(object sender, EventArgs e)
    {
        if (sender is Button btn && btn.BindingContext is ZadanieProste z)
            zadania.Remove(z);

        Odswiez();
    }

    private void Odswiez()
    {
        int wykonane = zadania.Count(z => z.Wykonane);
        PodsumowanieLabel.Text = $"Zadań: {zadania.Count}, wykonane: {wykonane}";
    }
}

```

35.18. Dziennik wydatków

Aplikacja dodaje wydatki z nazwą i kwotą, pokazuje listę oraz sumę. Ćwicz listę obiektów, walidację liczby i `StringFormat` w bindingu.

```

<?xml version="1.0" encoding="utf-8" ?>
<ContentPage xmlns="http://schemas.microsoft.com/dotnet/2021/maui"
              xmlns:x="http://schemas.microsoft.com/winfx/2009/xaml"
              x:Class="Miniaplikacje.WydatkiPage"
              Title="Dziennik wydatków">
    <Grid Padding="20" RowDefinitions="Auto,Auto,*" RowSpacing="12">
        <VerticalStackLayout Spacing="8">
            <Entry x:Name="NazwaEntry" Placeholder="Nazwa wydatku" />
            <Entry x:Name="KwotaEntry" Placeholder="Kwota" Keyboard="Numeric" />
            <Button Text="Dodaj wydatek" Clicked="OnDodaj" />
        </VerticalStackLayout>
        <Label x:Name="SumaLabel" Grid.Row="1" FontSize="20" FontAttributes="Bold" />
        <CollectionView x:Name="WydatkiView" Grid.Row="2">
            <CollectionView.ItemTemplate>
                <DataTemplate>
                    <Grid Padding="8" ColumnDefinitions="*,Auto">
                        <Label Text="{Binding Nazwa}" />
                        <Label Grid.Column="1" Text="{Binding Kwota, StringFormat='{0:0.00} zł'}" TextColor="DarkRed" />
                    </Grid>
                </DataTemplate>
            </CollectionView.ItemTemplate>
        </CollectionView>
    </Grid>
</ContentPage>

```

```

using System.Collections.ObjectModel;

namespace Miniaplikacje;

```

```

public class Wydatek
{
    public string Nazwa { get; set; }
    public double Kwota { get; set; }
}

public partial class WydatkiPage : ContentPage
{
    ObservableCollection<Wydatek> wydatki = new();

    public WydatkiPage()
    {
        InitializeComponent();
        WydatkiView.ItemsSource = wydatki;
        Odswiez();
    }

    private async void OnDodaj(object sender, EventArgs e)
    {
        if (string.IsNullOrEmpty(NazwaEntry.Text) || !double.TryParse(KwotaEntry.Text, out double kwota) || kwota <= 0)
        {
            await DisplayAlert("Błąd", "Podaj nazwę i poprawną kwotę.", "OK");
            return;
        }

        wydatki.Add(new Wydatek { Nazwa = NazwaEntry.Text.Trim(), Kwota = kwota });
        NazwaEntry.Text = KwotaEntry.Text = "";
        Odswiez();
    }

    private void Odswiez()
    {
        SumaLabel.Text = $"Suma: {wydatki.Sum(w => w.Kwota):0.00} zł";
    }
}

```

35.19. Licznik punktów graczy

Aplikacja prowadzi wynik dwóch graczy. Ćwicz pola stanu, kilka przycisków, reset i prostą logikę wyłaniania prowadzącego.

```

<?xml version="1.0" encoding="utf-8" ?>
<ContentPage xmlns="http://schemas.microsoft.com/dotnet/2021/maui"
    xmlns:x="http://schemas.microsoft.com/winfx/2009/xaml"
    x:Class="Miniaplikacje.PunktyPage"
    Title="Punkty graczy">
    <VerticalStackLayout Padding="24" Spacing="18">
        <Label Text="Licznik punktów" FontSize="26" FontAttributes="Bold" HorizontalOptions="Center" />

        <Grid ColumnDefinitions="*,*" ColumnSpacing="12">
            <VerticalStackLayout Spacing="10">
                <Label Text="Gracz A" FontSize="20" HorizontalOptions="Center" />
                <Label x:Name="PunktyALabel" Text="0" FontSize="42" HorizontalOptions="Center" />
                <Button Text="+1" Clicked="OnDodajA" />
            </VerticalStackLayout>
        </Grid>
    </VerticalStackLayout>
</ContentPage>

```

```

        <Button Text="-1" Clicked="OnOdejmijA" />
    </VerticalStackLayout>
    <VerticalStackLayout Grid.Column="1" Spacing="10">
        <Label Text="Gracz B" FontSize="20" HorizontalOptions="Center" />
        <Label x:Name="PunktyBLabel" Text="0" FontSize="42" HorizontalOptions="Center" />
        <Button Text="+1" Clicked="OnDodajB" />
        <Button Text="-1" Clicked="OnOdejmijB" />
    </VerticalStackLayout>
</Grid>

<Label x:Name="StatusLabel" HorizontalOptions="Center" FontAttributes="Bold" />
<Button Text="Reset" Clicked="OnReset" />
</VerticalStackLayout>
</ContentPage>

```

```

namespace Miniaplikacje;

public partial class PunktyPage : ContentPage
{
    int a = 0;
    int b = 0;

    public PunktyPage()
    {
        InitializeComponent();
        Odswiez();
    }

    private void OnDodajA(object sender, EventArgs e) { a++; Odswiez(); }
    private void OnOdejmijA(object sender, EventArgs e) { if (a > 0) a--; Odswiez(); }
    private void OnDodajB(object sender, EventArgs e) { b++; Odswiez(); }
    private void OnOdejmijB(object sender, EventArgs e) { if (b > 0) b--; Odswiez(); }
    private void OnReset(object sender, EventArgs e) { a = b = 0; Odswiez(); }

    private void Odswiez()
    {
        PunktyALabel.Text = a.ToString();
        PunktyBLabel.Text = b.ToString();
        StatusLabel.Text = a == b ? "Remis" : a > b ? "Prowadzi gracz A" : "Prowadzi gracz B";
    }
}

```

35.20. Obrazy, gry i interakcje

Tutaj pojawiają się obrazki, losowanie, quizy, reagowanie na kliknięcia i zmiana wyglądu interfejsu w trakcie działania aplikacji.

35.21. Gra w kości

Aplikacja losuje pięć kości i podmienia obrazy na podstawie wyniku. Wymaga plików `kostka1.png` do `kostka6.png` w `Resources/Images`.

```

<?xml version="1.0" encoding="utf-8" ?>
<ContentPage xmlns="http://schemas.microsoft.com/dotnet/2021/maui"
              xmlns:x="http://schemas.microsoft.com/winfx/2009/xaml"
              x:Class="Miniaplikacje.KosciPage"
              Title="Gra w kości">
  <VerticalStackLayout Padding="24" Spacing="20" VerticalOptions="Center">
    <HorizontalStackLayout Spacing="8" HorizontalOptions="Center">
      <Image x:Name="K0" Source="kostka1.png" WidthRequest="60" HeightRequest="60" />
      <Image x:Name="K1" Source="kostka1.png" WidthRequest="60" HeightRequest="60" />
      <Image x:Name="K2" Source="kostka1.png" WidthRequest="60" HeightRequest="60" />
      <Image x:Name="K3" Source="kostka1.png" WidthRequest="60" HeightRequest="60" />
      <Image x:Name="K4" Source="kostka1.png" WidthRequest="60" HeightRequest="60" />
    </HorizontalStackLayout>
    <Label x:Name="SumaLabel" Text="Suma: 5" FontSize="22" FontAttributes="Bold" HorizontalOptions="Center" />
    <Button Text="Rzuć" Clicked="OnRzut" />
  </VerticalStackLayout>
</ContentPage>

```

```

namespace Miniaplikacje;

public partial class KosciPage : ContentPage
{
    readonly Random los = new Random();
    Image[] obrazy;

    public KosciPage()
    {
        InitializeComponent();
        obrazy = new[] { K0, K1, K2, K3, K4 };
    }

    private void OnRzut(object sender, EventArgs e)
    {
        int suma = 0;
        foreach (Image obraz in obrazy)
        {
            int wynik = los.Next(1, 7);
            obraz.Source = $"kostka{wynik}.png";
            suma += wynik;
        }
        SumaLabel.Text = $"Suma: {suma}";
    }
}

```

35.22. Kości z blokowaniem

Rozszerzenie gry w kości: dotknięta kość zostaje zablokowana i nie zmienia wartości przy kolejnym rzucie. Ćwiczy

`TapGestureRecognizer`, tablice i `ClassId`.

```

<?xml version="1.0" encoding="utf-8" ?>
<ContentPage xmlns="http://schemas.microsoft.com/dotnet/2021/maui"
              xmlns:x="http://schemas.microsoft.com/winfx/2009/xaml"
              x:Class="Miniaplikacje.KosciBlokadaPage"
              Title="Kości z blokowaniem">
    <VerticalStackLayout Padding="24" Spacing="20" VerticalOptions="Center">
        <Label Text="Dotknij kość, aby ją zablokować" HorizontalOptions="Center" />
        <HorizontalStackLayout Spacing="8" HorizontalOptions="Center">
            <Image x:Name="K0" ClassId="0" Source="kostka1.png" WidthRequest="60" HeightRequest="60"><Image.GestureRecognizers>
<TapGestureRecognizer Tapped="OnDotknieto" /></Image.GestureRecognizers></Image>
            <Image x:Name="K1" ClassId="1" Source="kostka1.png" WidthRequest="60" HeightRequest="60"><Image.GestureRecognizers>
<TapGestureRecognizer Tapped="OnDotknieto" /></Image.GestureRecognizers></Image>
            <Image x:Name="K2" ClassId="2" Source="kostka1.png" WidthRequest="60" HeightRequest="60"><Image.GestureRecognizers>
<TapGestureRecognizer Tapped="OnDotknieto" /></Image.GestureRecognizers></Image>
            <Image x:Name="K3" ClassId="3" Source="kostka1.png" WidthRequest="60" HeightRequest="60"><Image.GestureRecognizers>
<TapGestureRecognizer Tapped="OnDotknieto" /></Image.GestureRecognizers></Image>
            <Image x:Name="K4" ClassId="4" Source="kostka1.png" WidthRequest="60" HeightRequest="60"><Image.GestureRecognizers>
<TapGestureRecognizer Tapped="OnDotknieto" /></Image.GestureRecognizers></Image>
        </HorizontalStackLayout>
        <Label x:Name="SumaLabel" FontSize="22" FontAttributes="Bold" HorizontalOptions="Center" />
        <Button Text="Rzucić" Clicked="OnRzut" />
    </VerticalStackLayout>
</ContentPage>

```

```

namespace Miniaplikacje;

public partial class KosciBlokadaPage : ContentPage
{
    readonly Random los = new Random();
    Image[] obrazy;
    int[] wartosci = { 1, 1, 1, 1, 1 };
    bool[] blokady = new bool[5];

    public KosciBlokadaPage()
    {
        InitializeComponent();
        obrazy = new[] { K0, K1, K2, K3, K4 };
        OdswiezSume();
    }

    private void OnRzut(object sender, EventArgs e)
    {
        for (int i = 0; i < obrazy.Length; i++)
        {
            if (blokady[i]) continue;
            wartosci[i] = los.Next(1, 7);
            obrazy[i].Source = $"kostka{wartosci[i]}.png";
        }
        OdswiezSume();
    }
}

```



```

private void OnDotknieto(object sender, EventArgs e)
{
    var obraz = (Image)sender;
    int i = int.Parse(obraz.ClassId);
    blokady[i] = !blokady[i];
    obraz.Opacity = blokady[i] ? 0.4 : 1.0;
}

private void OdswiezSume()
{
    SumaLabel.Text = $"Suma: {wartosci.Sum()}";
}
}

```

35.23. Galeria obrazów

Aplikacja pokazuje kolejne obrazy z zasobów aplikacji. Ćwiczy [Image](#), przyciski [Poprzedni](#) i [Następny](#) oraz indeks aktualnego elementu.

```

<?xml version="1.0" encoding="utf-8" ?>
<ContentPage xmlns="http://schemas.microsoft.com/dotnet/2021/maui"
    xmlns:x="http://schemas.microsoft.com/winfx/2009/xaml"
    x:Class="Miniaplikacje.GaleriaPage"
    Title="Galeria">
    <VerticalStackLayout Padding="24" Spacing="16">
        <Image x:Name="Obraz" HeightRequest="260" Aspect="AspectFit" />
        <Label x:Name="OpisLabel" HorizontalOptions="Center" FontAttributes="Bold" />
        <HorizontalStackLayout Spacing="12" HorizontalOptions="Center">
            <Button Text="Poprzedni" Clicked="OnPoprzedni" />
            <Button Text="Następny" Clicked="OnNastepny" />
        </HorizontalStackLayout>
    </VerticalStackLayout>
</ContentPage>

```

```

namespace Miniaplikacje;

public partial class GaleriaPage : ContentPage
{
    readonly string[] pliki = { "obraz1.png", "obraz2.png", "obraz3.png" };
    int indeks = 0;

    public GaleriaPage()
    {
        InitializeComponent();
        Pokaz();
    }

    private void OnNastepny(object sender, EventArgs e)
    {
        indeks = (indeks + 1) % pliki.Length;
        Pokaz();
    }
}

```

```

    }

    private void OnPoprzedni(object sender, EventArgs e)
    {
        indeks = (indeks - 1 + pliki.Length) % pliki.Length;
        Pokaz();
    }

    private void Pokaz()
    {
        Obraz.Source = pliki[indeks];
        OpisLabel.Text = $"Obraz {indeks + 1} z {pliki.Length}";
    }
}

```

35.24. Quiz z pytaniami

Aplikacja wyświetla pytania po kolei i liczy punkty. Ćwiczy tablice obiektów, **RadioButton**, indeks aktualnego pytania i walidację wyboru.

```

<?xml version="1.0" encoding="utf-8" ?>
<ContentPage xmlns="http://schemas.microsoft.com/dotnet/2021/maui"
              xmlns:x="http://schemas.microsoft.com/winfx/2009/xaml"
              x:Class="Miniaplikacje.QuizPage"
              Title="Quiz">
    <VerticalStackLayout Padding="24" Spacing="12">
        <Label x:Name="NumerLabel" TextColor="Gray" />
        <Label x:Name="PytanieLabel" FontSize="22" FontAttributes="Bold" />
        <RadioButton x:Name="Odp0" GroupName="odp" />
        <RadioButton x:Name="Odp1" GroupName="odp" />
        <RadioButton x:Name="Odp2" GroupName="odp" />
        <Button Text="Zatwierdź" Clicked="OnZatwierdz" />
        <Label x:Name="WynikLabel" FontSize="20" FontAttributes="Bold" />
    </VerticalStackLayout>
</ContentPage>

```

```

namespace Miniaplikacje;

public class PytanieQuizu
{
    public string Tresc { get; set; }
    public string[] Odpowiedzi { get; set; }
    public int Poprawna { get; set; }
}

public partial class QuizPage : ContentPage
{
    PytanieQuizu[] pytania =

    {
        new() { Tresc = "Która kontrolka pokazuje tekst?", Odpowiedzi = new[] { "Label", "Entry", "Slider" }, Poprawna = 0 },
        new() { Tresc = "Która kontrolka służy do wpisywania tekstu?", Odpowiedzi = new[] { "Image", "Entry", "BoxView" }, Poprawna

```

```

= 1 },
    new() { Tresc = "Która kolekcja odświeża listę?", Odpowiedzi = new[] { "Array", "List", "ObservableCollection" }, Poprawna
= 2 }
};
int indeks = 0;
int punkty = 0;

public QuizPage()
{
    InitializeComponent();
    Pokaz();
}

private async void OnZatwierdz(object sender, EventArgs e)
{
    int wybrana = Odp0.IsChecked ? 0 : Odp1.IsChecked ? 1 : Odp2.IsChecked ? 2 : -1;
    if (wybrana == -1)
    {
        await DisplayAlert("Uwaga", "Wybierz odpowiedź.", "OK");
        return;
    }

    if (wybrana == pytania[indeks].Poprawna) punkty++;
    indeks++;

    if (indeks ≥ pytania.Length)
    {
        WynikLabel.Text = $"Wynik: {punkty}/{pytania.Length}";
        return;
    }
    Pokaz();
}

private void Pokaz()
{
    Odp0.IsChecked = Odp1.IsChecked = Odp2.IsChecked = false;
    var p = pytania[indeks];
    NumerLabel.Text = $"Pytanie {indeks + 1}/{pytania.Length}";
    PytanieLabel.Text = p.Tresc;
    Odp0.Content = p.Odpowiedzi[0];
    Odp1.Content = p.Odpowiedzi[1];
    Odp2.Content = p.Odpowiedzi[2];
    WynikLabel.Text = "";
}
}

```

35.25. Wzornik kolorów RGB

Aplikacja składa kolor z trzech suwaków RGB. Ćwiczy `Slider`, `BoxView`, `Color.FromRgb` i aktualizację wyglądu na żywo.

```

<?xml version="1.0" encoding="utf-8" ?>
<ContentPage xmlns="http://schemas.microsoft.com/dotnet/2021/maui"
    xmlns:x="http://schemas.microsoft.com/winfx/2009/xaml"

```

```

        x:Class="Miniaplikacje.RgbPage"
        Title="Wzornik RGB">
<VerticalStackLayout Padding="24" Spacing="12">
    <BoxView x:Name="Podglad" HeightRequest="120" Color="Black" />
    <Label x:Name="RgbLabel" Text="RGB(0, 0, 0)" HorizontalOptions="Center" FontAttributes="Bold" />
    <Label Text="Czerwony" /><Slider x:Name="RSlider" Minimum="0" Maximum="255" ValueChanged="OnKoLor" />
    <Label Text="Zielony" /><Slider x:Name="GSlider" Minimum="0" Maximum="255" ValueChanged="OnKoLor" />
    <Label Text="Niebieski" /><Slider x:Name="BSlider" Minimum="0" Maximum="255" ValueChanged="OnKoLor" />
</VerticalStackLayout>
</ContentPage>

```

```

namespace Miniaplikacje;

public partial class RgbPage : ContentPage
{
    public RgbPage()
    {
        InitializeComponent();
    }

    private void OnKoLor(object sender, ValueChangedEventArgs e)
    {
        int r = (int)RSlider.Value;
        int g = (int)GSlider.Value;
        int b = (int)BSlider.Value;
        Podglad.Color = Color.FromRgb(r, g, b);
        RgbLabel.Text = $"RGB({r}, {g}, {b})";
    }
}

```

35.26. Pliki i dane lokalne

Ta część ćwiczy zapis i odczyt danych: zwykłe pliki tekstowe, ustawienia aplikacji oraz prostą bazę SQLite.

35.27. Notatnik z zapisem do pliku

Aplikacja zapisuje i odczytuje tekst z pliku w `FileSystem.AppDataDirectory`. Ćwiczy `Editor`, `File.WriteAllTextAsync`, `File.ReadAllTextAsync` i obsługę błędów.

```

<?xml version="1.0" encoding="utf-8" ?>
<ContentPage xmlns="http://schemas.microsoft.com/dotnet/2021/maui"
    xmlns:x="http://schemas.microsoft.com/winfx/2009/xaml"
    x:Class="Miniaplikacje.NotatnikPlikPage"
    Title="Notatnik">
<VerticalStackLayout Padding="24" Spacing="12">
    <Editor x:Name="TekstEditor" Placeholder="Wpisz notatkę" HeightRequest="220" />
    <HorizontalStackLayout Spacing="10">
        <Button Text="Zapisz" Clicked="OnZapisz" />
        <Button Text="Odczytaj" Clicked="OnOdczytaj" />
        <Button Text="Wyczyść" Clicked="OnWyczysc" />
    </HorizontalStackLayout>
    <Label x:Name="InfoLabel" TextColor="Gray" />

```

```
</VerticalStackLayout>

</ContentPage>
```

```
namespace Miniaplikacje;

public partial class NotatnikPlikPage : ContentPage
{
    string Sciezka => Path.Combine(FileSystem.AppDataDirectory, "notatka.txt");

    public NotatnikPlikPage()
    {
        InitializeComponent();
    }

    private async void OnZapisz(object sender, EventArgs e)
    {
        await File.WriteAllTextAsync(Sciezka, TekstEditor.Text ?? "");
        InfoLabel.Text = "Zapisano notatkę.";
    }

    private async void OnOdczytaj(object sender, EventArgs e)
    {
        if (!File.Exists(Sciezka))
        {
            InfoLabel.Text = "Plik jeszcze nie istnieje.";
            return;
        }
        TekstEditor.Text = await File.ReadAllTextAsync(Sciezka);
        InfoLabel.Text = "Odczytano notatkę.";
    }

    private void OnWyczysc(object sender, EventArgs e)
    {
        TekstEditor.Text = "";
    }
}
```

35.28. Szyfr Cezara z zapisem wyniku

Aplikacja szyfruje tekst prostym przesunięciem liter i zapisuje wynik do pliku. Ćwicz pracę z `char`, pętlę `foreach`, metody pomocnicze i zapis pliku.

```
<?xml version="1.0" encoding="utf-8" ?>
<ContentPage xmlns="http://schemas.microsoft.com/dotnet/2021/maui"
    xmlns:x="http://schemas.microsoft.com/winfx/2009/xaml"
    x:Class="Miniaplikacje.SzyfrPage"
    Title="Szyfr Cezara">
    <VerticalStackLayout Padding="24" Spacing="12">
        <Entry x:Name="TekstEntry" Placeholder="Tekst" />
        <Entry x:Name="PrzesuniecieEntry" Placeholder="Przesunięcie" Keyboard="Numeric" />
        <Button Text="Zaszyfruj i zapisz" Clicked="OnZaszyfruj" />
    </VerticalStackLayout>
</ContentPage>
```

```

        <Label x:Name="WynikLabel" FontSize="20" FontAttributes="Bold" />
        <Label x:Name="InfoLabel" TextColor="Gray" />
    </VerticalStackLayout>
</ContentPage>

```

```

namespace Miniaplikacje;

public partial class SzyfrPage : ContentPage
{
    string Sciezka => Path.Combine(FileSystem.AppDataDirectory, "szyfr.txt");

    public SzyfrPage()
    {
        InitializeComponent();
    }

    private async void OnZaszyfruj(object sender, EventArgs e)
    {
        if (!int.TryParse(PrzesuniecieEntry.Text, out int p)) p = 3;
        string wynik = Szyfruj(TekstEntry.Text ?? "", p);
        WynikLabel.Text = wynik;
        await File.WriteAllTextAsync(Sciezka, wynik);
        InfoLabel.Text = "Zapisano wynik do pliku.";
    }

    private string Szyfruj(string tekst, int przesuniecie)
    {
        var sb = new System.Text.StringBuilder();
        foreach (char c in tekst)
        {
            if (!char.IsLetter(c)) { sb.Append(c); continue; }
            char baza = char.IsUpper(c) ? 'A' : 'a';
            sb.Append((char)(baza + (c - baza + przesuniecie) % 26));
        }
        return sb.ToString();
    }
}

```

35.29. Przeglądarka albumów z pliku

Aplikacja odczytuje plik `albumy.txt` z `Resources/Raw`, parsuje linie w formacie `Tytuł;Autor` i pokazuje dane w `CollectionView`.

```

<?xml version="1.0" encoding="utf-8" ?>
<ContentPage xmlns="http://schemas.microsoft.com/dotnet/2021/maui"
    xmlns:x="http://schemas.microsoft.com/winfx/2009/xaml"
    x:Class="Miniaplikacje.AlbumyPage"
    Title="Albumy z pliku">
    <CollectionView x:Name="AlbumyView">
        <CollectionView.ItemTemplate>
            <DataTemplate>

```

```

        <VerticalStackLayout Padding="12">
            <Label Text="{Binding Tytuł}" FontSize="18" FontAttributes="Bold" />
            <Label Text="{Binding Autor}" TextColor="Gray" />
        </VerticalStackLayout>
    </DataTemplate>
</CollectionView.ItemTemplate>
</CollectionView>
</ContentPage>

```

```

using System.Collections.ObjectModel;

namespace Miniaplikacje;

public class AlbumPlik
{
    public string Tytuł { get; set; }
    public string Autor { get; set; }
}

public partial class AlbumyPage : ContentPage
{
    ObservableCollection<AlbumPlik> albumy = new();

    public AlbumyPage()
    {
        InitializeComponent();
        AlbumyView.ItemsSource = albumy;
    }

    protected override async void OnAppearing()
    {
        base.OnAppearing();
        if (albumy.Count == 0) await Wczytaj();
    }

    private async Task Wczytaj()
    {
        using var stream = await FileSystem.OpenAppPackageFileAsync("albumy.txt");
        using var reader = new StreamReader(stream);
        string linia;
        while ((linia = await reader.ReadLineAsync()) != null)
        {
            string[] czesci = linia.Split(';');
            if (czesci.Length == 2)
                albumy.Add(new AlbumPlik { Tytuł = czesci[0].Trim(), Autor = czesci[1].Trim() });
        }
    }
}

```

35.30. Ustawienia aplikacji w Preferences

Aplikacja zapamiętuje login, rozmiar czcionki i tryb ciemny w **Preferences**. Ćwiczy trwałe ustawienia i odczyt danych przy starcie strony.

```
<?xml version="1.0" encoding="utf-8" ?>
<ContentPage xmlns="http://schemas.microsoft.com/dotnet/2021/maui"
              xmlns:x="http://schemas.microsoft.com/winfx/2009/xaml"
              x:Class="Miniaplikacje.UstawieniaPage"
              Title="Ustawienia">
    <VerticalStackLayout Padding="24" Spacing="14">
        <Entry x:Name="LoginEntry" Placeholder="Login" />
        <HorizontalStackLayout><Switch x:Name="CiemnySwitch" /><Label Text="Tryb ciemny" VerticalOptions="Center" />
    </HorizontalStackLayout>
    <Label x:Name="RozmiarLabel" Text="Rozmiar czcionki: 16" />
    <Slider x:Name="RozmiarSlider" Minimum="12" Maximum="28" Value="16" ValueChanged="OnRozmiar" />
    <Button Text="Zapisz ustawienia" Clicked="OnZapisz" />
    <Button Text="Wczytaj ustawienia" Clicked="OnWczytaj" />
    <Label x:Name="InfoLabel" />
    </VerticalStackLayout>
</ContentPage>
```

```
namespace Miniaplikacje;

public partial class UstawieniaPage : ContentPage
{
    public UstawieniaPage()
    {
        InitializeComponent();
        Wczytaj();
    }

    private void OnRozmiar(object sender, ValueChangedEventArgs e)
    {
        RozmiarLabel.Text = $"Rozmiar czcionki: {(int)e.NewValue}";
    }

    private void OnZapisz(object sender, EventArgs e)
    {
        Preferences.Set("login", LoginEntry.Text ?? "");
        Preferences.Set("ciemny", CiemnySwitch.IsToggled);
        Preferences.Set("rozmiar", (int)RozmiarSlider.Value);
        InfoLabel.Text = "Zapisano ustawienia.";
    }

    private void OnWczytaj(object sender, EventArgs e) => Wczytaj();

    private void Wczytaj()
    {
        LoginEntry.Text = Preferences.Get("login", "");
        CiemnySwitch.IsToggled = Preferences.Get("ciemny", false);
        RozmiarSlider.Value = Preferences.Get("rozmiar", 16);
    }
}
```



```

        InfoLabel.Text = "Wczytano ustawienia.";
    }
}

```

35.31. Katalog produktów SQLite

Aplikacja zapisuje produkty w lokalnej bazie SQLite. Ćwiczy model z `PrimaryKey`, klasę bazy danych, dodawanie, usuwanie i odświeżanie `CollectionView`.

```

<?xml version="1.0" encoding="utf-8" ?>
<ContentPage xmlns="http://schemas.microsoft.com/dotnet/2021/maui"
    xmlns:x="http://schemas.microsoft.com/winfx/2009/xaml"
    x:Class="Miniaplikacje.ProduktySqlitePage"
    Title="Produkty SQLite">
    <Grid Padding="20" RowDefinitions="Auto,Auto,*" RowSpacing="12">
        <HorizontalStackLayout Spacing="8">
            <Entry x:Name="NazwaEntry" Placeholder="Nazwa" WidthRequest="150" />
            <Entry x:Name="CenaEntry" Placeholder="Cena" Keyboard="Numeric" WidthRequest="90" />
            <Button Text="Dodaj" Clicked="OnDodaj" />
        </HorizontalStackLayout>
        <Label x:Name="KomunikatLabel" Grid.Row="1" TextColor="Red" />
        <CollectionView x:Name="ProduktyView" Grid.Row="2">
            <CollectionView.ItemTemplate>
                <DataTemplate>
                    <Grid Padding="8" ColumnDefinitions="*,Auto,Auto">
                        <Label Text="{Binding Nazwa}" />
                        <Label Grid.Column="1" Text="{Binding Cena, StringFormat='{0:0.00} zł'}" Margin="0,0,12,0" />
                        <Button Grid.Column="2" Text="Usuń" Clicked="OnUsun" />
                    </Grid>
                </DataTemplate>
            </CollectionView.ItemTemplate>
        </CollectionView>
    </Grid>
</ContentPage>

```

```

using SQLite;

namespace Miniaplikacje;

public class ProduktSqlite
{
    [PrimaryKey, AutoIncrement]
    public int Id { get; set; }
    public string Nazwa { get; set; }
    public double Cena { get; set; }
}

public class ProduktyBazaSqlite
{
    SQLiteAsyncConnection db;
}

```

```

async Task Init()
{
    if (db != null) return;
    string sciezka = Path.Combine(FileSystem.AppDataDirectory, "produkty.db3");
    db = new SQLiteAsyncConnection(sciezka);
    await db.CreateTableAsync<ProduktSqlite>();
}

public async Task<List<ProduktSqlite>> Pobierz()
{ await Init(); return await db.Table<ProduktSqlite>().ToListAsync(); }

public async Task Dodaj(ProduktSqlite p)
{ await Init(); await db.InsertAsync(p); }

public async Task Usun(ProduktSqlite p)
{ await Init(); await db.DeleteAsync(p); }
}

public partial class ProduktySqlitePage : ContentPage
{
    ProduktyBazaSqlite baza = new();

    public ProduktySqlitePage()
    {
        InitializeComponent();
    }

    protected override async void OnAppearing()
    {
        base.OnAppearing();
        await Odswiez();
    }

    private async void OnDodaj(object sender, EventArgs e)
    {
        if (string.IsNullOrEmpty(NazwaEntry.Text) || !double.TryParse(CenaEntry.Text, out double cena))
        {
            KomunikatLabel.Text = "Podaj nazwę i cenę.";
            return;
        }

        await baza.Dodaj(new ProduktSqlite { Nazwa = NazwaEntry.Text.Trim(), Cena = cena });
        NazwaEntry.Text = CenaEntry.Text = "";
        KomunikatLabel.Text = "";
        await Odswiez();
    }

    private async void OnUsun(object sender, EventArgs e)
    {
        if (sender is Button btn && btn.BindingContext is ProduktSqlite p)
        {
            await baza.Usun(p);
            await Odswiez();
        }
    }
}

```

```
private async Task Odswiez()
{
    ProduktyView.ItemsSource = await baza.Pobierz();
}
}
```

35.32. Czas, API i funkcje urządzenia

Te przykłady pokazują zadania asynchroniczne, pobieranie danych z sieci, minutnik oraz odczytywanie informacji z urządzenia.

35.33. Minutnik

Aplikacja odlicza czas podany w sekundach. Ćwiczy `DispatcherTimer`, stan logiczny, blokowanie przycisków i aktualizację etykiety co sekundę.

```
<?xml version="1.0" encoding="utf-8" ?>
<ContentPage xmlns="http://schemas.microsoft.com/dotnet/2021/maui"
    xmlns:x="http://schemas.microsoft.com/winfx/2009/xaml"
    x:Class="Miniaplikacje.MinutnikPage"
    Title="Minutnik">
    <VerticalStackLayout Padding="24" Spacing="16" VerticalOptions="Center">
        <Entry x:Name="SekundyEntry" Placeholder="Liczba sekund" Keyboard="Numeric" />
        <Label x:Name="CzasLabel" Text="00:00" FontSize="48" FontAttributes="Bold" HorizontalOptions="Center" />
        <HorizontalStackLayout Spacing="10" HorizontalOptions="Center">
            <Button x:Name="StartButton" Text="Start" Clicked="OnStart" />
            <Button Text="Reset" Clicked="OnReset" />
        </HorizontalStackLayout>
    </VerticalStackLayout>
</ContentPage>
```

```
namespace Miniaplikacje;

public partial class MinutnikPage : ContentPage
{
    IDispatcherTimer timer;
    int pozostalo;

    public MinutnikPage()
    {
        InitializeComponent();
        timer = Dispatcher.CreateTimer();
        timer.Interval = TimeSpan.FromSeconds(1);
        timer.Tick += OnTick;
    }

    private async void OnStart(object sender, EventArgs e)
    {
        if (!int.TryParse(SekundyEntry.Text, out pozostalo) || pozostalo <= 0)
        {
            await DisplayAlert("Błąd", "Podaj liczbę sekund większą od 0.", "OK");
            return;
        }
    }
}
```

```

    }

    StartButton.IsEnabled = false;
    PokazCzas();
    timer.Start();
}

private void OnTick(object sender, EventArgs e)
{
    pozostalo--;
    PokazCzas();
    if (pozostalo ≤ 0)
    {
        timer.Stop();
        StartButton.IsEnabled = true;
    }
}

private void OnReset(object sender, EventArgs e)
{
    timer.Stop();
    pozostalo = 0;
    StartButton.IsEnabled = true;
    PokazCzas();
}

private void PokazCzas()
{
    CzasLabel.Text = TimeSpan.FromSeconds(pozostalo).ToString(@"mm\:ss");
}
}

```

35.34. Panel urządzenia domowego

Aplikacja symuluje sterowanie urządzeniem. Ćwicz **Switch**, **Slider**, stan logiczny i zmianę koloru interfejsu.

```

<?xml version="1.0" encoding="utf-8" ?>
<ContentPage xmlns="http://schemas.microsoft.com/dotnet/2021/maui"
    xmlns:x="http://schemas.microsoft.com/winfx/2009/xaml"
    x:Class="Miniaplikacje.PanelPage"
    Title="Panel urządzenia">
    <VerticalStackLayout Padding="24" Spacing="16">
        <Label Text="Panel urządzenia" FontSize="26" FontAttributes="Bold" />
        <HorizontalStackLayout>
            <Switch x:Name="Wlacznik" Toggled="OnZmieniono" />
            <Label Text="Włącz urządzenie" VerticalOptions="Center" />
        </HorizontalStackLayout>
        <Label x:Name="MocLabel" Text="Moc: 50%" />
        <Slider x:Name="MocSlider" Minimum="0" Maximum="100" Value="50" ValueChanged="OnMoc" />
        <BoxView x:Name="StatusBox" HeightRequest="80" Color="Gray" />
        <Label x:Name="StatusLabel" Text="Urządzenie wyłączone" FontAttributes="Bold" />
    </VerticalStackLayout>
</ContentPage>

```

```

namespace Miniaplikacje;

public partial class PanelPage : ContentPage
{
    public PanelPage()
    {
        InitializeComponent();

        private void OnZmieniono(object sender, ToggledEventArgs e) => Odswiez();
        private void OnMoc(object sender, ValueChangedEventArgs e)
        {
            MocLabel.Text = $"Moc: {(int)e.NewValue}%";
            Odswiez();
        }

        private void Odswiez()
        {
            bool wlaczone = Wlacznik.IsToggled;
            int moc = (int)MocSlider.Value;
            StatusLabel.Text = wlaczone ? $"Urządzenie działa z mocą {moc}%" : "Urządzenie wyłączone";
            StatusBox.Color = wlaczone ? Color.FromRgb(255 - moc * 2, 120 + moc, 80) : Colors.Gray;
        }
    }
}

```

35.35. Pogoda z API

Aplikacja pobiera aktualną pogodę z publicznego API Open-Meteo dla wybranego miasta. Ćwiczmy `HttpClient`, JSON, `ActivityIndicator` i obsługę błędów sieci.

```

<?xml version="1.0" encoding="utf-8" ?>
<ContentPage xmlns="http://schemas.microsoft.com/dotnet/2021/maui"
    xmlns:x="http://schemas.microsoft.com/winfx/2009/xaml"
    x:Class="Miniaplikacje.PogodaApiPage"
    Title="Pogoda API">
    <VerticalStackLayout Padding="24" Spacing="14">
        <Picker x:Name="MiastoPicker" Title="Wybierz miasto">
            <Picker.Items>
                <x:String>Warszawa</x:String>
                <x:String>Kraków</x:String>
                <x:String>Gdańsk</x:String>
            </Picker.Items>
        </Picker>
        <Button Text="Pobierz pogodę" Clicked="OnPobierz" />
        <ActivityIndicator x:Name="Loader" IsVisible="False" IsRunning="False" />
        <Label x:Name="TemperaturaLabel" FontSize="36" FontAttributes="Bold" HorizontalOptions="Center" />
        <Label x:Name="WiatrLabel" HorizontalOptions="Center" />
    </VerticalStackLayout>
</ContentPage>

```

```
using System.Net.Http.Json;
using System.Text.Json.Serialization;

namespace Miniaplikacje;

public class PogodaOdpowiedz
{
    [JsonPropertyName("current_weather")]
    public PogodaAktualna CurrentWeather { get; set; }
}

public class PogodaAktualna
{
    [JsonPropertyName("temperature")]
    public double Temperature { get; set; }
    [JsonPropertyName("windspeed")]
    public double Windspeed { get; set; }
}

public partial class PogodaApiPage : ContentPage
{
    static readonly HttpClient http = new HttpClient();

    public PogodaApiPage()
    {
        InitializeComponent();
        MiastoPicker.SelectedIndex = 0;
    }

    private async void OnPobierz(object sender, EventArgs e)
    {
        var (lat, lon) = MiastoPicker.SelectedItem.ToString() switch
        {
            "Kraków" => (50.06, 19.94),
            "Gdańsk" => (54.35, 18.65),
            _ => (52.23, 21.01)
        };

        Loader.IsRunning = Loader.IsVisible = true;
        try
        {
            string url = $"https://api.open-meteo.com/v1/forecast?latitude={lat}&longitude={lon}&current_weather=true";
            var dane = await http.GetFromJsonAsync<PogodaOdpowiedz>(url);
            TemperaturaLabel.Text = $"{dane.CurrentWeather.Temperature:0} °C";
            WiatrLabel.Text = $"Wiatr: {dane.CurrentWeather.Windspeed:0} km/h";
        }
        catch
        {
            await DisplayAlert("Błąd", "Nie udało się pobrać danych.", "OK");
        }
        finally
        {
            Loader.IsRunning = Loader.IsVisible = false;
        }
    }
}
```

```

    }
}
}

```

35.36. Lista użytkowników z API

Aplikacja pobiera listę użytkowników z przykładowego API i pokazuje ją w `CollectionView`. Ćwicz modele JSON, `GetFromJsonAsync`, listę obiektów i wskaźnik ładowania.

```

<?xml version="1.0" encoding="utf-8" ?>
<ContentPage xmlns="http://schemas.microsoft.com/dotnet/2021/maui"
    xmlns:x="http://schemas.microsoft.com/winfx/2009/xaml"
    x:Class="Miniaplikacje.UzytkownicyApiPage"
    Title="Użytkownicy API">
    <Grid Padding="20" RowDefinitions="Auto,Auto,*" RowSpacing="12">
        <Button Text="Pobierz użytkowników" Clicked="OnPobierz" />
        <ActivityIndicator x:Name="Loader" Grid.Row="1" IsVisible="False" IsRunning="False" />
        <CollectionView x:Name="UzytkownicyView" Grid.Row="2">
            <CollectionView.ItemTemplate>
                <DataTemplate>
                    <VerticalStackLayout Padding="10">
                        <Label Text="{Binding Name}" FontSize="18" FontAttributes="Bold" />
                        <Label Text="{Binding Email}" TextColor="Gray" />
                    </VerticalStackLayout>
                </DataTemplate>
            </CollectionView.ItemTemplate>
        </CollectionView>
    </Grid>
</ContentPage>

```

```

using System.Net.Http.Json;

namespace Miniaplikacje;

public class UzytkownikApi
{
    public string Name { get; set; }
    public string Email { get; set; }
}

public partial class UzytkownicyApiPage : ContentPage
{
    static readonly HttpClient http = new HttpClient();

    public UzytkownicyApiPage()
    {
        InitializeComponent();
    }

    private async void OnPobierz(object sender, EventArgs e)
    {

```

```

        Loader.IsRunning = Loader.IsVisible = true;
    try
    {
        var dane = await http.GetFromJsonAsync<List<UzytkownikApi>>("https://jsonplaceholder.typicode.com/users");
        UzytkownicyView.ItemsSource = dane;
    }
    catch
    {
        await DisplayAlert("Błąd", "Nie udało się pobrać użytkowników.", "OK");
    }
    finally
    {
        Loader.IsRunning = Loader.IsVisible = false;
    }
}
}

```

35.37. Informacje o urządzeniu i sieci

Aplikacja pokazuje podstawowe informacje o urządzeniu oraz stanie połączenia z internetem. Ćwicz [DeviceInfo](#), [Connectivity](#) i odświeżanie danych przyciskiem.

```

<?xml version="1.0" encoding="utf-8" ?>
<ContentPage xmlns="http://schemas.microsoft.com/dotnet/2021/maui"
    xmlns:x="http://schemas.microsoft.com/winfx/2009/xaml"
    x:Class="Miniaplikacje.InfoUrzadzeniaPage"
    Title="Informacje">
    <VerticalStackLayout Padding="24" Spacing="12">
        <Label Text="Informacje o urządzeniu" FontSize="26" FontAttributes="Bold" />
        <Label x:Name="ModelLabel" />
        <Label x:Name="PlatformaLabel" />
        <Label x:Name="WersjaLabel" />
        <Label x:Name="SiecLabel" FontAttributes="Bold" />
        <Button Text="Odśwież" Clicked="OnOdswiez" />
    </VerticalStackLayout>
</ContentPage>

```

```

namespace Miniaplikacje;

public partial class InfoUrzadzeniaPage : ContentPage
{
    public InfoUrzadzeniaPage()
    {
        InitializeComponent();
        Pokaz();
    }

    private void OnOdswiez(object sender, EventArgs e) => Pokaz();

    private void Pokaz()
    {

```



```

        ModelLabel.Text = $"Model: {DeviceInfo.Current.Model}";
        PlatformaLabel.Text = $"Platforma: {DeviceInfo.Current.Platform}";
        WersjaLabel.Text = $"Wersja systemu: {DeviceInfo.Current.VersionString}";

        bool internet = Connectivity.Current.NetworkAccess == NetworkAccess.Internet;
        SiecLabel.Text = internet ? "Internet: dostępny" : "Internet: niedostępny";
        SiecLabel.TextColor = internet ? Colors.Green : Colors.Red;
    }
}

```

36. Najczęstsze błędy (Wyjątki i błędy kompilacji)

Tworząc aplikacje w platformie MAUI (C# oraz XAML) możesz napotkać błędy w czasie kompilacji oraz działaniu na różnych urządzeniach. Poniżej przedstawiamy typowe wyjątki systemowe (Exceptions) wyrzucane przez framework oraz popularne komunikaty błędów widoczne w okienku Output/Error List Visual Studio.

36.1. NullReferenceException

Zgłaszany, gdy kod próbuje wywołać metodę lub uzyskać dostęp do właściwości obiektu (np. kontrolki lub zmiennej modelu), który jest równy `null`.

❌ Kod powodujący błąd:

```

Label mojLabel = null;
mojLabel.Text = "Witaj w MAUI";

```

Komunikat błędu:

```
System.NullReferenceException: 'Object reference not set to an instance of an object.'
```

✅ Poprawny kod:

```

Label mojLabel = new Label();
mojLabel.Text = "Witaj w MAUI";

```

Upewnij się, że kontrolka istnieje i została utworzona przed przypisaniem wartości, zwłaszcza jeśli odnosisz się do niej przed wywołaniem `InitializeComponent()`.

36.2. Błąd XFC0000 (Błąd parsowania XAML)

Błąd kompilatora XAML (XAML Compiler), sygnalizujący błędną składnię, brakujący atrybut lub nierozpoznany znacznik w pliku .xaml.

❌ Kod powodujący błąd:

```
<Label Text="Witaj" FontSize="BłędnyRozmiar" />
```

Komunikat błędu:

```
XFC0000: Cannot convert "BłędnyRozmiar" to type "double"
```

✅ Poprawny kod:

```
<Label Text="Witaj" FontSize="18" />
```

Popraw wskazaną przez błąd linijkę w pliku XAML podając wartość o prawidłowym typie oczekiwanym przez framework.

36.3. Błąd XFC0009 (Brak właściwości)

Zgłaszany, gdy wpiszesz w znaczniku XAML atrybut, który nie jest znany kompilatorowi MAUI (częsta literówka w nazwach właściwości).

❌ Kod powodujący błąd:

```
<Button Label="Zapisz" Clicked="OnZapisz" />
```

Komunikat błędu:

```
XFC0009: No property, BindableProperty, or event found for "Label", or mismatching type between value and property.
```

✅ Poprawny kod:

```
<Button Text="Zapisz" Clicked="OnZapisz" />
```

Zamień niewłaściwą nazwę atrybutu (np. `Label`) na poprawną (np. `Text`), zgodnie z dokumentacją kontrolki MAUI.

36.4. MissingMethodException (Brak konstruktora bezparametrowego)

Pojawia się podczas próby nawigacji do strony (lub tworzenia obiektu z XAML), która nie ma publicznego konstruktora bez argumentów.

❌ Kod powodujący błąd:

```
public partial class MojaStrona : ContentPage {  
    public MojaStrona(int id) { InitializeComponent(); }  
}
```

Komunikat błędu:

```
System.MissingMethodException: 'Constructor on type 'Miniaplikacje.MojaStrona' not found.'
```

✅ Poprawny kod:

```
public partial class MojaStrona : ContentPage {  
    public MojaStrona() { InitializeComponent(); }  
    public MojaStrona(int id) : this() { ... }  
}
```

Każda strona tworzona automatycznie z XAML lub podczas standardowej nawigacji musi posiadać publiczny konstruktor bez parametrów.

36.5. InvalidOperationException

Wyjątek ten występuje w różnych scenariuszach w MAUI, najczęściej przy próbie wykonania operacji w niewłaściwym stanie, np. aktualizacji UI z wątku tła na niektórych platformach.

✗ Kod powodujący błąd:

```
Task.Run(() => {  
    StatusLabel.Text = "Zakończono";  
});
```

Komunikat błędu:

```
System.InvalidOperationException: 'The current thread is not the UI thread.' (Na platformie iOS/Windows)
```

✓ Poprawny kod:

```
Task.Run(() => {  
    MainThread.BeginInvokeOnMainThread(() => {  
        StatusLabel.Text = "Zakończono";  
    });  
});
```

Zawsze modyfikuj kontrolki widoku (UI) korzystając z wątku głównego, używając `MainThread.BeginInvokeOnMainThread()`.

36.6. HttpRequestException

Częsty błąd przy komunikacji z API (metody HttpClient). Występuje, gdy serwer nie odpowiada, URL jest błędny lub brakuje połączenia internetowego.

✗ Kod powodujący błąd:

```
HttpClient client = new HttpClient();  
var odpowiedz = await client.GetStringAsync("http://zlyadres.brak");
```

Komunikat błędu:

```
System.Net.Http.HttpRequestException: 'No such host is known. (zlyadres.brak:80)'
```

✓ Poprawny kod:

```
try {  
    var odpowiedz = await client.GetStringAsync("https://api.github.com");  
} catch (HttpRequestException ex) {  
    await DisplayAlert("Błąd", "Brak połączenia z siecią.", "OK");  
}
```

Operacje sieciowe zawsze obejmuj blokiem `try...catch` i sprawdzaj uprzednio połączenie z internetem (np. przez `Connectivity.Current.NetworkAccess`).

36.7. TaskCanceledException

Związany z programowaniem asynchronicznym. Zgłaszany najczęściej, gdy wywołanie `HttpClient` przekroczyło limit czasu (timeout) lub operacja została anulowana za pomocą `CancellationToken`.

❌ Kod powodujący błąd:

```
HttpClient client = new HttpClient { Timeout = TimeSpan.FromSeconds(1) };  
await client.GetStringAsync("https://wolny-serwer.com");
```

Komunikat błędu:

```
System.Threading.Tasks.TaskCanceledException: 'A task was canceled.'
```

✅ Poprawny kod:

```
try {  
    await client.GetStringAsync("https://wolny-serwer.com");  
} catch (TaskCanceledException) {  
    await DisplayAlert("Błąd", "Przekroczono czas oczekiwania (Timeout).", "OK");  
}
```

Zwiększ limit czasu lub obsłuż ten wyjątek (szczególnie na wolnych połączeniach mobilnych), wyświetlając przyjazny komunikat dla użytkownika.

36.8. FileNotFoundException

Pojawia się przy próbie odczytu pliku z pamięci urządzenia (lokalnego Storage), który nie istnieje pod podaną ścieżką.

❌ Kod powodujący błąd:

```
string path = Path.Combine(FileSystem.AppDataDirectory, "dane.json");  
string json = File.ReadAllText(path);
```

Komunikat błędu:

```
System.IO.FileNotFoundException: 'Could not find file '/data/user/0/.../files/dane.json'.'
```

✅ Poprawny kod:

```
string path = Path.Combine(FileSystem.AppDataDirectory, "dane.json");  
if (File.Exists(path)) {  
    string json = File.ReadAllText(path);  
}
```

Przed próbą otwarcia pliku sprawdź metodą `File.Exists()` czy dany plik rzeczywiście tam się znajduje.

36.9. Błąd CS0103 (Kompilacja)

Klasyczny błąd kompilacji, sygnalizujący że zmienna lub nazwa podana w C# nie występuje w bieżącym kontekście lub kontrolka XAML nie ma przypisanego `x:Name`.

✗ Kod powodujący błąd:

```
WejscieUzytkownika.Text = ""; // Ale kontrolka ma x:Name="PoleWejscia" w XAML
```

Komunikat błędu:

```
CS0103: The name 'WejscieUzytkownika' does not exist in the current context
```

✓ Poprawny kod:

```
PoleWejscia.Text = "";
```

Upewnij się, że nazwa zmiennej w pliku C# dokładnie (z uwzględnieniem wielkości liter) zgadza się z atrybutem `x:Name` wpisanym w pliku `.xaml`.

36.10. Błąd CS1061 (Kompilacja)

Zgłaszany, gdy próbujemy odwołać się do metody lub właściwości, która nie istnieje dla danego typu obiektu (często błędne nazwy własności klas MAUI).

✗ Kod powodujący błąd:

```
Button btn = new Button();  
btn.Title = "Zapisz";
```

Komunikat błędu:

```
CS1061: 'Button' does not contain a definition for 'Title'...
```

✓ Poprawny kod:

```
Button btn = new Button();  
btn.Text = "Zapisz";
```

Sprawdź na co pozwala dana kontrolka. W MAUI większość elementów wyświetlających napisy ma właściwość `Text`, a nie `Title` czy `Content`.

36.11. FormatException

Pojawia się w trakcie uruchomienia aplikacji, gdy próbujesz przetworzyć ciąg znaków na inny typ danych (np. liczbę) przy użyciu metody `Parse`, a ciąg zawiera niewłaściwe znaki.

✗ Kod powodujący błąd:

```
int kwota = int.Parse("100 PLN");
```

Komunikat błędu:

```
System.FormatException: 'Input string was not in a correct format.'
```

✅ Poprawny kod:

```
if (int.TryParse("100 PLN", out int kwota)) {  
    // sukces  
} else {  
    // obsługa błędu wpisywania  
}
```

Używaj bezpieczniejszej funkcji `TryParse()` zamiast `Parse()`, szczególnie, gdy parsujesz dane wejściowe pobrane od użytkownika.

36.12. ArgumentOutOfRangeException

Zgłaszany przez kolekcje (np. `List`, `ObservableCollection`), gdy podany numer indeksu nie istnieje (jest mniejszy od 0 lub wykracza poza rozmiar kolekcji).

❌ Kod powodujący błąd:

```
var o = kolekcja[10]; // gdy kolekcja ma 5 elementów
```

Komunikat błędu:

```
System.ArgumentOutOfRangeException: 'Index was out of range. Must be non-negative and less than the size of the collection.'
```

✅ Poprawny kod:

```
if (kolekcja.Count > 10) {  
    var o = kolekcja[10];  
}
```

Kontroluj wielkość kolekcji przy użyciu właściwości `.Count` przed bezpośrednim odwoływaniem się poprzez indeksy.

36.13. UnauthorizedAccessException

Pojawia się na urządzeniach mobilnych, gdy program próbuje zapisać dane, odczytać kontakty lub użyć kamery bez przyznania przez użytkownika odpowiednich uprawnień w manifeście.

❌ Kod powodujący błąd:

```
var zdjecie = await MediaPicker.CapturePhotoAsync();
```

Komunikat błędu:

```
System.UnauthorizedAccessException: 'Access to the path or media is denied.' (lub podobne błędy systemowe)
```

✅ Poprawny kod:

```
var status = await Permissions.CheckStatusAsync<Permissions.Camera>();  
if (status != PermissionStatus.Granted)  
    status = await Permissions.RequestAsync<Permissions.Camera>();  
if (status == PermissionStatus.Granted)  
    var zdjecie = await MediaPicker.CapturePhotoAsync();
```

Dla każdej platformy (szczególnie Android/iOS) dodaj uprawnienia w pliku manifestu i poproś o nie użytkownika używając klasy `Permissions` z MAUI.

36.14. InvalidCastException

Występuje w obsłudze zdarzeń, gdy rzutujesz argument `sender` na niewłaściwy typ kontrolki.

❌ Kod powodujący błąd:

```
void OnKlick(object sender, EventArgs e) {  
    Label klikniety = (Label)sender; // błąd, jeśli to był Button  
}
```

Komunikat błędu:

```
System.InvalidCastException: 'Unable to cast object of type 'Microsoft.Maui.Controls.Button' to type  
'Microsoft.Maui.Controls.Label'.'
```

✅ Poprawny kod:

```
void OnKlick(object sender, EventArgs e) {  
    if (sender is Button klikniety) {  
        // tu bezpiecznie używamy 'klikniety' jako Button  
    }  
}
```

Używaj operatora `is` do bezpiecznego sprawdzania i rzutowania typów przy obsłudze uniwersalnych zdarzeń współdzielonych między różnymi kontrolkami.

36.15. JsonException (System.Text.Json)

Zgłaszany w procesie konwersji (deserializacji) ciągu JSON pobranego z sieci lub pliku do modelu C#, gdy JSON jest uszkodzony lub brakuje dopasowania typów.

❌ Kod powodujący błąd:

```
string json = "{ 'nazwa': 'Błędny JSON' }"; // cudzysłów pojedynczy  
var obj = JsonSerializer.Deserialize<Model>(json);
```

Komunikat błędu:

```
System.Text.Json.JsonException: ''{' is invalid after a single quoted string. Expected end of data. LineNumber: 0...'
```

✅ **Poprawny kod:**

```
string json = "{ \"nazwa\": \"Poprawny JSON\" }";  
try {  
    var obj = JsonSerializer.Deserialize<Model>(json);  
} catch (JsonException) { /* obsługa błędu */ }
```

Upewnij się, że odpowiedź z serwera jest prawidłowym obiektem JSON, właściwości w modelu są kompatybilne i poprawnie zdefiniowano pola do konwersji.

36.16. Błąd NETSDK1005 (Błąd Budowania)

Błąd na poziomie narzędzi kompilacji systemu .NET informujący o braku pliku konfiguracyjnego dla paczek (NuGet) używanych w projekcie.

❌ **Kod powodujący błąd:**

(Błąd pojawia się po pobraniu repozytorium z gita przed zrobieniem Restore)

Komunikat błędu:

NETSDK1005: Assets file '.../obj/project.assets.json' doesn't exist. Run a NuGet package restore to generate this file.

✅ **Poprawny kod:**

```
dotnet restore
```

Kliknij prawym przyciskiem myszy na rozwiązanie (Solution) w Visual Studio i wybierz **Restore NuGet Packages** lub odbuduj projekt (Rebuild).

36.17. Błąd CS0246 (Brak using)

Oznacza problem ze znalezieniem typu, klasy lub narzędzia, prawdopodobnie z powodu braku odwołania na górze pliku do właściwej przestrzeni nazw.

❌ **Kod powodujący błąd:**

```
ObservableCollection<string> napisy;
```

Komunikat błędu:

CS0246: The type or namespace name 'ObservableCollection<>' could not be found...

✅ **Poprawny kod:**

```
using System.Collections.ObjectModel;  
  
ObservableCollection<string> napisy;
```


Kliknij na podkreślony na czerwono element i wywołaj sugestie Visual Studio (zwykle żarówka lub skrót Ctrl+Kropka), aby automatycznie dodać brakujący `using`.

36.18. TargetInvocationException (Błędy wiązań i XAML)

Wyjątek opakowujący inny wyjątek. Często występuje podczas refleksji i wczytywania interfejsu (metoda `InitializeComponent()`), kryjąc wewnątrz prawdziwą przyczynę awarii.

✗ Kod powodujący błąd:

```
<!-- Błędne przypisanie w konstruktorze lub wiązaniu wewnątrz XAML -->
```

Komunikat błędu:

```
System.Reflection.TargetInvocationException: 'Exception has been thrown by the target of an invocation.'
```

✓ Poprawny kod:

```
// Należy zbadać InnerException
```

Gdy spotkasz ten wyjątek, kliknij opcję `View Detail` (lub w inspektora wyjątków) i sprawdź zawartość obiektu `InnerException`, która wskaże dokładny kod, plik lub XAML odpowiedzialny za błąd.

36.19. Java.Lang.Exception (Wyjątki specyficzne dla Androida)

Gdy stworzysz aplikację MAUI i uruchamiasz ją na Androidzie, błędy pochodzące od samego podsystemu operacyjnego Android (a nie .NET) będą owinięte w wyjątki Java.

✗ Kod powodujący błąd:

```
// Operacja specyficzna dla systemu Android kończąca się niepowodzeniem
```

Komunikat błędu:

```
Java.Lang.IllegalArgumentException: '...' (lub podobny Java Exception)
```

✓ Poprawny kod:

```
// Poprawienie natywnej logiki interfejsu
```

Są to typowe błędy ekosystemu Androida. Należy skopiować komunikat błędu (InnerException) do wyszukiwarki dopisując hasła `Android MAUI`, by odnaleźć błąd związany z użyciem platformy natywnej.

36.20. TypeLoadException

Zgłaszany, gdy środowisko wykonawcze (runtime) MAUI nie potrafi odnaleźć typu (klasy lub struktury) załadowanego ze skompilowanego zestawu (assemblies) lub biblioteki zewnętrznej NuGet.

✗ Kod powodujący błąd:

```
// Aktualizacja frameworku do nowszej wersji, pozostawienie starych bibliotek niezgodnych z .NET 8
```

Komunikat błędu:

```
System.TypeLoadException: 'Could not load type 'X' from assembly 'Y'.'
```

✅ Poprawny kod:

Należy zaktualizować paczki NuGet **do** nowszej wersji zgodnej z MAUI / .NET **8**.

Wykonaj komendę **Clean** (Wyczyść projekt) na całym Solution, zaktualizuj paczki NuGet, usuń foldery **bin** i **obj**, a następnie przebuduj całość od zera.